



PURLS OF WISDOM: SHOP MANAGEMENT SYSTEM

NEA WRITE-UP

Contents

Analysis	6
Problem.....	6
Client	6
Aim	6
Structured Interview	6
Document Study	9
Membership system.....	9
Workshop/events management system	10
Filtering orders (to help with searching through orders).....	10
Receipts.....	10
Objectives.....	12
Homepage	12
Customer Details Management	12
Customer Loyalty Scheme.....	12
Supplier Details Management.....	13
Product and Inventory Management.....	13
Customer Order Management.....	13
Supplier Order Management	15
Event Management System	17
Notice System	19
Critical Path	21
Documented Design.....	22
Form Navigation	22
Data Flow Diagram	22
Database Design.....	23
Relationships.....	23
Entity Relationship Diagram.....	24
Tables	24
SQL Design	38
Queries.....	38
Statements	41
UML Diagram	43

Class Diagrams	44
Algorithms.....	52
‘Big Algorithm’: Invoice Generator.....	52
Homepage Calendar.....	62
Filtering Products	63
Event Waiting List Allocation.....	66
Issuing and Updating Notices.....	70
Sorting Notices	72
Data Structures	77
Waiting List Queue	77
User Interface Design.....	78
Design Notes	78
Homepage.....	79
Customers	81
NewCustomer	83
Suppliers.....	84
NewSupplier.....	85
Products	86
NewProduct	87
Orders.....	88
NewCustOrder.....	90
NewSupplierOrder	91
Events.....	92
NewEvent	93
Notices	94
Hardware Selection.....	95
Testing Plan	95
Technical Solution	121
Forms	121
Homepage.cs.....	121
Customers.cs.....	145
NewCustomer.cs.....	157
Suppliers.cs	164

NewSupplier.cs	175
Products.cs	180
NewProduct.cs	213
Orders.cs	222
NewCustOrder.cs	275
NewSupplierOrder.cs	314
Events.cs	349
NewEvent.cs	377
Notices.cs	400
Controls	426
CalendarBlank	426
CalendarDay	427
CalendarBox	430
AttendeeBox	433
NoticeBox	435
Classes	439
Customer	439
Supplier	441
Product	443
Order	446
CustOrder	449
OrderItem	464
InvoiceLine	465
Event	467
EventAttendee	473
Notice	475
Node	483
WaitingListQueue	484
Testing	488
Evaluation	513
Testing Evaluation	513
Test 65 & 66	513
Test 119, 122, 124	514

Test 121	514
Test 155	516
Test 177	516
Objectives Evaluation	516
Client Feedback	518
Future Improvements & Extensions	519
User Access Levels.....	519
Address and Postcode Validation	519
Customer Postcodes.....	520
Event Time Management	520
Customer Loyalty Scheme	520
Printing Receipts	520
Final Evaluation	521
Appendix	522
1. Analysis	522
1.1 Full Structured Interview	522
1.2 Document Study.....	524
Testing	525
Test 97: Printed Receipt	525

Analysis

Problem

I will create a program for a crafts shop specialising in supplying wool/yarn for knitting and crocheting. It is a growing small business in our local area and is popular especially for the workshops they provide, as the shop owners work with various visitors to provide the local community with engaging events for those of all ages and experience.

The shop owner has expressed difficulty however in organising all inbound and customer orders, as well as managing workshops with attendees and payments. They find that due to the growing popularity of their business, keeping business up is proving difficult for a small team like theirs. This problem is exacerbated by keeping track of all customers, suppliers and workshop details all on different platforms: for example, workshops rely on customers using Facebook to RSVP to the event created on the shop's Facebook page.

They would like a program that will keep track of all orders and deliveries, customer, supplier and event details, and assist staff in managing stock. Though there may be programs out there that are similar, the shop owners would like a program that is more specific to the crafts shop's needs, like managing events and orders specific to the shop. They also want the program to notify them of any significant changes, such as stock levels depleting, or overdue orders, alongside a calendar view of all major events over a month period.

A program like this would overall allow the shop owner to manage the shop operations more efficiently and improve organisation of the shop.

Client

Position: Owner of the shop

Role description: Manages and oversees all shop operations.

Aim

Streamline the management of orders, stock and events within the shop to make daily operations more efficient for staff. Provide a user-friendly way to improve organisation, reduce delays and ensure a smoother shopping experience for customers.

Structured Interview

SEE 1.1 Full Structured Interview

This interview is my main source of information that I have analysed carefully in order to derive well structured, effective objectives. I gathered many points that I will be using to guide my project towards an efficient, well-designed program that will potentially prove useful to my client's business.

Question	Analysis	Main points
1. What information about the yarn would you prefer to be displayed?	This question helped me with my design of the Products table. From what my client has said, the fields I have in my Products table now include the colour, the weight and quantitative	Products table fields include the colour, weight, quantitative

	weight of the yarn, and the prices. I have ensured a wide range of products of varying colours and weights have been included to allow the filtering system to work efficiently. My client outlined the possible need for a “filter” option, to enable the user to easily search through products according to type, colour, material or weight etc. I will be implementing a filter system for products, as well as orders and events to allow users to easily find the information they need.	weight of yarn, and price. Need a filter system for products and orders to allow the user to find the information they require more efficiently.
2. What information would be useful to view in terms of orders?	From this response I decided what fields I would require in my Orders tables (later I decided I needed two separate tables for customers and suppliers). My client suggested being able to view the order date, the cost, the items of the order and the information of the customer/supplier which I have recorded in a separate table to avoid repeating groups of data. They have also suggested being able to right click on the order and view product information, which I will implement as a way to easily manage the items in any past order in store.	Orders tables (both customers and suppliers) include the order date, total cost, items of the order (customer orders) and the information of the customer/supplier linked. Products of an order should be viewed easily from the order selected on the Orders form.
3. What information would be useful to view in terms of deliveries?	I also thought that inbound deliveries should be viewed separately from all customer orders, so the information for each is recorded in separate tables. My client mentioned a calendar view would be useful for seeing all inbound deliveries of the month (or past or future months) using the order date field in the table. I think this would be a user-friendly efficient way for staff to stay on top of inbound deliveries.	Inbound deliveries and customer orders will be displayed separately. Having a calendar view for viewing all inbound deliveries of the month may be useful.
4. What other data would you want to be able to store and link?	My client’s response to this question first gave me the idea to add an event management system for the workshops and events the staff sometimes provide in the shop. They outlined that being able to type in all the attendee’s details (first name last name etc) as a customer, then link them to the event, it would prove useful in keeping track of attendees. Furthermore, staff could keep track of those that have paid for the event (where necessary) or those that may need extra requirements through a notes field.	Include an event management system for workshops and other events the shop hosts. Customers should be linked to the event when signing up, and those that have paid for the workshop should also be recorded. Event notes would prove useful for any extra information that is

		unique to the event being managed.
5. What would you like the main interface to have?	I feel like the main interface is one of the most important parts of my program since users will primarily need to interact with this part of the program. My client mainly suggested displaying important notices, the logo of the shop and an interactive calendar view of all events, orders and inbound deliveries all on the main interface.	The main feature of the main interface should be an interactive calendar consisting of all events, orders and inbound deliveries. Important notices such as low stock should also be displayed on the main interface.
6. What level of detail would be needed in the information displayed or would you prefer separate windows for separate categories?	Following on from the previous question, the client suggested that only basic information should be displayed on the calendar view to avoid it appearing cluttered. Instead, she advocated for a dropdown style view if there are multiple notices in one day, allowing the user to still access all the basic information they may need for one day. They also talked about being able to add quick notes to any events on the front calendar, which would allow her extra flexibility with orders/events/deliveries that may need extra information to allow other staff (and herself) to keep track of other essential information.	On the calendar view basic information should be displayed e.g. Order No.3 is due; Event: Stained Glass Butterfly workshop to avoid cluttering. A dropdown style view should be implemented to expand on the multiple orders/deliveries/events there may be in one day. Users should be able to add "quick notes" to all events/orders/deliveries through the front calendar to help with extra flexibility and ensure all staff can keep track of the same extra information.
7. What would you like to see when managing workshops?	This response helped me decide what information to store about each event. On the events page, she also would like a calendar view of just the workshops, to clearly see the events throughout the month alongside all the essential information required. She also would like to be able to see if a customer attending the workshop has paid or not and be able to add and delete customers easily from the event.	On the Events form, a calendar view should be used to display all events in a month view. By clicking on each event, the information about each event can be displayed accordingly. When managing an event, customers need to be added/removed

		from the event. Their paid status must also be tracked.
8. Are there any ways creating this program could help you increase your sales?	My client outlined that getting customers to return to the shop is rare, so implementing some sort of customer loyalty system would possibly help increase the number of returning customers. If this could be included in the program, it'd greatly help my client as she would not need to calculate membership details every time a customer places an order.	A customer loyalty system should be included and used for customer orders.

Document Study

Membership system

×

Create Account

Join our free Club today to enjoy 15% off your first purchase, as well as other exclusive rewards and discounts. [Find out more.](#)

First Name*

Last Name*

Email*

Confirm Email*

Password*

Password must be 11 or more characters, including at least one capital letter, one number and one special character

Confirm Password*

This is the Hobbycraft page for creating an account with them through their mobile application. All the fields are standard across most business like Hobbycraft needing to contact customers about orders, deals etc. and this still applies for small businesses like my client's.

Using this program, customers can be signed up by staff for the members' program by providing staff with their first name, last name and either their email address or phone number.

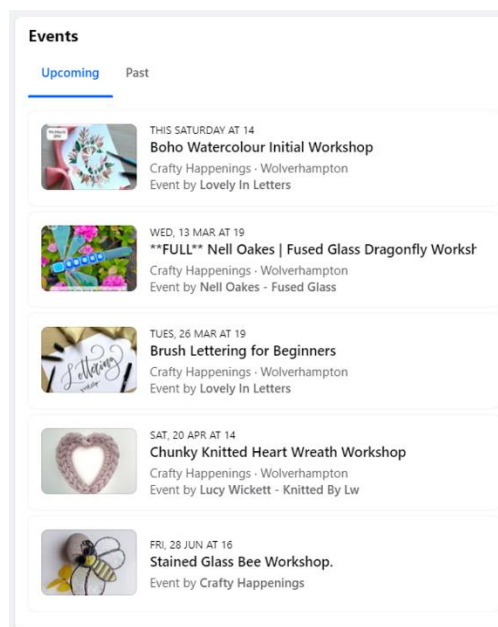
Customers do not need to sign up if they do not wish to, they will simply be recorded under the customer object with the name "Guest". Their order can still be recorded and searched for using their order ID, provided the customer can provide their order ID using the receipt of the order they made.

Workshop/events management system

This is an example of a crafts shop's Facebook page. This is how my client primarily manages attending guests and workshops together, and in store attendees would be recorded in a notebook along with paid status.

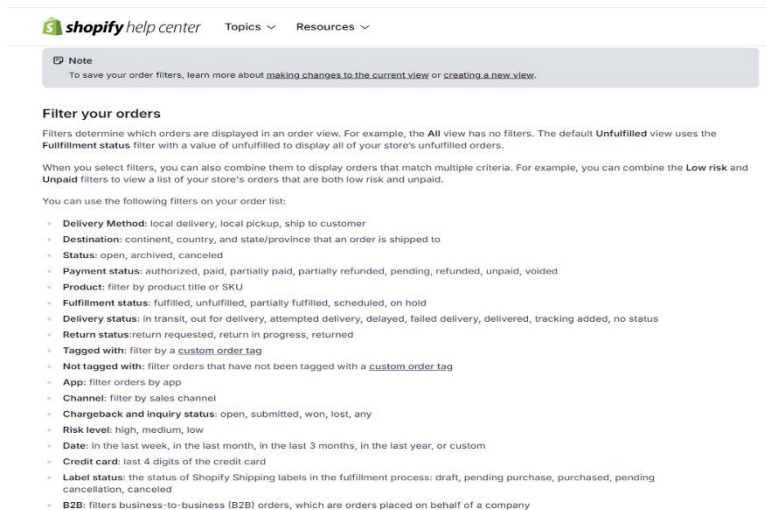
Instead, in the program staff will be able to register existing customers in the database for events (new customers can be created). The number of attendees and the number that have paid can also be kept track of efficiently. The capacity of each event can be recorded when each event is created: when customers are added it will alert the staff if places are low or have run out.

Notes about the event can also be stored in case staff need to record additional information.



Filtering orders (to help with searching through orders)

Shopify is a popular third-party order management service used by many businesses doing online orders through their own websites. The owner has said they are not looking to do orders through a website yet, however we noted that many of the filters that Shopify provide to users for management of orders would prove useful to staff. I picked out and decided on a few filters that I believe would be necessary: status, status, delivery status, date, and the presence of any order notes.



Receipts

In my client's shop, receipts are not automatically provided with orders. If a customer requests one, the owner manually records order details, including products, customer name, and ID, in an invoice book, keeping a carbon copy for store records (see Existing Invoice Format in appendix).

This manual process is inefficient, as records can be easily lost, and searching past orders is time-consuming. Additionally, with plans to implement a loyalty scheme, manually writing membership details for each order would further slow operations. To address this, I analysed automated receipt formats used in other stores (see Receipt Design in appendix). Furthermore, since my client wishes to

implement a loyalty scheme, writing all membership information on the invoice every time a customer places an order will consume much time which the client wishes to avoid.

Objectives

Homepage

1. View a calendar dashboard with all events, inbound deliveries and customer orders to be processed for each day for each month
2. Switch between each month
3. Switch back to the current month if a different month is currently being viewed
4. Access event information directly by clicking on an event in the calendar
5. Access notices, events, orders, products, customers and suppliers by clicking on each of their corresponding buttons

Customer Details Management

1. View all customers
2. Click on a customer and view their details: ID, name, Email Address, Phone Number, Membership Type, total stars, the number of vouchers available
3. Add a new customer with their full name, email address and/or phone number, and automatically set the membership type, total stars and number of vouchers unless set otherwise
 - 3.1. Inputted phone number and email address must be unique for each customer, if there is an existing customer with the contact details inputted an error message will be displayed
 - 3.2. Email address and phone number must be valid
4. Edit all customer details
5. All details entered for the customer must be valid inputs
6. Delete customer data from the system
 - 6.1. Customer should be removed from all events
 - 6.2. All orders made by the customer must have been collected, voided or returned before deleting their data
7. Search for the customer by name or email address

Customer Loyalty Scheme

1. Assign customers a membership status:
 - 1.1. Customers can hold Bronze, Silver, or Gold membership status.
 - 1.2. New customers are assigned Bronze status by default.
2. Customers can earn stars and vouchers to use in future orders:
 - 2.1. Customers earn 3 stars per £1 spent on any purchases.
 - 2.2. 1 voucher is awarded for every 150 stars earned.
3. Update customer membership status based on the total number of stars earned since holding their current membership status:
 - 3.1. Bronze to Silver: Achieved at 450 total stars earned.
 - 3.2. Silver to Gold: Achieved at 900 stars earned after reaching Silver.
 - 3.3. Each membership upgrade includes one voucher.
4. When voids or returns are made, update customer membership details as follows:
 - 4.1. If products are voided or returned, customers lose the stars earned from those products.
 - 4.2. Membership status is adjusted accordingly if stars drop below the required thresholds.
 - 4.3. Vouchers are revoked if total stars fall below the threshold for earning the last voucher.

Supplier Details Management

1. View all suppliers
2. Click on a supplier and view their details: ID, name, email address, phone number, address, postcode and URL
3. Add a new supplier with their name, email address and/or phone number, and if the user wishes they can add the address, postcode and URL
 - 3.1. Email address and phone number must be valid
4. Edit all supplier details
5. Delete supplier data from the system
 - 5.1. All orders made for the supplier must have been delivered, voided or returned before deleting their data
 - 5.2. All products by the supplier must be discontinued
6. Be able to search for a supplier by name

Product and Inventory Management

1. Be able to view all product details: ID, name, cost, stock, supplier, image,
 - 1.1. If yarn: ball weight, yarn weight, colour, fibre
 - 1.2. If accessory: size, category, colour, material
2. Switch between viewing all products, yarn products or accessory products
3. Highlight which products are out of stock or low on stock
4. Filter products:
 - 4.1. Yarn products: yarn weight, fibre, colour
 - 4.2. Accessories: category, material, colour
 - 4.3. All: In stock, low on stock, out of stock, supplier, discontinued
5. Add a new product with its name, cost, stock, supplier, ball weight/size, category, colour, fibre/material and uploaded image file to inventory
6. Edit all product information
 - 6.1. If stock is updated, issue and update notices
7. Permanently discontinue a product so it can't be added to new orders and stock cannot be updated when it runs out of stock
 - 7.1. Archive any existing stock notices for that product
8. Search for the product by name
 - 8.1. If filters are applied, the search applies to all products currently being shown
9. Be able to view, add, edit and remove the image of a product

Customer Order Management

1. View all customer orders
2. View all customer order details for each order: ID, order date, expected delivery date, collection window end date, order status (dependent on delivery status), delivery status, payment status, subtotal, total, stars earned from the order, if any voucher was used and the type, and order notes
3. View the customer of the order and their details
4. View all products in the order
 - 4.1. View all product details for each product in the order
 - 4.2. View all quantities of each product in the order:

- 4.2.1. Quantity of the product that has been sold
 - 4.2.2. Quantity that has been voided/returned
- 5. Filter all customer orders by status, delivery status, payment status, order date, expected delivery date, collection date end and if there are any order notes
- 6. Search for an order by customer name or email address
 - 6.1. If filters are applied, the search applies to all orders currently being shown
- 7. Place a customer order
 - 7.1. View all products that are not discontinued
 - 7.2. Switch between viewing all products, yarn products or accessory products
 - 7.3. Search for a product to add by product name
 - 7.4. Filter products:
 - 7.4.1. Yarn products: yarn weight, fibre, colour
 - 7.4.2. Accessories: category, material, colour
 - 7.4.3. All: In stock, low on stock, out of stock, supplier, discontinued
 - 7.5. View all product details when a product is selected
 - 7.6. Create a new product within the order if needed
 - 7.7. View all products added to the shopping list so far
 - 7.8. View the total cost of one product and its quantity in the shopping list
 - 7.9. Add products to the shopping list if there is enough stock
 - 7.10. Remove products from the shopping list
 - 7.11. Click on a product in the shopping list to show its details and add more of that product if needed
 - 7.12. Link a customer to the order
 - 7.13. Create a new customer within the order to link them to the order
 - 7.14. Select 'Guest' if the customer in store is not registered and doesn't wish to provide details
 - 7.15. Change the customer linked while creating the order
 - 7.16. Input the order date, expected delivery date, collection window end date, delivery status and order notes
 - 7.16.1. Limit the order notes to a character length of 255
 - 7.16.2. Automatically set the expected delivery date to 3 days after the order date and the collection window end date to a week after the expected delivery date
 - 7.17. Select 'In Store' if the order is paid for and collected in store on the same day automatically sets the delivery status and dates
 - 7.18. Be able to redeem a voucher in the order if the customer has vouchers available to use
 - 7.19. If the customer's membership status is:
 - 7.19.1. GOLD: apply a 15% discount
 - 7.19.2. SILVER: apply a 10% discount
 - 7.19.3. BRONZE: apply a 5% discount
 - 7.19.4. Only allow one voucher to be redeemed per order
 - 7.20. If a voucher was redeemed, display the amount saved by using the voucher
 - 7.21. Display the subtotal and update it automatically when products are added or removed
 - 7.22. Display the total cost and update it automatically when products are added or removed

- 7.23. Display the total cost and update it automatically when the subtotal is changed, or a voucher is added or removed
- 7.24. When the order is placed:
 - 7.24.1. Update inventory
 - 7.24.2. Issue and update stock notices
 - 7.24.3. Update the customer's membership details
 - 7.24.4. Issue and update customer order notices
- 8. Edit the order date, expected delivery date, collection date end, delivery status and order notes
 - 8.1. Only allow the user to edit the order notes if the order has been collected
 - 8.2. Limit the order notes to a character length of 255
- 9. Void order items of an order if the order hasn't been collected yet
- 10. Void a full order only if the order hasn't been collected yet
- 11. Return order items of an order if the order has been collected
- 12. Return a full order if the order has been collected
- 13. Issue and update any customer order notices if the expected delivery date, collection date end, delivery status has been changed or if an order has been returned or voided (see issuing and updating notices)
- 14. Update customer membership details when any voids/returns have been made
- 15. Update inventory when any voids/returns have been made
 - 15.1. Issue and update stock notices
- 16. Generate invoice for customer order
 - 16.1. Display the order date, order ID, date and time printed
 - 16.2. Display all products in the order and their total quantity
 - 16.3. Display any products that have been voided/returned and the quantity voided/returned
 - 16.4. Display the subtotal and total of the order
 - 16.5. Display the amount saved if a voucher was used
 - 16.6. Display the amount deducted from voids/returns
 - 16.7. If the customer of this order is under 'Guest' or was deleted, don't display any customer information except the stars earned from the order
 - 16.8. Display the customer ID, name, member status, stars earned from the order and the total amount of stars the customer has now
 - 16.9. If the customer used a voucher display how much the customer has saved altogether as a member
 - 16.10. If the customer didn't use a voucher, display how much the customer needs to spend to earn the next voucher
 - 16.11. Allow the user to print it or save it as a pdf file with multiple pages

Supplier Order Management

- 1. View all supplier orders
- 2. View all supplier order details for each order: ID, order date, expected delivery date, order status, delivery status, subtotal, total, order notes
- 3. View the supplier of the order and their details
- 4. View all products in the order
 - 4.1. View all product details for each product in the order

- 4.2. View all quantities of each product in the order:
 - 4.2.1. Quantity of the product that has been sold
 - 4.2.2. Quantity that has been voided/returned
5. Filter all supplier orders by status, delivery status, order date, expected delivery date, supplier and if there are any order notes
6. Search for an order by supplier name
 - 6.1. If filters are applied, the search applies to all orders currently shown
7. Place a supplier order for one supplier at a time:
 - 7.1. Change the supplier the order is being placed for, clearing all information entered
 - 7.2. Display all products that are not discontinued for the supplier selected
 - 7.3. Switch between viewing all products, yarn products or accessory products
 - 7.4. Search for a product to add by product name
 - 7.5. Filter products:
 - 7.5.1. Yarn products: yarn weight, fibre, colour
 - 7.5.2. Accessories: category, material, colour
 - 7.5.3. All: In stock, low on stock or out of stock
 - 7.6. View all product details when a product is selected
 - 7.7. If there are no products for the supplier selected, a new product must be created for that supplier in the products form.
 - 7.8. View all products added to the shopping list
 - 7.9. View the total cost of one product and its quantity in the shopping list
 - 7.10. Add products to the shopping list
 - 7.11. Remove products from the shopping list
 - 7.12. Edit the quantity of a product by adding or removing one of it
 - 7.13. Click on a product in the shopping list to show its details and add more of that product if needed
 - 7.14. Input the order date, expected delivery date, delivery status and order notes
 - 7.14.1. Limit the order notes to a character length of 255
 - 7.14.2. Automatically set the expected delivery date to 3 days after the order date
 - 7.15. Display the subtotal and total and update it automatically when the shopping list is updated
 - 7.16. Edit the total if required (for example, to account for any discounts when bulk ordering products)
 - 7.17. When the order is placed:
 - 7.17.1. If the delivery status is delivered, update inventory
 - 7.17.2. Issue and update stock notices
8. Edit the order date, expected delivery date, delivery status and order notes
9. Only the order notes can be edited if the order has been delivered
10. Void order items of an order if it hasn't been delivered
11. Void a full order if it hasn't been delivered
12. Return order items of an order if it's been delivered
13. Return a full order if it's been delivered
14. If any returns are made, update inventory
15. Issue and update notices:

- 15.1. Supplier order notices if the expected delivery date or delivery date has been changed
- 15.2. Stock notices and supplier order notices if any voids/returns have been made

Event Management System

1. View all events in either a weekly view or a view of all events
 - 1.1. Display all events that are active and upcoming by default
 - 1.2. Switch between each view by clicking either of the weekly view or all events buttons
2. Display all details of the event selected:
 - 2.1. View the Event ID, Name, Host, Date, Start and End time, Price, Capacity, Maximum number of guests per customer, Skill Level, Event notes
 - 2.2. View the number of customers that have paid in comparison to the total number of customers added (this does not include guests)
 - 2.3. Highlight this field in red if not all customers have paid
 - 2.4. View the total number of attendees for the event (including guests, excluding waiting list customers)
 - 2.5. View all attendees for the event:
 - 2.5.1. View the names and number of guests of customers attending the event and customers on the waiting list separately
 - 2.5.2. Click on a customer and view their details: Customer ID, Full Name, Phone Number, Email Address, Paid and Waiting List status
 - 2.5.3. Highlight customers that have not paid in the list in red
 - 2.5.4. Modify the paid status of a customer attending the event while viewing all details
 - 2.6. Edit event notes while viewing all details
3. Navigating the weekly view of events:
 - 3.1. Switch between each week
 - 3.2. Display the week start and end date of the week currently being viewed
 - 3.3. Switch back to the current week when a different week is currently being viewed
 - 3.4. Highlight today's date in the calendar
 - 3.5. Display all events for each day in the correct dates
 - 3.6. Click on an event in the calendar and display all details of that event
 - 3.7. Display all events for the current week displayed in the data grid view of events
 - 3.7.1. Be able to click on an event in the data grid view and display all details of that event
4. When viewing all events:
 - 4.1. Display all events that are active and upcoming by default
 - 4.2. Filter events by status, allocation of spaces and payments
 - 4.2.1. Status: Upcoming, Past, Active, Cancelled events
 - 4.2.2. Spaces: Events where there are spaces available, at capacity and/or have customers on a waiting list
 - 4.2.3. Payments: Events where all customers have paid, or where there are customers that haven't paid
5. Search for an event by name
 - 5.1. If filters are applied, the search applied to all events currently shown
6. Create a new event/Edit an existing event:
 - 6.1. View all customers

- 6.1.1.Highlight customers added to the event in green in the data grid view
- 6.2. Search for a customer by name or email address
- 6.3. Select a customer to be added and display all customer details: Customer ID, Full Name, Phone Number, Email Address
- 6.4. Select the number of guests the customer will be bringing
 - 6.4.1.This is limited by the maximum number of guests set for the event, which can be changed
- 6.5. View all customers currently added to the list of attendees for the event
 - 6.5.1.Display the customer's full name and the number of guests they are bringing
 - 6.5.2.Highlight customers that haven't paid in red
 - 6.5.3.Highlight customers that are on the waiting list in blue
 - 6.5.4.Add the selected customer to the list of attendees
 - 6.5.4.1. Automatically allocate customers added to the list to the waiting list by the following conditions:
 - 6.5.4.1.1. If the event is at capacity
 - 6.5.4.1.2. If adding the customer to the event will result in the event being oversubscribed i.e. the number of guests they are bringing results in this
 - 6.5.5.Indicate when the customer currently selected will be added to the waiting list as per the above conditions
- 6.6. Remove customers from the list of attendees
- 6.7. Allocate spaces to any customers on the waiting list according to the following conditions:
 - 6.7.1.There are enough spaces available for a customer and their guests (if any)
 - 6.7.2.Customers that have paid for the event have priority over customers that have not paid
 - 6.7.3.If all customers have either paid or not paid, then customers are allocated either by:
 - 6.7.3.1. First come first serve
 - 6.7.3.2. If not possible, then the first customer that can be added to the main list of attendees will be allocated a space, even if there is a customer with higher priority before them
- 6.8. Change the number of guests and paid status for customers currently added to the list of attendees
- 6.9. Input the name, host, date, start time, end time, price, skill level, notes, maximum number of guests allowed per customer and the capacity
 - 6.9.1.The start time and end time must be within the opening hours of the shop: 09:00-20:00
 - 6.9.2.Limit event notes to a character length of 255
 - 6.9.3.The capacity cannot be decreased if the total number of attendees added is equal to the capacity
 - 6.9.4.The maximum number of guests cannot be decreased until customers with guests are removed
- 6.10. Clear all information entered
 - 6.10.1. Clears all customers from the list of attendees
- 6.11. Refresh the list of attendees
 - 6.11.1. Allocates spaces to any customers on the waiting list automatically according to the conditions specifies above
- 7. Cancel an event in the future

Notice System

Viewing Notices

1. Create a dynamic notice system that consolidates stock levels, orders, and event information to keep users informed of critical updates.
 - 1.1. By default, show all active notices (not dismissed or archived)
 - 1.2. By default, notices should be displayed in descending order of review date
 - 1.2.1. For notices with the same review date, they should be displayed by descending priority
2. View all archived notices
3. View all dismissed notices
4. Filter all notices by:
 - 4.1. Priority
 - 4.2. Stock notices: low stock, no stock, selling fast and low sales
 - 4.3. Event notices: payments, spaces available
 - 4.4. Orders: collections, processing and supply
 - 4.5. Issue date and/or review date: past 1 day, 3 days, 1 week and older than 1 week
5. Sort currently shown notices by ascending or descending priority
6. Sort currently shown notices by ascending or descending review date
7. Right click on a notice and redirect the user to the appropriate form based on the notice type:
 - 7.1. Redirect the user to the supplier order creator if the notice is a stock related notice (low stock, no stock or selling fast)
 - 7.1.1. Display the relevant product and supplier information
 - 7.1.2. Automatically add the relevant product to the shopping list
 - 7.2. Redirect the user to the Events form if the notice is an event related notice
 - 7.2.1. Display the relevant event details, including the list of attendees
 - 7.3. Redirect the user to the Orders form if the notice is an order related notice
 - 7.3.1. If the notice relates to a customer order, display the relevant customer order
 - 7.3.2. If the notice relates to a supply delivery, display the relevant supplier order

Issuing and Updating Notices

Stock Notices

1. Generate stock notices for products based on the following conditions: low stock, out of stock, selling fast, or low sales
 - 1.1. Notices should be assigned a priority level as specified:
 - 1.1.1. **High Priority:** Out of Stock
 - 1.1.2. **Medium Priority:** Low Stock & Selling Fast
 - 1.1.3. **Low Priority:** Low Stock, Selling Fast, Low Sales
 - 1.2. Notices should be issued based on the following rules:
 - 1.2.1. Only generate notices for products that do not already have an active or dismissed notice of the same type
 - 1.2.2. If a product currently has a 'low stock' notice but is now out of stock, the 'low stock' notice should be archived and a new 'out of stock' notice should be issued for the product

- 1.2.3. Issue a 'selling fast' notice if more than half of a product's original stock (since its last order) has been depleted
- 1.2.4. Issue a 'low sales' notice if less than 20% of a product's full stock level has been sold in the past month
- 1.2.5. If a product is identified as low on stock and selling fast, a 'Low Stock and Selling Fast' notice should be issued for the product
 - 1.2.5.1. Any existing individual 'low stock' or 'selling fast' notices for that product should be archived
- 1.2.6. If a product identified as low on stock, out of stock or selling fast has a pending supply order that has not been delivered yet, a notice should be created but marked as dismissed until that order's expected delivery date
2. Update existing stock notices based on the following conditions:
 - 2.1. If a notice exists for a product identified as low on stock, out of stock, or selling fast, and a pending supply order for that product has just been marked as delivered in the orders form, archive the notice unless the product is still low on stock or out of stock.
 - 2.2. Archive a "low stock" notice if the product is no longer low on stock.
 - 2.3. Archive an "out of stock" notice if the product is no longer out of stock.
 - 2.4. Archive a "low sales" notice if the product's sales have sufficiently increased.
 - 2.5. Archive a "selling fast" notice if the product is no longer selling fast or if a supply order has been placed for the product.
 - 2.6. If a notice has been dismissed until a specific date and that date has passed, set the notice to active.

Order Notices

1. Generate customer order notices for orders under the following conditions:
 - 1.1. If an order has not been collected by the specified collection window end date
 - 1.2. If an order has not been processed and its expected delivery date has passed
 - 1.3. Only generate notices for orders that do not already have an active or dismissed notice of the same type
 - 1.4. Notices should be assigned priority levels based on the time elapsed since the delivery or collection date as specified:
 - 1.4.1. High Priority: More than 5 days overdue
 - 1.4.2. Medium Priority: More than 3 days overdue
 - 1.4.3. Low Priority: More than 1 day overdue
2. Update existing customer order notices based on the following conditions:
 - 2.1. If a notice exists for a specific overdue period (e.g. 1 day overdue) and the overdue period increases (e.g. now 3 days overdue)
 - 2.1.1. Archive the existing notice and create a new notice for the updated overdue period
 - 2.2. If the related order has now been collected, processed (status set to 'Ready To Collect') or voided
 - 2.2.1. Archive the related notice for the order
 - 2.3. If the expected delivery date or collection window end date for an order with an active notice has been modified:
 - 2.3.1. Archive the existing notice if its condition (e.g. overdue period) no longer applies.

- 2.3.2. Create new notices if required to reflect the updated conditions (e.g. new overdue status based on updated dates)
3. Generate supplier order notices for orders where its delivery date has passed, and the order has not been delivered yet:
 - 3.1. Notices should be assigned priority levels based on the time elapsed since the delivery or collection date as specified:
 - 3.1.1. High Priority: More than 5 days overdue
 - 3.1.2. Medium Priority: More than 3 days overdue
 - 3.1.3. Low Priority: More than 1 day overdue
 - 3.2. Update existing supplier order notices based on the following conditions:
 - 3.2.1. If an existing notice for an overdue order (e.g. 1 day overdue) and the overdue period increases (e.g. now 3 days overdue)
 - 3.2.1.1. Archive the existing notice and create a new notice for the updated overdue period
 - 3.2.2. If an order has been delivered or voided
 - 3.2.2.1. Archive the related notice for the order
 - 3.2.3. If the delivery date for an order with an active notice has been modified:
 - 3.2.3.1. Archive the existing notice if its condition (e.g. overdue period) no longer applies.
 - 3.2.4. Create new notices if required to reflect the updated conditions (e.g. new overdue status based on updated dates)

Event Notices

1. Generate notices for events based on the following conditions:
 - 1.1. If there are customers added to the event attendees list that have not paid
 - 1.2. If there are enough available spaces in the event for any customers on the waiting list
 - 1.3. Notices should be assigned **Medium Priority**
 - 1.4. Only generate notices for events that do not already have an active or dismissed notice of the same type
2. Archive existing event payment notices based on the following conditions:
 - 2.1. If there is an existing notice for an event where there are customers that haven't paid, but all customers that aren't on the waiting list have now paid
 - 2.2. If there is an existing notice for an event that is now cancelled
3. Archive existing event spaces notices based on the following conditions:
 - 3.1. If the relevant event is now at capacity
 - 3.2. If there are no longer customers on the waiting list for the relevant event
 - 3.3. If the relevant event is now cancelled

Critical Path

For this project, I will aim to complete the following critical path to produce an effective solution that fulfils the core requirements of this project.

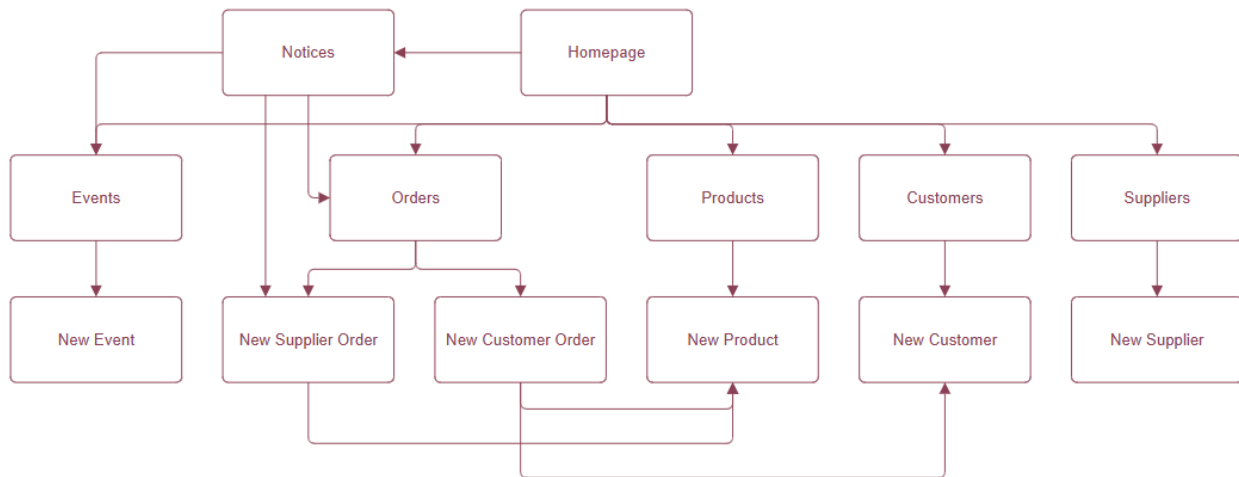
1. Database Creation & Normalization

Develop a fully normalized database in Microsoft Access to store customers, suppliers, products, orders, events, and notices.

2. **CRUD Implementation for Core Entities**
Implement Create, Read, Update, and Delete (CRUD) functionality for customers, suppliers, and products to ensure data integrity and easy management.
3. **Orders System Development**
Implement CRUD for orders, linking them to customers, suppliers, and products.
Enable multiple order creators to manage orders efficiently.
4. **Loyalty Scheme & Invoicing**
Apply the loyalty scheme to customer orders and customers.
Develop an invoice generation system with options to export to PDF or direct printing.
5. **Event Management System**
Implement the event creation and tracking system.
Track customer payment status and waiting list status efficiently.
6. **Homepage Calendar Integration**
Develop the homepage calendar to display events, deliveries, and collections for each date of the month, providing users with an overview.
7. **Notice Generation System**
Create a dynamic notice system that consolidates stock levels, orders, and event information to keep users informed of critical updates.

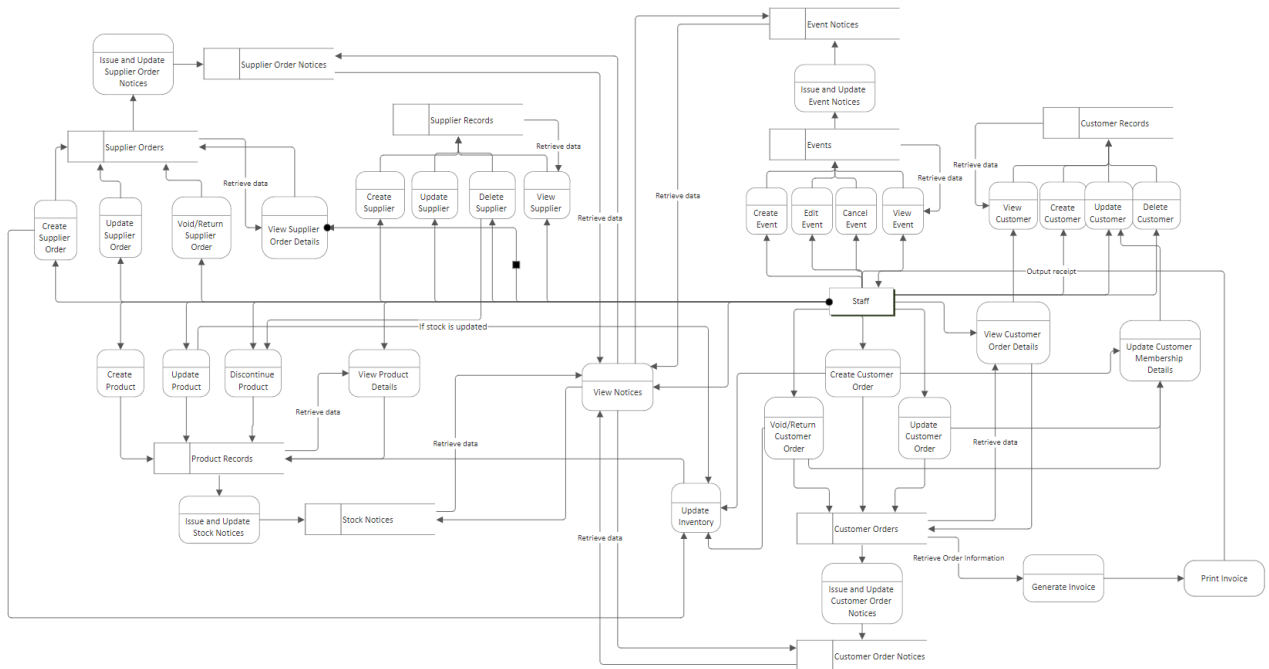
Documented Design

Form Navigation



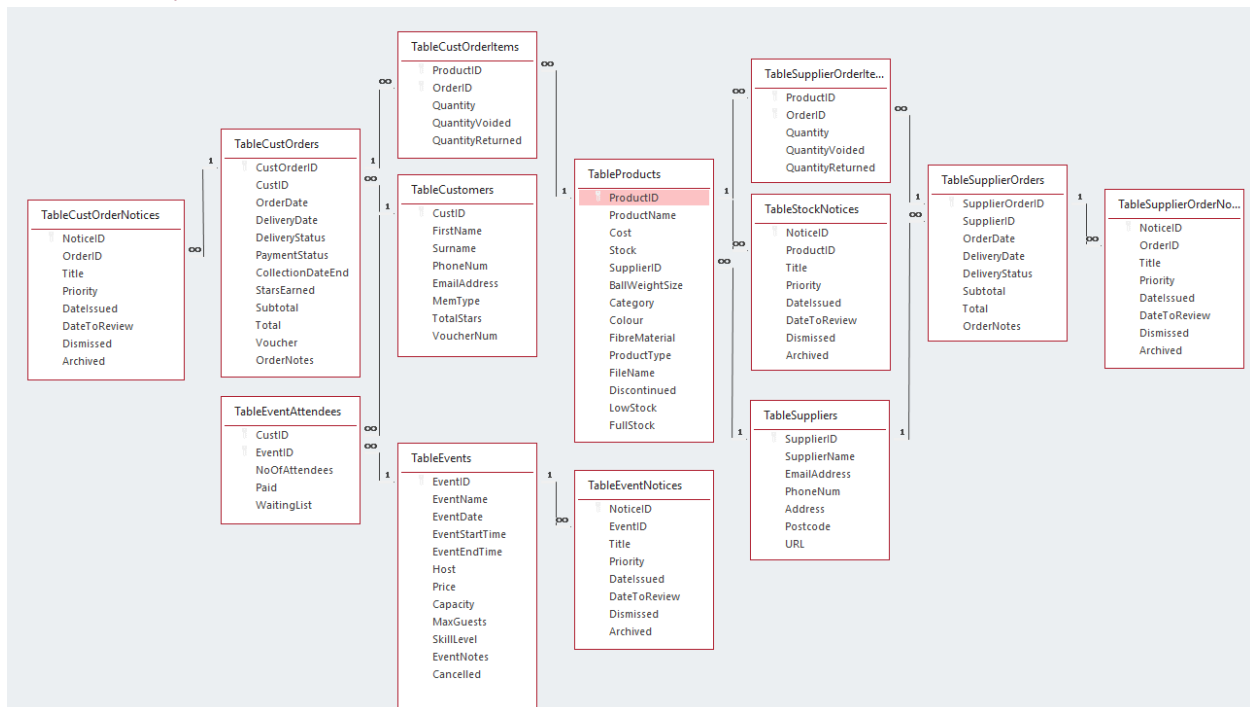
Data Flow Diagram

Below I have illustrated how CRUD will be implemented to link all fundamental components of my program.

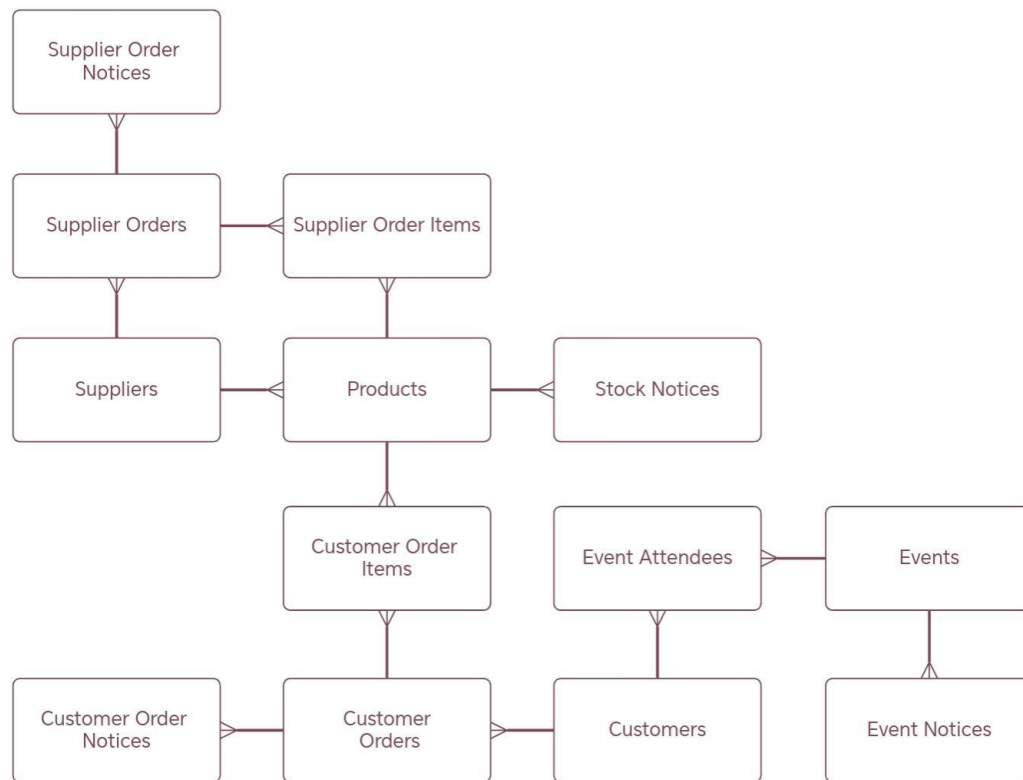


Database Design

Relationships



Entity Relationship Diagram



Tables

Data Integrity

In order to maintain referential integrity across the database, I do not delete records from certain tables.

TableCustomers: When a customer is deleted, the record is set to a 'deleted state'.

TableSuppliers: When a supplier is deleted, the record is set to a 'deleted state'.

TableProducts: Products are discontinued rather than deleted

TableCustOrders: Orders can only be voided/returned

TableCustOrderItems: Items are voided/returned

TableSupplierOrders: Orders can only be voided/returned

TableSupplierOrderItems: Items are voided/returned

TableEvents: Events are cancelled rather than deleted

TableEventAttendees: Attendees can be deleted as this does not affect referential integrity

TableStockNotices: Notices are archived

TableSupplierOrderNotices: Notices are archived

TableCustOrderNotices: Notices are archived

TableEventNotices: Notices are archived

Descriptions

- TableCustomers (CustID, FirstName, Surname, PhoneNum, EmailAddress, MemType, TotalStars, VoucherNum)
- TableSuppliers (SupplierID, SupplierName, EmailAddress, PhoneNum, Address, Postcode, URL)
- TableProducts (ProductID, ProductName, Cost, Stock, SupplierID, BallWeightSize, Category, Colour, FibreMaterial, ProductType, FileName, Discontinued, LowStock, FullStock)
- TableCustOrders (CustOrderID, CustID, Order Date, DeliveryDate, DeliveryStatus, PaymentStatus, CollectionDateEnd, StarsEarned, Subtotal, Total, Voucher, OrderNotes)
- TableCustOrderItems (ProductID, OrderID, Quantity, QuantityVoided, QuantityReturned)
- TableSupplierOrders (SupplierOrderID, SupplierID, OrderDate, DeliveryDate, DeliveryStatus, Subtotal, Total, OrderNotes)
- TableSupplierOrderItems (ProductID, OrderID, Quantity, QuantityVoided, QuantityReturned)
- TableEvents (EventID, EventName, EventDate, EventStartTime, EventEndTime, Host, Price, Capacity, MaxGuests, SkillLevel, EventNotes, Cancelled)
- TableEventAttendees (CustID, EventID, NoOfAttendees, Paid, WaitingList)
- TableStockNotices (NoticeID, ProductID, Title, Priority, DateIssued, DateToReview, Dismissed, Archived)
- TableSupplierOrderNotices (NoticeID, OrderID, Title, Priority, DateIssued, DateToReview, Dismissed, Archived)
- TableCustOrderNotices (NoticeID, OrderID, Title, Priority, DateIssued, DateToReview, Dismissed, Archived)
- TableEventNotices (NoticeID, EventID, Title, Priority, DateIssued, DateToReview, Dismissed, Archived)

TableCustomers

Stores the details of all customers that register with the shop.

	Field Name	Data Type
	CustID	AutoNumber
	FirstName	Short Text
	Surname	Short Text
	PhoneNum	Short Text
	EmailAddress	Short Text
	MemType	Short Text
	TotalStars	Number
	VoucherNum	Number

TableCustomers (CustID, FirstName, Surname, PhoneNum, EmailAddress, MemType, TotalStars, VoucherNum)

Field Name	Data Type	Description	Validation
<u>CustID</u>	Integer (Auto Number)	ID that uniquely identifies each customer in the table	Auto-increment Unique Not null
FirstName	String	Forename of the customer	Required

			Alphabetic characters only
Surname	<i>String</i>	Surname of the customer	Required Alphabetic characters only
PhoneNum	<i>String</i>	Phone number of the customer	Required Optional if email address is provided Valid phone number longer than 7 characters
EmailAddress	<i>String</i>	Email address of the customer	Required Valid email format
MemType	<i>String</i>	SEE Customer Customers can either have Bronze, Silver or Gold membership status.	Values: Gold, Silver, Bronze
TotalStars	<i>Integer</i>	SEE Customer The total number of stars the customer has earned through purchases with the shop.	Default: 0 Minimum: 0
VoucherNum	<i>Integer</i>	SEE Customer The total number of vouchers available for the customer to use.	Default: 0 Minimum: 0

TableSuppliers

Stores the details of all suppliers of all products in the shop.

	Field Name	Data Type
	SupplierID	AutoNumber
	SupplierName	Short Text
	EmailAddress	Short Text
	PhoneNum	Short Text
	Address	Short Text
	Postcode	Short Text
	URL	Long Text

TableSuppliers (SupplierID, SupplierName, EmailAddress, PhoneNum, Address, Postcode, URL)

Field Name	Data Type	Description	Validation
<u>SupplierID</u>	<i>Integer (Auto Number)</i>	ID that uniquely identifies each supplier in the table	Auto-increment Unique Not null
SupplierName	<i>String</i>	Name of supplier	Required Alphabetic characters only
EmailAddress	<i>String</i>	Email address	Required Valid email format
PhoneNum	<i>String</i>	Phone number	Required Valid phone number longer than 7 characters

Address	String	Address relevant to the supplier	Optional No validation
Postcode	String	Postcode relevant to the supplier	Optional No validation
URL	String	The relevant URL for the supplier (e.g. supplier website)	Optional No validation

TableProducts

Stores the details of all products. This table is used for making customer and supplier orders, and stock notices are generated based on stock levels and sales.

	Field Name	Data Type
	ProductID	AutoNumber
	ProductName	Short Text
	Cost	Number
	Stock	Number
	SupplierID	Number
	BallWeightSize	Short Text
	Category	Short Text
	Colour	Short Text
	FibreMaterial	Short Text
	ProductType	Short Text
	FileName	Short Text
	Discontinued	Yes/No
	LowStock	Number
	FullStock	Number

TableProducts (ProductID, ProductName, Cost, Stock, SupplierID, BallWeightSize, Category, Colour, FibreMaterial, ProductType, FileName, Discontinued, LowStock, FullStock)

Field Name	Data Type	Description	Validation
<u>ProductID</u>	Integer (Auto Number)	ID that uniquely identifies each product in the table	Auto-increment Unique Not null
ProductName	String	Product name	Required
Cost	Float	Product cost	Required Min: 0.00
Stock	Integer	Current number of that product available	Required Min: 0
SupplierID	Integer	Foreign key from TableSuppliers: Links the product to its supplier	Must exist in TableSuppliers
BallWeightSize	String	Yarn: Relates to the ball weight of the product e.g. '100g' or '500g'	Must only contain alphanumerical characters If left blank, 'N/A' is entered by default

		Accessory: Relates to the size of the product e.g. 8mm crochet hooks, 9 pieces	
Category	<i>String</i>	Yarn: Yarn Weight e.g. 'DK', '4 Ply' Accessory: General category e.g. 'Crochet Hooks' or 'Knitting Needles'	Must only contain alphanumerical characters If left blank, 'N/A' is entered by default
Colour	<i>String</i>	Colour of the product	Must only contain alphabetical characters If left blank, 'N/A' is entered by default
FibreMaterial	<i>String</i>	Yarn: Fibre of the product e.g. 'Acrylic' Accessory: Material of the product e.g. 'Resin'	Must only contain alphanumerical characters If left blank, 'N/A' is entered by default
ProductType	<i>String</i>	Specifies if the product is yarn or an accessory: affects filters shown	Required Values: "y" and "a"
FileName	<i>String</i>	The file name of the image saved for this product	Optional Valid file format e.g. .jpg or .png Default image is displayed if no file is uploaded
Discontinued	<i>Boolean</i>	True if the product is discontinued	Default: False
LowStock	<i>Integer</i>	Specifies the stock level the product should be at to be considered low on stock	Default: 0 Min: 0
FullStock	<i>Integer</i>	Specifies the stock level the product should be at to be considered as having full stock	Default: 0 Min: LowStock

TableCustOrders

Stores the details of all orders made by customers, whether in store or over the phone. This table is used to keep a record of all purchases made by customers, and to generate an invoice for each order. Order notices are generated based on the delivery date and collection window end date.

	Field Name	Data Type
	CustOrderID	AutoNumber
	CustID	Number
	OrderDate	Date/Time
	DeliveryDate	Date/Time
	DeliveryStatus	Short Text
	PaymentStatus	Short Text
	CollectionDateEnd	Date/Time
	StarsEarned	Number
	Subtotal	Number
	Total	Number
	Voucher	Short Text
	OrderNotes	Long Text

TableCustOrders (CustOrderID, *CustID*, Order Date, DeliveryDate, DeliveryStatus, PaymentStatus, CollectionDateEnd, StarsEarned, Subtotal, Total, Voucher, OrderNotes)

Field Name	Data Type	Description	Validation
<u>CustOrderID</u>	<i>Integer (Auto Number)</i>	ID that uniquely identifies each customer order in the table	Auto-increment Unique Not null
<i>CustID</i>	<i>Integer</i>	<i>Foreign key from TableCustomers</i> : Links the order to the customer that placed it	<i>Must exist in TableCustomers</i>
OrderDate	<i>DateTime</i>	The date this order was placed	Required
DeliveryDate	<i>DateTime</i>	The expected delivery date of this order: the date this order is expected to be processed by	Required Must be later than or equal to OrderDate
DeliveryStatus	<i>String</i>	Status of the order: Order Made, Ready To Collect, Collected, Voided, Returned	Values: Order Made, Ready To Collect, Collected, Voided, Returned
PaymentStatus	<i>String</i>	Payment status of the order: Unpaid, Paid, Cancelled, Refunded, Partially Refunded	Values: Unpaid, Paid, Cancelled, Refunded, Partially Refunded
CollectionDateEnd	<i>DateTime</i>	The date in which the collection window for this order ends	Required Must be later than or equal to DeliveryDate
StarsEarned	<i>Integer</i>	The number of stars earned by this order	Default: 0 Min: 0
Subtotal	<i>Float</i>	The original total cost of the order before any discounts, voids or returns	Required Min: 0.00
Total	<i>Float</i>	The final total paid by the customer after discounts, voids or returns	Required Min: 0.00
Voucher	<i>Integer</i>	If 'N/A', then no voucher was used	Required

		If a voucher was used, this field would be the membership status of the customer at the time this order was placed, specifying what discount was applied to this order	'N/A' if no voucher is used in the order
OrderNotes	String	Order notes to record any unique circumstances for an order	Nullable

TableCustOrderItems

Links the Customer Order and Products tables. Stores the quantities of a product within each order.

	Field Name	Data Type
	ProductID	Number
	OrderID	Number
	Quantity	Number
	QuantityVoided	Number
	QuantityReturned	Number

TableCustOrderItems (ProductID, OrderID, Quantity, QuantityVoided, QuantityReturned)

Field Name	Data Type	Description	Validation
<u>ProductID</u>	Integer	<u>Composite Primary Key:</u> Foreign key from TableProducts Links the product to the items in the order	Must exist in TableProducts
<u>OrderID</u>	Integer	<u>Composite Primary Key:</u> Foreign key from TableCustOrders Links the order to the items	Must exist in TableCustOrders
Quantity	Integer	The original quantity of this product in the order, including voided and returned quantities	Required Min: 1
QuantityVoided	Integer	The quantity of the product that has been voided	Default: 0 Min: 0 Must be less than or equal to Quantity-QuantityReturned
QuantityReturned	Integer	The quantity of the product that has been returned	Must be less than or equal to Quantity-QuantityVoided

TableSupplierOrders

Stores the details of all orders made with suppliers. Used to keep track of orders for products, and order notices are generated based on the delivery date.

	Field Name	Data Type
	SupplierOrderID	AutoNumber
	SupplierID	Number
	OrderDate	Date/Time
	DeliveryDate	Date/Time
	DeliveryStatus	Short Text
	Subtotal	Number
	Total	Number
	OrderNotes	Long Text

TableSupplierOrders (SupplierOrderID, SupplierID, OrderDate, DeliveryDate, DeliveryStatus, Subtotal, Total, OrderNotes)

Field Name	Data Type	Description	Validation
<u>SupplierOrderID</u>	Integer (Auto Number)	ID that uniquely identifies each supplier order in the table	Auto-increment Unique Not null
SupplierID	Integer	Foreign key from TableSuppliers: Links the order to the supplier it was placed with	Must exist in TableSuppliers
OrderDate	DateTime	The date this order was placed	Required
DeliveryDate	DateTime	The expected delivery date of this order	Required Must be later than or equal to Order Date
DeliveryStatus	String	Status of the order: Order Made, Out For Delivery, Delivered, Voided, Returned	Values: Order Made, Out For Delivery, Delivered, Voided, Returned
Subtotal	Float	The original total cost of the order before any discounts, voids or returns	Required Min: 0.00
Total	Float	The final total after discounts, voids or returns	Required Min: 0.00
OrderNotes	String	Order notes to record any unique circumstances for an order	Nullable

TableSupplierOrderItems

Links the Supplier Order and Products tables. Stores the quantities of a product within each order.

	Field Name	Data Type
	ProductID	Number
	OrderID	Number
	Quantity	Number
	QuantityVoided	Number
	QuantityReturned	Number

TableSupplierOrderItems (ProductID, OrderID, Quantity, QuantityVoided, QuantityReturned)

Field Name	Data Type	Description	Validation
<u>ProductID</u>	Integer	Composite Primary Key: Foreign key from TableProducts Links the product to the items in the order	Must exist in TableProducts
<u>OrderID</u>	Integer	Composite Primary Key: Foreign key from TableSupplierOrders Links the order to the items	Must exist in TableSupplierOrders
Quantity	Integer	The original quantity of this product in the order, including voided and returned quantities	Required Min: 1
QuantityVoided	Integer	The quantity of the product that has been voided	Default: 0 Min: 0 Must be less than or equal to Quantity-QuantityReturned
QuantityReturned	Integer	The quantity of the product that has been returned	Must be less than or equal to Quantity-QuantityVoided

TableEvents

Stores details of all events that take place in the shop (i.e. workshops).

	Field Name	Data Type
	EventID	AutoNumber
	EventName	Short Text
	EventDate	Date/Time
	EventStartTime	Date/Time
	EventEndTime	Date/Time
	Host	Short Text
	Price	Number
	Capacity	Number
	MaxGuests	Number
	SkillLevel	Short Text
	EventNotes	Short Text
	Cancelled	Yes/No

TableEvents (EventID, EventName, EventDate, EventStartTime, EventEndTime, Host, Price, Capacity, MaxGuests, SkillLevel, EventNotes, Cancelled)

Field Name	Data Type	Description	Validation
<u>EventID</u>	<i>Integer (Auto Number)</i>	ID that uniquely identifies each event in the table	Auto-increment Unique Not null
EventName	<i>String</i>	Name of event	Required
EventDate	<i>DateTime</i>	Date of event	Required Must be later than or equal to the current date
EventStartTime	<i>DateTime</i>	Start time of event	Required Min Time: 09:00
EventEndTime	<i>DateTime</i>	End time of event	Required Max Time: 20:00 Must be later than or equal to EventStartTime
Host	<i>String</i>	Host of event	Required
Price	<i>Float</i>	Price of event for each attendee	Min: 0.00
Capacity	<i>Integer</i>	Capacity of event	Required Default: 10 Min: 0
MaxGuests	<i>Integer</i>	Maximum number of guests allowed per customer	Default: 0 Min: 0 Must be less than or equal to Capacity
SkillLevel	<i>String</i>	Skill level of event (Beginner, Intermediate etc)	Values: Complete Beginner, Advanced Beginner, Intermediate, Expert
EventNotes	<i>String</i>	Event notes for recording any unique circumstances	Nullable
Cancelled	<i>Boolean</i>	True if the event is cancelled	Default: False

TableEventAttendees

Links the Events and Customers table. Used to keep track of the number of guests a customer has, and their paid and waiting list status.

	Field Name	Data Type
PK	CustID	Number
PK	EventID	Number
	NoOfAttendees	Number
	Paid	Yes/No
	WaitingList	Yes/No

TableEventAttendees (CustID, EventID, NoOfAttendees, Paid, WaitingList)

Field Name	Data Type	Description	Validation
<u>CustID</u>	Integer	<u>Composite Primary Key: Foreign key from TableCustomers</u> Links the customer	Must exist in TableCustomers
<u>EventID</u>	Integer	<u>Composite Primary Key: Foreign key from TableEvents</u> Links the order	Must exist in TableEvents
NoOfAttendees	Integer	Total number of attendees for that customer e.g. if a customer has one guest, the number of attendees would be 2	Required Min: 0 Must be less than MaxGuests of event + 1
Paid	Boolean	True if the customer has paid	Default: True unless set otherwise
WaitingList	Boolean	True if the customer is on the waiting list	Default: False

TableStockNotices

Stores details of notices related to a product and its stock levels/sales.

	Field Name	Data Type
PK	NoticeID	AutoNumber
	ProductID	Number
	Title	Short Text
	Priority	Number
	DateIssued	Date/Time
	DateToReview	Date/Time
	Dismissed	Yes/No
	Archived	Yes/No

TableStockNotices (NoticeID, ProductID, Title, Priority, DateIssued, DateToReview, Dismissed, Archived)

Field Name	Data Type	Description	Validation
------------	-----------	-------------	------------

<u>NoticeID</u>	<i>Integer (Auto Number)</i>	ID that uniquely identifies each stock notice in the table	Auto-increment Unique Not null
ProductID	<i>Integer</i>	<i>Foreign key from TableProducts:</i> Links the product this notice is related to	<i>Must exist in TableProducts</i>
Title	<i>String</i>	Title of the notice e.g. 'No Stock'	Required Values: "No Stock", "Low Stock", "Low Sales", "Selling Fast", "Low Stock: Selling Fast"
Priority	<i>Integer</i>	Priority of the notice: 1,2,3 (3 being highest priority)	Values: 1,2,3
DateIssued	<i>DateTime</i>	The date the notice was created	Required Must be earlier than or equal to the current date
DateToReview	<i>DateTime</i>	The date the notice is dismissed until	Required Must be later than or equal to DateIssued
Dismissed	<i>Boolean</i>	True if the notice has been dismissed (Will be set to false when the review date passes)	Default: False
Archived	<i>Boolean</i>	True if the notice has been archived	Default: False

TableSupplierOrderNotices

Stores details of notices related to a supplier order.

	Field Name	Data Type
	NoticeID	AutoNumber
	OrderID	Number
	Title	Short Text
	Priority	Number
	DateIssued	Date/Time
	DateToReview	Date/Time
	Dismissed	Yes/No
	Archived	Yes/No

TableSupplierOrderNotices (NoticeID, OrderID, Title, Priority, DateIssued, DateToReview, Dismissed, Archived)

Field Name	Data Type	Description	Validation
<u>NoticeID</u>	<i>Integer (Auto Number)</i>	ID that uniquely identifies each supplier order notice in the table	Auto-increment Unique Not null

OrderID	Integer	Foreign key from TableSupplierOrders: Links the supplier order this notice is related to	Must exist in TableSupplierOrders
Title	String	Title of the notice e.g. 'Supply Delivery Overdue: 1 day'	Required Values: "Supply Delivery Overdue" + "1 day", "3 days", ">5 days" depending on priority
Priority	Integer	Priority of the notice: 1,2,3 (3 being highest priority)	Values: 1,2,3
DateIssued	DateTime	The date the notice was created	Required Must be earlier than or equal to the current date
DateToReview	DateTime	The date the notice is dismissed until	Required Must be later than or equal to DateIssued
Dismissed	Boolean	True if the notice has been dismissed (Will be set to false when the review date passes)	Default: False
Archived	Boolean	True if the notice has been archived	Default: False

TableCustOrderNotices

Stores details of notices related to a customer order.

	Field Name	Data Type
	NoticeID	AutoNumber
	OrderID	Number
	Title	Short Text
	Priority	Number
	DateIssued	Date/Time
	DateToReview	Date/Time
	Dismissed	Yes/No
	Archived	Yes/No

TableCustOrderNotices (NoticeID, OrderID, Title, Priority, DateIssued, DateToReview, Dismissed, Archived)

Field Name	Data Type	Description	Validation
<u>NoticeID</u>	Integer (Auto Number)	ID that uniquely identifies each customer order notice in the table	Auto-increment Unique Not null
OrderID	Integer	Foreign key from TableCustOrders: Links the	Must exist in TableCustOrders

		customer order this notice is related to	
Title	<i>String</i>	Title of the notice e.g. 'Order Processing Overdue: 1 day'	Values: "Order Processing Overdue"/"Order Collection Overdue" + "1 day", "3 days", ">5 days" depending on priority
Priority	<i>Integer</i>	Priority of the notice: 1,2,3 (3 being highest priority)	Values: 1,2,3
DateIssued	<i>DateTime</i>	The date the notice was created	Required Must be earlier than or equal to the current date
DateToReview	<i>DateTime</i>	The date the notice is dismissed until	Required Must be later than or equal to DateIssued
Dismissed	<i>Boolean</i>	True if the notice has been dismissed (Will be set to false when the review date passes)	Default: False
Archived	<i>Boolean</i>	True if the notice has been archived	Default: False

TableEventNotices

Stores details of notices related to an event.

	Field Name	Data Type
	NoticeID	AutoNumber
	EventID	Number
	Title	Short Text
	Priority	Number
	DateIssued	Date/Time
	DateToReview	Date/Time
	Dismissed	Yes/No
	Archived	Yes/No

TableEventNotices (NoticeID, EventID, Title, Priority, DateIssued, DateToReview, Dismissed, Archived)

Field Name	Data Type	Description	Validation
<u>NoticeID</u>	<i>Integer (Auto Number)</i>	ID that uniquely identifies each event notice in the table	Auto-increment Unique Not null
EventID	<i>Integer</i>	Foreign key from TableEvents: Links the event this notice is related to	Must exist in TableEvents

Title	<i>String</i>	Title of the notice e.g. 'Event Payments'	Required Values: "Event Payments", "Event Spaces Available"
Priority	<i>Integer</i>	Priority of the notice: 1,2,3 (3 being highest priority)	Values: 1,2,3
DateIssued	<i>DateTime</i>	The date the notice was created	Required Must be earlier than or equal to the current date
DateToReview	<i>DateTime</i>	The date the notice is dismissed until	Required Must be later than or equal to DateIssued
Dismissed	<i>Boolean</i>	True if the notice has been dismissed (Will be set to false when the review date passes)	Default: False
Archived	<i>Boolean</i>	True if the notice has been archived	Default: False

SQL Design

Queries

SQL Statement	Description
<pre>SELECT TableProducts.ProductID, SUM(TableCustOrderItems.Quantity - TableCustOrderItems.QuantityVoided - TableCustOrderItems.QuantityReturned) FROM (TableProducts INNER JOIN TableCustOrderItems ON TableCustOrderItems.ProductID = TableProducts.ProductID) INNER JOIN TableCustOrders ON TableCustOrderItems.OrderID = TableCustOrders.CustOrderID WHERE TableCustOrders.DeliveryDate >= DateAdd('d', - 7, Date()) AND TableProducts.Discontinued = False GROUP BY TableProducts.ProductID;</pre>	Retrieves all products and the total number sold in the past week for each product Used to determine products that are selling fast
<pre>SELECT SUM(TableProducts.Stock+TableSupplierOrderItems .Quantity- TableSupplierOrderItems.QuantityVoided- TableSupplierOrderItems.QuantityReturned) FROM (TableProducts INNER JOIN TableSupplierOrderItems ON TableSupplierOrderItems.ProductID = TableProducts.ProductID) INNER JOIN TableSupplierOrders ON TableSupplierOrderItems.OrderID = TableSupplierOrders.SupplierOrderID WHERE TableSupplierOrders.DeliveryDate > (SELECT MAX(OrderDate) FROM TableSupplierOrders INNER</pre>	Retrieves the original amount of stock of the current product since its last supplier order Used to determine products that are selling fast

<pre>JOIN TableSupplierOrderItems ON TableSupplierOrders.SupplierOrderID = TableSupplierOrderItems.OrderID WHERE TableSupplierOrderItems.ProductID = TableProducts.ProductID) AND TableProducts.ProductID = @pid AND TableProducts.Discontinued = False GROUP BY TableProducts.ProductID;</pre>	
<pre>SELECT TableProducts.ProductID, TableProducts.FullStock, SUM(TableCustOrderItems.Quantity - TableCustOrderItems.QuantityVoided - TableCustOrderItems.QuantityReturned) FROM (TableProducts INNER JOIN TableCustOrderItems ON TableCustOrderItems.ProductID = TableProducts.ProductID) INNER JOIN TableCustOrders ON TableCustOrderItems.OrderID = TableCustOrders.CustOrderID WHERE TableCustOrders.DeliveryDate >= DateAdd('m', - 1, Date()) GROUP BY TableProducts.ProductID, TableProducts.FullStock;</pre>	Retrieves all products and the total number sold in the past month for each product Used to find products considered to have a low number of sales currently
<pre>ELECT DISTINCT TableProducts.ProductID,TableSupplierOrders.Del iveryDate FROM (TableProducts INNER JOIN TableSupplierOrderItems ON TableProducts.ProductID = TableSupplierOrderItems.ProductID) INNER JOIN TableSupplierOrders ON TableSupplierOrderItems.OrderID = TableSupplierOrders.SupplierOrderID WHERE TableSupplierOrderItems.QuantityVoided = 0 AND TableSupplierOrderItems.QuantityReturned = 0 AND (TableSupplierOrders.DeliveryStatus = 'Order Made' OR TableSupplierOrders.DeliveryStatus = 'Out For Delivery')</pre>	Retrieve all products in supplier orders that have yet to be delivered Used to dismiss products with related stock notices issued
<pre>SELECT * FROM TableCustomers WHERE (FirstName LIKE @qu OR Surname LIKE @qu OR EmailAddress LIKE @qu) AND (MemType <> 'Guest' OR MemType <> 'Deleted')</pre>	Retrieves all customers where their first or last name or email address matches the parameterised input Ensures no customers in a deleted or guest state are returned
<pre>SELECT TableCustOrders.* FROM (TableCustOrders INNER JOIN TableCustomers ON TableCustOrders.CustID = TableCustomers.CustID) WHERE TableCustomers.FirstName LIKE @in OR TableCustomers.Surname LIKE @in OR TableCustomers.EmailAddress LIKE @in;</pre>	Retrieves all customer orders where the customer name or email address matches the parameterised input
<pre>SELECT TableSupplierOrders.* FROM (TableSupplierOrders INNER JOIN TableSuppliers ON TableSupplierOrders.SupplierID =</pre>	Retrieves all supplier orders where the supplier name matches the parameterised input

TableSuppliers.SupplierID) WHERE TableSuppliers.SupplierName LIKE @in;	
SELECT DISTINCT TableProducts.ProductID, TableProducts.ProductName, TableCustOrderItems.Quantity,TableCustOrderItems.Qu antityVoided,TableCustOrderItems.QuantityReturned, TableProducts.BallWeightSize,TableProducts.Category ,TableProducts.Colour,TableProducts.FibreMaterial,T ableProducts.ProductType FROM (TableProducts INNER JOIN TableCustOrderItems ON TableProducts.ProductID = TableCustOrderItems.ProductID) INNER JOIN TableCustOrders ON TableCustOrderItems.OrderID = TableCustOrders.@oid;	Retrieves all products and their details from the selected customer order
SELECT DISTINCT TableProducts.ProductID, TableProducts.ProductName, TableSupplierOrderItems.Quantity,TableSupplierOrder Items.QuantityVoided,TableSupplierOrderItems.Quanti tyReturned, TableProducts.BallWeightSize,TableProducts.Category ,TableProducts.Colour,TableProducts.FibreMaterial,T ableProducts.ProductType FROM (TableProducts INNER JOIN TableSupplierOrderItems ON TableProducts.ProductID = TableSupplierOrderItems.ProductID) INNER JOIN TableSupplierOrders ON TableSupplierOrderItems.OrderID = TableSupplierOrders.@oid;	Retrieves all products and their details from the selected supplier order
SELECT TableEvents.EventID, TableEvents.Capacity, SUM (TableEventAttendees.NoOfAttendees) FROM TableEventAttendees LEFT JOIN TableEvents ON TableEventAttendees.EventID = TableEvents.EventID WHERE TableEventAttendees.WaitingList = False GROUP BY TableEvents.EventID, TableEvents.Capacity;	Retrieve IDs of all events and their total number of attendees and capacity
SELECT DISTINCT TableEvents.EventID FROM TableEvents INNER JOIN TableEventAttendees ON TableEvents.EventID = TableEventAttendees.EventID WHERE TableEventAttendees.WaitingList = True;	Retrieve IDs of all events with a waiting list
SELECT DISTINCT TableEvents.EventID FROM (TableCustomers INNER JOIN TableEventAttendees ON TableCustomers.CustID = TableEventAttendees.CustID) INNER JOIN TableEvents ON TableEventAttendees.EventID = TableEvents.EventID WHERE TableEventAttendees.Paid = False ORDER BY TableEvents.EventID;	Retrieve the IDs of all events where there are customers that haven't paid
SELECT DISTINCT TableEvents.EventID FROM TableEvents LEFT JOIN TableEventAttendees ON TableEvents.EventID = TableEventAttendees.EventID GROUP BY TableEvents.EventID HAVING SUM(IIf(TableEventAttendees.Paid = False, 1, 0)) = 0	Retrieve the IDs of all events where all customers have paid by checking the count of attendees that have not paid
SELECT *, 'TableStockNotices' AS Source FROM TableStockNotices WHERE Dismissed = False AND Archived = False UNION ALL SELECT *, 'TableCustOrderNotices' AS Source FROM TableCustOrderNotices WHERE Dismissed = False AND Archived = False UNION ALL SELECT *, 'TableSupplierOrderNotices' AS Source FROM TableSupplierOrderNotices WHERE Dismissed = False	Retrieve all active notices

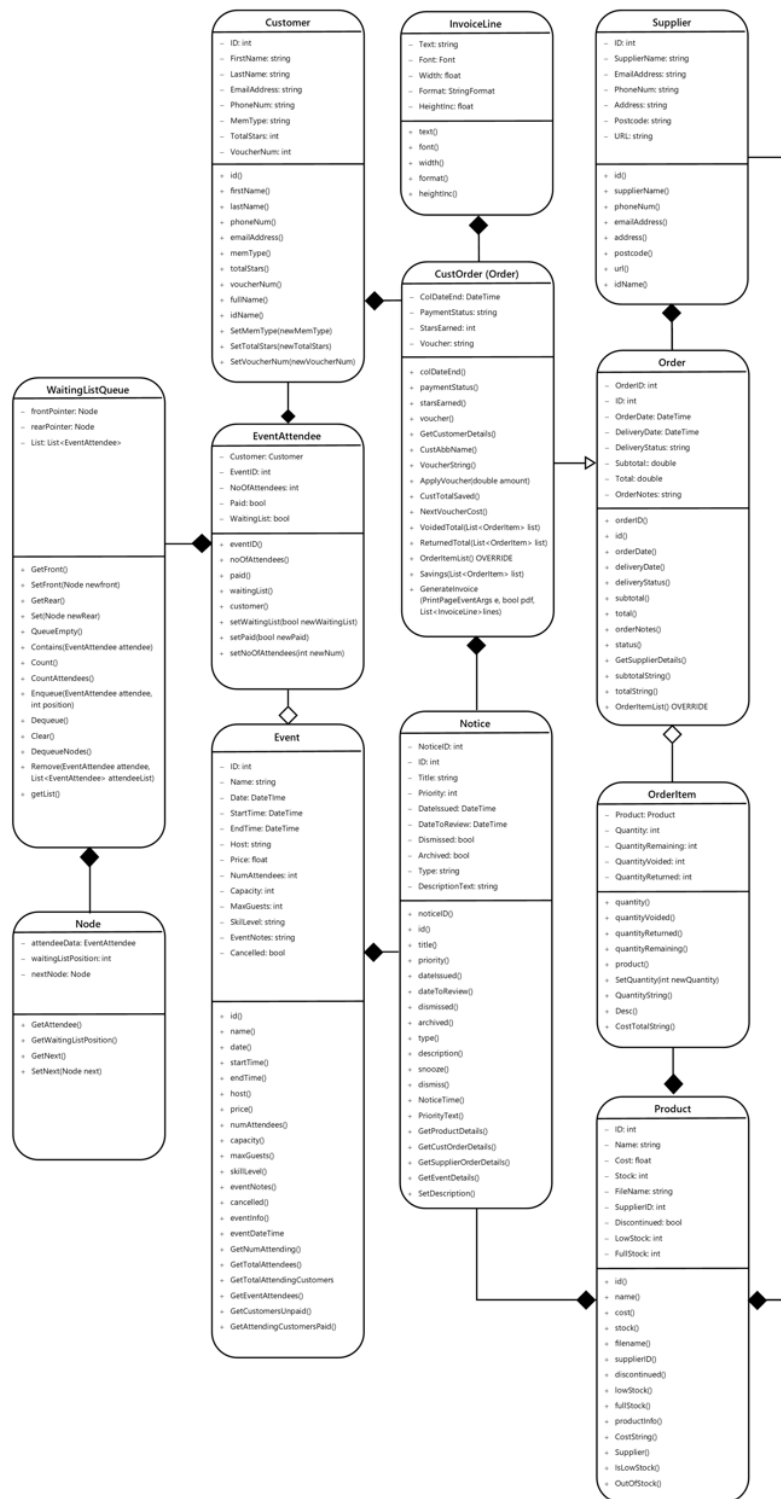
AND Archived = False UNION ALL SELECT *, 'TableEventNotices' AS Source FROM TableEventNotices WHERE Dismissed = False AND Archived = False ORDER BY Source ASC	
SELECT DISTINCT TableProducts.Cost, TableCustOrderItems.QuantityVoided,TableCustOrderIt ems.QuantityReturned FROM (TableProducts INNER JOIN TableCustOrderItems ON TableProducts.ProductID = TableCustOrderItems.ProductID) INNER JOIN TableCustOrders ON TableCustOrderItems.OrderID = TableCustOrders.@eid	Retrieve the cost, quantity voided and quantity returned for all products in one customer order
SELECT SUM (TableEventAttendees.NoOfAttendees) FROM (TableEventAttendees LEFT JOIN TableEvents ON TableEventAttendees.EventID = TableEvents.EventID) WHERE TableEventAttendees.WaitingList = False AND TableEvents.EventID = TableEvents.@eid	Retrieve total number of attendees that are not on the waiting list for an event
SELECT SUM (TableEventAttendees.NoOfAttendees) FROM (TableEventAttendees LEFT JOIN TableEvents ON TableEventAttendees.EventID = TableEvents.EventID) WHERE TableEvents.EventID = TableEvents.@eid	Retrieve total number of attendees (including the waiting list) for an event
SELECT COUNT(*) FROM (TableEventAttendees LEFT JOIN TableEvents ON TableEventAttendees.EventID = TableEvents.EventID) WHERE TableEvents.EventID = TableEvents.@eid	Retrieve total number of customers (including the waiting list, excluding guests)
SELECT DISTINCT TableCustomers.*, TableEventAttendees.NoOfAttendees, TableEventAttendees.Paid, TableEventAttendees.WaitingList FROM (TableCustomers INNER JOIN TableEventAttendees ON TableCustomers.CustID = TableEventAttendees.CustID) INNER JOIN TableEvents ON TableEventAttendees.EventID = TableEvents.@eid	Retrieve all customers and their customer and event attendee details for an event

Statements

SQL Query	Description
UPDATE TableCustomers SET FirstName = @fn, Surname = @sn, PhoneNum = @pn, EmailAddress = @ea, MemType = @mt, TotalStars = @ts, VoucherNum = @va WHERE CustID = @cid	Edits an existing customer
INSERT INTO TableCustomers(FirstName, Surname, PhoneNum, EmailAddress, MemType, TotalStars, VoucherNum) VALUES(@fn,@sn,@pn,@ea,@mt,@ts,@va)	Inserts new customer into customer records
UPDATE TableSuppliers SET SupplierName = @sn, PhoneNum = @pn, EmailAddress = @ea, Address = @ad, Postcode = @pc, URL = @url WHERE SupplierID = @ssid	Edits an existing supplier
INSERT INTO TableSuppliers(SupplierName, PhoneNum, EmailAddress, Address, Postcode, URL) VALUES(@sn,@pn,@ea,@ad,@pc,@url)	Inserts new supplier into supplier records
UPDATE TableProducts SET ProductName = @pn, Cost = @cos, Stock = @sto, SupplierID = @sup, BallWeightSize = @typ, Category = @cat, Colour = @col, FibreMaterial = @fib, FileName = @fn, LowStock = @ls, FullStock = @fs WHERE ProductID = @prid	Edits existing product
INSERT INTO TableProducts(ProductName, Cost, Stock, SupplierID, BallWeightSize, Category, Colour, FibreMaterial, ProductType, FileName,LowStock,FullStock)	Inserts new product into product records

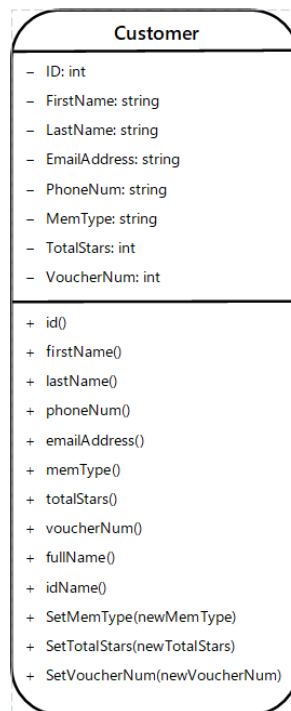
VALUES(@pn,@cos,@sto,@sup,@typ,@cat,@col,@fib,@pty,@fn,@ls,@fs)	
UPDATE TableCustOrders SET OrderDate = @od, DeliveryDate = @dda, DeliveryStatus = @dsa, PaymentStatus = @psa, CollectionDateEnd = @col, OrderNotes = @ordn WHERE CustOrderID = @orid	Edits existing customer order
UPDATE TableSupplierOrders SET OrderDate = @od, DeliveryDate = @dda, DeliveryStatus = @dsa, OrderNotes = @ordn WHERE SupplierOrderID = @orid	Edits existing supplier order
INSERT INTO TableCustOrders(CustID, OrderDate, DeliveryDate, DeliveryStatus, PaymentStatus, CollectionDateEnd, StarsEarned, Subtotal, Total, Voucher, OrderNotes) VALUES(@cid,@ord,@exd,@dsa,@psa,@cod,@ste,@sub,@cos,@vou,@orn)	Inserts new customer order into customer order records
UPDATE TableCustomers SET MemType = @met, TotalStars = @sta, VoucherNum = @von WHERE CustID = @cid	Update customer membership details
INSERT INTO TableCustOrderItems(ProductID, OrderID, Quantity) VALUES(@pid,@oid,@q)	Insert customer order items into customer order items records
INSERT INTO TableSupplierOrders(SupplierID, OrderDate, DeliveryDate, DeliveryStatus, Subtotal, Total, OrderNotes) VALUES(@sid,@ord,@exd,@dsa,@sub,@tot,@orn)	Inserts new supplier order into supplier order records
INSERT INTO TableSupplierOrderItems(ProductID, OrderID, Quantity) VALUES(@pid,@oid,@q)	Insert supplier order items into supplier order items records
UPDATE TableEvents SET EventName = @en, EventDate = @ed, EventStartTime = @est, EventEndTime = @eet, Host = @ho, Price = @pa, Capacity = @cap, MaxGuests = @max, SkillLevel = @ski, EventNotes = @evn WHERE EventID = @eid	Edits existing event
INSERT INTO TableEvents(EventName, EventDate, EventStartTime, EventEndTime, Host, Price, Capacity, MaxGuests, SkillLevel, EventNotes) VALUES(@en,@ed,@est,@eet,@ho,@pr,@cap,@max,@ski,@evn)	Insert new event into event records

UML Diagram

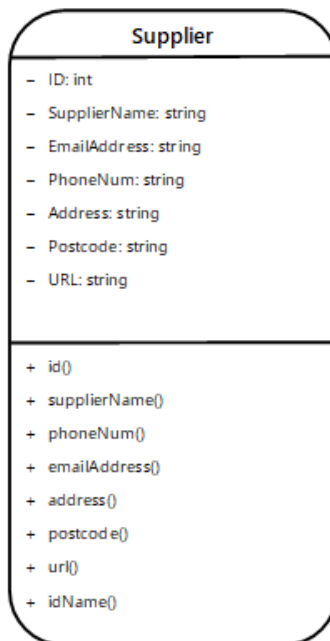


Class Diagrams

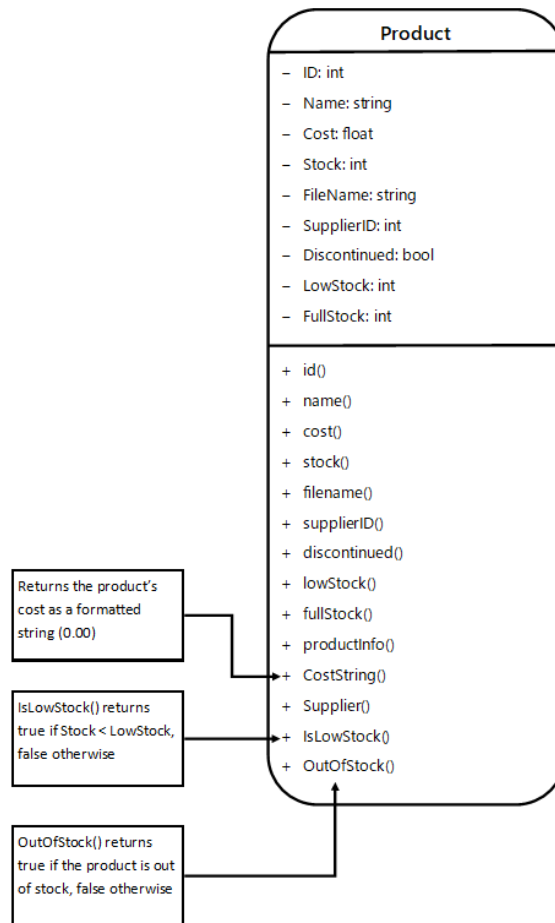
Customer



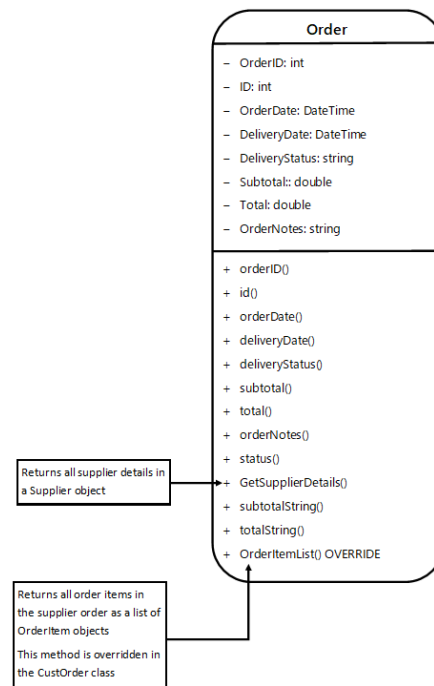
Supplier



Product

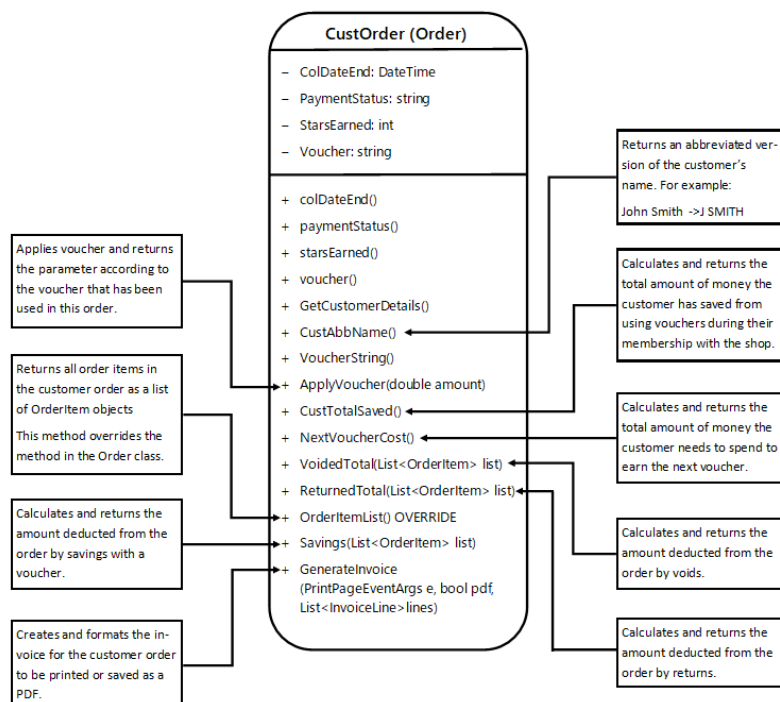


Order

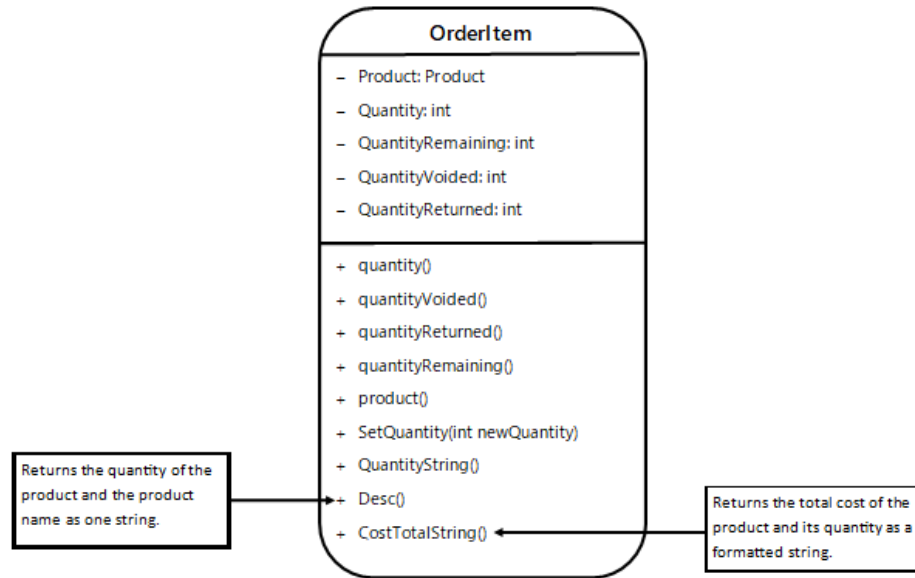


CustOrder

SEE PG.ERROR! BOOKMARK NOT DEFINED. ERROR! REFERENCE SOURCE NOT FOUND. FOR GENERATEINVOICE()

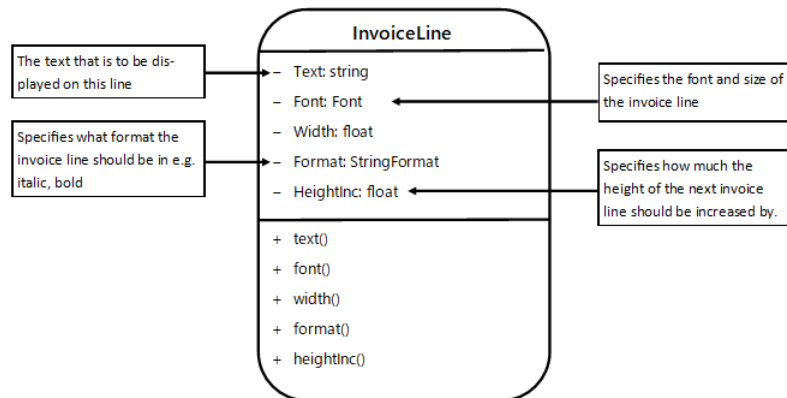


OrderItem

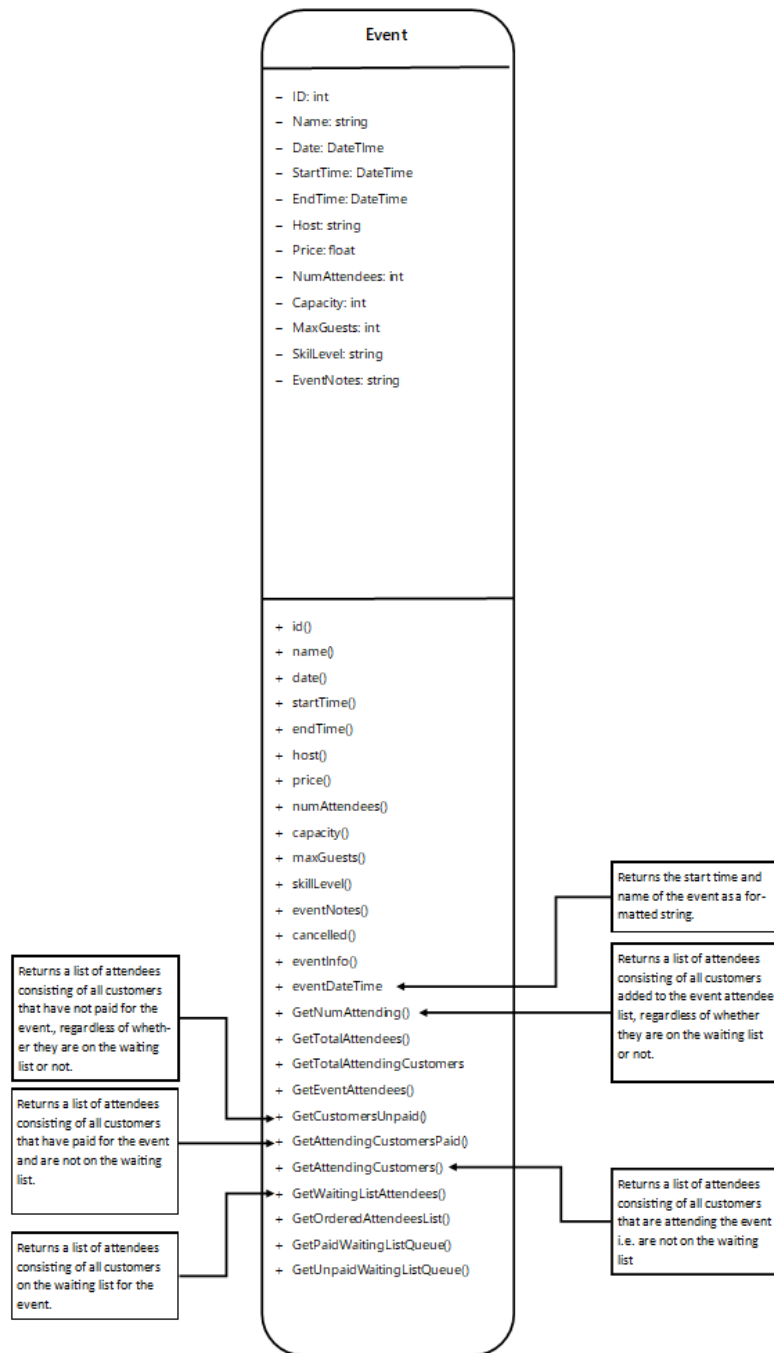


InvoiceLine

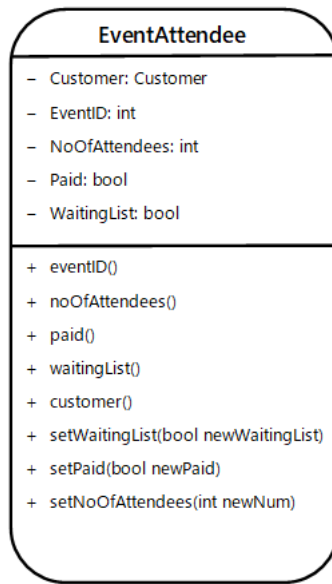
SEE PG.ERROR! BOOKMARK NOT DEFINED. ERROR! REFERENCE SOURCE NOT FOUND.



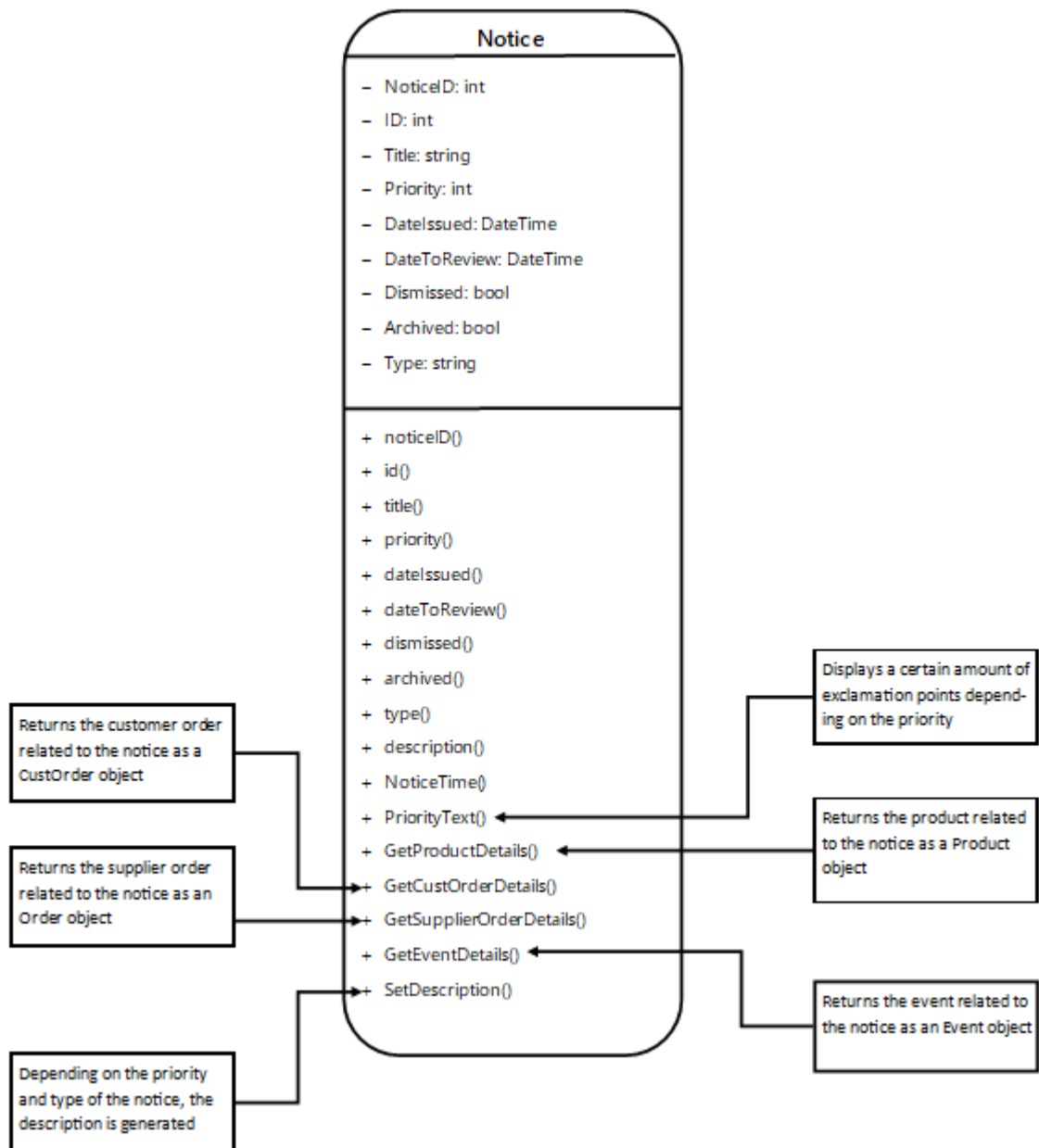
Event



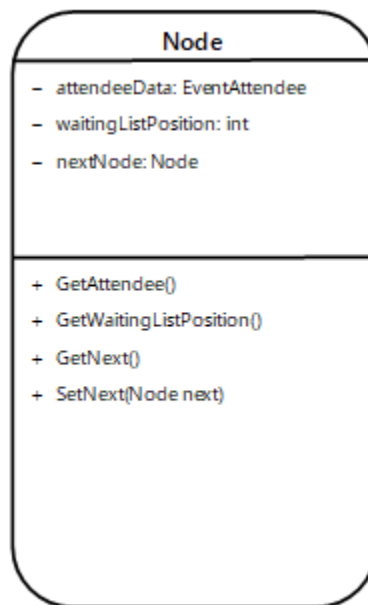
EventAttendee



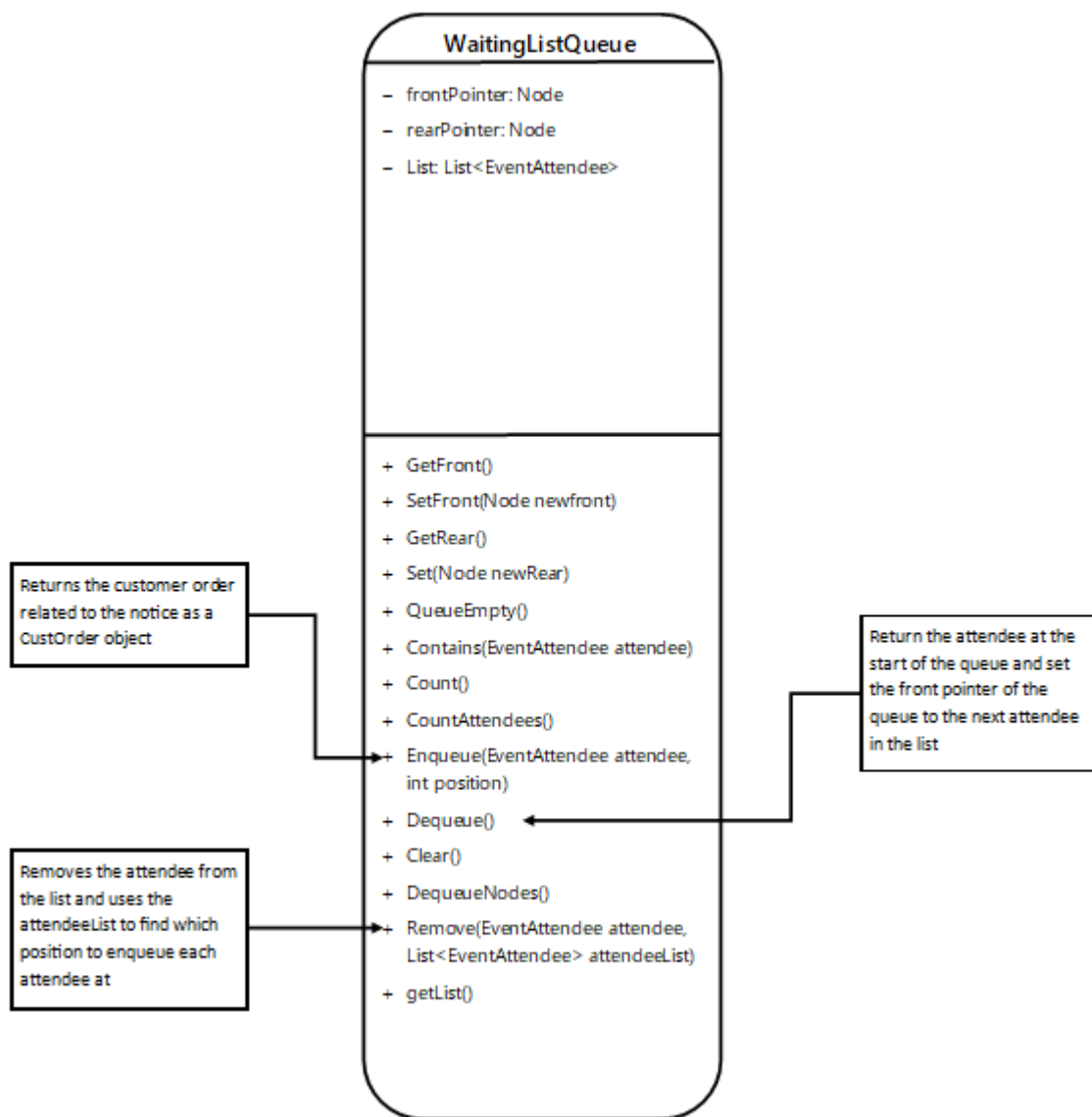
Notice



Node



WaitingListQueue



Algorithms

‘Big Algorithm’: Invoice Generator

RELATED OBJECTIVES: PG.15 GENERATE INVOICE FOR CUSTOMER ORDER

Description

This algorithm is used to generate and print receipts by formatting all information of the order, customer and organising text into a printable format, which can either be saved as a pdf or printed as a continuous receipt.

Explanation

The pseudocode shown explains how each line in the invoice is generated, formatted in the CustOrder class, and printed from the Orders form.

This invoice contains the following components:

- **Header:** Store name, location address
- **Order Details:** Invoice title, order number, date, print date and time
- **Products and Quantities:** All products and their quantities- wraps long product names to fit within the maximum length of the line
- **Voided and Returned items:** Includes the amount deducted by each product that was voided or returned
- **Financial Details:** Subtotal, Total, Voucher Savings, deductions from voids/returns
- **Customer Membership information:** All customer information (if not a Guest or Deleted) including their total stars, membership status and customer name and ID
- **Membership Insights:** Displays a message specific to the customer's membership details and their use of vouchers:
 - If they used a voucher in this order, display the total amount of money saved as a member with the shop
 - If not, display the total amount the customer must spend to earn the next voucher
 - All long messages are wrapped to fit within the maximum line length
- **Thank you message**

This algorithm is created as a method within the CustOrder class, so it can retrieve all information about the order and customer easily for the customer order that the user has currently selected in the Orders form.

Originally, I tested the generation and printing of this invoice by drawing strings as soon as the information required for each line was generated. However, this would cause an issue when the invoice is saved as a PDF, as all strings drawn to the page would be cut off after the height of each line relative to the page exceeded the height of the page. This meant that invoices requiring multiple pages to display all information would not be formatted correctly.

Therefore, I have chosen to implement printing each line by creating the InvoiceLine class: **SEE InvoiceLine**

Each line to be printed is represented as an instance of the InvoiceLine class. This class encapsulates:

- **Text:** The content of the line
- **Font:** Font and font size
- **Format:** Bold, italic, underline and alignment settings

- **Location:** The position of the line relative to the page, defined by x and y coordinates
- **Height increase:** What the height of the next line to be printed should be increased by relative to the height of the current line

I store all the lines in a list so I can systematically manage the printing process across multiple pages if the user is exporting the invoice as a pdf.

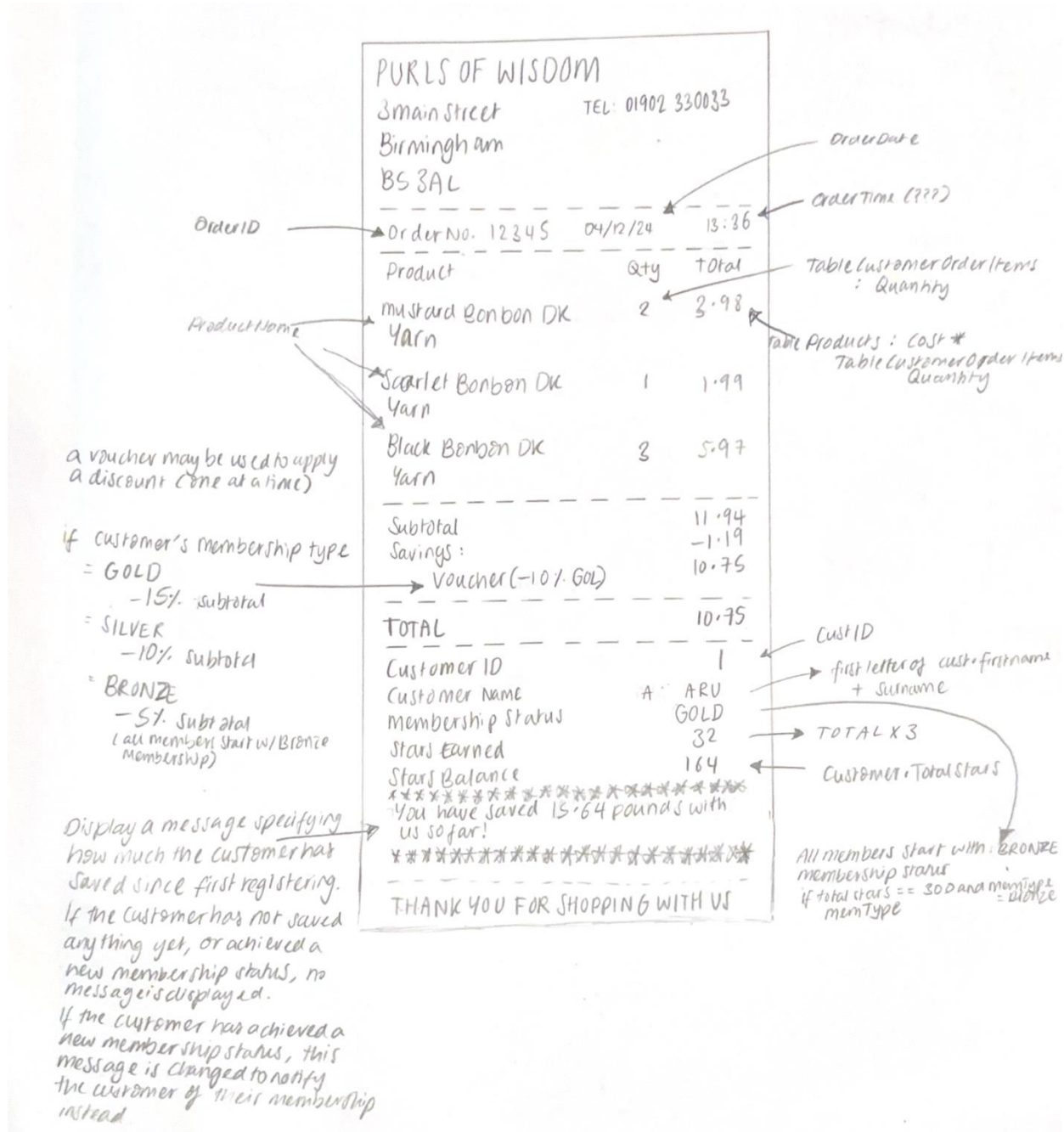
By maintaining a list of all InvoiceLine objects, I can track which lines have been printed and which lines have yet to be printed when a new page is created. This allows for continuity and ensures no lines are skipped or repeated. When the content exceeds the height of one page, the list allows me to resume printing from the correct line on the next page. This avoids having to reprocess the content.

The algorithm checks if the current page is full and then marks the page as incomplete, updating the start index to continue from the next line to be printed on a new page.

Regardless of whether the user is printing the receipt directly or saving it as a PDF, the list allows for flexibility and ensures all lines are correctly formatted and rendered.

Initial Design of Invoice

Before implementing the formatting of all the data in the invoice, I roughly designed how all the data required on an invoice would be laid out first. This provided me with a rough idea of how I should format all order data on the receipt in an organised manner.



Pseudocode

FILE: Orders.cs

```

# FILE: Orders.cs
DEFINE pageNumber
height ← 30
startIndex ← 0
DEFINE lines
DEFINE PrintDialog
SUBROUTINE printInvoiceClick(sender, e)
    # WHEN THE USER CLICKS PRINT INVOICE

    CLEAR lines
    # Ensures that an entirely new invoice is generated
    pageNumber ← 1
    startIndex ← 0
    printInvoice.DocumentName ← "Print Invoice"
    PrintDialog.Document ← "Print Invoice"

    # Configure the page size
    DEFINE PaperSize pageSize
    # Receipt width is fixed at 220
    pageSize.Width ← 220
    printInvoice.DefaultPageSettings.PaperSize ← pageSize

    # Configure print dialog settings
    AllowSelection ← TRUE
    AllowSomePages ← TRUE

    # Show print dialog and print invoice if confirmed by user
    IF PrintDialog user input == DialogResult.OK THEN
        printInvoice()
    END IF
END SUBROUTINE

SUBROUTINE printInvoice(sender, e)
    pdf ← FALSE
    # Initialise the height every time a new page is created
    height ← 30
    IF Print to PDF THEN
        pdf ← TRUE
    END IF
    currentCustomerOrder.GenerateInvoice(e, pdf, lines)
END SUBROUTINE

# FILE: CustOrder.cs
SUBROUTINE GenerateInvoice(e, pdf, lines)
    Customer ← GetCustomerDetails
    orderItemList ← GetOrderItemList
    voidedItems ← false
    returnedItems ← false
    CONSTANT maxLineLength ← 20

    # DEFINE FONTS, BRUSHES, FORMATS AND PENS
    DEFINE FONTS: titleFont, title2Font, boldFont, regularFont, italicFont
    DEFINE drawBrush
    DEFINE FORMATS: Center, Left, Right
    DEFINE blackDashedPen

    IF Customer IS NULL OR orderItemList IS NULL THEN

```

```

        OUTPUT "Invoice creation error: data is missing"
        RETURN
    END IF

    # GENERATE ALL LINES TO BE PRINTED IN A LIST OF LINES
    IF pageNumber == 1 THEN
        # RESETS ALL LINES
        IF LEN (lines) > 0 THEN
            CLEAR lines
        END IF
        startIndex ← 0
        morePages ← false

        # HEADER INFORMATION
        ADD header ('Purls of Wisdom', address, contact info) to lines
        ADD horizontal line to lines

        # ORDER INFORMATION
        ADD "Invoice" title and order details (date, order number, date
and time printed to lines
        ADD horizontal line to lines

        # ORDER ITEM HEADERS
        ADD "Product" and "Price" headers to lines
        ADD horizontal line to lines

        # PRODUCTS IN THE ORDER
        FOR EACH orderItem IN orderItemList
            ADD product cost and quantity to lines
            currentProductLine ← orderItem.product.name

            # WRAPPING PRODUCT NAME
            WHILE LEN (currentProductLine) > maxLineLength
                lastSpaceIndex ← index of the last space before
maxLineLength
                IF at least one space is found
                    ADD SUBSTRING (0, lastSpaceIndex,
currentProductLine)
                    currentProductLine ← SUBSTRING
(lastSpaceIndex, LEN (currentProductLine), currentProductLine)
                ELSE
                    ADD SUBSTRING (0, maxLineLength,
currentProductLine)
                    currentProductLine ← SUBSTRING
(lastSpaceIndex, LEN (currentProductLine), currentProductLine)
                END IF
            END WHILE
            # ADD THE REMAINING STRING TO LINES
            ADD currentProductLine to lines

            # ADD VOIDED ITEMS IF APPLICABLE AND THE ORDER IS NOT
VOIDED
            IF orderDeliveryStatus IS NOT "Voided" THEN
                ADD voided items to lines
                voidedItems ← TRUE
            END IF
        END IF
    END IF

```

```

# ADD RETURNED ITEMS IF APPLICABLE AND THE ORDER IS NOT
RETURNED
    IF orderDeliveryStatus IS NOT "Returned" THEN
        ADD returned items to lines
        returnedItems ← TRUE
    END IF
END FOR EACH

ADD horizontal line to lines

# SUBTOTAL, TOTAL, VOIDS, RETURNS
ADD subtotal to lines
IF orderDeliveryStatus IS "Voided" OR "Returned" THEN
    ADD voided or returned order details to lines
ELSE
    IF voucherRedeemed THEN
        ADD voucher savings to lines
    END IF
    IF voidedItems THEN
        ADD voided items deductions to lines
    END IF
    IF returnedItems THEN
        ADD returned items deductions to lines
    END IF
END IF

ADD horizontal line to lines
ADD total to lines
ADD horizontal line to lines

# CUSTOMER MEMBER INFORMATION
IF customer IS NOT (Deleted OR Guest)
    ADD customer details (ID, name, membership type, stars) to
lines
    IF customer used a voucher OR orderDeliveryStatus ==
Voided OR orderDeliveryStatus == Returned THEN
        currentLine ← amount of money saved in total by the
customer in the shop
    ELSE
        currentLine ← amount of money that needs to be spent
to earn the next voucher
    END IF

# WRAPPING MEMBERSHIP MESSAGE
WHILE LEN (currentLine) > maxLineLength
    lastSpaceIndex ← index of the last space before
maxLineLength
    IF at least one space is found
        ADD SUBSTRING (0, lastSpaceIndex, currentLine)
        currentLine ← SUBSTRING (lastSpaceIndex, LEN
(currentLine), currentLine)
    ELSE
        ADD SUBSTRING (0, maxLineLength, currentLine)
        currentLine ← SUBSTRING (lastSpaceIndex, LEN
(currentLine), currentLine)
    END IF
END WHILE

```

```

        END IF
        # THANK YOU MESSAGE
        ADD thank-you message to lines
    ELSE IF pageNumber != 1 THEN
        e.HasMorePages ← FALSE
    END IF

    # PRINT LINES BY LOOPING THROUGH THE LIST
    IF pdf THEN
        # IF PRINTING AS A PDF, CONSIDER PAGE BREAKS
        FOR line FROM startIndex TO END OF lines
            IF height of line < pageHeight THEN
                DRAW line on e.Graphics
                INCREMENT height of line
            ELSE
                e.HasMorePages ← TRUE
                INCREMENT pageNum
                startIndex ← currentLineIndex
                BREAK
            END IF
        END FOR
    ELSE
        FOR line IN lines
            DRAW line on e.Graphics
            INCREMENT height of line
        END FOR
    END IF
ENDSUBROUTINE

```

FILE: CustOrder.cs

```

SUBROUTINE GenerateInvoice(e, pdf, lines)
    Customer ← GetCustomerDetails
    orderItemList ← GetOrderItemList
    voidedItems ← false
    returnedItems ← false
    CONSTANT maxLineLength ← 20

    # DEFINE FONTS, BRUSHES, FORMATS AND PENS
    DEFINE FONTS: titleFont, title2Font, boldFont, regularFont, italicFont
    DEFINE drawBrush
    DEFINE FORMATS: Center, Left, Right
    DEFINE blackDashedPen

    IF Customer IS NULL OR orderItemList IS NULL THEN
        OUTPUT "Invoice creation error: data is missing"
        RETURN
    END IF

    # GENERATE ALL LINES TO BE PRINTED IN A LIST OF LINES
    IF pageNumber == 1 THEN
        # RESETS ALL LINES
        IF LEN (lines) > 0 THEN
            CLEAR lines
        END IF
        startIndex ← 0
    END IF

```

```

morePages ← false

# HEADER INFORMATION
ADD header ('Purls of Wisdom', address, contact info) to lines
ADD horizontal line to lines

# ORDER INFORMATION
ADD "Invoice" title and order details (date, order number, date
and time printed to lines
ADD horizontal line to lines

# ORDER ITEM HEADERS
ADD "Product" and "Price" headers to lines
ADD horizontal line to lines

# PRODUCTS IN THE ORDER
FOR EACH orderItem IN orderItemList
    ADD product cost and quantity to lines
    currentProductLine ← orderItem.product.name

    # WRAPPING PRODUCT NAME
    WHILE LEN (currentProductLine) > maxLineLength
        lastSpaceIndex ← index of the last space before
maxLineLength
        IF at least one space is found
            ADD SUBSTRING (0, lastSpaceIndex,
currentProductLine)
            currentProductLine ← SUBSTRING
(lastSpaceIndex, LEN (currentProductLine), currentProductLine)
        ELSE
            ADD SUBSTRING (0, maxLineLength,
currentProductLine)
            currentProductLine ← SUBSTRING
(lastSpaceIndex, LEN (currentProductLine), currentProductLine)
        END IF
    END WHILE
    # ADD THE REMAINING STRING TO LINES
    ADD currentProductLine to lines

    # ADD VOIDED ITEMS IF APPLICABLE AND THE ORDER IS NOT
VOIDED
    IF orderDeliveryStatus IS NOT "Voided" THEN
        ADD voided items to lines
        voidedItems ← TRUE
    END IF
    # ADD RETURNED ITEMS IF APPLICABLE AND THE ORDER IS NOT
RETURNED
    IF orderDeliveryStatus IS NOT "Returned" THEN
        ADD returned items to lines
        returnedItems ← TRUE
    END IF
END FOR EACH

ADD horizontal line to lines

# SUBTOTAL, TOTAL, VOIDS, RETURNS
ADD subtotal to lines

```



```

    IF orderDeliveryStatus IS "Voided" OR "Returned" THEN
        ADD voided or returned order details to lines
    ELSE
        IF voucherRedeemed THEN
            ADD voucher savings to lines
        END IF
        IF voidedItems THEN
            ADD voided items deductions to lines
        END IF
        IF returnedItems THEN
            ADD returned items deductions to lines
        END IF
    END IF

    ADD horizontal line to lines
    ADD total to lines
    ADD horizontal line to lines

    # CUSTOMER MEMBER INFORMATION
    IF customer IS NOT (Deleted OR Guest)
        ADD customer details (ID, name, membership type, stars) to
lines
        IF customer used a voucher OR orderDeliveryStatus ==
Voided OR orderDeliveryStatus == Returned THEN
            currentLine ← amount of money saved in total by the
customer in the shop
        ELSE
            currentLine ← amount of money that needs to be spent
to earn the next voucher
        END IF

        # WRAPPING MEMBERSHIP MESSAGE
        WHILE LEN (currentLine) > maxLineLength
            lastSpaceIndex ← index of the last space before
maxLineLength
            IF at least one space is found
                ADD SUBSTRING (0, lastSpaceIndex, currentLine)
                currentLine ← SUBSTRING (lastSpaceIndex, LEN
(currentLine), currentLine)
            ELSE
                ADD SUBSTRING (0, maxLineLength, currentLine)
                currentLine ← SUBSTRING (lastSpaceIndex, LEN
(currentLine), currentLine)
            END IF
        END WHILE
    END IF

    # THANK YOU MESSAGE
    ADD thank-you message to lines
ELSE IF pageNumber != 1 THEN
    e.HasMorePages ← FALSE
END IF

# PRINT LINES BY LOOPING THROUGH THE LIST
IF pdf THEN
    # IF PRINTING AS A PDF, CONSIDER PAGE BREAKS
    FOR line FROM startIndex TO END OF lines
        IF height of line < pageHeight THEN

```

```

        DRAW line on e.Graphics
        INCREMENT height of line
    ELSE
        e.HasMorePages ← TRUE
        INCREMENT pageNum
        startIndex ← currentLineIndex
        BREAK
    END IF
END FOR
ELSE
    FOR line IN lines
        DRAW line on e.Graphics
        INCREMENT height of line
    END FOR
END IF
ENDSUBROUTINE

```

Homepage Calendar

Description

The calendar system dynamically generates a monthly view that consolidates events, collections, and deliveries, fulfilling the client's requirement for a centralized schedule overview.

Each time the calendar updates, it clears previous data from the FlowLayoutPanel controls representing each day of the week. The system retrieves the month name and year and displays it, while a DateTime object determines the first day of the month and total days in that month.

To align dates correctly, blank placeholders (CalendarBlank) are inserted before the first actual date. Then, the system loops through each day of the month, creating CalendarDay objects that store the date, events, and relevant shop activities for that day. These objects are then assigned to the appropriate day-of-week panel, ensuring proper alignment. The calendar updates when the user switches months by calling this routine again, ensuring the displayed data is always accurate and formatted correctly.

Note: See CalendarDay and CalendarBox code for the DisplayEvents, DisplayNumDueDeliveries and DisplayNumCollections subroutines.

Pseudocode

```

SUBROUTINE UpdateCalendar(month, year)
    # Clear previous calendar data
    ClearControls()

    # Get first day of the month and total days in month
    startOfMonth ← Date(year, month, 1)
    daysInMonth ← GetDaysInMonth(year, month)
    daysOfWeek ← GetDayOfWeek(startOfMonth)

    # Adjust daysOfWeek to match indexing (Sunday = 7)
    IF daysOfWeek = 0 THEN
        daysOfWeek ← 7
    ENDIF

    # Add blank placeholders to align first date correctly
    FOR i ← 1 TO daysOfWeek - 1

```

```

        dayName ← GetDayName(i)
        blankControl ← CreateBlankControl()
        ADD blankControl TO FlowLayoutPanel(dayName)
    ENDFOR

    # Populate the calendar with dates and data for each day
    FOR i ← 1 TO daysInMonth
        dateString ← FormatDate(i, month, year)
        date ← ConvertToDate(dateString)

        // Create calendar day object with necessary details
        calendarDay ← CreateCalendarDay(i, month, year)
        calendarDay.DisplayEvents(LoadEvents(i, month, year))
        calendarDay.DisplayNumDueDeliveries(i, month, year)
        calendarDay.DisplayNumCollections(i, month, year)

        # Assign the calendar day to the correct weekday panel
        dayOfWeek ← GetDayOfWeek(date)
        ADD calendarDay TO FlowLayoutPanel(dayOfWeek)
    END FOR
END SUBROUTINE

```

Filtering Products

Description

To easily navigate through all products in the system, I have created a filtering system to easily find selections of products and allow the user to easily do so while creating orders and checking inventory.

This algorithm dynamically builds and executes a SQL query to filter products based on user selected criteria using the checked list boxes, check boxes and radio buttons on the user interface. It ensures that multiple filters can be applied simultaneously and results are shown in the data grid view.

Pseudocode

```

SUBROUTINE ApplyFilters()
    # Base query
    baseCommandText ← "SELECT * FROM TableProducts WHERE"

    # Initialize filter conditions
    categoryFilterCommand ← " AND Category IN ("
    fibreFilterCommand ← " AND FibreMaterial IN ("
    colourFilterCommand ← " AND Colour IN ("
    supplierFilterCommand ← " AND SupplierID IN ("

    # Get selected filters from UI
    selectedCategoryFilters ← GetCheckedItems(clsbCategory)
    selectedFibreFilters ← GetCheckedItems(clsbFibreMaterial)
    selectedColourFilters ← GetCheckedItems(clsbColour)
    selectedStockFilters ← GetCheckedItems(clsbStock)
    selectedSupplierFilters ← GetCheckedItems(clsbSuppliers)

    # Apply stock filters
    IF selectedStockFilters IS NOT EMPTY THEN
        filtersApplied ← TRUE
        FOR EACH stockFilter IN selectedStockFilters DO

```

```

        condition ← ""
        IF stockFilter = "In Stock" THEN
            condition ← " Stock > 0"
        ELSE IF stockFilter = "No Stock" THEN
            condition ← " Stock = 0"
        ELSE IF stockFilter = "Low Stock" THEN
            condition ← " Stock < LowStock"
        END IF

        # Append stock condition to query
        IF baseCommandText ENDS WITH "E" THEN
            baseCommandText ← baseCommandText + " (" + condition
        ELSE
            baseCommandText ← baseCommandText + " OR" + condition
        END IF
    END FOR
    baseCommandText ← baseCommandText + ")"
ELSE
    baseCommandText ← baseCommandText + " Stock >= 0"
END IF

# Apply product type filter
IF rbtnAll IS NOT CHECKED THEN
    baseCommandText ← baseCommandText + " AND ProductType = @ya"
ENDIF

# Apply category filters
IF selectedCategoryFilters IS NOT EMPTY THEN
    filtersApplied ← TRUE
    AppendFiltersToCommand(categoryFilterCommand,
selectedCategoryFilters)
    baseCommandText ← baseCommandText + categoryFilterCommand + ")"
END IF

# Apply fibre material filters
IF selectedFibreFilters IS NOT EMPTY THEN
    filtersApplied ← TRUE
    AppendFiltersToCommand(fibreFilterCommand, selectedFibreFilters)
    baseCommandText ← baseCommandText + fibreFilterCommand + ")"
END IF

# Apply colour filters
IF selectedColourFilters IS NOT EMPTY THEN
    filtersApplied ← TRUE
    AppendFiltersToCommand(colourFilterCommand, selectedColourFilters)
    baseCommandText ← baseCommandText + colourFilterCommand + ")"
END IF

# Apply supplier filters
IF selectedSupplierFilters IS NOT EMPTY THEN
    filtersApplied ← TRUE
    FOREACH supplierFilter IN selectedSupplierFilters DO
        IF supplierFilterCommand ENDS WITH "(" THEN
            supplierFilterCommand ← supplierFilterCommand +
supplierFilter.id
        ELSE

```

```

        supplierFilterCommand ← supplierFilterCommand + "," +
supplierFilter.id
        ENDIF
    END FOREACH
    baseCommandText ← baseCommandText + supplierFilterCommand + ")"
END IF

# Apply discontinued filter
IF chbxDiscontinuedFilter IS CHECKED THEN
    filtersApplied ← TRUE
    baseCommandText ← baseCommandText + " AND Discontinued = True"
ELSE
    baseCommandText ← baseCommandText + " AND Discontinued = False"
END IF

# Determine selected product type
IF rbtnYarn IS CHECKED THEN
    tempProductType ← "y"
ELSE IF rbtnAcc IS CHECKED THEN
    tempProductType ← "a"
END IF

# Execute query
connection ← OpenDatabaseConnection("shopDB.mdb")
cmd ← CreateCommand(baseCommandText, connection)
cmd.AddParameter("@ya", tempProductType)
reader ← ExecuteQuery(cmd)

# Load products into dgv, if any results
# If no results, show a message box
IF reader HAS ROWS THEN
    dgvProducts.DataSource ← reader
    currentCommand ← baseCommandText
    tbxSearch.Text ← ""
ELSE
    ShowMessage("No products found.", "No Results")
END IF

CloseDatabaseConnection(connection)
HighlightProductsStock()
END SUBROUTINE

# Function to append multiple filters to the command string
FUNCTION AppendFiltersToCommand(filterCommand, selectedFilters)
    FOREACH filter IN selectedFilters DO
        IF filterCommand ENDS WITH "(" THEN
            filterCommand ← filterCommand + "'" + filter + "'"
        ELSE
            filterCommand ← filterCommand + "," + "'" + filter + "'"
        END IF
    END FOREACH
    RETURN filterCommand
END FUNCTION

```

Event Waiting List Allocation

RELATED OBJECTIVES: PG. 18 ALLOCATE SPACES TO ANY CUSTOMERS ON THE WAITING LIST ACCORDING TO THE FOLLOWING CONDITIONS:

Description

This system is implemented when a user creates or edits an event and modifies the list of attendees. The algorithm automatically allocates spaces to and from the waiting list based on paid status and the number of guests a customer has.

To achieve this, a priority queue (see *WaitingListQueue*) is used to efficiently manage attendees.

Below, three flowcharts illustrate the allocation process. Since this system is primarily used when adding or removing customers from the list, I have included flowcharts detailing these operations. Additionally, the `updateWaitingList()` subroutine is provided to demonstrate the main allocation loop.

Flowcharts

Event: btnAddCustomer_Click



Figure 1: When a customer is added to the list of attendees

Event: removeCustomerToolStripMenuItem_Click_1

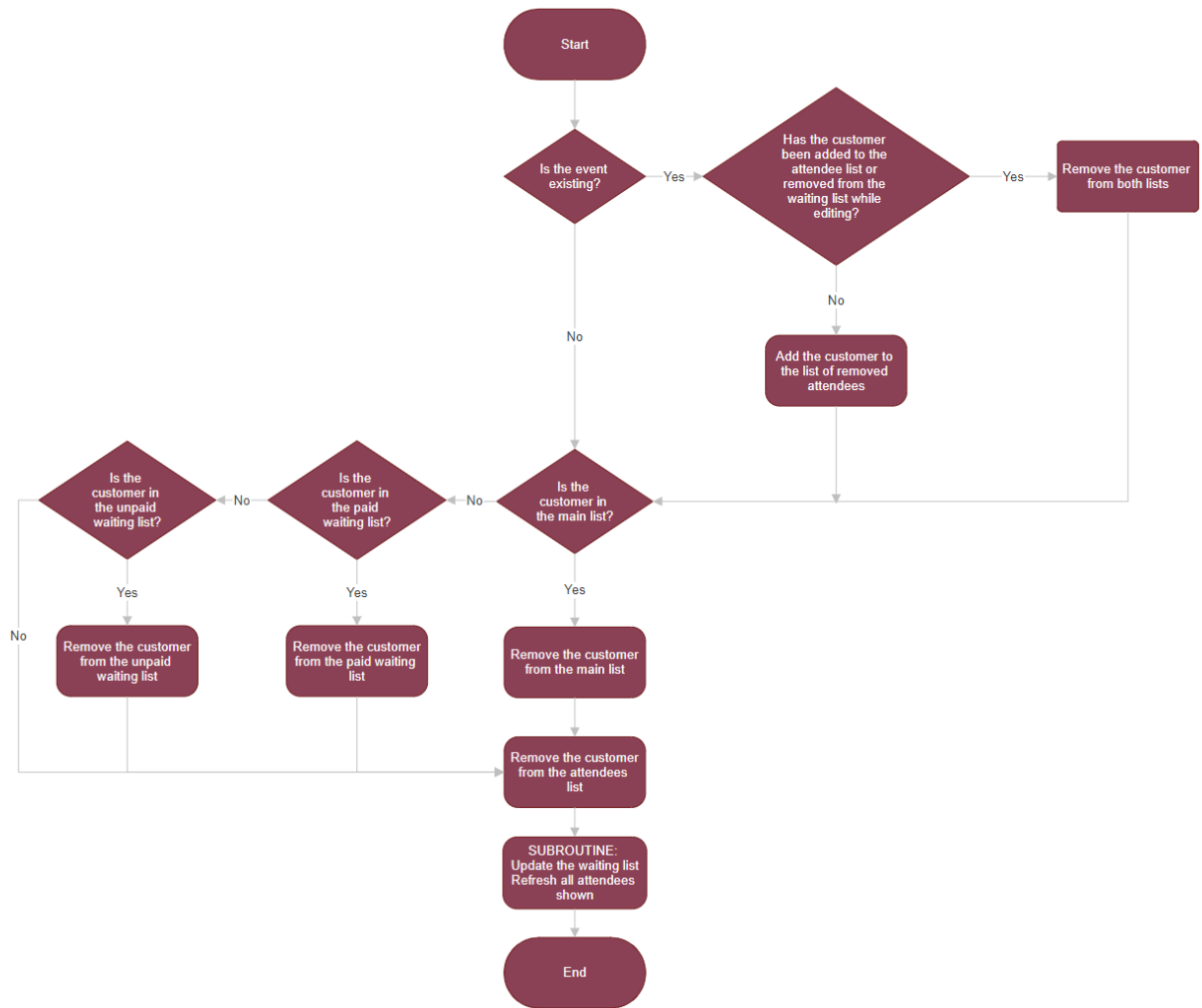


Figure 2 When a customer is removed from the list of attendees

Subroutine: updateWaitingList()



Figure 3 When a customer is added/removed, the capacity of the event has changed or a customer on the event has been deleted

Issuing and Updating Notices

RELATED OBJECTIVES: PG. 19 ISSUING AND UPDATING NOTICES

Description

Notices are generated for customer and supplier orders, events and stock. Each type of notice is generated similarly, with similar checks for the presence of a notice, and creating, archiving and dismissing notices accordingly. Below, an example of how notices are issued and updated is shown below, specifically referring to the stock notice system.

This flowchart describes the process of issuing and updating stock notices for one product.

Flowchart

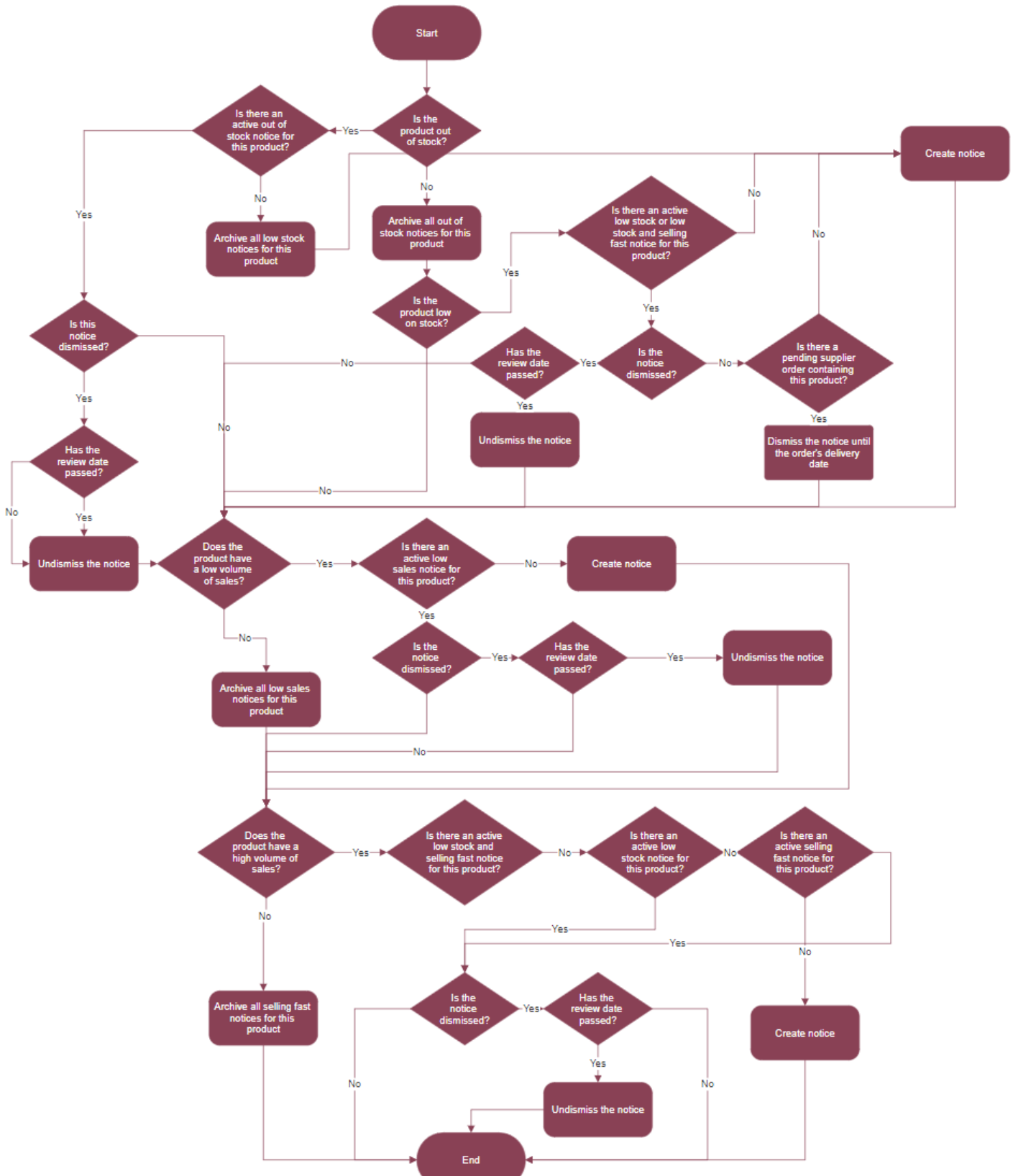


Figure 4 Issuing and Updating Stock Notices

Sorting Notices

RELATED OBJECTIVES: PG. 19 VIEWING NOTICES

Description

On the Notices form, a list of all active notices is displayed. Since there can be many active notices, managing them in a single list can become overwhelming for the user. To improve usability, I will implement sorting functionality based on priority and review date, allowing users to quickly identify the most important and recently issued notices.

I have chosen to implement a merge sort algorithm because it efficiently handles large datasets with a time complexity of $O(n \log n)$, making it well-suited for sorting a high volume of notices.

Explanation

The pseudocode shown explains how notices shown in the flow layout panel on the notices form are sorted, either by default or according to the sort settings selected by the user.

By default, the user will see all notices sorted by review date in descending order, and for notices with the same review date, they will be sorted in descending priority.

The notices will be sorted by review date first, then by priority.

All notices are first sorted based on their review date. This determines the primary order in which notices are displayed.

The Boolean variables `ascReview` and `ascPriority` are used to determine what order to sort notices in. If each of these variables are true, notices are sorted in ascending order of that particular attribute.

The merge sort function can be used to sort notices by review date or priority, depending on the parameters passed in. It can also be used to sort in both ascending and descending order, again depending on the parameters passed into the function.

This function is used to sort the array of notices by review date first, so all notices are grouped by their review date in the desired order in the list of notices.

Grouping all notices by review date allows for finer sorting by priority within each review date group.

The merge sort function is used again to sort each group of notices by priority in the desired order.

All notices have then been sorted by review date, then by priority. All groups are combined into a single list while preserving the order.

This final list is used to display notices in the flow layout panel in the desired order.

Pseudocode

```
FUNCTION merge(left, right, orderBy, asc)
  # Size of the new array
  mergedArraySize ← LEN (left) + LEN (right)
  ARRAY merged[mergedArraySize]

  indexLeft ← 0
```

```

indexRight ← 0
indexMerged ← 0

# While there are still items to merge
WHILE indexLeft < LEN (left) AND indexRight < LEN (right)
    SWITCH orderBy
        # If ordering by review date
        CASE "Date To Review":
            # Ascending order of review date
            IF asc THEN
                # Find the earlier date of the two dates being compared
                and add it to the merged array
                IF left[indexLeft].dateToReview <
right[indexRight].dateToReview THEN
                    merged[indexMerged] ← left[indexLeft]
                    indexLeft ← indexLeft + 1
                ELSE
                    merged[indexMerged] ← right[indexRight]
                    indexRight ← indexRight + 1
                END IF
            # Descending order of review date
            ELSE
                # Find the later date of the two dates being compared
                and add it to the merged array
                IF left[indexLeft].dateToReview >
right[indexRight].dateToReview THEN
                    merged[indexMerged] ← left[indexLeft]
                    indexLeft ← indexLeft + 1
                    indexMerged ← indexMerged + 1
                ELSE
                    merged[indexMerged] ← right[indexRight]
                    indexRight ← indexRight + 1
                    indexMerged ← indexMerged + 1
                END IF
            END IF
        # If ordering by priority
        CASE "Priority":
            # Ascending order of priority
            IF asc THEN
                # Find the notice with the lowest priority out of the
                two notices being compared and add it to the merged array
                IF left[indexLeft].priority < right[indexRight].priority
THEN
                    merged[indexMerged] ← left[indexLeft]
                    indexLeft ← indexLeft + 1
                ELSE
                    merged[indexMerged] ← right[indexRight]
                    indexRight ← indexRight + 1
                END IF
            # Descending order of priority
            ELSE
                # Find the notice with the highest priority out of the
                two notices being compared and add it to the merged array
                IF left[indexLeft].priority > right[indexRight].priority
THEN
                    merged[indexMerged] ← left[indexLeft]
                    indexLeft ← indexLeft + 1

```

```

        indexMerged ← indexMerged + 1
    ELSE
        merged[indexMerged] ← right[indexRight]
        indexRight ← indexRight + 1
        indexMerged ← indexMerged + 1
    END IF
END IF
END SWITCH
END WHILE

# Add any remaining data from the left array to the merged data
WHILE indexLeft < LEN (left)
    merged[indexMerged] ← left[indexLeft]
    indexLeft ← indexLeft + 1
    indexMerged ← indexMerged + 1
END WHILE

# Add any remaining data from the right array to the merged array
WHILE indexRight < LEN (right)
    merged[indexMerged] ← right[indexRight]
    indexRight ← indexRight + 1
    indexMerged ← indexMerged + 1
END WHILE

RETURN merged
END FUNCTION

FUNCTION mergeSort( ARRAY notices, orderBy, asc)
    ARRAY mergedItems [ LEN (notices)]

    # Recursion stops when each array has a length of one notice
    # BASE CASE
    IF LEN (notices) <= 1 THEN
        RETURN notices
    ELSE
        # Midpoint index
        midpoint ← ( LEN (notices) - 1 ) / 2
        # Size of left half of array
        leftSize ← midpoint + 1
        # Size of right half of array
        DEFINE rightSize

        IF LEN (notices) % 2 == 0 THEN
            # If the number of notices is even, the left and right arrays
            are equal in size
            rightSize ← midpoint + 1
        ELSE
            # If the number of notices is odd, the right array will have one
            less item than the left array
            rightSize ← midpoint
        END IF

        # Left and right half
        ARRAY leftHalf[leftSize]
        ARRAY rightHalf[rightSize]
    
```

```

    # Populate left half array with the items up to and including the
    midpoint of the notices array
    FOR i = 0 TO leftSize - 1
        leftHalf[i] ← notices[i]
    NEXT i

    # Populate right half array with the items after the midpoint of the
    notices array
    indexItems ← midpoint + 1
    FOR i = 0 TO rightSize - 1
        rightHalf[i] ← notices[indexItems]
        indexItems ← indexItems + 1
    NEXT i

    # Using recursion to sort both halves
    leftHalf ← mergeSort(leftHalf, orderBy, asc)
    rightHalf ← mergeSort(rightHalf, orderBy, asc)

    # Merge both halves
    mergedItems ← merge(leftHalf, rightHalf, orderBy, asc)
END IF

RETURN mergedItems

END FUNCTION

SUBROUTINE refreshPanel(def)

    # If notices are to be sorted by default: descending review date,
    descending priority
    IF def == TRUE THEN
        ascReview ← FALSE
        ascPriority ← FALSE
    END IF

    # Sorts notices by date to review ascending or descending depending on
    the current settings
    displayedNotices ← mergeSort(displayedNotices, "Date To Review",
    ascReview)

    # Create lists to group notices by review date
    # Stores lists of notices for each date
    LIST noticesLists ← []
    # Stores notices for one specific date
    LIST noticesDay ← []

    prevNotice ← NULL

    # If there are notices to sort
    IF LEN (displayedNotices) != 0 THEN

        # Separate all notices into lists of notices
        # Group notices by review date
        FOREACH notice IN displayedNotices

            # If the current notice has the same review date as the previous
            notice in the list

```

```

        # Or if the current notice is the first notice in the list
        IF prevNotice == NULL OR prevNotice.dateToReview ==
notice.dateToReview THEN
            # Add the notice to the list of notices if they belong to
the same date group
            ADD notice TO noticesDay
            # If the current notice does not have the same review date as
the previous notice
            ELSE
                # Add the current list of notices in the same date group to
the list of lists of all notices
                ADD noticesDay TO noticesLists
                # Creates a new list
                noticesDay ← []
                # Add the current notice to the new list
                ADD notice TO noticesDay
            END IF
            # Set the previous notice to the current notice so the next
notice can be compared to it
            prevNotice ← notice
        END FOREACH

        # Ensures the last list of notices is added
        IF LEN (noticesDay) > 0 THEN
            ADD noticesDay TO noticesLists
        END IF

        # Sorts each list of notices for each date by ascending or
descending priority depending on the current settings
        FOR i = 0 TO LEN (noticesLists) - 1
            noticesLists[i] ← mergeSort(noticesLists[i], "Priority",
ascPriority)
        NEXT i

        # Populate all groups into a single list
        displayedNotices ← []
        FOREACH list IN noticesLists
            FOREACH notice IN list
                ADD notice TO displayedNotices
            END FOREACH
        END FOREACH

    END IF

    # Clear the display and update with sorted notices
    CLEAR NoticesPanel
    DEFINE NoticeBox

    # If there are notices to display
    IF LEN (displayedNotices) > 0 THEN
        HIDE noNoticesIcon
        height ← 40
        FOR i = 0 TO LEN (displayedNotices) - 1
            NoticeBox ← new NoticeBox(displayedNotices[i], height);
            NoticeBox.Location ← (20, height)
            ADD NoticeBox TO NoticesPanel
            SUBSCRIBE TO NoticeBox NoticeUpdated event
        NEXT i
    END IF

```



```

        height ← height + 90
    NEXT i
    # If there are no notices
    ELSE
        SHOW noNoticesIcon
    END IF

    DISABLE NoticesPanel.HorizontalScroll

    END IF
END FUNCTION

```

Data Structures

Waiting List Queue

Use

I have implemented a priority queue for use in my Event Waiting List allocation algorithm, to determine which customers to allocate spaces to first while editing the attendees of an event.

Code

Here is the pseudocode explaining the enqueue, dequeue and removal processes of this data structure.

```

SUBROUTINE Enqueue(attendee, position):
    newNode = Node(attendee, position)

    IF frontPointer IS NULL THEN
        # If queue is empty
        frontPointer ← newNode
        rearPointer ← newNode
    ELSE
        IF newNode.WaitingListPosition < rearPointer.WaitingListPosition
THEN
            # New node becomes the front of the queue
            newNode.Next ← frontPointer
            frontPointer ← newNode
        ELSE IF newNode.WaitingListPosition >=
rearPointer.WaitingListPosition THEN
            # New node becomes the rear of the queue
            rearPointer.Next ← newNode
            rearPointer ← newNode
        ELSE
            # Find where to insert the new node
            current ← frontPointer
            prev ← NULL
            WHILE current.WaitingListPosition <= newNode.WaitingListPosition
                prev ← current
                current ← current.Next
            END WHILE
            newNode.Next ← current
            prev.Next ← newNode
        END IF
    END IF
END IF

```

```

        ADD attendee TO list
    END SUBROUTINE

SUBROUTINE Dequeue()
    IF frontPointer IS NULL THEN
        OUTPUT ("Waiting list is empty.")
        RETURN NULL
    ELSE
        dequeuedAttendee ← frontPointer.Attendee
        frontPointer ← frontPointer.Next
        REMOVE dequeuedAttendee FROM list
        RETURN dequeuedAttendee
    END IF
END SUBROUTINE

SUBROUTINE Remove(attendee, attendeeList):
    # Full list of attendees must be passed in as the indexes of each
    # customer in the list may have changed.
    tempList ← []

    IF frontPointer.Attendee == attendee THEN
        # Remove the front node
        DequeueNodes()
    ELSE
        WHILE attendee IS NOT IN tempList DO
            IF frontPointer IS NOT NULL THEN
                # Dequeue from the queue and add to tempList
                ADD Dequeue() TO tempList
            ELSE
                # If the queue is empty and attendee is not found
                BREAK
            END IF
        END WHILE

        # Remove the attendee from tempList
        REMOVE attendee FROM tempList

        # Re-enqueue the remaining attendees
        FOREACH attendee IN tempList
            Enqueue(attendee, attendeeList.IndexOf(attendee))
        END FOREACH
    END IF
END Remove

```

User Interface Design

Design Notes

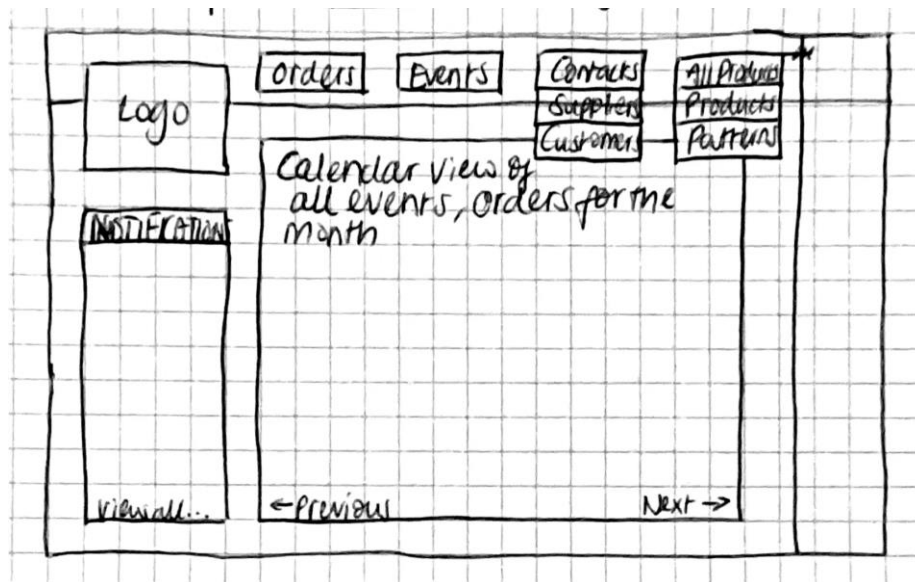
For most of my forms, I started by designing and prototyping them directly in Visual Studio. This approach allowed me to become familiar with the built-in controls provided by Visual Studio's designer and to consider how best to lay out the data on each form in a way that is both efficient and user-friendly.

As development progressed and my understanding of the built-in controls improved, I revisited and redesigned each form. These redesigns focused on presenting all the required information in a more organized and logical format. Additionally, based on feedback from my client after reviewing the initial designs, I implemented a new, consistent colour scheme across the entire application to improve the visual appeal and user experience.

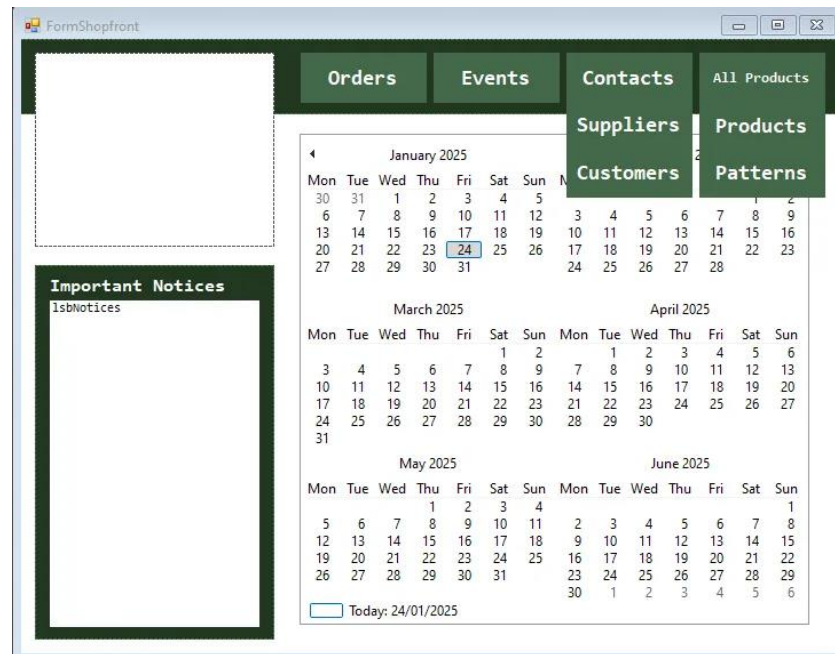
For many of the forms, a filtering system was included to allow users to narrow down data views. Initially, the filters were integrated into the main interface. However, my client suggested moving the filters to a separate sidebar. This adjustment made the filters more compact and allowed them to be easily hidden when not in use, resulting in a cleaner and more streamlined interface. This suggestion was incorporated into the redesigned forms.

Homepage

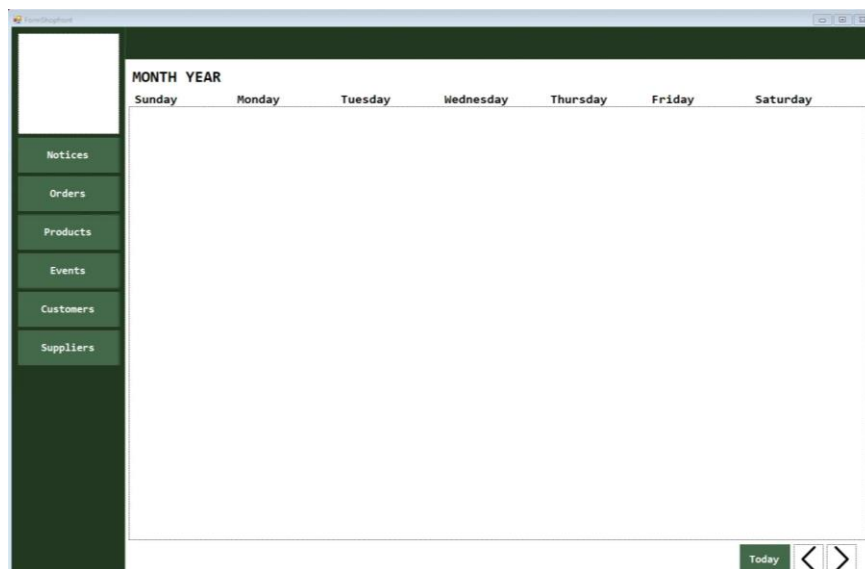
Initial Design



First Iteration



Second Iteration



Redesign

PURLS OF WISDOM

NOTICES

EVENTS

ORDERS

PRODUCTS

CUSTOMERS

SUPPLIERS

January 2025

Today

MON	TUE	WED	THU	FRI	SAT	SUN
		01 2 deliveries due 2 collections due	02 11:00 Sewing class Butterflies	03 17:30 Quilting Square Tutorials	04	05 5 deliveries due
06 18:00 Knitting for Beginners	07 3 deliveries due 1 collection due	08	09 18:00 Pattern Reading	10 18:00 Crochet for Bain Baby clothes	11 2 collections due	12
13	14 18:00 Knitting for Beginners	15 1 delivery due	16 1 collection due	17 18:00 Crochet for Bain Baby clothes	18	19
20 18:00 Sewing a city sweater	21 2 deliveries due	22 18:00 Advanced Crocheting Techniques	23	24	25 18:00 Sewing 101	26 18:00 Pattern Reading
27	28 18:00 Sewing class Insects	29	30 1 delivery due 2 collections due	31 18:00 Crocheting a blanket		

< >

Customers

Initial Design

CUSTOMERS

Search by customer

Filter by:

Customer Info

Customer ID

Forename

Surname

Email Address

Phone Number

membership Type

Total Stars

Vouchers Available

First Iteration

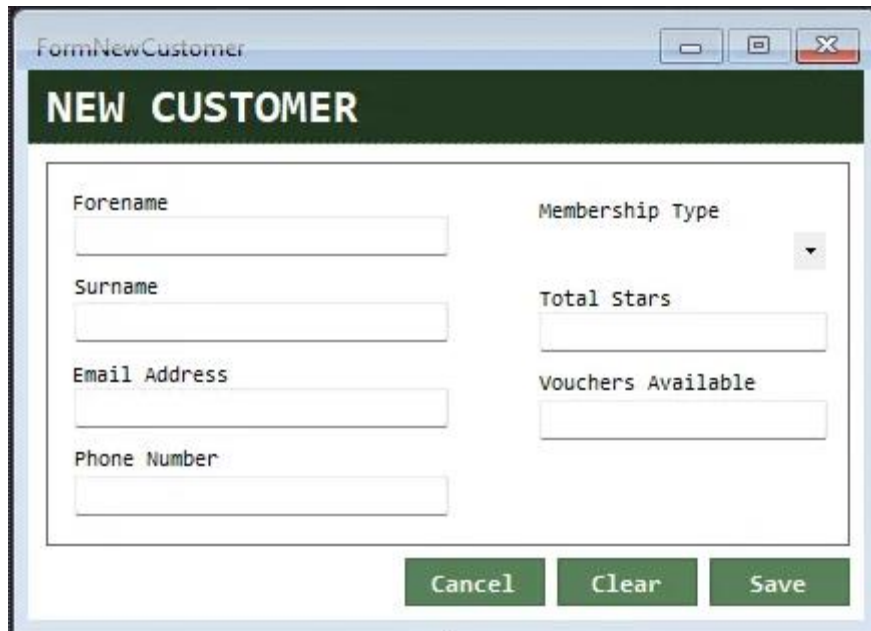
The screenshot shows a web application window titled 'FormCustomers'. The main header is 'CUSTOMERS'. Below the header, there is a search bar labeled 'Search for Customer' and a 'Filter by:' dropdown menu. A large green rectangular area represents the customer list. To the right, the 'Customer Info:' section contains several input fields: 'Customer ID', 'Forename', 'Surname', 'Email Address', 'Phone Number', 'Membership Type' (a dropdown menu), 'Total Stars', and 'Vouchers Available'. At the bottom right, there are two buttons: 'Cancel' and 'Add New Customer'.

Redesign

The redesign is a hand-drawn sketch of the 'CUSTOMERS' interface. It features a header bar with the title 'CUSTOMERS' and a close button (X). Below the header, there is a search bar with a magnifying glass icon. To the left of the search bar is a box labeled 'Customers View'. To the right, the 'Customer Info:' section is a form with the following fields and values: 'Customer ID' (1), 'Forename' (Annika), 'Surname' (Arumugam), 'Email Address' (a.aru@email.com), 'Phone Number' (07123 456 789), 'Membership Type' (Gold), 'Total stars' (363), and 'Vouchers Available' (3). At the bottom right, there is an 'Add' button.

NewCustomer

Initial Design



The initial design of the 'FormNewCustomer' window features a dark green header with the title 'NEW CUSTOMER'. The form is organized into two columns. The left column contains four text input fields labeled 'Forename', 'Surname', 'Email Address', and 'Phone Number'. The right column contains a 'Membership Type' dropdown menu, a 'Total Stars' text input field, and a 'Vouchers Available' text input field. At the bottom right, there are three green buttons labeled 'Cancel', 'Clear', and 'Save'.

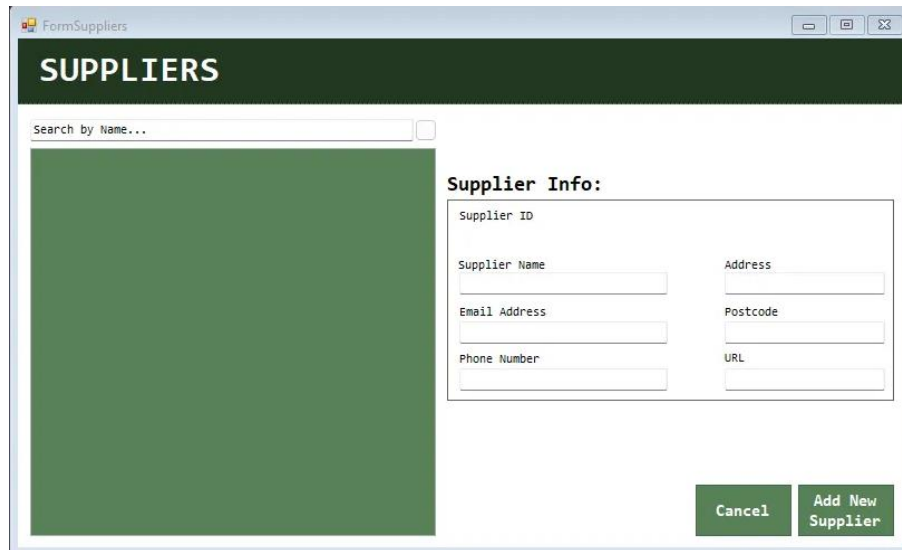
Redesign



The redesign of the 'NEW CUSTOMER' form features a pink header with the title 'NEW CUSTOMER' and a close button 'X'. The form is organized into two columns. The left column contains four text input fields labeled 'Forename', 'Surname', 'Email Address', and 'Phone Number'. The right column contains a 'membership type' dropdown menu with a 'V' icon, a 'Total stars' text input field, and a 'Vouchers Available' text input field. At the bottom right, there are two pink buttons labeled 'clear' and 'Add'.

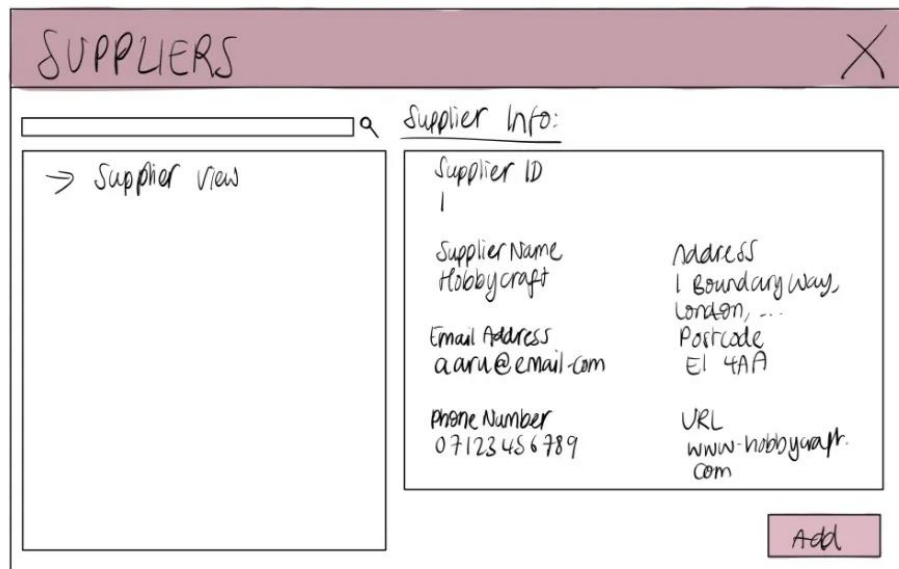
Suppliers

Initial Design



The initial design of the Suppliers form is shown in a window titled "FormSuppliers". The form has a dark green header with the title "SUPPLIERS". Below the header is a search bar labeled "Search by Name...". To the left of the search bar is a large green rectangular area. To the right of the search bar is a "Supplier Info:" section with a form containing the following fields: Supplier ID, Supplier Name, Address, Email Address, Postcode, Phone Number, and URL. At the bottom right of the form are two buttons: "Cancel" and "Add New Supplier".

Redesign

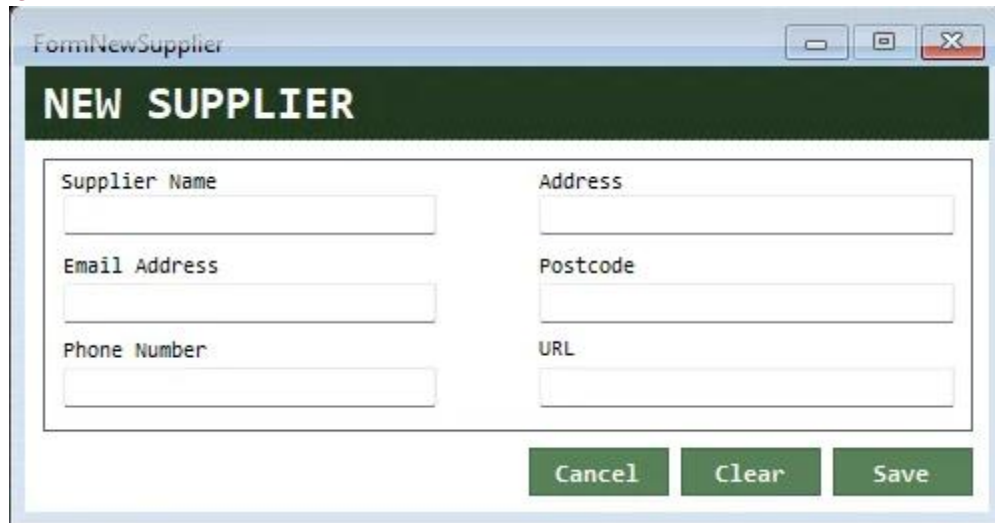


The redesign of the Suppliers form is shown in a window titled "SUPPLIERS" with a close button (X) in the top right corner. The form has a search bar with a magnifying glass icon. Below the search bar is a box labeled "→ Supplier view". To the right of the search bar is a "Supplier Info:" section with a form containing the following fields: Supplier ID, Supplier Name, Address, Email Address, Postcode, Phone Number, and URL. At the bottom right of the form is a button labeled "Add".

Supplier Info:	
Supplier ID	1
Supplier Name	Hobbycraft
Address	1 Boundary Way, London, ...
Email Address	aara@email.com
Postcode	E1 4AA
Phone Number	07123 456 789
URL	www-hobbycraft.com

NewSupplier

Initial Design



The initial design of the NewSupplier form is a standard web form with a title bar, a header, and a main content area. The title bar is light blue and contains the text 'FormNewSupplier' and three window control buttons (minimize, maximize, close). The header is a dark green bar with the text 'NEW SUPPLIER' in white. The main content area is white and contains six text input fields arranged in two columns. The left column contains 'Supplier Name', 'Email Address', and 'Phone Number'. The right column contains 'Address', 'Postcode', and 'URL'. At the bottom right of the form are three green buttons: 'Cancel', 'Clear', and 'Save'.

FormNewSupplier

NEW SUPPLIER

Supplier Name

Address

Email Address

Postcode

Phone Number

URL

Cancel Clear Save

Redesign



The redesign of the NewSupplier form is a hand-drawn sketch. It features a title bar with a pink background and the text 'NEW SUPPLIER' in black, followed by a large 'X' icon. The main content area is white and contains six text input fields arranged in two columns. The left column contains 'Supplier Name', 'Email Address', and 'Phone Number'. The right column contains 'Address', 'Postcode', and 'URL'. At the bottom right of the form are two pink buttons: 'Clear' and 'Add'.

NEW SUPPLIER X

Supplier Name

Address

Email Address

Postcode

Phone Number

URL

Clear Add

Products

Initial Design

The initial design of the FormProducts application window features a dark green header with the title "PRODUCTS". Below the header is a search bar. To the right of the search bar is a "Filter:" section with a "Product Type" dropdown menu and three radio buttons: "All Products", "Yarn", and "Accessories". Further right are three dropdown menus for "Category", "Fibre", and "Colour", each with a corresponding checkbox: "clsbCategory", "clsbFibreMaterial", and "clsbColour". Below the search bar is a large green rectangular area representing the product list. To the right of this area is a "Product Image:" section with a dashed border. To the right of the image section is a "Product Info:" section containing fields for "Product ID", "Product Name", "Cost", "Stock", and "Supplier". To the right of these fields are three dropdown menus for "Type", "Category", "Colour", and "Fibre". At the bottom right are two buttons: "Cancel" and "Add New Product".

First Iteration

The first iteration of the FormProducts application window includes the same elements as the initial design but with several enhancements. A "butt on" button is added to the top right corner. The "Filter:" section now includes a "Supplier" dropdown menu with a "clsbSupplier" checkbox. A "In Stock" checkbox is added below the filter section, along with "Clear" and "Apply" buttons. The "Product Info:" section now includes a "Supplier" text input field. The "Add New Product" button is now labeled "Add New" and "Product" on two lines.

Redesign

PRODUCTS

FILTER:

☐ All

☒ Yarn

☐ Accessories

☐ In Stock

YARN WEIGHT

☐ 4 Ply

☒ Aran

☐ Chunky

☐ DK

FIBRE

☐ Acrylic

☐ Cotton

☐ Wool

COLOUR

☐ Red

☒ Orange

☐ Yellow

☐ Green

SUPPLIER

☐ Hobbycraft

☐ Women's Institute

APPLY

CLEAR

Product ID

Product Name

Cost

Stock

Supplier

Ball Weight

Yarn Weight

Colour

Fibre

Product Info:

Product ID: 1

Product Name: DK Mustard Yarn

Cost: £1.75

Stock: 50

Supplier: Hobbycraft

Ball Weight: 100g

Yarn Weight: Aran

Colour: Yellow

Fibre: Acrylic

Product Image:



Add

NewProduct

Initial Design

FormNewProduct

NEW PRODUCT

Product Name

Cost

Stock

Supplier

Image

Save As:

Type

Category

Colour

Fibre/Material

Product Type

☐ Yarn

☐ Accessory

Upload

Cancel **Clear** **Save**

Redesign

NEW PRODUCT

Product Name

Type

Cost

Category

✓

Stock

☐

Low stock

☐

Full stock

☐

Colour

✓

Supplier

✓

Fibre/material

✓

Image

Upload

Remove

Product Type

☐ Yarn

☐ Accessory

Save As:

Clear

Add

Orders

Initial Design

FormOrders

ORDERS

Order Type

☐ Customer Orders

☐ Inbound Orders

Clear

Apply

Status

☐ clsbStatus

☐ Order Notes

Delivery Status

☐ clsbDelStatus

Payment Status

☐ clsbPayStatus

Order Date

Expected Delivery Date

Collection Date

Order Info:

Order ID

Order Date

Expected Delivery Date

Collection Date End

Stars Earned

Status

Delivery Status

Payment Status

Total Cost

Order Notes

Contact Info:

Full Name

Customer ID

Phone Number

Email Address

URL

View Order Items

Apply

Cancel

Add New Order

Redesign

ORDERS

FILTER:

☐ Customer Orders

☐ Supplier orders

☒ Order Notes

STATUS

☐ Archived

☐ Open

DELIVERY STATUS

☐ Delivered

☐ Order mode

☐ Out for Delivery

☐ Returned

SUPPLIER

☐ Hobbycraft

☐ Women's Institute

☐ Puris of Wisdom

ORDER DATE

☐ 1 January 2025

EXP DELIVERY DATE

☐ 1 January 2025

APPLY

CLEAR

Search by name/email address

Q

View Order Items

Order Info:

Order ID 1	Status Archived
Order Date 01/01/2025	Delivery Status Collected
Expected Delivery 02/01/2025	Payment status Paid
Subtotal £15-00	Voucher GOLD
Total £10-00	Order Notes

Contact Info:

Customer ID 1	Phone Number 07-123 456789
Full Name Anniika Arumugam	Email Address anniika@gmail.com
Membership Tier Gold	

New Order

NewCustOrder

Initial Design

FormMakeCustOrder

NEW CUSTOMER ORDER

☐ Filter:

Category
☐ clsbCategory

Fibre
☐ clsbFibreMater

Colour
☐ clsbColour

Supplier
☐ clsbSupplier

Product Type
☐ All ☐ Yarn ☐ Accessories ☒ In Stock

Clear Apply

Order Info: ☐ In Store

Order Date
 24 January 2025

Status
 Status

Order Notes
 Order Notes

Expected Delivery Date
 24 January 2025

Delivery Status
 Delivery Status

Collection Date End
 24 January 2025

Payment Status
 Payment Status

Customer Info:

☐ Existing Customer
☐ New Customer
☐ Guest

Customer ID
 Select Customer

Forename
 Forename

Membership Type
 Membership Type

Surname
 Surname

Total Stars
 Total Stars

Email Address
 Email Address

Vouchers Available
 Vouchers Available

Phone Number
 Phone Number

Order Items:

Product	Cost	Qty
lsbShoppingList	lsbCost	lsb

Product Info:

ID
 ID

Product Name
 Product Name

Quantity
 0

Type
 Type

Cost
 Cost

Category
 Category

Stock
 Stock

Colour
 Colour

Supplier
 Supplier

Fibre
 Fibre

Product Image:

Subtotal
 £0.00

Savings
 Savings

Voucher
 Voucher

Vouchers
 0

Total
 £0.00

Stars Earned
 0

Place Order

Redesign

CUSTOMER ORDER CREATOR

FILTER:

☐ All
☐ Yarn
☐ Accessories
☐ In Stock

YARN WEIGHT
☐ 4 Ply
☐ Aran
☐ Chunky
☐ DK

FIBRE
☐ acrylic
☐ cotton
☐ wool

COLOUR
☐ Red
☐ Orange
☐ Yellow
☐ Green

SUPPLIER
☐ Hobbycraft
☐ Womans Institute

APPLY
 CLEAR

→ Shows all products here

Order Info: ☐ In Store

Order Date
 01 January 2025

Status
 Open

Order Notes
 Order Notes

Expected Delivery Date
 04 January 2025

Delivery Status
 Order made

Collection Date End
 06 January 2025

Payment Status
 Unpaid

Customer Info:

☐ Existing Customer
☒ New Customer

Select Customer
 Anna Maria Aramugam

ID
 1

Name
 Anna Maria Aramugam

Email Address
 anna@email.com

Membership Type
 Gold

Phone Number
 07113 456789

Total Stars
 243

Vouchers Available
 3

Product Info:

ID	Product Name	Quantity	Cost	Stock	Supplier
1	Knitcraft Mustard Cotton Blend DK Yarn	100g	£3.99	50	Hobbycraft

Product Image:

Ball weight
 100g

Yarn weight
 DK

Colour
 Yellow

Fibre
 Acrylic

Order Items:

Product	Cost	Qty
Mustard Yarn	£3.99	2
Mustard Yarn	£2.99	3

Subtotal
 £13.97

Savings
 -£5.00

Total
 £8.97

Place Order

NewSupplierOrder

Initial Design

FormMakeSupplierOrder

NEW SUPPLIER ORDER

Filter:

Category

☐ clsbCategory

Fibre

☐ clsbFibreMaterial

Colour

☐ clsbColour

Product Type

☐ All Products

☐ Yarn

☐ Accessories

Clear

Apply

☒ In Stock

Order Info:

Order Date: 24 January 2025

Status:

Order Notes:

Expected Delivery Date: 24 January 2025

Delivery Status:

Product Info:

Product ID:

Product Name:

Type:

Cost:

Category:

Stock:

Colour:

Supplier:

Fibre:

Add

Product Image:

Supplier Info:

Supplier ID:

Select Supplier:

Supplier Name:

Email Address:

Address:

Phone Number:

URL:

Postcode:

Order Items:

Product	Cost	Qty
lsbShoppingList	lsbCost	lsb

Total:

Place Order

Redesign

SUPPLIER ORDER CREATOR

Filter:

☐ All

☐ Yarn

☐ Accessories

☐ In Stock

YARN WEIGHT

☐ 4 Ply

☐ Aran

☐ Chunky

☐ DK

FIBRE

☐ Acrylic

☐ Cotton

☐ Wool

COLOUR

☐ Red

☐ Orange

☐ Yellow

☐ Green

APPLY

CLEAR

→ Show all products here for the selected supplier

Order Info:

Order Date: 01 January 2025

Status: Open

Order Notes:

Expected Delivery Date: 01 January 2025

Delivery Status: Order made

Product Info:

ID:

Product Name: Knitcraft Mustard Cotton Blend DK Yarn

Quantity:

Ball Weight: 100g

Cost: £3.99

Yarn Weight: DK

Stock: 50

Colour: Yellow

Supplier: Hobbycraft

Fibre: Acrylic

New Product:

Product Image:

Supplier Info:

Select Supplier:

ID: 1

Name: Hobbycraft

Email Address: hobbycraft@co.uk

Address: 1 Widd Lane

Phone Number: 07113 456789

Postcode: B3 4A6

URL: www.hobbycraft.com

Order Items:

Product	Cost	Qty
Mustard Yarn	£3.99	2
Mustard Yarn	£3.99	3

Subtotal: £11.97

Total: £11.97

Place Order

Events

Initial Design

EVENTS

WEEKLY VIEW

ALL EVENTS

30/12/2024 - 05/01/2025

Today < >

MON	TUE	WED	THU	FRI	SAT	SUN

FILTER:

STATUS

☐ Upcoming
 ☐ Past
 ☐ Active
 ☐ Cancelled

SPACES

☐ Spaces Available
 ☐ Attending
 ☐ Waiting List

PAYMENTS

☐ Paid
 ☐ Unpaid

DATE RANGE

From:

01 January 2025

To:

01 January 2025

APPLY

CLEAR

→ show all events here

Event Info:

ID	Name	Date
Host	Price	Notes
Num Attending	No. Paid	
Capacity	Skill Level	
max Guests		

Attendees:

☐ View Waiting List

Attendee Info:

Customer ID
Full Name
Phone Number
Email Address
Paid ☐

Edit

Apply

First Iteration

FormEvents

EVENTS

WEEK

Today < >

MO	TU	WE	TH	FR	SA	SU

Attendees:

Attendee Info:

Event Info:

Customer ID

Full Name

Phone Number

Email Address

Paid ☐

Waiting List ☐

Event ID

Name	Date	Price	No. Paid
Host	Time	Notes	
No. Attending	Max Guests		
Capacity	Skill Level		

Edit Event

New Event

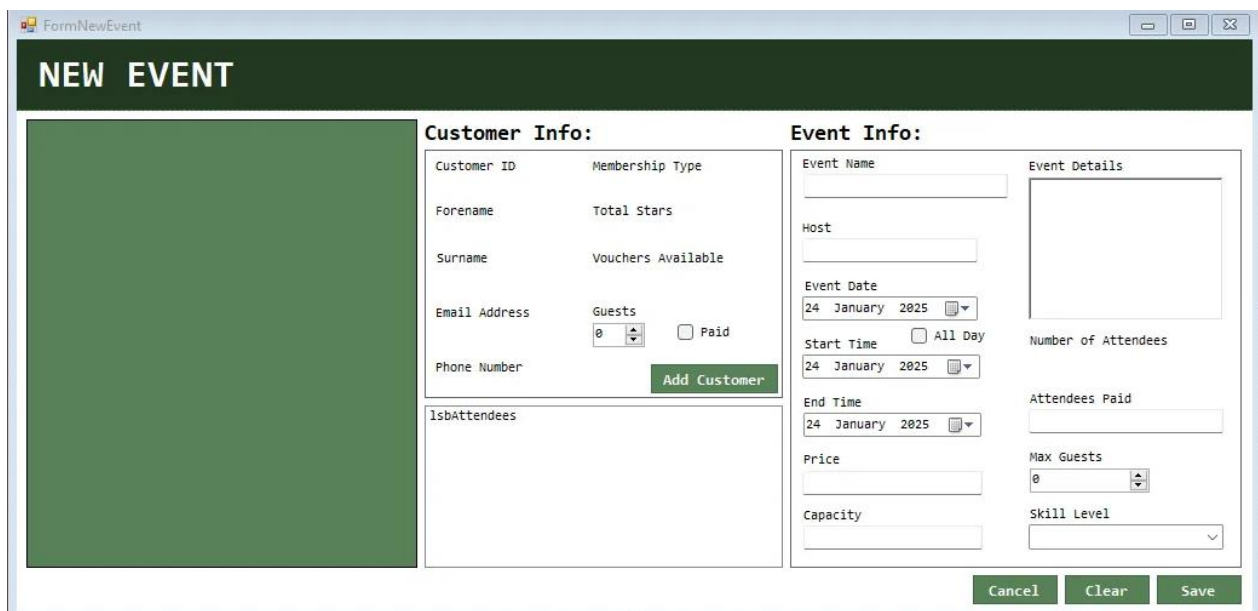
NewEvent

Initial Design



The initial design of the NewEvent form is a simple, single-column layout. It features a title bar with the text "FormNewEvent" and standard window controls. The main title "NEW EVENT" is displayed in a dark green header. Below the header, the form is divided into two columns. The left column contains input fields for "Event Name", "Host", "Event Date" (with a calendar icon), "Start Time" (with a calendar icon), "End Time" (with a calendar icon), and "Price". The right column contains a large "Event Details" text area, followed by input fields for "Number of Attendees", "Attendees Paid", "Max Guests", "Skill Level", and "Capacity". At the bottom of the form are three buttons: "Cancel", "Clear", and "Save".

First Iteration



The first iteration of the NewEvent form is a more complex, multi-column layout. It features a title bar with the text "FormNewEvent" and standard window controls. The main title "NEW EVENT" is displayed in a dark green header. Below the header, the form is divided into three main sections. On the left is a large green rectangular area. In the center is the "Customer Info:" section, which includes input fields for "Customer ID", "Forename", "Surname", "Email Address", and "Phone Number", as well as a "Membership Type" dropdown, "Total Stars" input, "Vouchers Available" input, and a "Guests" spinner. There is also a "Paid" checkbox and an "Add Customer" button. Below this is a "IsbAttendees" input field. On the right is the "Event Info:" section, which includes input fields for "Event Name", "Host", "Event Date" (with a calendar icon), "Start Time" (with a calendar icon), "End Time" (with a calendar icon), "Price", and "Capacity". It also includes a "Number of Attendees" input, "Attendees Paid" input, "Max Guests" input, and a "Skill Level" dropdown. There is also an "All Day" checkbox. At the bottom of the form are three buttons: "Cancel", "Clear", and "Save".

Redesign

EVENT CREATOR

Customer Info:

Customer ID	Name
Email Address	Phone Number
Guests ☐	Paid ☐
Add	

Event Info:

Name	Notes
Host	
Date ☐ All day	
Start Time	No of Attendees
End Time	No Paid
Price	Max Guests
Skill Level	Capacity

Notices

Initial Design

filter system for
Priority, type, date issued, date to be reviewed,
dismissed

ascending/descending
→ order by
priority
→ order by
date

NOTICES

Priority	Type	Date Issued	Date to be reviewed
☐ Pending			

FormNotices

NOTICES

listBox1

listBox2

Redesign

NOTICES

PRIORITY ↓ REVIEW DATE ↓

Active
Dismissed
Archived
Priority
High
Medium
Low
Stock
Low stock
No stock
Selling Fast
Events
Payments
Spaces
Orders
Collection
Processing
Supply
Issue
1 day
2 days
1 week
Older
Review
1 day
2 days
1 week
Older

APPLY
CLEAR

Q >> B

!!!

Q >> B

!!!

Q >> B

!!!

Q >> B

!!!

Hardware Selection

In my client's shop, invoices and receipts are not automatically provided with each order. Instead, if a customer requests a receipt, the order details are manually recorded in an invoice book, and a copy is given to the customer.

My invoice-generating algorithm streamlines this process by allowing the user to print a receipt directly with a single click using the 'Print Invoice' function. While the intended output is for a receipt printer, I did not have access to one during development. Instead, I will test the feature using a label printer, meaning the page width in the algorithm is optimized for the label paper used in my printer. The output works the exact same way it would with a receipt printer, since the printing is continuous.

Testing Plan

To make sure that the program works as intended and all data is created, displayed, updated and deleted correctly, I have created this test plan during development for use once development has been completed.

Test No	Form	Test Description	Test Type	Input	Expected Output	Output
1.	Homepage.cs	When the form loads, all events for the	Typical	Load form	All events in the correct boxes for their date	

		current month are displayed, all deliveries and collections are displayed in the correct dates and the current month is shown			All dates are shown correctly for this month All deliveries and collections due for this month are displayed in the correct boxes for their date	
2.	Homepage.cs	Switch to the next month	Typical	Click forward button	Correct dates for February 2025 should be shown All events, collections and deliveries should be shown for that month	
3.	Homepage.cs	Switch to the current month	Typical	Click 'Today'	Correct dates for the current month should be shown All events, collections and deliveries should be shown for that month	
4.	Homepage.cs	Switch to the previous month	Typical	Click backward button	Correct dates for December 2024 should be shown All events, collections and deliveries should be shown for that month	
5.	Homepage.cs	When right clicking on an event and clicking view event information, redirect the user to the and show all the information of that event	Typical	Right click on an event and click 'View Event Information'	Load Events.cs and display the information of the event clicked on	
6.	Homepage.cs	"View Event Information"	Erroneous	Right click on a delivery notice	No context menu strip should appear	

		should only be displayed when the user clicks on an event		box in the calendar		
CUSTOMERS.CS						
7.	Customers.cs	Load all details of the first customer in the data grid view	Typical	Click on 'Annika Arumugam' in the data grid view and display all details for that customer.	All details of customer 'Annika Arumugam' should be displayed in the correct textboxes.	
8.	Customers.cs	Click on a customer in the data grid view and display all details for that customer.	Typical	Click on a customer in the data grid view	All details of the customer selected should be displayed in the correct textboxes.	
9.	Customers.cs	Click on the header of the data grid view This causes the selected index of the dgv to be -1.	Erroneous	Click on 'CustID' at the top of the data grid view.	All details of the first customer in the list should be displayed.	
10.	Customers.cs	Search for a customer by first name	Typical	Type "Anni" in the search bar	The customer "Annika Arumugam" should be the only result	
11.	Customers.cs	Search for a customer by email address	Typical	Type thecure@gmail.com in the search bar	The customer "Robert Smith" should be the only result	
12.	Customers.cs	Search for a customer with details that do not match any customer's details	Erroneous	Type "abcdefg" in the search bar and click the search button	Error message is displayed: "No customers found"	
13.	Customers.cs	Delete a customer that is in an event	Typical	Delete customer 'Michael Braun'	Data of customer Michael Braun is deleted Customer is removed from all events the	

					customer was in: Event ID 25 and 26	
14.	Customers.cs	Attempt to delete a customer that has placed an order and has not collected it	Erroneous	Attempt to delete customer 'Annika Arumugam'	Error message displayed indicating that all open orders must be closed before deleting customer data	
NEWCUSTOMER.CS						
15.	NewCustomer.cs	Add a new customer	Typical	Enter valid inputs for all fields	Customer successfully added	
16.	NewCustomer.cs	Add a new customer with blank fields	Typical	Enter valid inputs for all fields except 'Phone Number'	Customer successfully added Where fields were left blank, 'N/A' is displayed	
17.	NewCustomer.cs	Attempt to add a new customer with an invalid email address	Erroneous	Add a customer with all valid details entered except for "asdfhjks" entered in 'Email Address' field	Error message is displayed: 'Enter a valid email address.'	
18.	NewCustomer.cs	Attempt to add a new customer with an invalid phone number	Boundary	Add a customer with all valid details entered except for "071234" entered in 'Phone Number' field	Error message is displayed: 'Phone number must be longer than 7 characters.'	
19.	NewCustomer.cs	Attempt to add a new customer with an invalid number of vouchers	Erroneous	Add a customer with all valid details entered except for "-1" entered in 'Vouchers Available' field	Error message is displayed: 'Enter a valid number.'	
20.	NewCustomer.cs	Clear all details inputted	Typical	Enter values in all fields then click 'Clear'	All fields cleared and set to default	
SUPPLIERS.CS						
21.	Suppliers.cs	Delete a supplier	Typical	Right click on a supplier	Supplier deleted:	

				Click 'Delete' Confirm deletion by clicking 'Yes'	Supplier no longer in data grid view In database: Set to a 'deleted state' All products by that supplier are discontinued	
22.	Suppliers.cs	Open supplier creator	Typical	Click 'Add'	NewSupplier form opens with all fields blank/ set to default	
NEWSUPPLIER.CS						
23.	NewSupplier.cs	Add a new supplier	Typical	Enter valid inputs for all fields	Supplier successfully added	
24.	NewSupplier.cs	Add a new supplier with some fields left blank	Typical	Enter valid inputs for all fields Leave 'URL' and 'Phone Number' blank	Supplier successfully added with 'N/A' saved in the fields left blank	
25.	NewSupplier.cs	Attempt to add a supplier with no details entered	Erroneous	Add a supplier with no details entered in any textboxes	Error message: 'Enter inputs in all required fields'	
26.	NewSupplier.cs	Attempt to add a supplier with no contact details	Erroneous	Add a supplier with all valid inputs Leave 'Email Address' and 'Phone Number' blank	Error message: 'Enter values in all required fields'	
PRODUCTS.CS						
27.	Products.cs	Viewing a 'yarn' product	Typical	Click on a 'yarn' product in the data grid view	All product details for the selected yarn product should be displayed in the correct fields with the correct field names	
28.	Products.cs	Viewing all yarn products	Typical	Click 'Yarn'	All products in the data grid view are yarn products All filters are changed to match yarn product attributes	

					All field names are changed to match yarn product attributes	
29.	Products.cs	Product stock levels indicated by colour in dgv	Typical	Click on a product highlighted in red or orange	Products in red are out of stock, products in orange are low on stock	
30.	Products.cs	Filter products by one supplier which has no products	Typical	Select 'Cygnet Ltd' only	Error message: 'No Products Found'	
31.	Products.cs	Filter by multiple suppliers that are in stock but discontinued	Typical	Select 'Discontinued' check box, 'In Stock' check box under 'Stock' and 'Hobbycraft' and 'Women's Institute' under 'Supplier'	All products shown are discontinued are are supplied by either Hobbycraft or Women's Institute All products shown are in stock (none are highlighted in the data grid view)	
32.	Products.cs	Filter yarn products by one of each filter	Typical	Select 'In Stock','Hobbycraft','4 Ply', 'ACRYLIC' and 'Red' checkboxes	One product found that fulfils all filters selected	
33.	Products.cs	Filter yarn products by two filters and one stock filter	Typical	Select 'Aran', 'Chunky' and 'ACRYLIC' checkboxes	All products shown have the yarn weight 'Aran' or 'Chunky' and are all of acrylic fibre All products shown are in stock, indicated by there being no products highlighted in red	
34.	Products.cs	Viewing all accessory products	Typical	Click 'Accessories'	All products in the data grid view are accessory products All filters are changed to match accessory product attributes	

					All field names are changed to match accessory product attributes	
35.	Products.cs	Viewing all products	Typical	Click 'All'	Clears all filters and shows all products regardless of whether they are a yarn or accessory product in the data grid view	
36.	Products.cs	Search for a product that exists in the database with the filters applied	Typical	Enter "knitcraft" in the search bar and click the search button	Only products that match the input out of the filtered products are shown	
37.	Products.cs	Search for a product that does not exist with the filters applied	Typical	Enter "mustard" in the search bar and click the search button	Error message: no products found	
38.	Products.cs	Search for a product with no input in search bar	Erroneous	Click search button with nothing entered in the search bar	No changes are made to the products shown	
39.	Products.cs	Clear all filters for accessory products	Typical	Click 'Clear'	All checked list boxes cleared All accessory products are shown	
40.	Products.cs	Search all products for products with similar names	Typical	Enter "Knitcraft" in the search bar	All products matching the input are shown	
41.	Products.cs	Search for a product that does not exist	Typical	Enter "abcdefg" in the search bar and click the search button	Error message: no products found	
42.	Products.cs	Editing a product: change the supplier	Typical	Edit Product 38 Change combo box item to 'Women's Institute' and save	Product edited successfully	

43.	Products.cs	Editing a product: change the image	Typical	Edit Product ID 15 Click 'Upload' Upload bow.jpg Save the product	Product image saved to bow.jpg	
44.	Products.cs	Editing a product: remove the image	Typical	While editing Product ID 15, click 'Remove'	Default image displayed	
45.	Products.cs	Editing a product: reset the image	Typical	While editing Product ID 15, click 'Reset'	Original image bow.jpg displayed	
46.	Products.cs	Editing a product: remove the image and save	Typical	While editing Product ID 15, click 'Remove' and save the product	Product image saved to default image	
47.	Products.cs	Editing a product: enter values and cancel editing	Typical	Change product ID 1: Product name to "abcdef" and the supplier to 'Women's Institute' Click 'Cancel'	No edits are saved	
48.	Products.cs	Discontinue a product	Typical	Right click product ID 34 Click 'Discontinue' Confirm discontinuation by clicking 'Yes'	Product ID 34 is discontinued Edits cannot be made	
NEWPRODUCT.CS						
49.	NewProduct.cs	Add a new product	Typical	Enter valid inputs for all fields	Product successfully added	
50.	NewProduct.cs	Attempt to add a new product with an invalid name	Erroneous	Enter valid inputs for all fields except 'Product Name' Enter "abcd\$£" in 'Product Name'	Error message	
51.	NewProduct.cs	Attempt to add a new product	Erroneous	Enter valid inputs in all	Error message	

		with an invalid file name		fields except 'Save As:' Upload image as usual Enter "abc>@" in 'Save As' field		
52.	NewProduct.cs	Clear all fields	Typical	Enter values in all fields Click 'Clear'	All fields cleared and set to default values	
ORDERS.CS						
53.	Orders.cs	Open Orders and view all customer orders Details of the first customer order in the data grid view should be displayed	Typical	Load Orders.cs	All order details of the first order in the data grid view should be displayed in the correct fields with the correct field names	
54.	Orders.cs	View all supplier orders	Typical	Click 'Supplier Orders'	All supplier orders should be displayed Details of the first order in the data grid view should be displayed All filters changed to supplier order specific filters i.e. collection date filter is hidden, payment status filter is hidden and supplier filter is shown	
55.	Orders.cs	All suppliers with orders are loaded into filters	Typical	When viewing all supplier orders	All suppliers with orders recorded displayed in the filter list	
56.	Orders.cs	Filter by suppliers	Typical	Select 'Hobbycraft' checkbox	All supplier orders for Hobbycraft shown	
57.	Orders.cs	Filter supplier orders by order date	Typical	Select 15/1/2025 in	All supplier orders for 15/1/2025 shown	

				order date filter		
58.	Orders.cs	Filter supplier orders by order date and supplier	Typical	Select 15/9/2024 in order date filter Select 'Women's Institute' checkbox	All orders with the order date 15/9/2024 and supplier Women's Institute shown	
59.	Orders.cs	Search for a supplier order by supplier name	Typical	Enter 'WYS' in search bar	All orders for the supplier 'WYS Woodland' shown	
60.	Orders.cs	Search for a supplier order by name of a supplier that does not exist	Erroneous	Enter 'Annika' in search bar and click search button	Error message	
61.	Orders.cs	Switch to viewing all customer orders	Typical	Click 'Customer Orders'	All customer orders should be displayed Details of the first order in the data grid view should be displayed All filters changed to customer order specific filters i.e. collection date filter is shown, payment status filter is shown and supplier filter is hidden	
62.	Orders.cs	Click on a customer order and view all details of the order	Typical	Click on any row in the data grid view	Shows all details of the customer order Shows all details of the customer related to the order	
63.	Orders.cs	View all products in the order	Typical	Click 'View Order Items'	Shows all products in the order and shows details of the first product in the order Hides all filters	

64.	Orders.cs	Click on a product and view all details of the product	Typical	Click on a product in the order	All details of that product should be displayed in the correct fields with the correct field names	
65.	Orders.cs	Search for a product in the order	Typical	Enter input in search bar that matches one of the products in the order	Only products that match input are displayed	
66.	Orders.cs	Search for a product that does not exist in the order	Erroneous	Enter "abcdefghi" in the search bar and click search	Error message displayed	
67.	Orders.cs	View order details	Typical	Click 'View Order Info'	Shows all order details of the order selected Shows all customer orders Shows all filters	
68.	Orders.cs	Search for a customer order by customer name	Typical	Input "Annika" in the search bar	All orders shown are linked to customer "Annika Arumugam"	
69.	Orders.cs	Search for a customer order by customer email address	Typical	Input "snaph" in the search bar	All orders shown are linked to customer "Sophie Naphray" whose email address matches the input	
70.	Orders.cs	Search for a customer order by customer name of a customer that does not exist	Erroneous	Input "abcdefg" in the search bar and click the search button	Error message: No orders found	
71.	Orders.cs	Filter all customer orders: archived orders	Typical	Select 'Archived'	All archived customer orders are shown	
72.	Orders.cs	Filter all customer	Typical	Select 'Open'	All open customer orders are shown	

		orders: open orders			i.e. all orders that have not been collected, voided or returned	
73.	Orders.cs	Filter all customer orders: paid and collected orders	Typical	Select 'Paid' and 'Collected'	All customer orders that have been paid for and collected are shown	
74.	Orders.cs	Clear all filters	Typical	Clears all filters and shows all customer orders	All customer orders are shown	
75.	Orders.cs	Apply filters with no filters selected	Typical	Click 'Apply' when no filters are selected	All customer orders are shown	
NEWCUSTORDER.CS						
76.	NewCustOrder.cs	Attempt to add a product that is out of stock to the order	Erroneous	Click 'Add' when a product out of stock is selected	Error message	
77.	NewCustOrder.cs	Add a product to the shopping list	Typical	Select Product ID 1 and click 'Add'	Product is added to the shopping list and totals are updated Subtotal and total equal £1.99 Stars earned equals 6	
78.	NewCustOrder.cs	Add multiple of one product to the shopping list	Typical	Select Product ID 6 and click 'Add' twice	Product is added to the shopping list with quantity 2 Subtotal and total equal £5.97 Stars earned equals 18	
79.	NewCustOrder.cs	Create new product and add to the shopping list	Typical	Click 'New Product', create new product and add the new product to the shopping list	Product successfully created and displayed in the shopping list when added	
80.	NewCustOrder.cs	View details of a product which is not	Typical	Click on 'Knitcraft Mustard	All product details of that product are displayed	

		currently selected in the shopping list		Cotton Blend Plain DK Yarn..' in the shopping list	Product is selected in the data grid view	
81.	NewCustOrder.cs	Increase quantity of existing product in shopping list	Typical	Click 'Add' to increase quantity of 'Knitcraft Mustard Cotton...' product	Quantity of that product is increased by 1 in the shopping list Totals updated accordingly	
82.	NewCustOrder.cs	Remove a product from the shopping list	Typical	Remove 'Knitcraft Pale Peach...' using context menu strip	Product removed and totals updated accordingly	
83.	NewCustOrder.cs	Remove all products from the shopping list	Typical	Remove all products from the shopping list using context menu strip	Totals updated to 0 Stars earned equals 0 No products in order	
84.	NewCustOrder.cs	Attempt to place order	Erroneous	Click place order	Error message	
85.	NewCustOrder.cs	Apply a voucher to the order, if the customer has vouchers available	Typical	Add products to the order Click 'up' on the numeric up down control for 'Vouchers' once	Vouchers used increases by 1 Vouchers available in customer info panel decreases by 1 Value in 'Savings' reflects the discount applied according to the customer's membership status Totals and stars earned are updated accordingly	
86.	NewCustOrder.cs	Add products to the order while a voucher is applied	Typical	Add any product to the order	Totals and savings updated accordingly while regarding the voucher	
87.	NewCustOrder.cs	Attempt to add another voucher	Erroneous	Click 'up' on the numeric up down control	Number of vouchers does not	

				for 'Vouchers' once	change, totals do not change	
88.	NewCustOrder.cs	Remove voucher from order	Typical	Click 'down' on the numeric up down control for 'Vouchers' once	Voucher is removed and all totals and savings updated accordingly Subtotal = total	
89.	NewCustOrder.cs	Attempt to add multiple of one product where there will be insufficient stock	Erroneous	Add Product ID 31 to the shopping list Click 'Add' 4 times	Error message: insufficient stock Only 3 lots of that product are in the shopping list	
90.	NewCustOrder.cs	Decrease the quantity of a product in the shopping list	Typical	Click on any product with multiple lots added to the order Use numeric up down control for 'Quantity' and decrease the number Click 'Apply'	Quantity of selected product is decreased Totals updated accordingly	
91.	NewCustOrder.cs	Input order date	Typical	Select 'Order Date' to be 26/01/2025	Order date is changed Expected delivery date is changed to date 3 days following order date Collection date end is changed to date 7 days following expected delivery date	
92.	NewCustOrder.cs	Set the order to be in store	Typical	Select 'In Store' check box	Delivery status is set to 'Delivered' Status is set to 'Archived' Payment status is set to 'Paid'	

					All dates are set to the current date and are not editable	
93.	NewCustOrder.cs	Select another customer for the order	Typical	Select a different customer in the combo box	All details of that customer are displayed and the order will be placed for that customer now	
94.	NewCustOrder.cs	Place customer order	Typical	Click 'Place Order'	Order placed successfully Inventory and membership details of related customer are updated accordingly	
ORDERS.CS (Returning customer orders and printing an invoice)						
95.	Orders.cs	Return product	Typical	Return product in customer order created in previous tests	Inventory and membership details of related customer are updated accordingly Order payment status is changed to 'Partially Refunded'	
96.	Orders.cs	Print invoice as a pdf	Typical	Print invoice for order made during testing Right click on order and click 'Print Invoice' Choose 'Microsoft Print to PDF' Click 'Print'	Print to PDF successful: all order information shown as required, including returns	
97.	Orders.cs	Print invoice to printer	Typical	Right click on order and click 'Print Invoice' Choose correct printer for printing invoice	Print successful: see appendix	
98.	Orders.cs	Return full order	Typical	Right click on order and click 'Return Order'	Return successful Delivery status set to 'Returned' Payment status set to 'Refunded'	

					Inventory updated accordingly Membership details of customer are updated accordingly	
99.	Orders.cs	Print invoice of returned order as a pdf	Typical	Print the invoice just returned as a pdf	Returns shown on invoice as required	
100.	Orders.cs	Print invoice for customer order for a guest	Typical	Print invoice for a customer order where there is no related customer i.e. order placed under 'Guest'	Invoice should show no membership details	
101.	Orders.cs	Print invoice with multiple pages	Typical	Print invoice for order ID 57 to pdf	PDF of invoice produced with multiple pages	
NEWSUPPLIERORDER.CS						
102.	NewSupplier Order.cs	Select a product and show all product details	Typical	Click on a product in the data grid view	All details of that products are shown in the correct fields	
103.	NewSupplier Order.cs	Switch supplier to a supplier with no products	Typical	Switch to supplier 'Sirdar Holdings' using the combo box	Error message 'No results found' Supplier is not changed	
104.	NewSupplier Order.cs	Change supplier to another supplier while creating the order	Typical	Add products to the shopping list Change supplier using the combo box	Supplier details of new supplier displayed All items in the shopping list are cleared and all totals are cleared All products for the supplier selected are displayed in the data grid view	
105.	NewSupplier Order.cs	Place supplier order	Typical	Click 'Place Order'	Supplier order placed successfully Inventory updated accordingly as order is delivered	

106.	NewSupplier Order.cs	Place a new supplier order where the order has not been delivered	Typical	Input all valid details and add products to order Set delivery status to 'Order Made' Place order	Supplier order placed successfully Inventory is not updated as the order has not been delivered	
ORDERS.CS (Voiding and returning supplier orders)						
107.	Orders.cs	Void items in a supplier order	Typical	Click 'View Order Items' and right click on the product to be voided Void items in the order that has been placed while testing	Order items successfully voided No inventory changes	
108.	Orders.cs	Void full supplier order	Typical	Right click on order and click 'Void Order'	Order successfully voided No inventory changes	
109.	Orders.cs	Return items in a supplier order	Typical	Click 'View Order Items' on the delivered supplier order created while testing Right click on the product to be returned Click 'Return'	Order items successfully returned Inventory updated accordingly	
110.	Orders.cs	Return full supplier order	Typical	Right click on order and click 'Return Order'	Order successfully returned Inventory updated accordingly	
EVENTS.CS						
111.	Events.cs	Open Events and view all active events for the current week	Typical	Load Events.cs	All active events for the current week displayed in the weekly calendar and in the data grid view	

112.	Events.cs	View all events for the next week	Typical	Click forward button	All active events for the next week should be displayed in the weekly calendar and in the data grid view If there are no events for the next week, display that	
113.	Events.cs	Switch to all events for the current week	Typical	Click 'Today'	All active events for the current week displayed in the weekly calendar and in the data grid view	
114.	Events.cs	Switch to the previous week	Typical	Click previous button	All active events for the previous week displayed in the weekly calendar and in the data grid view	
115.	Events.cs	View event details of each event	Typical	Click on an event in the calendar	All details of that event displayed in the correct textboxes All attendees of that event displayed in the attendees list If the event has a waiting list, display the 'View Waiting List' button Event is selected in the data grid view	
116.	Events.cs	View event details of each event	Typical	Click on an event in the data grid view	All details of that event displayed in the correct textboxes All attendees of that event displayed in the attendees list If the event has a waiting list, display the 'View Waiting List' button	

117.	Events.cs	View attendee details	Typical	Click on an attendee in the list of attendees for the event	All details of that attendee displayed, including paid status	
118.	Events.cs	View waiting list	Typical	Click 'View waiting list'	Displays attendees on the waiting list	
119.	Events.cs	Switch to viewing all events	Typical	Click 'All Events'	Weekly view is hidden All active and upcoming events are displayed in the data grid view All filters for events are shown	
120.	Events.cs	Filter cancelled events	Typical	Select 'Cancelled' Click 'Apply'	All events that are cancelled are displayed	
121.	Events.cs	Filter cancelled and upcoming events	Typical	Select 'Cancelled' and 'Upcoming' Click 'Apply'	All cancelled events in the future are displayed	
122.	Events.cs	Filter upcoming events with spaces available	Typical	Select 'Spaces Available' and 'Upcoming' Click 'Apply'	All events in the future that aren't at capacity are displayed	
123.	Events.cs	Filter cancelled events where there are customers that have paid for the event	Typical	Select 'Cancelled' and 'Paid' Click 'Apply'	All cancelled events where all customers have paid are displayed	
124.	Events.cs	Clear all filters	Typical	Click 'Clear'	All filters are cleared and all upcoming and active events are displayed	
125.	Events.cs	View the weekly calendar of events	Typical	Click 'Weekly View'	All active events for the current week displayed in the weekly calendar and in the data grid view	
126.	Events.cs	Edit order notes	Typical	Edit the order notes of an	Order notes successfully changed	

				event and click 'Apply'		
127.	Events.cs	Edit paid status of a customer	Typical	Click on an attendee that has not paid yet (highlighted in red) Check the 'Paid' checkbox and click 'Apply'	Customer should no longer be highlighted in list, indicating that the paid status of the customer has successfully been changed	
128.	Events.cs	Attempt to cancel an event in the past	Erroneous	Right click on an event in the past Click 'Cancel Event'	Error message displayed	
NEWEVENT.CS						
129.	NewEvent.cs	Add customer to attendee list	Typical	Click on a customer and click 'Add'	Customer added to attendee list successfully Customer highlighted in green in data grid view	
130.	NewEvent.cs	Attempt to add a guest to a customer when no guests are allowed		Click up on the numeric up down control for 'Guests'	Error message	
131.	NewEvent.cs	Add a guest to a customer and add the customer to the attendee list	Typical	Increase maximum number of guests Increase number of guests for the customer using the numeric up down control Add the customer	Customer added successfully to attendee list with the guest shown	
132.	NewEvent.cs	Add a customer that has not paid	Typical	Uncheck 'Paid' and add the customer	Customer added to attendee list successfully	

					Customer highlighted in red in attendee list	
133.	NewEvent.cs	Reach capacity of event	Typical	Add enough customers to reach the capacity of 10 Click on another customer	'ON WAITING LIST' displayed to indicate the customer will be added to the waiting list	
134.	NewEvent.cs	Add customers when the event is at capacity	Typical	Add customers when the event is at capacity	Customers are added to the waiting list Customers in the waiting list are highlighted blue in the list	
135.	NewEvent.cs	Increase the capacity of the event and reallocate spaces to customers	Typical	Increase the capacity by 1 Refresh the waiting list	First customer on the waiting list is no longer highlighted in blue, indicating they are no longer on the waiting list	
136.	NewEvent.cs	Attempt to decrease the capacity	Erroneous	Decrease 'Capacity'	Error message	
137.	NewEvent.cs	Remove customer from main list	Typical	Right click on customer and click 'remove'	Customer removed successfully	
138.	NewEvent.cs	Reallocate spaces to customers on the waiting list that have paid before those that have not paid	Typical	Add a customer that has not paid to the waiting list, followed by a customer that has paid to the waiting list Increase the capacity to free up space in the event Click 'Refresh'	The customer that was originally on the waiting list that has paid is removed from the waiting list first	
139.	NewEvent.cs	Remove customer from waiting list	Typical	Right click on customer in the waiting list	Customer removed successfully	

				and click 'remove'		
140.	NewEvent.cs	Attempt to save event with date in the past	Erroneous	Enter valid inputs in all required fields Input 01/01/2025 in 'Date' Click 'Save'	Error message	
141.	NewEvent.cs	Add event	Typical	Enter valid inputs in all required fields Click 'Save'	Event added successfully	
NOTICES.CS (Viewing notices)						
142.	Notices.cs	Notices are all shown in default order	Typical	Load Notices.cs	All notices shown are ordered by review date in descending order For notices with the same review date, they are ordered by priority in descending order Latest notices are shown first	
143.	Notices.cs	Order notices by ascending order of priority and ascending order of review date	Typical	Click 'Priority' and click 'Review Date'	Notices ordered by ascending order of review date and priority Earliest notices are shown first	
144.	Notices.cs	Order notices by ascending order of priority and descending order of review date	Typical	Click 'Priority' so the arrow is pointing up Click 'Review Date' so the arrow is pointing downwards	Notices shown in descending order of review date, and for notices with the same review date, they are shown in ascending priority (lowest first)	
145.	Notices.cs	Filter all notices by high priority	Typical	Select 'High' and click 'Apply'	All notices shown have high priority	
146.	Notices.cs	Filter all notices by	Typical	Select 'Collection' and click 'Apply'	All notices shown related to collections	

		collection related notices				
147.	Notices.cs	Filter all notices by 'Older' review date	Typical	Select 'Older' and click 'Apply'	All notices shown have a review date that is more than a week ago today	
148.	Notices.cs	View all archived notices	Typical	Click the 'archive' icon	All archived notices shown	
149.	Notices.cs	View all dismissed notices	Typical	Click the 'bell' icon	All dismissed but not archived notices shown	
150.	Notices.cs	Filter all notices by low priority and collection related notices	Typical	Select 'Low' and 'Collection' and click 'Apply'	All notices with low priority and are collection related are shown Title should be 'Order Collection Overdue: 1 day'	
151.	Notices.cs	Dismiss a notice for 3 days	Typical	Right click on a notice's header Click 'Dismiss' Click '3 days'	Dismisses notice successfully for 3 days- review date is set to 3 days later than today	
152.	Notices.cs	Clear all filters	Typical	Click 'Clear'	All filters cleared	
153.	Notices.cs	Go to the event for an event notice	Typical	Right click on an event notice	Events form opened All details of event related to notice are shown	
154.	Notices.cs	Go to the customer order for a customer order notice	Typical	Right click on a customer order notice	Orders form opened All details of order related to notice are shown	
155.	Notices.cs	Go to the supplier order for a supplier order notice	Typical	Right click on a supplier order notice	Orders form opened All details of order related to notice are shown	
156.	Notices.cs	Open a new supplier order for a product that is low/out of stock	Typical	Right click on a stock notice	Supplier order creator is opened Related product is added to the shopping list with quantity equal to the low stock	

					threshold for that product	
ISSUING AND UPDATING NOTICES						
157.	Products.cs	Issue a 'No Stock' notice for products out of stock	Typical	Change stock of product to 0	'No Stock' notice issued	
158.	Products.cs	Issue a 'Low Stock' notice for products low on stock	Typical	Change stock of product to below low stock threshold	'Low Stock' notice issued	
159.	Orders.cs	Archive stock notice when a supplier order for that notice has been delivered	Typical	Change delivery status of a supplier order containing a product with a notice to 'Delivered'	Notice related to product is archived	
160.	Products.cs	Archive 'No Stock' notice	Typical	Add stock to a product out of stock	Notice related to product is archived	
161.	NewSupplier Order.cs	Update 'Low Stock' notice	Typical	Place supplier order for a product low on stock	Notice related to product dismissed until delivery date of supplier order	
162.	NewCustOrder.cs	Archive 'Low Sales' notice	Typical	Place customer order for a product with low sales	Notice related to product archived	
163.	NewCustOrder.cs	Issue 'Selling Fast' notice	Typical	Order a large volume of one product in a customer order	Notice issued for the product	
164.	NewCustOrder.cs	Issue collection overdue by 1 day notice	Typical	Create a customer order where the collection date has passed by 2 days and the order is ready to collect	Notice for overdue collection issued with low priority linked to the order ID	
165.	NewCustOrder.cs	Issue collection overdue by more than 5 days notice	Typical	Create customer order where the collection date has passed by 6	Notice for overdue collection issued with high priority linked to the order ID	

				days and the order is ready to collect		
166.	Orders.cs	Archive notices for collected orders	Typical	Set delivery status of an order 'Ready to Collect' to 'Collected'	Any related notices are archived	
167.	NewCustOrder.cs	Issue order processing overdue by 3 days notice	Typical	Create a customer order where the delivery date has passed by 3 days	Notice for overdue processing issued with medium priority linked to the order ID	
168.	Orders.cs	Archive processing overdue notice when the order is processed	Typical	Update customer order with delivery status 'Order Made', change to 'Ready to Collect'	Archives related overdue processing notice	
169.	NewSupplierOrder.cs	Issue overdue supply delivery notice by 3 days notice	Typical	Create supplier order where the delivery date has passed by 3 days	Notice for overdue supply delivery issued with medium priority linked to the order ID	
170.	Orders.cs	Archive supply delivery notices for supplier orders that have been delivered	Typical	Update supplier order out for delivery to 'Delivered'	Archives related supply delivery notice	
171.	Orders.cs	Dismiss supply delivery notices until their delivery date if there was an existing notice for an overdue order	Typical	Update supplier order delivery date	Dismisses related notice until the delivery date	
172.	Orders.cs	Archive all notices for orders that are voided	Typical	Void supplier order with a notice	Archives related notice	

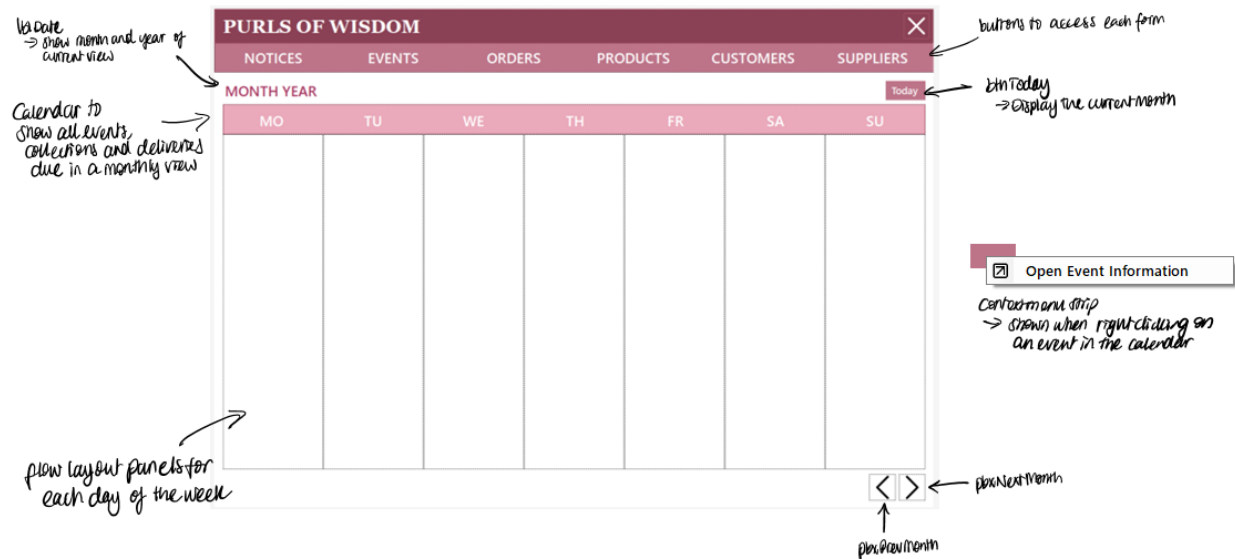
173.	NewSupplier Order.cs	Issue overdue supply delivery by 1 day notice	Typical	Create a supplier order where the order is out for delivery and the delivery date was 1 day ago	Notice issued for overdue supply delivery with low priority	
174.	NewEvent.cs	Issue event payments notice	Typical	Create an event where customers have not paid	Notice related to event ID issued for event payments	
175.	NewEvent.cs	Issue 'event spaces available' notice	Typical	Create an event where customers are on the waiting list but there are spaces available	Notice related to event ID issued for 'event spaces available'	
176.	NewEvent.cs	Allocate spaces to customers on the waiting list in an event with a notice for available spaces	Typical	Resolve an existing 'event spaces available' notice by refreshing the waiting list and saving the event	Notice related to event is archived	
177.	Events.cs	Cancel event with a notice	Typical	Cancel event with event payment notice	Archives related notice	
178.	Events.cs	Update payments for an event with a payment notice	Typical	In an event with an event payments notice, set all customers' paid status to paid	Archives related notice	

Technical Solution

Forms

Homepage.cs

Form Design



Code

```
using System;
using System.Collections;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Drawing.Text;
using System.Globalization;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public partial class Homepage : Form
    {
        static string connectionString = "Provider = 
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        static int month, year;
        public Homepage()
        {
            InitializeComponent();
        }
        // Make form draggable
```

```

    bool drag = false;
    Point dragCursor;
    Point dragForm;
    private void pbxBar_MouseDown(object sender, MouseEventArgs e) //
Detects when the mouse is down and allows the user to drag
    {
        drag = true;
        dragCursor = Cursor.Position;
        dragForm = Location;
    }
    private void pbxBar_MouseMove(object sender, MouseEventArgs e) //
Detects when the mouse is down and moving
    {
        if (drag)
        {
            Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
            Location = Point.Add(dragForm, new Size(difference));
        }
    }
    private void pbxBar_MouseUp(object sender, MouseEventArgs e) //
Detects when the mouse is not dragging the form
    {
        drag = false;
    }
    private void btnClose_Click(object sender, EventArgs e)
    {
        Close();
    }
    private void FormShopfront_Load(object sender, EventArgs e) //
Displays all events and deliveries/collections for the current month and
issues/updates all notices automatically
    {
        checkDismissedNotices();
        refreshCurrentMonth();
        refreshStockNotices();
        refreshOrderNotices();
        refreshEventNotices();
    }

    // Issuing and updating all notices
    public static void refreshOrderNotices() // Issues and updates all
order notices
    {
        generateCustOrderNotices();
        generateSupplyDeliveryNotices();
    }
    public static void refreshEventNotices() // Issues and updates all
event notices
    {
        processEventPayments();
        processEventSpaces();
    }
    public static void refreshStockNotices() // Issues and updates all
stock notices
    {

```

```

List<int> noStockProductIDs = new List<int>();
List<int> lowStockProductIDs = new List<int>();
List<int> lowStockSellingFastProductIDs = new List<int>();
List<int> productsSellingFast = new List<int>();
List<int> productsSalesLow = new List<int>();
bool updateNotice = false;

OleDbConnection connection = new OleDbConnection();
connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
OleDbCommand cmd = connection.CreateCommand();
OleDbDataReader reader;

// Retrieve all products that have low stock but are NOT out of
stock
connection.Open();
cmd.CommandText = "SELECT ProductID FROM TableProducts WHERE
Stock <= LowStock AND Stock <> 0 AND Discontinued = False";
cmd.Parameters.Clear();
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    lowStockProductIDs.Add(Convert.ToInt32(reader[0])); // Add
the product ID to the list for products on low stock
}
connection.Close();

connection.Open();
// Retrieve all products that have no stock.
cmd.CommandText = "SELECT ProductID FROM TableProducts WHERE
Stock = 0 AND Discontinued = False";
cmd.Parameters.Clear();
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    noStockProductIDs.Add(Convert.ToInt32(reader[0])); // Add
the product ID to the list for products on no stock.
}
connection.Close();

// Retrieve all products that are considered to be selling fast.
connection.Open();
cmd.CommandText = "SELECT TableProducts.ProductID,
SUM(TableCustOrderItems.Quantity - TableCustOrderItems.QuantityVoided -
TableCustOrderItems.QuantityReturned) FROM (TableProducts INNER JOIN
TableCustOrderItems ON TableCustOrderItems.ProductID =
TableProducts.ProductID) INNER JOIN TableCustOrders ON
TableCustOrderItems.OrderID = TableCustOrders.CustOrderID WHERE
TableCustOrders.DeliveryDate >= DateAdd('d', -7, Date()) AND
TableProducts.Discontinued = False GROUP BY TableProducts.ProductID;";
// Gets the total number of each product sold in the past week
cmd.Parameters.Clear();
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())

```

```

        {
            // Get the original amount of stock of the current product
            since its last order
            // If more than half of the original amount of stock has
            been sold in the past week, the product is considered as 'selling fast'
            OleDbConnection con = new OleDbConnection();
            con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0;
Data Source = shopDB.mdb";
            OleDbCommand cmd1 = con.CreateCommand();
            OleDbDataReader reader1;
            con.Open();
            cmd1.CommandText = "SELECT
SUM(TableProducts.Stock+TableSupplierOrderItems.Quantity-
TableSupplierOrderItems.QuantityVoided-
TableSupplierOrderItems.QuantityReturned) FROM (TableProducts INNER JOIN
TableSupplierOrderItems ON TableSupplierOrderItems.ProductID =
TableProducts.ProductID) INNER JOIN TableSupplierOrders ON
TableSupplierOrderItems.OrderID = TableSupplierOrders.SupplierOrderID WHERE
TableSupplierOrders.DeliveryDate > (SELECT MAX(OrderDate) FROM
TableSupplierOrders INNER JOIN TableSupplierOrderItems ON
TableSupplierOrders.SupplierOrderID = TableSupplierOrderItems.OrderID WHERE
TableSupplierOrderItems.ProductID = TableProducts.ProductID) AND
TableProducts.ProductID = @pid AND TableProducts.Discontinued = False GROUP
BY TableProducts.ProductID;";
            // Gets the original amount of stock of the current product
            since its last order
            cmd1.Parameters.Clear();
            cmd1.Parameters.AddWithValue("@pid",
Convert.ToInt32(reader[0]));
            cmd1.Connection = con;
            reader1 = cmd1.ExecuteReader();
            while (reader1.Read())
            {
                double soldPastWeek = Convert.ToDouble(reader[1]); //
Amount sold in the past week
                double ogStock = Convert.ToDouble(reader1[0]); //
Original amount of stock of that product since its last order
                if (soldPastWeek > 0.5 * ogStock)
                {
                    productsSellingFast.Add(Convert.ToInt32(reader[0]));
// Add the product ID to the list for products selling fast
                }
            }
        }
        connection.Close();

        // Retrieve all products that are considered to have a low
        number of sales.
        connection.Open();
        cmd.CommandText = "SELECT TableProducts.ProductID,
TableProducts.FullStock, SUM(TableCustOrderItems.Quantity -
TableCustOrderItems.QuantityVoided - TableCustOrderItems.QuantityReturned)
FROM (TableProducts INNER JOIN TableCustOrderItems ON
TableCustOrderItems.ProductID = TableProducts.ProductID) INNER JOIN
TableCustOrders ON TableCustOrderItems.OrderID = TableCustOrders.CustOrderID
WHERE TableCustOrders.DeliveryDate >= DateAdd('m', -1, Date()) GROUP BY
TableProducts.ProductID, TableProducts.FullStock;";

```



```

// Gets the total number of each product sold in the past month
cmd.Parameters.Clear();
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    // If the amount sold for each product is less than 20% of
the stock level considered to be full stock, the product is considered as
having 'low sales'
    int soldPastMonth = Convert.ToInt32(reader[2]); // Amount
sold in the past month.
    int fullStock = Convert.ToInt32(reader[1]); // Full stock
level

    if (soldPastMonth < 0.20 * fullStock)
    {
        productsSalesLow.Add(Convert.ToInt32(reader[0])); // Add
the product ID to the list for products with low sales
    }
    else
    {
        archiveStockNotices("Low Sales",
Convert.ToInt32(reader[0])); // Archives any existing notices for low sales
    }
}
connection.Close();

// Retrieve all products in supplier orders that have yet to be
delivered.
// All products that are yet to have arrived but have had an
order placed are removed from all of the lists, so duplicate notices are not
generated. Instead, the notice will be dismissed until the product's
delivery date
connection.Open();
cmd.CommandText = "SELECT DISTINCT
TableProducts.ProductID,TableSupplierOrders.DeliveryDate FROM (TableProducts
INNER JOIN TableSupplierOrderItems ON TableProducts.ProductID =
TableSupplierOrderItems.ProductID) INNER JOIN TableSupplierOrders ON
TableSupplierOrderItems.OrderID = TableSupplierOrders.SupplierOrderID WHERE
TableSupplierOrderItems.QuantityVoided = 0 AND
TableSupplierOrderItems.QuantityReturned = 0 AND
(TableSupplierOrders.DeliveryStatus = 'Order Made' OR
TableSupplierOrders.DeliveryStatus = 'Out For Delivery')";
cmd.Parameters.Clear();
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    updateNotice = false;
    if (noStockProductIDs.Contains(Convert.ToInt32(reader[0])))
    {
        noStockProductIDs.Remove(Convert.ToInt32(reader[0]));
        updateNotice = true;
    }
    else if
(lowStockProductIDs.Contains(Convert.ToInt32(reader[0])))
    {
        lowStockProductIDs.Remove(Convert.ToInt32(reader[0]));

```

```

        updateNotice = true;
    }
    else if
(productsSellingFast.Contains(Convert.ToInt32(reader[0])))
    {
        productsSellingFast.Remove(Convert.ToInt32(reader[0]));
        updateNotice = true;
    }
    if (updateNotice) // Dismisses notices for products that are
yet to be delivered until the order's delivery date.
    {
        OleDbConnection con = new OleDbConnection();
        con.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd1 = con.CreateCommand();
        con.Open();
        cmd1.CommandText = "UPDATE TableStockNotices SET
DateToReview = @del, Dismissed = True WHERE ProductID = @pid AND Archived =
False AND Title <> 'Low Sales'";
        cmd1.Parameters.Clear();
        cmd1.Parameters.AddWithValue("@del",
Convert.ToDateTime(reader[1]));
        cmd1.Parameters.AddWithValue("@pid",
Convert.ToInt32(reader[0]));
        cmd1.Connection = con;
        int status = cmd1.ExecuteNonQuery();
        con.Close();
    }
}
connection.Close();

// Retrieve all active notices for low stock.
// If an active notice is found for the product out of stock and
it's a low stock notice, this notice needs to be archived
// If an active notice is found for the product on low stock and
it's found to be selling fast, archive the current low stock notice and
create a new notice for low stock and selling fast
// Otherwise, if an active notice is found for the product low
on stock, a new notice doesn't need to be created
connection.Open();
cmd.CommandText = "SELECT ProductID FROM TableStockNotices WHERE
Archived = False AND Title = 'Low Stock' OR Title = 'Low Stock & Selling
Fast'";
cmd.Parameters.Clear();
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    if (noStockProductIDs.Contains(Convert.ToInt32(reader[0]))
|| productsSellingFast.Contains(Convert.ToInt32(reader[0])))
    {
        archiveStockNotices("Low Stock",
Convert.ToInt32(reader[0]));
    }
    if
(productsSellingFast.Contains(Convert.ToInt32(reader[0])))
    {

```

```

lowStockSellingFastProductIDs.Add(Convert.ToInt32(reader[0]));
        productsSellingFast.Remove(Convert.ToInt32(reader[0]));
    }
    if (lowStockProductIDs.Contains(Convert.ToInt32(reader[0])))
    {
        lowStockProductIDs.Remove(Convert.ToInt32(reader[0]));
    }
}
connection.Close();

// Retrieve all active out of stock notices.
// Remove products from the list of product IDs if there are
active notices found.
connection.Open();
cmd.CommandText = "SELECT ProductID FROM TableStockNotices WHERE
Archived = False AND Dismissed = False AND Title = 'No Stock'";
cmd.Parameters.Clear();
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    int productID = Convert.ToInt32(reader[0]);
    if (noStockProductIDs.Contains(productID)) // if an active
notice is found for the product, a new notice doesn't need to be created
    {
        noStockProductIDs.Remove(productID);
    }
    else // if an active notice is found for a product that is
no longer out of stock, the notice needs to be archived
    {
        archiveStockNotices("No Stock", productID);
    }
}
connection.Close();

// Retrieve all active selling fast notices.
// Remove products from the list of product IDs if there are
active notices found.
connection.Open();
cmd.CommandText = "SELECT ProductID FROM TableStockNotices WHERE
Archived = False AND Dismissed = False AND Title = 'Selling Fast'";
cmd.Parameters.Clear();
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    if
(productsSellingFast.Contains(Convert.ToInt32(reader[0]))) // if an active
notice is found for the product, a new notice doesn't need to be created
    {
        productsSellingFast.Remove(Convert.ToInt32(reader[0]));
    }
    else
    {
        archiveStockNotices("Selling Fast",
Convert.ToInt32(reader[0]));
    }
}
}

```

```

        }
    }
    connection.Close();

    // Retrieve all active low sales notices.
    // Remove products from the list of product IDs if there are
active notices found.
    connection.Open();
    cmd.CommandText = "SELECT ProductID FROM TableStockNotices WHERE
Archived = False AND Dismissed = False AND Title = 'Low Sales'"; // Retrieve
all active low sales notices.
    cmd.Parameters.Clear();
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        if (productsSalesLow.Contains(Convert.ToInt32(reader[0])))
// if an active notice is found for the product, a new notice doesn't need
to be created
        {
            productsSalesLow.Remove(Convert.ToInt32(reader[0]));
        }
        else
        {
            archiveStockNotices("Low Sales",
Convert.ToInt32(reader[0])); // If products with low sales have active
notices and the list of products doesn't contain that product any longer,
the notice can be archived
        }
    }
    connection.Close();

    foreach (int id in lowStockProductIDs) // Finds all products
that are low on stock and selling fast
    {
        if (productsSellingFast.Contains(id))
        {
            productsSellingFast.Remove(id);
            lowStockSellingFastProductIDs.Add(id);
        }
    }
    foreach (int id in lowStockSellingFastProductIDs) // Removes all
product ids of products that are low on stock and selling fast from the low
stock list
    {
        if (lowStockProductIDs.Contains(id))
        {
            lowStockProductIDs.Remove(id);
        }
    }
    createStockNotices("Low Stock & Selling Fast", 2,
lowStockSellingFastProductIDs);
    createStockNotices("Low Stock", 1, lowStockProductIDs);
    createStockNotices("No Stock", 3, noStockProductIDs);
    createStockNotices("Selling Fast", 3, productsSellingFast);
    createStockNotices("Low Sales", 1, productsSalesLow);

```

```

    }
    private static void processEventSpaces() // Issues and updates all
'event spaces available' notices
    {
        using (OleDbConnection con = new
OleDbConnection(connectionString))
        {
            con.Open();

            // Get all future events that haven't been cancelled
            OleDbCommand eventCmd = new OleDbCommand("SELECT * FROM
TableEvents WHERE Cancelled <> True AND EventDate >= @tod", con);
            eventCmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
            OleDbDataReader eventReader = eventCmd.ExecuteReader();

            // Get all active notices related to event spaces
            OleDbCommand noticeCmd = new OleDbCommand("SELECT * FROM
TableEventNotices WHERE Title = 'Event Spaces Available' AND Archived <>
True AND Dismissed <> True", con);

            // Store all notices in a dictionary, boolean set to true if
active
            Dictionary<int, bool> activeNotices = new Dictionary<int,
bool>();

            using (OleDbDataReader noticeReader =
noticeCmd.ExecuteReader())
            {
                while (noticeReader.Read())
                {
                    int eventID = Convert.ToInt32(noticeReader[1]);
                    activeNotices[eventID] = true;
                }
            }

            // Lists to bulk update and insert notices later
            List<int> noticesToArchive = new List<int>();
            List<int> noticesToInsert = new List<int>();

            while (eventReader.Read())
            {
                Event e = new Event(Convert.ToInt32(eventReader[0]),
Convert.ToString(eventReader[1]), Convert.ToDateTime(eventReader[2]),
Convert.ToDateTime(eventReader[3]), Convert.ToDateTime(eventReader[4]),
Convert.ToString(eventReader[5]), Convert.ToSingle(eventReader[6]),
Convert.ToInt32(eventReader[7]), Convert.ToInt32(eventReader[8]),
Convert.ToString(eventReader[9]), Convert.ToString(eventReader[10]),
Convert.ToBoolean(eventReader[11]));
                int eventID = e.id;
                int numattendees = e.numAttendees;
                int capacity = e.capacity;

                bool hasWaitingList = e.GetWaitingListAttendees.Count >
0;

                // Archive all notices for cancelled events

```

```

        if (e.cancelled)
        {
            noticesToArchive.Add(eventID);
            continue;
        }

        // Archive all notices for events where there is no
longer a waiting list
        // Create new notices for events that have a waiting
list but there is no active notice
        if(numattendees < capacity)
        {
            if (!hasWaitingList &&
activeNotices.ContainsKey(eventID))
            {
                noticesToArchive.Add(eventID);
            }
            else if (hasWaitingList &&
!activeNotices.ContainsKey(eventID))
            {
                noticesToInsert.Add(eventID);
            }
        }
        else
        {
            // If the event is at capacity, archive any active
notices

            if (activeNotices.ContainsKey(eventID))
            {
                noticesToArchive.Add(eventID);
            }
        }
    }

    // Bulk archive notices
    if (noticesToArchive.Count > 0)
    {
        OleDbCommand archiveCmd = new OleDbCommand("UPDATE
TableEventNotices SET Archived = True WHERE Title = 'Event Spaces Available'
AND EventID IN (" + string.Join(",", noticesToArchive) + ")", con);
        // Uses IN to archive all event payment notices in one
command

        archiveCmd.ExecuteNonQuery();
    }

    // Bulk insert notices
    if (noticesToInsert.Count > 0)
    {
        foreach (int id in noticesToInsert)
        {
            OleDbCommand insertCmd = new OleDbCommand("INSERT
INTO
TableEventNotices (EventID,Title,Priority,DateIssued,DateToReview,Dismissed,A
rchived) VALUES(@eid, 'Event Spaces Available',2,@tod,@tod,false,false)",
con);

            insertCmd.Parameters.AddWithValue("@eid", id);

```

```

        insertCmd.Parameters.AddWithValue("@tod",
DateTime.Today);
        insertCmd.ExecuteNonQuery();
    }
}

}

}

}
private static void processEventPayments() // Issues and updates all
'Event Payments' notices
{
    using (OleDbConnection con = new
OleDbConnection(connectionString))
    {
        con.Open();

        // Get all events that aren't cancelled

        OleDbCommand eventCmd = new OleDbCommand("SELECT * FROM
TableEvents WHERE Cancelled <> True", con);
        OleDbDataReader eventReader = eventCmd.ExecuteReader(); //
Create new reader and execute reader

        // Get all active notices related to events

        OleDbCommand noticeCmd = new OleDbCommand("SELECT * FROM
TableEventNotices WHERE Title = 'Event Payments' AND Archived <> True",
con);

        // Store all notices in a dictionary, boolean set to true if
active
        Dictionary<int, bool> activeNotices = new Dictionary<int,
bool>();
        using (OleDbDataReader noticeReader =
noticeCmd.ExecuteReader())
        {
            while (noticeReader.Read())
            {
                int eventID = Convert.ToInt32(noticeReader[1]);
                activeNotices[eventID] = true;
            }
        }

        // Lists to bulk update and insert notices later
        List<int> noticesToArchive = new List<int>();
        List<int> noticesToInsert = new List<int>();

        while (eventReader.Read())
        {
            Event e = new Event(Convert.ToInt32(eventReader[0]),
Convert.ToString(eventReader[1]), Convert.ToDateTime(eventReader[2]),
Convert.ToDateTime(eventReader[3]), Convert.ToDateTime(eventReader[4]),
Convert.ToString(eventReader[5]), Convert.ToSingle(eventReader[6]),
Convert.ToInt32(eventReader[7]), Convert.ToInt32(eventReader[8]),
Convert.ToString(eventReader[9]), Convert.ToString(eventReader[10]),
Convert.ToBoolean(eventReader[11]));

```

```

        int eventID = e.id;

        // Get number of unpaid customers

        int numUnpaid = e.GetCustomersUnpaid.Count;
        if(numUnpaid == 0 && activeNotices.ContainsKey(eventID))
// If all customers in the event have paid and the dictionary of active
notices contains the event ID, the notice needs to be archived
        {
            noticesToArchive.Add(eventID);
        }
        else if(numUnpaid>0 &&
!activeNotices.ContainsKey(eventID))
        {
            noticesToInsert.Add(eventID);
        }

    }

    // Bulk archive notices
    if(noticesToArchive.Count > 0)
    {
        OleDbCommand archiveCmd = new OleDbCommand("UPDATE
TableEventNotices SET Archived = True WHERE Title = 'Event Payments' AND
EventID IN (" + string.Join(",", noticesToArchive) + ")", con);
        // Uses IN to archive all event payment notices in one
command
        archiveCmd.ExecuteNonQuery();
    }

    // Bulk insert notices
    if(noticesToInsert.Count > 0)
    {
        foreach(int id in noticesToInsert)
        {
            OleDbCommand insertCmd = new OleDbCommand("INSERT
INTO
TableEventNotices (EventID,Title,Priority,DateIssued,DateToReview,Dismissed,A
rchived) VALUES(@eid, 'Event Payments',2,@tod,@tod,false,false)", con);
            insertCmd.Parameters.AddWithValue("@eid", id);
            insertCmd.Parameters.AddWithValue("@tod",
DateTime.Today);
            insertCmd.ExecuteNonQuery();
        }
    }

}

private static void checkDismissedNotices() // Reactivates notices
that have been dismissed and the review date has passed
{
    // Checks all dismissed notices and reactivates them when the
notice's review date has passed.
    List<string> tablesToUpdate = new List<string>();
    tablesToUpdate.Add("TableStockNotices");
    tablesToUpdate.Add("TableCustOrderNotices");
    tablesToUpdate.Add("TableSupplierOrderNotices");

```



```

        tablesToUpdate.Add("TableEventNotices");
        using (OleDbConnection con = new
OleDbConnection(connectionString))
        {
            con.Open();

            foreach(string table in tablesToUpdate)
            {
                OleDbCommand dismissCmd = new OleDbCommand("UPDATE " +
table + " SET Dismissed = False WHERE DateToReview <= @tod AND Archived =
False AND Dismissed = True",con);
                dismissCmd.Parameters.AddWithValue("@tod",
DateTime.Today);
                dismissCmd.ExecuteNonQuery();
            }
        }
        private static void createStockNotices(string title, int priority,
List<int>products) // Creates a new notice for each of the products passed
in
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            foreach (int id in products)
            {
                connection.Open();
                cmd.CommandText = "INSERT INTO TableStockNotices(ProductID,
Title, Priority, DateIssued, DateToReview, Dismissed, Archived)
VALUES (@pid,@title,@p,@tod,@tod,false,false)";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@pid", id);
                cmd.Parameters.AddWithValue("@title", title);
                cmd.Parameters.AddWithValue("@p", priority);
                cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
                cmd.Connection = connection;
                int status = cmd.ExecuteNonQuery();
                connection.Close();
            }
        }
        private static void archiveStockNotices(string title, int productID)
// Archive stock notice
        {
            OleDbConnection con = new OleDbConnection();
            con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
            OleDbCommand cmd = con.CreateCommand();
            con.Open();
            cmd.CommandText = "UPDATE TableStockNotices SET Archived = True
WHERE ProductID = @pid AND Archived = False AND Title = @title";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@pid", productID);
            cmd.Parameters.AddWithValue("@title", title);
            cmd.Connection = con;

```

```

        int status1 = cmd.ExecuteNonQuery();
        con.Close();
    }
    private static void generateCustOrderNotices() // Issues and updates
all customer order related notices
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT CustOrderID, DeliveryDate,
DeliveryStatus, CollectionDateEnd FROM TableCustOrders"; // Retrieve all
customer orders
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            DateTime deliveryDate = Convert.ToDateTime(reader[1]);
            string deliveryStatus = Convert.ToString(reader[2]);
            int orderID = Convert.ToInt32(reader[0]);
            DateTime collectionDate = Convert.ToDateTime(reader[3]);

            if(deliveryStatus == "Order Made") // If the order hasn't
been processed yet
            {
                if (deliveryDate < DateTime.Today) // If the delivery
date has passed
                {
                    processOverdueOrder(orderID, deliveryDate,
"Processing"); // Check for any active notices, and create any notices if
required
                }
                else
                {
                    //Dismisses any active order notices for the current
order until its delivery date
                    updateActiveOrderNotices(orderID, deliveryDate,
"Processing");
                }
            }
            else if(deliveryStatus == "Ready To Collect") // If the
order hasn't been collected yet
            {
                if (collectionDate < DateTime.Today) // If the
collection window end date has passed
                {
                    processOverdueOrder(orderID, collectionDate,
"Collection"); // Check for any active notices, and create any notices if
required
                }
                else

```

```

        {
            // Dismisses any active order notices for the
current order until its collection date
            updateActiveOrderNotices(orderID, collectionDate,
"Collection");
        }
    }
    else // If the delivery status is voided, returned, or
collected, then any notices for the order must be archived
    {
        archiveOrderNotices(orderID, true);
    }

}
connection.Close();
}
private static void generateSupplyDeliveryNotices() // Issues and
updates all supply delivery related notices
{
    // Issues and updates all supplier order related notices.
OleDbConnection connection = new OleDbConnection();
connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
OleDbCommand cmd = connection.CreateCommand();
OleDbDataReader reader;
connection.Open();
cmd.CommandText = "SELECT SupplierOrderID, DeliveryDate,
DeliveryStatus FROM TableSupplierOrders"; // Retrieve all orders where the
collection date has passed.
cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    int orderID = Convert.ToInt32(reader[0]);
    DateTime deliveryDate = Convert.ToDateTime(reader[1]);
    string deliveryStatus = Convert.ToString(reader[2]);
    if(deliveryStatus == "Order Made" || deliveryStatus == "Out
For Delivery") // If the order hasn't been delivered yet
    {
        if (deliveryDate < DateTime.Today) // If the delivery
has passed
        {
            processOverdueOrder(orderID, deliveryDate,
"Supply"); // Check for any active notices, and create any notices if
required
        }
        else
        {
            // Dismisses any active order notices for the
current order until its delivery date
            updateActiveOrderNotices(orderID, deliveryDate,
"Supply");
        }
    }
}
}

```

```

        }
        else // If the delivery status is voided, returned, or
delivered, then any notices for the order must be archived
        {
            archiveOrderNotices(orderID, false);
        }

    }
    connection.Close();
}

private static void archiveOrderNotices(int orderID, bool
customerOrder) // Archive order notice
{
    OleDbConnection con = new OleDbConnection();
    con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
    OleDbCommand cmd = con.CreateCommand();
    con.Open();
    if (customerOrder)
    {
        cmd.CommandText = "UPDATE TableCustOrderNotices SET Archived
= True WHERE OrderID = @oid AND Archived <> True";
    }
    else
    {
        cmd.CommandText = "UPDATE TableSupplierOrderNotices SET
Archived = True WHERE OrderID = @oid AND Archived <> True";
    }
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@oid", orderID);
    cmd.Connection = con;
    cmd.ExecuteNonQuery();
    con.Close();
}

private static void processOverdueOrder(int orderID, DateTime date,
string type) // Creates the title based on the type of notice, assigns a
priority
{
    // Creates the title based on the type of notice
    string title = "";
    switch (type)
    {
        case "Collection":
            title = "Order Collection Overdue: ";
            break;
        case "Processing":
            title = "Order Processing Overdue: ";
            break;
        case "Supply":
            title = "Supply Delivery Overdue: ";
            break;
    }
    // Assigns a priority and sets the title
    if (date < DateTime.Today.AddDays(-5))
    {

```

```

        processOrderNotice(orderID, title + ">5 days", 3);
    }
    else if (date < DateTime.Today.AddDays(-3))
    {
        processOrderNotice(orderID, title + "3 days", 2);
    }
    else if (date < DateTime.Today.AddDays(-1))
    {
        processOrderNotice(orderID, title + "1 day", 1);
    }
}

private static void processOrderNotice(int orderID, string title,
int priority)
{
    // If there are existing notices for all the other overdue
    periods other than the overdue period of the notice to be processed, all
    those notices should be archived.
    // If there is not an existing notice for the overdue period of
    the notice to be processed, then a new notice should be created.
    if (!existingOrderNotice(orderID, title, false)) // If there is
    an existing notice for the current overdue period
    {
        createOrderNotice(orderID, title, priority); // Create new
        notice if there is no existing notice for that overdue period
    }
    if (existingOrderNotice(orderID, title, true)) // If there are
    existing notices for other overdue periods
    {
        archiveOtherNotices(orderID, title); // Archive all other
        notices relating to a different overdue period
    }
}

private static void archiveOtherNotices(int orderID, string title)
// Archives all notices that are of the same type but are referencing the
wrong overdue period.

{
    // This is used so that only one notice of one type is active.
    OleDbConnection con = new OleDbConnection();
    con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
    OleDbCommand cmd = con.CreateCommand();
    con.Open();
    if (title.Contains("Collection"))
    {
        cmd.CommandText = "UPDATE TableCustOrderNotices SET Archived
= True WHERE OrderID = @oid AND Title <> @title AND Title LIKE
'%Collection%' AND Archived <> True";
    }
    if (title.Contains("Processing"))
    {
        cmd.CommandText = "UPDATE TableCustOrderNotices SET Archived
= True WHERE OrderID = @oid AND Title <> @title AND Title LIKE
'%Processing%' AND Archived <> True";
    }
    if (title.Contains("Supply"))
    {

```

```

        cmd.CommandText = "UPDATE TableSupplierOrderNotices SET
Archived = True WHERE OrderID = @oid AND Title <> @title AND Title LIKE
'%Supply%' AND Archived <> True";
    }
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@oid", orderID);
    cmd.Parameters.AddWithValue("@title", title);
    cmd.Connection = con;
    cmd.ExecuteNonQuery();
    con.Close();
}
private static bool existingOrderNotice(int orderID, string title,
bool other) // Checks if an order notice exists
{
    if (other) // If looking for other overdue notices
    {
        OleDbConnection con = new OleDbConnection();
        con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0;
Data Source = shopDB.mdb";
        OleDbCommand cmd = con.CreateCommand();
        OleDbDataReader reader;
        con.Open();
        if (title.Contains("Collection"))
        {
            cmd.CommandText = "SELECT OrderID FROM
TableCustOrderNotices WHERE OrderID = @oid AND Title <> @title AND Title
LIKE '%Collection%' AND Archived <> True";
        }
        if (title.Contains("Processing"))
        {
            cmd.CommandText = "SELECT OrderID FROM
TableCustOrderNotices WHERE OrderID = @oid AND Title <> @title AND Title
LIKE '%Processing%' AND Archived <> True";
        }
        if (title.Contains("Supply"))
        {
            cmd.CommandText = "SELECT OrderID FROM
TableSupplierOrderNotices WHERE OrderID = @oid AND Title <> @title AND Title
LIKE '%Supply%' AND Archived <> True";
        }
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@oid", orderID);
        cmd.Parameters.AddWithValue("@title", title);
        cmd.Connection = con;
        reader = cmd.ExecuteReader();
        bool noticeExists = reader.Read();
        con.Close();
        return noticeExists;
    }
    else // Finding if the notice passed in exists itself
    {
        OleDbConnection con = new OleDbConnection();
        con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0;
Data Source = shopDB.mdb";
        OleDbCommand cmd = con.CreateCommand();
        OleDbDataReader reader;
        con.Open();

```

```

        cmd.CommandText = "SELECT OrderID FROM TableCustOrderNotices
WHERE OrderID = @oid AND Title = @title AND Archived <> True";
        if (title.Contains("Collection") ||
title.Contains("Processing"))
        {
            cmd.CommandText = "SELECT OrderID FROM
TableCustOrderNotices WHERE OrderID = @oid AND Title = @title AND Archived
<> True";
        }
        if (title.Contains("Supply"))
        {
            cmd.CommandText = "SELECT OrderID FROM
TableSupplierOrderNotices WHERE OrderID = @oid AND Title = @title AND
Archived <> True";
        }
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@oid", orderID);
        cmd.Parameters.AddWithValue("@title", title);
        cmd.Connection = con;
        reader = cmd.ExecuteReader();
        bool noticeExists = reader.Read();
        con.Close();
        return noticeExists;
    }
}

private static void createOrderNotice(int orderID, string title, int
priority) // Creates a new order notice
{
    // Creates a new notice with the parameters passed in
    OleDbConnection con = new OleDbConnection();
    con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
    OleDbCommand cmd = con.CreateCommand();
    con.Open();
    if (title.Contains("Collection") ||
title.Contains("Processing"))
    {
        cmd.CommandText = "INSERT INTO
TableCustOrderNotices(OrderID, Title, Priority, DateIssued, DateToReview,
Dismissed, Archived) VALUES(@oid, @title, @priority, @tod, @tod, false,
false)";
    }
    if (title.Contains("Supply"))
    {
        cmd.CommandText = "INSERT INTO
TableSupplierOrderNotices(OrderID, Title, Priority, DateIssued,
DateToReview, Dismissed, Archived) VALUES(@oid, @title, @priority, @tod,
@tod, false, false)";
    }
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@oid", orderID);
    cmd.Parameters.AddWithValue("@title", title);
    cmd.Parameters.AddWithValue("@priority", priority);
    cmd.Parameters.AddWithValue("@tod", DateTime.Today);
    cmd.Connection = con;
    cmd.ExecuteNonQuery();
    con.Close();
}

```

```

    }
    private static void updateActiveOrderNotices(int orderID, DateTime
date, string title)// Updates all active order notices where the
collection/delivery date has been changed. Dismisses the notice until that
date.
    {
        OleDbConnection con = new OleDbConnection();
        con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
        OleDbCommand cmd = con.CreateCommand();
        con.Open();
        if (title.Contains("Collection") ||
title.Contains("Processing"))
        {
            cmd.CommandText = "UPDATE TableCustOrderNotices SET
DateToReview = @del, Dismissed = True WHERE OrderID = @oid AND Dismissed <>
True AND Archived <> True";
        }
        if (title.Contains("Supply"))
        {
            cmd.CommandText = "UPDATE TableSupplierOrderNotices SET
DateToReview = @del, Dismissed = True WHERE OrderID = @oid AND Dismissed <>
True AND Archived <> True";
        }
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@del",
DateTime.Parse(date.ToShortDateString()));
        cmd.Parameters.AddWithValue("@oid", orderID);
        cmd.Connection = con;
        int status = cmd.ExecuteNonQuery();
        con.Close();
    }

    // Displaying the calendar
    public List<Event> loadEventsMonthList(int monthNum, int year) //
Loads all events for the month
    {
        //loads all events for the month displayed
        List<Event> eventsMonthList = new List<Event>();
        //load all yarn weights into a list
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;

        connection.Open();
        cmd.CommandText = "SELECT * FROM TableEvents WHERE
Month(EventDate) = @mo AND Year(EventDate) = @ye AND Cancelled = False";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@mo", monthNum);
        cmd.Parameters.AddWithValue("@ye", year);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            bool cancelled = false;

```



```

        if (Convert.ToInt32(reader[11]) == -1 ||
Convert.ToBoolean(reader[11]))
        {
            cancelled = true;
        }
        if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
        {
            eventsMonthList.Add(new
Event(Convert.ToInt32(reader[0]), Convert.ToString(reader[1]),
Convert.ToDateTime(reader[2]), Convert.ToDateTime(reader[3]),
Convert.ToDateTime(reader[4]), Convert.ToString(reader[5]),
Convert.ToSingle(reader[6]), Convert.ToInt32(reader[7]),
Convert.ToInt32(reader[8]), Convert.ToString(reader[9]),
Convert.ToString(reader[10]), cancelled));
        }
        else
        {
            continue;
        }
    }
    connection.Close();
    return eventsMonthList;
}

public List<Event> loadEventsDayList(int dayNum,
List<Event>eventsOfMonth) // Loads all events for the day
{
    List<Event> eventsOfDay = new List<Event>();
    foreach (Event e in eventsOfMonth)
    {
        if (dayNum == e.date.Day)
        {
            eventsOfDay.Add(e);
        }
    }
    return eventsOfDay;
}

private void refreshCurrentMonth() // Loads all events for the
current month
{
    month = DateTime.Now.Month;
    year = DateTime.Now.Year;
    refreshCalendar();
}

private void refreshCalendar()
{
    flpMonday.Controls.Clear();
    flpTuesday.Controls.Clear();
    flpWednesday.Controls.Clear();
    flpThursday.Controls.Clear();
    flpFriday.Controls.Clear();
    flpSaturday.Controls.Clear();
    flpSunday.Controls.Clear();

    string monthName =
DateTimeFormatInfo.CurrentInfo.GetMonthName(month);
    lblDate.Text = monthName + " " + year;
}

```

```

        // first day of month
        DateTime startOfMonth = new DateTime(year, month, 1);
        int daysInMonth = DateTime.DaysInMonth(year, month); // number
of days in the month
        int daysOfWeek =
Convert.ToInt32(startOfMonth.DayOfWeek.ToString("d")); // gets the start day
of the month
        if(daysOfWeek == 0)
        {
            daysOfWeek = 7;
        }
        for (int i = 1; i < daysOfWeek; i++) // Loops through each day
of the week and adds the event/notice to the correct day
        {
            string dayName = Enum.GetName(typeof(DayOfWeek), i);
            UserControlBlank ucblank = new UserControlBlank();
            switch (dayName)
            {
                case "Monday":
                    flpMonday.Controls.Add(ucblank);
                    break;
                case "Tuesday":
                    flpTuesday.Controls.Add(ucblank);
                    break;
                case "Wednesday":
                    flpWednesday.Controls.Add(ucblank);
                    break;
                case "Thursday":
                    flpThursday.Controls.Add(ucblank);
                    break;
                case "Friday":
                    flpFriday.Controls.Add(ucblank);
                    break;
                case "Saturday":
                    flpSaturday.Controls.Add(ucblank);
                    break;
                case "Sunday":
                    flpSunday.Controls.Add(ucblank);
                    break;
            }
        }
        for (int i = 1; i <= daysInMonth; i++)
        {
            string dateString = startOfMonth.ToString("MM/yyyy");
            if (i < 10)
            {
                dateString = "0" + i.ToString() + "/" + dateString;
            } //gets date in dd/MM/yyyy format
            else
            {
                dateString = i.ToString() + "/" + dateString;
            }
            DateTime date = Convert.ToDateTime(dateString);
            //displays each of the dates
            CalendarDay ucdays = new CalendarDay(true);
            ucdays.days(i, month, year);

```

```

        ucdays.displayEvents(loadEventsDayList(i,
loadEventsMonthList(month, year)));
        ucdays.displayNumDueDeliveries(i, month, year);
        ucdays.displayNumCollections(i, month, year);
        switch (date.ToString("dddd"))
        {
            case "Monday":
                flpMonday.Controls.Add(ucdays);
                break;
            case "Tuesday":
                flpTuesday.Controls.Add(ucdays);
                break;
            case "Wednesday":
                flpWednesday.Controls.Add(ucdays);
                break;
            case "Thursday":
                flpThursday.Controls.Add(ucdays);
                break;
            case "Friday":
                flpFriday.Controls.Add(ucdays);
                break;
            case "Saturday":
                flpSaturday.Controls.Add(ucdays);
                break;
            case "Sunday":
                flpSunday.Controls.Add(ucdays);
                break;
        }
    }
}
private void pbxNextMonth_Click(object sender, EventArgs e) // View
the next month
{
    if (month == 12)
    {
        month = 1;
        year++;
    }
    else
    {
        month++;
    }
    refreshCalendar();
}
private void pbxPrevMonth_Click(object sender, EventArgs e) // View
the previous month
{
    if (month == 1)
    {
        month = 12;
        year--;
    }
    else
    {
        month--;
    }
    refreshCalendar();
}

```

```

    }
    private void btnToday_Click(object sender, EventArgs e) // View this
month
    {
        refreshCurrentMonth();
    }

    // Accessing forms
    private void btnOrders_Click(object sender, EventArgs e)
    {
        Orders newFormOrders = new Orders(null, null);
        newFormOrders.ShowDialog();

        // Issue and update all order notices and stock notices
        refreshOrderNotices();
        refreshStockNotices();
    }
    private void btnSuppliers_Click(object sender, EventArgs e)
    {
        Suppliers newFormSuppliers = new Suppliers();
        newFormSuppliers.ShowDialog();
        refreshStockNotices();
    }
    private void btnCustomers_Click(object sender, EventArgs e)
    {
        Customers newFormCustomers = new Customers();
        newFormCustomers.ShowDialog();
        refreshEventNotices();
    }
    private void btnProducts_Click(object sender, EventArgs e)
    {
        Products newFormProducts = new Products(null);
        newFormProducts.ShowDialog();

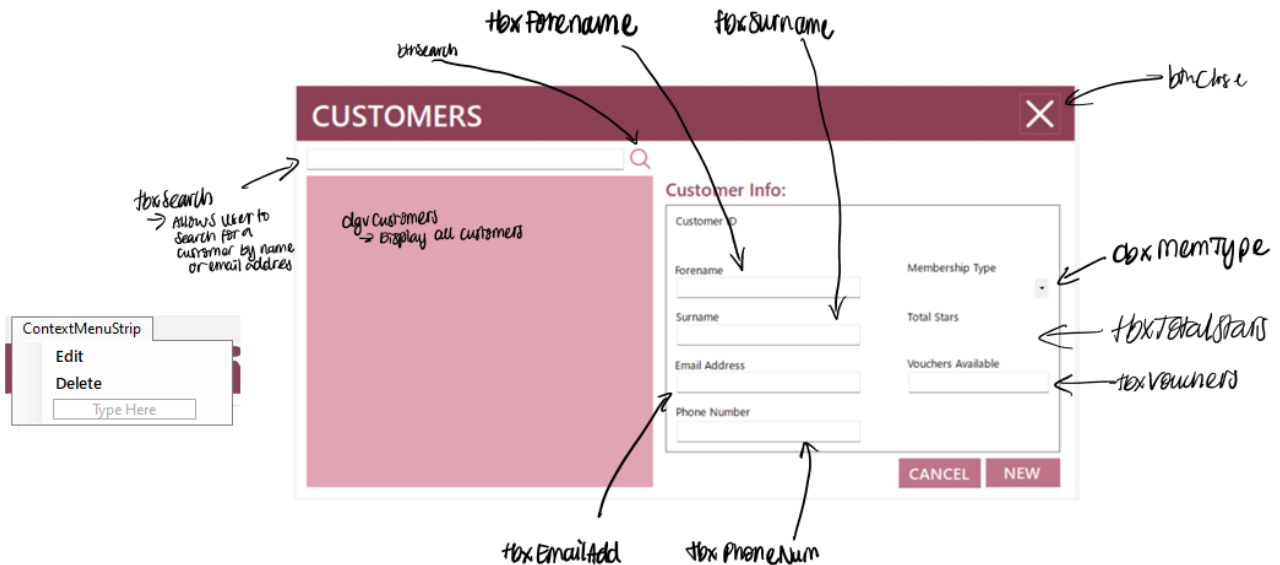
        // Issue and update all stock notices
        refreshStockNotices();
    }
    private void btnNotices_Click(object sender, EventArgs e)
    {
        Notices newFormNotices = new Notices();
        newFormNotices.ShowDialog();
    }
    private void btnEvents_Click(object sender, EventArgs e)
    {
        Events newFormEvents = new Events(null);
        newFormEvents.ShowDialog();

        // Issue and update all event notices
        refreshEventNotices();
    }
}
}

```

Customers.cs

Form Design



Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Linq;
using System.Net.Mail;
using System.Reflection;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public partial class Customers : Form
    {
        int index;
        Customer currentCust;
        bool editMode = false;
        List<string> memTypes = new List<string>();

        // Make form draggable
        bool drag = false;
        Point dragCursor;
        Point dragForm;

        public Customers()
        {
            Visible = false;
        }
    }
}
```

```

        InitializeComponent();
    }

    // CUSTOM DRAGGABLE FORM BORDER
    private void pbxBar_MouseDown(object sender, MouseEventArgs e) //
Detects when the mouse is down and allows the user to drag
    {
        drag = true;
        dragCursor = Cursor.Position;
        dragForm = Location;
    }
    private void pbxBar_MouseMove(object sender, MouseEventArgs e) //
Detects when the mouse is down and moving
    {
        if (drag)
        {
            Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
            Location = Point.Add(dragForm, new Size(difference));
        }
    }
    private void pbxBar_MouseUp(object sender, MouseEventArgs e) //
Detects when the mouse is not dragging the form
    {
        drag = false;
    }
    private void btnClose_Click(object sender, EventArgs e) // Closes
the form
    {
        if (editMode)
        {
            DialogResult result = MessageBox.Show("Are you sure you want
to discard all changes?", "Discard Edits", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
            if (result == DialogResult.Yes)
            {
                Close();
            }
        }
        else
        {
            Close();
        }
    }

    // VIEW ALL CUSTOMER INFORMATION
    private void FormCustomers_Load(object sender, EventArgs e)
    {
        refresh();
        Visible = true;
    }
    private void refresh() // Disables editing and displays all
customers.
    {
        disableEditing();
        editMode = false;
        OleDbConnection connection = new OleDbConnection();

```

```

        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableCustomers WHERE MemType <>
'Guest' AND MemType <> 'Deleted'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvCustomers.DataSource = bindingSource;
        connection.Close();

        //ensures there is always something in the textboxes so null
values don't appear when you edit another customer straight after
        refreshCustomerInfo(0);

    }
    private void tbxSearch_TextChanged(object sender, EventArgs e) // As
the user types in the search bar, customers are searched for according to
the input.
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableCustomers WHERE
(FirstName LIKE @qu OR Surname LIKE @qu OR EmailAddress LIKE @qu) AND
(MemType <> 'Guest' AND MemType <> 'Deleted')";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@qu", "%" + tbxSearch.Text +
"%");

            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                BindingSource bindingSource = new BindingSource();
                bindingSource.DataSource = reader;
                dgvCustomers.DataSource = bindingSource;
                refreshCustomerInfo(0);
            }
            connection.Close();
        }
        else
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();

```

```

        cmd.CommandText = "SELECT * FROM TableCustomers WHERE
MemType <> 'Guest' AND MemType <> 'Deleted'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvCustomers.DataSource = bindingSource;
        connection.Close();
        refreshCustomerInfo(0);
    }
}

private void btnSearch_Click(object sender, EventArgs e) // Search
for the customer by name or email address
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableCustomers WHERE
(FirstName LIKE @qu OR Surname LIKE @qu OR EmailAddress LIKE @qu) AND
(MemType <> 'Guest' AND MemType <> 'Deleted')";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@qu", "%" + tbxSearch.Text + "%");
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new BindingSource();
            bindingSource.DataSource = reader;
            dgvCustomers.DataSource = bindingSource;
            refreshCustomerInfo(0);
        }
        else
        {
            MessageBox.Show("No customers found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
    else
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableCustomers WHERE
MemType <> 'Guest' AND MemType <> 'Deleted'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
    }
}

```



```

        dgvCustomers.DataSource = bindingSource;
        connection.Close();
        refreshCustomerInfo(0);
    }
}

private void dgvCustomers_CellClick(object sender,
DataGridViewCellEventArgs e) // Allows the user to click on each customer
and view all details of each customer.
{
    disableEditing(); // If the user clicks on another customer
while editing, disable editing
    index = e.RowIndex;
    refreshCustomerInfo(index);
}

private void refreshCustomerInfo(int index)
{
    if (index < 0) // Index cannot be negative
    {
        index = 0;
        dgvCustomers.ClearSelection();
        dgvCustomers.CurrentCell = dgvCustomers.Rows[0].Cells[0];
        dgvCustomers[0,0].Selected = true;
        dgvCustomers.FirstDisplayedScrollingRowIndex = 0;
        dgvCustomers.Focus();
    }
    DataGridViewRow selectedRow = dgvCustomers.Rows[index];
    currentCust = new
Customer(Convert.ToInt32(selectedRow.Cells[0].Value),
selectedRow.Cells[1].Value.ToString(),
selectedRow.Cells[2].Value.ToString(),
selectedRow.Cells[3].Value.ToString(),
selectedRow.Cells[4].Value.ToString(),
selectedRow.Cells[5].Value.ToString(),
Convert.ToInt32(selectedRow.Cells[6].Value),
Convert.ToInt32(selectedRow.Cells[7].Value));
    tbxCustID.Text = currentCust.id.ToString();
    tbxForename.Text = currentCust.firstName;
    tbxSurname.Text = currentCust.lastName;
    tbxPhoneNum.Text = currentCust.phoneNum;
    tbxEmailAdd.Text = currentCust.emailAddress;
    cbxMemType.Text = currentCust.memType;
    tbxTotalStars.Text = currentCust.totalStars.ToString();
    tbxVouchers.Text = currentCust.voucherNum.ToString();
}

// ADDING, EDITING AND DELETING
private void disableEditing()
{
    editMode = false;
    btnAddSave.Text = "Add New Customer";
    tbxCustID.ReadOnly = true;
    tbxForename.ReadOnly = true;
    tbxSurname.ReadOnly = true;
    tbxEmailAdd.ReadOnly = true;
    tbxPhoneNum.ReadOnly = true;
    cbxMemType.Enabled = false;
    tbxTotalStars.ReadOnly = true;
}

```

```

        tbxVouchers.ReadOnly = true;
        btnCancel.Visible = false;
        tbxCustID.BackColor = SystemColors.ControlLightLight;
        tbxForename.BackColor = SystemColors.ControlLightLight;
        tbxSurname.BackColor = SystemColors.ControlLightLight;
        tbxEmailAdd.BackColor = SystemColors.ControlLightLight;
        tbxPhoneNum.BackColor = SystemColors.ControlLightLight;
        cbxMemType.BackColor = SystemColors.ControlLightLight;
        tbxTotalStars.BackColor = SystemColors.ControlLightLight;
        tbxVouchers.BackColor = SystemColors.ControlLightLight;
        tbxCustID.BorderStyle = BorderStyle.None;
        tbxForename.BorderStyle = BorderStyle.None;
        tbxSurname.BorderStyle = BorderStyle.None;
        tbxEmailAdd.BorderStyle = BorderStyle.None;
        tbxPhoneNum.BorderStyle = BorderStyle.None;
        cbxMemType.FlatStyle = FlatStyle.Flat;
        tbxTotalStars.BorderStyle = BorderStyle.None;
        tbxVouchers.BorderStyle = BorderStyle.None;
    }
    private void enableEditing()
    {
        tbxForename.BackColor = Color.White;
        tbxSurname.BackColor = Color.White;
        tbxEmailAdd.BackColor = Color.White;
        tbxPhoneNum.BackColor = Color.White;
        cbxMemType.BackColor = Color.White;
        tbxTotalStars.BackColor = Color.White;
        tbxVouchers.BackColor = Color.White;
        tbxForename.BorderStyle = BorderStyle.Fixed3D;
        tbxSurname.BorderStyle = BorderStyle.Fixed3D;
        tbxEmailAdd.BorderStyle = BorderStyle.Fixed3D;
        tbxPhoneNum.BorderStyle = BorderStyle.Fixed3D;
        cbxMemType.FlatStyle = FlatStyle.System;
        tbxTotalStars.BorderStyle = BorderStyle.Fixed3D;
        tbxVouchers.BorderStyle = BorderStyle.Fixed3D;
        tbxForename.ReadOnly = false;
        tbxSurname.ReadOnly = false;
        tbxEmailAdd.ReadOnly = false;
        tbxPhoneNum.ReadOnly = false;
        cbxMemType.Enabled = true;
        tbxTotalStars.ReadOnly = false;
        tbxVouchers.ReadOnly = false;
        btnCancel.Visible = true;
    }
    private void editToolStripMenuItem_Click(object sender, EventArgs e)
// Edit all customer details
    {
        //changes the add button to a save button
        editMode = true;
        enableEditing();
        btnAddSave.Text = "Save";
    }
    private void deleteToolStripMenuItem_Click(object sender, EventArgs
e) // Delete customer data from the system
    {
        if (dgvCustomers.SelectedRows != null)
        {

```

```

        //ensures user does not accidentally delete customer
        DialogResult result = MessageBox.Show("Are you sure you want
to permanently delete this customer?", "Delete Customer",
MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
        if (result == DialogResult.Yes)
        {
            bool openOrders = false;

            // All orders made by the customer must have been
collected, voided or returned before deleting their data
            OleDbConnection con = new OleDbConnection();
            con.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = con.CreateCommand();
            OleDbDataReader reader;
            con.Open();
            cmd.CommandText = "SELECT * FROM TableCustOrders WHERE
CustID = @cid AND (DeliveryStatus = 'Order Made' OR DeliveryStatus = 'Ready
To Collect')";

            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@cid", currentCust.id);
            cmd.Connection = con;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                openOrders = true;
            }
            con.Close();
            if (openOrders)
            {
                MessageBox.Show("Review all open orders before
deleting customer data.", "Open Orders", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
            }
            else
            {
                // Deletes the customer from all events
                OleDbConnection conEventAtt = new OleDbConnection();
                conEventAtt.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmdEventAtt =
conEventAtt.CreateCommand();
                conEventAtt.Open();
                cmdEventAtt.CommandText = "DELETE FROM
TableEventAttendees WHERE CustID = @cid";
                cmdEventAtt.Parameters.Clear();
                cmdEventAtt.Parameters.AddWithValue("@cid",
currentCust.id);

                cmdEventAtt.Connection = conEventAtt;
                int statusEventAtt = cmdEventAtt.ExecuteNonQuery();
                conEventAtt.Close();

                // Sets the customer record to a 'deleted state' to
maintain referential integrity
                OleDbConnection conCust = new OleDbConnection();
                conCust.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";

```

```

        OleDbCommand cmdCust = conCust.CreateCommand();
        conCust.Open();
        cmdCust.CommandText = "UPDATE TableCustomers SET
FirstName = 'Deleted', Surname = 'Customer', PhoneNum = 'N/A', EmailAddress
= 'N/A', MemType = 'Deleted', TotalStars = 0, VoucherNum = 0 WHERE CustID =
@cid";

        cmdCust.Parameters.Clear();
        cmdCust.Parameters.AddWithValue("@cid",
currentCust.id);

        cmdCust.Connection = conCust;
        int statusCust = cmdCust.ExecuteNonQuery();
        conCust.Close();
        MessageBox.Show("Customer deleted.", "Delete
Customer", MessageBoxButtons.OK, MessageBoxIcon.Information);
        refresh();
    }

    }
    else
    {
        MessageBox.Show("Customer not deleted.", "Delete
Customer", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    }
    else
    {
        MessageBox.Show("Select a customer to delete.", "Customer
Not Selected", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}

private void btnCancel_Click(object sender, EventArgs e) // Disable
editing and clear all changes
{
    int index = dgvCustomers.SelectedRows[0].Index;
    refresh();
    dgvCustomers.ClearSelection();
    dgvCustomers.CurrentCell = dgvCustomers.Rows[index].Cells[0];
    dgvCustomers[0, index].Selected = true;
    dgvCustomers.FirstDisplayedScrollingRowIndex = index;
    dgvCustomers.Focus();
    refreshCustomerInfo(index);
}

// Validation

private bool validName(string text)
{
    string allowedCharacters =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ-";

    // Check each character
    foreach (char c in text)
    {
        bool found = false;
        for (int i = 0; i < allowedCharacters.Length; i++)
        {
            if (c == allowedCharacters[i])
            {

```

```

        found = true;
    }
}
if (!found)
{
    MessageBox.Show("Input valid details in all fields.",
"Invalid Inputs", MessageBoxButtons.OK, MessageBoxIcon.Error);
    return found;
}

return true;
}
private bool validContactInfo()
{
    // Checks if there is at least a phone number or email address
entered.
    // Checks that any inputs are valid.
    if (!string.IsNullOrEmpty(tbxEmailAdd.Text) &&
!string.IsNullOrEmpty(tbxPhoneNum.Text)) // If both fields have inputs
in, both need to be validated
    {
        if (!validEmailAddress(tbxEmailAdd.Text))
        {
            return false;
        }
        if (!validPhoneNumber(tbxPhoneNum.Text))
        {
            return false;
        }
        return true;
    }
    else if (!string.IsNullOrEmpty(tbxEmailAdd.Text))
    {
        if (!validEmailAddress(tbxEmailAdd.Text))
        {
            return false;
        }
        return true;
    }
    else if (!string.IsNullOrEmpty(tbxPhoneNum.Text))
    {
        if (!validPhoneNumber(tbxPhoneNum.Text))
        {
            return false;
        }
        return true;
    }
    else
    {
        // No inputs for both fields
        return false;
    }
}
private bool validEmailAddress(string email)
{
    try

```

```

        {
            var mailAddress = new MailAddress(email);
            return true;
        }
        catch
        {
            MessageBox.Show("Enter a valid email address.", "Invalid
Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
    }
    private bool validPhoneNumber(string phoneNumber)
    {
        // Checks that the phone number entered is of a valid length
        i.e. longer than 7
        // Checks that the phone number entered does not consist of any
        letters
        // Uses a regular expression to check all characters in the
        inputted phone number are valid
        string validCharacters = @"^[0-9+\s\-\(\)]+$";
        if (phoneNumber.Length > 7)
        {
            // Allows digits, plus signs, spaces, dashes, opening and
            closing parentheses
            if (Regex.Match(phoneNumber, validCharacters).Success)
            {
                return true;
            }
            else
            {
                MessageBox.Show("Enter a valid phone number.", "Invalid
Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
                return false;
            }
            // Suppliers are often international, so validating all
            phone numbers for all international phone numbers would require the use of
            an external API
        }
        else
        {
            MessageBox.Show("Phone number must be longer than 7
characters.", "Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
    }
    private bool validNumber(string str)
    {
        if (!uint.TryParse(str, out uint result)) // Checks if the
        string can be converted into a positive integer
        {
            MessageBox.Show("Enter a valid number.", "Invalid Input",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
        else
        {

```

```

        return true;
    }
}

private void btnAddSave_Click(object sender, EventArgs e) // Add a
new customer or save changes made to customer details
{
    if (!editMode)
    {
        NewCustomer formNewCustomer = new NewCustomer();
        formNewCustomer.ShowDialog();
        refresh();
    }
    if (editMode)
    {
        //editMode is set to true when the user uses the context
        menu strip and clicks edit
        //editMode is set to false whenever the form refreshes
        (after deleting, adding or editing a record) or when the user clicks on
        another customer
        if (string.IsNullOrEmpty(tbxForename.Text) ||
string.IsNullOrEmpty(tbxSurname.Text) ||
(string.IsNullOrEmpty(tbxEmailAdd.Text) &&
string.IsNullOrEmpty(tbxPhoneNum.Text))) // Checks that name fields are
filled in and one of phone number or email address is filled in
        {
            MessageBox.Show("Enter values in all required fields.",
"Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else if (!validContactInfo() ||
!validNumber(tbxTotalStars.Text) || !validNumber(tbxVouchers.Text) ||
!validName(tbxForename.Text) || !validName(tbxSurname.Text))
        {
            // Message boxes are shown when the subroutines are
called
        }
        else
        {
            if (string.IsNullOrEmpty(tbxEmailAdd.Text))
            {
                tbxEmailAdd.Text = "N/A";
            }
            if (string.IsNullOrEmpty(tbxPhoneNum.Text))
            {
                tbxPhoneNum.Text = "N/A";
            }
            if (string.IsNullOrEmpty(tbxTotalStars.Text))
            {
                tbxTotalStars.Text = "0";
            }
            if (string.IsNullOrEmpty(cbxMemType.Text))
            {
                cbxMemType.Text = "Bronze"; // Sets the customer's
membership status to bronze by default.
            }
            if (string.IsNullOrEmpty(tbxVouchers.Text))
            {
                tbxVouchers.Text = "0";
            }
        }
    }
}

```


NewCustomer.cs

Form Design

The form design for `NewCustomer.cs` includes the following elements and annotations:

- Form Title:** NEW CUSTOMER (with a close button 'X').
- Input Fields:**
 - Forename (labeled `tbxForename`)
 - Surname (labeled `tbxSurname`)
 - Email Address (labeled `tbxEmailAdd`)
 - Phone Number (labeled `tbxPhoneNum`)
 - Membership Type (dropdown menu, labeled `cbxMemType` with note: "Allows user to select membership from 'Gold', 'Silver' or 'Bronze'")
 - Total Stars (labeled `tbxTotalStars`)
 - Vouchers Available (labeled `tbxVouchers`)
- Buttons:**
 - CLEAR** (labeled `btnClear` with note: "Clears all details entered & sets to default")
 - ADD** (labeled `btnAdd` with note: "Adds customer to database")
- Validation:**
 - A red text label "Customer exists with these contact details" is shown below the Phone Number field.
 - A handwritten note `lblExisting` points to this label, stating: "Displayed when the user enters an email address that is already registered".

Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Linq;
using System.Net.Mail;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public partial class NewCustomer : Form
    {
        List<string> memTypes = new List<string>();
        bool existingEmailAddress = false;
        bool existingPhoneNumber = false;
        // Make form draggable
        bool drag = false;
        Point dragCursor;
        Point dragForm;

        // CUSTOM DRAGGABLE FORM BORDER
        private void pbxBar_MouseDown(object sender, MouseEventArgs e)
        {
            drag = true;
            dragCursor = Cursor.Position;
        }
    }
}
```

```

        dragForm = Location;
    }
    private void pbxBar_MouseMove(object sender, MouseEventArgs e)
    {
        if (drag)
        {
            Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
            Location = Point.Add(dragForm, new Size(difference));
        }
    }
    private void pbxBar_MouseUp(object sender, MouseEventArgs e)
    {
        drag = false;
    }
    private void btnClose_Click(object sender, EventArgs e)
    {
        DialogResult result = MessageBox.Show("Are you sure you want to
discard all changes?", "Discard Creator", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
        if (result == DialogResult.Yes)
        {
            Close();
        }
    }

    public NewCustomer()
    {
        Visible = false;
        InitializeComponent();
    }
    private void FormNewCustomer_Load(object sender, EventArgs e)
    {
        Visible = true;

        // Validation

        private bool validName(string text)
        {
            string allowedCharacters =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ-";

            // Check each character
            foreach (char c in text)
            {
                bool found = false;
                for (int i = 0; i < allowedCharacters.Length; i++)
                {
                    if (c == allowedCharacters[i])
                    {
                        found = true;
                    }
                }
                if (!found)
                {

```

```

        MessageBox.Show("Input valid details in all fields.",
"Invalid Inputs", MessageBoxButtons.OK, MessageBoxIcon.Error);
        return found;
    }
}

return true;
}
private bool validContactInfo()
{
    // Checks if there is at least a phone number or email address
entered.
    // Checks that any inputs are valid.
    if (!string.IsNullOrEmpty(tbxEmailAdd.Text) &&
!string.IsNullOrEmpty(tbxPhoneNum.Text)) // If both fields have inputs
in, both need to be validated
    {
        if (!validEmailAddress(tbxEmailAdd.Text))
        {
            return false;
        }
        if (!validPhoneNumber(tbxPhoneNum.Text))
        {
            return false;
        }
        return true;
    }
    else if (!string.IsNullOrEmpty(tbxEmailAdd.Text))
    {
        if (!validEmailAddress(tbxEmailAdd.Text))
        {
            return false;
        }
        return true;
    }
    else if (!string.IsNullOrEmpty(tbxPhoneNum.Text))
    {
        if (!validPhoneNumber(tbxPhoneNum.Text))
        {
            return false;
        }
        return true;
    }
    else
    {
        // No inputs for both fields
        return false;
    }
}
private bool validEmailAddress(string email)
{
    try
    {
        var mailAddress = new MailAddress(email);
        return true;
    }
    catch

```

```

        {
            MessageBox.Show("Enter a valid email address.", "Invalid
Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
    }
    private bool validPhoneNumber(string phoneNumber)
    {
        // Checks that the phone number entered is of a valid length
        i.e. longer than 7
        // Checks that the phone number entered does not consist of any
        letters
        string validCharacters = @"^[0-9+\s\-\(\)]+$";
        if (phoneNumber.Length > 7)
        {
            // Allows digits, plus signs, spaces, dashes, opening and
            closing parentheses
            if (Regex.Match(phoneNumber, validCharacters).Success)
            {
                return true;
            }
            else
            {
                MessageBox.Show("Enter a valid phone number.", "Invalid
Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
                return false;
            }
            // Suppliers are often international, so validating all
            phone numbers for all international phone numbers would require the use of
            an external API
        }
        else
        {
            MessageBox.Show("Phone number must be longer than 7
characters.", "Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
    }
    private bool validNumber(string str)
    {
        if (string.IsNullOrEmpty(str))
        {
            return true;
        }
        else
        {
            if (!uint.TryParse(str, out uint result)) // Checks if the
            string can be converted into a positive integer
            {
                MessageBox.Show("Enter a valid number.", "Invalid
Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
                return false;
            }
            else
            {
                return true;
            }
        }
    }
}

```

```

    }
}

}

private void tbxEmailAdd_TextChanged(object sender, EventArgs e)
{
    // Checks if there is an existing customer with the contact
    details.

    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = "SELECT * FROM TableCustomers WHERE
EmailAddress = @ea";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@ea", tbxEmailAdd.Text);
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    if (reader.Read())
    {
        existingEmailAddress = true;
        btnAdd.Enabled = false;
        lblExisting.Visible = true;
    }
    else
    {
        existingEmailAddress = false;
        if (!existingPhoneNumber)
        {
            btnAdd.Enabled = true;
            lblExisting.Visible = false;
        }
    }
    connection.Close();
}

private void tbxPhoneNum_TextChanged(object sender, EventArgs e)
{
    // Checks if there is an existing customer with the contact
    details.

    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = "SELECT * FROM TableCustomers WHERE PhoneNum =
@pn";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@pn", tbxPhoneNum.Text);
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    if (reader.Read())
    {

```

```

        existingPhoneNumber = true;
        btnAdd.Enabled = false;
        lblExisting.Visible = true;
    }
    else
    {
        existingPhoneNumber = false;
        if (!existingEmailAddress)
        {
            btnAdd.Enabled = true;
            lblExisting.Visible = false;
        }
    }
    connection.Close();
}
// Save
private void btnAddSave_Click(object sender, EventArgs e)
{
    if(string.IsNullOrEmpty(tbxForename.Text) ||
string.IsNullOrEmpty(tbxSurname.Text) ||
(string.IsNullOrEmpty(tbxEmailAdd.Text) &&
string.IsNullOrEmpty(tbxPhoneNum.Text))) // Checks that name fields are
filled in and one of phone number or email address is filled in
    {
        MessageBox.Show("Enter values in all required fields.",
"Enter All Fields", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    else if (!tbxForename.Text.All(char.IsLetter) ||
!tbxSurname.Text.All(char.IsLetter))
    {
        MessageBox.Show("Input valid details in all fields.",
"Invalid Inputs", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    else if (!validContactInfo() || !validNumber(tbxTotalStars.Text)
|| !validNumber(tbxVouchers.Text) ||
!validName(tbxForename.Text) || !validName(tbxSurname.Text))
    {
        // Message boxes are shown when the subroutines are called
    }
    else
    {
        if (string.IsNullOrEmpty(tbxEmailAdd.Text))
        {
            tbxEmailAdd.Text = "N/A";
        }
        if (string.IsNullOrEmpty(tbxPhoneNum.Text))
        {
            tbxPhoneNum.Text = "N/A";
        }
        if (string.IsNullOrEmpty(tbxTotalStars.Text))
        {
            tbxTotalStars.Text = "0";
        }
        if (string.IsNullOrEmpty(cbxCmemType.Text))
        {
            cbxCmemType.Text = "Bronze"; // Sets the customer's
membership status to bronze by default.

```

```

    }
    if (string.IsNullOrEmpty(tbxVouchers.Text))
    {
        tbxVouchers.Text = "0";
    }

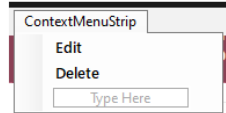
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    connection.Open();
    cmd.CommandText = "INSERT INTO TableCustomers(FirstName,
Surname, PhoneNum, EmailAddress, MemType, TotalStars, VoucherNum)
VALUES (@fn,@sn,@pn,@ea,@mt,@ts,@va)";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@fn", tbxForename.Text);
    cmd.Parameters.AddWithValue("@sn", tbxSurname.Text);
    cmd.Parameters.AddWithValue("@pn", tbxPhoneNum.Text);
    cmd.Parameters.AddWithValue("@ea", tbxEmailAdd.Text);
    cmd.Parameters.AddWithValue("@mt", cbxMemType.Text);
    cmd.Parameters.AddWithValue("@ts", tbxTotalStars.Text);
    cmd.Parameters.AddWithValue("@va", tbxVouchers.Text);
    cmd.Connection = connection;
    int status = cmd.ExecuteNonQuery();
    connection.Close();
    MessageBox.Show("New customer added successfully.", "New
Customer", MessageBoxButtons.OK, MessageBoxIcon.Information);
    Close();
}

}

private void btnClear_Click(object sender, EventArgs e) // Clears
whole form
{
    tbxForename.Text = "";
    tbxEmailAdd.Text = "";
    tbxSurname.Text = "";
    tbxMemType.Text = "";
    tbxPhoneNum.Text = "";
    tbxVouchers.Text = "";
    tbxTotalStars.Text = "";
    cbxMemType.SelectedIndex = 0;
}
}
}

```

Form Design



```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Linq;
using System.Net.Mail;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public partial class Suppliers : Form
    {
        int index;
        Supplier currentSupplier;
        bool editMode = false;

        // Make form draggable
        bool drag = false;
        Point dragCursor;
        Point dragForm;

        public Suppliers()
        {
            Visible = false;
            InitializeComponent();
        }

        // CUSTOM DRAGGABLE FORM BORDER
    }
}
```



```

private void pbxBar_MouseDown(object sender, MouseEventArgs e)
{
    drag = true;
    dragCursor = Cursor.Position;
    dragForm = Location;
}
private void pbxBar_MouseMove(object sender, MouseEventArgs e)
{
    if (drag)
    {
        Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
        Location = Point.Add(dragForm, new Size(difference));
    }
}
private void pbxBar_MouseUp(object sender, MouseEventArgs e)
{
    drag = false;
}
private void btnClose_Click(object sender, EventArgs e)
{
    if (editMode)
    {
        DialogResult result = MessageBox.Show("Are you sure you want
to discard all changes?", "Discard Edits", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
        if (result == DialogResult.Yes)
        {
            Close();
        }
    }
    else
    {
        Close();
    }
}

// VIEW ALL SUPPLIER INFORMATION
private void FormSuppliers_Load(object sender, EventArgs e)
{
    refresh();
    Visible = true;
}
private void refresh()
{
    //ensures textboxes cannot be edited until editmode is true

    disableEditing();
    editMode = false;
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();

```

```

        cmd.CommandText = "SELECT * FROM TableSuppliers WHERE
SupplierName <> 'Deleted' AND EmailAddress <> 'Deleted'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvSuppliers.DataSource = bindingSource;
        connection.Close();

        //ensures there is always something in the textboxes so null
values don't appear when you edit another supplier straight after
refreshSupplierInfo(0);

    }
    private void tbxSearch_TextChanged(object sender, EventArgs e)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableSuppliers WHERE
SupplierName LIKE @sn AND (SupplierName <> 'Deleted' AND EmailAddress <>
'Deleted')";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@sn", "%" + tbxSearch.Text +
"%");

            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                BindingSource bindingSource = new BindingSource();
                bindingSource.DataSource = reader;
                dgvSuppliers.DataSource = bindingSource;
                refreshSupplierInfo(0);
            }
            connection.Close();
        }
        else
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableSuppliers WHERE
(SupplierName <> 'Deleted' AND EmailAddress <> 'Deleted')";
            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            BindingSource bindingSource = new BindingSource();
            bindingSource.DataSource = reader;
            dgvSuppliers.DataSource = bindingSource;
            connection.Close();
        }
    }
}

```

```

        refreshSupplierInfo(0);
    }
}
private void btnSearch_Click(object sender, EventArgs e) // Search
for the supplier by name
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableSuppliers WHERE
SupplierName LIKE @sn AND (SupplierName <> 'Deleted' AND EmailAddress <>
'Deleted')";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@sn", "%" + tbxSearch.Text +
"%");
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new BindingSource();
            bindingSource.DataSource = reader;
            dgvSuppliers.DataSource = bindingSource;
            refreshSupplierInfo(0);
        }
        else
        {
            MessageBox.Show("No suppliers found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
    else
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableSuppliers WHERE
(SupplierName <> 'Deleted' AND EmailAddress <> 'Deleted')";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvSuppliers.DataSource = bindingSource;
        connection.Close();
        refreshSupplierInfo(0);
    }
}
private void dgvSuppliers_CellClick(object sender,
DataGridViewCellEventArgs e)

```

```

    {
        disableEditing();
        editMode = false;
        index = e.RowIndex;
        refreshSupplierInfo(index);
    }
    private void refreshSupplierInfo(int index)
    {
        if (index < 0) // ensures index is non negative
        {
            index = 0;
        }
        DataGridViewRow selectedRow = dgvSuppliers.Rows[index];
        currentSupplier = new
Supplier(Convert.ToInt32(selectedRow.Cells[0].Value),
selectedRow.Cells[1].Value.ToString(),
selectedRow.Cells[2].Value.ToString(),
selectedRow.Cells[3].Value.ToString(),
selectedRow.Cells[4].Value.ToString(),
selectedRow.Cells[5].Value.ToString(),
selectedRow.Cells[6].Value.ToString());
        tbxSupplierID.Text = currentSupplier.id.ToString();
        tbxSupplierName.Text = currentSupplier.supplierName;
        tbxPhoneNum.Text = currentSupplier.phoneNum;
        tbxEmailAdd.Text = currentSupplier.emailAddress;
        tbxPostcode.Text = currentSupplier.postcode;
        tbxAddress.Text = currentSupplier.address;
        tbxURL.Text = currentSupplier.url;
    }

    // ADDING, EDITING AND DELETING
    private void disableEditing()
    {
        btnAddSave.Text = "Add New Supplier";
        tbxSupplierID.ReadOnly = true;
        tbxSupplierName.ReadOnly = true;
        tbxEmailAdd.ReadOnly = true;
        tbxPhoneNum.ReadOnly = true;
        tbxAddress.ReadOnly = true;
        tbxPostcode.ReadOnly = true;
        tbxURL.ReadOnly = true;
        tbxSearch.ReadOnly = false;
        btnCancel.Visible = false;
        tbxSupplierID.BackColor = SystemColors.ControlLightLight;
        tbxSupplierName.BackColor = SystemColors.ControlLightLight;
        tbxEmailAdd.BackColor = SystemColors.ControlLightLight;
        tbxPhoneNum.BackColor = SystemColors.ControlLightLight;
        tbxAddress.BackColor = SystemColors.ControlLightLight;
        tbxPostcode.BackColor = SystemColors.ControlLightLight;
        tbxURL.BackColor = SystemColors.ControlLightLight;
        tbxSupplierID.BorderStyle = BorderStyle.None;
        tbxSupplierName.BorderStyle = BorderStyle.None;
        tbxEmailAdd.BorderStyle = BorderStyle.None;
        tbxPhoneNum.BorderStyle = BorderStyle.None;
        tbxAddress.BorderStyle = BorderStyle.None;
        tbxPostcode.BorderStyle = BorderStyle.None;
        tbxURL.BorderStyle = BorderStyle.None;
    }

```

```

    }
    private void enableEditing()
    {
        tbxSupplierName.BackColor = Color.White;
        tbxEmailAdd.BackColor = Color.White;
        tbxPhoneNum.BackColor = Color.White;
        tbxAddress.BackColor = Color.White;
        tbxPostcode.BackColor = Color.White;
        tbxURL.BackColor = Color.White;
        tbxSupplierName.BorderStyle = BorderStyle.Fixed3D;
        tbxEmailAdd.BorderStyle = BorderStyle.Fixed3D;
        tbxPhoneNum.BorderStyle = BorderStyle.Fixed3D;
        tbxAddress.BorderStyle = BorderStyle.Fixed3D;
        tbxPostcode.BorderStyle = BorderStyle.Fixed3D;
        tbxURL.BorderStyle = BorderStyle.Fixed3D;
        tbxSupplierName.ReadOnly = false;
        tbxEmailAdd.ReadOnly = false;
        tbxPhoneNum.ReadOnly = false;
        tbxAddress.ReadOnly = false;
        tbxPostcode.ReadOnly = false;
        tbxURL.ReadOnly = false;
        tbxSearch.ReadOnly = true;
        btnCancel.Visible = true;
    }
    private void editToolStripMenuItem_Click(object sender, EventArgs e)
    // Edit all supplier details
    {
        //changes the add button to a save button
        editMode = true;
        enableEditing();
        btnAddSave.Text = "Save";
    }
    private void deleteToolStripMenuItem_Click(object sender, EventArgs
e) // Delete supplier data from the system
    {
        // The supplier record is set to a 'deleted state' to maintain
referential integrity
        if (dgvSuppliers.SelectedRows != null)
        {
            //ensures user does not accidentally delete supplier
            DialogResult result = MessageBox.Show("Are you sure you want
to permanently delete this supplier?", "Delete Supplier",
MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
            if (result == DialogResult.Yes)
            {
                bool openOrders = false;
                OleDbConnection con = new OleDbConnection();
                con.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = con.CreateCommand();
                OleDbDataReader reader;
                con.Open();
                cmd.CommandText = "SELECT * FROM TableSupplierOrders
WHERE SupplierID = @sid AND (DeliveryStatus = 'Order Made' OR DeliveryStatus
= 'Out For Delivery')";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@sid", currentSupplier.id);
            }
        }
    }

```

```

        cmd.Connection = con;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            openOrders = true;
        }
        con.Close();
        if (openOrders)
        {
            MessageBox.Show("Review all open orders before
deleting supplier data.", "Open Orders", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
        }
        else
        {
            // Discontinues all products by the supplier
            OleDbConnection conProducts = new OleDbConnection();
            conProducts.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmdProducts =
conProducts.CreateCommand();
            conProducts.Open();
            cmdProducts.CommandText = "UPDATE TableProducts SET
Discontinued = True WHERE SupplierID = @sid";
            cmdProducts.Parameters.Clear();
            cmdProducts.Parameters.AddWithValue("@sid",
currentSupplier.id);

            cmdProducts.Connection = conProducts;
            int status = cmdProducts.ExecuteNonQuery();
            conProducts.Close();

            OleDbConnection conSuppliers = new
OleDbConnection();
            conSuppliers.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmdSuppliers =
conSuppliers.CreateCommand();
            conSuppliers.Open();
            cmdSuppliers.CommandText = "UPDATE TableSuppliers
SET SupplierName = 'Deleted', EmailAddress = 'N/A', PhoneNum = 'N/A',
Address = 'N/A', Postcode = 'N/A', URL = 'N/A' WHERE SupplierID = @sid";
            cmdSuppliers.Parameters.Clear();
            cmdSuppliers.Parameters.AddWithValue("@sid",
currentSupplier.id);

            cmdSuppliers.Connection = conSuppliers;
            int status1 = cmdSuppliers.ExecuteNonQuery();
            conSuppliers.Close();

            MessageBox.Show("Supplier deleted.", "Delete
Supplier", MessageBoxButtons.OK, MessageBoxIcon.Information);
            refresh();
        }
    }
    else
    {
        MessageBox.Show("Supplier not deleted.", "Delete
Supplier", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}

```

```

    }
}
else
{
    MessageBox.Show("Select a supplier to delete.", "Supplier
Not Selected", MessageBoxButtons.OK, MessageBoxIcon.Warning);
}
}
private void btnCancel_Click(object sender, EventArgs e)
{
    int index = dgvSuppliers.SelectedRows[0].Index;
    refresh();
    refreshSupplierInfo(index);
}
// Validation
private bool validString(string text) // Checks if the string is
valid i.e. only contains valid alphanumerical characters and certain special
characters
{
    string allowedCharacters =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 -_.,&/():'";

    // Check each character
    foreach (char c in text)
    {
        bool found = false;
        for (int i = 0; i < allowedCharacters.Length; i++)
        {
            if (c == allowedCharacters[i])
            {
                found = true;
            }
        }
        if (!found)
        {
            MessageBox.Show("Input valid details in all fields.",
"Invalid Inputs", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return found;
        }
    }

    return true;
}

}
private bool validContactInfo()
{
    // Checks if there is at least a phone number or email address
entered.
    // Checks that any inputs are valid.
    if (!string.IsNullOrEmpty(tbxEmailAdd.Text) &&
!string.IsNullOrEmpty(tbxPhoneNum.Text)) // If both fields have inputs
in, both need to be validated
    {
        if (!validEmailAddress(tbxEmailAdd.Text))
        {
            return false;

```

```

        }
        if (!validPhoneNumber(tbxPhoneNum.Text))
        {
            return false;
        }
        return true;
    }
    else if (!string.IsNullOrEmpty(tbxEmailAdd.Text))
    {
        if (!validEmailAddress(tbxEmailAdd.Text))
        {
            return false;
        }
        return true;
    }
    else if (!string.IsNullOrEmpty(tbxPhoneNum.Text))
    {
        if (!validPhoneNumber(tbxPhoneNum.Text))
        {
            return false;
        }
        return true;
    }
    else
    {
        // No inputs for both fields
        return false;
    }
}

private bool validEmailAddress(string email)
{
    try
    {
        var mailAddress = new MailAddress(email);
        return true;
    }
    catch
    {
        MessageBox.Show("Enter a valid email address.", "Invalid
Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
        return false;
    }
}

private bool validPhoneNumber(string phoneNumber)
{
    // Checks that the phone number entered is of a valid length
    i.e. longer than 7
    // Checks that the phone number entered does not consist of any
    letters

    string validCharacters = @"^[0-9+\s\-\(\)]+$";
    if (phoneNumber.Length > 7)
    {
        // Allows digits, plus signs, spaces, dashes, opening and
        closing parentheses
        if (Regex.Match(phoneNumber, validCharacters).Success)
        {
            return true;
        }
    }
}

```



```

    }
    else
    {
        MessageBox.Show("Enter a valid phone number.", "Invalid
Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
        return false;
    }
    // Suppliers are often international, so validating all
phone numbers for all international phone numbers would require the use of
an external API

    }
    else
    {
        MessageBox.Show("Phone number must be longer than 7
characters.", "Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
        return false;
    }
}
private void btnAddSave_Click(object sender, EventArgs e) // Add a
new supplier or save changes made to supplier details
{
    if (!editMode)
    {
        NewSupplier formNewSupplier = new NewSupplier();
        formNewSupplier.ShowDialog();
        refresh();
    }
    else
    {
        if (string.IsNullOrEmpty(tbxSupplierName.Text) ||
(string.IsNullOrEmpty(tbxEmailAdd.Text) &&
string.IsNullOrEmpty(tbxPhoneNum.Text))) // Checks that name fields are
filled in and one of phone number or email address is filled in
        {
            MessageBox.Show("Enter values in all required fields.",
"Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else if(!validContactInfo() ||
!validString(tbxSupplierName.Text) || !validString(tbxAddress.Text) ||
!validString(tbxPostcode.Text))
        {
        }
        else
        {
            if (string.IsNullOrEmpty(tbxEmailAdd.Text))
            {
                tbxEmailAdd.Text = "N/A";
            }
            if (string.IsNullOrEmpty(tbxPhoneNum.Text))
            {
                tbxPhoneNum.Text = "N/A";
            }
            if (string.IsNullOrEmpty(tbxAddress.Text))
            {
                tbxAddress.Text = "N/A";
            }
        }
    }
}

```

```

    }
    if (string.IsNullOrEmpty(tbxPostcode.Text))
    {
        tbxPostcode.Text = "N/A";
    }
    if (string.IsNullOrEmpty(tbxURL.Text))
    {
        tbxURL.Text = "N/A";
    }

    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    connection.Open();
    cmd.CommandText = "UPDATE TableSuppliers SET
SupplierName = @sn, PhoneNum = @pn, EmailAddress = @ea, Address = @ad,
Postcode = @pc, URL = @url WHERE SupplierID = @ssid";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@sn",
tbxSupplierName.Text);
    cmd.Parameters.AddWithValue("@pn", tbxPhoneNum.Text);
    cmd.Parameters.AddWithValue("@ea", tbxEmailAdd.Text);
    cmd.Parameters.AddWithValue("@ad", tbxAddress.Text);
    cmd.Parameters.AddWithValue("@pc", tbxPostcode.Text);
    cmd.Parameters.AddWithValue("@url", tbxURL.Text);
    cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);
    cmd.Connection = connection;
    int status = cmd.ExecuteNonQuery();
    connection.Close();
    MessageBox.Show("Supplier edited successfully.", "Edit
Supplier", MessageBoxButtons.OK, MessageBoxIcon.Information);

    editMode = false;
    int index = dgvSuppliers.SelectedRows[0].Index;
    refresh();
    refreshSupplierInfo(index);
    dgvSuppliers.ClearSelection();
    dgvSuppliers.CurrentCell =
dgvSuppliers.Rows[0].Cells[0];
    dgvSuppliers.Rows[index].Selected = true;
    }
    }
    }
}

```

NewSupplier.cs

Form Design

The image shows a Windows Form titled "NEW SUPPLIER" with a dark red header bar containing a close button (X). The form has six text input fields arranged in two columns. Handwritten annotations with arrows point to each field: "tbxSupplierName" for the first field, "tbxAddress" for the second, "tbxEmailAdd" for the third, "tbxPostcode" for the fourth, "tbxPhoneNum" for the fifth, and "tbxURL" for the sixth. At the bottom right, there are two buttons: "CLEAR" and "ADD". Handwritten annotations point to these buttons: "btnClear -> Clears all details entered" for the "CLEAR" button and "btnAdd -> Adds supplier to database" for the "ADD" button.

Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Linq;
using System.Net.Mail;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public partial class NewSupplier : Form
    {
        // Make form draggable
        bool drag = false;
        Point dragCursor;
        Point dragForm;

        // CUSTOM DRAGGABLE FORM BORDER
        private void pbxBar_MouseDown(object sender, MouseEventArgs e)
        {
            drag = true;
            dragCursor = Cursor.Position;
            dragForm = Location;
        }
        private void pbxBar_MouseMove(object sender, MouseEventArgs e)
        {
            if (drag)
            {

```

```

        Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
        Location = Point.Add(dragForm, new Size(difference));
    }
}
private void pbxBar_MouseUp(object sender, MouseEventArgs e)
{
    drag = false;
}
private void btnClose_Click(object sender, EventArgs e)
{
    DialogResult result = MessageBox.Show("Are you sure you want to
discard all changes?", "Discard Creator", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
    if (result == DialogResult.Yes)
    {
        Close();
    }
}
public NewSupplier()
{
    Visible = false;
    InitializeComponent();
}

// CREATE SUPPLIER
private void FormNewSupplier_Load(object sender, EventArgs e)
{
    Visible = true;
}
private bool validString(string text) // Checks if the string is
valid i.e. only contains valid alphanumerical characters and certain special
characters
{
    string allowedCharacters =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 -_., &/():'";

    // Check each character
    foreach (char c in text)
    {
        bool found = false;
        for (int i = 0; i < allowedCharacters.Length; i++)
        {
            if (c == allowedCharacters[i])
            {
                found = true;
            }
        }
        if (!found)
        {
            MessageBox.Show("Input valid details in all fields.",
"Invalid Inputs", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return found;
        }
    }
}

```

```

        return true;
    }
    private bool validContactInfo()
    {
        // Checks if there is at least a phone number or email address
        entered.
        // Checks that any inputs are valid.
        if (!string.IsNullOrEmpty(tbxEmailAdd.Text) &&
            !string.IsNullOrEmpty(tbxPhoneNum.Text)) // If both fields have inputs
            in, both need to be validated
        {
            if (!validEmailAddress(tbxEmailAdd.Text))
            {
                return false;
            }
            if (!validPhoneNumber(tbxPhoneNum.Text))
            {
                return false;
            }
            return true;
        }
        else if (!string.IsNullOrEmpty(tbxEmailAdd.Text))
        {
            if (!validEmailAddress(tbxEmailAdd.Text))
            {
                return false;
            }
            return true;
        }
        else if (!string.IsNullOrEmpty(tbxPhoneNum.Text))
        {
            if (!validPhoneNumber(tbxPhoneNum.Text))
            {
                return false;
            }
            return true;
        }
        else
        {
            // No inputs for both fields
            return false;
        }
    }
    private bool validEmailAddress(string email)
    {
        try
        {
            var mailAddress = new MailAddress(email);
            return true;
        }
        catch
        {
            MessageBox.Show("Enter a valid email address.", "Invalid
            Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
    }

```

```

    }
    private bool validPhoneNumber(string phoneNumber)
    {
        // Checks that the phone number entered is of a valid length
        i.e. longer than 7
        // Checks that the phone number entered does not consist of any
        letters
        string validCharacters = @"^[0-9+\s\-\(\)]+$";
        if (phoneNumber.Length > 7)
        {
            // Allows digits, plus signs, spaces, dashes, opening and
            closing parentheses
            if (Regex.Match(phoneNumber, validCharacters).Success)
            {
                return true;
            }
            else
            {
                MessageBox.Show("Enter a valid phone number.", "Invalid
                Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
                return false;
            }
            // Suppliers are often international, so validating all
            phone numbers for all international phone numbers would require the use of
            an external API
        }
        else
        {
            MessageBox.Show("Phone number must be longer than 7
            characters.", "Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
    }
    private void btnAddSave_Click(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(tbxSupplierName.Text) ||
        (string.IsNullOrEmpty(tbxEmailAdd.Text) &&
        string.IsNullOrEmpty(tbxPhoneNum.Text))) // Checks that name fields are
        filled in and one of phone number or email address is filled in
        {
            MessageBox.Show("Enter values in all required fields.",
            "Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else if (!validContactInfo() ||
        !validString(tbxSupplierName.Text) || !validString(tbxAddress.Text) ||
        !validString(tbxPostcode.Text))
        {
        }
        else
        {
            if (string.IsNullOrEmpty(tbxEmailAdd.Text))
            {
                tbxEmailAdd.Text = "N/A";
            }
            if (string.IsNullOrEmpty(tbxPhoneNum.Text))

```

```

        {
            tbxPhoneNum.Text = "N/A";
        }
        if (string.IsNullOrEmpty(tbxAddress.Text))
        {
            tbxAddress.Text = "N/A";
        }
        if (string.IsNullOrEmpty(tbxPostcode.Text))
        {
            tbxPostcode.Text = "N/A";
        }
        if (string.IsNullOrEmpty(tbxURL.Text))
        {
            tbxURL.Text = "N/A";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        connection.Open();
        cmd.CommandText = "INSERT INTO TableSuppliers(SupplierName,
PhoneNum, EmailAddress, Address, Postcode, URL)
VALUES (@sn,@pn,@ea,@ad,@pc,@url)";
        cmd.Parameters.AddWithValue("@sn", tbxSupplierName.Text);
        cmd.Parameters.AddWithValue("@pn", tbxPhoneNum.Text);
        cmd.Parameters.AddWithValue("@ea", tbxEmailAdd.Text);
        cmd.Parameters.AddWithValue("@ad", tbxAddress.Text);
        cmd.Parameters.AddWithValue("@pc", tbxPostcode.Text);
        cmd.Parameters.AddWithValue("@url", tbxURL.Text);
        cmd.Connection = connection;
        int status = cmd.ExecuteNonQuery();
        connection.Close();
        MessageBox.Show("New supplier added successfully.", "New
Supplier", MessageBoxButtons.OK, MessageBoxIcon.Information);
        Close();
    }

}

private void btnClear_Click(object sender, EventArgs e)
{
    {
        tbxAddress.Text = "";
        tbxEmailAdd.Text = "";
        tbxPhoneNum.Text = "";
        tbxPostcode.Text = "";
        tbxSupplierName.Text = "";
        tbxURL.Text = "";
    }
}
}

```

Products.cs

Form Design

Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.ComponentModel.Design;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.IO;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Net.Mime.MediaTypeNames;
using static System.Net.WebRequestMethods;
using static
System.Windows.Forms.VisualStyles.VisualStyleElement.ProgressBar;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.Window;

namespace YarnShop
{
    public partial class Products : Form
    {
        // Filter lists
        List<Supplier> supplierList = new List<Supplier>();
        List<string> YarnWeightList = new List<string>();
        List<string> CategoryList = new List<string>();
        List<string> FibreList = new List<string>();
        List<string> MaterialList = new List<string>();
    }
}
```



```

        List<string> ColourList = new List<string>();
        bool filtersApplied; // True if filters have been applied, used in
searches
        string currentCommand; // Stores the command used to retrieve the
data currently shown in the dgv

        int index; //index used for finding row of information about a
product
        Product currentProduct = null; // Stores the currently selected
product
        Supplier currentSupplier = null; // Stores the supplier of the
currently selected product
        string tempProductType; // Stores the type of product that is
currently selected or is to be shown
        bool editMode = false; // True when editing a product

        // Used to upload, view, edit and remove the image of the product
being edited
        OpenFileDialog dialog;
        FileInfo path = null;
        System.Drawing.Image currentImage = null;
        string currentProductImagePath;

        // Make form draggable
        bool drag = false;
        Point dragCursor;
        Point dragForm;

        public Products(Product noticeProduct)
        {
            InitializeComponent();
            currentProduct = noticeProduct;
        }

        // CUSTOM DRAGGABLE FORM BORDER
        private void pbxBar_MouseDown(object sender, MouseEventArgs e)
        {
            drag = true;
            dragCursor = Cursor.Position;
            dragForm = Location;
        }
        private void pbxBar_MouseMove(object sender, MouseEventArgs e)
        {
            if (drag)
            {
                Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
                Location = Point.Add(dragForm, new Size(difference));
            }
        }
        private void pbxBar_MouseUp(object sender, MouseEventArgs e)
        {
            drag = false;
        }
        private void btnClose_Click(object sender, EventArgs e)

```

```

    {
        if (editMode)
        {
            DialogResult result = MessageBox.Show("Are you sure you want
to discard all changes?", "Discard Edits", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
            if (result == DialogResult.Yes)
            {
                Close();
            }
        }
        else
        {
            Close();
        }
    }

    // VIEW ALL PRODUCTS
    private void FormProducts_Load(object sender, EventArgs e)
    {
        //load all yarn weights into a list
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;

        connection.Open();
        cmd.CommandText = "SELECT DISTINCT Category FROM TableProducts
WHERE ProductType = 'y'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
            {
                YarnWeightList.Add(Convert.ToString(reader[0]));
            }
            else
            {
                continue;
            }
        }
        connection.Close();

        //load all accessory categories into a list
        connection.Open();
        cmd.CommandText = "SELECT DISTINCT Category FROM TableProducts
WHERE ProductType = 'a'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
            {

```

```

        CategoryList.Add(Convert.ToString(reader[0]));
    }
    else
    {
        continue;
    }
}
connection.Close();

//load all yarn fibres into a list
connection.Open();
cmd.CommandText = "SELECT DISTINCT FibreMaterial FROM
TableProducts WHERE ProductType = 'y'";
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
    {
        FibreList.Add(Convert.ToString(reader[0]));
    }
    else
    {
        continue;
    }
}
connection.Close();

//load all accessory materials into a list
connection.Open();
cmd.CommandText = "SELECT DISTINCT FibreMaterial FROM
TableProducts WHERE ProductType = 'a'";
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
    {
        MaterialList.Add(Convert.ToString(reader[0]));
    }
    else
    {
        continue;
    }
}
connection.Close();

//load all colours into a list
connection.Open();
cmd.CommandText = "SELECT DISTINCT Colour FROM TableProducts";
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{

```

```

        if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
        ""))
        {
            ColourList.Add(Convert.ToString(reader[0]));
        }
        else
        {
            continue;
        }
    }
    connection.Close();

    //loads suppliers into the combo box
    //should be done in refresh so it can update anything changed
    OleDbConnection conSuppliers = new OleDbConnection();
    conSuppliers.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmdSuppliers = conSuppliers.CreateCommand();
    OleDbDataReader readerSuppliers;
    conSuppliers.Open();
    cmdSuppliers.CommandText = "SELECT * FROM TableSuppliers WHERE
SupplierName <> 'Deleted'";
    cmdSuppliers.Connection = conSuppliers;
    readerSuppliers = cmdSuppliers.ExecuteReader();
    while (readerSuppliers.Read())
    {
        //instantiates new supplier every time the loop iterates
        supplierList.Add(new
Supplier(Convert.ToInt32(readerSuppliers[0]),
Convert.ToString(readerSuppliers[1]), Convert.ToString(readerSuppliers[2]),
Convert.ToString(readerSuppliers[3]), Convert.ToString(readerSuppliers[4]),
Convert.ToString(readerSuppliers[5]),
Convert.ToString(readerSuppliers[6])));
    }
    cbxSuppliers.DataSource = supplierList;
    cbxSuppliers.DisplayMember = "Suppliers";
    cbxSuppliers.ValueMember = "supplierName";
    tbxSupplier.Text = "supplierName";
    conSuppliers.Close();

    refresh();
}
private void refresh()
{
    disableEditing();
    filtersApplied = false;
    rbtnAll.Checked = true;
    // Cannot filter products until either yarn or accessories is
selected
    // This is because the filters for yarn and accessories are
different

    clearChecked(clsbCategory);
    clearChecked(clsbFibreMaterial);
    clearChecked(clsbColour);
    clearChecked(clsbSuppliers);
}

```

```

clearChecked(clsbStock);

chbxDiscontinuedFilter.Checked = false;
pnlFilters.AutoScroll = false;
clsbCategory.Visible = false;
clsbFibreMaterial.Visible = false;
clsbColour.Visible = false;

clsbSuppliers.DataSource = supplierList;
clsbSuppliers.DisplayMember = "Suppliers";
clsbSuppliers.ValueMember = "supplierName";

refreshProductDGV();
if (currentProduct != null) // displays the product passed into
this form by notices
{
    foreach (DataGridViewRow row in dgvProducts.Rows)
    {
        if (row.Cells[0].Value.ToString() ==
currentProduct.id.ToString())
        {
            dgvProducts.ClearSelection();
            dgvProducts.CurrentCell =
dgvProducts.Rows[row.Index].Cells[0];
            dgvProducts[0, row.Index].Selected = true;
        }
    }
    refreshProductInfo(dgvProducts.CurrentRow.Index);
}
else
{
    //ensures there is always something in the textboxes so null
values don't appear when you edit another customer straight after
    refreshProductInfo(0);
}
}

private void tbxSearch_TextChanged(object sender, EventArgs e) // As
the user types in the search bar, products are searched for according to the
input
{
    // Search applies to all currently shown products, even if
filters are applied
    if (!filtersApplied)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            if (rbtnAll.Checked)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductName LIKE @pn AND Discontinued = False";
                cmd.Parameters.Clear();
            }
        }
    }
}

```

```

cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
cmd.Connection = connection;
reader = cmd.ExecuteReader();
if (reader.HasRows)
{
    BindingSource bindingSource = new
BindingSource();

    bindingSource.DataSource = reader;
    dgvProducts.DataSource = bindingSource;
}
connection.Close();
}
else
{
    if (rbtnYarn.Checked)
    {
        tempProductType = "y";
    }
    if (rbtnAcc.Checked)
    {
        tempProductType = "a";
    }
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND ProductName LIKE @pn AND Discontinued = False";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@ya", tempProductType);
    cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
cmd.Connection = connection;
reader = cmd.ExecuteReader();
if (reader.HasRows)
{
    BindingSource bindingSource = new
BindingSource();

    bindingSource.DataSource = reader;
    dgvProducts.DataSource = bindingSource;
}
connection.Close();
}
}
else
{
    if (rbtnAll.Checked)
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;

```

```

        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
Discontinued = False";
        cmd.Parameters.Clear();
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
    else
    {
        if (rbtnYarn.Checked)
        {
            tempProductType = "y";
        }
        if (rbtnAcc.Checked)
        {
            tempProductType = "a";
        }

        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND Discontinued = False";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
}
else
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        if (rbtnAll.Checked)
        {
            OleDbConnection connection = new OleDbConnection();

```

```

        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
    else
    {
        if (rbtnYarn.Checked)
        {
            tempProductType = "y";
        }
        if (rbtnAcc.Checked)
        {
            tempProductType = "a";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
}
else
{

```



```

        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand;
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        if (reader.HasRows)
        {
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
}
highlightProductsStock();
}
private void btnSearch_Click(object sender, EventArgs e) // Searches
for products when the user clicks the button
{
    if (!filtersApplied)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            if (rbtnAll.Checked)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductName LIKE @pn AND Discontinued = False";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
                cmd.Connection = connection;
                reader = cmd.ExecuteReader();
                if (reader.HasRows)
                {
                    BindingSource bindingSource = new
BindingSource();
                    bindingSource.DataSource = reader;
                    dgvProducts.DataSource = bindingSource;
                }
                else
                {
                    MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
                }
            }

```

```

        connection.Close();
    }
    else
    {
        if (rbtnYarn.Checked)
        {
            tempProductType = "y";
        }
        if (rbtnAcc.Checked)
        {
            tempProductType = "a";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND ProductName LIKE @pn AND Discontinued = False";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
}
else
{
    if (rbtnAll.Checked)
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
Discontinued = False";
        cmd.Parameters.Clear();
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)

```

```

        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
    else
    {
        if (rbtnYarn.Checked)
        {
            tempProductType = "y";
        }
        if (rbtnAcc.Checked)
        {
            tempProductType = "a";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND Discontinued = False";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
}
}
else
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        if (rbtnAll.Checked)
        {

```

```

OleDbConnection connection = new OleDbConnection();
connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
OleDbCommand cmd = connection.CreateCommand();
OleDbDataReader reader;
connection.Open();
cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";

cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

cmd.Connection = connection;
reader = cmd.ExecuteReader();
if (reader.HasRows)
{
    BindingSource bindingSource = new

    bindingSource.DataSource = reader;
    dgvProducts.DataSource = bindingSource;
}
else
{
    MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
connection.Close();
}
else
{
    if (rbtnYarn.Checked)
    {
        tempProductType = "y";
    }
    if (rbtnAcc.Checked)
    {
        tempProductType = "a";
    }
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = currentCommand + " AND ProductName
    LIKE @pn";

    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@ya", tempProductType);
    cmd.Parameters.AddWithValue("@pn", "%" +
    tbxSearch.Text + "%");

    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    if (reader.HasRows)
    {
        BindingSource bindingSource = new

        bindingSource.DataSource = reader;
        dgvProducts.DataSource = bindingSource;
    }
}

```

```

        }
        else
        {
            MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
}
else
{
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = currentCommand;
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@ya", tempProductType);
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    BindingSource bindingSource = new BindingSource();
    if (reader.HasRows)
    {
        bindingSource.DataSource = reader;
        dgvProducts.DataSource = bindingSource;
    }
    else
    {
        MessageBox.Show("No products found.", "No Results",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    connection.Close();
}
}
highlightProductsStock();
}
private void dgvProducts_CellClick(object sender,
DataGridViewCellEventArgs e) // Allows the user to click on a product and
view all product details for that product
{
    disableEditing();
    editMode = false;
    index = e.RowIndex;
    if (index < 0)
    {
        index = 0;
    }
    refreshProductInfo(index);
}
private void refreshProductInfo(int index) // Show all product
details
{
    if (index < 0)
    {
        index = 0;
    }
}

```

```

    }
    DataGridViewRow selectedRow = dgvProducts.Rows[index];
    currentProduct = new
Product(Convert.ToInt32(selectedRow.Cells[0].Value),
selectedRow.Cells[1].Value.ToString(),
Convert.ToSingle(selectedRow.Cells[2].Value),
Convert.ToInt32(selectedRow.Cells[3].Value),
selectedRow.Cells[10].Value.ToString(),
Convert.ToInt32(selectedRow.Cells[4].Value),
Convert.ToBoolean(selectedRow.Cells[11].Value),
Convert.ToInt32(selectedRow.Cells[12].Value),
Convert.ToInt32(selectedRow.Cells[13].Value));
    currentSupplier = currentProduct.Supplier;
    tbxProductID.Text = currentProduct.id.ToString();
    tbxProductName.Text = currentProduct.name;
    tbxCost.Text = currentProduct.CostString;
    tbxStock.Text = currentProduct.stock.ToString();
    tbxSupplier.Text = currentSupplier.supplierName;
    tbxType.Text = selectedRow.Cells[5].Value.ToString();
    tbxCategory.Text = selectedRow.Cells[6].Value.ToString();
    tbxColour.Text = selectedRow.Cells[7].Value.ToString();
    tbxFibre.Text = selectedRow.Cells[8].Value.ToString();
    tempProductType = selectedRow.Cells[9].Value.ToString();
    currentProductImagePath = Path.GetFullPath("productImages/" +
currentProduct.filename);
    tbxFileName.Text = currentProduct.filename;
    numStock.Value = currentProduct.stock;
    numFullStock.Value = currentProduct.fullStock;
    numLowStock.Value = currentProduct.lowStock;
    pbxImage.Refresh();
    if (currentProduct.discontinued)
    {
        lblDiscontinued.Visible = true;
        editToolStripMenuItem.Visible = false;
        discontinueToolStripMenuItem.Visible = false;
    }
    else
    {
        lblDiscontinued.Visible = false;
        editToolStripMenuItem.Visible = true;
        discontinueToolStripMenuItem.Visible = true;
    }

    if (currentProduct.OutOfStock)
    {
        lblStockLevel.Visible = true;
        lblStockLevel.Text = "Out Of Stock";
    }
    else if (currentProduct.IsLowStock)
    {
        lblStockLevel.Visible = true;
        lblStockLevel.Text = "Low Stock";
    }
    else
    {
        lblStockLevel.Visible = false;
    }
}

```

```

        if (tempProductType == "y")
        {
            showYarnFields(false);
        }
        if (tempProductType == "a")
        {
            showAccFields(false);
        }
    }
    private void refreshProductDGV() // Refresh all products
    {
        //this is a separate subroutine because while making an order,
        new products may be added, so this subroutine is required to update the
        // dgv of products each time a new product is added
        //loads up all the data into the dgv
        tbxSearch.Text = "";
        string type = "";
        if (rbtnAcc.Checked)
        {
            type = "a";
        }
        if (rbtnYarn.Checked)
        {
            type = "y";
        }
        if (rbtnAll.Checked)
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableProducts WHERE
Discontinued = False";
            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            BindingSource bindingSource = new BindingSource();
            if (reader.HasRows)
            {
                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
            }
            else
            {
                MessageBox.Show("No results found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            connection.Close();
        }
        else
        {
            OleDbConnection connection = new OleDbConnection();

```

```

        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND Discontinued = False";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", type);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        if (reader.HasRows)
        {
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No results found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
    highlightProductsStock();
}
private void highlightProductsStock() // Highlights which products
are out of stock or low on stock in the dgv
{
    foreach (DataGridViewRow row in dgvProducts.Rows)
    {
        if (Convert.ToInt32(row.Cells[3].Value) == 0)
        {
            row.DefaultCellStyle.BackColor = Color.DarkRed;
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlLightLight;
        }
        else if (Convert.ToInt32(row.Cells[3].Value) <
Convert.ToInt32(row.Cells[12].Value))
        {
            row.DefaultCellStyle.BackColor = Color.FromArgb(199,
110, 0);
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlLightLight;
        }
        else
        {
            row.DefaultCellStyle.BackColor =
SystemColors.ControlLightLight;
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlText;
        }
    }
    // Highlights products in the dgv that have low stock or no
stock.
}

```



```

// FILTERS
private void showYarnFields(bool filter) // Changes fields shown to
suit yarn products.
{
    if (filter)
    {
        clsbCategory.Visible = true;
        clsbFibreMaterial.Visible = true;
        clsbColour.Visible = true;

        clsbCategory.DataSource = YarnWeightList;
        clsbFibreMaterial.DataSource = FibreList;
        clsbColour.DataSource = ColourList;

        lblCategoryFilter.Text = "YARN WEIGHT";
        lblFibreMaterialFilter.Text = "FIBRE";
        lblCategoryFilter.Visible = true;
        lblFibreMaterialFilter.Visible = true;
        lblColourFilter.Visible = true;

        clearChecked(clsbCategory);
        clearChecked(clsbFibreMaterial);
        clearChecked(clsbColour);
        clearChecked(clsbSuppliers);

        chbxDiscontinuedFilter.Checked = false;

        pnlFilters.AutoScroll = true;
    }

    lblType.Text = "Ball Weight";
    lblCategory.Text = "Yarn Weight";
    lblFibre.Text = "Fibre";

    cbxCategory.DataSource = YarnWeightList;
    cbxFibre.DataSource = FibreList;
    cbxColour.DataSource = ColourList;
}

private void showAccFields(bool filter) // Changes fields shown to
suit accessory products.
{
    if (filter)
    {
        clsbCategory.Visible = true;
        clsbFibreMaterial.Visible = true;
        clsbColour.Visible = true;

        lblCategoryFilter.Text = "CATEGORY";
        lblFibreMaterialFilter.Text = "MATERIAL";
        lblCategoryFilter.Visible = true;
        lblFibreMaterialFilter.Visible = true;
        lblColourFilter.Visible = true;

        clsbCategory.DataSource = CategoryList;
        clsbFibreMaterial.DataSource = MaterialList;
    }
}

```

```

        clsbColour.DataSource = ColourList;

        clearChecked(clsbCategory);
        clearChecked(clsbFibreMaterial);
        clearChecked(clsbColour);
        clearChecked(clsbSuppliers);

        chbxDiscontinuedFilter.Checked = false;

        pnlFilters.AutoScroll = true;
    }

    lblType.Text = "Size";
    lblCategory.Text = "Category";
    lblFibre.Text = "Material";

    cbxCategory.DataSource = CategoryList;
    cbxFibre.DataSource = MaterialList;
    cbxColour.DataSource = ColourList;

}
private void clearChecked(CheckedListBox clsbToClear) // Clears all
checked items in the checked list box.
{
    for (int i = 0; i < clsbToClear.Items.Count; i++)
    {
        clsbToClear.SetSelected(i, false);
        clsbToClear.SetItemChecked(i, false);
    }
    clsbToClear.ClearSelected();
}
private void rbtnYarn_CheckedChanged(object sender, EventArgs e) //
Shows all yarn products
{
    if (rbtnYarn.Checked == true)
    {
        showYarnFields(true);
        disableEditing();
        refreshProductDGV();
        refreshProductInfo(0);
    }
}
private void rbtnAcc_CheckedChanged(object sender, EventArgs e) //
Shows all accessory products
{
    if (rbtnAcc.Checked == true)
    {
        showAccFields(true);
        disableEditing();
        refreshProductDGV();
        refreshProductInfo(0);
    }
}
private void rbtnAll_Click(object sender, EventArgs e) // Shows all
products and clears all filters
{

```

```

        // If there are any other filters selected, clicking the all
radio button will clear all filters selected.
        // A message box will be shown if any filters apart from Yarn
and Accessories are selected.
        DialogResult result = MessageBox.Show("This will clear all
filters. Proceed?", "Clear Filters", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
        tbxSearch.Text = "";
        if (result == DialogResult.Yes)
        {
            refresh();
        }
        else
        {
            MessageBox.Show("No filters were cleared.", "Clear
Filters", MessageBoxButtons.OK, MessageBoxIcon.Information);
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND Discontinued = False";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@ya", tempProductType);
            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows) //in case all accessories or yarn
products have been deleted
            {
                BindingSource bindingSource = new BindingSource();
                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
                if (tempProductType == "y")
                {
                    rbtnYarn.Checked = true;
                    showYarnFields(false);
                }
                if (tempProductType == "a")
                {
                    rbtnAcc.Checked = true;
                    showAccFields(false);
                }
            }
            else
            {
                MessageBox.Show("No results found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
                rbtnAll.Checked = true;
                rbtnYarn.Checked = false;
            }
            connection.Close();
        }
        highlightProductsStock();
    }
}

```

```

        private void btnApplyFilters_Click(object sender, EventArgs e) //
Apply filters selected
        {
            filtersApplied = false;
            string baseCommandText = "SELECT * FROM TableProducts WHERE";
            string categoryFilterCommand = " AND Category IN (";
            string fibreFilterCommand = " AND FibreMaterial IN (";
            string colourFilterCommand = " AND Colour IN (";
            string supplierFilterCommand = " AND SupplierID IN (";

            // Need these lists to store the selected items for each checked
list box.
            // Since a new list is created each time the user clicks the
apply button, the list will be cleared each time.

            List<string> selectedCategoryFilters = new List<string>();
            List<string> selectedFibreFilters = new List<string>();
            List<string> selectedColourFilters = new List<string>();
            List<string> selectedStockFilters = new List<string>();

            // Load all the checked filters into their corresponding lists.

            selectedCategoryFilters =
clsbCategory.CheckedItems.Cast<string>().ToList();
            selectedFibreFilters =
clsbFibreMaterial.CheckedItems.Cast<string>().ToList();
            selectedColourFilters =
clsbColour.CheckedItems.Cast<string>().ToList();
            selectedStockFilters =
clsbStock.CheckedItems.Cast<string>().ToList();
            List<Supplier> selectedSupplierFilters =
clsbSuppliers.CheckedItems.Cast<Supplier>().ToList();
            if (selectedStockFilters.Count != 0)
            {
                filtersApplied = true;
                foreach (string stockFilter in selectedStockFilters)
                {
                    string condition = "";
                    switch (stockFilter)
                    {
                        case "In Stock":
                            condition = " Stock > 0";
                            break;
                        case "No Stock":
                            condition = " Stock = 0";
                            break;
                        case "Low Stock":
                            condition = " Stock < LowStock";
                            break;
                    }
                    if (baseCommandText.EndsWith("E"))
                    {
                        baseCommandText += " (" + condition;
                    }
                    else
                    {
                        baseCommandText += " OR" + condition;
                    }
                }
            }
        }

```

```

    }
    }
    baseCommandText += ")";
}
else
{
    baseCommandText += " Stock >= 0"; // If no specific stock
filters are selected, results will only include products in stock.
}
if (!rbtnAll.Checked)
{
    baseCommandText += " AND ProductType = @ya";
}
// Building the command string

if (selectedCategoryFilters.Count != 0)
{
    baseCommandText +=
appendFilterCommand(categoryFilterCommand,selectedCategoryFilters) + ")";
}
if (selectedFibreFilters.Count != 0)
{
    baseCommandText += appendFilterCommand(fibreFilterCommand,
selectedFibreFilters) + ")";
}
if (selectedColourFilters.Count != 0)
{
    baseCommandText += appendFilterCommand(colourFilterCommand,
selectedColourFilters) + ")";
}
if (selectedSupplierFilters.Count != 0)
{
    filtersApplied = true;
    foreach (Supplier supplierFilter in selectedSupplierFilters)
    {
        // If there isn't anything in the string so far, don't
want it to start with a comma.
        if (supplierFilterCommand.EndsWith("("))
        {
            supplierFilterCommand += supplierFilter.id;
        }
        // Add the checked filter to the command string.
        else
        {
            supplierFilterCommand += "," + supplierFilter.id;
        }
    }
    baseCommandText += supplierFilterCommand + ")";
}
if (chbxDiscontinuedFilter.Checked)
{
    filtersApplied = true;
    baseCommandText += " AND Discontinued = True";
}
else
{
    baseCommandText += " AND Discontinued = False";
}

```

```

    }

    // Running the queries
    if (rbtnYarn.Checked)
    {
        tempProductType = "y";
    }
    if (rbtnAcc.Checked)
    {
        tempProductType = "a";
    }

    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = baseCommandText;
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@ya", tempProductType);
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    BindingSource bindingSource = new BindingSource();
    if (reader.HasRows)
    {
        bindingSource.DataSource = reader;
        dgvProducts.DataSource = bindingSource;
        currentCommand = baseCommandText;
        tbxSearch.Text = "";
    }
    else
    {
        MessageBox.Show("No products found.", "No Results",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    connection.Close();
    highlightProductsStock();
}

private string appendFilterCommand(string filterCommand,
List<string> selectedFilters)
{
    filtersApplied = true;
    foreach (string filter in selectedFilters)
    {
        // If there isn't anything in the string so far, don't want
it to start with a comma.
        if (filterCommand.EndsWith("("))
        {
            filterCommand += "'" + filter + "'";
        }
        // Add the checked filter to the command string.
        else
        {
            filterCommand += "," + "'" + filter + "'";
        }
    }
}

```

```

        return filterCommand;
    }
    private void btnClearFilters_Click(object sender, EventArgs e) //
Clears all filters selected
    {
        DialogResult result = MessageBox.Show("This will clear all
filters. Proceed?", "Clear All Filters", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
        if (result == DialogResult.Yes)
        {
            clearChecked(clsbCategory);
            clearChecked(clsbFibreMaterial);
            clearChecked(clsbColour);
            clearChecked(clsbSuppliers);
            clearChecked(clsbStock);
            chbxDiscontinuedFilter.Checked = false;

            if (rbtnAll.Checked)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableProducts WHERE
Discontinued = False";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@ya", "y");
                cmd.Connection = connection;
                reader = cmd.ExecuteReader();
                BindingSource bindingSource = new BindingSource();
                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
                connection.Close();
            }
            else
            {
                if (rbtnYarn.Checked)
                {
                    tempProductType = "y";
                }
                if (rbtnAcc.Checked)
                {
                    tempProductType = "a";
                }
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableProducts WHERE
Discontinued = False AND ProductType = @ya";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@ya", tempProductType);
                cmd.Connection = connection;
            }
        }
    }

```

```

        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvProducts.DataSource = bindingSource;
        connection.Close();
    }
    filtersApplied = false;
    currentCommand = "";
}
else
{
    MessageBox.Show("No filters were deleted.", "Clear All
Filters", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
highlightProductsStock();
}

// ADDING, EDITING AND DISCONTINUING PRODUCTS
private void disableEditing() // Disables editing of all fields
{
    editMode = false;
    btnAddSave.Text = "Add New Product";

    //text boxes are all read only when not in edit mode

    lblLowStock.Visible = false;
    lblFullStock.Visible = false;
    numFullStock.Visible = false;
    numLowStock.Visible = false;
    numStock.Visible = false;
    tbxStock.Visible = true;

    tbxProductID.ReadOnly = true;
    tbxProductName.ReadOnly = true;
    tbxCost.ReadOnly = true;
    tbxStock.ReadOnly = true;
    tbxSupplier.ReadOnly = true;

    tbxType.ReadOnly = true;
    tbxCategory.ReadOnly = true;
    tbxColour.ReadOnly = true;
    tbxFibre.ReadOnly = true;

    tbxSupplier.Visible = true;
    tbxCategory.Visible = true;
    tbxColour.Visible = true;
    tbxFibre.Visible = true;

    cbxSuppliers.Visible = false;
    cbxCategory.Visible = false;
    cbxColour.Visible = false;
    cbxFibre.Visible = false;

    btnApplyFilters.Visible = true;
    btnClearFilters.Visible = true;
    btnCancel.Visible = false;
    btnUpload.Visible = false;

```



```

        btnRemove.Visible = false;
        btnReset.Visible = false;
        tbxFileName.Visible = false;
        lblSaveAs.Visible = false;

        pbxHideFilter.Visible = false;

        //change design to make it clear it is not editable
        tbxProductID.BackColor = SystemColors.ControlLightLight;
        tbxProductName.BackColor = SystemColors.ControlLightLight;
        tbxCost.BackColor = SystemColors.ControlLightLight;
        tbxStock.BackColor = SystemColors.ControlLightLight;
        tbxSupplier.BackColor = SystemColors.ControlLightLight;
        tbxType.BackColor = SystemColors.ControlLightLight;
        cbxSuppliers.BackColor = SystemColors.ControlLightLight;
        cbxCategory.BackColor = SystemColors.ControlLightLight;
        cbxColour.BackColor = SystemColors.ControlLightLight;
        cbxFibre.BackColor = SystemColors.ControlLightLight;

        tbxProductID.BorderStyle = BorderStyle.None;
        tbxProductName.BorderStyle = BorderStyle.None;
        tbxCost.BorderStyle = BorderStyle.None;
        tbxStock.BorderStyle = BorderStyle.None;
        tbxSupplier.BorderStyle = BorderStyle.None;
        tbxType.BorderStyle = BorderStyle.None;
    }
    private void enableEditing() // Makes fields editable
    {
        //changes design to make it clear to type in tbxs
        tbxProductName.BackColor = Color.White;
        tbxCost.BackColor = Color.White;
        tbxStock.BackColor = Color.White;
        tbxSupplier.BackColor = Color.White;
        tbxType.BackColor = Color.White;
        cbxSuppliers.BackColor = Color.White;
        cbxCategory.BackColor = Color.White;
        cbxColour.BackColor = Color.White;
        cbxFibre.BackColor = Color.White;

        tbxProductName.BorderStyle = BorderStyle.Fixed3D;
        tbxCost.BorderStyle = BorderStyle.Fixed3D;
        tbxStock.BorderStyle = BorderStyle.Fixed3D;
        tbxSupplier.BorderStyle = BorderStyle.Fixed3D;
        tbxType.BorderStyle = BorderStyle.Fixed3D;

        //allows user to edit/add
        lblLowStock.Visible = true;
        lblFullStock.Visible = true;
        numFullStock.Visible = true;
        numLowStock.Visible = true;
        numStock.Visible = true;
        tbxStock.Visible = false;

        tbxProductName.ReadOnly = false;
        tbxCost.ReadOnly = false;
        tbxStock.ReadOnly = false;
        tbxSupplier.ReadOnly = false;
    }

```

```

        tbxType.ReadOnly = false;

        tbxSupplier.Visible = false;
        tbxCategory.Visible = false;
        tbxColour.Visible = false;
        tbxFibre.Visible = false;

        cbxSuppliers.Visible = true;
        cbxCategory.Visible = true;
        cbxColour.Visible = true;
        cbxFibre.Visible = true;

        cbxSuppliers.Text = tbxSupplier.Text;
        cbxCategory.Text = tbxCategory.Text;
        cbxColour.Text = tbxColour.Text;
        cbxFibre.Text = tbxFibre.Text;

        btnApplyFilters.Visible = false;
        btnClearFilters.Visible = false;
        btnCancel.Visible = true;
        btnUpload.Visible = true;
        btnRemove.Visible = true;
        btnReset.Visible = true;
        tbxFileName.Visible = true;
        lblSaveAs.Visible = true;

        pbxHideFilter.Visible = true;
    }
    private void numLowStock_ValueChanged(object sender, EventArgs e) //
Ensures low stock levels cannot be bigger than full stock levels
    {
        if (numLowStock.Value > numFullStock.Value)
        {
            numLowStock.Value = numFullStock.Value;
        }
    }
    private void numFullStock_ValueChanged(object sender, EventArgs e)
// Ensures low stock levels cannot be bigger than full stock levels
    {
        if (numLowStock.Value > numFullStock.Value)
        {
            numFullStock.Value = numLowStock.Value;
        }
    }
    private void editToolStripMenuItem_Click(object sender, EventArgs e)
// Enables the user to edit a product
    {
        editMode = true;
        enableEditing();
        cbxSuppliers.Text = currentSupplier.supplierName;
        cbxCategory.Text = tbxCategory.Text;
        cbxColour.Text = tbxColour.Text;
        cbxFibre.Text = tbxFibre.Text;
        btnAddSave.Text = "Save";
    }
}

```

```

        private void discontinueToolStripMenuItem_Click(object sender,
EventArgs e) // Discontinue a product
        {
            // Products are discontinued rather than deleted to maintain
referential integrity.
            if (dgvProducts.SelectedRows != null)
            {
                DialogResult result = MessageBox.Show("Are you sure you want
to discontinue this product?", "Discontinue Product",
MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
                if (result == DialogResult.Yes)
                {
                    OleDbConnection connection = new OleDbConnection();
                    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                    OleDbCommand cmd = connection.CreateCommand();
                    connection.Open();
                    cmd.CommandText = "UPDATE TableProducts SET Discontinued
= True WHERE ProductID = @pid";
                    cmd.Parameters.AddWithValue("@pid", currentProduct.id);
                    cmd.Connection = connection;
                    int status = cmd.ExecuteNonQuery();
                    connection.Close();
                    MessageBox.Show("Product discontinued.", "Discontinue
Product", MessageBoxButtons.OK, MessageBoxIcon.Information);
                    refresh();
                }
                else
                {
                    MessageBox.Show("No changes were made.", "Discontinue
Product", MessageBoxButtons.OK, MessageBoxIcon.Information);
                }
            }
            else
            {
                MessageBox.Show("Select a product to discontinue.", "Product
Not Selected", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }
        private void btnCancel_Click(object sender, EventArgs e) // Cancel
editing a product and discard all changes
        {
            if (editMode)
            {
                DialogResult result = MessageBox.Show("Are you sure you want
to discard all changes?", "Discard Edits", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
                if (result == DialogResult.Yes)
                {
                    refreshProductInfo(index);
                    disableEditing();
                    pbxImage.ImageLocation = currentProduct.filename;
                }
            }
            else
            {
                refresh();
            }
        }
    }
}

```

```

    }
}

private bool supplierExists(string text)
{
    foreach(Supplier sup in supplierList)
    {
        if(text == sup.supplierName)
        {
            return true;
        }
    }
    return false;
}

private void btnAddSave_Click(object sender, EventArgs e) // Add a
new product, or save changes made if editing
{
    //editMode is set to true when the user uses the context menu
strip and clicks edit
    //editMode is set to false whenever the form refreshes (after
deleting, adding or editing a record) or when the user clicks on another
customer
    if (!editMode)
    {
        NewProduct formNewProduct = new NewProduct(false,
currentSupplier.id);
        formNewProduct.ShowDialog();
        refresh();
    }
    if (editMode)
    {
        if (string.IsNullOrEmpty(tbxProductName.Text) ||
string.IsNullOrEmpty(tbxCost.Text) ||
string.IsNullOrEmpty(cbxSuppliers.Text))
        {
            MessageBox.Show("Enter values in all required fields.",
"Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else if (!supplierExists(cbxSuppliers.Text))
        {
            MessageBox.Show("Select an existing supplier.", "Invalid
Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else if (invalidFileName(tbxFileName.Text))
        {
            MessageBox.Show("Enter a valid filename.", "Invalid
Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else if (!ValidCost(tbxCost.Text))
        {
            MessageBox.Show("Enter a valid cost.", "Invalid Input",
MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else
        {

```

```

        if (string.IsNullOrEmpty(tbxType.Text))
        {
            tbxType.Text = "N/A";
        }
        if (string.IsNullOrEmpty(cbxCategory.Text))
        {
            cbxCategory.Text = "N/A";
        }
        if (string.IsNullOrEmpty(cbxColour.Text))
        {
            cbxColour.Text = "N/A";
        }
        if (string.IsNullOrEmpty(cbxFibre.Text))
        {
            cbxFibre.Text = "N/A";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        connection.Open();
        cmd.CommandText = "UPDATE TableProducts SET ProductName
= @pn, Cost = @cos, Stock = @sto, SupplierID = @sup, BallWeightSize = @typ,
Category = @cat, Colour = @col, FibreMaterial = @fib, FileName = @fn,
LowStock = @ls, FullStock = @fs WHERE ProductID = @prid";
        cmd.Parameters.AddWithValue("@pn", tbxProductName.Text);
        if (tbxCost.Text.StartsWith("£"))
        {
            // If the user enters a £ sign in
            tbxCost.Text = tbxCost.Text.Remove(0, 1);
        }
        cmd.Parameters.AddWithValue("@cos",
Math.Round(Convert.ToDouble(tbxCost.Text), 2));
        cmd.Parameters.AddWithValue("@sto", numStock.Value);
        cmd.Parameters.AddWithValue("@sup",
((Supplier)cbxSuppliers.SelectedItem).id);
        cmd.Parameters.AddWithValue("@typ", tbxType.Text);
        cmd.Parameters.AddWithValue("@cat", cbxCategory.Text);
        cmd.Parameters.AddWithValue("@col", cbxColour.Text);
        cmd.Parameters.AddWithValue("@fib", cbxFibre.Text);
        if (path != null && tbxFileName.Text != "N/A")
        {
            if (tbxFileName.Text.Contains(".jpg") ||
tbxFileName.Text.Contains(".png"))
            {
                cmd.Parameters.AddWithValue("@fn",
tbxFileName.Text);
            }
            else
            {
                cmd.Parameters.AddWithValue("@fn",
tbxFileName.Text + ".jpg");
            }

            byte[] data =
System.IO.File.ReadAllBytes(path.FullName);

```

```

        if (tbxFileName.Text != currentProduct.filename) //
if a different image was uploaded or if the name of the image was changed
        {
            using (Stream str = new MemoryStream(data))
            {
                Bitmap image = new Bitmap(str);
                image.Save(@"productImages\" +
tbxFileName.Text + ".jpg");
            }
        }
    }
    else if(currentProduct.filename == null ||
tbxFileName.Text == "N/A")
    {
        cmd.Parameters.AddWithValue("@fn", "N/A");
    }
    else
    {
        cmd.Parameters.AddWithValue("@fn",
currentProduct.filename);
    }
    cmd.Parameters.AddWithValue("@ls", numLowStock.Value);
    cmd.Parameters.AddWithValue("@fs", numFullStock.Value);
    cmd.Parameters.AddWithValue("@prid", currentProduct.id);
    cmd.Connection = connection;
    int status = cmd.ExecuteNonQuery();
    connection.Close();
    editMode = false;
    MessageBox.Show("Product edited successfully.", "Edit
Product", MessageBoxButtons.OK, MessageBoxIcon.Information);
    refresh();
    refreshProductInfo(index);
    dgvProducts.ClearSelection();
    dgvProducts.CurrentCell =
dgvProducts.Rows[index].Cells[0];
    dgvProducts.Rows[index].Selected = true;
}
}
}
private bool invalidFileName(string filename) // Checks if a
filename is valid or not
{
    string invalidChars = @"!@%^*~|#<>~\`\";
    bool invalid = false;
    for(int i = 0; i < invalidChars.Length; i++)
    {
        if (filename.Contains(invalidChars[i]))
        {
            invalid = true;
            break;
        }
    }
    return invalid;
}
public bool ValidCost(string cost)
{

```

```

        // Checks if the cost entered can be converted to a double,
        which can then be rounded to 2 decimal places if required.
        if (cost.StartsWith("£"))
        {
            // If the user enters a £ sign in
            cost = cost.Remove(0, 1);
        }
        TypeConverter conv =
        TypeDescriptor.GetConverter(typeof(double));
        return conv.IsValid(cost);
    }
    private void pbxImage_Paint(object sender, PaintEventArgs e) //
    Loads image into the picture box
    {
        if (!editMode)
        {
            bool imageError = false;
            try
            {
                if (currentProduct.filename != null)
                {
                    try
                    {
                        System.Drawing.Image image =
                        System.Drawing.Image.FromFile("productImages/" + currentProduct.filename);
                        e.Graphics.DrawImage(image, 0, 0, 217, 300);
                    }
                    catch
                    {
                        System.Drawing.Image image =
                        System.Drawing.Image.FromFile("productImages/noImage.png");
                        e.Graphics.DrawImage(image, 0, 0, 217, 300);
                    }
                }
                imageError = false;
            }
            catch
            {
                if (!imageError)
                {
                    MessageBox.Show("Product image could not be
                    loaded.", "Image Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
                    imageError = true;
                }
            }
        }
        else
        {
            bool imageError = false;
            try
            {
                if (currentProduct.filename != null)
                {
                    try
                    {
                        if(path != null)
                    }

```

```

        System.Drawing.Image image =
System.Drawing.Image.FromFile(path.FullName);
        e.Graphics.DrawImage(image, 0, 0, 217, 300);
    }
    else
    {
        System.Drawing.Image image =
System.Drawing.Image.FromFile("productImages/noImage.png");
        e.Graphics.DrawImage(image, 0, 0, 217, 300);
    }
}
catch
{
    System.Drawing.Image image =
System.Drawing.Image.FromFile("productImages/noImage.png");
    e.Graphics.DrawImage(image, 0, 0, 217, 300);
}
imageError = false;
}
catch
{
    if (!imageError)
    {
        MessageBox.Show("Product image could not be
loaded.", "Image Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        imageError = true;
    }
}
}
}
private void btnUpload_Click(object sender, EventArgs e) // Allows
the user to upload a new image to replace the current one
{
    try
    {
        dialog = new OpenFileDialog();
        dialog.Filter = "jpg files(*.jpg)|*.jpg| PNG
files(*.png)|*.png| All Files(*.*)|*.*";
        if (dialog.ShowDialog() == DialogResult.OK)
        {
            path = new FileInfo(dialog.FileName);
            tbxFileName.Text = path.Name;
            pbxImage.Refresh();
        }
    }
    catch
    {
        MessageBox.Show("Upload error occurred. Please try again.",
"Upload Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
private void btnRemove_Click(object sender, EventArgs e) // Removes
any image from the product selected
{
    tbxFileName.Text = "N/A";
}

```



```

        path = null;
        pbxImage.Refresh();
    }
    private void btnReset_Click(object sender, EventArgs e) // Displays
the original image of the product selected
    {
        tbxFileName.Text = currentProduct.filename;
        path = new FileInfo(currentProductImagePath);
        pbxImage.Refresh();
    }
}
}

```

NewProduct.cs

Form Design

The form is titled "NEW PRODUCT" and contains the following controls and annotations:

- Product Name:** Text box labeled *tbxProductName*.
- Type:** Text box labeled *tbxType*.
- Cost:** Text box labeled *tbxCost*.
- Category:** Dropdown menu labeled *cbxCategory* with a note "shows".
- Stock:** Numeric up/down control labeled *numeric updown controls: Configure Stock levels*.
- Low Stock:** Numeric up/down control.
- Full Stock:** Numeric up/down control.
- Colour:** Text box labeled *cbxColour*.
- Fibre/Material:** Text box labeled *cbxFibre*.
- Supplier:** Dropdown menu.
- Image:** Image box with **UPLOAD** and **REMOVE** buttons. Annotations include *btnUpload*, *btnRemove*, and *Upload and remove image*.
- Save As:** Text box labeled *tbxFileName name of file saved in database*.
- Product Type:** Radio buttons for **Yarn** and **Accessory**, labeled *gpbx Product Type select between "yarn" or "accessory" product*.
- CLEAR:** Button labeled *btnClear -> Clears all details entered/sets to default*.
- ADD:** Button labeled *btnAdd -> Adds product to database*.
- pbxImage:** Image box labeled *pbxImage -> Displays preview of image uploaded*.

Code

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Drawing.Imaging;
using System.IO;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows.Forms;

```

```

namespace YarnShop
{
    public partial class NewProduct : Form
    {
        List<Supplier> suppliers = new List<Supplier>();
        List<string> YarnWeightList = new List<string>();
        List<string> CategoryList = new List<string>();
        List<string> FibreList = new List<string>();
        List<string> MaterialList = new List<string>();
        List<string> ColourList = new List<string>();

        OpenFileDialog dialog;
        FileInfo path = null;
        string tempProductType;

        // Make form draggable
        bool drag = false;
        Point dragCursor;
        Point dragForm;
        private void pbxBar_MouseDown(object sender, MouseEventArgs e)
        {
            drag = true;
            dragCursor = Cursor.Position;
            dragForm = Location;
        }
        private void pbxBar_MouseMove(object sender, MouseEventArgs e)
        {
            if (drag)
            {
                Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
                Location = Point.Add(dragForm, new Size(difference));
            }
        }
        private void pbxBar_MouseUp(object sender, MouseEventArgs e)
        {
            drag = false;
        }
        public NewProduct(bool newSupplier, int selectedSupplierID)
        {
            InitializeComponent();

            //loads suppliers into the combo box
            //should be done in refresh so it can update anything changed
            OleDbConnection conSuppliers = new OleDbConnection();
            conSuppliers.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmdSuppliers = conSuppliers.CreateCommand();
            OleDbDataReader readerSuppliers;
            conSuppliers.Open();
            cmdSuppliers.CommandText = "SELECT * FROM TableSuppliers WHERE
SupplierName <> 'Deleted'";
            cmdSuppliers.Connection = conSuppliers;
            readerSuppliers = cmdSuppliers.ExecuteReader();
            while (readerSuppliers.Read())

```

```

        {
            //instantiates new supplier every time the loop iterates
            suppliers.Add(new
Supplier(Convert.ToInt32(readerSuppliers[0]),
Convert.ToString(readerSuppliers[1]), Convert.ToString(readerSuppliers[2]),
Convert.ToString(readerSuppliers[3]), Convert.ToString(readerSuppliers[4]),
Convert.ToString(readerSuppliers[5]),
Convert.ToString(readerSuppliers[6])));
        }
        cbxSuppliers.DataSource = suppliers;
        cbxSuppliers.DisplayMember = "Suppliers";
        cbxSuppliers.ValueMember = "supplierName";
        conSuppliers.Close();

        newSupplierProduct(newSupplier,selectedSupplierID);
    }

    private void newSupplierProduct(bool newSupplier, int
selectedSupplierID)
    {
        if (newSupplier)
        {
            foreach (Supplier supplier in suppliers)
            {
                if (selectedSupplierID == supplier.id)
                {
                    cbxSuppliers.SelectedIndex =
suppliers.IndexOf(supplier);
                    cbxSuppliers.Enabled = false;
                }
            }
        }
    }

    private void FormNewProduct_Load(object sender, EventArgs e)
    {
        //load all yarn weights into a list
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;

        connection.Open();
        cmd.CommandText = "SELECT DISTINCT Category FROM TableProducts
WHERE ProductType = 'y'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
            {
                YarnWeightList.Add(Convert.ToString(reader[0]));
            }
            else
            {
                continue;
            }
        }
    }

```

```

    }
}
connection.Close();

//load all accessory categories into a list
connection.Open();
cmd.CommandText = "SELECT DISTINCT Category FROM TableProducts
WHERE ProductType = 'a'";
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
    {
        CategoryList.Add(Convert.ToString(reader[0]));
    }
    else
    {
        continue;
    }
}
connection.Close();

//load all yarn fibres into a list
connection.Open();
cmd.CommandText = "SELECT DISTINCT FibreMaterial FROM
TableProducts WHERE ProductType = 'y'";
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
    {
        FibreList.Add(Convert.ToString(reader[0]));
    }
    else
    {
        continue;
    }
}
connection.Close();

//load all accessory materials into a list
connection.Open();
cmd.CommandText = "SELECT DISTINCT FibreMaterial FROM
TableProducts WHERE ProductType = 'a'";
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
    {
        MaterialList.Add(Convert.ToString(reader[0]));
    }
}

```

```

        }
        else
        {
            continue;
        }
    }
    connection.Close();

    //load all colours into a list
    connection.Open();
    cmd.CommandText = "SELECT DISTINCT Colour FROM TableProducts";
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
        {
            ColourList.Add(Convert.ToString(reader[0]));
        }
        else
        {
            continue;
        }
    }
    connection.Close();

    rbtnYarn.Checked = true;
}
public bool ValidCost(string cost)
{
    // Checks if the cost entered can be converted to a double,
    which can then be rounded to 2 decimal places if required.
    if (cost.StartsWith("£"))
    {
        // If the user enters a £ sign in
        cost = cost.Remove(0, 1);
    }
    TypeConverter conv =
TypeDescriptor.GetConverter(typeof(double));
    return conv.IsValid(cost);
}
private bool validString(string text) // Checks if the string is
valid i.e. only contains valid alphanumerical characters and certain special
characters
{
    string allowedCharacters =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 -_.,&/():'";

    // Check each character
    foreach (char c in text)
    {
        bool found = false;
        for (int i = 0; i < allowedCharacters.Length; i++)
        {

```

```

        if(c == allowedCharacters[i])
        {
            found = true;
        }
    }
    if (!found)
    {
        return found;
    }
}

return true;
}

private bool invalidFileName(string filename) // Checks if a
filename is valid or not
{
    string invalidChars = @"!@%^*~|#<>~\`\";
    bool invalid = false;
    for (int i = 0; i < invalidChars.Length; i++)
    {
        if (filename.Contains(invalidChars[i]))
        {
            invalid = true;
            break;
        }
    }
    return invalid;
}

private bool supplierExists(string text)
{
    foreach (Supplier sup in suppliers)
    {
        if (text == sup.supplierName)
        {
            return true;
        }
    }
    return false;
}

private void btnAddSave_Click(object sender, EventArgs e) // Adds
the product
{
    // Ensures there are valid inputs in all required fields
    if(string.IsNullOrEmpty(tbxProductName.Text) ||
string.IsNullOrEmpty(tbxCost.Text) ||
string.IsNullOrEmpty(cbxSuppliers.Text))
    {
        MessageBox.Show("Enter values in all required fields.",
"Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    else if (!supplierExists(cbxSuppliers.Text))
    {
        MessageBox.Show("Select an existing supplier.", "Invalid
Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```

```

    }
    else if (!ValidCost(tbxCost.Text))
    {
        MessageBox.Show("Enter a valid cost.", "Invalid Input",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    else if (!validString(tbxType.Text) ||
    !validString(cbxCategory.Text) || !validString(cbxColour.Text) ||
    !validString(cbxFibre.Text) || !validString(tbxProductName.Text))
    {
        MessageBox.Show("Enter valid inputs in all fields.", "Invalid
        Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    else if (invalidFileName(tbxFileName.Text))
    {
        MessageBox.Show("Enter a valid filename.", "Invalid Input",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    else
    {
        if (string.IsNullOrEmpty(tbxType.Text))
        {
            tbxType.Text = "N/A";
        }
        if (string.IsNullOrEmpty(cbxCategory.Text))
        {
            cbxCategory.Text = "N/A";
        }
        if (string.IsNullOrEmpty(cbxColour.Text))
        {
            cbxColour.Text = "N/A";
        }
        if (string.IsNullOrEmpty(cbxFibre.Text))
        {
            cbxFibre.Text = "N/A";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
        Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        connection.Open();
        cmd.CommandText = "INSERT INTO TableProducts(ProductName,
        Cost, Stock, SupplierID, BallWeightSize, Category, Colour, FibreMaterial,
        ProductType, FileName, LowStock, FullStock)
        VALUES (@pn, @cos, @sto, @sup, @typ, @cat, @col, @fib, @pty, @fn, @ls, @fs)";
        cmd.Parameters.AddWithValue("@pn", tbxProductName.Text);
        if (tbxCost.Text.StartsWith("£"))
        {
            // If the user enters a £ sign in
            tbxCost.Text = tbxCost.Text.Remove(0, 1);
        }
        cmd.Parameters.AddWithValue("@cos",
        Math.Round(Convert.ToDouble(tbxCost.Text), 2));
        cmd.Parameters.AddWithValue("@sto", numStock.Value);
        cmd.Parameters.AddWithValue("@sup",
        ((Supplier)cbxSuppliers.SelectedItem).id);
        cmd.Parameters.AddWithValue("@typ", tbxType.Text);
    }
}

```

```

cmd.Parameters.AddWithValue("@cat", cbxCategory.Text);
cmd.Parameters.AddWithValue("@col", cbxColour.Text);
cmd.Parameters.AddWithValue("@fib", cbxFibre.Text);
cmd.Parameters.AddWithValue("@pty", tempProductType);
if (path != null)
{
    cmd.Parameters.AddWithValue("@fn", tbxFileName.Text +
".jpg");

    byte[] data = File.ReadAllBytes(path.FullName);

    using (Stream str = new MemoryStream(data))
    {
        Bitmap image = new Bitmap(str);
        try
        {
            image.Save(@"productImages\" + tbxFileName.Text
+ ".jpg");
        }
        catch
        {
            MessageBox.Show("Image could not be saved.",
"Save Image", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
else
{
    cmd.Parameters.AddWithValue("@fn", "N/A");
}
cmd.Parameters.AddWithValue("@ls", numLowStock.Value);
cmd.Parameters.AddWithValue("@fs", numFullStock.Value);
cmd.Connection = connection;
int status = cmd.ExecuteNonQuery();
connection.Close();
MessageBox.Show("New product added successfully.", "Product
Added", MessageBoxButtons.OK, MessageBoxIcon.Information);
Close();
}
}
private void btnClear_Click(object sender, EventArgs e) // Clears
all fields
{
    tbxProductName.Text = "";
    tbxCost.Text = "";
    cbxSuppliers.SelectedIndex = 0;
    pbxImage.Image = null;
    tbxFileName.Text = "";
    tbxType.Text = "";
    cbxCategory.SelectedIndex = 0;
    cbxColour.SelectedIndex = 0;
    cbxFibre.SelectedIndex = 0;
    numStock.Value = 0;
    numLowStock.Value = 0;
    numFullStock.Value = 0;
}
private void rbtnYarn_CheckedChanged(object sender, EventArgs e) //
Changes all fields to yarn related fields

```



```

    {
        if (rbtnYarn.Checked)
        {
            tempProductType = "y";
            lblType.Text = "Ball Weight";
            lblCategory.Text = "Yarn Weight";
            lblFibre.Text = "Fibre";

            cbxCategory.DataSource = YarnWeightList;
            cbxFibre.DataSource = FibreList;
            cbxColour.DataSource = ColourList;
        }
    }
    private void rbtnAccessory_CheckedChanged(object sender, EventArgs
e) // Changes all fields to accessory related fields
    {
        if (rbtnAccessory.Checked)
        {
            tempProductType = "a";
            lblType.Text = "Size";
            lblCategory.Text = "Category";
            lblFibre.Text = "Material";

            cbxCategory.DataSource = CategoryList;
            cbxFibre.DataSource = MaterialList;
            cbxColour.DataSource = ColourList;
        }
    }
    private void btnUpload_Click(object sender, EventArgs e) // Upload
an image
    {
        string imageLocation = "";
        try
        {
            dialog = new OpenFileDialog();
            dialog.Filter = "jpg files (*.jpg)|*.jpg| PNG
files (*.png)|*.png| All Files (*.*)|*.*";
            if (dialog.ShowDialog() == DialogResult.OK)
            {
                path = new FileInfo(dialog.FileName);
                imageLocation = dialog.FileName;
                tbxFileName.Text = path.Name; // To be saved in the
database

                pbxImage.ImageLocation = imageLocation;
            }
        }
        catch
        {
            MessageBox.Show("Upload error occurred. Please try again.",
"Upload Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
    private void btnRemove_Click(object sender, EventArgs e) // Remove
any image uploaded
    {
        pbxImage.ImageLocation = null;
    }

```

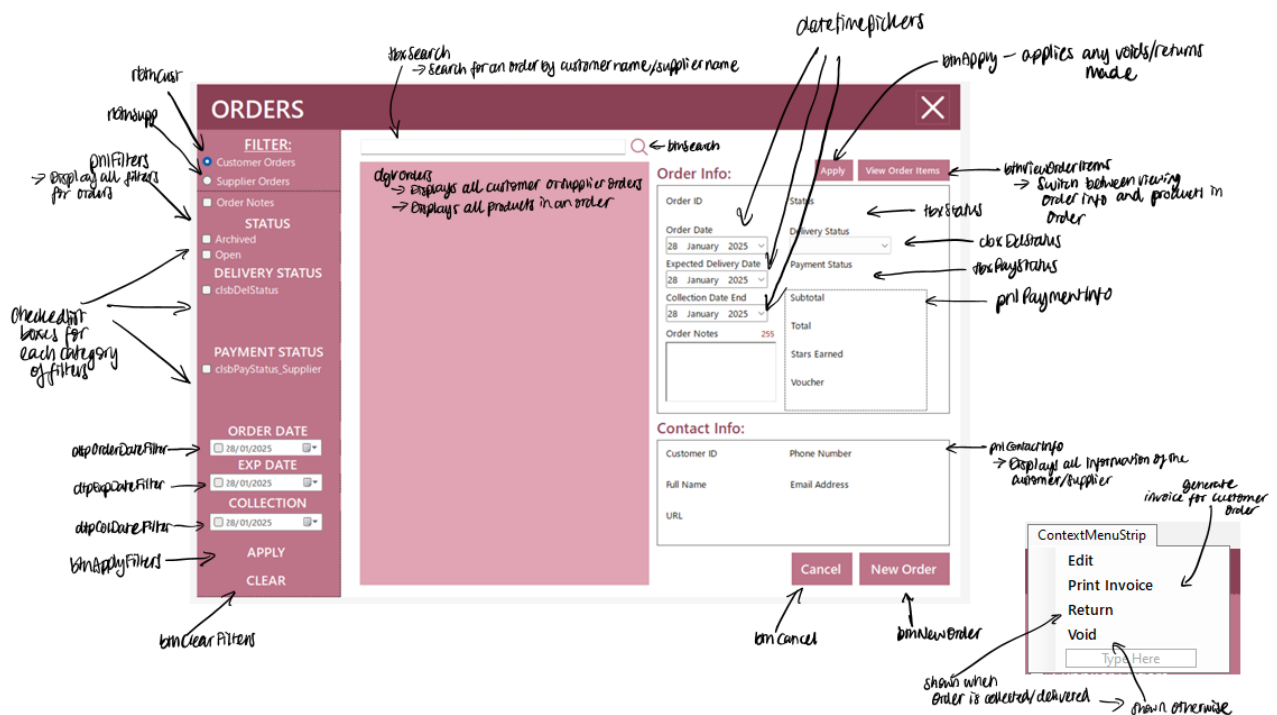
```

        tbxFileName.Text = "";
    }
    private void btnClose_Click(object sender, EventArgs e) // Close the
form
    {
        Close();
    }
    private void numLowStock_ValueChanged(object sender, EventArgs e) //
Low stock cannot be less than full stock
    {
        if (numLowStock.Value > numFullStock.Value)
        {
            numLowStock.Value = numFullStock.Value;
        }
    }
    private void numFullStock_ValueChanged(object sender, EventArgs e)
// Full stock cannot be more than low stock
    {
        if (numLowStock.Value > numFullStock.Value)
        {
            numFullStock.Value = numLowStock.Value;
        }
    }
}
}

```

Orders.cs

Form Design



Code

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Drawing.Printing;
using System.Linq;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace YarnShop
{
    public partial class Orders : Form
    {
        List<Supplier> supplierList = new List<Supplier>();
        List<Customer> customerList = new List<Customer>();
        List<string> SupplierNameList = new List<string>();

        //defining variables that will be used throughout
        List<string> orderList = new List<string>();
        List<string> custDelStatus = new List<string>();
        List<string> supplierDelStatus = new List<string>();
        List<string> payStatus = new List<string>();
        bool filtersApplied;
        string currentCommand;

        List<OrderItem> currentOrderList = new List<OrderItem>();

        Customer currentCust = null;
        Supplier currentSupp = null;
        CustOrder currentCustOrder = null;
        Order currentSupplierOrder = null;
        OrderItem currentOrderItem = null;

        //used for finding row of information about product
        int index;
        int tempQuantity;

        // These variables are static so they can be accessed by the
        CustOrder class to create an invoice.
        public static int pageNum;
        public static float height = 30;
        public static int startIndex = 0;
        public static List<InvoiceLine> lines = new List<InvoiceLine>();

        string selectedDelStatus;
        string selectedPayStatus;
        bool editMode = false;
        bool returnMode = false;
        bool voidMode = false;
        bool supplierRefresh;
        PrintDialog printDlg = new PrintDialog();

        BindingSource bindingSourceItems = new BindingSource();
```

```

// if order items are being viewed then events will occur
differently e.g. when a cell in dgvOrders is clicked it will not be able to
show order info

// Make form draggable
bool drag = false;
Point dragCursor;
Point dragForm;
private void pbxBar_MouseDown(object sender, MouseEventArgs e)
{
    drag = true;
    dragCursor = Cursor.Position;
    dragForm = Location;
}
private void pbxBar_MouseMove(object sender, MouseEventArgs e)
{
    if (drag)
    {
        Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
        Location = Point.Add(dragForm, new Size(difference));
    }
}
private void pbxBar_MouseUp(object sender, MouseEventArgs e)
{
    drag = false;
}
private void btnClose_Click(object sender, EventArgs e)
{
    if (editMode)
    {
        DialogResult result = MessageBox.Show("Are you sure you want
to discard all changes?", "Discard Changes", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
        if (result == DialogResult.Yes)
        {
            Close();
        }
    }
    else
    {
        Close();
    }
}
public Orders(CustOrder custOrder, Order order)
{
    InitializeComponent();
    updateCustomerList();
    updateSupplierList();
    currentCustOrder = custOrder;
    currentSupplierOrder = order;
}

private void FormOrders_Load(object sender, EventArgs e)
{
    pnlProductInfo.Visible = false;
}

```

```

custDelStatus.Add("Order Made");
custDelStatus.Add("Ready To Collect");
custDelStatus.Add("Collected");
custDelStatus.Add("Voided");
custDelStatus.Add("Returned");

supplierDelStatus.Add("Order Made");
supplierDelStatus.Add("Out For Delivery");
supplierDelStatus.Add("Delivered");
supplierDelStatus.Add("Voided");
supplierDelStatus.Add("Returned");

payStatus.Add("Cancelled");
payStatus.Add("Paid");
payStatus.Add("Partially Refunded");
payStatus.Add("Refunded");
payStatus.Add("Unpaid");

clsbPayStatus_Supplier.DataSource = payStatus;

    if (currentCustOrder!=null || currentSupplierOrder != null) //
If this form was opened from a notice, load the order directly
    {
        string command = "";
        if (currentCustOrder != null) // If this form was opened
from a customer order notice
        {
            command = "SELECT * FROM TableCustOrders";
            rbtnCust.Checked = true;
        }
        if (currentSupplierOrder != null) // If this form was opened
from a supplier order notice
        {
            command = "SELECT * FROM TableSupplierOrders";
            rbtnSuppliers.Checked = true;
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = command;
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvOrders.DataSource = bindingSource;
        connection.Close();

        if(currentCustOrder != null)
        {
            foreach (DataGridViewRow row in dgvOrders.Rows)
            {
                if (Convert.ToInt32(row.Cells[0].Value) ==
currentCustOrder.orderID)
                {

```

```

        dgvOrders.ClearSelection();
        dgvOrders.CurrentCell =
dgvOrders.Rows[row.Index].Cells[0];
        dgvOrders[0, row.Index].Selected = true;
    }
}
DataGridViewRow selectedRow = dgvOrders.CurrentRow;
displayCustomerOrderInfo(selectedRow);
lblPayStatus_Supplier.Text = "PAYMENT STATUS";
clsbPayStatus_Supplier.DataSource = null;
clsbPayStatus_Supplier.DataSource = payStatus;
clsbDelStatus.DataSource = null;
clsbDelStatus.DataSource = custDelStatus;
displayCustomerOrders();
}
if(currentSupplierOrder!= null)
{
    foreach (DataGridViewRow row in dgvOrders.Rows)
    {
        if (Convert.ToInt32(row.Cells[0].Value) ==
currentSupplierOrder.orderID)
        {
            dgvOrders.ClearSelection();
            dgvOrders.CurrentCell =
dgvOrders.Rows[row.Index].Cells[0];
            dgvOrders[0, row.Index].Selected = true;
        }
    }
    DataGridViewRow selectedRow = dgvOrders.CurrentRow;
    displaySupplierOrderInfo(selectedRow);
    lblPayStatus_Supplier.Text = "SUPPLIER";
    clsbPayStatus_Supplier.DataSource = null;
    clsbPayStatus_Supplier.DataSource = supplierList;
    clsbPayStatus_Supplier.DisplayMember = "Suppliers";
    clsbPayStatus_Supplier.ValueMember = "supplierName";
    clsbDelStatus.DataSource = null;
    clsbDelStatus.DataSource = supplierDelStatus;
    displaySupplierOrders();
}
}
else
{
    refresh();
}
clearChecked(clsbStatus);
clearChecked(clsbDelStatus);
clearChecked(clsbPayStatus_Supplier);
dtpColDateFilter.Checked = false;
dtpExpDateFilter.Checked = false;
dtpOrderDateFilter.Checked = false;
}
private double applyVoucher(double amount) // Apply voucher
according to the membership of the customer of the current customer order
{
    if (amount < 0)

```

```

    {
        return 0;
    }
    else
    {
        //reapplying the vouchers
        if (currentCustOrder.voucher == "GOLD")
        {
            return Math.Round(amount * 0.85, 2);
        }
        if (currentCustOrder.voucher == "SILVER")
        {
            return Math.Round(amount * 0.9, 2);
        }
        if (currentCustOrder.voucher == "BRONZE")
        {
            return Math.Round(amount * 0.95, 2);
        }
        else
        {
            return amount;
        }
    }
}

private void disableEditing()
{
    editMode = false;
    btnAddSave.Text = "New Order";

    //textboxes are all read only when not in edit mode

    //hide all editable fields
    dtpColDate.Visible = false;
    dtpExpDeliveryDate.Visible = false;
    dtpOrderDate.Visible = false;
    cbxDelStatus.Visible = false;
    tbxOrderDate.Visible = true;
    tbxExpectedDelivery.Visible = true;
    tbxColDate.Visible = true;
    tbxDelStatus.Visible = true;
    rtbxOrderNotes.ReadOnly = true;
    rtbxOrderNotes.BorderStyle = BorderStyle.None;
    rtbxOrderNotes.BackColor = SystemColors.ControlLightLight;

    btnCancel.Visible = false;
    btnViewOrderItems.Visible = true;

    //filters are only usable when not editing
    pnlFilters.Visible = true;
    if (rbtnCust.Checked)
    {
        lblColDateFilter.Visible = true;
        dtpColDateFilter.Visible = true;
    }
    rbtnCust.Visible = true;
    rbtnSuppliers.Visible = true;
}

```

```

        btnAddSave.Text = "New";

    }
    private void enableEditing()
    {
        if(currentCustOrder != null)
        {
            if (currentCustOrder.deliveryStatus != "Voided" &&
currentCustOrder.deliveryStatus != "Returned" &&
currentCustOrder.deliveryStatus != "Collected")
            {
                // When an order has been returned or voided, only order
notes can be edited.
                dtpColDate.Visible = true;
                dtpExpDeliveryDate.Visible = true;
                dtpOrderDate.Visible = true;
                cbxDelStatus.Visible = true;
                tbxOrderDate.Visible = false;
                tbxExpectedDelivery.Visible = false;
                tbxColDate.Visible = false;
                tbxDelStatus.Visible = false;
                cbxDelStatus.Items.Clear();
                cbxDelStatus.Items.AddRange(new string[] { "Collected",
"Ready To Collect", "Order Made"});
                cbxDelStatus.Text = currentCustOrder.deliveryStatus;
            }
        }
        if(currentSupplierOrder != null)
        {
            if(currentSupplierOrder.deliveryStatus != "Voided" &&
currentSupplierOrder.deliveryStatus != "Returned" &&
currentSupplierOrder.deliveryStatus!="Delivered")
            {
                dtpColDate.Visible = false;

                tbxDelStatus.Visible = false;
                tbxPayStatus.Visible = false;

                dtpExpDeliveryDate.Visible = true;

                tbxExpectedDelivery.BackColor = Color.White;
                tbxDelStatus.BackColor = Color.White;
                tbxColDate.BackColor = Color.White;

                cbxDelStatus.Visible = true;

                cbxDelStatus.Items.Clear();
                cbxDelStatus.Items.AddRange(new string[] { "Delivered",
"Out For Delivery", "Order Made" });
                dtpColDate.Visible = false;
                cbxDelStatus.Text = currentSupplierOrder.deliveryStatus;
            }
        }

        // Order notes are always editable.
        rtbxOrderNotes.ReadOnly = false;
    }

```



```

rtbxOrderNotes.BackColor = Color.White;
rtbxOrderNotes.BorderStyle = BorderStyle.Fixed3D;

btnCancel.Visible = true;
btnViewOrderItems.Visible = false;

//filters are only usable when not editing
pnlFilters.Visible = false;
lblColDateFilter.Visible = false;
dtpColDateFilter.Visible = false;
rbtnCust.Visible = false;
rbtnSuppliers.Visible = false;

btnAddSave.Text = "Save";
}

// REFRESHING INFORMATION SHOWN
private void refresh()
{
    disableEditing();
    if (supplierRefresh)
    {
        // ensures the checked change event is triggered incase the
radio button isnt selected at all
        rbtnSuppliers.Checked = false;
        rbtnSuppliers.Checked = true;
    }
    else
    {
        rbtnCust.Checked = false;
        rbtnCust.Checked = true;
    }
    updateCustomerList();
}
private void tbxSearch_TextChanged(object sender, EventArgs e)// As
the user types in the search bar, orders are searched for according to the
input.
{
    if (dgvOrders.DataSource == bindingSourceItems)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            if (rbtnCust.Checked)
            {
                cmd.CommandText = "SELECT DISTINCT
TableProducts.ProductID, TableProducts.ProductName,
TableCustOrderItems.Quantity,TableCustOrderItems.QuantityVoided,TableCustOrd
erItems.QuantityReturned,
TableProducts.BallWeightSize,TableProducts.Category,TableProducts.Colour,Tab
leProducts.FibreMaterial,TableProducts.ProductType FROM (TableProducts INNER

```

```

JOIN TableCustOrderItems ON TableProducts.ProductID =
TableCustOrderItems.ProductID) INNER JOIN TableCustOrders ON
TableCustOrderItems.OrderID = TableCustOrders.@oid WHERE
TableProducts.ProductName LIKE @pn";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@oid",
currentCustOrder.orderID);

    }
    else
    {
        cmd.CommandText = "SELECT DISTINCT
TableProducts.ProductID, TableProducts.ProductName,
TableSupplierOrderItems.Quantity,TableSupplierOrderItems.QuantityVoided,Tabl
eSupplierOrderItems.QuantityReturned,
TableProducts.BallWeightSize,TableProducts.Category,TableProducts.Colour,Tab
leProducts.FibreMaterial,TableProducts.ProductType FROM (TableProducts INNER
JOIN TableSupplierOrderItems ON TableProducts.ProductID =
TableSupplierOrderItems.ProductID) INNER JOIN TableSupplierOrders ON
TableSupplierOrderItems.OrderID = TableSupplierOrders.@oid WHERE
TableProducts.ProductName LIKE @pn";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@oid",
currentSupplierOrder.orderID);

    }
    cmd.Parameters.AddWithValue("@pn", tbxSearch.Text);
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    if (reader.HasRows)
    {
        bindingSourceItems.DataSource = reader;
        dgvOrders.DataSource = bindingSourceItems;
    }
    connection.Close();
    numQuantity.Enabled = false;
}
else
{
    refreshOrderItemsDGV();
}
}
else
{
    if (!filtersApplied)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();

```

```

        if (rbtnCust.Checked)
        {
            cmd.CommandText = "SELECT TableCustOrders.* FROM
(TableCustOrders INNER JOIN TableCustomers ON TableCustOrders.CustID =
TableCustomers.CustID) WHERE TableCustomers.FirstName LIKE @in OR
TableCustomers.Surname LIKE @in OR TableCustomers.EmailAddress LIKE @in;";

        }
        if (rbtnSuppliers.Checked)
        {
            cmd.CommandText = "SELECT TableSupplierOrders.*
FROM (TableSupplierOrders INNER JOIN TableSuppliers ON
TableSupplierOrders.SupplierID = TableSuppliers.SupplierID) WHERE
TableSuppliers.SupplierName LIKE @in";

        }
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@in", "%" +
tbxSearch.Text + "%");
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvOrders.DataSource = bindingSource;
        }
        connection.Close();
    }
    else
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        if (rbtnCust.Checked)
        {
            cmd.CommandText = "SELECT * FROM
TableCustOrders";
        }
        if (rbtnSuppliers.Checked)
        {
            cmd.CommandText = "SELECT * FROM
TableSupplierOrders";
        }
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvOrders.DataSource = bindingSource;
        }
    }
}

```

```

        connection.Close();
    }
}
else
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        if (rbtnCust.Checked)
        {
            cmd.CommandText = currentCommand + " AND
(TableCustomers.FirstName LIKE @in OR TableCustomers.Surname LIKE @in OR
TableCustomers.EmailAddress LIKE @in)";
        }
        if (rbtnSuppliers.Checked)
        {
            cmd.CommandText = currentCommand + " AND
(TableSuppliers.SupplierName LIKE @in OR TableSuppliers.EmailAddress LIKE
@in)";
        }
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ord",
DateTime.Parse(dtpOrderDateFilter.Value.ToShortDateString()));
        cmd.Parameters.AddWithValue("@exd",
DateTime.Parse(dtpExpDateFilter.Value.ToShortDateString()));
        cmd.Parameters.AddWithValue("@col",
DateTime.Parse(dtpColDateFilter.Value.ToShortDateString()));
        cmd.Parameters.AddWithValue("@in", "%" +
tbxSearch.Text + "%");
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();
            bindingSource.DataSource = reader;
            dgvOrders.DataSource = bindingSource;
        }
        connection.Close();
    }
    else
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand;
        cmd.Parameters.Clear();
    }
}

```

```

        cmd.Parameters.AddWithValue("@ord",
DateTime.Parse(dtpOrderDateFilter.Value.ToShortDateString()));
        cmd.Parameters.AddWithValue("@exd",
DateTime.Parse(dtpExpDateFilter.Value.ToShortDateString()));
        cmd.Parameters.AddWithValue("@col",
DateTime.Parse(dtpColDateFilter.Value.ToShortDateString()));
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();
            bindingSource.DataSource = reader;
            dgvOrders.DataSource = bindingSource;
        }
        connection.Close();
    }
}
}
}
private void btnSearch_Click(object sender, EventArgs e) // Search
for orders by customer name/email address or supplier name
{
    if(dgvOrders.DataSource == bindingSourceItems)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            if (rbtnCust.Checked)
            {
                currentOrderList = currentCustOrder.OrderItemList();
//refresh the list
                cmd.CommandText = "SELECT DISTINCT
TableProducts.ProductID, TableProducts.ProductName,
TableCustOrderItems.Quantity,TableCustOrderItems.QuantityVoided,TableCustOrd
erItems.QuantityReturned,
TableProducts.BallWeightSize,TableProducts.Category,TableProducts.Colour,Tab
leProducts.FibreMaterial,TableProducts.ProductType FROM (TableProducts INNER
JOIN TableCustOrderItems ON TableProducts.ProductID =
TableCustOrderItems.ProductID) INNER JOIN TableCustOrders ON
TableCustOrderItems.OrderID = TableCustOrders.@oid WHERE
TableProducts.ProductName LIKE @pn";

                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@oid",
currentCustOrder.orderID);

            }
            else
            {
                currentOrderList =
currentSupplierOrder.OrderItemList(); //refresh the list

```

```

cmd.CommandText = "SELECT DISTINCT
TableProducts.ProductID, TableProducts.ProductName,
TableSupplierOrderItems.Quantity,TableSupplierOrderItems.QuantityVoided,Tabl
eSupplierOrderItems.QuantityReturned,
TableProducts.BallWeightSize,TableProducts.Category,TableProducts.Colour,Tab
leProducts.FibreMaterial,TableProducts.ProductType FROM (TableProducts INNER
JOIN TableSupplierOrderItems ON TableProducts.ProductID =
TableSupplierOrderItems.ProductID) INNER JOIN TableSupplierOrders ON
TableSupplierOrderItems.OrderID = TableSupplierOrders.@oid WHERE
TableProducts.ProductName LIKE @pn;";

cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@oid",
currentSupplierOrder.orderID);

}
cmd.Parameters.AddWithValue("@pn", tbxSearch.Text);
cmd.Connection = connection;
reader = cmd.ExecuteReader();
if (reader.HasRows)
{
    bindingSourceItems.DataSource = reader;
    dgvOrders.DataSource = bindingSourceItems;
}
else
{
    MessageBox.Show("No products found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
}
connection.Close();
numQuantity.Enabled = false;
}
else
{
    refreshOrderItemsDGV();
}
}
else
{
    if (!filtersApplied)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            if (rbtnCust.Checked)
            {
                cmd.CommandText = "SELECT TableCustOrders.* FROM
(TableCustOrders INNER JOIN TableCustomers ON TableCustOrders.CustID =
TableCustomers.CustID) WHERE TableCustomers.FirstName LIKE @in OR
TableCustomers.Surname LIKE @in OR TableCustomers.EmailAddress LIKE @in;";

```

```

        }
        if (rbtnSuppliers.Checked)
        {
            cmd.CommandText = "SELECT TableSupplierOrders.*
FROM (TableSupplierOrders INNER JOIN TableSuppliers ON
TableSupplierOrders.SupplierID = TableSuppliers.SupplierID) WHERE
TableSuppliers.SupplierName LIKE @in";

            }
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@in", "%" +
tbxSearch.Text + "%");
            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                BindingSource bindingSource = new
BindingSource();

                bindingSource.DataSource = reader;
                dgvOrders.DataSource = bindingSource;
            }
            else
            {
                MessageBox.Show("No orders found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            connection.Close();
        }
        else
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            if (rbtnCust.Checked)
            {
                cmd.CommandText = "SELECT * FROM
TableCustOrders";
            }
            if (rbtnSuppliers.Checked)
            {
                cmd.CommandText = "SELECT * FROM
TableSupplierOrders";
            }
            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                BindingSource bindingSource = new
BindingSource();

                bindingSource.DataSource = reader;
                dgvOrders.DataSource = bindingSource;
            }
            else

```

```

        {
            MessageBox.Show("No orders found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
}
else
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        if (rbtnCust.Checked)
        {
            cmd.CommandText = currentCommand + " AND
(TableCustomers.FirstName LIKE @in OR TableCustomers.Surname LIKE @in OR
TableCustomers.EmailAddress LIKE @in)";
        }
        if (rbtnSuppliers.Checked)
        {
            cmd.CommandText = currentCommand + " AND
TableSuppliers.SupplierName LIKE @in";
        }
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ord",
DateTime.Parse(dtpOrderDateFilter.Value.ToShortDateString()));
        cmd.Parameters.AddWithValue("@exd",
DateTime.Parse(dtpExpDateFilter.Value.ToShortDateString()));
        cmd.Parameters.AddWithValue("@col",
DateTime.Parse(dtpColDateFilter.Value.ToShortDateString()));
        cmd.Parameters.AddWithValue("@in", "%" +
tbxSearch.Text + "%");
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvOrders.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No orders found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
}
else
{

```



```

        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand;
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ord",
DateTime.Parse(dtpOrderDateFilter.Value.ToShortDateString()));
        cmd.Parameters.AddWithValue("@exd",
DateTime.Parse(dtpExpDateFilter.Value.ToShortDateString()));
        cmd.Parameters.AddWithValue("@col",
DateTime.Parse(dtpColDateFilter.Value.ToShortDateString()));
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvOrders.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No orders found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
}

private void dgvOrders_CellClick(object sender,
DataGridViewCellEventArgs e) // Allows the user to click on each order and
view all details of each order
{
    //when you click one cell it obtains info from the cell and puts
it in the separate textboxes to display the info
    index = e.RowIndex;
    editMode = false;
    if (index < 0)
    {
        //Index cannot be negative.
        index = 0;
    }
    if (dgvOrders.DataSource == bindingSourceItems) // When viewing
order items
    {
        displayOrderItemInfo(dgvOrders.Rows[index]);
        numQuantity.Enabled = false;
    }
    else
    {
        disableEditing(); // Disable editing when user clicks on
another order
        if (rbtnCust.Checked == true)

```

```

        {
            displayCustomerOrderInfo(dgvOrders.Rows[index]);
        }
        if (rbtnSuppliers.Checked == true)
        {
            displaySupplierOrderInfo(dgvOrders.Rows[index]);
        }
        else // this is when an order has been passed into the form
by notices
        {
            if(currentCustOrder != null)
            {
                displayCustomerOrderInfo(dgvOrders.Rows[index]);
            }
            if(currentSupplierOrder != null)
            {
                displaySupplierOrderInfo(dgvOrders.Rows[index]);
            }
        }
    }
}

private void displayCustomerOrders() // Displays all fields for
customer orders
{
    tbxStarsEarned.Visible = true;
    lblStarsEarned.Visible = true;

    tbxPayStatus.Visible = true;
    lblPayStatus.Visible = true;
    pnlPaymentInfo.Location = new Point(180, 147);

    lblVoucherUsed.Visible = true;
    tbxVoucher.Visible = true;

    lblPayStatus_Supplier.Text = "PAYMENT STATUS";
    clsbPayStatus_Supplier.DataSource = null;
    clsbPayStatus_Supplier.DataSource = payStatus;

    clsbDelStatus.DataSource = null;
    clsbDelStatus.DataSource = custDelStatus;

    lblOrderNotes.Location = new Point(9, 205);
    rtbxOrderNotes.Location = new Point(11, 226);

    tbxColDate.Visible = true;
    lblCollectionDate.Visible = true;
    dtpColDateFilter.Visible = true;
    lblColDateFilter.Visible = true;

    lblID.Text = "Customer ID";
    lblName.Text = "Full Name";
    lblURL.Text = "Membership Type";

    supplierRefresh = false;
}

```

```

        if (editMode)
        {
            dtpColDate.Visible = true;
        }
        printInvoiceToolStripMenuItem.Visible = true;
    }
    private void updateCustomerList() // Refreshes the list of customers
    {
        //loads customers into a list of instances
        customerList.Clear();
        OleDbConnection conCustomers = new OleDbConnection();
        conCustomers.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmdCustomers = conCustomers.CreateCommand();
        OleDbDataReader readerCustomers;
        conCustomers.Open();
        cmdCustomers.CommandText = "SELECT * FROM TableCustomers WHERE
MemType <> 'Deleted' AND MemType <> 'Guest'";
        cmdCustomers.Connection = conCustomers;
        readerCustomers = cmdCustomers.ExecuteReader();
        while (readerCustomers.Read())
        {
            //instantiates new customer every time the loop iterates
            customerList.Add(new
Customer(Convert.ToInt32(readerCustomers[0]),
Convert.ToString(readerCustomers[1]), Convert.ToString(readerCustomers[2]),
Convert.ToString(readerCustomers[3]), Convert.ToString(readerCustomers[4]),
Convert.ToString(readerCustomers[5]), Convert.ToInt32(readerCustomers[6]),
Convert.ToInt32(readerCustomers[7])));
        }
        conCustomers.Close();
    }
    private void displayCustomerOrderInfo(DataGridViewRow selectedRow)
// Displays all customer order details
    {
        // Loads all the information of the selected customer order into
an object to be used to display all the order and customer information.
        currentCustOrder = new
CustOrder(Convert.ToInt32(selectedRow.Cells[0].Value),
Convert.ToInt32(selectedRow.Cells[1].Value),
Convert.ToDateTime(selectedRow.Cells[2].Value),
Convert.ToDateTime(selectedRow.Cells[3].Value),
selectedRow.Cells[4].Value.ToString(),
Convert.ToDouble(selectedRow.Cells[8].Value),
Convert.ToDouble(selectedRow.Cells[9].Value),
Convert.ToString(selectedRow.Cells[11].Value),
Convert.ToDateTime(selectedRow.Cells[6].Value),
selectedRow.Cells[5].Value.ToString(),
Convert.ToInt32(selectedRow.Cells[7].Value),
selectedRow.Cells[10].Value.ToString());
        currentOrderList = currentCustOrder.OrderItemList();
        tbxOrderID.Text = currentCustOrder.orderID.ToString();
        dtpOrderDate.Value = currentCustOrder.orderDate;
        tbxOrderDate.Text =
currentCustOrder.orderDate.ToString("dd/MM/yyyy");
        dtpExpDeliveryDate.Value = currentCustOrder.deliveryDate;
    }

```

```

        tbxExpectedDelivery.Text =
currentCustOrder.deliveryDate.ToString("dd/MM/yyyy");
        tbxDelStatus.Text = currentCustOrder.deliveryStatus;
        selectedDelStatus = tbxDelStatus.Text;
        tbxStatus.Text = currentCustOrder.status;
        tbxPayStatus.Text = currentCustOrder.paymentStatus;
        selectedPayStatus = tbxPayStatus.Text;
        dtpColDate.Value = currentCustOrder.colDateEnd;
        tbxColDate.Text =
currentCustOrder.colDateEnd.ToString("dd/MM/yyyy");
        tbxStarsEarned.Text = currentCustOrder.starsEarned.ToString();
        tbxSubtotal.Text = currentCustOrder.subtotalString();
        tbxTotal.Text = currentCustOrder.totalString();
        tbxVoucher.Text = currentCustOrder.voucher;
        rtbxOrderNotes.Text = currentCustOrder.orderNotes;

        currentCust = currentCustOrder.GetCustomerDetails;

        tbxID.Text = currentCust.id.ToString();
        tbxFullName.Text = currentCust.fullName;
        tbxPhoneNum.Text = currentCust.phoneNum;
        tbxEmailAdd.Text = currentCust.emailAddress;
        tbxURL.Text = currentCust.memType;

        if(currentCustOrder.deliveryStatus == "Voided" ||
currentCustOrder.deliveryStatus == "Returned")
        {
            // If the order is voided or returned, this status cannot be
modified.

            voidToolStripMenuItem.Visible = false;
            returnOrderToolStripMenuItem.Visible = false;
        }
        else if (currentCustOrder.deliveryStatus == "Collected")
        {
            // The order can only be returned when it's been delivered.
            voidToolStripMenuItem.Visible = false;
            returnOrderToolStripMenuItem.Visible = true;
        }
        else
        {
            voidToolStripMenuItem.Visible = true;
            returnOrderToolStripMenuItem.Visible = false;
        }
    }
    private void displaySupplierOrders() // Displays all fields for
supplier orders
    {
        //moving textboxes and labels so visually appealing
        tbxStarsEarned.Visible = false;
        lblStarsEarned.Visible = false;

        tbxColDate.Visible = false;
        lblCollectionDate.Visible = false;
        lblColDateFilter.Visible = false;
        dtpColDateFilter.Visible = false;

        tbxPayStatus.Visible = false;

```

```

lblPayStatus.Visible = false;

lblVoucherUsed.Visible = false;
tbxVoucher.Visible = false;

pnlPaymentInfo.Location = new Point(180, 99);
lblPayStatus_Supplier.Text = "SUPPLIER";
clsbPayStatus_Supplier.DataSource = null;
clsbPayStatus_Supplier.DisplayMember = null;
clsbPayStatus_Supplier.ValueMember = null;
clsbPayStatus_Supplier.DataSource = supplierList;
clsbPayStatus_Supplier.DisplayMember = "Suppliers";
clsbPayStatus_Supplier.ValueMember = "supplierName";

clsbDelStatus.DataSource = null;
clsbDelStatus.DataSource = supplierDelStatus;

lblOrderNotes.Location = lblCollectionDate.Location;
rtbxOrderNotes.Location = tbxColDate.Location;
rtbxOrderNotes.Text = "";

lblID.Text = "Supplier ID";
lblName.Text = "Supplier Name";
lblURL.Text = "URL";

supplierRefresh = true;
printInvoiceToolStripMenuItem.Visible = false;
}
private void updateSupplierList() // Refreshes the list of suppliers
{
    //loads suppliers into a list of instances
    OleDbConnection conSuppliers = new OleDbConnection();
    conSuppliers.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmdSuppliers = conSuppliers.CreateCommand();
    OleDbDataReader readerSuppliers;
    conSuppliers.Open();
    cmdSuppliers.CommandText = "SELECT * FROM TableSuppliers WHERE
SupplierName <> 'Deleted' AND EmailAddress <> 'Deleted'";
    cmdSuppliers.Connection = conSuppliers;
    readerSuppliers = cmdSuppliers.ExecuteReader();
    while (readerSuppliers.Read())
    {
        //instantiates new supplier every time the loop iterates
        supplierList.Add(new
Supplier(Convert.ToInt32(readerSuppliers[0]),
Convert.ToString(readerSuppliers[1]), Convert.ToString(readerSuppliers[2]),
Convert.ToString(readerSuppliers[3]), Convert.ToString(readerSuppliers[4]),
Convert.ToString(readerSuppliers[5]),
Convert.ToString(readerSuppliers[6])));
    }
    conSuppliers.Close();

    foreach (Supplier sup in supplierList)
    {
        SupplierNameList.Add(sup.supplierName);
    }
}

```

```

    }
    private void displaySupplierOrderInfo(DataGridViewRow selectedRow)
// Displays all supplier order details
    {
        // Loads all the information of the selected supplier order into
        an object to be used to display all the order and supplier information.
        currentSupplierOrder = new
Order(Convert.ToInt32(selectedRow.Cells[0].Value),
Convert.ToInt32(selectedRow.Cells[1].Value),
Convert.ToDateTime(selectedRow.Cells[2].Value),
Convert.ToDateTime(selectedRow.Cells[3].Value),
selectedRow.Cells[4].Value.ToString(),
Convert.ToDouble(selectedRow.Cells[5].Value),
Convert.ToDouble(selectedRow.Cells[6].Value),
selectedRow.Cells[7].Value.ToString());
        tbxOrderID.Text = currentSupplierOrder.orderID.ToString();
        dtpOrderDate.Value = currentSupplierOrder.orderDate;
        tbxOrderDate.Text =
currentSupplierOrder.orderDate.ToString("dd/MM/yyyy");
        dtpExpDeliveryDate.Value = currentSupplierOrder.deliveryDate;
        tbxExpectedDelivery.Text =
currentSupplierOrder.deliveryDate.ToString("dd/MM/yyyy");
        tbxDelStatus.Text = currentSupplierOrder.deliveryStatus;
        selectedDelStatus = tbxDelStatus.Text;
        tbxStatus.Text = currentSupplierOrder.status;
        tbxSubtotal.Text = currentSupplierOrder.subtotalString();
        tbxTotal.Text = currentSupplierOrder.totalString();
        rtbxOrderNotes.Text = currentSupplierOrder.orderNotes;

        currentSupp = currentSupplierOrder.GetSupplierDetails();

        tbxID.Text = currentSupp.id.ToString();
        tbxFullName.Text = currentSupp.supplierName;
        tbxPhoneNum.Text = currentSupp.phoneNum;
        tbxEmailAdd.Text = currentSupp.emailAddress;
        tbxURL.Text = currentSupp.url;

        if (currentSupplierOrder.deliveryStatus == "Voided" ||
currentSupplierOrder.deliveryStatus == "Returned")
        {
            voidToolStripMenuItem.Visible = false;
            returnOrderToolStripMenuItem.Visible = false;
        }
        else if(currentSupplierOrder.deliveryStatus == "Delivered")
        {
            voidToolStripMenuItem.Visible = false;
            returnOrderToolStripMenuItem.Visible = true;
        }
        else
        {
            voidToolStripMenuItem.Visible = true;
            returnOrderToolStripMenuItem.Visible = false;
        }
    }
    private void btnViewOrderItems_Click(object sender, EventArgs e) //
Switches between order details and details of the products in the order
    {

```

```

        switchPanels();
    }
    private void switchPanels() // Switch between viewing order
information and viewing order items
    {
        if (dgvOrders.DataSource != bindingSourceItems)
        {
            rbtnCust.Visible = false;
            rbtnSuppliers.Visible = false;
            pnlFilters.Visible = false;
            editToolStripMenuItem.Visible = false;
            voidToolStripMenuItem.Text = "Void Item";
            returnOrderToolStripMenuItem.Text = "Return Item";
            printInvoiceToolStripMenuItem.Visible = false;

            pnlProductInfo.Visible = true;
            pnlOrderInfo.Visible = false;
            lblOrderInfo.Text = "Product Info";

            btnViewOrderItems.Text = "View Order Info";

            refreshOrderItemsDGV();
            displayOrderItemInfo((dgvOrders.Rows[0]));
        }
        else
        {
            rbtnCust.Visible = true;
            rbtnSuppliers.Visible = true;
            pnlFilters.Visible = true;
            editToolStripMenuItem.Text = "Edit";
            editToolStripMenuItem.Visible = true;
            voidToolStripMenuItem.Text = "Void Order";
            returnOrderToolStripMenuItem.Text = "Return Order";
            printInvoiceToolStripMenuItem.Visible = true;

            pnlProductInfo.Visible = false;
            pnlOrderInfo.Visible = true;
            lblOrderInfo.Text = "Order Info";

            btnViewOrderItems.Text = "View Order Items";

            if (rbtnCust.Checked)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableCustOrders";
                cmd.Connection = connection;
                reader = cmd.ExecuteReader();
                BindingSource bindingSource = new BindingSource();
                bindingSource.DataSource = reader;
                dgvOrders.DataSource = bindingSource;
                connection.Close();
            }
        }
    }

```

```

    }
    if (rbtnSuppliers.Checked)
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableSupplierOrders";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvOrders.DataSource = bindingSource;
        connection.Close();
    }

    // so the order record is selected when the user switches
back to the order view
    foreach (DataGridViewRow row in dgvOrders.Rows)
    {
        if (tbxOrderID.Text == row.Cells[0].Value.ToString())
        {
            dgvOrders.ClearSelection();
            row.Selected = true;
        }
    }
}

private void refreshOrderItemsDGV() // Shows all products in the
order in the dgv.
{
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    if (rbtnCust.Checked)
    {
        currentOrderList = currentCustOrder.OrderItemList();
//refresh the list
        cmd.CommandText = "SELECT DISTINCT TableProducts.ProductID,
TableProducts.ProductName,
TableCustOrderItems.Quantity,TableCustOrderItems.QuantityVoided,TableCustOrd
erItems.QuantityReturned,
TableProducts.BallWeightSize,TableProducts.Category,TableProducts.Colour,Tab
leProducts.FibreMaterial,TableProducts.ProductType FROM (TableProducts INNER
JOIN TableCustOrderItems ON TableProducts.ProductID =
TableCustOrderItems.ProductID) INNER JOIN TableCustOrders ON
TableCustOrderItems.OrderID = TableCustOrders.@oid;";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@oid",
currentCustOrder.orderID);
    }
}

```



```

    }
    else
    {
        currentOrderList = currentSupplierOrder.OrderItemList();
//refresh the list
        cmd.CommandText = "SELECT DISTINCT TableProducts.ProductID,
TableProducts.ProductName,
TableSupplierOrderItems.Quantity,TableSupplierOrderItems.QuantityVoided,Tabl
eSupplierOrderItems.QuantityReturned,
TableProducts.BallWeightSize,TableProducts.Category,TableProducts.Colour,Tab
leProducts.FibreMaterial,TableProducts.ProductType FROM (TableProducts INNER
JOIN TableSupplierOrderItems ON TableProducts.ProductID =
TableSupplierOrderItems.ProductID) INNER JOIN TableSupplierOrders ON
TableSupplierOrderItems.OrderID = TableSupplierOrders.@oid;";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@oid",
currentSupplierOrder.orderID);

    }
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    bindingSourceItems.DataSource = reader;
    dgvOrders.DataSource = bindingSourceItems;
    connection.Close();
    numQuantity.Enabled = false;
}
private void displayOrderItemInfo(DataGridViewRow selectedRow) //
Display information of the product currently selected
{
    foreach (OrderItem item in currentOrderList)
    {
        if (Convert.ToInt32(selectedRow.Cells[0].Value) ==
item.product.id)
        {
            currentOrderItem = item;
        }
    }
    if (currentOrderItem != null)
    {
        tbxProductID.Text = currentOrderItem.product.id.ToString();
        tbxProductName.Text = currentOrderItem.product.name;
        tbxProductCost.Text = currentOrderItem.product.CostString;
        tbxStock.Text = currentOrderItem.product.stock.ToString();

        //this loop searches through the array of Suppliers and
matches the selected product's supplier
        // to the corresponding supplier in the array using their
ID, then returns the supplier name to the textbox

        foreach (Supplier selectingSup in supplierList)
        {
            if (selectingSup.id ==
currentOrderItem.product.supplierID)
            {
                tbxSupplier.Text = selectingSup.supplierName;
            }
        }
    }
}

```

```

    }

    tbxType.Text = selectedRow.Cells[5].Value.ToString();
    tbxCategory.Text = selectedRow.Cells[6].Value.ToString();
    tbxColour.Text = selectedRow.Cells[7].Value.ToString();
    tbxFibre.Text = selectedRow.Cells[8].Value.ToString();
    numQuantity.Maximum = currentOrderItem.quantityRemaining;
    numQuantity.Value = numQuantity.Maximum;
    tbxQuantityVoided.Text =
currentOrderItem.quantityVoided.ToString();
    tbxQuantityReturned.Text =
currentOrderItem.quantityReturned.ToString();

    //changes the field names based on the type of product
selected

    if (selectedRow.Cells[9].Value.ToString() == "y")
    {
        lblType.Text = "Ball Weight";
        lblCategory.Text = "Yarn Weight";
        lblFibre.Text = "Fibre";
    }
    if (selectedRow.Cells[9].Value.ToString() == "a")
    {
        lblType.Text = "Size";
        lblCategory.Text = "Category";
        lblFibre.Text = "Material";
    }
    }
}

// FILTERS
private void rbtnCust_CheckedChanged(object sender, EventArgs e)
{
    currentSupplierOrder = null;
    if (rbtnCust.Checked == true)
    {
        pnlFilters.Visible = true;
        if (pnlProductInfo.Visible)
        {
            pnlFilters.Visible = false;
            chbxOrderNotes.Enabled = true;
            editToolStripMenuItem.Text = "Edit";
            voidToolStripMenuItem.Text = "Void Order";
            returnOrderToolStripMenuItem.Text = "Return Order";

            pnlContactInfo.Enabled = true;
            pnlProductInfo.Visible = false;
            pnlOrderInfo.Visible = true;
            lblOrderInfo.Text = "Order Info";

            btnViewOrderItems.Text = "View Order Items";
        }

        lblPayStatus_Supplier.Text = "PAYMENT STATUS";
        clsbPayStatus_Supplier.DataSource = null;
        clsbPayStatus_Supplier.DataSource = payStatus;
    }
}

```

```

        clsbDelStatus.DataSource = null;
        clsbDelStatus.DataSource = custDelStatus;

        clearChecked(clsbStatus);
        clearChecked(clsbDelStatus);
        clearChecked(clsbPayStatus_Supplier);
        dtpOrderDateFilter.Value = DateTime.Today;
        dtpExpDateFilter.Value = DateTime.Today;
        dtpColDateFilter.Value = DateTime.Today;

        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableCustOrders";
        cmd.Parameters.Clear();
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvOrders.DataSource = bindingSource;
        connection.Close();
        filtersApplied = false;
        printInvoiceToolStripMenuItem.Visible = true;

        //ensures there is always something in the textboxes so null
        values don't appear when you edit another customer straight after
        displayCustomerOrderInfo(dgvOrders.Rows[0]);
        displayCustomerOrders();
    }
}
e) private void rbtnSuppliers_CheckedChanged(object sender, EventArgs
{
    currentCustOrder = null;
    if (rbtnSuppliers.Checked == true)
    {
        pnlFilters.Visible = true;
        if (pnlProductInfo.Visible)
        {
            pnlFilters.Visible = false;
            chbxOrderNotes.Enabled = true;
            editToolStripMenuItem.Text = "Edit";
            voidToolStripMenuItem.Text = "Void Order";
            returnOrderToolStripMenuItem.Text = "Return Order";

            pnlContactInfo.Enabled = true;
            pnlProductInfo.Visible = false;
            pnlOrderInfo.Visible = true;
            lblOrderInfo.Text = "Order Info";

            btnViewOrderItems.Text = "View Order Items";
        }
    }
}

```

```

clearChecked(clsbStatus);
clearChecked(clsbDelStatus);
clearChecked(clsbPayStatus_Supplier);
dtpOrderDateFilter.Value = DateTime.Today;
dtpExpDateFilter.Value = DateTime.Today;
dtpColDateFilter.Value = DateTime.Today;

lblPayStatus_Supplier.Text = "SUPPLIER";
clsbPayStatus_Supplier.DataSource = null;
clsbPayStatus_Supplier.DataSource = supplierList;
clsbPayStatus_Supplier.DisplayMember = "Suppliers";
clsbPayStatus_Supplier.ValueMember = "supplierName";

clsbDelStatus.DataSource = null;
clsbDelStatus.DataSource = supplierDelStatus;

OleDbConnection connection = new OleDbConnection();
connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
OleDbCommand cmd = connection.CreateCommand();
OleDbDataReader reader;
connection.Open();
cmd.CommandText = "SELECT * FROM TableSupplierOrders";
cmd.Parameters.Clear();
cmd.Connection = connection;
reader = cmd.ExecuteReader();
BindingSource bindingSource = new BindingSource();
bindingSource.DataSource = reader;
dgvOrders.DataSource = bindingSource;
connection.Close();
filtersApplied = false;
printInvoiceToolStripMenuItem.Visible = false;
//ensures there is always something in the textboxes so null
values don't appear when you edit another customer straight after
displaySupplierOrderInfo(dgvOrders.Rows[0]);
displaySupplierOrders();
}
}
private void btnApplyFilters_Click(object sender, EventArgs e)
{
    filtersApplied = false;
    if (clsbStatus.CheckedItems.Count == 0 &&
        clsbDelStatus.CheckedItems.Count == 0 &&
        clsbPayStatus_Supplier.CheckedItems.Count == 0 &&
        !dtpOrderDateFilter.Checked && !dtpExpDateFilter.Checked &&
        !dtpColDateFilter.Checked && !chbxOrderNotes.Checked)
    {
        // If none of the filters are selected, display all records.
        This condition is here to prevent the error that occurs since WHERE is still
        in the base command statement.
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        if (rbtnCust.Checked)

```

```

        {
            cmd.CommandText = "SELECT * FROM TableCustOrders";
        }
        if (rbtnSuppliers.Checked)
        {
            cmd.CommandText = "SELECT * FROM TableSupplierOrders";
        }
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", "a");
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvOrders.DataSource = bindingSource;
        connection.Close();
    }
    else
    {
        if (rbtnCust.Checked)
        {
            string baseCommandText = "SELECT * FROM TableCustOrders
WHERE";

            string delStatusFilterCommand = " DeliveryStatus IN (";
            string payStatusFilterCommand = " PaymentStatus IN (";

            // Need these lists to store the selected items for each
checked list box.
            // Since a new list is created each time the user clicks
the apply button, the list will be cleared each time.
            List<string> selectedStatusFilters = new List<string>();
            List<string> selectedDelStatusFilters = new
List<string>();
            List<string> selectedPayStatusFilters = new
List<string>();

            // Load all the checked filters into their corresponding
lists
            selectedStatusFilters =
clsbStatus.CheckedItems.Cast<string>().ToList();
            if (selectedStatusFilters.Count == 2)
            {
                // If both Archived and Open are selected, add all
delivery statuses to the delivery status filters.
                selectedDelStatusFilters.AddRange(clsbDelStatus.Items.Cast<string>().ToList(
));
            }
            else if (selectedStatusFilters.Contains("Archived"))
            {
                filtersApplied = true;
                selectedDelStatusFilters.AddRange(new List<string> {
"Collected", "Voided", "Returned" });
            }
            else if (selectedStatusFilters.Contains("Open"))
            {
                filtersApplied = true;

```

```

        selectedDelStatusFilters.AddRange(new List<string> {
"Order Made", "Ready To Collect"});
    }

    if(selectedDelStatusFilters.Count != 0 &&
selectedDelStatusFilters != clsbDelStatus.Items.Cast<string>().ToList())
    {
        filtersApplied = true;
        // If filters have already been added as a result of
any statuses being selected
        // And archived and open have not both been selected
resulting in all delivery statuses already being added
        List<string> tempDelStatusFilters =
clsbDelStatus.CheckedItems.Cast<string>().ToList();
        foreach (string delStatusFilter in
tempDelStatusFilters)
        {
            if
(!selectedDelStatusFilters.Contains(delStatusFilter))
            {
selectedDelStatusFilters.Add(delStatusFilter);
            }
        }
    }
    else if (selectedDelStatusFilters !=
selectedStatusFilters)
    {
        filtersApplied = true;
        // If there is nothing in delivery status filters
selectedDelStatusFilters =
clsbDelStatus.CheckedItems.Cast<string>().ToList();
    }
    selectedPayStatusFilters =
clsbPayStatus_Supplier.CheckedItems.Cast<string>().ToList();

    // Building the command string
    if (selectedDelStatusFilters.Count != 0)
    {
        filtersApplied = true;
        if (!baseCommandText.EndsWith("E"))
        {
            delStatusFilterCommand = " AND " +
delStatusFilterCommand;
        }
        foreach (string delStatusFilter in
selectedDelStatusFilters)
        {
            // If there isn't anything in the string so far,
don't want it to start with a comma.
            if (delStatusFilterCommand.EndsWith("("))
            {
                delStatusFilterCommand += "' " +
delStatusFilter + "'";
            }
            // Add the checked filter to the command string.
            else

```

```

        {
            delStatusFilterCommand += "," + "'" +
delStatusFilter + "'";
        }
    }
    delStatusFilterCommand += ")";
    baseCommandText += delStatusFilterCommand;
}
if (selectedPayStatusFilters.Count != 0)
{
    if (!baseCommandText.EndsWith("E"))
    {
        payStatusFilterCommand = " AND " +
payStatusFilterCommand;
    }
    foreach (string payStatusFilter in
selectedPayStatusFilters)
    {
        // If there isn't anything in the string so far,
        // don't want it to start with a comma.
        if (payStatusFilterCommand.EndsWith("("))
        {
            payStatusFilterCommand += "'" +
payStatusFilter + "'";
        }
        // Add the checked filter to the command string.
        else
        {
            payStatusFilterCommand += "," + "'" +
payStatusFilter + "'";
        }
    }
    payStatusFilterCommand += ")";
    baseCommandText += payStatusFilterCommand;
}

// Commands need to end with a closing bracket- SQL
syntax.

if (dtpOrderDateFilter.Checked)
{
    filtersApplied = true;
    if (!baseCommandText.EndsWith("E"))
    {
        baseCommandText += " AND OrderDate = @ord";
    }
    else
    {
        baseCommandText += " OrderDate = @ord";
    }
}
if (dtpExpDateFilter.Checked)
{
    filtersApplied = true;
    if (!baseCommandText.EndsWith("E"))
    {

```

```

        baseCommandText += " AND DeliveryDate = @exd";
    }
    else
    {
        baseCommandText += " DeliveryDate = @exd";
    }
}
if (dtpColDateFilter.Checked)
{
    filtersApplied = true;
    if (!baseCommandText.EndsWith("E"))
    {
        baseCommandText += " AND CollectionDateEnd =
@col";
    }
    else
    {
        baseCommandText += " CollectionDateEnd = @col";
    }
}
if (chbxOrderNotes.Checked)
{
    filtersApplied = true;
    if (!baseCommandText.EndsWith("E"))
    {
        baseCommandText += " AND OrderNotes IS NOT
NULL";
    }
    else
    {
        baseCommandText += " OrderNotes IS NOT NULL";
    }
}

OleDbConnection connection = new OleDbConnection();
connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
OleDbCommand cmd = connection.CreateCommand();
OleDbDataReader reader;
connection.Open();
cmd.CommandText = baseCommandText;
cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@ord",
DateTime.Parse(dtpOrderDateFilter.Value.ToShortDateString()));
cmd.Parameters.AddWithValue("@exd",
DateTime.Parse(dtpExpDateFilter.Value.ToShortDateString()));
cmd.Parameters.AddWithValue("@col",
DateTime.Parse(dtpColDateFilter.Value.ToShortDateString()));
cmd.Connection = connection;
reader = cmd.ExecuteReader();
BindingSource bindingSource = new BindingSource();
if (reader.HasRows)
{
    bindingSource.DataSource = reader;
    dgvOrders.DataSource = bindingSource;
    displayCustomerOrderInfo(dgvOrders.Rows[0]);
}

```



```

    }
    else
    {
        MessageBox.Show("No results found.", "No Results",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    connection.Close();
}
if (rbtnSuppliers.Checked)
{
    string baseCommandText = "SELECT * FROM
TableSupplierOrders WHERE";

    string delStatusFilterCommand = " DeliveryStatus IN (";
    string supplierFilterCommand = " SupplierID IN (";
    //string orderDateFilterCommand = "";

    // Need these lists to store the selected items for each
checked list box.
    // Since a new list is created each time the user clicks
the apply button, the list will be cleared each time.
    List<string> selectedStatusFilters = new List<string>();
    List<string> selectedDelStatusFilters = new
List<string>();

    List<int> selectedSupplierFilters = new List<int>();

    // Load all the checked filters into their corresponding
lists
    selectedStatusFilters =
clsbStatus.CheckedItems.Cast<string>().ToList();
    if (selectedStatusFilters.Count == 2)
    {
        // If both Archived and Open are selected, add all
delivery statuses to the delivery status filters.
selectedDelStatusFilters.AddRange(clsbDelStatus.Items.Cast<string>().ToList(
));
    }
    else if (selectedStatusFilters.Contains("Archived"))
    {
        filtersApplied = true;
        selectedDelStatusFilters.AddRange(new List<string> {
"Delivered", "Voided", "Returned" });
    }
    else if (selectedStatusFilters.Contains("Open"))
    {
        filtersApplied = true;
        selectedDelStatusFilters.AddRange(new List<string> {
"Order Made", "Out For Delivery" });
    }

    if (selectedDelStatusFilters.Count != 0 &&
selectedDelStatusFilters != clsbDelStatus.Items.Cast<string>().ToList())
    {
        filtersApplied = true;
        // If filters have already been added as a result of
any statuses being selected

```

```

// And archived and open have not both been selected
resulting in all delivery statuses already being added
List<string> tempDelStatusFilters =
clsbDelStatus.CheckedItems.Cast<string>().ToList();
foreach (string delStatusFilter in
tempDelStatusFilters)
{
    if
(!selectedDelStatusFilters.Contains(delStatusFilter))
    {
selectedDelStatusFilters.Add(delStatusFilter);
    }
}
else if (selectedDelStatusFilters !=
selectedStatusFilters)
{
    filtersApplied = true;
    // If there is nothing in delivery status filters
    selectedDelStatusFilters =
clsbDelStatus.CheckedItems.Cast<string>().ToList();
}

foreach (Supplier supplier in
clsbPayStatus_Supplier.CheckedItems)
{
    filtersApplied = true;
    foreach (Supplier findSupplier in supplierList)
    {
        if (supplier == findSupplier)
        {
            selectedSupplierFilters.Add(supplier.id);
        }
    }
}
// Building the command string

if (selectedDelStatusFilters.Count != 0)
{
    filtersApplied = true;
    if (!baseCommandText.EndsWith("E"))
    {
        delStatusFilterCommand = " AND " +
delStatusFilterCommand;
    }
    foreach (string delStatusFilter in
selectedDelStatusFilters)
    {
        // If there isn't anything in the string so far,
        don't want it to start with a comma.
        if (delStatusFilterCommand.EndsWith("("))
        {
            delStatusFilterCommand += "' +
delStatusFilter + "'";
        }
        // Add the checked filter to the command string.

```

```

else
{
    delStatusFilterCommand += "," + "'" +
delStatusFilter + "'";
}
}
delStatusFilterCommand += ")";
baseCommandText += delStatusFilterCommand;
}
if (selectedSupplierFilters.Count != 0)
{
    filtersApplied = true;
    if (!baseCommandText.EndsWith("E"))
    {
        supplierFilterCommand = " AND " +
supplierFilterCommand;
    }
    foreach (int supplierFilter in
selectedSupplierFilters)
    {
        // If there isn't anything in the string so far,
        // don't want it to start with a comma.
        if (supplierFilterCommand.EndsWith("("))
        {
            supplierFilterCommand += supplierFilter;
        }
        // Add the checked filter to the command string.
        else
        {
            supplierFilterCommand += "," +
supplierFilter;
        }
    }
    supplierFilterCommand += ")";
    baseCommandText += supplierFilterCommand;
}

// Commands need to end with a closing bracket- SQL
syntax.

if (dtpOrderDateFilter.Checked)
{
    filtersApplied = true;
    if (!baseCommandText.EndsWith("E"))
    {
        baseCommandText += " AND OrderDate = @ord";
    }
    else
    {
        baseCommandText += " OrderDate = @ord";
    }
}
if (dtpExpDateFilter.Checked)
{
    filtersApplied = true;
    if (!baseCommandText.EndsWith("E"))

```

```

        {
            baseCommandText += " AND DeliveryDate = @exd";
        }
        else
        {
            baseCommandText += " DeliveryDate = @exd";
        }
    }
    if (chbxOrderNotes.Checked)
    {
        filtersApplied = true;
        if (!baseCommandText.EndsWith("E"))
        {
            baseCommandText += " AND OrderNotes IS NOT
NULL";
        }
        else
        {
            baseCommandText += " OrderNotes IS NOT NULL";
        }
    }

    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = baseCommandText;
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@ord",
DateTime.Parse(dtpOrderDateFilter.Value.ToShortDateString()));
    cmd.Parameters.AddWithValue("@exd",
DateTime.Parse(dtpExpDateFilter.Value.ToShortDateString()));
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    BindingSource bindingSource = new BindingSource();
    if (reader.HasRows)
    {
        bindingSource.DataSource = reader;
        dgvOrders.DataSource = bindingSource;
        displaySupplierOrderInfo(dgvOrders.Rows[0]);
        currentCommand = baseCommandText;
    }
    else
    {
        MessageBox.Show("No results found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    connection.Close();
}

}

}
private void clearChecked(CheckedListBox clsbToClear)
{

```

```

        for (int i = 0; i < clsbToClear.Items.Count; i++)
        {
            clsbToClear.SetSelected(i, false);
            clsbToClear.SetItemChecked(i, false);
        }
        clsbToClear.ClearSelected();
    }
    private void btnClearFilters_Click(object sender, EventArgs e)
    {
        DialogResult result = MessageBox.Show("This will clear all
filters. Proceed?", "Clear All Filters", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
        if (result == DialogResult.Yes)
        {
            clearChecked(clsbStatus);
            clearChecked(clsbDelStatus);
            clearChecked(clsbPayStatus_Supplier);
            chbxOrderNotes.Checked = false;
            dtpOrderDateFilter.Value = DateTime.Today;
            dtpExpDateFilter.Value = DateTime.Today;
            dtpColDateFilter.Value = DateTime.Today;
            dtpOrderDateFilter.Checked = false;
            dtpExpDateFilter.Checked = false;
            dtpColDateFilter.Checked = false;

            if (rbtnCust.Checked)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableCustOrders";
                cmd.Parameters.Clear();
                cmd.Connection = connection;
                reader = cmd.ExecuteReader();
                BindingSource bindingSource = new BindingSource();
                bindingSource.DataSource = reader;
                dgvOrders.DataSource = bindingSource;
                connection.Close();
            }
            if (rbtnSuppliers.Checked)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableSupplierOrders";
                cmd.Parameters.Clear();
                cmd.Connection = connection;
                reader = cmd.ExecuteReader();
                BindingSource bindingSource = new BindingSource();
                bindingSource.DataSource = reader;
                dgvOrders.DataSource = bindingSource;
            }
        }
    }

```

```

        connection.Close();
    }
}
else
{
    MessageBox.Show("No filters were deleted.", "Clear All
Filters", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
filtersApplied = false;
}

// EDITING, VOIDING, RETURNING ORDERS
private void editToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (dgvOrders.DataSource != bindingSourceItems) // When viewing
order items and the quantity of an order item is to be edited.
    {
        //TO EDIT THE ORDER

        //changes the add button to a save button
        editMode = true;
        enableEditing();
    }
}
private void btnAddSave_Click(object sender, EventArgs e)
{
    if (!editMode)
    {
        if (rbtnCust.Checked)
        {
            NewCustOrder formMakeCustOrder = new NewCustOrder();
            formMakeCustOrder.ShowDialog();
        }
        if (rbtnSuppliers.Checked)
        {
            NewSupplierOrder formMakeSupplierOrder = new
NewSupplierOrder(null);
            formMakeSupplierOrder.ShowDialog();
            supplierRefresh = true;
        }
        refresh();
    }
    else
    //EDITS PRODUCT IN DATABASE
    {
        if (rtbxOrderNotes.Text == "")
        {
            rtbxOrderNotes.Text = " ";
        }
        if (dtpExpDeliveryDate.Value < dtpOrderDate.Value)
        {
            MessageBox.Show("You cannot enter an expected delivery
date that falls before the order date.", "Invalid Date",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
            dtpExpDeliveryDate.Value =
dtpOrderDate.Value.AddDays(3);
        }
    }
}

```

```

        dtpColDate.Value = dtpExpDeliveryDate.Value.AddDays(7);
    }
    else if (dtpColDate.Value < dtpOrderDate.Value)
    {
        MessageBox.Show("You cannot enter a collection end date
that falls before the order date.", "Invalid Date", MessageBoxButtons.OK,
MessageBoxIcon.Error);
        dtpColDate.Value = dtpExpDeliveryDate.Value.AddDays(7);
    }
    else if (dtpColDate.Value <
dtpExpDeliveryDate.Value.AddDays(-7))
    {
        MessageBox.Show("You cannot enter a collection end date
that falls before the expected delivery date.", "Invalid Date",
MessageBoxButtons.OK, MessageBoxIcon.Error);
        dtpColDate.Value = dtpExpDeliveryDate.Value.AddDays(7);
    }
    else
    {
        if (rbtnCust.Checked)
        {
            if (cbxDelStatus.Text == "Ready To Collect" &&
currentCustOrder.deliveryStatus != cbxDelStatus.Text &&
dtpExpDeliveryDate.Value == currentCustOrder.deliveryDate)
            {
                // If the order's delivery status has been
changed to 'Ready to Collect'
                // If the delivery date has not been changed by
the user
                // Set the delivery date to the day that it
became ready for collection and the collection date end for a week after
                dtpExpDeliveryDate.Value = DateTime.Today;
                dtpColDate.Value =
dtpExpDeliveryDate.Value.AddDays(7);
            }
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            connection.Open();
            cmd.CommandText = "UPDATE TableCustOrders SET
OrderDate = @od, DeliveryDate = @dda, DeliveryStatus = @dsa, PaymentStatus =
@psa, CollectionDateEnd = @col, OrderNotes = @ordn WHERE CustOrderID =
@orid";

            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@od",
Convert.ToString(dtpOrderDate.Value));
            cmd.Parameters.AddWithValue("@dda",
Convert.ToString(dtpExpDeliveryDate.Value));
            cmd.Parameters.AddWithValue("@dsa",
cbxDelStatus.Text);
            cmd.Parameters.AddWithValue("@psa",
tbxPayStatus.Text);
            cmd.Parameters.AddWithValue("@col",
Convert.ToString(dtpColDate.Value));
            cmd.Parameters.AddWithValue("@ordn",
rtbxOrderNotes.Text);

```

```

        cmd.Parameters.AddWithValue("@orid",
currentCustOrder.orderID);
        cmd.Connection = connection;
        int status = cmd.ExecuteNonQuery();
        connection.Close();
        supplierRefresh = false;
    }
    if (rbtnSuppliers.Checked)
    {

        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        connection.Open();
        cmd.CommandText = "UPDATE TableSupplierOrders SET
OrderDate = @od, DeliveryDate = @dda, DeliveryStatus = @dsa, OrderNotes =
@ordn WHERE SupplierOrderID = @orid";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@od",
dtpOrderDate.Value.ToString());
        cmd.Parameters.AddWithValue("@dda",
dtpExpDeliveryDate.Value.ToString());
        cmd.Parameters.AddWithValue("@dsa",
cbxDelStatus.Text);
        cmd.Parameters.AddWithValue("@ordn",
rtbxOrderNotes.Text);
        cmd.Parameters.AddWithValue("@orid",
currentSupplierOrder.orderID);
        cmd.Connection = connection;
        int status = cmd.ExecuteNonQuery();
        connection.Close();

        if (cbxDelStatus.Text == "Delivered" &&
(cbxDelStatus.Text != currentSupplierOrder.deliveryStatus))
        {
            // If the delivery status has been changed to
'Delivered' and it wasn't this previously, then stock can be updated.
            foreach (OrderItem item in currentOrderList)
            {
                updateInventory(item.quantityRemaining,
item.product.id); // quantity remaining is used incase any products were
voided before delivery
            }
        }
        supplierRefresh = true;
    }

    editMode = false;
    MessageBox.Show("Order edited successfully.", "Order
Edited", MessageBoxButtons.OK, MessageBoxIcon.Information);
    refresh();
}
}
}

```



```

        private void updateInventory(int quantityChange, int productID) //
Update stock
    {
        // Updates inventory stock.
        OleDbConnection conUpdateStock = new OleDbConnection();
        conUpdateStock.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
        OleDbCommand cmdUpdateStock = conUpdateStock.CreateCommand();
        conUpdateStock.Open();
        cmdUpdateStock.CommandText = "UPDATE TableProducts SET Stock =
Stock + @sto WHERE ProductID = @prid";
        cmdUpdateStock.Parameters.Clear();
        cmdUpdateStock.Parameters.AddWithValue("@sto", quantityChange);
// If the stock change is negative
        cmdUpdateStock.Parameters.AddWithValue("@prid", productID);
        cmdUpdateStock.Connection = conUpdateStock;
        int statusUpdateStock = cmdUpdateStock.ExecuteNonQuery();
        conUpdateStock.Close();
    }
    private void btnCancel_Click(object sender, EventArgs e)
    {
        if (editMode)
        {
            //ensures user does not accidentally delete product
            DialogResult result = MessageBox.Show("Are you sure you want
to discard all changes?", "Discard Changes", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
            if (result == DialogResult.Yes)
            {
                disableEditing();
                if (rbtnCust.Checked)
                {
                    displayCustomerOrderInfo(dgvOrders.Rows[index]);
                }
                if (rbtnSuppliers.Checked)
                {
                    displaySupplierOrderInfo(dgvOrders.Rows[index]);
                }
            }
            else
            {
                MessageBox.Show("No changes discarded.", "Discard
Changes", MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
        }
    }
    private void updateMembershipDetails(int quantityChange, double
productCost)
    {
        // Update membership details of the customer when voids/returns
are made

        // Calculate the total cost change due to the quantity change
and apply any vouchers.
        // The negative sign ensures the calculation reflects a decrease
in quantity for returns

```

```

        double totalCostChange = -applyVoucher(-quantityChange *
productCost);

        // Store the customer's original total stars before applying the
change.
        int originalTotalStars = currentCust.totalStars;

        // Update the customer's total stars based on the total cost
change.
        // £1 = 3 stars
        currentCust.SetTotalStars(currentCust.totalStars +
Convert.ToInt32(totalCostChange * 3));

        // Change in voucher count based on the change in total stars
        // Customers earn 1 voucher for every 150 stars
        int voucherChange = (currentCust.totalStars -
originalTotalStars) / 150;

        // This decreases the stars earned and may affect the customer's
membership status
        if (quantityChange < 0)
        {
            // Ensure the customer does not have negative total stars
            // If total stars go below zero, adjust the membership type
and restore stars accordingly.
            while (currentCust.totalStars < 0)
            {
                // If the customer is Gold, downgrade to Silver and add
900 stars (the threshold for Gold)
                if (currentCust.memType == "Gold")
                {
                    currentCust.SetMemType("Silver");
                    currentCust.SetTotalStars(currentCust.totalStars +
900);

                    // Adjust the voucher count, ensuring it doesn't
drop below zero
                    if (currentCust.voucherNum > 0)
                    {
                        currentCust.SetVoucherNum(currentCust.voucherNum
+ voucherChange - 1);
                    }
                }
                // If the customer is Silver, downgrade to Bronze and
add 450 stars (the threshold for Silver)
                else if (currentCust.memType == "Silver")
                {
                    currentCust.SetMemType("Bronze");
                    currentCust.SetTotalStars(currentCust.totalStars +
450);

                    // Adjust the voucher count, ensuring it doesn't
drop below zero.
                    if (currentCust.voucherNum > 0)
                    {
                        currentCust.SetVoucherNum(currentCust.voucherNum
+ voucherChange - 1);

```

```

        }
    }
    // If the customer is already Bronze, set their total
stars to zero
    else
    {
        currentCust.SetTotalStars(0);
    }
}

// Ensure the voucher count does not go below zero
if (currentCust.voucherNum < 0)
{
    currentCust.SetVoucherNum(0);
}
}

// Updating the total stars, membership type and vouchers of the
customer in the database
OleDbConnection conCust = new OleDbConnection();
conCust.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0;
Data Source =shopDB.mdb";
OleDbCommand cmdCust = conCust.CreateCommand();
conCust.Open();
cmdCust.CommandText = "UPDATE TableCustomers SET MemType =
@met,TotalStars = @sta, VoucherNum = @von WHERE CustID = @cid";
cmdCust.Parameters.Clear();
cmdCust.Parameters.AddWithValue("@met", currentCust.memType);
cmdCust.Parameters.AddWithValue("@sta", currentCust.totalStars);
cmdCust.Parameters.AddWithValue("@von", currentCust.voucherNum);
cmdCust.Parameters.AddWithValue("@cid", currentCust.id);
cmdCust.Connection = conCust;
int status2 = cmdCust.ExecuteNonQuery();
conCust.Close();
}
private void btnApply_Click(object sender, EventArgs e)
{
    if (voidMode)
    {
        int quantityVoided = currentOrderItem.quantityRemaining -
(int)numQuantity.Value;
        if (quantityVoided != 0) // If the quantity hasn't been
changed.
        {
            // Checks how many products there are in the order. If
there is only one, nothing will be updated as the user must delete the whole
order for there to be 0 products in the order.
            if (dgvOrders.Rows.Count == 1 && numQuantity.Value == 0)
            {
                //If there's only one product in the order, and the
user has attempted to remove another product, the order will have no
products.

                //Therefore it will not allow the user to remove the
product, the user must delete the whole order itself.
            }
        }
    }
}

```

```

        DialogResult result = MessageBox.Show("Voiding this
product will result in a fully voided order. Continue?", "Void Order",
MessageBoxButtons.YesNo, MessageBoxIcon.Question);
        if (result == DialogResult.Yes)
        {
            voidReturnOrder();
            switchPanels();
        }
        else
        {
            MessageBox.Show("No items/orders were voided.",
"Void Order", MessageBoxButtons.OK, MessageBoxIcon.Information);
            numQuantity.Value = tempQuantity; //sets back to
original quantity
        }
    }
    else
    {
        //If there's more than one product in the order, the
product is removed, stock is updated, total cost of order is updated, stars
earned from the order is updated, total stars, membership type and voucher
num of customer is updated
        DialogResult result = MessageBox.Show("Do you want
to permanently void this product?", "Void Item", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
        if (result == DialogResult.Yes)
        {
            if (rbtnCust.Checked)
            {
                updateInventory(quantityVoided,
currentOrderItem.product.id);
                updateMembershipDetails(-quantityVoided,
currentOrderItem.product.cost);

                double newTotal =
Math.Round(currentCustOrder.total -
(applyVoucher(currentOrderItem.product.cost) * quantityVoided), 2);
                int newStarsEarned =
Convert.ToInt32(newTotal * 3);

                OleDbConnection conQuantity = new
OleDbConnection();
                conQuantity.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
                OleDbCommand cmdQuantity =
conQuantity.CreateCommand();
                conQuantity.Open();
                cmdQuantity.CommandText = "UPDATE
TableCustOrderItems SET QuantityVoided = QuantityVoided + @qu WHERE OrderID
= @orid AND ProductID = @prid";
                cmdQuantity.Parameters.Clear();
                cmdQuantity.Parameters.AddWithValue("@qu",
quantityVoided);
                cmdQuantity.Parameters.AddWithValue("@orid",
currentCustOrder.orderID);
                cmdQuantity.Parameters.AddWithValue("@prid",
currentOrderItem.product.id);

```

```

cmdQuantity.Connection = conQuantity;
int statusQuantity =
cmdQuantity.ExecuteNonQuery();
conQuantity.Close();

OleDbConnection conVoid = new
OleDbConnection();
conVoid.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
OleDbCommand cmdVoid =
conVoid.CreateCommand();
conVoid.Open();
cmdVoid.CommandText = "UPDATE
TableCustOrders SET StarsEarned = @st, Total = @to WHERE CustOrderID =
@orid";
cmdVoid.Parameters.Clear();
cmdVoid.Parameters.AddWithValue("@st",
newStarsEarned);
cmdVoid.Parameters.AddWithValue("@to",
newTotal);
cmdVoid.Parameters.AddWithValue("@orid",
currentCustOrder.orderID); // copy to return
cmdVoid.Connection = conVoid;
int status1 = cmdVoid.ExecuteNonQuery();
conVoid.Close();
}
if (rbtnSuppliers.Checked)
{
double newTotal =
Math.Round(currentSupplierOrder.total - (currentOrderItem.product.cost *
quantityVoided), 2);

OleDbConnection conQuantity = new
OleDbConnection();
conQuantity.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
OleDbCommand cmdQuantity =
conQuantity.CreateCommand();
conQuantity.Open();
cmdQuantity.CommandText = "UPDATE
TableSupplierOrderItems SET QuantityVoided = QuantityVoided + @qu WHERE
OrderID = @orid AND ProductID = @prid";
cmdQuantity.Parameters.Clear();
cmdQuantity.Parameters.AddWithValue("@qu",
quantityVoided);
cmdQuantity.Parameters.AddWithValue("@orid",
currentSupplierOrder.orderID);
cmdQuantity.Parameters.AddWithValue("@prid",
currentOrderItem.product.id);
cmdQuantity.Connection = conQuantity;
int statusQuantity =
cmdQuantity.ExecuteNonQuery();
conQuantity.Close();

OleDbConnection conVoid = new
OleDbConnection();

```

```

        conVoid.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
        OleDbCommand cmdVoid =
conVoid.CreateCommand();

        conVoid.Open();
        cmdVoid.CommandText = "UPDATE
TableSupplierOrders SET Total = @to WHERE SupplierOrderID = @orid";
        cmdVoid.Parameters.Clear();
        cmdVoid.Parameters.AddWithValue("@to",
newTotal);

        cmdVoid.Parameters.AddWithValue("@orid",
currentSupplierOrder.orderID); // copy to return
        cmdVoid.Connection = conVoid;
        int status1 = cmdVoid.ExecuteNonQuery();
        conVoid.Close();
    }

    refreshOrderItemsDGV();
}
else
{
    MessageBox.Show("No items were voided.", "Void
Item", MessageBoxButtons.OK, MessageBoxIcon.Information);
    displayOrderItemInfo((dgvOrders.Rows[0]));
}
}
else
{
    MessageBox.Show("No items were voided.", "Void Item",
MessageBoxButtons.OK, MessageBoxIcon.Information);
    displayOrderItemInfo((dgvOrders.Rows[0]));
}
voidMode = false;
tbxQuantityVoided.Text =
Convert.ToString(Convert.ToInt32(tbxQuantityVoided.Text) + quantityVoided);
}
if (returnMode)
{
    int quantityReturned = currentOrderItem.quantityRemaining -
(int)numQuantity.Value;
    if (quantityReturned != 0) // If the quantity hasn't been
changed.
    {
        // Checks how many products there are in the order. If
there is only one, nothing will be updated as the user must delete the whole
order for there to be 0 products in the order.
        if (dgvOrders.Rows.Count == 1)
        {
            //If there's only one product in the order, and the
user has attempted to remove another product, the order will have no
products.

            //Therefore it will not allow the user to remove the
product, the user must delete the whole order itself.
            DialogResult result = MessageBox.Show("Returning
this product will result in a fully returned order. Continue?", "Return
Order", MessageBoxButtons.YesNo, MessageBoxIcon.Question);

```

```

        if (result == DialogResult.Yes)
        {
            voidReturnOrder();
            switchPanels();
        }
        else
        {
            MessageBox.Show("No items/orders were
returned.", "Return Order", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            numQuantity.Value = tempQuantity; //sets back to
original quantity
        }
    }
    else
    {
        //If there's more than one product in the order, the
product is removed, stock is updated, total cost of order is updated, stars
earned from the order is updated, total stars, membership type and voucher
num of customer is updated
        DialogResult result = MessageBox.Show("Do you want
to permanently return this product?", "Return Item",
MessageBoxButtons.YesNo, MessageBoxIcon.Question);
        if (result == DialogResult.Yes)
        {
            if (rbtnCust.Checked)
            {
                updateInventory(quantityReturned,
currentOrderItem.product.id); // If a product is returned to the store,
stock increases
                updateMembershipDetails(-quantityReturned,
currentOrderItem.product.cost);

                double newTotal =
Math.Round(currentCustOrder.total -
(applyVoucher(currentOrderItem.product.cost) * quantityReturned), 2);
                int newStarsEarned =
Convert.ToInt32(newTotal * 3);
                OleDbConnection conQuantity = new
OleDbConnection();
                conQuantity.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
                OleDbCommand cmdQuantity =
conQuantity.CreateCommand();
                conQuantity.Open();
                cmdQuantity.CommandText = "UPDATE
TableCustOrderItems SET QuantityReturned = QuantityReturned + @qu WHERE
OrderID = @orid AND ProductID = @prid";
                cmdQuantity.Parameters.Clear();
                cmdQuantity.Parameters.AddWithValue("@qu",
quantityReturned);
                cmdQuantity.Parameters.AddWithValue("@orid",
currentCustOrder.orderID);
                cmdQuantity.Parameters.AddWithValue("@prid",
currentOrderItem.product.id);
                cmdQuantity.Connection = conQuantity;
            }
        }
    }
}

```

```

cmdQuantity.ExecuteNonQuery();
int statusQuantity =
conQuantity.Close();

OleDbConnection conReturn = new
conReturn.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
OleDbCommand cmdReturn =
conReturn.CreateCommand();
conReturn.Open();
cmdReturn.CommandText = "UPDATE
TableCustOrders SET PaymentStatus = 'Partially Refunded', StarsEarned = @st,
Total = @to WHERE CustOrderID = @orid";
cmdReturn.Parameters.Clear();
cmdReturn.Parameters.AddWithValue("@st",
newStarsEarned);
cmdReturn.Parameters.AddWithValue("@to",
newTotal);
cmdReturn.Parameters.AddWithValue("@orid",
currentCustOrder.orderID);
cmdReturn.Connection = conReturn;
int status1 = cmdReturn.ExecuteNonQuery();
conReturn.Close();
}
if (rbtnSuppliers.Checked)
{
updateInventory(-quantityReturned,
currentOrderItem.product.id); // If a product is returned to the supplier,
stock decreases

double newTotal =
Math.Round(currentSupplierOrder.total - (currentOrderItem.product.cost *
quantityReturned), 2);
OleDbConnection conQuantity = new
conQuantity.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
OleDbCommand cmdQuantity =
conQuantity.CreateCommand();
conQuantity.Open();
cmdQuantity.CommandText = "UPDATE
TableSupplierOrderItems SET QuantityReturned = QuantityReturned + @qu WHERE
OrderID = @orid AND ProductID = @prid";
cmdQuantity.Parameters.Clear();
cmdQuantity.Parameters.AddWithValue("@qu",
quantityReturned);
cmdQuantity.Parameters.AddWithValue("@orid",
currentSupplierOrder.orderID);
cmdQuantity.Parameters.AddWithValue("@prid",
currentOrderItem.product.id);
cmdQuantity.Connection = conQuantity;
int statusQuantity =
cmdQuantity.ExecuteNonQuery();
conQuantity.Close();

```



```

OleDbConnection conReturn = new
OleDbConnection();
conReturn.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
OleDbCommand cmdReturn =
conReturn.CreateCommand();
conReturn.Open();
cmdReturn.CommandText = "UPDATE
TableSupplierOrders SET DeliveryStatus = 'Delivered', Total = @to WHERE
SupplierOrderID = @orid";
cmdReturn.Parameters.Clear();
cmdReturn.Parameters.AddWithValue("@to",
newTotal);
cmdReturn.Parameters.AddWithValue("@orid",
currentSupplierOrder.orderID);
cmdReturn.Connection = conReturn;
int status1 = cmdReturn.ExecuteNonQuery();
conReturn.Close();
}
refreshOrderItemsDGV();
}
else
{
    MessageBox.Show("No items were returned.",
"Return Item", MessageBoxButtons.OK, MessageBoxIcon.Information);
    displayOrderItemInfo((dgvOrders.Rows[0]));
}
}
else
{
    MessageBox.Show("No items were returned.", "Return
Item", MessageBoxButtons.OK, MessageBoxIcon.Information);
    displayOrderItemInfo((dgvOrders.Rows[0]));
}
returnMode = false;
tbxQuantityReturned.Text =
Convert.ToString(Convert.ToInt32(tbxQuantityReturned.Text) +
quantityReturned);
}

refreshOrderItemsDGV();
numQuantity.Enabled = false;
btnApply.Visible = false;
}
private void voidToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (dgvOrders.DataSource == bindingSourceItems) // If the void
button has been clicked on a specific order item
    {
        numQuantity.Enabled = true;
        numQuantity.Maximum = currentOrderItem.quantityRemaining; //
prevents user from increasing the quantity, can only decrease
        btnApply.Visible = true;
        voidMode = true;
    }
    else

```

```

        {
            voidReturnOrder();
            refresh();
        }
    }
    private void voidReturnOrder()
    {
        if (rbtnCust.Checked)
        {
            // Voids/returns all order items and the entire order
            // itself, and adjusts the membership information of the customer linked to the
            // order.
            // This event occurs when the user has clicked "Void Order"
            // or "Return Order" or when the user attempts to remove a product from an
            // order resulting in there being no products left in the order.
            foreach (OrderItem item in currentOrderList)
            {
                updateInventory(item.quantityRemaining,
item.product.id);
                updateMembershipDetails(-item.quantityRemaining,
item.product.cost);
            }

            if (currentCustOrder.voucher != "N/A" &&
currentCustOrder.deliveryStatus != "Collected") // If the customer used a
voucher and the order is being voided, the customer cannot get their voucher
back if they are returning an order
            {
                OleDbConnection con1 = new OleDbConnection();
                con1.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
                OleDbCommand cmd1 = con1.CreateCommand();
                con1.Open();
                cmd1.CommandText = "UPDATE TableCustomers SET VoucherNum
= VoucherNum + 1 WHERE CustID = @cid";
                cmd1.Parameters.Clear();
                cmd1.Parameters.AddWithValue("@cid",
currentCustOrder.id);
                cmd1.Connection = con1;
                int status2 = cmd1.ExecuteNonQuery();
                con1.Close();
            }

            // Void/return all items in the order
            OleDbConnection conVoidReturnItems = new OleDbConnection();
            conVoidReturnItems.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
            OleDbCommand cmdVoidReturnItems =
conVoidReturnItems.CreateCommand();
            conVoidReturnItems.Open();
            if (currentCustOrder.deliveryStatus == "Collected") // If
the order has been collected, then it is returned
            {
                cmdVoidReturnItems.CommandText = "UPDATE
TableCustOrderItems SET QuantityReturned = Quantity - QuantityVoided WHERE
OrderID = @orid";
            }
        }
    }

```

```

        else
        {
            cmdVoidReturnItems.CommandText = "UPDATE
TableCustOrderItems SET QuantityVoided = Quantity - QuantityReturned WHERE
OrderID = @orid";
        }
        cmdVoidReturnItems.Parameters.Clear();
        cmdVoidReturnItems.Parameters.AddWithValue("@orid",
currentCustOrder.orderID);
        cmdVoidReturnItems.Connection = conVoidReturnItems;
        int status = cmdVoidReturnItems.ExecuteNonQuery();
        conVoidReturnItems.Close();

        // Void/return the order
        OleDbConnection conVoidReturn = new OleDbConnection();
        conVoidReturn.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
        OleDbCommand cmdVoidReturn = conVoidReturn.CreateCommand();
        conVoidReturn.Open();
        if(currentCustOrder.deliveryStatus == "Collected") // If the
order has been collected, then it is returned
        {
            cmdVoidReturn.CommandText = "UPDATE TableCustOrders SET
DeliveryStatus = 'Returned', PaymentStatus = 'Refunded', StarsEarned = 0,
Total = 0 WHERE CustOrderID = @orid";
        }
        else //otherwise it is being voided
        {
            cmdVoidReturn.CommandText = "UPDATE TableCustOrders SET
DeliveryDate = OrderDate, DeliveryStatus = 'Voided', PaymentStatus =
'Cancelled', CollectionDateEnd = OrderDate, StarsEarned = 0, Total = 0 WHERE
CustOrderID = @orid";

        }
        cmdVoidReturn.Parameters.Clear();
        cmdVoidReturn.Parameters.AddWithValue("@orid",
currentCustOrder.orderID);
        cmdVoidReturn.Connection = conVoidReturn;
        int status1 = cmdVoidReturn.ExecuteNonQuery();
        conVoidReturn.Close();
    }
    if (rbtnSuppliers.Checked)
    {
        if(currentSupplierOrder.deliveryStatus == "Delivered")
        {
            foreach (OrderItem item in currentOrderList)
            {
                updateInventory(-item.quantityRemaining,
item.product.id);
            }
        }

        // Void/return all items in the order
        OleDbConnection conVoidReturnItems = new OleDbConnection();
        conVoidReturnItems.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
    }

```

```

        OleDbCommand cmdVoidReturnItems =
conVoidReturnItems.CreateCommand();
        conVoidReturnItems.Open();
        if(currentSupplierOrder.deliveryStatus == "Delivered") // If
the order has been delivered, it will be returned
        {
            cmdVoidReturnItems.CommandText = "UPDATE
TableSupplierOrderItems SET QuantityReturned = Quantity - QuantityVoided
WHERE OrderID = @orid";
        }
        else // If the order has not been delivered, it will be
voided
        {
            cmdVoidReturnItems.CommandText = "UPDATE
TableSupplierOrderItems SET QuantityVoided = Quantity - QuantityReturned
WHERE OrderID = @orid";
        }
        cmdVoidReturnItems.Parameters.Clear();
        cmdVoidReturnItems.Parameters.AddWithValue("@orid",
currentSupplierOrder.orderID);
        cmdVoidReturnItems.Connection = conVoidReturnItems;
        int status = cmdVoidReturnItems.ExecuteNonQuery();
        conVoidReturnItems.Close();

        // Void/return the order
        OleDbConnection conVoidReturn = new OleDbConnection();
        conVoidReturn.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
        OleDbCommand cmdVoidReturn = conVoidReturn.CreateCommand();
        conVoidReturn.Open();
        if (currentSupplierOrder.deliveryStatus == "Delivered") //
If the order has been delivered, it will be returned
        {
            cmdVoidReturn.CommandText = "UPDATE TableSupplierOrders
SET DeliveryStatus = 'Returned' WHERE SupplierOrderID = @orid";
        }
        else // If the order has not been delivered, it will be
voided
        {
            cmdVoidReturn.CommandText = "UPDATE TableSupplierOrders
SET DeliveryStatus = 'Voided' WHERE SupplierOrderID = @orid";
        }
        cmdVoidReturn.Parameters.Clear();
        cmdVoidReturn.Parameters.AddWithValue("@orid",
currentSupplierOrder.orderID);
        cmdVoidReturn.Connection = conVoidReturn;
        int status1 = cmdVoidReturn.ExecuteNonQuery();
        conVoidReturn.Close();
    }
}
private void returnOrderToolStripMenuItem_Click(object sender,
EventArgs e)
{
    if (dgvOrders.DataSource == bindingSourceItems) // If the void
button has been clicked on a specific order item
    {
        numQuantity.Enabled = true;
    }
}

```

```

        numQuantity.Maximum = currentOrderItem.quantityRemaining; //
prevents user from increasing the quantity, can only decrease
        btnApply.Visible = true;
        returnMode = true;
    }
    else
    {
        voidReturnOrder();
        refresh();
    }
}
private void dtpOrderDate_ValueChanged(object sender, EventArgs e)
{
    // The order date can be a previous date, if a past order has
not been recorded.
    // It cannot be later than the expected delivery date, but can
be the same if all products in the order are already in store.
    // The collection end date will always be at least a week later
than the order date, if the order date and expected delivery date are the
same.

    if (editMode)
    {
        //Automatically changes both values so
dtpExpDeliveryDate_ValueChanged does not return an error when initiated.
        dtpColDate.Value = dtpOrderDate.Value.AddDays(10);
        dtpExpDeliveryDate.Value = dtpOrderDate.Value.AddDays(3);
    }

}
private void dtpExpDeliveryDate_ValueChanged(object sender,
EventArgs e)
{
    // The expected delivery date cannot be earlier than the order
date, or later than the collection end date.
    // If editing or adding, automatically set the collection end
date to a week after the expected delivery date.
    // Does not change the order date.
    dtpColDate.Value = dtpExpDeliveryDate.Value.AddDays(7);
}
private void cbxDelStatus_SelectionChangeCommitted(object sender,
EventArgs e)
{
    if (rbtnCust.Checked)
    {
        if (cbxDelStatus.SelectedItem.ToString() == "Collected")
        {
            tbxStatus.Text = "Archived";
            tbxPayStatus.Text = "Paid";
        }
        else
        {
            tbxStatus.Text = "Open";
            tbxPayStatus.Text = "Unpaid";
        }
    }
}

```

```

    }
    if (rbtnSuppliers.Checked)
    {
        if (cbxDelStatus.SelectedItem.ToString() == "Delivered")
        {
            tbxStatus.Text = "Archived";
            tbxPayStatus.Text = "Paid";
        }
        else
        {
            tbxStatus.Text = "Open";
            tbxPayStatus.Text = "Unpaid";
        }
    }
}

private void rtbxOrderNotes_TextChanged(object sender, EventArgs e)
{
    // Implements character length limit on order notes. It will
    show the user when the order notes' text length is reaching the limit of
    255.

    if (rtbxOrderNotes.TextLength > 244)
    {
        lblCharLimit.Visible = true;
        lblCharLimit.Text = rtbxOrderNotes.TextLength + "/255";
    }
    else
    {
        lblCharLimit.Visible = false;
    }
}

// PRINTING INVOICES
private void printInvoiceToolStripMenuItem_Click(object sender,
EventArgs e)
{
    // START GENERATING INVOICE

    lines.Clear(); // Ensures that an entirely new invoice is
generated
    pageNum = 1; // Sets the page number to the first page
    startIndex = 0; // Sets the line start index to 0
    printInvoice.DocumentName = "Print Invoice";
    printDlg.Document = printInvoice;
    PaperSize pageSize = new PaperSize();
    pageSize.Width = 220; // Fixes the width of the invoice: this
works with the printer used to print these invoices
    printInvoice.DefaultPageSettings.PaperSize = pageSize;
    printDlg.AllowSelection = true;
    printDlg.AllowSomePages = true;
    if (printDlg.ShowDialog() == DialogResult.OK)
    {
        printInvoice.Print();
    }
}

private void printInvoice_PrintPage(object sender,
PrintPageEventArgs e)
{

```

```

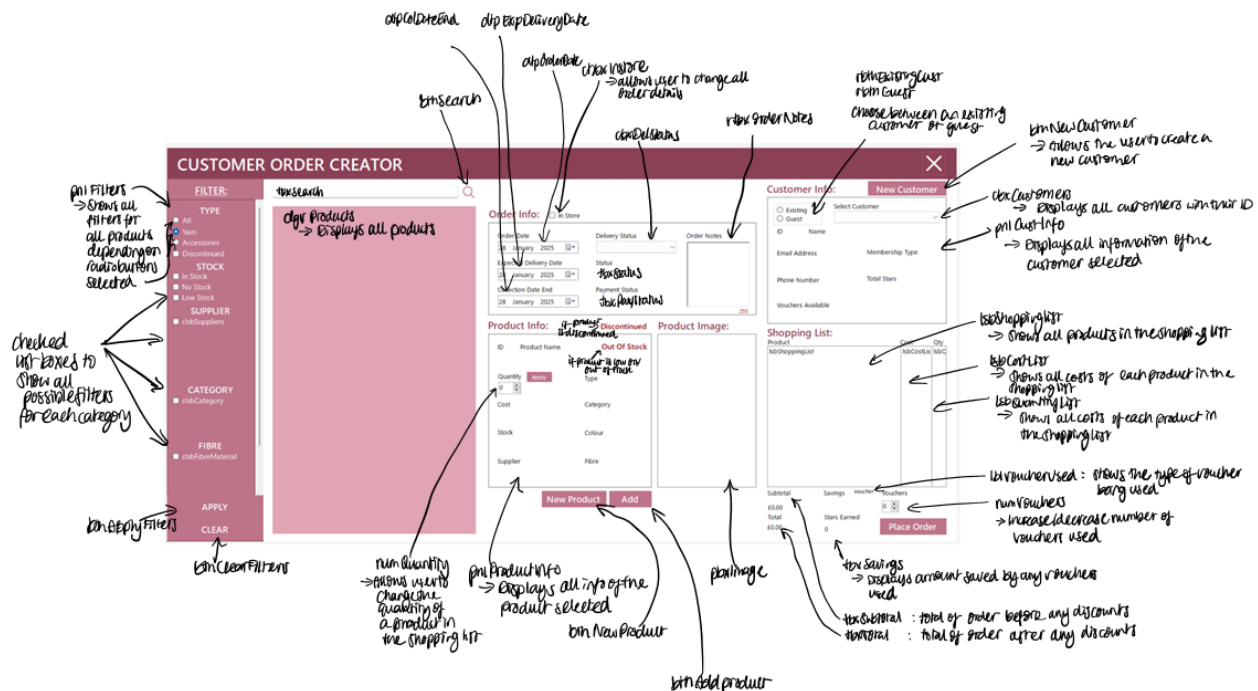
        bool pdf = false;
        height = 30; // initialises the height every time a new page is
        created

        if (printDlg.PrinterSettings.PrinterName == "Microsoft Print to
        PDF")
        {
            pdf = true;
        }
        currentCustOrder.GenerateInvoice(e, pdf, lines);
    }
}

```

NewCustOrder.cs

Form Design



Code

```

using System;
using System.CodeDom.Compiler;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Text;

```

```

using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;
using static YarnShop.NewCustOrder;

namespace YarnShop
{
    public partial class NewCustOrder : Form
    {
        int index;
        double subtotal; //to keep track of these for the subroutine which
calculates the final total
        double total;
        string tempProductType;
        bool custChanged;
        bool addingProduct;
        bool quantityChanged;
        List<Customer> customerList = new List<Customer>();
        Customer currentCust = null;
        Supplier currentSupplier = null;
        Product currentProduct = null;
        List<Supplier> supplierList = new List<Supplier>();
        List<Product> productList = new List<Product>();
        List<OrderItem> orderList = new List<OrderItem>();
        List<string> filterList = new List<string>();
        List<string> YarnWeightList = new List<string>();
        List<string> CategoryList = new List<string>();
        List<string> FibreList = new List<string>();
        List<string> MaterialList = new List<string>();
        List<string> ColourList = new List<string>();
        bool filtersApplied; // True if filters have been applied, used in
searches
        string currentCommand; // Stores the command used to retrieve the
data currently shown in the dgv
        List<string> custDelStatus = new List<string>();
        List<string> payStatus = new List<string>();

        // Make form draggable
        bool drag = false;
        Point dragCursor;
        Point dragForm;
        private void pbxBar_MouseDown(object sender, MouseEventArgs e)
        {
            drag = true;
            dragCursor = Cursor.Position;
            dragForm = Location;
        }
        private void pbxBar_MouseMove(object sender, MouseEventArgs e)
        {
            if (drag)
            {
                Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
                Location = Point.Add(dragForm, new Size(difference));
            }
        }
    }
}

```



```

private void pbxBar_MouseUp(object sender, MouseEventArgs e)
{
    drag = false;
}
private void btnClose_Click(object sender, EventArgs e)
{
    Close();
}
public NewCustOrder()
{
    InitializeComponent();
    lsbShoppingList.DataSource = orderList;
}
private void FormMakeCustOrder_Load(object sender, EventArgs e)
{
    //load all yarn weights into a list
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;

    connection.Open();
    cmd.CommandText = "SELECT DISTINCT Category FROM TableProducts
WHERE ProductType = 'y'";
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
        {
            YarnWeightList.Add(Convert.ToString(reader[0]));
        }
        else
        {
            continue;
        }
    }
    connection.Close();

    //load all accessory categories into a list
    connection.Open();
    cmd.CommandText = "SELECT DISTINCT Category FROM TableProducts
WHERE ProductType = 'a'";
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
        {
            CategoryList.Add(Convert.ToString(reader[0]));
        }
        else
        {

```

```

        continue;
    }
}
connection.Close();

//load all yarn fibres into a list
connection.Open();
cmd.CommandText = "SELECT DISTINCT FibreMaterial FROM
TableProducts WHERE ProductType = 'y'";
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
    {
        FibreList.Add(Convert.ToString(reader[0]));
    }
    else
    {
        continue;
    }
}
connection.Close();

//load all accessory materials into a list
connection.Open();
cmd.CommandText = "SELECT DISTINCT FibreMaterial FROM
TableProducts WHERE ProductType = 'a'";
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
    {
        MaterialList.Add(Convert.ToString(reader[0]));
    }
    else
    {
        continue;
    }
}
connection.Close();

//load all colours into a list
connection.Open();
cmd.CommandText = "SELECT DISTINCT Colour FROM TableProducts";
cmd.Connection = connection;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
    {

```

```

        ColourList.Add(Convert.ToString(reader[0]));
    }
    else
    {
        continue;
    }
}
connection.Close();

refresh();
showCustomerInfo(0);
rbtnExistingCust.Checked = true;
cbxDelStatus.Text = "Order Made";
tbxStatus.Text = "Open";
tbxPayStatus.Text = "Unpaid";
}

// REFRESHING INFORMATION SHOWN
private void refresh()
{
    lsbShoppingList.Refresh();
    lsbCostList.Refresh();
    lsbQuantityList.Refresh();

    cbxCustomers.DataSource = null;
    cbxCustomers.DisplayMember = null;
    cbxCustomers.ValueMember = null;

    clearChecked(clsbCategory);
    clearChecked(clsbFibreMaterial);
    clearChecked(clsbColour);
    clearChecked(clsbSuppliers);
    chbxDiscontinuedFilter.Checked = false;
    pnlFilters.AutoScroll = false;
    clsbCategory.Visible = false;
    clsbFibreMaterial.Visible = false;
    clsbColour.Visible = false;
    lblCategoryFilter.Visible = false;
    lblFibreMaterialFilter.Visible = false;
    lblColourFilter.Visible = false;

    //loads customers into a list
    customerList.Clear();
    OleDbConnection conCustomers = new OleDbConnection();
    conCustomers.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmdCustomers = conCustomers.CreateCommand();
    OleDbDataReader readerCustomers;
    conCustomers.Open();
    cmdCustomers.CommandText = "SELECT * FROM TableCustomers WHERE
MemType <> 'Deleted'";
    cmdCustomers.Connection = conCustomers;
    readerCustomers = cmdCustomers.ExecuteReader();
    while (readerCustomers.Read())
    {
        //instantiates new customer every time the loop iterates

```

```

        customerList.Add(new
Customer(Convert.ToInt32(readerCustomers[0]),
Convert.ToString(readerCustomers[1]), Convert.ToString(readerCustomers[2]),
Convert.ToString(readerCustomers[3]), Convert.ToString(readerCustomers[4]),
Convert.ToString(readerCustomers[5]), Convert.ToInt32(readerCustomers[6]),
Convert.ToInt32(readerCustomers[7])));

    }
    conCustomers.Close();
    cbxCustomers.DataSource = customerList;
    cbxCustomers.ValueMember = "idName";
    cbxCustomers.DisplayMember = "CustInfo";
    cbxCustomers.Refresh();

    //loads suppliers into a list
    supplierList.Clear();
    OleDbConnection conSuppliers = new OleDbConnection();
    conSuppliers.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmdSuppliers = conSuppliers.CreateCommand();
    OleDbDataReader readerSuppliers;
    conSuppliers.Open();
    cmdSuppliers.CommandText = "SELECT * FROM TableSuppliers WHERE
SupplierName <> 'Deleted'";
    cmdSuppliers.Connection = conSuppliers;
    readerSuppliers = cmdSuppliers.ExecuteReader();
    while (readerSuppliers.Read())
    {
        //instantiates new supplier every time the loop iterates
        supplierList.Add(new
Supplier(Convert.ToInt32(readerSuppliers[0]),
Convert.ToString(readerSuppliers[1]), Convert.ToString(readerSuppliers[2]),
Convert.ToString(readerSuppliers[3]), Convert.ToString(readerSuppliers[4]),
Convert.ToString(readerSuppliers[5]),
Convert.ToString(readerSuppliers[6])));
    }
    clsbSuppliers.DataSource = supplierList;
    clsbSuppliers.DisplayMember = "Suppliers";
    clsbSuppliers.ValueMember = "supplierName";
    tbxSupplier.Text = "supplierName";
    conSuppliers.Close();

    refreshProductDGV();

    //ensures there is always something in the textboxes so null
values don't appear when you edit another customer straight after
    refreshProductInfo(0);
    rbtnAll.Checked = true;

}
private void tbxSearch_TextChanged(object sender, EventArgs e) // As
the user types in the search bar, products are searched for according to the
input.
{
    // Search applies to all products currently being shown.
    // If filters have been applied, the search applies to the
products shown.

```

```

        if (!filtersApplied)
        {
            if (!string.IsNullOrEmpty(tbxSearch.Text))
            {
                if (rbtnAll.Checked)
                {
                    OleDbConnection connection = new OleDbConnection();
                    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                    OleDbCommand cmd = connection.CreateCommand();
                    OleDbDataReader reader;
                    connection.Open();
                    cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductName LIKE @pn AND (Discontinued = False OR (Discontinued = True AND
Stock > 0))";

                    cmd.Parameters.Clear();
                    cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

                    cmd.Connection = connection;
                    reader = cmd.ExecuteReader();
                    if (reader.HasRows)
                    {
                        BindingSource bindingSource = new
BindingSource();

                        bindingSource.DataSource = reader;
                        dgvProducts.DataSource = bindingSource;
                    }
                    connection.Close();
                }
                else
                {
                    if (rbtnYarn.Checked)
                    {
                        tempProductType = "y";
                    }
                    if (rbtnAcc.Checked)
                    {
                        tempProductType = "a";
                    }
                    OleDbConnection connection = new OleDbConnection();
                    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                    OleDbCommand cmd = connection.CreateCommand();
                    OleDbDataReader reader;
                    connection.Open();
                    cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND ProductName LIKE @pn AND (Discontinued = False OR
(Discontinued = True AND Stock > 0))";
                    cmd.Parameters.Clear();
                    cmd.Parameters.AddWithValue("@ya", tempProductType);
                    cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

                    cmd.Connection = connection;
                    reader = cmd.ExecuteReader();
                    if (reader.HasRows)
                    {

```

```

BindingSource bindingSource = new
BindingSource();

        bindingSource.DataSource = reader;
        dgvProducts.DataSource = bindingSource;
    }
    connection.Close();
}
else
{
    if (rbtnAll.Checked)
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
(Discontinued = False OR (Discontinued = True AND Stock > 0))";
        cmd.Parameters.Clear();
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
            }
            connection.Close();
        }
        else
        {
            if (rbtnYarn.Checked)
            {
                tempProductType = "y";
            }
            if (rbtnAcc.Checked)
            {
                tempProductType = "a";
            }

            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND (Discontinued = False OR (Discontinued = True AND
Stock > 0))";

            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@ya", tempProductType);
            cmd.Connection = connection;
            reader = cmd.ExecuteReader();

```

```

        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
}
else
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        if (rbtnAll.Checked)
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";

            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                BindingSource bindingSource = new
BindingSource();

                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
            }
            connection.Close();
        }
        else
        {
            if (rbtnYarn.Checked)
            {
                tempProductType = "y";
            }
            if (rbtnAcc.Checked)
            {
                tempProductType = "a";
            }
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";

```

```

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
    else
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand;
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        if (reader.HasRows)
        {
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
}
highlightProductsStock();
}
private void btnSearch_Click(object sender, EventArgs e)
{
    if (!filtersApplied)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            if (rbtnAll.Checked)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductName LIKE @pn AND (Discontinued = False OR (Discontinued = True AND
Stock > 0))";

```



```

cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
cmd.Connection = connection;
reader = cmd.ExecuteReader();
if (reader.HasRows)
{
    BindingSource bindingSource = new
BindingSource();

    bindingSource.DataSource = reader;
    dgvProducts.DataSource = bindingSource;
}
else
{
    MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
connection.Close();
}
else
{
    if (rbtnYarn.Checked)
    {
        tempProductType = "y";
    }
    if (rbtnAcc.Checked)
    {
        tempProductType = "a";
    }
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND ProductName LIKE @pn AND (Discontinued = False OR
(Discontinued = True AND Stock > 0))";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@ya", tempProductType);
    cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
cmd.Connection = connection;
reader = cmd.ExecuteReader();
if (reader.HasRows)
{
    BindingSource bindingSource = new
BindingSource();

    bindingSource.DataSource = reader;
    dgvProducts.DataSource = bindingSource;
}
else
{
    MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
connection.Close();

```

```

    }
}
else
{
    if (rbtnAll.Checked)
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts AND
(Discontinued = False OR (Discontinued = True AND Stock > 0))";
        cmd.Parameters.Clear();
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
    else
    {
        if (rbtnYarn.Checked)
        {
            tempProductType = "y";
        }
        if (rbtnAcc.Checked)
        {
            tempProductType = "a";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND (Discontinued = False OR (Discontinued = True AND
Stock > 0))";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {

```

```

BindingSource bindingSource = new
BindingSource();

        bindingSource.DataSource = reader;
        dgvProducts.DataSource = bindingSource;
    }
    else
    {
        MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    connection.Close();
}
}
else
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        if (rbtnAll.Checked)
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";

            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                BindingSource bindingSource = new
BindingSource();

                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
            }
            else
            {
                MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            connection.Close();
        }
        else
        {
            if (rbtnYarn.Checked)
            {
                tempProductType = "y";
            }
            if (rbtnAcc.Checked)
            {
                tempProductType = "a";
            }
        }
    }
}

```

```

OleDbConnection connection = new OleDbConnection();
connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
OleDbCommand cmd = connection.CreateCommand();
OleDbDataReader reader;
connection.Open();
cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";

cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@ya", tempProductType);
cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

cmd.Connection = connection;
reader = cmd.ExecuteReader();
if (reader.HasRows)
{
    BindingSource bindingSource = new
BindingSource();

    bindingSource.DataSource = reader;
    dgvProducts.DataSource = bindingSource;
}
else
{
    MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
connection.Close();
}
else
{
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = currentCommand;
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@ya", tempProductType);
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    BindingSource bindingSource = new BindingSource();
    if (reader.HasRows)
    {
        bindingSource.DataSource = reader;
        dgvProducts.DataSource = bindingSource;
    }
    else
    {
        MessageBox.Show("No products found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    connection.Close();
}
}
highlightProductsStock();

```

```

    }
    private void dgvProducts_CellClick(object sender,
DataGridViewCellEventArgs e)
    {
        index = e.RowIndex;
        int orderItemsIndex = 0;
        if (index < 0)
        {
            index = 0;
        }
        refreshProductInfo(index);

        if (lsbShoppingList.Items.Count != 0) // Highlights the selected
product if it has been added to the shopping list
        {
            OrderItem orderItem = new OrderItem(currentProduct,
Convert.ToInt32(numQuantity.Value), 0, 0);
            bool found = false;
            for (int i = 0; i < lsbShoppingList.Items.Count; i++)
            {
                if (((OrderItem)lsbShoppingList.Items[i]).product.id ==
orderItem.product.id)
                {
                    orderItemsIndex = i;
                    found = true;
                    break;
                }
            }
            if (!found)
            {
                lsbShoppingList.SelectedItem = null;
                lsbQuantityList.SelectedItem = null;
                lsbCostList.SelectedItem = null;
                numQuantity.Value = 0;
            }
            else
            {
                numQuantity.Value =
Convert.ToInt32(((OrderItem)lsbQuantityList.Items[orderItemsIndex]).quantity
);
                lsbShoppingList.SelectedIndex = orderItemsIndex;
                lsbQuantityList.SelectedIndex = orderItemsIndex;
                lsbCostList.SelectedIndex = orderItemsIndex;
            }
        }
    }
    private void refreshProductInfo(int index)
    {
        if (index < 0)
        {
            index = 0;
        }
        DataGridViewRow selectedRow = dgvProducts.Rows[index];
        currentProduct = new
Product(Convert.ToInt32(selectedRow.Cells[0].Value),
selectedRow.Cells[1].Value.ToString(),
Convert.ToSingle(selectedRow.Cells[2].Value),

```

```

Convert.ToInt32(selectedRow.Cells[3].Value),
selectedRow.Cells[10].Value.ToString(),
Convert.ToInt32(selectedRow.Cells[4].Value),
Convert.ToBoolean(selectedRow.Cells[11].Value),
Convert.ToInt32(selectedRow.Cells[12].Value),
Convert.ToInt32(selectedRow.Cells[13].Value));

    currentSupplier = currentProduct.Supplier;
    tbxProductID.Text = currentProduct.id.ToString();
    tbxProductName.Text = currentProduct.name;
    tbxCost.Text = currentProduct.CostString;
    tbxStock.Text = currentProduct.stock.ToString();
    tbxSupplier.Text = currentSupplier.supplierName;
    tbxType.Text = selectedRow.Cells[5].Value.ToString();
    tbxCategory.Text = selectedRow.Cells[6].Value.ToString();
    tbxColour.Text = selectedRow.Cells[7].Value.ToString();
    tbxFibre.Text = selectedRow.Cells[8].Value.ToString();
    tempProductType = selectedRow.Cells[9].Value.ToString();
    pbxImage.Refresh();

    if (currentProduct.discontinued) // Indicates if the product has
been discontinued
    {
        lblDiscontinued.Visible = true;
    }
    else
    {
        lblDiscontinued.Visible = false;
    }

    if (currentProduct.OutOfStock) // Indicates if the product is
out of stock
    {
        lblStockLevel.Visible = true;
        lblStockLevel.Text = "Out Of Stock";
    }
    else if (currentProduct.IsLowStock) // Indicates if the product
is low on stock
    {
        lblStockLevel.Visible = true;
        lblStockLevel.Text = "Low Stock";
    }
    else
    {
        lblStockLevel.Visible = false;
    }

    if (tempProductType == "y")
    {
        showYarnFields(false);
    }
    if (tempProductType == "a")
    {
        showAccFields(false);
    }

}
private void refreshProductDGV()

```

```

{
    //loads all products into the dgv
    tbxSearch.Text = "";
    string type = "";
    if (rbtnAcc.Checked)
    {
        type = "a";
    }
    if (rbtnYarn.Checked)
    {
        type = "y";
    }
    if (rbtnAll.Checked)
    {

        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
Discontinued = False OR (Discontinued = True AND Stock > 0)";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        if (reader.HasRows)
        {
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No results found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
    else
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND Discontinued = False OR (Discontinued = True AND Stock
> 0)";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", type);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        if (reader.HasRows)
        {
            bindingSource.DataSource = reader;

```

```

        dgvProducts.DataSource = bindingSource;
    }
    else
    {
        MessageBox.Show("No results found.", "No Results",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    connection.Close();
}
highlightProductsStock();
}
private void highlightProductsStock() // Highlights which products
are out of stock or low on stock in the dgv
{
    foreach (DataGridViewRow row in dgvProducts.Rows)
    {
        if (Convert.ToInt32(row.Cells[3].Value) == 0)
        {
            row.DefaultCellStyle.BackColor = Color.DarkRed;
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlLightLight;
        }
        else if (Convert.ToInt32(row.Cells[3].Value) <
Convert.ToInt32(row.Cells[12].Value))
        {
            row.DefaultCellStyle.BackColor = Color.FromArgb(199,
110, 0);
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlLightLight;
        }
        else
        {
            row.DefaultCellStyle.BackColor =
SystemColors.ControlLightLight;
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlText;
        }
    }
}
private void showCustomerInfo(int index)
{
    tbxCustID.Text = customerList[index].id.ToString();
    tbxFullName.Text = customerList[index].fullName.ToString();
    tbxPhoneNum.Text = customerList[index].phoneNum.ToString();
    tbxEmailAdd.Text = customerList[index].emailAddress.ToString();
    tbxMemType.Text = customerList[index].memType.ToString();
    tbxTotalStars.Text = customerList[index].totalStars.ToString();
    tbxVouchers.Text = customerList[index].voucherNum.ToString();
    int tempCustID = Convert.ToInt32(tbxCustID.Text);
    foreach (Customer selectCust in customerList)
    {
        if (selectCust.id == tempCustID)
        {
            currentCust = selectCust;
        }
    }
    if(currentCust.memType == "Guest")

```



```

        {
            rbtnExistingCust.Checked = false;
            rbtnGuest.Checked = true;
            cbxCustomers.Enabled = false;
        }
    }

    private void updateOrderTotals(double totalToCalc) // Automatically
updates totals shown when a voucher is added or removed
    {
        if(numudVouchers.Value == 1) //if adding a voucher
        {
            if (currentCust.memType == "Gold")
            {
                lblVoucherUsed.Text = "-15% OFF";
                tbxSavings.Text = "-£"+Math.Round(totalToCalc * 0.15,
2).ToString("0.00");
                totalToCalc = 0.85 * totalToCalc;
            }
            else if (currentCust.memType == "Silver")
            {
                lblVoucherUsed.Text = "-10% OFF";
                tbxSavings.Text = "-£" + Math.Round(total * 0.1,
2).ToString("0.00");
                totalToCalc = 0.9 * totalToCalc;
            }
            else if (currentCust.memType == "Bronze")
            {
                lblVoucherUsed.Text = "-5% OFF";
                tbxSavings.Text = "-£" + Math.Round(total * 0.05,
2).ToString("0.00");
                totalToCalc = 0.95 * totalToCalc;
            }
        }
        else
        {
            lblVoucherUsed.Text = "";
            tbxSavings.Text = "";
            totalToCalc = subtotal;
        }
        // if none of these
        tbxSubtotal.Text = "£" + Math.Round(subtotal,
2).ToString("0.00");
        total = Math.Round(totalToCalc, 2);
        tbxTotal.Text = "£" + Math.Round(totalToCalc,
2).ToString("0.00");
        tbxStarsEarned.Text = Convert.ToString(Math.Round(total * 3));
    }

    // FILTERS
    private void showYarnFields(bool filter)
    {
        if (filter) // If there are other filters in place
        {
            clsbCategory.Visible = true;
            clsbFibreMaterial.Visible = true;
            clsbColour.Visible = true;
        }
    }

```

```

        clsbCategory.DataSource = YarnWeightList;
        clsbFibreMaterial.DataSource = FibreList;
        clsbColour.DataSource = ColourList;

        lblCategoryFilter.Text = "YARN WEIGHT";
        lblFibreMaterialFilter.Text = "FIBRE";
        lblCategoryFilter.Visible = true;
        lblFibreMaterialFilter.Visible = true;
        lblColourFilter.Visible = true;

        clearChecked(clsbCategory);
        clearChecked(clsbFibreMaterial);
        clearChecked(clsbColour);
        clearChecked(clsbSuppliers);

        chbxDiscontinuedFilter.Checked = false;

        pnlFilters.AutoScroll = true;
        lblCategoryFilter.Visible = true;
        lblFibreMaterialFilter.Visible = true;
        lblColourFilter.Visible = true;
    }

    lblType.Text = "Ball Weight";
    lblCategory.Text = "Yarn Weight";
    lblFibre.Text = "Fibre";
}

private void showAccFields(bool filter)
{
    if (filter) // If there are other filters in place
    {
        clsbCategory.Visible = true;
        clsbFibreMaterial.Visible = true;
        clsbColour.Visible = true;

        clsbCategory.DataSource = CategoryList;
        clsbFibreMaterial.DataSource = MaterialList;
        clsbColour.DataSource = ColourList;

        lblCategoryFilter.Text = "CATEGORY";
        lblFibreMaterialFilter.Text = "MATERIAL";
        lblCategoryFilter.Visible = true;
        lblFibreMaterialFilter.Visible = true;
        lblColourFilter.Visible = true;

        clearChecked(clsbCategory);
        clearChecked(clsbFibreMaterial);
        clearChecked(clsbColour);
        clearChecked(clsbSuppliers);

        chbxDiscontinuedFilter.Checked = false;

        pnlFilters.AutoScroll = true;
        lblCategoryFilter.Visible = true;
        lblFibreMaterialFilter.Visible = true;
    }
}

```

```

        lblColourFilter.Visible = true;
    }

    lblType.Text = "Size";
    lblCategory.Text = "Category";
    lblFibre.Text = "Material";
}
private void clearChecked(CheckedListBox clsbToClear)
{
    for (int i = 0; i < clsbToClear.Items.Count; i++)
    {
        clsbToClear.SetSelected(i, false);
        clsbToClear.SetItemChecked(i, false);
    }
    clsbToClear.ClearSelected();
}
private void rbtnYarn_CheckedChanged(object sender, EventArgs e) //
Shows all yarn related fields
{
    if (rbtnYarn.Checked == true)
    {
        showYarnFields(true);
        refreshProductDGV();
        refreshProductInfo(0);
    }
}
private void rbtnAcc_CheckedChanged(object sender, EventArgs e) //
Shows all accessory related fields
{
    if (rbtnAcc.Checked == true)
    {
        showAccFields(true);
        refreshProductDGV();
        refreshProductInfo(0);
    }
}
private void rbtnAll_Click(object sender, EventArgs e) // Shows
general fields for all products and clears filters
{
    // If there are any other filters selected, clicking the all
    radio button will clear all filters selected.
    // A message box will be shown if any filters apart from Yarn
    and Accessories are selected.
    DialogResult result = MessageBox.Show("This will clear all
    filters. Proceed?", "Clear Filters", MessageBoxButtons.YesNo,
    MessageBoxIcon.Warning);
    tbxSearch.Text = "";
    if (result == DialogResult.Yes)
    {
        refresh();
    }
    else
    {
        MessageBox.Show("Filters not cleared.");
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
        Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    }
}

```

```

        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND (Discontinued = False OR (Discontinued = True AND
Stock > 0))";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows) //in case all accessories or yarn
products have been deleted
        {
            BindingSource bindingSource = new BindingSource();
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
            if (tempProductType == "y")
            {
                rbtnYarn.Checked = true;
                showYarnFields(false);
            }
            if (tempProductType == "a")
            {
                rbtnAcc.Checked = true;
                showAccFields(false);
            }
        }
        else
        {
            DialogResult noProducts = MessageBox.Show("No results
found.", "No Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
            rbtnAll.Checked = true;
            rbtnYarn.Checked = false;
        }
        connection.Close();
    }
    highlightProductsStock();
}
private void btnApplyFilters_Click(object sender, EventArgs e)
{
    filtersApplied = false;
    string baseCommandText = "SELECT * FROM TableProducts WHERE";
    string categoryFilterCommand = " AND Category IN (";
    string fibreFilterCommand = " AND FibreMaterial IN (";
    string colourFilterCommand = " AND Colour IN (";
    string supplierFilterCommand = " AND SupplierID IN (";

    // Need these lists to store the selected items for each checked
list box.
    // Since a new list is created each time the user clicks the
apply button, the list will be cleared each time.
    // Load all the checked filters into their corresponding lists.
    List<string> selectedCategoryFilters =
clsbCategory.CheckedItems.Cast<string>().ToList();
    List<string> selectedFibreFilters =
clsbFibreMaterial.CheckedItems.Cast<string>().ToList();

```

```

        List<string> selectedColourFilters =
clsbColour.CheckedItems.Cast<string>().ToList();
        List<string> selectedStockFilters =
clsbStock.CheckedItems.Cast<string>().ToList();
        List<Supplier> selectedSupplierFilters =
clsbSuppliers.CheckedItems.Cast<Supplier>().ToList();

        bool noStock = false;
        if (selectedStockFilters.Count != 0)
        {
            filtersApplied = true;
            foreach (string stockFilter in selectedStockFilters)
            {
                string condition = "";
                switch (stockFilter)
                {
                    case "In Stock":
                        condition = " Stock > 0";
                        break;
                    case "No Stock":
                        condition = " Stock = 0";
                        noStock = true;
                        break;
                    case "Low Stock":
                        condition = " Stock < LowStock";
                        break;
                }
                if (baseCommandText.EndsWith("E"))
                {
                    baseCommandText += " (" + condition;
                }
                else
                {
                    baseCommandText += " OR" + condition;
                }
            }
        }
        else
        {
            baseCommandText += " (Stock > 0";
        }
        if (chbxDiscontinuedFilter.Checked)
        {
            filtersApplied = true;
            baseCommandText += " AND Discontinued = True)";
        }
        else // Shows active products that are in stock by default
        {
            if (noStock)
            {
                baseCommandText += ") AND (Discontinued = False OR
Discontinued = True)";
            }
            else
            {
                baseCommandText += ") AND (Discontinued = False OR
(Discontinued = True AND Stock > 0))";
            }
        }
    }
}

```

```

    }
}
if (!rbtnAll.Checked)
{
    if (baseCommandText.EndsWith("E"))
    {
        baseCommandText += " ProductType = @ya";
    }
    else
    {
        baseCommandText += " AND ProductType = @ya";
    }
}
// Building the command string

if (selectedCategoryFilters.Count != 0)
{
    baseCommandText +=
appendFilterCommand(categoryFilterCommand, selectedCategoryFilters) + ",";
}
if (selectedFibreFilters.Count != 0)
{
    baseCommandText += appendFilterCommand(fibreFilterCommand,
selectedFibreFilters) + ",";
}
if (selectedColourFilters.Count != 0)
{
    baseCommandText += appendFilterCommand(colourFilterCommand,
selectedColourFilters) + ",";
}
if (selectedSupplierFilters.Count != 0)
{
    filtersApplied = true;
    foreach (Supplier supplierFilter in selectedSupplierFilters)
    {
        // If there isn't anything in the string so far, don't
want it to start with a comma.
        if (supplierFilterCommand.EndsWith("("))
        {
            supplierFilterCommand += supplierFilter.id;
        }
        // Add the checked filter to the command string.
        else
        {
            supplierFilterCommand += "," + supplierFilter.id;
        }
    }
    baseCommandText += supplierFilterCommand + ",";
}

// Running the queries
if (rbtnYarn.Checked)
{
    tempProductType = "y";
}

```

```

        if (rbtnAcc.Checked)
        {
            tempProductType = "a";
        }

        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = baseCommandText;
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        if (reader.HasRows)
        {
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
            currentCommand = baseCommandText;
            tbxSearch.Text = "";
            refreshProductInfo(0);
        }
        else
        {
            MessageBox.Show("No products found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
        highlightProductsStock();

    }
    private string appendFilterCommand(string filterCommand,
List<string> selectedFilters)
    {
        filtersApplied = true;
        foreach (string filter in selectedFilters)
        {
            // If there isn't anything in the string so far, don't want
it to start with a comma.
            if (filterCommand.EndsWith("("))
            {
                filterCommand += "'" + filter + "'";
            }
            // Add the checked filter to the command string.
            else
            {
                filterCommand += "," + "'" + filter + "'";
            }
        }
        return filterCommand;
    }
}

```

```

        private void btnClearFilters_Click(object sender, EventArgs e) //
Clears all filters
        {
            DialogResult result = MessageBox.Show("This will clear all
filters. Proceed?", "Clear All Filters", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
            if (result == DialogResult.Yes)
            {
                clearChecked(clsbCategory);
                clearChecked(clsbFibreMaterial);
                clearChecked(clsbColour);
                clearChecked(clsbSuppliers);
                clearChecked(clsbStock);
                chbxDiscontinuedFilter.Checked = false;

                if (rbtnAll.Checked)
                {
                    OleDbConnection connection = new OleDbConnection();
                    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                    OleDbCommand cmd = connection.CreateCommand();
                    OleDbDataReader reader;
                    connection.Open();
                    cmd.CommandText = "SELECT * FROM TableProducts WHERE
(Discontinued = False OR (Discontinued = True AND Stock > 0))";
                    cmd.Parameters.Clear();
                    cmd.Parameters.AddWithValue("@ya", "y");
                    cmd.Connection = connection;
                    reader = cmd.ExecuteReader();
                    BindingSource bindingSource = new BindingSource();
                    bindingSource.DataSource = reader;
                    dgvProducts.DataSource = bindingSource;
                    connection.Close();
                }
                else
                {
                    if (rbtnYarn.Checked)
                    {
                        tempProductType = "y";
                    }
                    if (rbtnAcc.Checked)
                    {
                        tempProductType = "a";
                    }
                    OleDbConnection connection = new OleDbConnection();
                    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                    OleDbCommand cmd = connection.CreateCommand();
                    OleDbDataReader reader;
                    connection.Open();
                    cmd.CommandText = "SELECT * FROM TableProducts WHERE
(Discontinued = False OR (Discontinued = True AND Stock > 0)) AND
ProductType = @ya";
                    cmd.Parameters.Clear();
                    cmd.Parameters.AddWithValue("@ya", tempProductType);
                    cmd.Connection = connection;
                    reader = cmd.ExecuteReader();

```



```

        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvProducts.DataSource = bindingSource;
        connection.Close();
    }
    filtersApplied = false;
    currentCommand = "";
}
else
{
    MessageBox.Show("No filters were deleted.", "Clear All
Filters", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
highlightProductsStock();
}

// CUSTOMER INFORMATION
private void cbxCustomers_SelectionChangeCommitted(object sender,
EventArgs e)
{
    if (cbxCustomers.SelectedIndex < 0)
    {
        cbxCustomers.SelectedIndex = 0;
    }
    showCustomerInfo(cbxCustomers.SelectedIndex);

    tbxTotal.Text = tbxSubtotal.Text;
    total = subtotal;
    numudVouchers.Value = 0;
    updateOrderTotals(subtotal);
    tbxSavings.Text = "";
    lblVoucherUsed.Text = "";
    custChanged = true;
}
private void rbtnExistingCust_CheckedChanged(object sender,
EventArgs e)
{
    if (rbtnExistingCust.Checked)
    {
        cbxCustomers.SelectedIndex = 1;
        cbxCustomers.SelectedIndex = 0;
        cbxCustomers.Enabled = true;
    }
}
private void btnNewCustomer_Click(object sender, EventArgs e)
{
    NewCustomer formNewCustomer = new NewCustomer();
    formNewCustomer.ShowDialog();
    refresh();
}
private void rbtnGuest_CheckedChanged(object sender, EventArgs e)
{
}

// ORDER DATE INFORMATION
private void chbxInStore_CheckedChanged(object sender, EventArgs e)

```

```

    {
        if (chbxInStore.Checked)
        {
            dtpExpDeliveryDate.Value = dtpOrderDate.Value;
            dtpColDate.Value = dtpOrderDate.Value;
            dtpExpDeliveryDate.Enabled = false;
            dtpColDate.Enabled = false;
            cbxDelStatus.Text = "Collected";
            tbxStatus.Text = "Archived";
            tbxPayStatus.Text = "Paid";

        }
        else
        {
            dtpExpDeliveryDate.Enabled = true;
            dtpColDate.Enabled = true;
            cbxDelStatus.Enabled = true;
            dtpExpDeliveryDate.Value = dtpOrderDate.Value.AddDays(3);
            dtpColDate.Value = dtpExpDeliveryDate.Value.AddDays(7);
        }
    }
    private void dtpOrderDate_ValueChanged(object sender, EventArgs e)
    {
        if (chbxInStore.Checked)
        {
            dtpExpDeliveryDate.Value = dtpOrderDate.Value;
            dtpColDate.Value = dtpOrderDate.Value;
        }
        else
        {
            dtpColDate.Value = dtpOrderDate.Value.AddDays(10);
            dtpExpDeliveryDate.Value = dtpOrderDate.Value.AddDays(3);
        }
    }
    private void dtpExpDeliveryDate_ValueChanged(object sender,
    EventArgs e)
    {
        dtpColDate.Value = dtpExpDeliveryDate.Value.AddDays(7);
    }

    // ADDING ITEMS TO ORDER AND PLACING ORDER
    private void btnAddProduct_Click(object sender, EventArgs e)
    {
        bool productInOrder = false;
        int initialQuantity = Convert.ToInt32(numQuantity.Value);
        int finalIndex = 0;
        if (currentProduct.stock < initialQuantity+1)
        {
            MessageBox.Show("Insufficient stock.", "Insufficient Stock",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        }
        else
        {
            if (orderList.Count == 0) // If there are no products in the
            order, create an order item, set the quantity to 1 and add it to the order
            list. Add the cost of one product to the subtotal.

```

```

        {
            OrderItem orderItem = new OrderItem(currentProduct, 1,
0, 0);

            numQuantity.Value = 1;
            orderList.Add(orderItem);
            subtotal += orderItem.product.cost;
            lsbShoppingList.SelectedIndex =
lsbShoppingList.Items.Count - 1;
            lsbQuantityList.SelectedIndex =
lsbShoppingList.SelectedIndex;
            lsbCostList.SelectedIndex =
lsbShoppingList.SelectedIndex;
        }
        else
        {

            if (initialQuantity == 0)
            {
                initialQuantity = 1;
            }
            else
            {
                initialQuantity =
Convert.ToInt32(numQuantity.Value);
            }
            foreach (OrderItem item in orderList) // Loops through
each item in the order to find the product that has been selected by the
user.
            {
                if (item.product.id == currentProduct.id) // If the
product selected by the user is already in the order, the order item
quantity is increased by 1 and the search for the product selected is
broken.
                {
                    // There is a maximum value set to the numeric
up down button.

                    // If incrementing the quantity results in
numQuantity storing a value larger than its maximum,
                    //it will notify that there is no stock left for
that product and set the item quantity to its maximum.
                    productInOrder = true;
                    if (!quantityChanged) // Ensures that one extra
product is not added if the quantity has manually been changed.
                    {
                        initialQuantity++;
                        subtotal += item.product.cost;
                    }
                    numQuantity.Value = initialQuantity;

                    item.SetQuantity(initialQuantity);
                    int orderItemsIndex = 0;
                    for (int i = 0; i < lsbShoppingList.Items.Count;
i++)
                    {
                        if
(((OrderItem)lsbShoppingList.Items[i]).product.id == item.product.id)
                        {

```

```

                orderItemsIndex = i;
                break;
            }
        }
        finalIndex = orderItemsIndex;
        break;
    }
}
if (!productInOrder) // If the product selected by the
user is not in the order
{
    OrderItem orderItem = new OrderItem(currentProduct,
initialQuantity, 0, 0);
    numQuantity.Value = initialQuantity;
    orderList.Add(orderItem);
    subtotal +=
orderItem.product.cost*Convert.ToDouble(initialQuantity);
    addingProduct = true;
    lsbQuantityList.DataSource = null;
    lsbQuantityList.DisplayMember = "QuantityString";
    lsbQuantityList.DataSource = orderList;
    lsbQuantityList.Refresh();
    lsbCostList.DataSource = null;
    lsbCostList.DisplayMember = "CostString";
    lsbCostList.DataSource = orderList;
    lsbCostList.Refresh();
    lsbShoppingList.DataSource = null;
    lsbShoppingList.DisplayMember = "name";
    lsbShoppingList.DataSource = orderList;
    lsbShoppingList.Refresh();
    finalIndex = lsbShoppingList.Items.Count - 1;
}
}
lsbQuantityList.DataSource = null;
lsbQuantityList.DisplayMember = "QuantityString";
lsbQuantityList.DataSource = orderList;
lsbQuantityList.Refresh();
lsbCostList.DataSource = null;
lsbCostList.DisplayMember = "CostString";
lsbCostList.DataSource = orderList;
lsbCostList.Refresh();
lsbShoppingList.DataSource = null;
lsbShoppingList.DisplayMember = "name";
lsbShoppingList.DataSource = orderList;
lsbShoppingList.Refresh();
if (lsbShoppingList.Items.Count > 0)
{
    lsbShoppingList.SelectedIndex = finalIndex;
    lsbQuantityList.SelectedIndex =
lsbShoppingList.SelectedIndex;
    lsbCostList.SelectedIndex = lsbShoppingList.SelectedIndex;
}
updateOrderTotals(subtotal);
quantityChanged = false;
}

```

```

        private void btnApply_Click(object sender, EventArgs e) // Applies
the quantity of the product that the user has set
        {
            int index = lsbShoppingList.SelectedIndex;
            if(index > -1 && lsbShoppingList.Items.Count > 0)
            {
                // Adjusts the total of the order accordingly
                int originalQuantity =
                ((OrderItem)lsbShoppingList.Items[lsbShoppingList.SelectedIndex]).quantity;
                if (numQuantity.Value == 0)
                {
                    DialogResult result = MessageBox.Show("Remove product?",
"Remove Product", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
                    if (result == DialogResult.Yes)
                    {
                        index = 0;
                        subtotal += -originalQuantity *
                ((OrderItem)lsbShoppingList.Items[lsbShoppingList.SelectedIndex]).product.co
st;

orderList.Remove((OrderItem)lsbShoppingList.Items[lsbShoppingList.SelectedIn
dex]);

                        numQuantity.Enabled = false;
                    }
                }
                else
                {
                    if (lsbShoppingList.SelectedItem != null)
                    {
                        subtotal += ((float)numQuantity.Value -
originalQuantity) *
                ((OrderItem)lsbShoppingList.Items[lsbShoppingList.SelectedIndex]).product.co
st;

orderList[lsbShoppingList.SelectedIndex].SetQuantity(Convert.ToInt32(numQuan
tity.Value));

                    }
                }
                lsbQuantityList.DataSource = null;
                lsbQuantityList.DisplayMember = "QuantityString";
                lsbQuantityList.DataSource = orderList;
                lsbQuantityList.Refresh();
                lsbCostList.DataSource = null;
                lsbCostList.DisplayMember = "CostString";
                lsbCostList.DataSource = orderList;
                lsbCostList.Refresh();
                lsbShoppingList.DataSource = null;
                lsbShoppingList.DisplayMember = "name";
                lsbShoppingList.DataSource = orderList;
                lsbShoppingList.Refresh();
                if (lsbShoppingList.Items.Count > 0)
                {
                    lsbShoppingList.SelectedIndex = index;
                    lsbQuantityList.SelectedIndex =
lsbShoppingList.SelectedIndex;

```

```

        lsbCostList.SelectedIndex =
lsbShoppingList.SelectedIndex;
        numQuantity.Enabled = false;
    }
    updateOrderTotals(subtotal);
}
quantityChanged = false;

}
private void numQuantity_ValueChanged(object sender, EventArgs e)
{
    if(lsbShoppingList.SelectedIndex != -1)
    {
        if(lsbShoppingList.SelectedItem != null)
        {
            if(numQuantity.Value ==
((OrderItem)lsbShoppingList.SelectedItem).quantity)
            {
                quantityChanged = false;
            }
            else
            {
                quantityChanged = true;
            }
        }
    }
    if (numQuantity.Value > currentProduct.stock)
    {
        MessageBox.Show("Insufficient stock.", "Insufficient Stock",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
        numQuantity.Value = currentProduct.stock;
    }
}
private void btnNewProduct_Click(object sender, EventArgs e) //
Allows the user to create a new product to be added to the order
{
    NewProduct formNewProduct = new
NewProduct(false,currentSupplier.id);
    formNewProduct.ShowDialog();
    refreshProductDGV();
}
private void btnPlaceOrder_Click(object sender, EventArgs e)
{
    //These conditions will mainly come in place when the user
changes the expected delivery date themselves, and the date is invalid.
    if (lsbShoppingList.Items.Count == 0)
    {
        MessageBox.Show("There are no products added to the order.",
"No Products", MessageBoxButtons.OK,MessageBoxIcon.Warning);
    }
    else if (dtpExpDeliveryDate.Value < dtpOrderDate.Value)
    {
        MessageBox.Show("You cannot enter an expected delivery date
that falls before the order date.", "Invalid Date", MessageBoxButtons.OK,
MessageBoxIcon.Error);
        dtpExpDeliveryDate.Value = dtpOrderDate.Value.AddDays(3);
        dtpColDate.Value = dtpExpDeliveryDate.Value.AddDays(7);
    }
}

```

```

    }
    else if (dtpColDate.Value < dtpOrderDate.Value)
    {
        MessageBox.Show("You cannot enter a collection end date that
falls before the order date.", "Invalid Date", MessageBoxButtons.OK,
MessageBoxIcon.Error);
        dtpColDate.Value = dtpExpDeliveryDate.Value.AddDays(7);
    }
    else if (dtpColDate.Value < dtpExpDeliveryDate.Value.AddDays(-
7))
    {
        MessageBox.Show("You cannot enter a collection end date that
falls before the expected delivery date.", "Invalid Date",
MessageBoxButtons.OK, MessageBoxIcon.Error);
        dtpColDate.Value = dtpExpDeliveryDate.Value.AddDays(7);
    }
    else if (string.IsNullOrEmpty(tbxTotal.Text) ||
string.IsNullOrEmpty(cbxDelStatus.Text))
    {
        MessageBox.Show("Enter values in all required fields.",
"Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    else
    {
        if (rtbxOrderNotes.Text == "")
        {
            rtbxOrderNotes.Text = " ";
        }
        if (tbxStarsEarned.Text == "")
        {
            tbxStarsEarned.Text = "0";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        connection.Open();
        cmd.CommandText = "INSERT INTO TableCustOrders(CustID,
OrderDate, DeliveryDate, DeliveryStatus, PaymentStatus, CollectionDateEnd,
StarsEarned, Subtotal, Total, Voucher, OrderNotes)
VALUES (@cid,@ord,@exd,@dsa,@psa,@cod,@ste,@sub,@cos,@vou,@orn)";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@cid", currentCust.id);
        cmd.Parameters.AddWithValue("@ord",
dtpOrderDate.Value.ToString("dd/MM/yyyy"));
        cmd.Parameters.AddWithValue("@exd",
dtpExpDeliveryDate.Value.ToString("dd/MM/yyyy"));
        cmd.Parameters.AddWithValue("@dsa", cbxDelStatus.Text);
        cmd.Parameters.AddWithValue("@psa", tbxPayStatus.Text);
        cmd.Parameters.AddWithValue("@cod",
dtpColDate.Value.ToString("dd/MM/yyyy"));
        cmd.Parameters.AddWithValue("@ste",
Convert.ToInt32(tbxStarsEarned.Text));
        cmd.Parameters.AddWithValue("@sub", Math.Round(subtotal,
2));
        cmd.Parameters.AddWithValue("@cos", Math.Round(total, 2));
    }
}

```

```

        if (numudVouchers.Value == 0)
        {
            cmd.Parameters.AddWithValue("@vou", "N/A");
        }
        else
        {
            cmd.Parameters.AddWithValue("@vou",
currentCust.memType.ToUpper());
        }
        cmd.Parameters.AddWithValue("@orn", rtbxOrderNotes.Text);
        cmd.Connection = connection;
        int status = cmd.ExecuteNonQuery();

        // this is to get the order ID of the order that has just
        // been made, so that it can be used for the order items table
        cmd.CommandText = "SELECT CustOrderID FROM TableCustOrders
ORDER BY CustOrderID DESC";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@cid", currentCust.id);
        cmd.Parameters.AddWithValue("@od", dtpOrderDate.Value);
        cmd.Parameters.AddWithValue("@dd",
dtpExpDeliveryDate.Value);
        int tempOrderID = Convert.ToInt32(cmd.ExecuteScalar());

        // MEMBERSHIP SYSTEM

        // update membership details of customer
        int originalTotalStars = currentCust.totalStars;
        currentCust.SetTotalStars(currentCust.totalStars +
Convert.ToInt32(Convert.ToDouble(total) * 3));
        int voucherChange = (currentCust.totalStars -
originalTotalStars) / 150;
        if (currentCust.memType == "Bronze" &&
currentCust.totalStars >= 450 && currentCust.totalStars <= 1350)
        {
            //When a Bronze customer reaches 450 stars, they go up
            //to the next tier and earn 1 extra voucher for going up one tier.
            currentCust.SetMemType("Silver");
            currentCust.SetTotalStars(currentCust.totalStars-450);
            currentCust.SetVoucherNum(currentCust.voucherNum +
voucherChange + 1);
        }
        if (currentCust.memType == "Bronze" &&
currentCust.totalStars >= 1350)
        {
            //When a Bronze customer reaches 1350 stars, they go up
            //to the Gold tier and earn 2 extra vouchers for going up two tiers.
            currentCust.SetMemType("Gold");
            currentCust.SetTotalStars(currentCust.totalStars -
1350);
            currentCust.SetVoucherNum(currentCust.voucherNum +
voucherChange + 2);
        }
        else if (currentCust.memType == "Silver" &&
currentCust.totalStars >= 900)
        {

```



```

        //When a Silver customer reaches 900 stars, they go up
        to the next tier and earn 1 extra voucher for going up one tier.
        currentCust.SetMemType("Gold");
        currentCust.SetTotalStars(currentCust.totalStars - 900);
        currentCust.SetVoucherNum(currentCust.voucherNum +
voucherChange + 1);
    }

    //this command will update the total stars the customer has
    cmd.CommandText = "UPDATE TableCustomers SET MemType =
@met,TotalStars = @sta, VoucherNum = @von WHERE CustID = @cid";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@met", currentCust.memType);
    cmd.Parameters.AddWithValue("@sta", currentCust.totalStars);
    cmd.Parameters.AddWithValue("@von", currentCust.voucherNum);
    cmd.Parameters.AddWithValue("@cid", currentCust.id);
    cmd.Connection = connection;
    int status1 = cmd.ExecuteNonQuery();

    // QUANTITY CALCULATING

    List<int> savedProductsID = new List<int>(); // this is to
    keep track of which products have been saved in the shopping list
    foreach (OrderItem shoppingListItem in orderList)
    {
        if
        (savedProductsID.Contains(shoppingListItem.product.id)) // if the product
        has not already been searched for and recorded
        {
            continue;
        }
        else
        {
            savedProductsID.Add(shoppingListItem.product.id);
            cmd.CommandText = "INSERT INTO
TableCustOrderItems(ProductID,OrderID,Quantity) VALUES(@pid,@oid,@q)";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@pid",
((OrderItem)shoppingListItem).product.id);
            cmd.Parameters.AddWithValue("@oid", tempOrderID);
            cmd.Parameters.AddWithValue("@q",
shoppingListItem.quantity);
            int status2 = cmd.ExecuteNonQuery();

            cmd.CommandText = "UPDATE TableProducts SET Stock =
Stock - @q WHERE ProductID = @pid";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@q",
shoppingListItem.quantity);
            cmd.Parameters.AddWithValue("@pid",
((OrderItem)shoppingListItem).product.id);
            int statusStock = cmd.ExecuteNonQuery();
        }
    }
    connection.Close();

```

```

        MessageBox.Show("Order placed successfully.", "Order
Placed", MessageBoxButtons.OK, MessageBoxIcon.Information);
        Close();
    }
}
private void removeToolStripMenuItem1_Click(object sender, EventArgs
e)
{
    OrderItem orderItem = (OrderItem)lsbShoppingList.SelectedItem;
    if (orderItem != null)
    {
        // removes the product and adjusts the totals shown
        orderList.Remove(orderItem);
        lsbShoppingList.DataSource = null;
        lsbShoppingList.DisplayMember = "name";
        lsbShoppingList.DataSource = orderList;
        lsbShoppingList.Refresh();
        lsbQuantityList.DataSource = null;
        lsbQuantityList.DisplayMember = "QuantityString";
        lsbQuantityList.DataSource = orderList;
        lsbQuantityList.Refresh();
        lsbCostList.DataSource = null;
        lsbCostList.DisplayMember = "CostString";
        lsbCostList.DataSource = orderList;
        lsbCostList.Refresh();
        subtotal -= orderItem.product.cost* orderItem.quantity;
        updateOrderTotals(subtotal);
        numQuantity.Value = 0;
    }
}
private void lsbShoppingList_SelectedIndexChanged(object sender,
EventArgs e)
{
    // When the user clicks on another item in the list, the item is
highlighted in the dgv and all information for that product is shown
    if (lsbShoppingList.Items.Count > 0)
    {
        btnApply.Visible = true;
        numQuantity.Enabled = true;
        bool productInDGV = false;
        lsbQuantityList.SelectedIndex =
lsbShoppingList.SelectedIndex;
        lsbCostList.SelectedIndex = lsbShoppingList.SelectedIndex;
        if (lsbShoppingList.SelectedItem != null)
        {
            foreach (DataGridViewRow row in dgvProducts.Rows)
            {
                if (addingProduct)
                {
                    if (row.Cells[0].Value.ToString() ==
tbxProductID.Text)
                    {
                        dgvProducts.ClearSelection();
                        dgvProducts.CurrentCell =
dgvProducts.Rows[row.Index].Cells[0];
                        dgvProducts[0, row.Index].Selected = true;

```

```

        productInDGV = true;
    }
}
else
{
    if (row.Cells[0].Value.ToString() ==
((OrderItem)lsbShoppingList.SelectedItem).product.id.ToString())
    {
        dgvProducts.ClearSelection();
        dgvProducts.CurrentCell =
dgvProducts.Rows[row.Index].Cells[0];
        dgvProducts[0, row.Index].Selected = true;
        productInDGV = true;
    }
}
}
if (!productInDGV)
{
    if (rbtnYarn.Checked)
    {
        rbtnAcc.Checked = true;
        foreach (DataGridViewRow row in
dgvProducts.Rows)
        {
            if (row.Cells[0].Value.ToString() ==
((OrderItem)lsbShoppingList.SelectedItem).product.id.ToString())
            {
                dgvProducts.ClearSelection();
                dgvProducts.CurrentCell =
dgvProducts.Rows[row.Index].Cells[0];
                dgvProducts[0, row.Index].Selected =
true;
            }
        }
    }
    else if (rbtnAcc.Checked)
    {
        rbtnYarn.Checked = true;
        foreach (DataGridViewRow row in
dgvProducts.Rows)
        {
            if (row.Cells[0].Value.ToString() ==
((OrderItem)lsbShoppingList.SelectedItem).product.id.ToString())
            {
                dgvProducts.ClearSelection();
                dgvProducts.CurrentCell =
dgvProducts.Rows[row.Index].Cells[0];
                dgvProducts[0, row.Index].Selected =
true;
            }
        }
    }
}
numQuantity.Value =
Convert.ToInt32(((OrderItem)lsbQuantityList.Items[((int)lsbShoppingList.Sele
ctedIndex])).quantity);
refreshProductInfo(dgvProducts.CurrentRow.Index);

```

```

        addingProduct = false;
    }
}
else
{
    btnApply.Visible = false;
    numQuantity.Enabled = false;
}
}
private void numudVouchers_ValueChanged(object sender, EventArgs e)
{
    if (numudVouchers.Value == 0)
    {
        if (!custChanged)
        {
            currentCust.SetVoucherNum(currentCust.voucherNum + 1);
// if the selected customer for the order has been changed then they
shouldn't gain an extra voucher from this, as voucher is set to 0
        }
        tbxVouchers.Text = Convert.ToString(currentCust.voucherNum);
        updateOrderTotals(subtotal);
    }
    if (numudVouchers.Value == 1)
    {
        if (currentCust.voucherNum < 1)
        {
            numudVouchers.Value = 0;
        }
        else
        {
            currentCust.SetVoucherNum(currentCust.voucherNum - 1);
            tbxVouchers.Text =
Convert.ToString(currentCust.voucherNum);
            updateOrderTotals(subtotal);
        }
    }
}
private void cbxDelStatus_SelectionChangeCommitted(object sender,
EventArgs e) // Automatically sets the order status according to the
delivery status
{
    if (cbxDelStatus.SelectedItem.ToString() == "Collected")
    {
        tbxStatus.Text = "Archived";
        tbxPayStatus.Text = "Paid";
    }
    else
    {
        tbxStatus.Text = "Open";
        tbxPayStatus.Text = "Unpaid";
    }
}
private void pbxImage_Paint(object sender, PaintEventArgs e) //
Paints the image of the product currently shown
{
    bool imageError = false;

```

```

        try
        {
            if (currentProduct.filename != null)
            {
                try
                {
                    Image image = Image.FromFile("productImages/" +
currentProduct.filename);
                    e.Graphics.DrawImage(image, 0, 0, 217, 300);
                }
                catch
                {
                    Image image =
Image.FromFile("productImages/noImage.png");
                    e.Graphics.DrawImage(image, 0, 0, 217, 300);
                }
            }
            imageError = false;
        }
        catch
        {
            if (!imageError)
            {
                MessageBox.Show("Product image could not be loaded.",
"Image Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
                imageError = true;
            }
        }
    }

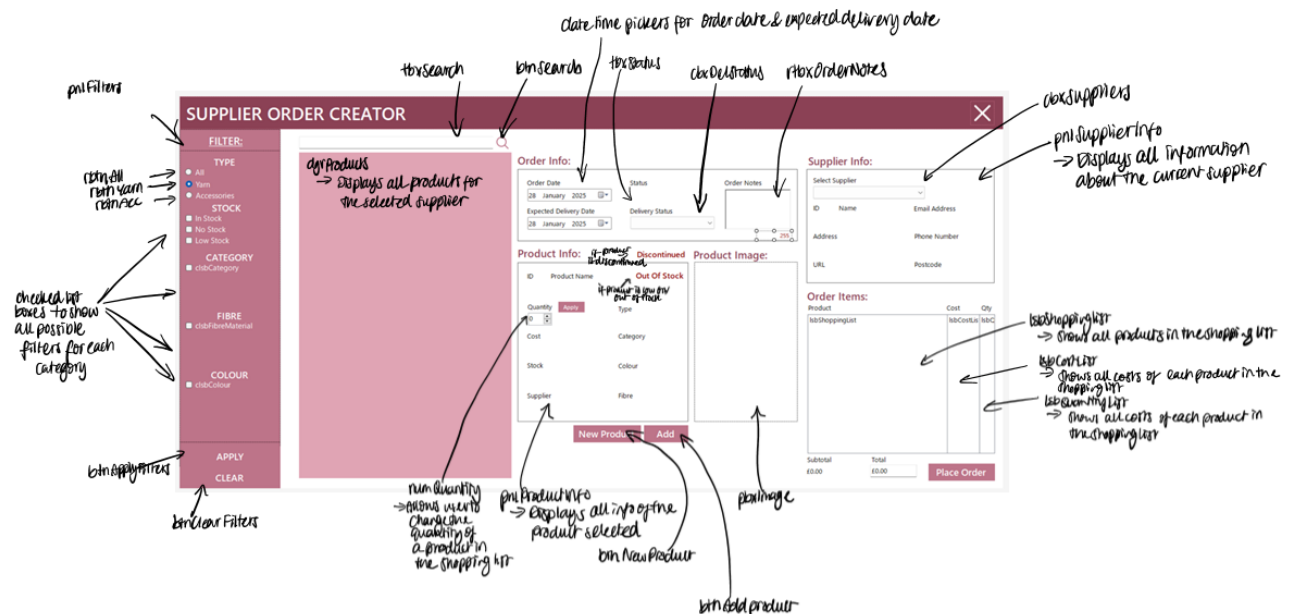
    private void rtbxOrderNotes_TextChanged(object sender, EventArgs e)
    {
        // Implements character length limit on order notes. It will
        show the user when the order notes' text length is reaching the limit of
        255.

        if (rtbxOrderNotes.TextLength > 244)
        {
            lblCharLimit.Visible = true;
            lblCharLimit.Text = rtbxOrderNotes.TextLength + "/255";
        }
        else
        {
            lblCharLimit.Visible = false;
        }
    }
}

```

NewSupplierOrder.cs

Form Design



Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace YarnShop
{
    public partial class NewSupplierOrder : Form
    {
        int index;
        double subtotal;
        double total;
        bool addingProduct;
        Supplier currentSupplier = null;
        Product currentProduct = null;
        Product resolveProduct = null;
        string tempProductType;
        List<Supplier> supplierList = new List<Supplier>();
        List<OrderItem> orderList = new List<OrderItem>();

        // These lists are used for the filter system.
        List<string> filterList = new List<string>();
    }
}
```

```

List<string> YarnWeightList = new List<string>();
List<string> CategoryList = new List<string>();
List<string> FibreList = new List<string>();
List<string> MaterialList = new List<string>();
List<string> ColourList = new List<string>();
List<string> supplierDelStatus = new List<string>();
bool filtersApplied; // True if filters have been applied, used in
searches
string currentCommand; // Stores the command used to retrieve the
data currently shown in the dgv

// Make form draggable
bool drag = false;
Point dragCursor;
Point dragForm;
private void pbxBar_MouseDown(object sender, MouseEventArgs e)
{
    drag = true;
    dragCursor = Cursor.Position;
    dragForm = Location;
}
private void pbxBar_MouseMove(object sender, MouseEventArgs e)
{
    if (drag)
    {
        Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
        Location = Point.Add(dragForm, new Size(difference));
    }
}
private void pbxBar_MouseUp(object sender, MouseEventArgs e)
{
    drag = false;
}
private void btnClose_Click(object sender, EventArgs e)
{
    Close();
}
public NewSupplierOrder(Product product)
{
    InitializeComponent();
    resolveProduct = product;
}
private void FormMakeSupplierOrder_Load(object sender, EventArgs e)
{
    //loading all attributes into lists

    //load all yarn weights into a list
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;

    connection.Open();

```

```

        cmd.CommandText = "SELECT DISTINCT Category FROM TableProducts
WHERE ProductType = 'y'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
            {
                YarnWeightList.Add(Convert.ToString(reader[0]));
            }
            else
            {
                continue;
            }
        }
        connection.Close();

        //load all accessory categories into a list
        connection.Open();
        cmd.CommandText = "SELECT DISTINCT Category FROM TableProducts
WHERE ProductType = 'a'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
            {
                CategoryList.Add(Convert.ToString(reader[0]));
            }
            else
            {
                continue;
            }
        }
        connection.Close();

        //load all yarn fibres into a list
        connection.Open();
        cmd.CommandText = "SELECT DISTINCT FibreMaterial FROM
TableProducts WHERE ProductType = 'y'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
            {
                FibreList.Add(Convert.ToString(reader[0]));
            }
            else
            {
                continue;
            }
        }
        connection.Close();

```



```

        //load all accessory materials into a list
        connection.Open();
        cmd.CommandText = "SELECT DISTINCT FibreMaterial FROM
TableProducts WHERE ProductType = 'a'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
            {
                MaterialList.Add(Convert.ToString(reader[0]));
            }
            else
            {
                continue;
            }
        }
        connection.Close();

        //load all colours into a list
        connection.Open();
        cmd.CommandText = "SELECT DISTINCT Colour FROM TableProducts";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
            {
                ColourList.Add(Convert.ToString(reader[0]));
            }
            else
            {
                continue;
            }
        }
        connection.Close();

        refresh();
        cbxDelStatus.Text = "Order Made";
        tbxStatus.Text = "Open";
    }

    // REFRESHING INFORMATION SHOWN
    private void refresh()
    {
        lsbShoppingList.Refresh();
        lsbCostList.Refresh();
        lsbQuantityList.Refresh();

        cbxSuppliers.DataSource = null;
        cbxSuppliers.DisplayMember = null;

```

```

        cbxSuppliers.ValueMember = null;

        clearChecked(clsbCategory);
        clearChecked(clsbFibreMaterial);
        clearChecked(clsbColour);
        clearChecked(clsbStock);
        pnlFilters.AutoScroll = false;
        clsbCategory.Visible = false;
        clsbFibreMaterial.Visible = false;
        clsbColour.Visible = false;
        lblCategoryFilter.Visible = false;
        lblFibreMaterialFilter.Visible = false;
        lblColourFilter.Visible = false;

        //loads suppliers into a list
        supplierList.Clear();
        OleDbConnection conSuppliers = new OleDbConnection();
        conSuppliers.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmdSuppliers = conSuppliers.CreateCommand();
        OleDbDataReader readerSuppliers;
        conSuppliers.Open();
        cmdSuppliers.CommandText = "SELECT * FROM TableSuppliers WHERE
SupplierName <> 'Deleted'";
        cmdSuppliers.Connection = conSuppliers;
        readerSuppliers = cmdSuppliers.ExecuteReader();
        while (readerSuppliers.Read())
        {
            //instantiates new supplier every time the loop iterates
            supplierList.Add(new
Supplier(Convert.ToInt32(readerSuppliers[0]),
Convert.ToString(readerSuppliers[1]), Convert.ToString(readerSuppliers[2]),
Convert.ToString(readerSuppliers[3]), Convert.ToString(readerSuppliers[4]),
Convert.ToString(readerSuppliers[5]),
Convert.ToString(readerSuppliers[6])));
        }
        conSuppliers.Close();
        cbxSuppliers.DataSource = supplierList;
        cbxSuppliers.DisplayMember = "SupplierInfo";
        cbxSuppliers.ValueMember = "idName";

        if (resolveProduct != null) // When resolving a stock notice,
the product that requires stock will be added and the supplier for that
product will be selected.
        {
            foreach (Supplier sup in supplierList) // Set current
supplier
            {
                if (sup.id == resolveProduct.supplierID)
                {
                    currentSupplier = sup;
                    showSupplierInfo(supplierList.IndexOf(sup));
                }
                refreshProductDGV();
            }
        }
    }

```

```

        foreach (DataGridViewRow row in dgvProducts.Rows) // Select
the product in the dgv
        {
            if (Convert.ToInt32(row.Cells[0].Value) ==
resolveProduct.id)
            {
                dgvProducts.ClearSelection();
                dgvProducts.CurrentCell =
dgvProducts.Rows[row.Index].Cells[0];
                dgvProducts[0, row.Index].Selected = true;
                index = row.Index;
                refreshProductInfo(0);
            }
        }
        // Adds the minimum quantity that needs to be added for no
stock and low stock products. This can be changed by the user.
        OrderItem orderItem = new OrderItem(resolveProduct,
resolveProduct.lowStock, 0, 0);
        numQuantity.Value = orderItem.quantityRemaining;
        orderList.Add(orderItem);
        subtotal += orderItem.product.cost *
orderItem.product.lowStock;
        updateOrderTotals(subtotal);
        lsbQuantityList.DataSource = null;
        lsbQuantityList.DisplayMember = "QuantityString";
        lsbQuantityList.DataSource = orderList;
        lsbQuantityList.Refresh();
        lsbCostList.DataSource = null;
        lsbCostList.DisplayMember = "CostString";
        lsbCostList.DataSource = orderList;
        lsbCostList.Refresh();
        lsbShoppingList.DataSource = null;
        lsbShoppingList.DisplayMember = "name";
        lsbShoppingList.DataSource = orderList;
        lsbShoppingList.Refresh();
        lsbShoppingList.SelectedIndex = lsbShoppingList.Items.Count
- 1;

        lsbQuantityList.SelectedIndex =
lsbShoppingList.SelectedIndex;
        lsbCostList.SelectedIndex = lsbShoppingList.SelectedIndex;
    }
    else if (currentSupplier == null)
    {
        currentSupplier = supplierList[0];
        tbxSupplier.Text = currentSupplier.supplierName;
        showSupplierInfo(0);
        index = 0;
    }
    refreshProductDGV();
    refreshProductInfo(0);
    rbtnAll.Checked = true;
}

private void tbxSearch_TextChanged(object sender, EventArgs e) // As
the user types in the search bar, products are searched for according to the
input.
{
    if (!filtersApplied)

```

```

        {
            if (!string.IsNullOrEmpty(tbxSearch.Text))
            {
                if (rbtnAll.Checked)
                {
                    OleDbConnection connection = new OleDbConnection();
                    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                    OleDbCommand cmd = connection.CreateCommand();
                    OleDbDataReader reader;
                    connection.Open();
                    cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductName LIKE @pn AND Discontinued = False AND SupplierID = @ssid";
                    cmd.Parameters.Clear();
                    cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
                    cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);

                    cmd.Connection = connection;
                    reader = cmd.ExecuteReader();
                    if (reader.HasRows)
                    {
                        BindingSource bindingSource = new
BindingSource();

                        bindingSource.DataSource = reader;
                        dgvProducts.DataSource = bindingSource;
                    }
                    connection.Close();
                }
                else
                {
                    if (rbtnYarn.Checked)
                    {
                        tempProductType = "y";
                    }
                    if (rbtnAcc.Checked)
                    {
                        tempProductType = "a";
                    }
                    OleDbConnection connection = new OleDbConnection();
                    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                    OleDbCommand cmd = connection.CreateCommand();
                    OleDbDataReader reader;
                    connection.Open();
                    cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND ProductName LIKE @pn AND Discontinued = False AND
SupplierID = @ssid";
                    cmd.Parameters.Clear();
                    cmd.Parameters.AddWithValue("@ya", tempProductType);
                    cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
                    cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);

                    cmd.Connection = connection;
                    reader = cmd.ExecuteReader();
                    if (reader.HasRows)

```

```

        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
    else
    {
        if (rbtnAll.Checked)
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableProducts WHERE
Discontinued = False AND SupplierID = @ssid";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);

            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                BindingSource bindingSource = new
BindingSource();

                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
            }
            connection.Close();
        }
        else
        {
            if (rbtnYarn.Checked)
            {
                tempProductType = "y";
            }
            if (rbtnAcc.Checked)
            {
                tempProductType = "a";
            }

            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND Discontinued = False AND SupplierID = @ssid";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@ya", tempProductType);

```

```

currentSupplier.id);
cmd.Parameters.AddWithValue("@ssid",
BindingSource();
cmd.Connection = connection;
reader = cmd.ExecuteReader();
if (reader.HasRows)
{
    BindingSource bindingSource = new
    bindingSource.DataSource = reader;
    dgvProducts.DataSource = bindingSource;
}
connection.Close();
}
}
else
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        if (rbtnAll.Checked)
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);
            cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                BindingSource bindingSource = new
                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
            }
            connection.Close();
        }
        else
        {
            if (rbtnYarn.Checked)
            {
                tempProductType = "y";
            }
            if (rbtnAcc.Checked)
            {
                tempProductType = "a";
            }
            OleDbConnection connection = new OleDbConnection();

```

```

        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);

        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
    else
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand;
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);

        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        if (reader.HasRows)
        {
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        connection.Close();
    }
}
highlightProductsStock();
}
private void btnSearch_Click(object sender, EventArgs e)
{
    if (!filtersApplied)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {

```

```

        if (rbtnAll.Checked)
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductName LIKE @pn AND Discontinued = False AND SupplierID = @ssid";
            MessageBox.Show(cmd.CommandText);
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");
            cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);

            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                BindingSource bindingSource = new
BindingSource();

                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
            }
            else
            {
                MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            connection.Close();
        }
        else
        {
            if (rbtnYarn.Checked)
            {
                tempProductType = "y";
            }
            if (rbtnAcc.Checked)
            {
                tempProductType = "a";
            }
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND ProductName LIKE @pn AND Discontinued = False AND
SupplierID = @ssid";

            MessageBox.Show(cmd.CommandText);
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@ya", tempProductType);
            cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

```



```

currentSupplier.id);
cmd.Parameters.AddWithValue("@ssid",
cmd.Connection = connection;
reader = cmd.ExecuteReader();
if (reader.HasRows)
{
    BindingSource bindingSource = new
BindingSource();

    bindingSource.DataSource = reader;
    dgvProducts.DataSource = bindingSource;
}
else
{
    MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
connection.Close();
}
}
else
{
    if (rbtnAll.Checked)
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts AND
Discontinued = False AND SupplierID = @ssid";
        MessageBox.Show(cmd.CommandText);
        cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);

        cmd.Parameters.Clear();
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
    else
    {
        if (rbtnYarn.Checked)
        {
            tempProductType = "y";
        }
    }
}
}

```

```

        if (rbtnAcc.Checked)
        {
            tempProductType = "a";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND Discontinued = False AND SupplierID = @ssid";
        MessageBox.Show(cmd.CommandText);
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);

        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
}
else
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        if (rbtnAll.Checked)
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";

            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);
            cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)

```

```

        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
    else
    {
        if (rbtnYarn.Checked)
        {
            tempProductType = "y";
        }
        if (rbtnAcc.Checked)
        {
            tempProductType = "a";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand + " AND ProductName
LIKE @pn";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);

        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Parameters.AddWithValue("@pn", "%" +
tbxSearch.Text + "%");

        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No products found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
}
else
{
    OleDbConnection connection = new OleDbConnection();

```

```

        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand;
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        if (reader.HasRows)
        {
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No products found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
}
highlightProductsStock();
}
private void dgvProducts_CellClick(object sender,
DataGridViewCellEventArgs e)
{
    index = e.RowIndex;
    int orderItemsIndex = 0;
    refreshProductInfo(index);

    if (lsbShoppingList.Items.Count != 0) // Highlights the product
selected in the shopping list, if the product is in the shopping list
    {
        OrderItem orderItem = new OrderItem(currentProduct,
Convert.ToInt32(numQuantity.Value), 0, 0);
        bool found = false;
        for (int i = 0; i < lsbShoppingList.Items.Count; i++)
        {
            if (((OrderItem)lsbShoppingList.Items[i]).product.id ==
orderItem.product.id)
            {
                orderItemsIndex = i;
                found = true;
                break;
            }
        }
        if (!found)
        {
            lsbShoppingList.SelectedItem = null;
            lsbQuantityList.SelectedItem = null;
            lsbCostList.SelectedItem = null;
            numQuantity.Value = 0;
        }
    }
}

```

```

        }
        else
        {
            numQuantity.Value =
Convert.ToInt32(((OrderItem)lsbQuantityList.Items[orderItemsIndex]).quantity
);

            lsbShoppingList.SelectedIndex = orderItemsIndex;
            lsbQuantityList.SelectedIndex = orderItemsIndex;
            lsbCostList.SelectedIndex = orderItemsIndex;
        }
    }

}

private void refreshProductInfo(int index)
{
    if (index < 0)
    {
        index = 0;
    }
    DataGridViewRow selectedRow = dgvProducts.Rows[index];
    currentProduct = new
Product(Convert.ToInt32(selectedRow.Cells[0].Value),
selectedRow.Cells[1].Value.ToString(),
Convert.ToSingle(selectedRow.Cells[2].Value),
Convert.ToInt32(selectedRow.Cells[3].Value),
selectedRow.Cells[10].Value.ToString(),
Convert.ToInt32(selectedRow.Cells[4].Value),
Convert.ToBoolean(selectedRow.Cells[11].Value),
Convert.ToInt32(selectedRow.Cells[12].Value),
Convert.ToInt32(selectedRow.Cells[13].Value));
    currentSupplier = currentProduct.Supplier;
    tbxProductID.Text = currentProduct.id.ToString();
    tbxProductName.Text = currentProduct.name;
    tbxCost.Text = currentProduct.CostString;
    tbxStock.Text = currentProduct.stock.ToString();
    tbxSupplier.Text = currentSupplier.supplierName;
    tbxType.Text = selectedRow.Cells[5].Value.ToString();
    tbxCategory.Text = selectedRow.Cells[6].Value.ToString();
    tbxColour.Text = selectedRow.Cells[7].Value.ToString();
    tbxFibre.Text = selectedRow.Cells[8].Value.ToString();
    tempProductType = selectedRow.Cells[9].Value.ToString();
    pbxImage.Refresh();
    if (currentProduct.OutOfStock)
    {
        lblStockLevel.Visible = true;
        lblStockLevel.Text = "Out Of Stock";
    }
    else if (currentProduct.IsLowStock)
    {
        lblStockLevel.Visible = true;
        lblStockLevel.Text = "Low Stock";
    }
    else
    {
        lblStockLevel.Visible = false;
    }
    if (tempProductType == "y")

```

```

        {
            showYarnFields(false);
        }
        if (tempProductType == "a")
        {
            showAccFields(false);
        }
    }
    private bool refreshProductDGV()
    {
        //this is a separate subroutine because while making an order,
        new products may be added, so this subroutine is required to update the
        // dgv of products each time a new product is added
        //loads up all the data into the dgv
        tbxSearch.Text = "";
        string type = "";
        bool changed = false;
        if (rbtnAcc.Checked)
        {
            type = "a";
        }
        if (rbtnYarn.Checked)
        {
            type = "y";
        }
        if (rbtnAll.Checked)
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableProducts WHERE
Discontinued = False AND SupplierID = @ssid";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@ssid",
supplierList[cbxSuppliers.SelectedIndex].id);
            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            BindingSource bindingSource = new BindingSource();
            if (reader.HasRows)
            {
                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
                refreshProductInfo(0);
                changed = true;
            }
            else
            {
                MessageBox.Show("No results found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            connection.Close();
        }
        else
        {

```

```

        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND Discontinued = False AND SupplierID = @ssid";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", type);
        cmd.Parameters.AddWithValue("@ssid",
supplierList[cbxSuppliers.SelectedIndex].id);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        if (reader.HasRows)
        {
            bindingSource.DataSource = reader;
            dgvProducts.DataSource = bindingSource;
            refreshProductInfo(0);
            changed = true;
        }
        else
        {
            MessageBox.Show("No results found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
            rbtnAll.Checked = true;
            rbtnYarn.Checked = false;
            rbtnAcc.Checked = false;
            refresh();
        }
        connection.Close();
    }
    highlightProductsStock();
    return changed;
}

private void highlightProductsStock() // Highlights which products
are out of stock or low on stock in the dgv
{
    foreach (DataGridViewRow row in dgvProducts.Rows)
    {
        if (Convert.ToInt32(row.Cells[3].Value) == 0)
        {
            row.DefaultCellStyle.BackColor = Color.DarkRed;
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlLightLight;
        }
        else if (Convert.ToInt32(row.Cells[3].Value) <
Convert.ToInt32(row.Cells[12].Value))
        {
            row.DefaultCellStyle.BackColor = Color.FromArgb(199,
110, 0);
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlLightLight;
        }
        else

```

```

        {
            row.DefaultCellStyle.BackColor =
SystemColors.ControlLightLight;
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlText;
        }
    }
}
private void showSupplierInfo(int index)
{
    tbxSupplierID.Text = supplierList[index].id.ToString();
    tbxSupplierName.Text =
supplierList[index].supplierName.ToString();
    tbxEmailAdd.Text = supplierList[index].emailAddress.ToString();
    tbxPhoneNum.Text = supplierList[index].phoneNum.ToString();
    tbxAddress.Text = supplierList[index].address.ToString();
    tbxPostcode.Text = supplierList[index].postcode.ToString();
    tbxURL.Text = supplierList[index].url.ToString();
    int tempSupplierID = Convert.ToInt32(tbxSupplierID.Text);
    foreach (Supplier selectSup in supplierList)
    {
        if (selectSup.id == tempSupplierID)
        {
            currentSupplier = selectSup;
        }
    }
}
// FILTERS
private void showYarnFields(bool filter)
{
    if (filter) // If there are other filters in place
    {

        clsbCategory.Visible = true;
        clsbFibreMaterial.Visible = true;
        clsbColour.Visible = true;

        clsbCategory.DataSource = YarnWeightList;
        clsbFibreMaterial.DataSource = FibreList;
        clsbColour.DataSource = ColourList;

        lblCategoryFilter.Text = "YARN WEIGHT";
        lblFibreMaterialFilter.Text = "FIBRE";
        lblCategoryFilter.Visible = true;
        lblFibreMaterialFilter.Visible = true;
        lblColourFilter.Visible = true;

        clsbCategory.ClearSelected();
        clsbFibreMaterial.ClearSelected();
        clsbColour.ClearSelected();
        clearChecked(clsbCategory);
        clearChecked(clsbFibreMaterial);
        clearChecked(clsbColour);
        clearChecked(clsbStock);

        lblCategoryFilter.Visible = true;
        lblFibreMaterialFilter.Visible = true;
    }
}

```



```

        lblColourFilter.Visible = true;
    }
    lblType.Text = "Ball Weight";
    lblCategory.Text = "Yarn Weight";
    lblFibre.Text = "Fibre";
}
private void showAccFields(bool filter)
{
    if (filter) // If there are other filters in place
    {
        clsbCategory.Visible = true;
        clsbFibreMaterial.Visible = true;
        clsbColour.Visible = true;

        clsbCategory.DataSource = CategoryList;
        clsbFibreMaterial.DataSource = MaterialList;
        clsbColour.DataSource = ColourList;

        lblCategoryFilter.Text = "CATEGORY";
        lblFibreMaterialFilter.Text = "MATERIAL";
        lblCategoryFilter.Visible = true;
        lblFibreMaterialFilter.Visible = true;
        lblColourFilter.Visible = true;

        clearChecked(clsbCategory);
        clearChecked(clsbFibreMaterial);
        clearChecked(clsbColour);
        clearChecked(clsbStock);

        lblCategoryFilter.Visible = true;
        lblFibreMaterialFilter.Visible = true;
        lblColourFilter.Visible = true;
    }

    lblType.Text = "Size";
    lblCategory.Text = "Category";
    lblFibre.Text = "Material";

}
private void clearChecked(CheckedListBox clsbToClear)
{
    for (int i = 0; i < clsbToClear.Items.Count; i++)
    {
        clsbToClear.SetSelected(i, false);
        clsbToClear.SetItemChecked(i, false);
    }
    clsbToClear.ClearSelected();
}
private void rbtnYarn_CheckedChanged(object sender, EventArgs e) //
Shows all yarn related fields
{
    if (rbtnYarn.Checked == true)
    {
        showYarnFields(true);
        refreshProductDGV();
        refreshProductInfo(0);
    }
}

```

```

    }
    private void rbtnAcc_CheckedChanged(object sender, EventArgs e) //
Shows all accessory related fields
    {
        if (rbtnAcc.Checked == true)
        {
            if (rbtnAcc.Checked == true)
            {
                showAccFields(true);
                refreshProductDGV();
                refreshProductInfo(0);
            }
        }
    }
    private void rbtnAll_Click(object sender, EventArgs e) // Shows
general fields for all products and clears filters
    {
        DialogResult result = MessageBox.Show("This will clear all
filters. Proceed?", "Clear Filters", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
        tbxSearch.Text = "";
        if (result == DialogResult.Yes)
        {
            currentProduct = null;
            refresh();
        }
        else
        {
            MessageBox.Show("Filters not cleared.");
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND Discontinued = False AND SupplierID = @ssid";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@ya", tempProductType);
            cmd.Parameters.AddWithValue("@ssid", currentSupplier.id);
            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows) //in case all accessories or yarn
products have been deleted
            {
                BindingSource bindingSource = new BindingSource();
                bindingSource.DataSource = reader;
                dgvProducts.DataSource = bindingSource;
                if (tempProductType == "y")
                {
                    rbtnYarn.Checked = true;
                    showYarnFields(false);
                }
                if (tempProductType == "a")
                {
                    rbtnAcc.Checked = true;
                    showAccFields(false);
                }
            }
        }
    }

```

```

    }
    }
    else
    {
        MessageBox.Show("No results found.", "No Results",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
        rbtnAll.Checked = true;
        rbtnYarn.Checked = false;
    }
    connection.Close();
}
highlightProductsStock();
}
private void btnApplyFilters_Click(object sender, EventArgs e)
{
    filtersApplied = false;
    string baseCommandText = "SELECT * FROM TableProducts WHERE
Discontinued = False AND SupplierID = @ssid";
    string categoryFilterCommand = "AND Category IN (";
    string fibreFilterCommand = "AND FibreMaterial IN (";
    string colourFilterCommand = "AND Colour IN (";

    // Need these lists to store the selected items for each checked
list box.
    // Since a new list is created each time the user clicks the
apply button, the list will be cleared each time.
    // Load all the checked filters into their corresponding lists.
    List<string> selectedCategoryFilters =
clsbCategory.CheckedItems.Cast<string>().ToList();
    List<string> selectedFibreFilters =
clsbFibreMaterial.CheckedItems.Cast<string>().ToList();
    List<string> selectedColourFilters =
clsbColour.CheckedItems.Cast<string>().ToList();
    List<string> selectedStockFilters =
clsbStock.CheckedItems.Cast<string>().ToList();

    if (selectedStockFilters.Count != 0)
    {
        filtersApplied = true;
        foreach (string stockFilter in selectedStockFilters)
        {
            string condition = "";
            switch (stockFilter)
            {
                case "In Stock":
                    condition = " Stock > 0";
                    break;
                case "No Stock":
                    condition = " Stock = 0";
                    break;
                case "Low Stock":
                    condition = " Stock < LowStock";
                    break;
            }
            if (baseCommandText.EndsWith("d"))
            {
                baseCommandText += " AND (" + condition;
            }
        }
    }
}

```

```

        }
        else
        {
            baseCommandText += " OR" + condition;
        }
    }
    baseCommandText += ")";
}
else
{
    baseCommandText += " AND Stock >= 0 "; // If no specific
stock filters are selected, results will only include products in stock.
}
if (!rbtnAll.Checked)
{
    baseCommandText += " AND ProductType = @ya ";
}
// Building the command string

if (selectedCategoryFilters.Count != 0)
{
    baseCommandText +=
appendFilterCommand(categoryFilterCommand, selectedCategoryFilters) + ")";
}
if (selectedFibreFilters.Count != 0)
{
    baseCommandText += appendFilterCommand(fibreFilterCommand,
selectedFibreFilters) + ")";
}
if (selectedColourFilters.Count != 0)
{
    baseCommandText += appendFilterCommand(colourFilterCommand,
selectedColourFilters) + ")";
}

// Running the queries
if (rbtnYarn.Checked)
{
    tempProductType = "y";
}
if (rbtnAcc.Checked)
{
    tempProductType = "a";
}

OleDbConnection connection = new OleDbConnection();
connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
OleDbCommand cmd = connection.CreateCommand();
OleDbDataReader reader;
connection.Open();
cmd.CommandText = baseCommandText;
cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@ssid", currentSupplier.id);
cmd.Parameters.AddWithValue("@ya", tempProductType);
cmd.Connection = connection;
reader = cmd.ExecuteReader();

```

```

BindingSource bindingSource = new BindingSource();
if (reader.HasRows)
{
    bindingSource.DataSource = reader;
    dgvProducts.DataSource = bindingSource;
    currentCommand = baseCommandText;
    tbxSearch.Text = "";
    refreshProductInfo(0);
}
else
{
    MessageBox.Show("No products found.", "No Results",
    MessageBoxButtons.OK, MessageBoxIcon.Information);
}
connection.Close();
highlightProductsStock();

}
private string appendFilterCommand(string filterCommand,
List<string> selectedFilters)
{
    filtersApplied = true;
    foreach (string filter in selectedFilters)
    {
        // If there isn't anything in the string so far, don't want
it to start with a comma.
        if (filterCommand.EndsWith("("))
        {
            filterCommand += "'" + filter + "'";
        }
        // Add the checked filter to the command string.
        else
        {
            filterCommand += "," + "'" + filter + "'";
        }
    }
    return filterCommand;
}
private void btnClearFilters_Click(object sender, EventArgs e) //
Clears all filters
{
    DialogResult result = MessageBox.Show("This will clear all
filters. Proceed?", "Clear All Filters", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
    if (result == DialogResult.Yes)
    {
        clearChecked(clsbCategory);
        clearChecked(clsbFibreMaterial);
        clearChecked(clsbColour);
        clearChecked(clsbStock);

        if (rbtnAll.Checked)
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";

```

```

        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
Discontinued = False AND SupplierID = @ssid";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvProducts.DataSource = bindingSource;
        connection.Close();
    }
    else
    {
        if (rbtnYarn.Checked)
        {
            tempProductType = "y";
        }
        if (rbtnAcc.Checked)
        {
            tempProductType = "a";
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableProducts WHERE
ProductType = @ya AND Discontinued = False AND SupplierID = @ssid";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ya", tempProductType);
        cmd.Parameters.AddWithValue("@ssid",
currentSupplier.id);

        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvProducts.DataSource = bindingSource;
        connection.Close();
    }
    filtersApplied = false;
    currentCommand = "";
}
else
{
    MessageBox.Show("No filters were deleted.", "Clear All
Filters", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
highlightProductsStock();
}

// SUPPLIER INFORMATION

```

```

        private void cbxSuppliers_SelectionChangeCommitted(object sender,
EventArgs e) // When the supplier is changed, all order information entered
is cleared and a new order must be created
        {
            if (cbxSuppliers.SelectedIndex < 0)
            {
                cbxSuppliers.SelectedIndex = 0;
            }
            if (lsbShoppingList.Items.Count != 0)
            {
                DialogResult result = MessageBox.Show("This will create a
new order and delete all information entered prior. Proceed?", "New Supplier
Order", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
                if (result == DialogResult.Yes)
                {
                    currentSupplier =
supplierList[cbxSuppliers.SelectedIndex];
                    showSupplierInfo(cbxSuppliers.SelectedIndex);
                    orderList.Clear();
                    lsbShoppingList.DataSource = null;
                    lsbShoppingList.Items.Clear();
                    lsbCostList.DataSource = null;
                    lsbCostList.Items.Clear();
                    lsbQuantityList.DataSource = null;
                    lsbQuantityList.Items.Clear();
                    subtotal = 0;
                    updateOrderTotals(subtotal);
                    refreshProductDGV();
                    refreshProductInfo(0);
                }
                else
                {
                    MessageBox.Show("No information has been deleted.", "New
Supplier Order", MessageBoxButtons.OK, MessageBoxIcon.Information);
                }
            }
            else
            {
                bool changed = refreshProductDGV();
                if (changed) // If there are no products for the supplier
that was selected, then the current supplier won't be changed.
                {
                    showSupplierInfo(cbxSuppliers.SelectedIndex);
                }
                else
                {
                    refreshProductInfo(0);
                    foreach(Supplier s in cbxSuppliers.Items)
                    {
                        if(s.id == currentSupplier.id)
                        {
                            cbxSuppliers.SelectedIndex =
cbxSuppliers.Items.IndexOf(s);
                        }
                    }
                }
            }
        }

```

```

        cbxSuppliers.Text =
supplierList[cbxSuppliers.SelectedIndex].idName; // Sets the currently
selected item in the list to the supplier selected before
    }
}

}
private void updateOrderTotals(double subtotal)
{
    tbxSubtotal.Text = "£" + Math.Round(subtotal,
2).ToString("0.00");
    total = Math.Round(subtotal, 2);
    tbxTotal.Text = "£" + Math.Round(subtotal, 2).ToString("0.00");
}

// ORDER DATE INFORMATION
private void dtpOrderDate_ValueChanged(object sender, EventArgs e)
// Automatically sets the expected delivery date to 3 days following the
order date.
{
    dtpExpDeliveryDate.Value = dtpOrderDate.Value.AddDays(3);
}

// ADDING ITEMS TO ORDER AND PLACING ORDER
private void btnAddProduct_Click(object sender, EventArgs e)
{
    bool productInOrder = false;
    int initialQuantity = Convert.ToInt32(numQuantity.Value);
    int finalIndex = 0;
    if (orderList.Count == 0) // If there are no products in the
order, create an order item, set the quantity to 1 and add it to the order
list. Add the cost of one product to the subtotal.
    {
        OrderItem orderItem = new OrderItem(currentProduct, 1, 0,
0);
        numQuantity.Value = 1;
        orderList.Add(orderItem);
        subtotal += orderItem.product.cost;
        updateOrderTotals(subtotal);
        lsbShoppingList.SelectedIndex = lsbShoppingList.Items.Count
- 1;
        lsbQuantityList.SelectedIndex =
lsbShoppingList.SelectedIndex;
        lsbCostList.SelectedIndex = lsbShoppingList.SelectedIndex;
    }
    else
    {
        if (initialQuantity == 0)
        {
            initialQuantity = 1;
        }
        else
        {
            initialQuantity = Convert.ToInt32(numQuantity.Value);
        }
    }
}

```



```

        foreach (OrderItem item in orderList) // Loops through each
item in the order to find the product that has been selected by the user.
        {
            if (item.product.id == currentProduct.id) // If the
product selected by the user is already in the order, the order item
quantity is increased by 1 and the search for the product selected is
broken.
            {
                // There is a maximum value set to the numeric up
down button.
                // If incrementing the quantity results in
numQuantity storing a value larger than its maximum,
                // it will notify that there is no stock left for
that product and set the item quantity to its maximum.
                productInOrder = true;
                initialQuantity++;
                numQuantity.Value = initialQuantity;

                item.SetQuantity(initialQuantity);
                subtotal += item.product.cost;
                updateOrderTotals(subtotal);
                int orderItemsIndex = 0;
                for (int i = 0; i < lsbShoppingList.Items.Count;
i++)
                {
                    if
(((OrderItem)lsbShoppingList.Items[i]).product.id == item.product.id)
                    {
                        orderItemsIndex = i;
                        break;
                    }
                }
                finalIndex = orderItemsIndex;
                break;
            }
        }
        if (!productInOrder) // If the product selected by the user
is not in the order
        {
            OrderItem orderItem = new OrderItem(currentProduct,
initialQuantity, 0, 0);
            numQuantity.Value = initialQuantity;
            orderList.Add(orderItem);
            subtotal += orderItem.product.cost *
Convert.ToDouble(initialQuantity);
            updateOrderTotals(subtotal);
            addingProduct = true;
            lsbQuantityList.DataSource = null;
            lsbQuantityList.DisplayMember = "QuantityString";
            lsbQuantityList.DataSource = orderList;
            lsbQuantityList.Refresh();
            lsbCostList.DataSource = null;
            lsbCostList.DisplayMember = "CostString";
            lsbCostList.DataSource = orderList;
            lsbCostList.Refresh();
            lsbShoppingList.DataSource = null;
            lsbShoppingList.DisplayMember = "name";

```

```

        lsbShoppingList.DataSource = orderList;
        lsbShoppingList.Refresh();
        finalIndex = lsbShoppingList.Items.Count - 1;
    }
}

lsbQuantityList.DataSource = null;
lsbQuantityList.DisplayMember = "QuantityString";
lsbQuantityList.DataSource = orderList;
lsbQuantityList.Refresh();
lsbCostList.DataSource = null;
lsbCostList.DisplayMember = "CostString";
lsbCostList.DataSource = orderList;
lsbCostList.Refresh();
lsbShoppingList.DataSource = null;
lsbShoppingList.DisplayMember = "name";
lsbShoppingList.DataSource = orderList;
lsbShoppingList.Refresh();
lsbShoppingList.SelectedIndex = finalIndex;
lsbQuantityList.SelectedIndex = lsbShoppingList.SelectedIndex;
lsbCostList.SelectedIndex = lsbShoppingList.SelectedIndex;
updateOrderTotals(subtotal);
}

private void btnApply_Click(object sender, EventArgs e) // Applies
the quantity of the product that the user has set
{
    int index = lsbShoppingList.SelectedIndex;
    if (index > -1 && lsbShoppingList.Items.Count > 0)
    {
        // Adjusts the total of the order accordingly
        int originalQuantity =
((OrderItem)lsbShoppingList.Items[lsbShoppingList.SelectedIndex]).quantity;
        if (numQuantity.Value == 0)
        {
            DialogResult result = MessageBox.Show("Remove product?",
"Remove Product", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
            if (result == DialogResult.Yes)
            {
                index = 0;
                subtotal += -originalQuantity *
((OrderItem)lsbShoppingList.Items[lsbShoppingList.SelectedIndex]).product.co
st;

orderList.Remove((OrderItem)lsbShoppingList.Items[lsbShoppingList.SelectedIn
dex]);

                numQuantity.Enabled = false;
            }
        }
        else
        {
            if (lsbShoppingList.SelectedItem != null)
            {
                subtotal += ((float)numQuantity.Value -
originalQuantity) *
((OrderItem)lsbShoppingList.Items[lsbShoppingList.SelectedIndex]).product.co
st;

```

```

orderList[lsbShoppingList.SelectedIndex].SetQuantity(Convert.ToInt32(numQuantity.Value));

        }
    }
    lsbQuantityList.DataSource = null;
    lsbQuantityList.DisplayMember = "QuantityString";
    lsbQuantityList.DataSource = orderList;
    lsbQuantityList.Refresh();
    lsbCostList.DataSource = null;
    lsbCostList.DisplayMember = "CostString";
    lsbCostList.DataSource = orderList;
    lsbCostList.Refresh();
    lsbShoppingList.DataSource = null;
    lsbShoppingList.DisplayMember = "name";
    lsbShoppingList.DataSource = orderList;
    lsbShoppingList.Refresh();
    if (lsbShoppingList.Items.Count > 0)
    {
        lsbShoppingList.SelectedIndex = index;
        lsbQuantityList.SelectedIndex =
lsbShoppingList.SelectedIndex;
        lsbCostList.SelectedIndex =
lsbShoppingList.SelectedIndex;
        numQuantity.Enabled = false;
    }
    updateOrderTotals(subtotal);
}

}

private void btnNewProduct_Click(object sender, EventArgs e) //
Allows the user to create a new product for the current supplier selected
{
    NewProduct formNewProduct = new NewProduct(false,
currentSupplier.id);
    formNewProduct.ShowDialog();
    refreshProductDGV();
}

public bool ValidCost(string cost)
{
    // Checks if the cost entered can be converted to a double,
which can then be rounded to 2 decimal places if required.
    if (cost.StartsWith("£"))
    {
        // If the user enters a £ sign in
cost = cost.Remove(0, 1);
    }
    TypeConverter conv =
TypeDescriptor.GetConverter(typeof(double));
    return conv.IsValid(cost);
}

private void btnPlaceOrder_Click(object sender, EventArgs e)
{
    //These conditions will mainly come in place when the user
changes the expected delivery date themselves, and the date is invalid.

    if (lsbShoppingList.Items.Count == 0)

```

```

        {
            MessageBox.Show("There are no products added to the order.",
                "No Products", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        }
        else if (dtpExpDeliveryDate.Value < dtpOrderDate.Value)
        {
            MessageBox.Show("You cannot enter an expected delivery date
                that falls before the order date.", "Invalid Date", MessageBoxButtons.OK,
                MessageBoxIcon.Error);
            dtpExpDeliveryDate.Value = dtpOrderDate.Value.AddDays(3);
        }
        else if (string.IsNullOrEmpty(tbxTotal.Text) ||
            string.IsNullOrEmpty(cbxDelStatus.Text))
        {
            MessageBox.Show("Enter values in all required fields.",
                "Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else if (!ValidCost(tbxTotal.Text))
        {
            MessageBox.Show("Enter a valid total.", "Invalid Input",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else
        {
            if (rtbxOrderNotes.Text == "")
            {
                rtbxOrderNotes.Text = " ";
            }
            List<Product> savedProducts = new List<Product>(); // this
            is to keep track of which products have been saved in the shopping list

            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
                Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            connection.Open();
            cmd.CommandText = "INSERT INTO
                TableSupplierOrders(SupplierID, OrderDate, DeliveryDate, DeliveryStatus,
                Subtotal, Total, OrderNotes) VALUES(@sid,@ord,@exd,@dsa,@sub,@tot,@orn)";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@sid", currentSupplier.id);
            cmd.Parameters.AddWithValue("@ord",
                dtpOrderDate.Value.ToString("dd/MM/yyyy"));
            cmd.Parameters.AddWithValue("@exd",
                dtpExpDeliveryDate.Value.ToString("dd/MM/yyyy"));
            cmd.Parameters.AddWithValue("@dsa", cbxDelStatus.Text);
            cmd.Parameters.AddWithValue("@tot", Math.Round(subtotal,
                2));
            cmd.Parameters.AddWithValue("@cos", Math.Round(total, 2));
            cmd.Parameters.AddWithValue("@orn", rtbxOrderNotes.Text);
            cmd.Connection = connection;
            int status = cmd.ExecuteNonQuery();

            cmd.CommandText = "SELECT SupplierOrderID FROM
                TableSupplierOrders ORDER BY SupplierOrderID DESC";
            cmd.Parameters.Clear();

```

```

        cmd.Parameters.AddWithValue("@sid", currentSupplier.id);
        cmd.Parameters.AddWithValue("@od", dtpOrderDate.Value);
        cmd.Parameters.AddWithValue("@dd",
dtpExpDeliveryDate.Value);
        int tempOrderID = Convert.ToInt32(cmd.ExecuteScalar());

        List<int> savedProductsID = new List<int>(); // this is to
keep track of which products have been saved in the shopping list
        foreach (OrderItem shoppingListItem in orderList)
        {
            if
(savedProductsID.Contains(shoppingListItem.product.id)) // if the product
has not already been searched for and recorded
            {
                continue;
            }
            else
            {
                savedProductsID.Add(shoppingListItem.product.id);
                cmd.CommandText = "INSERT INTO
TableSupplierOrderItems (ProductID,OrderID,Quantity) VALUES (@pid,@oid,@q)";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@pid",
((OrderItem)shoppingListItem).product.id);
                cmd.Parameters.AddWithValue("@oid", tempOrderID);
                cmd.Parameters.AddWithValue("@q",
shoppingListItem.quantity);
                int status2 = cmd.ExecuteNonQuery();

                if(cbxDelStatus.Text == "Delivered") // If the order
has already been delivered, then stock can be updated.
                {
                    cmd.CommandText = "UPDATE TableProducts SET
Stock = Stock + @q WHERE ProductID = @pid";
                    cmd.Parameters.Clear();
                    cmd.Parameters.AddWithValue("@q",
shoppingListItem.quantity);
                    cmd.Parameters.AddWithValue("@pid",
((OrderItem)shoppingListItem).product.id);
                    int statusStock = cmd.ExecuteNonQuery();
                }
            }
        }
        connection.Close();
        MessageBox.Show("Order placed successfully.", "Order
Placed", MessageBoxButtons.OK, MessageBoxIcon.Information);
        Close();
    }
}
private void removeToolStripMenuItem_Click(object sender, EventArgs
e)
{
    OrderItem orderItem = (OrderItem)lsbShoppingList.SelectedItem;
    if (orderItem != null)
    {
        // removes the product and adjusts the total shown
        orderList.Remove(orderItem);
    }
}

```

```

        lsbShoppingList.DataSource = null;
        lsbShoppingList.DisplayMember = "name";
        lsbShoppingList.DataSource = orderList;
        lsbShoppingList.Refresh();
        lsbQuantityList.DataSource = null;
        lsbQuantityList.DisplayMember = "QuantityString";
        lsbQuantityList.DataSource = orderList;
        lsbQuantityList.Refresh();
        lsbCostList.DataSource = null;
        lsbCostList.DisplayMember = "CostString";
        lsbCostList.DataSource = orderList;
        lsbCostList.Refresh();
        subtotal -= orderItem.product.cost * orderItem.quantity;
        updateOrderTotals(subtotal);
        numQuantity.Value = 0;
    }
}

private void lsbShoppingList_SelectedIndexChanged(object sender,
EventArgs e)
{
    // When the user clicks on another item in the list, the item is
    highlighted in the dgv and all information for that product is shown
    if (lsbShoppingList.Items.Count > 0)
    {
        btnApply.Visible = true;
        numQuantity.Enabled = true;
        bool productInDGV = false;
        lsbQuantityList.SelectedIndex =
lsbShoppingList.SelectedIndex;
        lsbCostList.SelectedIndex = lsbShoppingList.SelectedIndex;
        if (lsbShoppingList.SelectedItem != null)
        {
            foreach (DataGridViewRow row in dgvProducts.Rows)
            {
                if (addingProduct)
                {
                    if (row.Cells[0].Value.ToString() ==
tbxProductID.Text)
                    {
                        dgvProducts.ClearSelection();
                        dgvProducts.CurrentCell =
dgvProducts.Rows[row.Index].Cells[0];
                        dgvProducts[0, row.Index].Selected = true;
                        productInDGV = true;
                    }
                }
                else
                {
                    if (row.Cells[0].Value.ToString() ==
((OrderItem)lsbShoppingList.SelectedItem).product.id.ToString())
                    {
                        dgvProducts.ClearSelection();
                        dgvProducts.CurrentCell =
dgvProducts.Rows[row.Index].Cells[0];
                        dgvProducts[0, row.Index].Selected = true;
                        productInDGV = true;
                    }
                }
            }
        }
    }
}

```

```

        }
    }
}
if (!productInDGV)
{
    if (rbtnYarn.Checked)
    {
        rbtnAcc.Checked = true;
        foreach (DataGridViewRow row in
dgvProducts.Rows)
        {
            if (row.Cells[0].Value.ToString() ==
((OrderItem)lsbShoppingList.SelectedItem).product.id.ToString())
            {
                dgvProducts.ClearSelection();
                dgvProducts.CurrentCell =
dgvProducts.Rows[row.Index].Cells[0];
                dgvProducts[0, row.Index].Selected =
true;
            }
        }
    }
    else if (rbtnAcc.Checked)
    {
        rbtnYarn.Checked = true;
        foreach (DataGridViewRow row in
dgvProducts.Rows)
        {
            if (row.Cells[0].Value.ToString() ==
((OrderItem)lsbShoppingList.SelectedItem).product.id.ToString())
            {
                dgvProducts.ClearSelection();
                dgvProducts.CurrentCell =
dgvProducts.Rows[row.Index].Cells[0];
                dgvProducts[0, row.Index].Selected =
true;
            }
        }
    }
}
numQuantity.Value =
Convert.ToInt32(((OrderItem)lsbQuantityList.Items[((int)lsbShoppingList.Sele
ctedIndex])).quantity);
refreshProductInfo(dgvProducts.CurrentRow.Index);
addingProduct = false;
    }
}
else
{
    btnApply.Visible = false;
    numQuantity.Enabled = false;
}
}
private void cbxDelStatus_SelectionChangeCommitted(object sender,
EventArgs e) // Automatically sets the order status according to the
delivery status
{

```

```

        if (cbxDelStatus.SelectedItem.ToString() == "Delivered")
        {
            tbxStatus.Text = "Archived";
        }
        else
        {
            tbxStatus.Text = "Open";
        }
    }

    private void pbxImage_Paint(object sender, PaintEventArgs e) //
Paints the image of the product currently shown
    {
        bool imageError = false;
        try
        {
            if (currentProduct.filename != null)
            {
                try
                {
                    Image image = Image.FromFile("productImages/" +
currentProduct.filename);
                    e.Graphics.DrawImage(image, 0, 0, 217, 300);
                }
                catch
                {
                    Image image =
Image.FromFile("productImages/noImage.png");
                    e.Graphics.DrawImage(image, 0, 0, 217, 300);
                }
            }
            imageError = false;
        }
        catch
        {
            if (!imageError)
            {
                MessageBox.Show("Product image could not be loaded.",
"Image Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
                imageError = true;
            }
        }
    }

    private void rtbxOrderNotes_TextChanged(object sender, EventArgs e)
    {
        // Implements character length limit on order notes. It will
show the user when the order notes' text length is reaching the limit of
255.

        if (rtbxOrderNotes.TextLength > 244)
        {
            lblCharLimit.Visible = true;
            lblCharLimit.Text = rtbxOrderNotes.TextLength + "/255";
        }
        else
        {
            lblCharLimit.Visible = false;
        }
    }
}

```



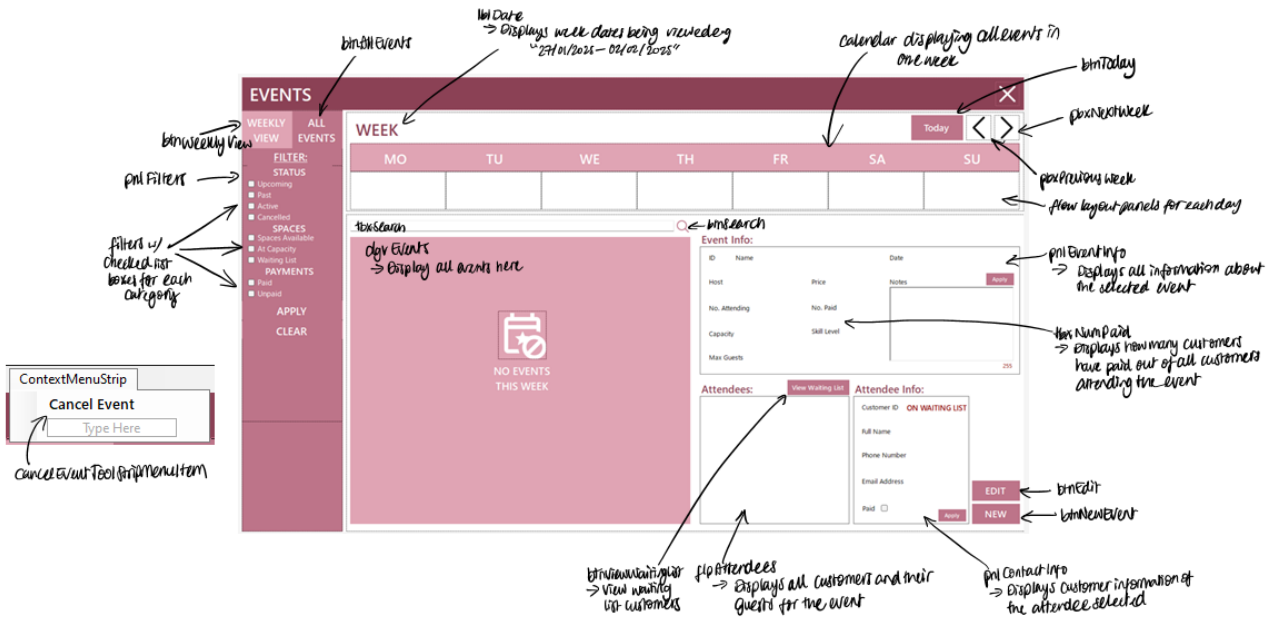
```

    }
}

```

Events.cs

Form Design



Code

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Globalization;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public partial class Events : Form
    {
        int index; // Keeps track of the index of the event in the dgv
        DateTime weekStartDate; // The start date of the week currently
        being viewed
        int month, year; // Month and year of the week currently being
        viewed

        Event selectedEvent = null;
        EventAttendee selectedAttendee = null;
    }
}

```

```

List<EventAttendee> attendeesList = new List<EventAttendee>(); //
List of attendees for the currently selected event
List<EventAttendee> waitingList = new List<EventAttendee>(); // List
of attendees on the waiting list for the currently selected event

bool filtersApplied = false; // Determines if filters are applied so
searches can be applied to the results of the filter
string currentCommand;

private CalendarDay ucdays; // Calendar box containing the events
for one day
private AttendeeBox attendeeBox; // Object displaying attendee names

bool viewWaitingList = false; // Used to determine if the waiting
list is currently being viewed or not

// Make form draggable
bool drag = false;
Point dragCursor;
Point dragForm;

// CUSTOM DRAGGABLE FORM BORDER
private void pbxBar_MouseDown(object sender, MouseEventArgs e)
{
    drag = true;
    dragCursor = Cursor.Position;
    dragForm = Location;
}
private void pbxBar_MouseMove(object sender, MouseEventArgs e)
{
    if (drag)
    {
        Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
        Location = Point.Add(dragForm, new Size(difference));
    }
}
private void pbxBar_MouseUp(object sender, MouseEventArgs e)
{
    drag = false;
}
private void btnClose_Click(object sender, EventArgs e)
{
    Close();
}
public Events(Event @event)
{
    selectedEvent = @event;
    InitializeComponent();
}

// VIEW ALL EVENTS
private void FormEvents_Load(object sender, EventArgs e)
{

```

```

        if(selectedEvent != null) // If directed to this form from the
main calendar or a notice
        {
            // Hide the weekly view, show all events
            btnWeeklyView.BackColor = Color.FromArgb(189, 116, 137);
            btnAllEvents.BackColor = Color.FromArgb(224, 164, 180);
            pnlFilters.Visible = true;
            pnlEvents.Location = new Point(206, 64);
            this.Size = new Size(1554, 687);
            pnlWeeklyView.Visible = false;
            filtersApplied = true;
            currentCommand = "SELECT * FROM TableEvents WHERE Cancelled
= False";

            Event tempEvent = selectedEvent;
            loadEventsDGV();
            selectedEvent = tempEvent;
            foreach (DataGridViewRow row in dgvEvents.Rows)
            {
                if (Convert.ToInt32(row.Cells[0].Value) ==
selectedEvent.id)
                {
                    dgvEvents.ClearSelection();
                    dgvEvents.CurrentCell =
dgvEvents.Rows[row.Index].Cells[0];
                    dgvEvents[0, row.Index].Selected = true;
                    dgvEvents.FirstDisplayedScrollingRowIndex =
row.Index;

                    dgvEvents.Focus();
                }
            }
            displayEventInfo(selectedEvent);
        }
        else // If this form was opened by clicking the Events button on
the homepage, show all events for the current week by default
        {
            refreshCurrentWeek();
        }
    }

    private void loadEventsDGV() // Refreshes the events dgv according
to the current query for events
    {
        if (!filtersApplied)
        {
            if (!pnlWeeklyView.Visible)
            {
                currentCommand = "SELECT * FROM TableEvents WHERE
Cancelled = False AND EventDate > @tod";
            }
            else
            {
                currentCommand = "SELECT * FROM TableEvents WHERE
EventDate >= @ed AND EventDate < @end AND Cancelled = False";
            }
        }
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";

```

```

        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand; // Shows all active events by
default.

        cmd.Parameters.Clear();
        if (!pnlWeeklyView.Visible)
        {
            cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
        }
        cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(weekStartDate.ToShortDateString()));
        cmd.Parameters.AddWithValue("@end",
DateTime.Parse(weekStartDate.AddDays(7).ToShortDateString()));
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            btnEdit.Visible = true;
            BindingSource bindingSource = new BindingSource();
            bindingSource.DataSource = reader;
            dgvEvents.DataSource = bindingSource;
            // Format the columns and cells
            dgvEvents.Columns[1].HeaderText = "Name";
            dgvEvents.Columns[2].HeaderText = "Date";
            dgvEvents.Columns[3].DefaultCellStyle.Format = "HH:mm:ss";
            dgvEvents.Columns[3].HeaderText = "Start Time";
            dgvEvents.Columns[4].DefaultCellStyle.Format = "HH:mm:ss";
            dgvEvents.Columns[4].HeaderText = "End Time";
            pbxNoEvents.Visible = false;
            lblNoEvents.Visible = false;
            DataGridViewRow selectedRow = dgvEvents.Rows[0];
            selectedEvent = new
Event(Convert.ToInt32(selectedRow.Cells[0].Value),
selectedRow.Cells[1].Value.ToString(),
Convert.ToDateTime(selectedRow.Cells[2].Value),
Convert.ToDateTime(selectedRow.Cells[3].Value),
Convert.ToDateTime(selectedRow.Cells[4].Value),
selectedRow.Cells[5].Value.ToString(),
Convert.ToSingle(selectedRow.Cells[6].Value),
Convert.ToInt32(selectedRow.Cells[7].Value),
Convert.ToInt32(selectedRow.Cells[8].Value),
selectedRow.Cells[9].Value.ToString(),
selectedRow.Cells[10].Value.ToString(),
Convert.ToBoolean(selectedRow.Cells[11].Value));
            displayEventInfo(selectedEvent);

        }
        else
        {
            // If no results are shown i.e. there are no future events
that have not been cancelled or no results for filters
            btnEdit.Visible = false;
            dgvEvents.DataSource = null;
            pbxNoEvents.Visible = true;
            lblNoEvents.Visible = true;
        }
    }
}

```

```

    }
    connection.Close();
}
private void pbxNextWeek_Click(object sender, EventArgs e) // Show
all events for the next week
{
    weekStartDate = weekStartDate.AddDays(7);
    month = weekStartDate.Month;
    year = weekStartDate.Year;
    refreshCalendar(weekStartDate);
}
private void pbxPrevWeek_Click(object sender, EventArgs e) // Show
all events for the previous week
{
    weekStartDate = weekStartDate.AddDays(-7);
    month = weekStartDate.Month;
    year = weekStartDate.Year;
    refreshCalendar(weekStartDate);
}
private void btnToday_Click(object sender, EventArgs e) // Show all
events for the current week
{
    refreshCurrentWeek();
}
private void refreshCurrentWeek() // Show all events for the current
week
{
    // Shows all the events for the current week
    DateTime now = DateTime.Now;
    month = now.Month;
    year = now.Year;
    DayOfWeek currentDay = DateTime.Now.DayOfWeek; // gets current
day
    int daysTillCurrentDay = currentDay - DayOfWeek.Monday; // gets
the number of days since monday
    weekStartDate = DateTime.Now.AddDays(-daysTillCurrentDay);
//gets the date of the start of this week
    refreshCalendar(weekStartDate);
}
private void refreshCalendar(DateTime currentWeekStartDate) //
Refresh the calendar
{
    flpMonday.Controls.Clear();
    flpTuesday.Controls.Clear();
    flpWednesday.Controls.Clear();
    flpThursday.Controls.Clear();
    flpFriday.Controls.Clear();
    flpSaturday.Controls.Clear();
    flpSunday.Controls.Clear();

    // Display the date range for the week currently shown
    lblDate.Text = "WEEK " +
currentWeekStartDate.ToShortDateString() + " - " +
currentWeekStartDate.AddDays(6).ToShortDateString();

    // Load all events for the week currently shown

```

```

        List<Event> eventsWeekList =
loadEventsWeekList(currentWeekStartDate);

        if (eventsWeekList.Count > 0) // Show the information for the
first event in the list
        {
            displayEventInfo(eventsWeekList[0]);
        }

        for (int i = 0; i < 7; i++) // Loop through each of the days of
the week and fill in the calendar with the correct events for that day
        {
            string dayName = currentWeekStartDate.DayOfWeek.ToString();
            ucdays = new CalendarDay(false);
            ucdays.days(currentWeekStartDate.Day, month, year);

ucdays.displayEvents(loadEventsDayList(currentWeekStartDate.Day,
eventsWeekList));
            switch (dayName)
            {
                case "Monday":
                    flpMonday.Controls.Add(ucdays);
                    break;
                case "Tuesday":
                    flpTuesday.Controls.Add(ucdays);
                    break;
                case "Wednesday":
                    flpWednesday.Controls.Add(ucdays);
                    break;
                case "Thursday":
                    flpThursday.Controls.Add(ucdays);
                    break;
                case "Friday":
                    flpFriday.Controls.Add(ucdays);
                    break;
                case "Saturday":
                    flpSaturday.Controls.Add(ucdays);
                    break;
                case "Sunday":
                    flpSunday.Controls.Add(ucdays);
                    break;
            }
            ucdays.EventSelected += ucdays_EventSelected;
            currentWeekStartDate = currentWeekStartDate.AddDays(1);
        }
        disableHorizontalScroll();
        loadEventsDGV(); // Refresh the dgv for the week
    }
    private void disableHorizontalScroll()
    {
        flpMonday.HorizontalScroll.Visible = false;
        flpMonday.HorizontalScroll.Enabled = false;
        flpTuesday.HorizontalScroll.Visible = false;
        flpTuesday.HorizontalScroll.Enabled = false;
        flpWednesday.HorizontalScroll.Visible = false;
        flpWednesday.HorizontalScroll.Enabled = false;
        flpThursday.HorizontalScroll.Visible = false;
    }

```

```

        flpThursday.HorizontalScroll.Enabled = false;
        flpFriday.HorizontalScroll.Visible = false;
        flpFriday.HorizontalScroll.Enabled = false;
        flpSaturday.HorizontalScroll.Visible = false;
        flpSaturday.HorizontalScroll.Enabled = false;
        flpSunday.HorizontalScroll.Visible = false;
        flpSunday.HorizontalScroll.Enabled = false;
    }
    private void ucdays_EventSelected(object sender, EventArgs e) // If
an event has been clicked on in the calendar
    {
        if(sender is CalendarBox cbx)
        {
            selectedEvent = cbx.getEvent;
            displayEventInfo(selectedEvent);
            foreach (DataGridViewRow row in dgvEvents.Rows)
            {
                if (Convert.ToInt32(row.Cells[0].Value) ==
selectedEvent.id)
                {
                    dgvEvents.ClearSelection();
                    dgvEvents.CurrentCell =
dgvEvents.Rows[row.Index].Cells[0];
                    dgvEvents[0, row.Index].Selected = true; //
Highlight the event that has been selected in the dgv
                }
            }
        }
    }
    private void attendeeBox_AttendeeSelected(object sender, EventArgs
e) // If an attendee has been clicked on in the list
    {
        if (sender is AttendeeBox attendeeBox)
        {
            selectedAttendee = attendeeBox.attendee;
            displayAttendeeInfo(selectedAttendee);
        }
    }
    public void displayEventInfo(Event @event)
    {
        if (@event != null)
        {
            viewWaitingList = false;
            tbxEventID.Text = @event.id.ToString();
            tbxName.Text = @event.name;
            tbxHost.Text = @event.host.ToString();
            tbxNoOfAttendees.Text = @event.numAttendees.ToString();
            tbxCapacity.Text = @event.capacity.ToString();
            tbxDate.Text = @event.eventDateTime;
            tbxMaxGuests.Text = @event.maxGuests.ToString();
            tbxSkillLevel.Text = @event.skillLevel.ToString();
            tbxPrice.Text = @event.price.ToString();
            tbxNumPaid.Text =
@event.GetAttendingCustomersPaid.Count.ToString() + "/" +
@event.GetTotalAttendingCustomers;
            rtbxNotes.Text = @event.eventNotes;

```

```

        attendeesList = @event.GetAttendingCustomers;
        if (@event.GetWaitingListAttendees.Count > 0) // If there
are attendees on the waiting list
        {
            waitingList = @event.GetWaitingListAttendees;
            btnViewWaitingList.Visible = true;
        }
        else
        {
            waitingList.Clear();
            btnViewWaitingList.Visible = false;
        }
        if (@event.GetAttendingCustomersPaid.Count <
@event.GetAttendingCustomers.Count) //
        {
            lblNumPaid.ForeColor = Color.DarkRed;
        }
        else
        {
            lblNumPaid.ForeColor = SystemColors.ControlText;
        }
        loadAttendees();
    }
}

private void loadAttendees() // By default will show all attendees
that are not on the waiting list. If the 'view waiting list' button has been
clicked, then all attendees on the waiting list will be shown instead.
{
    List<EventAttendee> list = new List<EventAttendee>();

    if (viewWaitingList) // If the waiting list is to be shown
    {
        list = waitingList;
    }
    else
    {
        list = attendeesList;
    }

    flpAttendees.Controls.Clear();
    foreach (EventAttendee a in list)
    {
        attendeeBox = new AttendeeBox(a);
        flpAttendees.Controls.Add(attendeeBox);
        attendeeBox.AttendeeSelected +=
attendeeBox_AttendeeSelected;

    }
    if (list.Count > 0)
    {
        displayAttendeeInfo(attendeesList[0]);
    }
}

private void btnViewWaitingList_Click(object sender, EventArgs e) //
Allows the user to switch between viewing all attending customers and the
waiting list
{

```



```

        if (viewWaitingList)
        {
            viewWaitingList = false;
            btnViewWaitingList.Text = "View Waiting List";
            // switch back to main list of attendees
        }
        else
        {
            viewWaitingList = true;
            btnViewWaitingList.Text = "View Attendees";
            // switch to waiting list
        }
        loadAttendees();
    }
    private void displayAttendeeInfo(EventAttendee attendee)
    {
        tbxFullName.Text = attendee.customer.fullName;
        tbxID.Text = attendee.customer.id.ToString();
        tbxPhoneNum.Text = attendee.customer.phoneNum.ToString();
        tbxEmailAdd.Text = attendee.customer.emailAddress.ToString();
        chbxPaid.Checked = attendee.paid;
        lblOnWaitingList.Visible = attendee.waitingList;

        if (!attendee.paid)
        {
            lblNumPaid.ForeColor = Color.DarkRed;
        }
        else
        {
            lblNumPaid.ForeColor = SystemColors.ControlText;
        }
        btnApplyPaid.Visible = false;
    }
    private void btnWeeklyView_Click(object sender, EventArgs e) //
    Switch between the weekly view and the view of all events
    {
        if (!pnlWeeklyView.Visible)
        {
            // If the weekly view is not visible, shows the weekly view
            and the events for that week only
            btnWeeklyView.BackColor = Color.FromArgb(224, 164, 180);
            btnAllEvents.BackColor = Color.FromArgb(189, 116, 137);
            pnlFilters.Visible = false;
            clearFilters();
            pnlEvents.Location = new Point(206, 272);
            this.Size = new Size(1554, 903);
            pnlWeeklyView.Visible = true;
            refreshCurrentWeek();
            loadEventsDGV();
        }
    }
    private void btnAllEvents_Click(object sender, EventArgs e) //
    Switch between the weekly view and the view of all events
    {
        if (pnlWeeklyView.Visible)
        {

```

```

// If the weekly view is visible, hide it and show all
events

    btnWeeklyView.BackColor = Color.FromArgb(189, 116, 137);
    btnAllEvents.BackColor = Color.FromArgb(224, 164, 180);
    pnlFilters.Visible = true;
    pnlEvents.Location = new Point(206, 64);
    this.Size = new Size(1554, 687);
    pnlWeeklyView.Visible = false;
    loadEventsDGV();
}

}

public List<Event> loadEventsWeekList(DateTime weekStartDate) //
Returns a list of all events for the week displayed
{
    //loads all events for the week displayed
    List<Event> eventsWeekList = new List<Event>();
    for (int i = 0; i < 7; i++)
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableEvents WHERE EventDate
= @ed AND Cancelled = False";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(weekStartDate.ToShortDateString()));
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            bool cancelled = false;
            if (Convert.ToInt32(reader[11]) == -1 ||
Convert.ToBoolean(reader[11]))
            {
                cancelled = true;
            }
            if ((reader[0] != null) || (Convert.ToString(reader[0])
!= ""))
            {
                eventsWeekList.Add(new
Event(Convert.ToInt32(reader[0]), Convert.ToString(reader[1]),
Convert.ToDateTime(reader[2]), Convert.ToDateTime(reader[3]),
Convert.ToDateTime(reader[4]), Convert.ToString(reader[5]),
Convert.ToSingle(reader[6]), Convert.ToInt32(reader[7]),
Convert.ToInt32(reader[8]), Convert.ToString(reader[9]),
Convert.ToString(reader[10]), cancelled));
            }
            else
            {
                continue;
            }
        }
        connection.Close();
        weekStartDate = weekStartDate.AddDays(1);
    }
}

```

```

    }
    return eventsWeekList;
}
public List<Event> loadEventsDayList(int dayNum, List<Event>
eventsOfMonth) // Returns a list of all events for one day
{
    List<Event> eventsOfDay = new List<Event>();
    foreach (Event e in eventsOfMonth)
    {
        if (dayNum == e.date.Day)
        {
            eventsOfDay.Add(e);
        }
    }
    return eventsOfDay;
}
private void btnNewEvent_Click(object sender, EventArgs e) // Allows
user to create a new event
{
    NewEvent newFormNewEvent = new NewEvent(null);
    newFormNewEvent.ShowDialog();
    refreshCurrentWeek();
}
private void dgvEvents_CellClick(object sender,
DataGridViewCellEventArgs e) // Allows the user to click on an event in the
dgv and view all the information for that event
{
    index = e.RowIndex;
    if (index < 0)
    {
        index = 0;
    }
    DataGridViewRow selectedRow = dgvEvents.Rows[index];
    selectedEvent = new
Event(Convert.ToInt32(selectedRow.Cells[0].Value),
selectedRow.Cells[1].Value.ToString(),
Convert.ToDateTime(selectedRow.Cells[2].Value),
Convert.ToDateTime(selectedRow.Cells[3].Value),
Convert.ToDateTime(selectedRow.Cells[4].Value),
selectedRow.Cells[5].Value.ToString(),
Convert.ToSingle(selectedRow.Cells[6].Value),
Convert.ToInt32(selectedRow.Cells[7].Value),
Convert.ToInt32(selectedRow.Cells[8].Value),
selectedRow.Cells[9].Value.ToString(),
selectedRow.Cells[10].Value.ToString(),
Convert.ToBoolean(selectedRow.Cells[11].Value));
    displayEventInfo(selectedEvent);
}

// FILTERS
private void tbxSearch_TextChanged(object sender, EventArgs e) // As
the user types in the search bar, events are searched for according to the
input.
{
    if (!filtersApplied)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))

```

```

        {
            if (!pnlWeeklyView.Visible)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableEvents WHERE
EventName LIKE @en AND Cancelled = False AND EventDate > @tod";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@en", "%" +
tbxSearch.Text + "%");
                cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
                cmd.Connection = connection;
                reader = cmd.ExecuteReader();
                if (reader.HasRows)
                {
                    BindingSource bindingSource = new
BindingSource();
                    bindingSource.DataSource = reader;
                    dgvEvents.DataSource = bindingSource;
                }
                connection.Close();
            }
            else
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableEvents WHERE
EventName LIKE @en AND EventDate >= @ed AND EventDate < @end AND Cancelled =
False";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@en", "%" +
tbxSearch.Text + "%");
                cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(weekStartDate.ToShortDateString()));
                cmd.Parameters.AddWithValue("@end",
DateTime.Parse(weekStartDate.AddDays(7).ToShortDateString()));
                cmd.Connection = connection;
                reader = cmd.ExecuteReader();
                if (reader.HasRows)
                {
                    BindingSource bindingSource = new
BindingSource();
                    bindingSource.DataSource = reader;
                    dgvEvents.DataSource = bindingSource;
                }
                connection.Close();
            }
        }
    }
}

```

```

else
{
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    if (!pnlWeeklyView.Visible)
    {
        cmd.CommandText = "SELECT * FROM TableEvents WHERE
Cancelled = False AND EventDate > @tod";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
    }
    else
    {
        cmd.CommandText = "SELECT * FROM TableEvents WHERE
EventDate >= @ed AND EventDate < @end AND Cancelled = False";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(weekStartDate.ToShortDateString()));
        cmd.Parameters.AddWithValue("@end",
DateTime.Parse(weekStartDate.AddDays(7).ToShortDateString()));
    }
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    if (reader.HasRows)
    {
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvEvents.DataSource = bindingSource;
    }
    connection.Close();
}
} // If no filters have been applied, then all events will be
shown and results found with the input will be shown
else // If filters have been applied, then the search will apply
to the events currently being shown
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        if (!pnlWeeklyView.Visible)
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = currentCommand + " AND EventName
LIKE @en";

            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));

```

```

        cmd.Parameters.AddWithValue("@en", "%" +
tbxSearch.Text + "%");
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvEvents.DataSource = bindingSource;
        }
        connection.Close();
    }
    else
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand + " AND EventName
LIKE @en";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(weekStartDate.ToShortDateString()));
        cmd.Parameters.AddWithValue("@end",
DateTime.Parse(weekStartDate.AddDays(7).ToShortDateString()));
        cmd.Parameters.AddWithValue("@en", "%" +
tbxSearch.Text + "%");

        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvEvents.DataSource = bindingSource;
        }
        connection.Close();
    }

}
else
{
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = currentCommand;
    cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
    cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(weekStartDate.ToShortDateString()));

```

```

        cmd.Parameters.AddWithValue("@end",
DateTime.Parse(weekStartDate.AddDays(7).ToShortDateString()));
        cmd.Parameters.Clear();
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new BindingSource();
            bindingSource.DataSource = reader;
            dgvEvents.DataSource = bindingSource;
        }
        connection.Close();
    }
}

private void btnSearch_Click(object sender, EventArgs e)
{
    if (!filtersApplied)
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            if (!pnlWeeklyView.Visible)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = "SELECT * FROM TableEvents WHERE
EventName LIKE @en AND Cancelled = False AND EventDate > @tod";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@en", "%" +
tbxSearch.Text + "%");
                cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
                cmd.Connection = connection;
                reader = cmd.ExecuteReader();
                if (reader.HasRows)
                {
                    BindingSource bindingSource = new
BindingSource();

                    bindingSource.DataSource = reader;
                    dgvEvents.DataSource = bindingSource;
                }
                else
                {
                    MessageBox.Show("No events found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
                }
                connection.Close();
            }
            else
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";

```

```

OleDbCommand cmd = connection.CreateCommand();
OleDbDataReader reader;
connection.Open();
cmd.CommandText = "SELECT * FROM TableEvents WHERE
EventName LIKE @en AND EventDate >= @ed AND EventDate < @end AND Cancelled =
False";

cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@en", "%" +
tbxSearch.Text + "%");
cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(weekStartDate.ToShortDateString()));
cmd.Parameters.AddWithValue("@end",
DateTime.Parse(weekStartDate.AddDays(7).ToShortDateString()));
cmd.Connection = connection;
reader = cmd.ExecuteReader();
if (reader.HasRows)
{
    BindingSource bindingSource = new
BindingSource();

    bindingSource.DataSource = reader;
    dgvEvents.DataSource = bindingSource;
}
else
{
    MessageBox.Show("No events found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
connection.Close();
}
else
{
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
OleDbCommand cmd = connection.CreateCommand();
OleDbDataReader reader;
connection.Open();
if (!pnlWeeklyView.Visible)
{
    cmd.CommandText = "SELECT * FROM TableEvents WHERE
Cancelled = False AND EventDate > @tod";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
}
else
{
    cmd.CommandText = "SELECT * FROM TableEvents WHERE
EventDate >= @ed AND EventDate < @end AND Cancelled = False";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(weekStartDate.ToShortDateString()));
    cmd.Parameters.AddWithValue("@end",
DateTime.Parse(weekStartDate.AddDays(7).ToShortDateString()));
}
cmd.Connection = connection;

```



```

        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new BindingSource();
            bindingSource.DataSource = reader;
            dgvEvents.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No events found.", "No Results",
                MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
    else
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            if (!pnlWeeklyView.Visible)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
                Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;
                connection.Open();
                cmd.CommandText = currentCommand + " AND EventName
                LIKE @en";

                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@tod",
                DateTime.Parse(DateTime.Today.ToShortDateString()));
                cmd.Parameters.AddWithValue("@en", "%" +
                tbxSearch.Text + "%");

                cmd.Connection = connection;
                reader = cmd.ExecuteReader();
                if (reader.HasRows)
                {
                    BindingSource bindingSource = new
                    BindingSource();

                    bindingSource.DataSource = reader;
                    dgvEvents.DataSource = bindingSource;
                }
                else
                {
                    MessageBox.Show("No events found.", "No
                    Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
                }
                connection.Close();
            }
            else
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
                Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                OleDbDataReader reader;

```

```

        connection.Open();
        cmd.CommandText = currentCommand + " AND EventName
LIKE @en";

        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(weekStartDate.ToShortDateString()));
        cmd.Parameters.AddWithValue("@end",
DateTime.Parse(weekStartDate.AddDays(7).ToShortDateString()));
        cmd.Parameters.AddWithValue("@en", "%" +
tbxSearch.Text + "%");

        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new
BindingSource();

            bindingSource.DataSource = reader;
            dgvEvents.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No events found.", "No
Results", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }

    }
    else
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = currentCommand;
        cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
        cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(weekStartDate.ToShortDateString()));
        cmd.Parameters.AddWithValue("@end",
DateTime.Parse(weekStartDate.AddDays(7).ToShortDateString()));
        cmd.Parameters.Clear();
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        if (reader.HasRows)
        {
            bindingSource.DataSource = reader;
            dgvEvents.DataSource = bindingSource;
        }
        else
        {
            MessageBox.Show("No events found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
}

```

```

        connection.Close();
    }
}

private void clearChecked(CheckedListBox clsbToClear) // Clears all
checked items in the checked list box.
{
    for (int i = 0; i < clsbToClear.Items.Count; i++)
    {
        clsbToClear.SetSelected(i, false);
        clsbToClear.SetItemChecked(i, false);
    }
    clsbToClear.ClearSelected();
}

private void clearFilters()
{
    filtersApplied = false;
    clearChecked(clsbStatus);
    clearChecked(clsbSpaces);
    clearChecked(clsbPayments);
    loadEventsDGV();
}

private void btnApplyFilters_Click(object sender, EventArgs e)
{
    filtersApplied = false;
    string baseCommandText = "SELECT TableEvents.* FROM TableEvents
WHERE";

    List<string> selectedStatusFilters =
clsbStatus.CheckedItems.Cast<string>().ToList();
    List<string> selectedSpacesFilters =
clsbSpaces.CheckedItems.Cast<string>().ToList();
    List<string> selectedPaymentFilters =
clsbPayments.CheckedItems.Cast<string>().ToList();
    List<int> eventIDsSpacesAvailable = new List<int>();
    List<int> eventIDsAtCapacity = new List<int>();
    List<int> eventIDsWaitingList = new List<int>();
    List<int> eventIDs = new List<int>();

    // If filters for spaces have been selected
    if (selectedSpacesFilters.Count != 0)
    {
        filtersApplied = true;

        foreach (string spaceFilter in selectedSpacesFilters)
        {
            switch (spaceFilter)
            {
                case "Spaces Available":
                    // Finds all events that have available spaces
                    OleDbConnection con = new OleDbConnection();
                    con.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                    OleDbCommand cmd1 = con.CreateCommand();
                    OleDbDataReader reader1;
                    con.Open();
                    cmd1.CommandText = "SELECT TableEvents.EventID,
TableEvents.Capacity, SUM (TableEventAttendees.NoOfAttendees) FROM

```

```

TableEventAttendees LEFT JOIN TableEvents ON TableEventAttendees.EventID =
TableEvents.EventID WHERE TableEventAttendees.WaitingList = False GROUP BY
TableEvents.EventID, TableEvents.Capacity;";
        cmd1.Parameters.Clear();
        cmd1.Connection = con;
        reader1 = cmd1.ExecuteReader();
        while (reader1.Read())
        {
            if (Convert.ToInt32(reader1[2]) <
Convert.ToInt32(reader1[1])) // If the number of attendees is less than the
capacity, they will be added to the list of event IDs
            {
                eventIDsSpacesAvailable.Add(Convert.ToInt32(reader1[0]));
            }
        }
        con.Close();
        break;
        case "At Capacity":
            // Find all events that are oversubscribed or at
capacity
            OleDbConnection con1 = new OleDbConnection();
            con1.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd2 = con1.CreateCommand();
            OleDbDataReader reader2;
            con1.Open();
            cmd2.CommandText = "SELECT TableEvents.EventID,
TableEvents.Capacity, SUM (TableEventAttendees.NoOfAttendees) FROM
TableEventAttendees LEFT JOIN TableEvents ON TableEventAttendees.EventID =
TableEvents.EventID WHERE TableEventAttendees.WaitingList = False GROUP BY
TableEvents.EventID, TableEvents.Capacity;";
            cmd2.Parameters.Clear();
            cmd2.Connection = con1;
            reader2 = cmd2.ExecuteReader();
            while (reader2.Read())
            {
                if (Convert.ToInt32(reader2[2]) >=
Convert.ToInt32(reader2[1])) // If the number of attendees is equal to or
more than the capacity, the event ID will be added to the list of event IDs
                {
                    eventIDsAtCapacity.Add(Convert.ToInt32(reader2[0]));
                }
            }
            con1.Close();
            break;
        case "Waiting List":
            // Find all events that have a waiting list
            OleDbConnection con2 = new OleDbConnection();
            con2.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd3 = con2.CreateCommand();
            OleDbDataReader reader3;
            con2.Open();
            cmd3.CommandText = "SELECT DISTINCT
TableEvents.EventID FROM TableEvents INNER JOIN TableEventAttendees ON

```

```

TableEvents.EventID = TableEventAttendees.EventID WHERE
TableEventAttendees.WaitingList = True;";
        cmd3.Parameters.Clear();
        cmd3.Connection = con2;
        reader3 = cmd3.ExecuteReader();
        while (reader3.Read())
        {
            eventIDsWaitingList.Add(Convert.ToInt32(reader3[0]));
        }
        con2.Close();
        break;
    }

    // Sets the list of event IDs to all ids that satisfy all
    // conditions selected by the user
    // If both 'at capacity' and 'spaces available' have been
    // selected, then no specific filter is applied
    // If both 'at capacity' and 'waiting list' have been
    // selected, then the intersection of the lists of event IDs is used
    // Same applies to 'spaces available' and 'waiting list'
    if((eventIDsAtCapacity.Count != 0 &&
eventIDsWaitingList.Count != 0 && eventIDsSpacesAvailable.Count != 0) ||
(eventIDsAtCapacity.Count == 0 && eventIDsSpacesAvailable.Count == 0 &&
eventIDsWaitingList.Count != 0))
    {
        eventIDs = eventIDsWaitingList;
    }
    else if (eventIDsAtCapacity.Count != 0 &&
eventIDsWaitingList.Count != 0)
    {
        eventIDs =
eventIDsAtCapacity.Intersect(eventIDsWaitingList).ToList();
    }
    else if(eventIDsSpacesAvailable.Count != 0 &&
eventIDsWaitingList.Count != 0)
    {
        eventIDs =
eventIDsSpacesAvailable.Intersect(eventIDsWaitingList).ToList();
    }
    else if(eventIDsAtCapacity.Count != 0 &&
eventIDsSpacesAvailable.Count == 0)
    {
        eventIDs = eventIDsAtCapacity;
    }
    else if(eventIDsSpacesAvailable.Count != 0)
    {
        eventIDs = eventIDsSpacesAvailable;
    }
}

List<int> eventIDPayments = new List<int>();
if(selectedPaymentFilters.Count != 0)
{
    if(selectedPaymentFilters.Count != 2) // if both filters are
selected then there is no need for a filter

```

```

        {
            filtersApplied = true;
            foreach(string paymentFilter in selectedPaymentFilters)
            {
                OleDbConnection con = new OleDbConnection();
                con.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
                OleDbCommand cmd1 = con.CreateCommand();
                OleDbDataReader reader1;
                con.Open();
                if (paymentFilter == "Unpaid")
                {
                    // Gets the IDs of all events where there are
customers that haven't paid.
                    cmd1.CommandText = "SELECT DISTINCT
TableEvents.EventID FROM (TableCustomers INNER JOIN TableEventAttendees ON
TableCustomers.CustID = TableEventAttendees.CustID) INNER JOIN TableEvents
ON TableEventAttendees.EventID = TableEvents.EventID WHERE
TableEventAttendees.Paid = False ORDER BY TableEvents.EventID;";

                }
                else
                {
                    // Gets the IDs of all events where all
customers have paid by checking the count of attendees that have not paid
                    cmd1.CommandText = "SELECT DISTINCT
TableEvents.EventID FROM TableEvents LEFT JOIN TableEventAttendees ON
TableEvents.EventID = TableEventAttendees.EventID GROUP BY
TableEvents.EventID HAVING SUM(IIf(TableEventAttendees.Paid = False, 1, 0))
= 0";

                }
                cmd1.Parameters.Clear();
                cmd1.Connection = con;
                reader1 = cmd1.ExecuteReader();
                while (reader1.Read())
                {
                    eventIDSPayments.Add(Convert.ToInt32(reader1[0]));
                }
                con.Close();
            }
        }

        if(eventIDSPayments.Count != 0 && eventIDs.Count !=0) // If
there are event ids from other filters, intersect both lists
        {
            eventIDs = eventIDs.Intersect(eventIDSPayments).ToList();
        }
        else if(eventIDs.Count==0) // If there are no event ids from
other filters, set the list equal to the ids from payment filters
        {
            eventIDs = eventIDSPayments;
        }

        if(eventIDs.Count > 0)
        {

```

```

baseCommandText += " TableEvents.EventID IN (";
foreach(int id in eventIDs)
{
    if(eventIDs.IndexOf(id) == eventIDs.Count - 1)
    {
        baseCommandText += id;
        break; // so an extra comma is not included
    }
    else
    {
        // if the id to be added is the first id in the
list, there should be no preceding comma
        baseCommandText += id;
    }
    baseCommandText += ",";
}
baseCommandText += ")";

string statusCondition = "";
if (selectedStatusFilters.Count != 0)
{
    filtersApplied = true;
    foreach (string statusFilter in selectedStatusFilters)
    {
        switch (statusFilter)
        {
            case "Upcoming":
                statusCondition = " TableEvents.EventDate >=
@tod";

                break;
            case "Past":
                statusCondition = " TableEvents.EventDate <
@tod";

                break;
            case "Active":
                statusCondition = " TableEvents.Cancelled =
False";

                break;
            case "Cancelled":
                statusCondition = " TableEvents.Cancelled =
True";

                break;
        }
        if (baseCommandText.EndsWith("E"))
        {
            if (selectedStatusFilters.Count > 1)
            {
                baseCommandText += " (" + statusCondition;
            }
            else
            {
                baseCommandText += " " + statusCondition;
            }
        }
        else if (baseCommandText.EndsWith("("))
        {

```

```

        if(selectedStatusFilters.Count > 1)
        {
            baseCommandText += " AND (" + statusCondition;
        }
        else
        {
            baseCommandText += " AND " + statusCondition;
        }
    }
    else
    {
        baseCommandText += " OR" + statusCondition;
    }
}
if (selectedStatusFilters.Count > 1)
{
    baseCommandText += ")";
}
}
else // If no filters are selected, upcoming events are shown by
default
{
    if (baseCommandText.EndsWith("("))
    {
        baseCommandText += " AND TableEvents.EventDate >= @tod
AND TableEvents.Cancelled = False";
    }
    else if(baseCommandText.EndsWith("E"))
    {
        baseCommandText += " TableEvents.EventDate >= @tod AND
TableEvents.Cancelled = False";
    }
}
OleDbConnection connection = new OleDbConnection();
connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
OleDbCommand cmd = connection.CreateCommand();
OleDbDataReader reader;
connection.Open();
cmd.CommandText = baseCommandText;
cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@tod",
DateTime.Parse(DateTime.Today.ToShortDateString()));
cmd.Connection = connection;
reader = cmd.ExecuteReader();
BindingSource bindingSource = new BindingSource();
if (reader.HasRows)
{
    bindingSource.DataSource = reader;
    dgvEvents.DataSource = bindingSource;
    currentCommand = baseCommandText;
    tbxSearch.Text = "";
}
else
{

```



```

        MessageBox.Show("No events found.", "No Results",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    connection.Close();
}
private void btnClearFilters_Click(object sender, EventArgs e) //
Clears all filters
{
    DialogResult result = MessageBox.Show("This will clear all
filters. Proceed?", "Clear All Filters", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
    if (result == DialogResult.Yes)
    {
        clearFilters();
    }
}

// EDITING
private void chbxPaid_CheckedChanged(object sender, EventArgs e)
{
    if(selectedAttendee != null)
    {
        if (chbxPaid.Checked != selectedAttendee.paid)
        {
            // If the user has unchecked/checked the paid checkbox,
show the apply button to allow the user to change the paid status of the
attendee

            btnApplyPaid.Visible = true;

        }
        else
        {
            btnApplyPaid.Visible = false;

        }

    }
}
private void btnApplyPaid_Click(object sender, EventArgs e) //
Applies the changes made to the customer's paid status
{
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    connection.Open();
    cmd.CommandText = "UPDATE TableEventAttendees SET Paid = True
WHERE CustID = @cid AND EventID = @eid";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@cid",
selectedAttendee.customer.id);
    cmd.Parameters.AddWithValue("@eid", selectedEvent.id);
    int status = cmd.ExecuteNonQuery();
    connection.Close();
    Event tempEvent = selectedEvent;
    loadEventsDGV();
    selectedEvent = tempEvent;
}

```

```

DataGridViewRow selectedRow = dgvEvents.Rows[0];
foreach (DataGridViewRow row in dgvEvents.Rows)
{
    if (Convert.ToInt32(row.Cells[0].Value) == selectedEvent.id)
    {
        dgvEvents.ClearSelection();
        dgvEvents.CurrentCell =
dgvEvents.Rows[row.Index].Cells[0];
        dgvEvents[0, row.Index].Selected = true;
        selectedRow = dgvEvents.Rows[row.Index];
        dgvEvents.FirstDisplayedScrollingRowIndex = row.Index;
        dgvEvents.Focus();
    }
}
selectedEvent = new
Event(Convert.ToInt32(selectedRow.Cells[0].Value),
selectedRow.Cells[1].Value.ToString(),
Convert.ToDateTime(selectedRow.Cells[2].Value),
Convert.ToDateTime(selectedRow.Cells[3].Value),
Convert.ToDateTime(selectedRow.Cells[4].Value),
selectedRow.Cells[5].Value.ToString(),
Convert.ToSingle(selectedRow.Cells[6].Value),
Convert.ToInt32(selectedRow.Cells[7].Value),
Convert.ToInt32(selectedRow.Cells[8].Value),
selectedRow.Cells[9].Value.ToString(),
selectedRow.Cells[10].Value.ToString(),
Convert.ToBoolean(selectedRow.Cells[11].Value));
displayEventInfo(selectedEvent);
}
private void rtbxNotes_TextChanged(object sender, EventArgs e)
{
    if (selectedEvent != null)
    {
        if (rtbxNotes.Text != selectedEvent.eventNotes) // If the
notes have been changed, show the apply button
        {
            btnApplyNotes.Visible = true;
        }
        else
        {
            btnApplyNotes.Visible = false;
        }
    }
    // Implements character length limit on order notes. It will
show the user when the order notes' text length is reaching the limit of
255.
    if (rtbxNotes.TextLength > 244)
    {
        lblCharLimit.Visible = true;
        lblCharLimit.Text = rtbxNotes.TextLength + "/255";
    }
    else
    {
        lblCharLimit.Visible = false;
    }
}

```

```

    }
    private void btnApplyNotes_Click(object sender, EventArgs e) //
Applies the changes made to the order notes
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        connection.Open();
        cmd.CommandText = "UPDATE TableEvents SET EventNotes = @en WHERE
EventID = @eid";
        cmd.Parameters.Clear();
        if (string.IsNullOrEmpty(rtbxNotes.Text))
        {
            rtbxNotes.Text = " ";
        }
        cmd.Parameters.AddWithValue("@en", rtbxNotes.Text);
        cmd.Parameters.AddWithValue("@eid", selectedEvent.id);
        int status = cmd.ExecuteNonQuery();
        connection.Close();
        Event tempEvent = selectedEvent;
        loadEventsDGV();
        selectedEvent = tempEvent;
        DataGridViewRow selectedRow = dgvEvents.Rows[0];
        foreach (DataGridViewRow row in dgvEvents.Rows)
        {
            if (Convert.ToInt32(row.Cells[0].Value) == selectedEvent.id)
            {
                dgvEvents.ClearSelection();
                dgvEvents.CurrentCell =
dgvEvents.Rows[row.Index].Cells[0];
                dgvEvents[0, row.Index].Selected = true;
                selectedRow = dgvEvents.Rows[row.Index];
                dgvEvents.FirstDisplayedScrollingRowIndex = row.Index;
                dgvEvents.Focus();
            }
        }
        selectedEvent = new
Event(Convert.ToInt32(selectedRow.Cells[0].Value),
selectedRow.Cells[1].Value.ToString(),
Convert.ToDateTime(selectedRow.Cells[2].Value),
Convert.ToDateTime(selectedRow.Cells[3].Value),
Convert.ToDateTime(selectedRow.Cells[4].Value),
selectedRow.Cells[5].Value.ToString(),
Convert.ToSingle(selectedRow.Cells[6].Value),
Convert.ToInt32(selectedRow.Cells[7].Value),
Convert.ToInt32(selectedRow.Cells[8].Value),
selectedRow.Cells[9].Value.ToString(),
selectedRow.Cells[10].Value.ToString(),
Convert.ToBoolean(selectedRow.Cells[11].Value));
        displayEventInfo(selectedEvent);
    }

    private void cancelEventToolStripMenuItem_Click(object sender,
EventArgs e)
    {
        if(selectedEvent.date < DateTime.Today)

```

```

        {
            MessageBox.Show("Cannot cancel a past event.", "Cancel
Event", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else
        {
            DialogResult result = MessageBox.Show("Cancel event?",
"Cancel Event", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
            if (result == DialogResult.Yes)
            {
                OleDbConnection connection = new OleDbConnection();
                connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
                OleDbCommand cmd = connection.CreateCommand();
                connection.Open();
                cmd.CommandText = "UPDATE TableEvents SET Cancelled =
True WHERE EventID = @eid";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@eid", selectedEvent.id);
                int status = cmd.ExecuteNonQuery();
                connection.Close();
                loadEventsDGV();
            }
        }
    }

    private void btnEdit_Click(object sender, EventArgs e) // Allows the
user to edit the event
    {
        NewEvent newFormNewEvent = new NewEvent(selectedEvent);
        newFormNewEvent.ShowDialog();
        refreshCurrentWeek();
    }
}
}

```

NewEvent.cs

Form Design

The screenshot shows a Windows Form titled "EVENT CREATOR" with a close button (X) in the top right corner. The form is divided into three main sections: "Customer Info:", "ON WAITING LIST", and "Event Info:".

- Customer Info:** Contains fields for "Customer ID", "Name", "Email Address", and "Phone Number". Below these are "Guests" (a spinner box set to 0) and a "Paid" checkbox. There are "Apply" and "Add" buttons. A "Refresh List" button is at the bottom left of this section.
- ON WAITING LIST:** A large pink rectangular area on the left side of the form.
- Event Info:** Contains fields for "Name", "Notes", "Host", "Date" (with an "All Day" checkbox), "Start Time", "End Time", "Price", "Skill Level" (a dropdown menu), "Max Guests" (a spinner box set to 0), and "Capacity" (a spinner box set to 1). There are "CANCEL", "CLEAR", and "ADD" buttons at the bottom right.

Handwritten annotations include:

- btnSearch:** Points to a search icon in the top left.
- numGuests:** Points to the "Guests" spinner box, with a note: "Choose number of guests the Customer has".
- btnAddCustomer:** Points to the "Add" button in the Customer Info section.
- btnRefreshList:** Points to the "Refresh List" button, with a note: "Refreshes waiting list and full list when clicked".
- btnApplyAttendeeChanges:** Points to the "Apply" button, with a note: "Applies any changes made to the selected attendee's guests/paid status".
- btnAttendees:** Points to the "ON WAITING LIST" area, with a note: "Displays all customers and their guests currently added to the list".
- txtName:** Points to the "Name" field in the Event Info section.
- txtHost:** Points to the "Host" field in the Event Info section.
- txtEventNotes:** Points to the "Notes" field in the Event Info section.
- dtpEventDate:** Points to the "Date" field in the Event Info section.
- dtgStartTime:** Points to the "Start Time" field in the Event Info section.
- txtNoOfAttendees:** Points to the "No. Attendees" label in the Event Info section.
- dtgEndTime:** Points to the "End Time" field in the Event Info section.
- txtNumPaid:** Points to the "No. Paid" label in the Event Info section.
- numMaxGuests:** Points to the "Max Guests" spinner box.
- numCapacity:** Points to the "Capacity" spinner box.
- prcEventInfo:** Points to the "Event Info:" section header.
- txtPrice:** Points to the "Price" field in the Event Info section.
- chkSkillLevel:** Points to the "Skill Level" dropdown menu.
- btnClear:** Points to the "CLEAR" button.
- btnCancel:** Points to the "CANCEL" button.
- btnAddSave:** Points to the "ADD" button.

Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Diagnostics.Eventing.Reader;
using System.Drawing;
using System.Linq;
using System.Reflection;
using System.Security.Cryptography;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Xml.Linq;

namespace YarnShop
{
    public partial class NewEvent : Form
    {
        int index;
        int capacity = 5;
        List<string> skillLevel = new List<string>();

        List<EventAttendee> attendeeList = new List<EventAttendee>();
    }
}
```

```

List<EventAttendee> mainList = new List<EventAttendee>();
WaitingListQueue paidWaitingList = new WaitingListQueue();
WaitingListQueue unpaidWaitingList = new WaitingListQueue();
List<EventAttendee> removedAttendees = new List<EventAttendee>(); //
Used to keep track of attendees that have been removed from an existing
event that is being edited, so records can be updated.
List<EventAttendee> addedAttendees = new List<EventAttendee>(); //
Used to keep track of attendees that are newly added to an existing event
that is being edited
List<EventAttendee> attendeesOffWaitingList = new
List<EventAttendee>(); // Used to keep track of attendees that have been
moved off the waiting list when editing an existing event
EventAttendee selectedAttendee = null;
AttendeeBox selectedBox;
Customer currentCust = null;

// Avoid recursive triggers of the value changed event
bool updatingEndTime = false;
bool updatingStartTime = false;

bool waitinglist = false;
bool selectingCust = false;
bool paidStatusChanged = false;
bool numGuestsChanged = false;
private AttendeeBox attendeeBox;
Event editEvent { get; set; }

// Make form draggable
bool drag = false;
Point dragCursor;
Point dragForm;

// CUSTOM DRAGGABLE FORM BORDER
private void pbxBar_MouseDown(object sender, MouseEventArgs e)
{
    drag = true;
    dragCursor = Cursor.Position;
    dragForm = Location;
}
private void pbxBar_MouseMove(object sender, MouseEventArgs e)
{
    if (drag)
    {
        Point difference = Point.Subtract(Cursor.Position, new
Size(dragCursor));
        Location = Point.Add(dragForm, new Size(difference));
    }
}
private void pbxBar_MouseUp(object sender, MouseEventArgs e)
{
    drag = false;
}
private void btnClose_Click(object sender, EventArgs e)
{
    Close();
}
public NewEvent(Event editEvent)

```

```

{
    InitializeComponent();
    this.editEvent = editEvent;
}
private void FormNewEvent_Load(object sender, EventArgs e)
{
    if (editEvent != null)
    {
        lblMakeEvent.Text = "EVENT EDITOR";
        btnAddSave.Text = "Save";
    }
    else
    {
        lblMakeEvent.Text = "EVENT CREATOR";
        btnAddSave.Text = "Add";
    }

    lblWaitingList.Visible = false;
    dtpStartTime.Format = DateTimePickerFormat.Time;
    dtpStartTime.ShowUpDown = true;
    dtpEndTime.Format = DateTimePickerFormat.Time;
    dtpEndTime.ShowUpDown = true;

    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;

    connection.Open();
    cmd.CommandText = "SELECT DISTINCT SkillLevel FROM TableEvents";
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        if ((reader[0] != null) || (Convert.ToString(reader[0]) !=
""))
        {
            skillLevel.Add(Convert.ToString(reader[0]));
        }
        else
        {
            continue;
        }
    }
    connection.Close();
    cbxSkillLevel.DataSource = null;
    cbxSkillLevel.DataSource = skillLevel;

    refresh();
}

private void refreshAttendees() // Shows all attendees in the list
{
    flpAttendees.Controls.Clear();
    foreach (EventAttendee a in attendeeList)
    {

```

```

        attendeeBox = new AttendeeBox(a);
        flpAttendees.Controls.Add(attendeeBox);
        attendeeBox.AttendeeSelected +=
attendeeBox_AttendeeSelected;

    }
    refreshEventTbxs();
    highlightCustomersAdded();
}
private void refresh()
{
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = "SELECT * FROM TableCustomers WHERE MemType <>
'Deleted' OR MemType <> 'Guest'";
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    BindingSource bindingSource = new BindingSource();
    bindingSource.DataSource = reader;
    dgvCustomers.DataSource = bindingSource;
    connection.Close();

    if (editEvent != null) // Shows all event details if editing an
event
    {
        attendeeList = editEvent.GetOrderedAttendeesList;
        mainList = editEvent.GetAttendingCustomers;
        paidWaitingList = editEvent.GetPaidWaitingListQueue;
        unpaidWaitingList = editEvent.GetUnpaidWaitingListQueue;
        tbxName.Text = editEvent.name;
        tbxHost.Text = editEvent.host;
        tbxNoOfAttendees.Text = editEvent.numAttendees.ToString();
        numCapacity.Value = editEvent.capacity;
        dtpEventDate.Value = editEvent.date;
        dtpStartTime.Value = editEvent.date +
editEvent.startTime.TimeOfDay;
        dtpEndTime.Value = editEvent.date +
editEvent.endTime.TimeOfDay;
        numMaxGuests.Text = editEvent.maxGuests.ToString();
        cbxSkillLevel.Text = editEvent.skillLevel.ToString();
        tbxPrice.Text = editEvent.price.ToString();
        tbxNoPaid.Text =
editEvent.GetAttendingCustomersPaid.Count.ToString() + " / "+
CountAttendingCustomers(attendeeList);
        rtbxNotes.Text = editEvent.eventNotes;
        refreshAttendees();

        if (attendeeList.Count > 0)
        {
            currentCust = attendeeList[0].customer;
        }
    }
}

```



```

else
{
    //Default
    tbxName.Text = string.Empty;
    tbxHost.Text = "Purls Of Wisdom";
    tbxPrice.Text = string.Empty;
    rtbxNotes.Text = string.Empty;
    tbxNoOfAttendees.Text = string.Empty;
    numCapacity.Value = 10;
    numMaxGuests.Value = 0;
    cbxSkillLevel.Text = string.Empty;

    index = 0;
    refreshCustomerInfo(0);
    refreshAttendees();

    dtpStartTime.Value.Subtract(dtpStartTime.Value);
    dtpStartTime.Value = DateTime.Today.Date.AddHours(9);
    dtpEndTime.Value.Subtract(dtpEndTime.Value);
    dtpEndTime.Value = DateTime.Today.Date.AddHours(10);
}

}

private bool CustomerAdded(int id, List<EventAttendee> list) //
Checks the list of attendees to see if a customer has been added or not.
{
    foreach (EventAttendee a in list)
    {
        if(a.customer.id == id)
        {
            return true;
        }
    }
    return false;
}

private int CountAttendees(List<EventAttendee> attendees)
{
    // Returns the total number of people in the list.

    int i = 0;
    foreach (EventAttendee attendee in attendees)
    {
        i += attendee.noOfAttendees;
    }
    return i;
}

private int CountAttendingCustomers(List<EventAttendee> attendees)
{
    // Returns the total number of people in the list.

    int i = 0;
    foreach (EventAttendee attendee in attendees)
    {
        if (!attendee.waitingList)
        {

```

```

        i += attendee.noOfAttendees;
    }
}
return i;
}
private void highlightCustomersAdded()// Searches through each
customer in the dgv and highlights the customers that have already been
added to the event attendees list.
{
    foreach (DataGridViewRow row in dgvCustomers.Rows)
    {
        if
(CustomerAdded(Convert.ToInt32(row.Cells[0].Value),attendeeList))
        {
            row.DefaultCellStyle.BackColor = Color.FromArgb(50, 105,
50);
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlLightLight;
        }
        else
        {
            row.DefaultCellStyle.BackColor =
SystemColors.ControlLightLight;
            row.DefaultCellStyle.ForeColor =
SystemColors.ControlText;
        }
    }
}
private void refreshEventTbxs()// Updates the textboxes showing how
many attendees are in the event and how many have paid.
{
    tbxNoOfAttendees.Text = CountAttendees(mainList).ToString();
    int numPaid = 0;
    foreach(EventAttendee a in mainList)
    {
        if (a.paid)
        {
            numPaid ++;
        }
    }
    tbxNoPaid.Text = numPaid.ToString() + "/" + mainList.Count; //
Sets label colour to red if the number of customers that have paid is less
than the number of customers in the list
    if (editEvent != null)
    {
        if (editEvent.GetAttendingCustomersPaid.Count <
editEvent.GetAttendingCustomers.Count)
        {
            lblAttendeesPaid.ForeColor = Color.DarkRed;
        }
        else
        {
            lblAttendeesPaid.ForeColor = SystemColors.ControlText;
        }
    }
}
}

```

```

        private void updateWaitingListLabel() // Indicates when a customer
will be added to the waiting list depending on the number of attendees
currently, the capacity and the number of guests of the customer to be added
    {
        // Determines whether to show that the next customer to be added
will be on the waiting list.

        if (CountAttendees(mainList) < numCapacity.Value)
        {
            lblWaitingList.Visible = false;
            waitinglist = false;
        }
        else
        {
            lblWaitingList.Visible = true;
            waitinglist = true;
        }
    }

    public bool ValidCost(string cost) // Checks if the cost entered can
be converted to a double, which can then be rounded to 2 decimal places if
required.
    {
        TypeConverter conv =
TypeDescriptor.GetConverter(typeof(double));
        return conv.IsValid(cost);
    }

    private bool validString(string text) // Checks if the string is
valid i.e. only contains valid alphanumerical characters and certain special
characters
    {

        string allowedCharacters =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 -_.,&/():'";

        // Check each character
        foreach (char c in text)
        {
            bool found = false;
            for (int i = 0; i < allowedCharacters.Length; i++)
            {
                if (c == allowedCharacters[i])
                {
                    found = true;
                }
            }
            if (!found)
            {
                MessageBox.Show("Input valid details in all fields.",
"Invalid Inputs", MessageBoxButtons.OK, MessageBoxIcon.Error);
                return found;
            }
        }

        return true;
    }

    private void btnAddSave_Click(object sender, EventArgs e)

```

```

        {
            if (string.IsNullOrEmpty(tbxName.Text) ||
string.IsNullOrEmpty(tbxHost.Text) ||
string.IsNullOrEmpty(cbxSkillLevel.Text))
            {
                MessageBox.Show("Enter values in all required fields.",
"Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
            else if (!ValidCost(tbxPrice.Text))
            {
                MessageBox.Show("Enter a valid cost.", "Invalid Input",
MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
            else if (!validString(tbxName.Text) ||
!validString(tbxHost.Text))
            {
            }
            else
            {
                if (string.IsNullOrEmpty(rtbxNotes.Text))
                {
                    rtbxNotes.Text = " ";
                }
                DateTime eventStartTime = dtpEventDate.Value.Date +
dtpStartTime.Value.TimeOfDay;
                DateTime eventEndTime = dtpEventDate.Value.Date +
dtpEndTime.Value.TimeOfDay;
                DialogResult result = MessageBox.Show("Save all changes?",
"Save Event", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
                if (result == DialogResult.Yes)
                {
                    if (editEvent != null)
                    {
                        OleDbConnection connection = new OleDbConnection();
                        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source =shopDB.mdb";
                        OleDbCommand cmd = connection.CreateCommand();
                        connection.Open();
                        cmd.CommandText = "UPDATE TableEvents SET EventName
= @en, EventDate = @ed, EventStartTime = @est, EventEndTime = @eet, Host =
@ho, Price = @pa, Capacity = @cap, MaxGuests = @max, SkillLevel = @ski,
EventNotes = @evn WHERE EventID = @eid";
                        cmd.Parameters.Clear();
                        cmd.Parameters.AddWithValue("@en", tbxName.Text);
                        cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(dtpEventDate.Value.ToShortDateString()));
                        cmd.Parameters.AddWithValue("@est",
DateTime.Parse(eventStartTime.ToShortTimeString()));
                        cmd.Parameters.AddWithValue("@eet",
DateTime.Parse(eventEndTime.ToShortTimeString()));
                        cmd.Parameters.AddWithValue("@ho", tbxHost.Text);
                        cmd.Parameters.AddWithValue("@pr", tbxPrice.Text);
                        cmd.Parameters.AddWithValue("@cap",
numCapacity.Value.ToString());
                        cmd.Parameters.AddWithValue("@max",
numMaxGuests.Value.ToString());

```

```

        cmd.Parameters.AddWithValue("@ski",
cbxSkillLevel.Text);

        cmd.Parameters.AddWithValue("@evn", rtbxNotes.Text);
        cmd.Parameters.AddWithValue("@eid", editEvent.id);
        cmd.Connection = connection;
        int status = cmd.ExecuteNonQuery();

        // If attendees have been removed off the waiting
list, change their waiting list status.
        if (attendeesOffWaitingList.Count != 0)
        {
            foreach (EventAttendee offWaitingList in
attendeesOffWaitingList)
            {
                cmd.CommandText = "UPDATE
TableEventAttendees SET WaitingList = False WHERE CustID = @cid AND EventID
= @eid";

                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@cid",
offWaitingList.customer.id);

                cmd.Parameters.AddWithValue("@eid",
editEvent.id);

                int status3 = cmd.ExecuteNonQuery();

            }
        }
        // If attendees have been removed, delete them from
the attendees records.
        if (removedAttendees.Count != 0)
        {
            foreach (EventAttendee removedAttendee in
removedAttendees)
            {
                cmd.CommandText = "DELETE FROM
TableEventAttendees WHERE CustID = @cid AND EventID = @eid";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@cid",
removedAttendee.customer.id);

                cmd.Parameters.AddWithValue("@eid",
editEvent.id);

                int status1 = cmd.ExecuteNonQuery();

            }
        }
        // If attendees have been added, insert new attendee
records.
        if (addedAttendees.Count != 0)
        {
            foreach (EventAttendee addedAttendee in
addedAttendees)
            {
                cmd.CommandText = "INSERT INTO
TableEventAttendees (CustID,EventID,NoOfAttendees,Paid,WaitingList)
VALUES (@cid,@eid,@nat,@pai,@wal) ";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@cid",
addedAttendee.customer.id);

```

```

editEvent.id);
cmd.Parameters.AddWithValue("@eid",
addedAttendee.noOfAttendees);
cmd.Parameters.AddWithValue("@nat",
addedAttendee.paid);
cmd.Parameters.AddWithValue("@wal",
addedAttendee.waitingList);
int status2 = cmd.ExecuteNonQuery();
}
}
connection.Close();
MessageBox.Show("Order edited successfully.", "Edit
Order", MessageBoxButtons.OK, MessageBoxIcon.Information);
Close();
}
else
{
if(dtpEventDate.Value < DateTime.Today)
{
MessageBox.Show("Event must be in the future.",
"Invalid Date", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
else
{
// Adds the event record to the database.
OleDbConnection connection = new
OleDbConnection();
connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
OleDbCommand cmd = connection.CreateCommand();
connection.Open();
cmd.CommandText = "INSERT INTO
TableEvents(EventName, EventDate, EventStartTime, EventEndTime, Host,
Price,Capacity,MaxGuests,SkillLevel,EventNotes)
VALUES (@en,@ed,@est,@eet,@ho,@pr,@cap,@max,@ski,@evn)";
cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@en",
tbxName.Text);
cmd.Parameters.AddWithValue("@ed",
DateTime.Parse(dtpEventDate.Value.ToShortDateString()));
cmd.Parameters.AddWithValue("@est",
DateTime.Parse(eventStartTime.ToShortTimeString()));
cmd.Parameters.AddWithValue("@eet",
DateTime.Parse(eventEndTime.ToShortTimeString()));
cmd.Parameters.AddWithValue("@ho",
tbxHost.Text);
cmd.Parameters.AddWithValue("@pr",
tbxPrice.Text);
cmd.Parameters.AddWithValue("@cap",
numCapacity.Value.ToString());
cmd.Parameters.AddWithValue("@max",
numMaxGuests.Value.ToString());
cmd.Parameters.AddWithValue("@ski",
cbxSkillLevel.Text);
cmd.Parameters.AddWithValue("@evn",
rtbxNotes.Text);

```

```

        cmd.Connection = connection;
        int status = cmd.ExecuteNonQuery();

        // Gets the ID of the event that has just been
created.
        cmd.CommandText = "SELECT EventID FROM
TableEvents ORDER BY EventID DESC";
        cmd.Parameters.Clear();
        int tempEventID =
Convert.ToInt32(cmd.ExecuteScalar());

        // If there are attendees, add each attendee and
link them to the event in the database.
        if (attendeeList.Count != 0)
        {
            foreach (EventAttendee attendee in
attendeeList)
            {
                cmd.CommandText = "INSERT INTO
TableEventAttendees (CustID,EventID,NoOfAttendees,Paid,WaitingList)
VALUES (@cid,@eid,@nat,@pai,@wal)";
                cmd.Parameters.Clear();
                cmd.Parameters.AddWithValue("@cid",
attendee.customer.id);
                cmd.Parameters.AddWithValue("@eid",
tempEventID);
                cmd.Parameters.AddWithValue("@nat",
attendee.noOfAttendees);
                cmd.Parameters.AddWithValue("@pai",
attendee.paid);
                cmd.Parameters.AddWithValue("@wal",
attendee.waitingList);

                int status1 = cmd.ExecuteNonQuery();
            }
        }
        connection.Close();
        MessageBox.Show("New event added successfully.",
"New Event", MessageBoxButtons.OK, MessageBoxIcon.Information);
        Close();
    }
}

}

}

}

}

private void dgvCustomers_CellClick(object sender,
DataGridViewCellEventArgs e) // Show customer details
{
    updateWaitingListLabel();
    index = e.RowIndex;
    refreshCustomerInfo(index);
    selectedAttendee = null;
}

private void attendeeBox_AttendeeSelected(object sender, EventArgs
e) // If an attendee has been clicked on in the list

```

```

    {
        if (sender is AttendeeBox attendeeBox)
        {
            selectedBox = attendeeBox;
            selectedAttendee = attendeeBox.attendee;
            refreshAttendeeInfo(selectedAttendee);
            selectingCust = true;
            foreach (DataGridViewRow row in dgvCustomers.Rows)
            {
                if (selectedAttendee.customer.id ==
Convert.ToInt32(row.Cells[0].Value))
                {
                    dgvCustomers.ClearSelection();
                    row.Selected = true;
                    dgvCustomers.FirstDisplayedScrollingRowIndex =
row.Index;

                    dgvCustomers.Focus();
                    btnAddCustomer.Visible = false;
                }
            }
        }
        private void refreshAttendeeInfo(EventAttendee attendee) // Show
customer details
        {
            tbxCustID.Text = attendee.customer.id.ToString();
            tbxFullName.Text = attendee.customer.fullName;
            tbxPhoneNum.Text = attendee.customer.phoneNum;
            tbxEmailAdd.Text = attendee.customer.emailAddress;
            if (CustomerAdded(attendee.customer.id, attendeeList))
            {
                numGuests.Value = attendee.noOfAttendees - 1;
            }
            else
            {
                numGuests.Value = 0;
            }
            chbxPaid.Checked = attendee.paid;
            if (attendee.waitingList) { lblWaitingList.Visible = true; }
            else { lblWaitingList.Visible = false; }
            btnAddCustomer.Visible = false;
        }
        private void refreshCustomerInfo(int index)
        {
            // Sets the current customer to the customer selected and
displays the information of the customer in the customer panel.
            if (index < 0) // Index cannot be negative.
            {
                index = 0;
            }
            DataGridViewRow selectedRow = dgvCustomers.Rows[index];
            currentCust = new
Customer(Convert.ToInt32(selectedRow.Cells[0].Value),
selectedRow.Cells[1].Value.ToString(),
selectedRow.Cells[2].Value.ToString(),
selectedRow.Cells[3].Value.ToString(),

```



```

selectedRow.Cells[4].Value.ToString(),
selectedRow.Cells[5].Value.ToString(),
Convert.ToInt32(selectedRow.Cells[6].Value),
Convert.ToInt32(selectedRow.Cells[7].Value));
    if (CustomerAdded(currentCust.id, attendeeList)) // If the
selected customer in the dgv has been added to the attendees list already,
then their event attendee details must be displayed instead.
    {
        foreach (EventAttendee a in attendeeList)
        {
            if (a.customer.id == currentCust.id)
            {
                refreshAttendeeInfo(a);
            }
        }
    }
else
{
    tbxCustID.Text = currentCust.id.ToString();
    tbxFullName.Text = currentCust.fullName;
    tbxPhoneNum.Text = currentCust.phoneNum;
    tbxEmailAdd.Text = currentCust.emailAddress;
    numGuests.Value = 0;
    chbxPaid.Checked = true;
    btnAddCustomer.Visible = true;
}
}
private void btnClear_Click(object sender, EventArgs e) // Clears
all fields
{
    DialogResult clearAll = MessageBox.Show("Are you sure you want
to discard all changes?", "Clear",
MessageBoxButtons.YesNo, MessageBoxIcon.Question); // Ensures the user
definitely wishes to clear all data.
    if (clearAll == DialogResult.Yes)
    {
        attendeeList.Clear();
        paidWaitingList.Clear();
        unpaidWaitingList.Clear();
        mainList.Clear();
        tbxName.Text = string.Empty;
        tbxHost.Text = "Purls Of Wisdom";
        tbxPrice.Text = string.Empty;
        rtbxNotes.Text = string.Empty;
        tbxNoOfAttendees.Text = string.Empty;
        numCapacity.Value = 10;
        numMaxGuests.Value = 0;
        cbxSkillLevel.Text = string.Empty;
        flpAttendees.Controls.Clear();
    }
    waitinglist = false;
    lblWaitingList.Visible = false;
    refreshAttendees();
}
private void btnCancel_Click(object sender, EventArgs e) // Closes
the form
{

```

```

        DialogResult result = MessageBox.Show("Are you sure you want to
discard all changes?", "Discard All
Changes", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
        if (result == DialogResult.Yes)
        {
            Close();
        }
    }
    private void chbxAllDay_CheckedChanged(object sender, EventArgs e)
    // Sets the start and end time to opening hours
    {
        if (chbxAllDay.Checked)
        {
            dtpStartTime.Enabled = false;
            dtpEndTime.Enabled = false;
            dtpStartTime.Value = dtpEventDate.Value.Date.AddHours(9);
            dtpEndTime.Value = dtpEventDate.Value.Date.AddHours(20);

        }
        if (!chbxAllDay.Checked)
        {
            dtpStartTime.Enabled = true;
            dtpEndTime.Enabled = true;

        }
    }
    private void dtpStartTime_ValueChanged(object sender, EventArgs e)
    // Ensures the start time is not after the end time and is within opening
    hours
    {
        if (updatingStartTime) // Avoids recursively triggering the
        value changed event
            return;

        try
        {
            updatingStartTime = true;

            if (dtpEndTime.Value < dtpStartTime.Value)
            {
                dtpEndTime.Value = dtpStartTime.Value;
            }
            if (dtpStartTime.Value <
dtpEventDate.Value.Date.AddHours(9))
            {
                dtpStartTime.Value =
dtpEventDate.Value.Date.AddHours(20);
            }
            if (dtpStartTime.Value >
dtpEventDate.Value.Date.AddHours(20))
            {
                dtpStartTime.Value =
dtpEventDate.Value.Date.AddHours(9);
            }
        }
        finally
        {

```

```

        updatingStartTime = false;
    }

    }
    private void dtpEndTime_ValueChanged(object sender, EventArgs e) //
Ensures the start time is not after the end time and is within opening hours
    {
        if (updatingEndTime) // Avoids recursively triggering the value
changed event
            return;

        try
        {
            updatingEndTime = true;

            if (dtpEndTime.Value < dtpStartTime.Value)
            {
                dtpEndTime.Value = dtpStartTime.Value;
            }
            if (dtpEndTime.Value < dtpEventDate.Value.Date.AddHours(9))
            {
                dtpEndTime.Value = dtpEventDate.Value.Date.AddHours(20);
            }
            if (dtpEndTime.Value > dtpEventDate.Value.Date.AddHours(20))
            {
                dtpEndTime.Value = dtpEventDate.Value.Date.AddHours(9);
            }
        }
        finally
        {
            updatingEndTime = false;
        }
    }

    private void btnAddCustomer_Click(object sender, EventArgs e)
    {
        selectedAttendee = new EventAttendee(currentCust, 0,
Convert.ToInt32(numGuests.Value+1), chbxPaid.Checked, waitinglist); // Set
the currently selected attendee to a new instance of attendee with the
current customer

        if (!CustomerAdded(selectedAttendee.customer.id,attendeeList))
// if the attendee isn't in the list, add them to the list
        {
            attendeeList.Add(selectedAttendee); // add the attendee to
the list

            if (editEvent != null) // If editing an event and the
attendee hasn't been removed so far
            {
                if (CustomerAdded(selectedAttendee.customer.id,
removedAttendees))
                {
                    removedAttendees.Remove(selectedAttendee);
                }
                addedAttendees.Add(selectedAttendee);
            }
        }
    }

```

```

        int pos = attendeeList.IndexOf(selectedAttendee); // get the
position of the attendee in the list
        if (CountAttendees(mainList)+selectedAttendee.noOfAttendees
<= numCapacity.Value) // if there are spaces available for the attendee and
their guests
        {
            mainList.Add(selectedAttendee);
        }
        else // if there are not enough spaces add the attendee to
the waiting list
        {
            if (!selectedAttendee.waitingList) // sets the
attendee's waiting list status to true incase previously it was false
            {
                selectedAttendee.setWaitingList(true);
            }
            if (selectedAttendee.paid) // enqueues them in the paid
waiting list if they have paid
            {
                paidWaitingList.Enqueue(selectedAttendee,pos);
            }
            else // enqueues them in the unpaid waiting list if they
have not paid
            {
                unpaidWaitingList.Enqueue(selectedAttendee,pos);
            }
        }

        // Creates new list box item and adds it to the panel of
customers in the event
        attendeeBox = new AttendeeBox(selectedAttendee);
        flpAttendees.Controls.Add(attendeeBox);
        attendeeBox.AttendeeSelected +=
attendeeBox_AttendeeSelected; // Subscribes to attendeeBox's
AttendeeSelected event
    }
    refreshAttendees();
}
private void removeCustomerToolStripMenuItem_Click_1(object sender,
EventArgs e)
{
    if (editEvent != null) // when editing an event's attendees, if
a customer that was just added is then removed
    {
        if
(CustomerAdded(selectedAttendee.customer.id,addedAttendees)||
CustomerAdded(selectedAttendee.customer.id, attendeesOffWaitingList))
        {
            if (CustomerAdded(selectedAttendee.customer.id,
addedAttendees))
            {
                addedAttendees.Remove(selectedAttendee);
            }
            if (CustomerAdded(selectedAttendee.customer.id,
attendeesOffWaitingList))
            {
                attendeesOffWaitingList.Remove(selectedAttendee);
            }
        }
    }
}

```

```

        }
    }
    else // If a customer was added while editing and then
removed, they are not added to the list of removed customers as there would
be no need to remove their record in the database.
    {
        // When editing an event and removing customers,
customers that have been removed need to be kept track of so records can be
deleted.
        removedAttendees.Add(selectedAttendee);
    }

    if (CustomerAdded(selectedAttendee.customer.id,
attendeesOffWaitingList))
    {
        attendeesOffWaitingList.Remove(selectedAttendee);
    }
}

if (CustomerAdded(selectedAttendee.customer.id,mainList)) // If
the customer to be removed is in the main list, they will be removed from
the main list.
{
    mainList.Remove(selectedAttendee);
}
else if (CustomerAdded(selectedAttendee.customer.id,
paidWaitingList.getList())) // If the customer to be removed is in the paid
waiting list, they will be removed from the paid waiting list and it will be
reordered automatically.
{
    paidWaitingList.Remove(selectedAttendee, attendeeList);
}
else if (CustomerAdded(selectedAttendee.customer.id,
unpaidWaitingList.getList())) // If the customer to be removed is in the
unpaid waiting list, they will be removed from the unpaid waiting list and
it will be reordered automatically.
{
    unpaidWaitingList.Remove(selectedAttendee, attendeeList);
}

attendeeList.Remove(selectedAttendee);
updateWaitingList();
refreshAttendees();
}
private void btnRefreshList_Click(object sender, EventArgs e)
{
    updateWaitingList();
    refreshAttendees();
}
private void updateWaitingList()
{
    // If there is space on the main list of attendees, this will
search through the waiting list to allocate attendees to the main list.
    // Priority is given to attendees that have paid, then by their
position in the full list of attendees (order they were added in).
    // Search through those that have paid first

```

```

        while (paidWaitingList.CountAttendees() > 0 &&
CountAttendees(mainList) +
paidWaitingList.GetFront().GetAttendee().noOfAttendees <= numCapacity.Value)
// While there are customers on the waiting list that have paid and the main
list of attendees will not exceed the event capacity
        {
            checkWaitingList(paidWaitingList.Dequeue());
        }
        // Search through those that have not paid if there is still
space in the main list and all paid customers have been allocated a space
        while (unpaidWaitingList.CountAttendees() > 0 &&
CountAttendees(mainList) +
unpaidWaitingList.GetFront().GetAttendee().noOfAttendees <=
numCapacity.Value)
        {
            checkWaitingList(unpaidWaitingList.Dequeue());
        }
        updateWaitingListLabel();
    }
    private void checkWaitingList(EventAttendee waitingListCust) //
Refreshes the list and automatically allocates spaces to any customers on
the waiting list
    {
        foreach (EventAttendee a in attendeeList) // removes the
customer from the attendee list
        {
            if (a.customer.id == waitingListCust.customer.id)
            {
                attendeeList.RemoveAt(attendeeList.IndexOf(a));
                break;
            }
        }
        waitingListCust.setWaitingList(false); // Sets the customer's
waiting list status to false
        mainList.Add(waitingListCust); // Adds the customer to the main
list
        if (editEvent != null)
        {
            attendeesOffWaitingList.Add(waitingListCust);
        }

        bool waitingListExists = false; // used to check if there are
any customers left on the waiting list

        foreach (EventAttendee a in attendeeList) // Insert the customer
back into the whole list at the end of the main list before the start of the
waiting list
        {
            if (a.waitingList) // Gets the index of where to insert the
customer before the waiting list customers
            {
                attendeeList.Insert(attendeeList.IndexOf(a),
waitingListCust); // Inserts the customer
                waitingListExists = true; //
                break;
            }
        }
    }

```

```

        if (!waitingListExists)
        {
            attendeeList.Add(waitingListCust);
        }
    }
    private void numCapacity_ValueChanged(object sender, EventArgs e)
    {
        updateWaitingListLabel();
        if(flpAttendees.Controls.Count>0) // ensures this event only
occurs when there are customers added to the list
        {
            if(capacity > numCapacity.Value) // If the capacity has been
decreased
            {
                if (CountAttendingCustomers(attendeeList) >
numCapacity.Value) // Capacity cannot be less than the number of attendees
in the list currently
                {
                    MessageBox.Show("Remove customers before changing
the capacity.", "Remove Customers", MessageBoxButtons.OK,
MessageBoxIcon.Information);
                    numCapacity.Value =
CountAttendingCustomers(attendeeList);
                }
                else
                {
                    updateWaitingList();
                    refreshAttendees();
                    capacity = (int)numCapacity.Value;
                }
            }
            else
            {
                capacity = (int)numCapacity.Value;
            }
        }
    }
    private void chbxPaid_CheckedChanged(object sender, EventArgs e)
    {
        paidStatusChanged = true;
        btnApplyAttendeeChanges.Visible = true;
    }
    private void numGuests_ValueChanged(object sender, EventArgs e)
    {
        if(numGuests.Value > numMaxGuests.Value)
        {
            MessageBox.Show("Number of guests cannot be more than the
maximum number of guests for this event.", "Maximum Guests",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
            numGuests.Value = numMaxGuests.Value;
        }
        else
        {
            numGuestsChanged = true;
            btnApplyAttendeeChanges.Visible = true;
        }
    }

```

```

    }

    }
    private void rtbxNotes_TextChanged(object sender, EventArgs e)
    {
        // Implements character length limit on order notes. It will
        show the user when the order notes' text length is reaching the limit of
        255.

        if (rtbxNotes.TextLength > 244)
        {
            lblCharLimit.Visible = true;
            lblCharLimit.Text = rtbxNotes.TextLength + "/255";
        }
        else
        {
            lblCharLimit.Visible = false;
        }
    }
    private void tbxSearch_TextChanged(object sender, EventArgs e) //
    Search for the customer by name or email address
    {
        if (!string.IsNullOrEmpty(tbxSearch.Text))
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableCustomers WHERE
(FirstName LIKE @qu OR Surname LIKE @qu OR EmailAddress LIKE @qu) AND
(MemType <> 'Guest' OR MemType <> 'Deleted')";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@qu", "%" + tbxSearch.Text +
"%");

            cmd.Connection = connection;
            reader = cmd.ExecuteReader();
            if (reader.HasRows)
            {
                BindingSource bindingSource = new BindingSource();
                bindingSource.DataSource = reader;
                dgvCustomers.DataSource = bindingSource;
                refreshCustomerInfo(0);
            }
            connection.Close();
        }
        else
        {
            OleDbConnection connection = new OleDbConnection();
            connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmd = connection.CreateCommand();
            OleDbDataReader reader;
            connection.Open();
            cmd.CommandText = "SELECT * FROM TableCustomers WHERE
MemType <> 'Guest' OR MemType <> 'Deleted'";
            cmd.Connection = connection;

```



```

        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvCustomers.DataSource = bindingSource;
        connection.Close();
        refreshCustomerInfo(0);
    }
    highlightCustomersAdded();
}
private void btnSearch_Click(object sender, EventArgs e) // Search
for the customer by name or email address
{
    if (!string.IsNullOrEmpty(tbxSearch.Text))
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableCustomers WHERE
(FirstName LIKE @qu OR Surname LIKE @qu OR EmailAddress LIKE @qu) AND
(MemType <> 'Guest' OR MemType <> 'Deleted')";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@qu", "%" + tbxSearch.Text +
"%");

        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        if (reader.HasRows)
        {
            BindingSource bindingSource = new BindingSource();
            bindingSource.DataSource = reader;
            dgvCustomers.DataSource = bindingSource;
            refreshCustomerInfo(0);
        }
        else
        {
            MessageBox.Show("No customers found.", "No Results",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        connection.Close();
    }
    else
    {
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        OleDbDataReader reader;
        connection.Open();
        cmd.CommandText = "SELECT * FROM TableCustomers WHERE
MemType <> 'Guest' OR MemType <> 'Deleted'";
        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        BindingSource bindingSource = new BindingSource();
        bindingSource.DataSource = reader;
        dgvCustomers.DataSource = bindingSource;
    }
}

```

```

        connection.Close();
        refreshCustomerInfo(0);
    }
    highlightCustomersAdded();
}
private void btnApplyAttendeeChanges_Click(object sender, EventArgs
e) // Applies changes made to the number of guests or paid status of a
customer
{
    if (!selectingCust)
    {
        if (numGuestsChanged)
        {
            bool numGuestsDesc = false;
            int numGuestsCurrent = 0;
            foreach (EventAttendee a in attendeeList)
            {
                if (a.noOfAttendees - 1 > numGuests.Value &&
a.customer == currentCust)
                {
                    numGuestsDesc = true;
                    numGuestsCurrent = a.noOfAttendees - 1;
                }
            }
            if (numGuestsDesc)
            {
                MessageBox.Show("Remove customers with guests before
decreasing the maximum number of guests.");
                numGuests.Value = numGuestsCurrent;
            }
            else
            {
                if (numGuests.Value > numMaxGuests.Value) // If the
user tries to add more guests for a customer than the maximum number of
guests they have set
                {
                    MessageBox.Show("This exceeds the maximum number
of guests allowed per customer for this event.");
                    numGuests.Value = numMaxGuests.Value;
                }
                else if (selectedAttendee != null)
                {
                    if (CountAttendees(mainList) + 1 >
numCapacity.Value) // Won't allow the user to add guests to a customer if at
capacity
                    {
                        MessageBox.Show("Guest could not be added to
customer as event at capacity.");
                    }
                    else
                    {
                        selectedAttendee.setNoOfAttendees(Convert.ToInt32(numGuests.Value + 1));
                    }
                    refreshAttendees();
                    updateWaitingListLabel();
                }
            }
        }
    }
}

```

```

        }
        numGuestsChanged = false;
        btnApplyAttendeeChanges.Visible = false;
    }
    if (selectedAttendee != null && paidStatusChanged)
    {
        // Updates every list

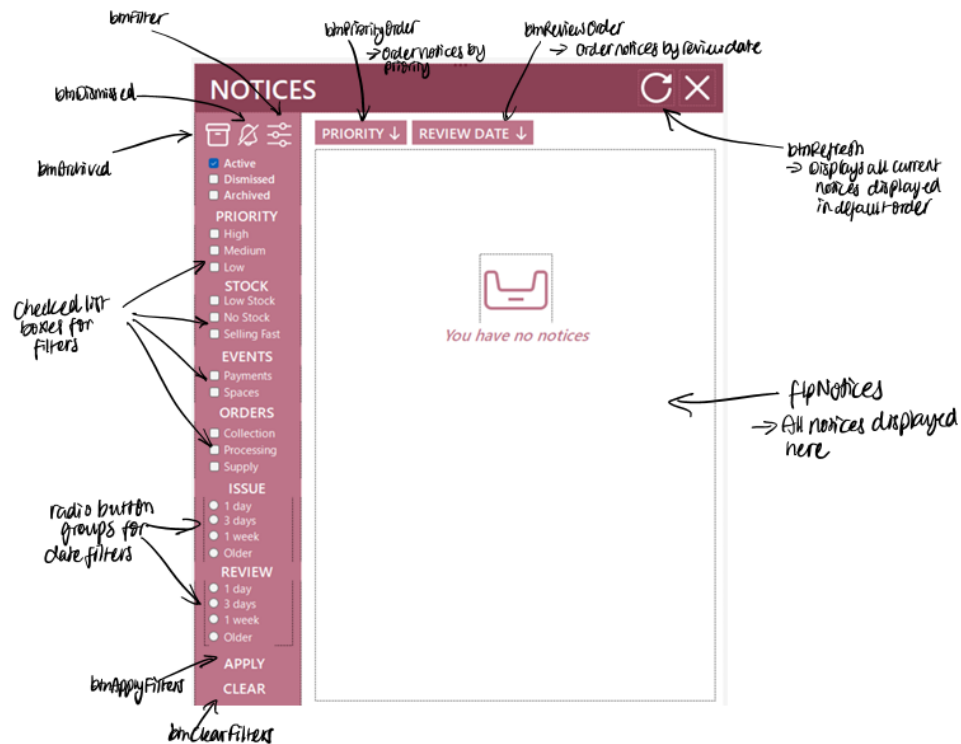
        if (CustomerAdded(selectedAttendee.customer.id,
attendeeList))
        {
            foreach (EventAttendee attendee in attendeeList)
            {
                if (attendee.customer.id ==
selectedAttendee.customer.id)
                {
                    attendee.setPaid(chbxPaid.Checked);
                    break;
                }
            }
            selectedAttendee.setPaid(chbxPaid.Checked);
        }
        if (CustomerAdded(selectedAttendee.customer.id,
mainList))
        {
            foreach (EventAttendee attendee in mainList)
            {
                if (attendee.customer.id ==
selectedAttendee.customer.id)
                {
                    attendee.setPaid(chbxPaid.Checked);
                    break;
                }
            }
            selectedAttendee.setPaid(chbxPaid.Checked);
        }
        if (chbxPaid.Checked)
        {
            if (CustomerAdded(selectedAttendee.customer.id,
unpaidWaitingList.getList()))
            {
                unpaidWaitingList.Remove(selectedAttendee,
attendeeList);
                paidWaitingList.Enqueue(selectedAttendee,
attendeeList.IndexOf(selectedAttendee));
            }
        }
        else
        {
            if (CustomerAdded(selectedAttendee.customer.id,
paidWaitingList.getList()))
            {
                paidWaitingList.Remove(selectedAttendee,
attendeeList);
                unpaidWaitingList.Enqueue(selectedAttendee,
attendeeList.IndexOf(selectedAttendee));
            }
        }
    }
}

```

```
    }  
    refreshAttendees();  
    paidStatusChanged = false;  
    btnApplyAttendeeChanges.Visible = false;  
}  
  
}  
selectingCust = false;  
}  
}
```

Notices.cs

Form Design



Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
```

```

using System.Drawing.Text;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.Tab;

namespace YarnShop
{
    public partial class Notices : Form
    {
        List<Notice> displayedNotices = new List<Notice>();
        bool showDismissed = false;
        bool showArchived = false;
        bool defaultOrder = true;
        DateTime dateToReviewFilter = DateTime.Today.Date;
        DateTime dateIssuedFilter = DateTime.Today.Date;

        bool ascPriority = false;
        bool ascReview = false;

        string currentCommand = "";

        // Make form draggable
        bool drag = false;
        Point dragCursor;
        Point dragForm;

        private void btnClose_Click(object sender, EventArgs e)
        {
            Close();
        }
        private void pbxBar_MouseDown(object sender, MouseEventArgs e)
        {
            drag = true;
            dragCursor = Cursor.Position;
            dragForm = Location;
        }
        private void pbxBar_MouseMove(object sender, MouseEventArgs e)
        {
            if (drag)
            {
                Point difference = Point.Subtract(Cursor.Position, new
                Size(dragCursor));
                Location = Point.Add(dragForm, new Size(difference));
            }
        }
        private void pbxBar_MouseUp(object sender, MouseEventArgs e)
        {
            drag = false;
        }
        public Notices()
        {
            InitializeComponent();
        }
        private void FormNotices_Load(object sender, EventArgs e)
    }
}

```

```

    {
        refreshAllNotices(false);
    }
    private void btnFilter_Click(object sender, EventArgs e) // Show
order by filters
    {
        if (btnPriorityOrder.Visible)
        {
            defaultOrder = true;
            btnPriorityOrder.Visible = false;
            btnReviewOrder.Visible = false;
            ascPriority = false;
            ascReview = false;
            refreshPanel(defaultOrder);
        }
        else
        {
            defaultOrder = false;
            btnPriorityOrder.Visible = true;
            btnReviewOrder.Visible = true;
            btnPriorityOrder.Image = Image.FromFile("down.png");
            btnReviewOrder.Image = Image.FromFile("down.png");
            ascPriority = false;
            ascReview = false;
        }
    }
    private void btnPriorityOrder_Click(object sender, EventArgs e) //
Changes the order of all notices shown according to priority
    {
        if (ascPriority) // If notices are currently ordered by
ascending priority
        {
            btnPriorityOrder.Image = Image.FromFile("down.png");
            ascPriority = false;
        }
        else // If notices are not being ordered by ascending priority
        {
            btnPriorityOrder.Image = Image.FromFile("up.png");
            ascPriority = true;
        }
        displayedNotices = mergeSort(displayedNotices.ToArray<Notice>(),
"Priority", ascPriority).ToList(); // Sorts the notices by priority
        refreshPanel(false);
    }
    private void btnReviewOrder_Click(object sender, EventArgs e) //
Changes the order of all notices shown according to review date
    {
        if (ascReview) // If notices are currently ordered by ascending
review date
        {
            btnReviewOrder.Image = Image.FromFile("down.png");
            ascReview = false;
        }
        else // If notices are currently ordered by descending review
date
        {

```

```

        btnReviewOrder.Image = Image.FromFile("up.png"); ascReview =
true;
    }
    displayedNotices = mergeSort(displayedNotices.ToArray<Notice>(),
    "Date To Review", ascReview).ToList(); // Sorts the notice by review date
    refreshPanel(false);
}
private Notice[] merge(Notice[] left, Notice[] right, string
orderBy, bool asc)
{
    Notice[] merged = new Notice[left.Length + right.Length]; //
merge the items and store in new array
    int indexLeft = 0; // left current position
    int indexRight = 0; // right current position
    int indexMerged = 0; // merged current position

    // while notices left to merge
    while (indexLeft < left.Length && indexRight < right.Length)
    {
        switch (orderBy)
        {
            case "Date To Review": // If ordering by review date
                if (asc) // If ascending order of review date
                {
                    if (left[indexLeft].dateToReview <
right[indexRight].dateToReview)
                    {
                        merged[indexMerged] = left[indexLeft];
                        indexLeft++;
                        indexMerged++;
                    }
                    else
                    {
                        merged[indexMerged] = right[indexRight];
                        indexRight++;
                        indexMerged++;
                    }
                }
                else // Descending order of review date
                {
                    if (left[indexLeft].dateToReview >
right[indexRight].dateToReview)
                    {
                        merged[indexMerged] = left[indexLeft];
                        indexLeft++;
                        indexMerged++;
                    }
                    else
                    {
                        merged[indexMerged] = right[indexRight];
                        indexRight++;
                        indexMerged++;
                    }
                }
            }
        }
        break;
    case "Priority": // If ordering by priority
        if (asc) // If ascending order of priority

```

```

        {
            if (left[indexLeft].priority <
right[indexRight].priority)
            {
                merged[indexMerged] = left[indexLeft];
                indexLeft++;
                indexMerged++;
            }
            else
            {
                merged[indexMerged] = right[indexRight];
                indexRight++;
                indexMerged++;
            }
        }
        else // descending order of priority
        {
            if (left[indexLeft].priority >
right[indexRight].priority)
            {
                merged[indexMerged] = left[indexLeft];
                indexLeft++;
                indexMerged++;
            }
            else
            {
                merged[indexMerged] = right[indexRight];
                indexRight++;
                indexMerged++;
            }
        }
        break;
    }

}

// Add any remaining data from the left array to the merged data
while (indexLeft < left.Length)
{
    merged[indexMerged] = left[indexLeft];
    indexLeft++;
    indexMerged++;
}

// Add any remaining data from the right array to the merged
data
while (indexRight < right.Length)
{
    merged[indexMerged] = right[indexRight];
    indexRight++;
    indexMerged++;
}

return merged;
}
private Notice[] mergeSort(Notice[] notices, string orderBy, bool
asc)

```



```

    {
        Notice[] leftHalf; // left half of items
        Notice[] rightHalf; // right half of items
        Notice[] mergedItems = new Notice[notices.Length]; // store
merged items with each recursive call

        if (notices.Length <= 1) // base case, recursion stops when each
array contains one item
        {
            return notices;
        }
        else
        {
            int midpoint = (notices.Length - 1) / 2; // midpoint index
            int leftSize = midpoint + 1; // Size of the left half array
            int rightSize; // Size of the right half array

            // check if the total number of items is even to determine
sizes of left and right arrays
            if (notices.Length % 2 == 0)
            {
                rightSize = midpoint + 1; // Size of left and right
arrays are equal since the length is even
            }
            else
            {
                rightSize = midpoint; // Right array has one less item
than left array since the length is odd
            }

            leftHalf = new Notice[leftSize]; // Left half
            rightHalf = new Notice[rightSize]; // Right half

            // Left half array populated up to and including the
midpoint
            for (int i = 0; i < leftSize; i++)
            {
                leftHalf[i] = notices[i];
            }

            // Right half array populated after midpoint
            int indexItems = midpoint + 1;
            for (int i = 0; i < rightSize; i++)
            {
                rightHalf[i] = notices[indexItems];
                indexItems++;
            }

            // Using recursion to sort both halves
            leftHalf = mergeSort(leftHalf, orderBy, asc);
            rightHalf = mergeSort(rightHalf, orderBy, asc);

            mergedItems = merge(leftHalf, rightHalf, orderBy, asc); //
Merge both halves

            return mergedItems;
        }
    }

```

```

    }
    public void refreshPanel(bool def)
    {
        if (def) // If all notices are to be sorted by default:
        descending review date, descending priority // if not being sorted by
        directly clicking the sort buttons
        {
            ascReview = false;
            ascPriority = false;
        }
        displayedNotices = mergeSort(displayedNotices.ToArray<Notice>(),
        "Date To Review", ascReview).ToList(); // sorts notices by date to review
        descending
        List<List<Notice>> noticesLists = new List<List<Notice>>(); //
        will store lists of notices for each individual date
        List<Notice> noticesDay = new List<Notice>(); // stores a list
        of notices for one specific date
        Notice prevNotice = null;
        if (displayedNotices.Count != 0) // If there are notices to sort
        {
            foreach (Notice notice in displayedNotices) // Separates all
            notices into lists where every notice in one list has the same review date
            {
                if (prevNotice == null || prevNotice.dateToReview ==
                notice.dateToReview)
                {
                    // If the current notice has the same review date as
                    the previous notice in the list
                    // If the current notice is the first notice in the
                    list
                    noticesDay.Add(notice); // Add the notice to the
                    list of notices for that specific review date
                }
                else
                {
                    // If the current notice does not have the same
                    review date as the previous notice
                    noticesLists.Add(new List<Notice>(noticesDay)); //
                    Add the current list of notices for the specific date to the list of lists
                    of all notices
                    noticesDay = new List<Notice>(); // Creates a new
                    list
                    noticesDay.Add(notice); // Add the current notice to
                    the new list
                }
                prevNotice = notice; // set the previous notice to the
                current notice so the next notice can be compared to it
            }
            if (noticesDay.Count > 0) // ensures last list of notices
            are added
            {
                noticesLists.Add(new List<Notice>(noticesDay));
            }

            for (int i = 0; i < noticesLists.Count; i++) // goes through
            each list of notices for each date and sorts them by priority

```

```

        {
            noticesLists[i] =
mergeSort(noticesLists[i].ToArray<Notice>(), "Priority",
ascPriority).ToList();
        }

        displayedNotices = new List<Notice>();
        foreach (List<Notice> noticesList in noticesLists)
        {
            foreach(Notice notice in noticesList)
            {
                displayedNotices.Add(notice);
            }
        }
        flpNotices.Controls.Clear(); // Clears all notices displayed
        NoticeBox noticeBox;
        if (displayedNotices.Count > 0) // If there are notices to
display
        {
            lblNoNotices.Visible = false;
            pbxNoNotices.Visible = false;
            int height = 40;
            for (int i = 0; i < displayedNotices.Count; i++) // Loops
through the list of notices
            {
                noticeBox = new NoticeBox(displayedNotices[i], height);
                noticeBox.Location = new Point(20, height);
                flpNotices.Controls.Add(noticeBox);
                noticeBox.NoticeUpdated += notice_NoticeUpdated; //
Subscribes to the notice box's event so
                height += 90;
            }
        }
        else // If there are no notices
        {
            lblNoNotices.Visible = true;
            pbxNoNotices.Visible = true;
        }
        flpNotices.HorizontalScroll.Visible = false;
        flpNotices.HorizontalScroll.Enabled = false;
    }
    private void notice_NoticeUpdated(object sender, EventArgs e)
    {
        if(sender is NoticeBox nbx)
        {
            // Updates all notices shown
            refreshAllNotices(true);
        }
    }
    private void refreshAllNotices(bool filter)
    {
        // Gets all notices ordered by date to review, then by priority,
unfiltered.
        displayedNotices.Clear();
        bool error = false;
        OleDbConnection con = new OleDbConnection();

```

```

        con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
        OleDbCommand cmd = con.CreateCommand();
        OleDbDataReader reader;
        con.Open();
        if (!filter) // If all notices are to be refreshed without a
filter, all active notices are retrieved
        {
            currentCommand = "SELECT *, 'TableStockNotices' AS Source
FROM TableStockNotices WHERE Dismissed = False AND Archived = False UNION
ALL SELECT *, 'TableCustOrderNotices' AS Source FROM TableCustOrderNotices
WHERE Dismissed = False AND Archived = False UNION ALL SELECT *,
'TableSupplierOrderNotices' AS Source FROM TableSupplierOrderNotices WHERE
Dismissed = False AND Archived = False UNION ALL SELECT *,
'TableEventNotices' AS Source FROM TableEventNotices WHERE Dismissed = False
AND Archived = False ORDER BY Source ASC";
        }
        cmd.CommandText = currentCommand;
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@dtr",
DateTime.Parse(dateToReviewFilter.ToString()));
        cmd.Parameters.AddWithValue("@dti",
DateTime.Parse(dateIssuedFilter.ToString()));
        cmd.Connection = con;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            try
            {
                bool archived = false;
                bool dismissed = false;
                // Access often uses -1 and 0 instead of True or False
for boolean fields. This ensures data retrieved in this different format
will be used correctly
                if (Convert.ToInt32(reader[7]) == -1 ||
Convert.ToBoolean(reader[7]))
                {
                    dismissed = true;
                }
                if (Convert.ToInt32(reader[8]) == -1 ||
Convert.ToBoolean(reader[8]))
                {
                    archived = true;
                }
                string type = "";
                if (reader[3].ToString().Contains("Stock") ||
reader[3].ToString().Contains("Selling") ||
reader[3].ToString().Contains("Sales"))
                {
                    type = "Stock";
                }
                if (reader[3].ToString().Contains("Collection"))
                {
                    type = "Order Collection";
                }
                if (reader[3].ToString().Contains("Processing"))
                {

```

```

        type = "Order Processing";
    }
    if (reader[3].ToString().Contains("Supply"))
    {
        type = "Supply Delivery";
    }
    if (reader[3].ToString().Contains("Payments"))
    {
        type = "Event Payments";
    }
    if (reader[3].ToString().Contains("Spaces"))
    {
        type = "Event Spaces Available";
    }
    displayedNotices.Add(new
Notice(Convert.ToInt32(reader[1]), Convert.ToInt32(reader[2]),
reader[3].ToString(), Convert.ToInt32(reader[4]),
Convert.ToDateTime(reader[5]), Convert.ToDateTime(reader[6]), dismissed,
archived, type));
    }
    catch
    {
        if (!error)
        {
            MessageBox.Show("Notice data is missing.", "Missing
Data");
            error = true;
        }
    }
    }
    con.Close();
    refreshPanel(defaultOrder);
}
private void clearChecked(CheckedListBox clsbToClear)
{
    for (int i = 0; i < clsbToClear.Items.Count; i++)
    {
        clsbToClear.SetSelected(i, false);
        clsbToClear.SetItemChecked(i, false);
    }
    clsbToClear.ClearSelected();
}
private void btnClear_Click(object sender, EventArgs e) // Clear all
filters
{
    DialogResult result = MessageBox.Show("This will clear all
filters. Proceed?", "Clear All Filters", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
    if (result == DialogResult.Yes)
    {
        clearAllFilters();
        refreshAllNotices(false);
    }
}
private void clearAllFilters()
{
    clearChecked(clsbStock);
}

```

```

        clearChecked(clsbOrders);
        clearChecked(clsbPriority);
        clearChecked(clsbEvents);
        rbtn1dayReview.Checked = false;
        rbtn3daysReview.Checked = false;
        rbtn1weekReview.Checked = false;
        rbtnOlderReview.Checked = false;
        rbtnDay.Checked = false;
        rbtn3days.Checked = false;
        rbtn1week.Checked = false;
        rbtnOlder.Checked = false;
        chbxActive.Checked = false;
        chbxArchived.Checked = false;
        chbxDismissed.Checked = false;
    }
    private void btnApply_Click(object sender, EventArgs e) // Applies
filters
    {
        bool query = false;
        int numTables = 0;
        string commandText = "";
        string eventCommandText = "SELECT *, 'TableEventNotices' AS
Source FROM TableEventNotices ";
        string custOrderCommandText = "SELECT *, 'TableCustOrderNotices'
AS Source FROM TableCustOrderNotices ";
        string supplierOrderCommandText = "SELECT *,
'TableSupplierOrderNotices' AS Source FROM TableSupplierOrderNotices ";
        string stockCommandText = "SELECT *, 'TableStockNotices' AS
Source FROM TableStockNotices ";
        string priorityCommandText = " AND Priority IN (";

        string condition = "WHERE Title IN (";
        List<string> selectedEventFilters =
clsbEvents.CheckedItems.Cast<string>().ToList();
        List<string> selectedOrderFilters =
clsbOrders.CheckedItems.Cast<string>().ToList();
        List<string> selectedStockFilters =
clsbStock.CheckedItems.Cast<string>().ToList();
        List<string> selectedPriorityFilters =
clsbPriority.CheckedItems.Cast<string>().ToList();
        List<string> fullQuery = new List<string>
        {
            eventCommandText,
            custOrderCommandText,
            supplierOrderCommandText,
            stockCommandText
        };

        // REVIEW DATE
        bool dateToReview = false;
        string dateToReviewCondition = "DateToReview ";
        if (rbtn1dayReview.Checked)
        {
            dateToReview = true;
            dateToReviewFilter =
DateTime.Parse(DateTime.Today.Date.AddDays(-1).ToString());
            dateToReviewCondition += "> @dtr ";

```

```

    }
    if (rbtn3daysReview.Checked)
    {
        dateToReview = true;
        dateToReviewFilter =
DateTime.Parse(DateTime.Today.Date.AddDays(-3).ToString());
        dateToReviewCondition += "> @dtr ";
    }
    if (rbtn1weekReview.Checked)
    {
        dateToReview = true;
        dateToReviewFilter =
DateTime.Parse(DateTime.Today.Date.AddDays(-7).ToString());
        dateToReviewCondition += "> @dtr ";
    }
    if (rbtnOlderReview.Checked)
    {
        dateToReview = true;
        dateToReviewFilter =
DateTime.Parse(DateTime.Today.Date.AddDays(-7).ToString());
        dateToReviewCondition += "< @dtr ";
    }

    // ISSUE DATE
    bool dateIssued = false;
    string dateIssuedCondition = "DateIssued ";
    if (rbtnDay.Checked)
    {
        dateIssued = true;
        dateIssuedFilter =
DateTime.Parse(DateTime.Today.Date.AddDays(-1).ToString());
        dateIssuedCondition += "> @dti ";
    }
    if (rbtn3days.Checked)
    {
        dateIssued = true;
        dateIssuedFilter =
DateTime.Parse(DateTime.Today.Date.AddDays(-3).ToString());
        dateIssuedCondition += "> @dti ";
    }
    if (rbtn1week.Checked)
    {
        dateIssued = true;
        dateIssuedFilter =
DateTime.Parse(DateTime.Today.Date.AddDays(-7).ToString());
        dateIssuedCondition += "> @dti ";
    }
    if (rbtnOlder.Checked)
    {
        dateIssued = true;
        dateIssuedFilter =
DateTime.Parse(DateTime.Today.Date.AddDays(-7).ToString());
        dateIssuedCondition += "< @dti ";
    }

    // EVENT NOTICES

```

```

        if (clsbEvents.CheckedItems.Count > 0)
        {
            query = true;
            numTables++;

            commandText += eventCommandText + condition;
            foreach(string eventFilter in selectedEventFilters)
            {
                if (commandText.EndsWith("("))
                {
                    switch (eventFilter)
                    {
                        case "Payments":
                            commandText += "'Event Payments'";
                            break;
                        case "Spaces":
                            commandText += "'Event Spaces Available'";
                            break;
                    }
                }
                // Add the checked filter to the command string.
                else
                {
                    switch (eventFilter)
                    {
                        case "Payments":
                            commandText += ", 'Event Payments'";
                            break;
                        case "Spaces":
                            commandText += ", 'Event Spaces Available'";
                            break;
                    }
                }
            }
            commandText += ")";
            if (clsbPriority.CheckedItems.Count > 0)
            {
                commandText += priorityCommandText;
                foreach(string priorityFilter in
selectedPriorityFilters)
                {
                    if (commandText.EndsWith("("))
                    {
                        switch (priorityFilter)
                        {
                            case "High":
                                commandText += "3";
                                break;
                            case "Medium":
                                commandText += "2";
                                break;
                            case "Low":
                                commandText += "1";
                                break;
                        }
                    }
                }
                // Add the checked filter to the command string.
            }
        }
    }
}

```



```

        else
        {
            switch (priorityFilter)
            {
                case "High":
                    commandText += ",3";
                    break;
                case "Medium":
                    commandText += ",2";
                    break;
                case "Low":
                    commandText += ",1";
                    break;
            }
        }
        commandText += ")";

    }
    if (dateToReview && dateIssued)
    {
        commandText += " AND " + dateToReviewCondition + " AND "
+ dateIssuedCondition + " ";
    }
    else if (dateToReview)
    {
        commandText += " AND " + dateToReviewCondition + " ";
    }
    else if (dateIssued)
    {
        commandText += " AND " + dateIssuedCondition + " ";
    }

    if (!chbxActive.Checked && !chbxArchived.Checked &&
!chbxDismissed.Checked)
    {
        commandText += " AND Archived = False AND Dismissed =
False ";
    }
    else
    {
        if (!(chbxActive.Checked && chbxDismissed.Checked &&
chbxArchived.Checked)) // if all of them arent selected
        {
            if (chbxActive.Checked && chbxDismissed.Checked &&
!chbxArchived.Checked)
            {
                commandText += " AND Archived = False ";
            }
            else if (chbxActive.Checked &&
!chbxDismissed.Checked && chbxArchived.Checked)
            {
                commandText += " AND Dismissed = False ";
            }
            else if (chbxActive.Checked &&
!chbxDismissed.Checked && !chbxArchived.Checked)
            {

```

```

commandText += " AND Archived = False AND
Dismissed = False ";
    }
    else if(!chbxActive.Checked && chbxArchived.Checked)
    {
        commandText += " AND Archived = True ";
    }
    else if(!chbxActive.Checked && chbxDismissed.Checked
&& !chbxArchived.Checked)
    {
        commandText += " AND Dismissed = True ";
    }
    }
}
if (clsbStock.CheckedItems.Count > 0)
{
    numTables++;
    if (query)
    {
        commandText += " UNION ALL ";
    }
    else
    {
        query = true;
    }
    commandText += stockCommandText + condition;
    foreach (string stockFilter in selectedStockFilters)
    {
        if (commandText.EndsWith("("))
        {
            if (selectedStockFilters.Contains("Low Stock") &&
selectedStockFilters.Contains("Selling Fast"))
            {
                commandText += "'Low Stock & Selling Fast','Low
Stock'";
            }
            else
            {
                commandText += "'" + stockFilter + "'";
            }
        }
        // Add the checked filter to the command string.
        else
        {
            commandText += ", '" + stockFilter + "'";
        }
    }
    commandText += ")";
    if (clsbPriority.CheckedItems.Count > 0)
    {
        commandText += priorityCommandText;
        foreach (string priorityFilter in
selectedPriorityFilters)
        {
            if (commandText.EndsWith("("))

```

```

        {
            switch (priorityFilter)
            {
                case "High":
                    commandText += "3";
                    break;
                case "Medium":
                    commandText += "2";
                    break;
                case "Low":
                    commandText += "1";
                    break;
            }
        }
        // Add the checked filter to the command string.
    else
    {
        switch (priorityFilter)
        {
            case "High":
                commandText += ",3";
                break;
            case "Medium":
                commandText += ",2";
                break;
            case "Low":
                commandText += ",1";
                break;
        }
    }
    commandText += ")";

}
if (dateToReview && dateIssued)
{
    commandText += " AND " + dateToReviewCondition + " AND "
+ dateIssuedCondition + " ";
}
else if (dateToReview)
{
    commandText += " AND " + dateToReviewCondition + " ";
}
else if (dateIssued)
{
    commandText += " AND " + dateIssuedCondition + " ";
}

if (!chbxActive.Checked && !chbxArchived.Checked &&
!chbxDismissed.Checked)
{
    commandText += " AND Archived = False AND Dismissed =
False ";
}
else
{

```

```

        if (!(chbxActive.Checked && chbxDismissed.Checked &&
chbxArchived.Checked))// if all of them arent selected
        {
            if (chbxActive.Checked && chbxDismissed.Checked &&
!chbxArchived.Checked)
            {
                commandText += " AND Archived = False ";
            }
            else if (chbxActive.Checked &&
!chbxDismissed.Checked && chbxArchived.Checked)
            {
                commandText += " AND Dismissed = False ";
            }
            else if (chbxActive.Checked &&
!chbxDismissed.Checked && !chbxArchived.Checked)
            {
                commandText += " AND Archived = False AND
Dismissed = False ";
            }
            else if (!chbxActive.Checked &&
chbxArchived.Checked)
            {
                commandText += " AND Archived = True ";
            }
            else if (!chbxActive.Checked &&
chbxDismissed.Checked && !chbxArchived.Checked)
            {
                commandText += " AND Dismissed = True ";
            }
        }
    }
    if (clsbOrders.CheckedItems.Count > 0 &&
selectedOrderFilters.Contains("Supply"))
    {
        numTables++;
        if (query)
        {
            commandText += " UNION ALL ";
        }
        else
        {
            query = true;
        }
        commandText += supplierOrderCommandText + "WHERE Title LIKE
'%Supply Delivery%'";
        if (clsbPriority.CheckedItems.Count > 0)
        {
            commandText += priorityCommandText;
            foreach (string priorityFilter in
selectedPriorityFilters)
            {
                if (commandText.EndsWith("("))
                {
                    switch (priorityFilter)
                    {
                        case "High":

```

```

        commandText += "3";
        break;
    case "Medium":
        commandText += "2";
        break;
    case "Low":
        commandText += "1";
        break;
    }
}
// Add the checked filter to the command string.
else
{
    switch (priorityFilter)
    {
        case "High":
            commandText += ",3";
            break;
        case "Medium":
            commandText += ",2";
            break;
        case "Low":
            commandText += ",1";
            break;
    }
}
commandText += ")";

}
if (dateToReview && dateIssued)
{
    commandText += " AND " + dateToReviewCondition + " AND "
+ dateIssuedCondition + " ";
}
else if (dateToReview)
{
    commandText += " AND " + dateToReviewCondition + " ";
}
else if (dateIssued)
{
    commandText += " AND " + dateIssuedCondition + " ";
}

if (!chbxActive.Checked && !chbxArchived.Checked &&
!chbxDismissed.Checked)
{
    commandText += " AND Archived = False AND Dismissed =
False ";
}
else
{
    if (!(chbxActive.Checked && chbxDismissed.Checked &&
chbxArchived.Checked))// if all of them arent selected
    {
        if (chbxActive.Checked && chbxDismissed.Checked &&
!chbxArchived.Checked)

```

```

        {
            commandText += " AND Archived = False ";
        }
        else if (chbxActive.Checked &&
!chbxDismissed.Checked && chbxArchived.Checked)
        {
            commandText += " AND Dismissed = False ";
        }
        else if (chbxActive.Checked &&
!chbxDismissed.Checked && !chbxArchived.Checked)
        {
            commandText += " AND Archived = False AND
Dismissed = False ";
        }
        else if (!chbxActive.Checked &&
chbxArchived.Checked)
        {
            commandText += " AND Archived = True ";
        }
        else if (!chbxActive.Checked &&
chbxDismissed.Checked && !chbxArchived.Checked)
        {
            commandText += " AND Dismissed = True ";
        }
    }
}
}
if (clsbOrders.CheckedItems.Count > 0 &&
(selectedOrderFilters.Contains("Collection") ||
selectedOrderFilters.Contains("Processing")))
{
    numTables++;
    if (query)
    {
        commandText += " UNION ALL ";
    }
    else
    {
        query = true;
    }
    commandText += custOrderCommandText + "WHERE Title LIKE";
    foreach (string custOrderFilter in selectedOrderFilters)
    {
        if (commandText.EndsWith("E"))
        {
            switch (custOrderFilter)
            {
                case "Collection":
                    commandText += " '%Order Collection
Overdue%'";
                    break;
                case "Processing":
                    commandText += " '%Order Processing
Overdue%'";
                    break;
            }
        }
    }
}

```

```

        // Add the checked filter to the command string.
    else
    {
        switch (custOrderFilter)
        {
            case "Collection":
                commandText += " OR Title LIKE '%Order
Collection Overdue%'";
                break;
            case "Processing":
                commandText += " OR Title LIKE '%Order
Processing Overdue%'";
                break;
        }
    }
    if (clsbPriority.CheckedItems.Count > 0)
    {
        commandText += priorityCommandText;
        foreach (string priorityFilter in
selectedPriorityFilters)
        {
            if (commandText.EndsWith("("))
            {
                switch (priorityFilter)
                {
                    case "High":
                        commandText += "3";
                        break;
                    case "Medium":
                        commandText += "2";
                        break;
                    case "Low":
                        commandText += "1";
                        break;
                }
            }
        }
        // Add the checked filter to the command string.
    else
    {
        switch (priorityFilter)
        {
            case "High":
                commandText += ",3";
                break;
            case "Medium":
                commandText += ",2";
                break;
            case "Low":
                commandText += ",1";
                break;
        }
    }
    }
    commandText += ")";
}
}

```

```

        if (dateToReview && dateIssued)
        {
            commandText += " AND " + dateToReviewCondition + " AND "
+ dateIssuedCondition;
        }
        else if (dateToReview)
        {
            commandText += " AND " + dateToReviewCondition;
        }
        else if (dateIssued)
        {
            commandText += " AND " + dateIssuedCondition;
        }
        if (!chbxActive.Checked && !chbxArchived.Checked &&
!chbxDismissed.Checked)
        {
            commandText += " AND Archived = False AND Dismissed =
False ";
        }
        else
        {
            if (!(chbxActive.Checked && chbxDismissed.Checked &&
chbxArchived.Checked))// if all of them arent selected
            {
                if (chbxActive.Checked && chbxDismissed.Checked &&
!chbxArchived.Checked)
                {
                    commandText += " AND Archived = False ";
                }
                else if (chbxActive.Checked &&
!chbxDismissed.Checked && chbxArchived.Checked)
                {
                    commandText += " AND Dismissed = False ";
                }
                else if (chbxActive.Checked &&
!chbxDismissed.Checked && !chbxArchived.Checked)
                {
                    commandText += " AND Archived = False AND
Dismissed = False ";
                }
                else if (!chbxActive.Checked &&
chbxArchived.Checked)
                {
                    commandText += " AND Archived = True ";
                }
                else if (!chbxActive.Checked &&
chbxDismissed.Checked && !chbxArchived.Checked)
                {
                    commandText += " AND Dismissed = True ";
                }
            }
        }
    }
    if (!query) // if no specific filter is selected
    {
        numTables = 4;
        int numUnions = 0;
    }

```



```

foreach(string str in fullQuery)
{
    bool conditionPresent = false;
    commandText += str;
    if (clsbPriority.CheckedItems.Count > 0)
    {
        conditionPresent = true;
        commandText += "WHERE Priority IN (";
        foreach (string priorityFilter in
selectedPriorityFilters)
        {
            if (commandText.EndsWith("("))
            {
                switch (priorityFilter)
                {
                    case "High":
                        commandText += "3";
                        break;
                    case "Medium":
                        commandText += "2";
                        break;
                    case "Low":
                        commandText += "1";
                        break;
                }
            }
            // Add the checked filter to the command string.
            else
            {
                switch (priorityFilter)
                {
                    case "High":
                        commandText += ",3";
                        break;
                    case "Medium":
                        commandText += ",2";
                        break;
                    case "Low":
                        commandText += ",1";
                        break;
                }
            }
        }
        commandText += ")";
    }
    if (commandText.EndsWith("(") && (dateToReview ||
dateIssued))
    {
        conditionPresent = true;
        if (dateToReview && dateIssued)
        {
            commandText += " AND " + dateToReviewCondition +
" AND " + dateIssuedCondition + " ";
        }
        else if (dateToReview)

```

```

        {
            commandText += " AND " + dateToReviewCondition +
" ";
        }
        else if (dateIssued)
        {
            commandText += " AND " + dateIssuedCondition + "
";
        }
    }
    else
    {
        if (dateToReview && dateIssued)
        {
            conditionPresent = true;
            commandText += "WHERE " + dateToReviewCondition
+ " AND " + dateIssuedCondition + " ";
        }
        else if (dateToReview)
        {
            conditionPresent = true;
            commandText += "WHERE " + dateToReviewCondition
+ " ";
        }
        else if (dateIssued)
        {
            conditionPresent = true;
            commandText += "WHERE " + dateIssuedCondition +
" ";
        }
    }

    if (!chbxActive.Checked && !chbxArchived.Checked &&
!chbxDismissed.Checked)
    {
        if (!conditionPresent)
        {
            commandText += "WHERE Archived = False AND
Dismissed = False ";
        }
        else
        {
            commandText += " AND Archived = False AND
Dismissed = False ";
        }
    }
    else
    {
        if (!(chbxActive.Checked && chbxDismissed.Checked &&
chbxArchived.Checked))// if all of them arent selected
        {
            if (!conditionPresent)
            {
                if (chbxActive.Checked &&
chbxDismissed.Checked && !chbxArchived.Checked)
                {

```

```

                                commandText += "WHERE Archived = False
";
                                }
                                else if (chbxActive.Checked &&
!chbxDismissed.Checked && chbxArchived.Checked)
                                {
                                    commandText += "WHERE Dismissed = False
";
                                }
                                else if (chbxActive.Checked &&
!chbxDismissed.Checked && !chbxArchived.Checked)
                                {
                                    commandText += "WHERE Archived = False
AND Dismissed = False ";
                                }
                                else if (!chbxActive.Checked &&
chbxArchived.Checked)
                                {
                                    commandText += "WHERE Archived = True ";
                                }
                                else if (!chbxActive.Checked &&
chbxDismissed.Checked && !chbxArchived.Checked)
                                {
                                    commandText += "WHERE Dismissed = True
";
                                }
                                }
                                else
                                {
                                    if (chbxActive.Checked &&
chbxDismissed.Checked && !chbxArchived.Checked)
                                    {
                                        commandText += " AND Archived = False ";
                                    }
                                    else if (chbxActive.Checked &&
!chbxDismissed.Checked && chbxArchived.Checked)
                                    {
                                        commandText += " AND Dismissed = False
";
                                    }
                                    else if (chbxActive.Checked &&
!chbxDismissed.Checked && !chbxArchived.Checked)
                                    {
                                        commandText += " AND Archived = False
AND Dismissed = False ";
                                    }
                                    else if (!chbxActive.Checked &&
chbxArchived.Checked)
                                    {
                                        commandText += " AND Archived = True ";
                                    }
                                    else if (!chbxActive.Checked &&
chbxDismissed.Checked && !chbxArchived.Checked)
                                    {
                                        commandText += " AND Dismissed = True ";
                                    }
                                }
                                }

```

```

        }
    }
    numUnions++;
    if (numUnions < 4)
    {
        commandText += "UNION ALL ";
    }

    }

    }
    if (numTables > 1)
    {
        commandText += " ORDER BY DateToReview DESC, Priority DESC,
Source ASC";
    }
    else
    {
        commandText += " ORDER BY DateToReview DESC, Priority DESC";
    }

    currentCommand = commandText;
    refreshAllNotices(true);
}
private void btnArchive_Click(object sender, EventArgs e) // Clears
all filters and shows all archived notices
{
    DialogResult result = MessageBox.Show("This will clear all
filters. Proceed?", "Clear All Filters", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
    if (result == DialogResult.Yes)
    {
        if (!showArchived) // If archived notices aren't currently
being shown, show all archived notices
        {
            currentCommand = "SELECT *, 'TableStockNotices' AS
Source FROM TableStockNotices WHERE Archived = True UNION ALL SELECT *,
'TableCustOrderNotices' AS Source FROM TableCustOrderNotices WHERE Archived
= True UNION ALL SELECT *, 'TableSupplierOrderNotices' AS Source FROM
TableSupplierOrderNotices WHERE Archived = True UNION ALL SELECT *,
'TableEventNotices' AS Source FROM TableEventNotices WHERE Archived = True
ORDER BY DateToReview DESC, Priority DESC, Source ASC;";
            chbxActive.Checked = false;
            chbxArchived.Checked = true;
            chbxDismissed.Checked = false;
            refreshAllNotices(true);
        }
        else
        {
            refreshAllNotices(false);
        }
        clearAllFilters();
    }
}
}

```

```

        private void btnDismissed_Click(object sender, EventArgs e) //
Clears all filters and shows all dismissed notices
        {
            DialogResult result = MessageBox.Show("This will clear all
filters. Proceed?", "Clear All Filters", MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);
            if (result == DialogResult.Yes)
            {
                if (!showDismissed) // If dismissed notices aren't currently
being shown, show all dismissed notices
                {
                    currentCommand = "SELECT *, 'TableStockNotices' AS
Source FROM TableStockNotices WHERE Archived = False AND Dismissed = True
UNION ALL SELECT *, 'TableCustOrderNotices' AS Source FROM
TableCustOrderNotices WHERE Archived = False AND Dismissed = True UNION ALL
SELECT *, 'TableSupplierOrderNotices' AS Source FROM
TableSupplierOrderNotices WHERE Archived = False AND Dismissed = True UNION
ALL SELECT *, 'TableEventNotices' AS Source FROM TableEventNotices WHERE
Archived = False AND Dismissed = True ORDER BY DateToReview DESC, Priority
DESC, Source ASC;";
                    refreshAllNotices(true);
                }
                else
                {
                    refreshAllNotices(false);
                }
                clearAllFilters();
            }
        }

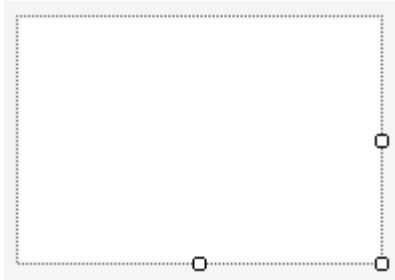
        private void btnRefresh_Click(object sender, EventArgs e) // Show
all current results by default order
        {
            defaultOrder = true;
            btnPriorityOrder.Visible = false;
            btnReviewOrder.Visible = false;
            ascPriority = false;
            ascReview = false;
            refreshPanel(defaultOrder);
        }
    }
}

```

Controls

CalendarBlank

Control Design



Uses

This control has no function other than to fill the main monthly view calendar on Homepage.cs, where there are no dates for the certain box. This control ensures that all dates are aligned correctly in the calendar.

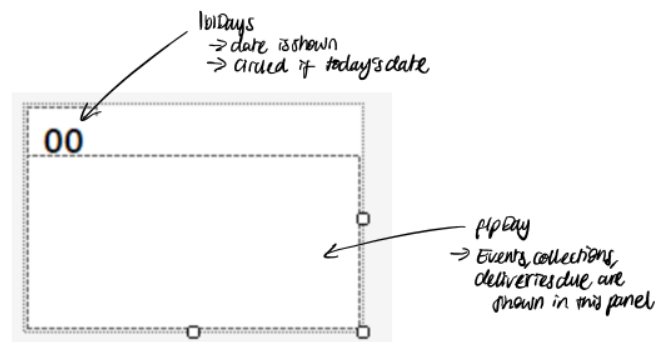
Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public partial class UserControlBlank : UserControl
    {
        // This user control is created to fill the calendar where there is
        not a particular day to be shown.
        public UserControlBlank()
        {
            InitializeComponent();
        }
        private void UserControlBlank_Load(object sender, EventArgs e)
        {
        }
    }
}
```

CalendarDay

Control Design



Uses

This control is used in the calendars in Homepage.cs and Events.cs.

Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Drawing.Imaging;
using System.Linq;
using System.Runtime.Remoting.Metadata.W3cXsd2001;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    // Used in the calendars on the homepage and events forms.
    public partial class CalendarDay : UserControl
    {
        // Attributes
        int day, month, year;
        private bool homepageCalendar { get; set; } // If this object is
        // created on the homepage, a context strip menu must be visible for the user.
        // If it's created on the homepage, a context strip menu is not needed.
        // to pass the Event object to the form
        public delegate void EventSelectedHandler(object sender, EventArgs
        e);

        public event EventSelectedHandler EventSelected;
        private CalendarBox cbx;
        public CalendarDay(bool mainCalendar) // Constructor
        {
            InitializeComponent();
            homepageCalendar = mainCalendar;
        }
        private void UserControlDays_Load(object sender, EventArgs e)
        {
            HorizontalScroll.Visible = false;
        }
    }
}
```

```

        HorizontalScroll.Enabled = false;
    }
    public void days(int numDays,int currentMonth, int currentYear) //
Displays the specific date in the calendar
    {
        lblDays.Visible = true;
        int today = DateTime.Now.Day;
        year = currentYear;
        month = currentMonth;
        day = numDays; //for panel
        // date label is the parameter
        if (numDays < 10)
        {
            lblDays.Text = "0"+numDays;
        }
        else
        {
            lblDays.Text = numDays + "";
        }
    }
    public void displayEvents(List<Event> events) // Displays all the
events for this day
    {
        foreach(Event ev in events)
        {
            cbx = new CalendarBox(ev, homepageCalendar);
            cbx.ShowEvent();
            flpDay.Controls.Add(cbx);
            cbx.EventSelected += cbx_EventSelected; // subscribe to
calendar box's event
        }
    }
    private void cbx_EventSelected(object sender, EventArgs e) //
Invokes the event selected event on this control, allows the user to access
the event shown and go to the events form directly
    {
        EventSelected?.Invoke(sender,e);
    }
    public void displayNumDueDeliveries(int numDays, int currentMonth,
int currentYear) // Gets the number of due deliveries for this specific date
and displays a message
    {
        // Get the specific date
        DateTime date = new DateTime(currentYear, currentMonth,
numDays);
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        connection.Open();
        cmd.CommandText = "SELECT COUNT(*) FROM TableSupplierOrders
WHERE DeliveryDate = @dat";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@dat",
DateTime.Parse(date.ToShortDateString()));
        cmd.Connection = connection;
    }

```



```

        int num = Convert.ToInt32(cmd.ExecuteScalar());
        if (num != 0)
        {
            cbx = new CalendarBox(null, homepageCalendar);
            cbx.ShowDeliveryDue(num);
            flpDay.Controls.Add(cbx);
            cbx.EventSelected += cbx_EventSelected; // subscribe to
calendar box's event
        }
        connection.Close();
    }

    public void displayNumCollections(int numDays, int currentMonth, int
currentYear) // Gets the number of due collections for this specific date
and displays a message
    {
        // Get the specific date
        DateTime date = new DateTime(currentYear, currentMonth,
numDays);

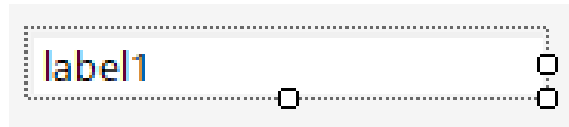
        OleDbConnection connection = new OleDbConnection();
        connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmd = connection.CreateCommand();
        connection.Open();
        cmd.CommandText = "SELECT COUNT(*) FROM TableCustOrders WHERE
CollectionDateEnd = @dat AND DeliveryStatus = 'Ready To Collect'";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@dat",
DateTime.Parse(date.ToShortDateString()));
        cmd.Connection = connection;
        int num = Convert.ToInt32(cmd.ExecuteScalar());
        if (num != 0)
        {
            cbx = new CalendarBox(null, homepageCalendar);
            cbx.ShowCollectionDue(num);
            flpDay.Controls.Add(cbx);
            cbx.EventSelected += cbx_EventSelected; // subscribe to
calendar box's event
        }
        connection.Close();
    }

    private void panell1_Paint(object sender, PaintEventArgs e) //
Highlights today's date
    {
        if (day == DateTime.Now.Day && month == DateTime.Now.Month &&
year == DateTime.Now.Year)
        {
            lblDays.Visible = false;
            Graphics g = this.panell1.CreateGraphics();
            Brush b = new SolidBrush(Color.FromArgb(139, 65, 85));
            g.FillEllipse(b, 0, 0, 26, 26);
            b = new SolidBrush(Color.White);
            g.DrawString(lblDays.Text, lblDays.Font, b, 1, 0);
        }
    }
}

```

CalendarBox

Control Design



Uses

Used to display an event in a different colour in the CalendarDay control, deliveries and collections, and correctly wrap the text.

Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    // This user control displays all events for its day and the number of
    // collections/deliveries due (if any and this control is shown on the
    // homepage)
    public partial class CalendarBox : UserControl
    {
        // Attributes
        public Event @event { get; private set; }
        public string messageType { get; private set; }
        private bool homepageCalendar { get; set; } // If this object is
        // created on the homepage, a context strip menu must be visible for the user.
        // If it's created on the homepage, a context strip menu is not needed.

        // to pass the Event object to the parent form
        public delegate void EventSelectedHandler(object sender, EventArgs
e);
        public event EventSelectedHandler EventSelected;

        public CalendarBox(Event @event, bool mainCalendar) // Constructor
        {
            InitializeComponent();
            this.@event = @event;
            homepageCalendar = mainCalendar;
            textLabel.MaximumSize = new Size(Width - 10, 0); // Allows
            // height to grow but not the width
        }
    }
}
```

```

public Event getEvent
{
    get
    {
        return @event;
    }
}

private void CalendarBox_Load(object sender, EventArgs e)
{
    // If this control shown on the homepage, the user needs to be
    // able to right click this control and access the event information directly
    if (homepageCalendar && @event != null)
    {
        ContextMenuStrip contextMenuStrip = new ContextMenuStrip();
        contextMenuStrip.Font = new Font("Yu Gothic UI",
(float)9.75, FontStyle.Bold);
        contextMenuStrip.Items.Add("Open Event Information",
Image.FromFile("goToFormToolStripMenuItem.png"), GoToEvent);
        this.ContextMenuStrip = contextMenuStrip;
    }
}

private void GoToEvent(object sender, EventArgs e) // Right clicking
on an event in this box allows the user to directly access the event in the
events form
{
    Events newFormEvents = new Events(@event);
    newFormEvents.ShowDialog();
}

public void ShowEvent() // Shows events in a different style
{
    if (@event != null)
    {
        textLabel.Text = @event.eventInfo;
        UpdateSize();
        textLabel.BackColor = Color.FromArgb(189, 116, 137);
        textLabel.ForeColor = SystemColors.ControlLightLight;
        textLabel.Font = new Font("Yu Gothic UI", (float)9.75,
FontStyle.Bold);
    }
}

private void UpdateSize() // If the event name is too long to fit in
the original box size, this will dynamically increase the size of the
control so the full name of the event can be shown
{
    using (Graphics g = this.CreateGraphics())
    {
        SizeF textSize = g.MeasureString(textLabel.Text,
textLabel.Font, textLabel.MaximumSize.Width); // Measures the width of the
event name
        textLabel.Height =
Convert.ToInt32(Math.Ceiling(textSize.Height));
        Height = textLabel.Bottom; // Sets the height of the whole
control to the height of the event name label
    }
}

```

```

        public void ShowDeliveryDue(int num) // Shows the number of
        deliveries due for the day
        {
            messageType = "Delivery Due";
            textLabel.Text = num + " deliveries due";
            UpdateSize();
        }

        public void ShowCollectionDue(int num) // Shows the number of
        collections due for the day
        {
            messageType = "Collection Ends";
            textLabel.Text = num + " collections due";
            textLabel.ForeColor = SystemColors.ControlText;
            UpdateSize();
        }

        private void textLabel_MouseEnter(object sender, EventArgs e) //
        Indicates when the mouse is hovering over an event/notice
        {
            if (@event != null) // If to display an event
            {
                textLabel.BackColor = Color.FromArgb(184, 112, 133);
                textLabel.ForeColor = SystemColors.ControlLight;
            }
            else
            {
                textLabel.BackColor = SystemColors.ControlLight;
            }
        }

        private void textLabel_MouseLeave(object sender, EventArgs e) //
        Indicates when the mouse is hovering over an event/notice
        {
            if (@event != null) // If to display an event
            {
                textLabel.BackColor = Color.FromArgb(189, 116, 137);
                textLabel.ForeColor = SystemColors.ControlLightLight;
            }
            else
            {
                textLabel.BackColor = SystemColors.ControlLightLight;
            }
        }

        private void textLabel_Click(object sender, EventArgs e) // When an
        attendee is clicked, the OnAttendeeClicked event is triggered so the
        information of the attendee can be passed to the parent form.
        {
            // If on the events form and the user has clicked an event, this
            will pass the event object to the form
            if (@event != null) // Passes through parent control
            UserControlDays then to FormEvents
            {
                OnEventClicked(EventArgs.Empty);
            }
        }
    }

```

```

        private void OnEventClicked(EventArgs e) // Invokes the
EventSelected event
        {
            // checks if the event is null before invoking
            EventSelected?.Invoke(this, e);
        }
    }
}

```

AttendeeBox

Control Design



Uses

Used to display customers in an event in different colours according to their paid status and waiting list status.

Code

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    // This is used in Events and NewEvent.
    // Used in the list of attendees.
    // Displays the full name of the customer and the number of guests that
customer has.
    public partial class AttendeeBox : UserControl
    {
        public EventAttendee attendee { get; private set; }

        // to pass the Attendee object to the form
        public delegate void AttendeeSelectedHandler(object sender,
EventArgs e);
        public event AttendeeSelectedHandler AttendeeSelected;
        public AttendeeBox(EventAttendee attendee)
        {
            InitializeComponent();
            this.attendee = attendee;
        }
        private void AttendeeBox_Load(object sender, EventArgs e)
        {
            text();
        }
    }
}

```

```

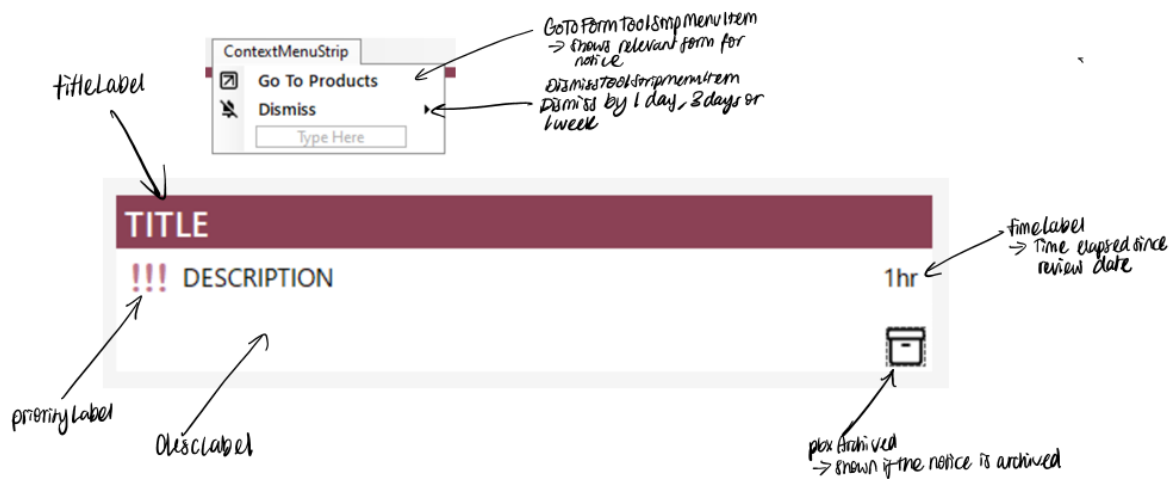
        setColor();
    }
    public void text() // Displays the customer's name and number of
guests (if any)
    {
        if (attendee.noOfAttendees > 1)
        {
            lblAttendee.Text = attendee.customer.idName + " + " +
(attendee.noOfAttendees - 1);
        }
        else
        {
            lblAttendee.Text = attendee.customer.idName;
        }
    }
    public void setColor() // Highlights the customer's name in red if
the customer hasn't paid, or in blue if the customer is on the waiting list.
    {
        if (attendee != null)
        {
            if (!attendee.paid)
            {
                lblAttendee.ForeColor = Color.DarkRed;
            }
            if (attendee.waitingList)
            {
                lblAttendee.ForeColor = Color.DarkBlue;
            }
        }
    }
    private void OnAttendeeClicked(EventArgs e) // When an attendee is
clicked on in the parent forms, this object is passed to the parent form so
the attendee information can be accessed.
    {
        // checks if the attendee is null before invoking
AttendeeSelected?.Invoke(this, e);
    }
    private void lblAttendee_Click(object sender, EventArgs e) // When
an attendee is clicked, the OnAttendeeClicked event is triggered so the
information of the attendee can be passed to the parent form.
    {
        if (attendee != null)
        {
            OnAttendeeClicked(EventArgs.Empty);
        }
    }
    private void lblAttendee_MouseLeave(object sender, EventArgs e) //
Indicates when the mouse isn't hovering over the attendee
    {
        lblAttendee.BackColor = SystemColors.ControlLightLight;
    }
    private void lblAttendee_MouseEnter(object sender, EventArgs e) //
Indicates when the mouse is hovering over the attendee
    {
        lblAttendee.BackColor = SystemColors.Control;
    }
}

```

```
}
```

NoticeBox

Control Design



Uses

This control is used to display each notice in a user-friendly form in the flow layout panel in Notices.

Code

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.OleDb;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    // This is used in Notices.
    // Displays the title, description, priority, time since the review date
    and whether it's been archived or dismissed.
    // Allows the user to access the forms relevant to the notice.
    public partial class NoticeBox : UserControl
    {
        // Attributes
        public Notice notice { get; private set; }

        // to pass the Event object to the parent form
    }
}
```

```

e);
public delegate void NoticeUpdatedHandler(object sender, EventArgs
public event NoticeUpdatedHandler NoticeUpdated;

public NoticeBox(Notice notice, float height) // Constructor
{
    this.notice = notice;
    InitializeComponent();
}

private void NoticeBox1_Load(object sender, EventArgs e)
{
    titleLabel.Text = notice.title;
    descLabel.Text = notice.description;

    if (notice.dismissed) // Shows if the notice has been dismissed
    {
        timeLabel.Text = "DISMISSED UNTIL: " + notice.NoticeTime();
    }
    else
    {
        timeLabel.Text = notice.NoticeTime();
    }

    if (notice.archived) // Shows if the notice has been archived
    {
        pbxArchived.Visible = true;
    }
    else
    {
        pbxArchived.Visible = false;
    }

    priorityLabel.Text = notice.PriorityText(); // Shows the
priority of the notice

    switch (notice.type) // Depending on the type of notice, the
user can access different forms directly from the notice.
    {
        case "Stock":
            if (notice.type.Contains("Sales"))
            {
                GoToFormToolStripMenuItem.Visible = false;
            }
            else
            {
                GoToFormToolStripMenuItem.Text = "Create New
Supplier Order";
            }
            break;
        case "Order Collection":
            GoToFormToolStripMenuItem.Text = "Customer Orders";
            break;
        case "Order Processing":
            GoToFormToolStripMenuItem.Text = "Customer Orders";
            break;
        case "Supply Delivery":

```



```

        GoToFormToolStripMenuItem.Text = "Supplier Orders";
        break;
    case "Event Payments":
        GoToFormToolStripMenuItem.Text = "Events";
        break;
    case "Event Spaces Available":
        GoToFormToolStripMenuItem.Text = "Events";
        break;
    }
}

private void GoToFormToolStripMenuItem_Click(object sender,
EventArgs e)
{
    switch (notice.type) // Depending on the type of notice, the
user can access different forms directly from the notice.
    {
        case "Stock":
            if (!notice.type.Contains("Sales")) // Create a new
supplier order and automatically add the minimum amount of that product
required to clear the notice
            {
                Product product = notice.GetProductDetails();
                NewSupplierOrder newSupOrder = new
NewSupplierOrder(product);
                newSupOrder.ShowDialog();
                Homepage.refreshOrderNotices();
            }
            Homepage.refreshStockNotices();
            break;
        case "Order Collection": // Access the relevant order
CustOrder custOrder = notice.GetCustOrderDetails();
Orders newFormOrders = new Orders(custOrder, null);
newFormOrders.ShowDialog();
Homepage.refreshOrderNotices();
Homepage.refreshStockNotices();
            break;
        case "Order Processing": // Access the relevant order
custOrder = notice.GetCustOrderDetails();
Orders nformOrders = new Orders(custOrder, null);
nformOrders.ShowDialog();
Homepage.refreshOrderNotices();
Homepage.refreshStockNotices();
            break;
        case "Supply Delivery": // Access the relevant order
Order order = notice.GetSupplierOrderDetails();
Orders formOrders = new Orders(null, order);
formOrders.ShowDialog();
Homepage.refreshOrderNotices();
Homepage.refreshStockNotices();
            break;
        case "Event Payments": // Access the relevant event
Event @event = notice.GetEventDetails();
Events newFormEvents = new Events(@event);
newFormEvents.ShowDialog();
Homepage.refreshEventNotices();
            break;
        case "Event Spaces Available": // Access the relevant event

```

```

        @event = notice.GetEventDetails();
        Events formEvents = new Events(@event);
        formEvents.ShowDialog();
        Homepage.refreshEventNotices();
        break;
    }

}

// Dismissing notices
private void dayToolStripMenuItem_Click(object sender, EventArgs e)
{
    // Dismisses the notice for 1 day
    updateNotice(1);
}
private void daysToolStripMenuItem_Click(object sender, EventArgs e)
{
    // Dismisses the notice for 3 days
    updateNotice(3);
}
private void weekToolStripMenuItem_Click(object sender, EventArgs e)
{
    // Dismisses the notice for 1 week
    updateNotice(7);
}
private void updateNotice(int days)
{
    // Dismisses the notice and sets the review date to the number
    of days specified by which tool strip menu item the user clicked.

    string command = "UPDATE ";
    switch (notice.type)
    {
        case "Stock":
            command += "TableStockNotices";
            break;
        case "Order Collection":
            command += "TableCustOrderNotices";
            break;
        case "Order Processing":
            command += "TableCustOrderNotices";
            break;
        case "Supply Delivery":
            command += "TableSupplierOrderNotices";
            break;
        case "Event Payments":
            command += "TableEventNotices";
            break;
        case "Event Spaces Available":
            command += "TableEventNotices";
            break;
    }

    OleDbConnection con = new OleDbConnection();
    con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";

```

```

        OleDbCommand cmd = con.CreateCommand();
        con.Open();
        cmd.CommandText = command + " SET Dismissed = True, DateToReview
= DateAdd('d'," + days + ",DateToReview) WHERE NoticeID = @nid";
        MessageBox.Show(cmd.CommandText);
        cmd.Parameters.AddWithValue("@nid", notice.noticeID);
        cmd.Connection = con;
        int status = cmd.ExecuteNonQuery();
        con.Close();

        // When a notice has been dismissed, the notices shown on the
parent form must update so that this dismissed notice will not be shown.
        // This will trigger an event on the parent form to refresh the
notice panel.
        OnNoticeUpdated(EventArgs.Empty);
    }
    private void OnNoticeUpdated(EventArgs e)
    {
        NoticeUpdated?.Invoke(this, e);
    }
    private void titleLabel_MouseEnter(object sender, EventArgs e) //
Indicates when the mouse is hovering over the notice
    {
        titleLabel.BackColor = Color.FromArgb(153, 72, 93);
    }
    private void titleLabel_MouseLeave(object sender, EventArgs e) //
Indicates when the mouse is hovering over the notice
    {
        titleLabel.BackColor = Color.FromArgb(139, 65, 85);
    }
}
}
}

```

Classes

Customer

Code

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace YarnShop
{
    public class Customer
    {
        // Attributes
    }
}

```

```

private int ID { get; set; }
private string FirstName { get; set; }
private string LastName { get; set; }
private string EmailAddress { get; set; }
private string PhoneNum { get; set; }
private string MemType { get; set; }
private int TotalStars { get; set; }
private int VoucherNum { get; set; }

public Customer(int newID, string newFirstName, string newLastName,
string newPhoneNum, string newEmailAddress, string newMemType, int
newTotalStars, int newVoucherNum) // Constructor
{
    ID = newID;
    FirstName = newFirstName;
    LastName = newLastName;
    PhoneNum = newPhoneNum;
    EmailAddress = newEmailAddress;
    MemType = newMemType;
    TotalStars = newTotalStars;
    VoucherNum = newVoucherNum;
}

// Getters
public int id
{
    get{return ID;}
}
public string firstName
{
    get { return FirstName; }
}
public string lastName
{
    get { return LastName; }
}
public string phoneNum
{
    get { return PhoneNum; }
}
public string emailAddress
{
    get { return EmailAddress; }
}
public string memType
{
    get { return MemType; }
}
public int totalStars
{
    get
    {
        return TotalStars;
    }
}
public int voucherNum
{

```

```

        get
        {
            return VoucherNum;
        }
    }
    public string fullName
    {
        get
        {
            return (FirstName + " " + LastName);
        }
    }
    public string idName
    {
        get
        {
            return "(" + ID + ")" + " " + fullName;
        }
    }

    // Setters
    public void SetMemType(string newMemType)
    {
        MemType = newMemType;
    }
    public void SetTotalStars(int newTotalStars)
    {
        TotalStars = newTotalStars;
    }
    public void SetVoucherNum(int newVoucherNum)
    {
        VoucherNum = newVoucherNum;
    }
}

```

Supplier

Code

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public class Supplier
    {
        // Attributes
        private int ID { get; set; }
        private string SupplierName { get; set; }
        private string EmailAddress { get; set; }
    }
}

```

```

private string PhoneNum { get; set; }
private string Address { get; set; }
private string Postcode { get; set; }
private string URL { get; set; }

public Supplier(int newID, string newSupplierName, string
newEmailAddress, string newPhoneNum, string newAddress, string newPostcode,
string newurl) // Constructor
{
    ID = newID;
    SupplierName = newSupplierName;
    EmailAddress = newEmailAddress;
    PhoneNum = newPhoneNum;
    Address = newAddress;
    Postcode = newPostcode;
    URL = newurl;
}

// Getters
public int id
{
    get { return ID; }
}
public string supplierName
{
    get { return SupplierName; }
}
public string emailAddress
{
    get { return EmailAddress; }
}
public string phoneNum
{
    get { return PhoneNum; }
}
public string address
{
    get { return Address; }
}
public string postcode
{
    get { return Postcode; }
}
public string url
{
    get { return URL; }
}
public string idName
{
    get { return "[" + id + "]" + supplierName; }
}
}
}

```

Product

Code

```
using System;
using System.Collections.Generic;
using System.Data.OleDb;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public class Product
    {
        // Attributes
        private int ID { get; set; }
        private string Name { get; set; }
        private float Cost { get; set; }
        private int Stock { get; set; }
        private string FileName { get; set; }
        private int SupplierID { get; set; }
        private bool Discontinued { get; set; }
        private int LowStock { get; set; }
        private int FullStock { get; set; }
        public Product(int newID, string newName, Single newCost, int
newStock, string newFileName, int newSupplierID, bool discontinued, int
lowStock, int fullStock) // Constructor
        {
            ID = newID;
            Name = newName;
            Cost = newCost;
            Stock = newStock;
            FileName = newFileName;
            SupplierID = newSupplierID;
            this.Discontinued = discontinued;
            this.LowStock = lowStock;
            this.FullStock = fullStock;
        }

        // Getters
        public int id
        {
            get { return ID; }
        }
        public string name
        {
            get { return Name; }
        }
        public float cost
        {
            get { return Cost; }
        }
        public int stock
        {
            get { return Stock; }
        }
    }
}
```

```

    }
    public string filename
    {
        get { return FileName; }
    }
    public int supplierID
    {
        get { return SupplierID; }
    }
    public bool discontinued
    {
        get { return Discontinued; }
    }
    public int lowStock
    {
        get { return LowStock; }
    }
    public int fullStock
    {
        get { return FullStock; }
    }
    public string productInfo
    {
        get
        {
            return (name + " £" + cost.ToString("0.00"));
        }
    }
    public string CostString
    {
        // Returns the total cost of this specific order item in the
order.
        get
        {
            return "£" + cost.ToString("0.00");
        }
    }
    public Supplier Supplier
    {
        get
        {
            Supplier supplier = null;
            bool error = false;
            OleDbConnection con = new OleDbConnection();
            con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0;
Data Source =shopDB.mdb";
            OleDbCommand cmd = con.CreateCommand();
            OleDbDataReader reader;
            con.Open();
            cmd.CommandText = "SELECT * FROM TableSuppliers WHERE
SupplierID = @ssid";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@ssid", supplierID);
            cmd.Connection = con;
            reader = cmd.ExecuteReader();
            while (reader.Read())
            {

```



```

        try
        {
            supplier = new Supplier(Convert.ToInt32(reader[0]),
reader[1].ToString(), reader[2].ToString(), reader[3].ToString(),
reader[4].ToString(), reader[5].ToString(), reader[6].ToString());
        }
        catch
        {
            if (!error)
            {
                MessageBox.Show("Supplier data is missing.",
"Missing Data");
                error = true;
            }
        }
    }
    con.Close();
    return supplier;
}

// Check stock levels
public bool IsLowStock
{
    get
    {
        if (stock < lowStock)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}

public bool OutOfStock
{
    get
    {
        if (stock == 0)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}
}
}
}

```

Order

Code

```
using System;
using System.Collections.Generic;
using System.Data.OleDb;
using System.Deployment.Internal;
using System.Drawing;
using System.Drawing.Printing;
using System.Linq;
using System.Runtime.InteropServices;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.LinkLabel;

namespace YarnShop
{
    public class Order
    {
        // Attributes
        private int OrderID { get; set; }
        private int ID { get; set; }
        private DateTime OrderDate { get; set; }
        private DateTime DeliveryDate { get; set; }
        private string DeliveryStatus { get; set; }
        private double Subtotal { get; set; }
        private double Total { get; set; }
        private string OrderNotes { get; set; }

        public Order(int orderID, int id, DateTime orderDate, DateTime
deliveryDate, string deliveryStatus, double subtotal, double total, string
orderNotes) // Constructor
        {
            this.OrderID = orderID;
            this.ID = id;
            this.OrderDate = orderDate;
            this.DeliveryDate = deliveryDate;
            this.DeliveryStatus = deliveryStatus;
            this.Subtotal = Math.Round(subtotal, 2);
            this.Total = Math.Round(total, 2);
            this.OrderNotes = orderNotes;
        }

        // Getters
        public int orderID
        {
            get { return OrderID; }
        }
        public int id
        {
            get { return ID; }
        }
        public DateTime orderDate
        {

```

```

        get { return OrderDate; }
    }
    public DateTime deliveryDate
    {
        get { return DeliveryDate; }
    }
    public string deliveryStatus
    {
        get { return DeliveryStatus; }
    }
    public double subtotal
    {
        get { return Subtotal; }
    }
    public double total
    {
        get { return Total; }
    }
    public string orderNotes
    {
        get { return OrderNotes; }
    }
    public string status
    {
        get
        {
            if (deliveryStatus == "Order Made" || deliveryStatus ==
"Ready To Collect")
            {
                return "Open";
            }
            else
            {
                return "Archived";
            }
        }
    }
    public Supplier GetSupplierDetails() // Gets the supplier details if
this order is a supplier order
    {
        bool error = false;
        Supplier supplier = null;
        OleDbConnection con = new OleDbConnection();
        con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
        OleDbCommand cmd = con.CreateCommand();
        OleDbDataReader reader;
        con.Open();
        cmd.CommandText = "SELECT * FROM TableSuppliers WHERE SupplierID
= @sid";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@sid", id);
        cmd.Connection = con;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            try

```

```

        {
            supplier = new Supplier(Convert.ToInt32(reader[0]),
Convert.ToString(reader[1]), Convert.ToString(reader[2]),
Convert.ToString(reader[3]), Convert.ToString(reader[4]),
Convert.ToString(reader[5]), Convert.ToString(reader[6]));
        }
        catch
        {
            if (!error)
            {
                MessageBox.Show("Supplier data is missing.",
"Missing Data");
            }
        }
    }
    return supplier;
}
public string subtotalString()
{
    return "£" + subtotal.ToString("0.00");
}
public string totalString()
{
    return "£" + total.ToString("0.00");
}
public virtual List<OrderItem> OrderItemList() // Overridden by
OrderItemList() in CustOrder
{
    // Loads all products of the order into a list of OrderItem
objects.
    bool error = false;
    List<OrderItem> orderItemList = new List<OrderItem>();
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = "SELECT DISTINCT TableProducts.ProductID,
TableProducts.ProductName, TableProducts.Cost, TableProducts.Stock,
TableProducts.FileName, TableProducts.SupplierID,
TableProducts.Discontinued, TableProducts.LowStock,TableProducts.FullStock,
TableSupplierOrderItems.Quantity, TableSupplierOrderItems.QuantityVoided,
TableSupplierOrderItems.QuantityReturned FROM (TableProducts INNER JOIN
TableSupplierOrderItems ON TableProducts.ProductID =
TableSupplierOrderItems.ProductID) INNER JOIN TableSupplierOrders ON
TableSupplierOrderItems.OrderID = TableSupplierOrders.@oid";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@oid", orderID);
    cmd.Connection = connection;
    reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        try
        {
            orderItemList.Add(new OrderItem(new
Product(Convert.ToInt32(reader[0]), Convert.ToString(reader[1]),

```

```

(float)Math.Round(Convert.ToDouble(reader[2]), 2),
Convert.ToInt32(reader[3]), Convert.ToString(reader[4]),
Convert.ToInt32(reader[5]), Convert.ToBoolean(reader[6]),
Convert.ToInt32(reader[7]), Convert.ToInt32(reader[8]),
Convert.ToInt32(reader[9]), Convert.ToInt32(reader[10]),
Convert.ToInt32(reader[11]));
    }
    catch
    {
        if (!error)
        {
            MessageBox.Show("Order item data is missing.",
"Missing Data");
            error = true;
        }
    }
    }
    connection.Close();
    return orderItemList;
}
}
}

```

CustOrder

Uses

SEE: ERROR! REFERENCE SOURCE NOT FOUND.

Code

```

using System;
using System.Collections.Generic;
using System.Data.OleDb;
using System.Drawing.Printing;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public class CustOrder : Order
    {
        // Attributes (rest are inherited)
        private DateTime ColDateEnd { get; set; }
        private string PaymentStatus { get; set; }
        private int StarsEarned { get; set; }
        private string Voucher { get; set; }

        public CustOrder(int orderID, int id, DateTime orderDate, DateTime
deliveryDate, string deliveryStatus, double subtotal, double total, string
orderNotes, DateTime colDateEnd, string paymentStatus, int starsEarned,

```

```

string voucher) : base(orderID, id, orderDate, deliveryDate, deliveryStatus,
subtotal, total, orderNotes) // Constructor
{
    this.ColDateEnd = colDateEnd;
    this.PaymentStatus = paymentStatus;
    this.StarsEarned = starsEarned;
    this.Voucher = voucher;
}

// RETRIEVING DATA
public DateTime colDateEnd
{
    get { return ColDateEnd; }
}
public string paymentStatus
{
    get { return PaymentStatus; }
}
public int starsEarned
{
    get { return StarsEarned; }
}
public string voucher
{
    get { return Voucher; }
}
public Customer GetCustomerDetails // Loads customer details of the
order into a Customer object.
{
    get
    {
        bool error = false;
        Customer cust = null;
        OleDbConnection conCustomers = new OleDbConnection();
        conCustomers.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
        OleDbCommand cmdCustomers = conCustomers.CreateCommand();
        OleDbDataReader readerCustomers;
        conCustomers.Open();
        cmdCustomers.CommandText = "SELECT * FROM TableCustomers
WHERE CustID = @cid";
        cmdCustomers.Parameters.Clear();
        cmdCustomers.Parameters.AddWithValue("@cid", id);
        cmdCustomers.Connection = conCustomers;
        readerCustomers = cmdCustomers.ExecuteReader();
        while (readerCustomers.Read())
        {
            try
            {
                cust = new
Customer(Convert.ToInt32(readerCustomers[0]),
Convert.ToString(readerCustomers[1]), Convert.ToString(readerCustomers[2]),
Convert.ToString(readerCustomers[3]), Convert.ToString(readerCustomers[4]),
Convert.ToString(readerCustomers[5]), Convert.ToInt32(readerCustomers[6]),
Convert.ToInt32(readerCustomers[7]));
            }
            catch

```

```

        {
            if (!error)
            {
                MessageBox.Show("Customer data is missing.",
"Missing Data");
                error = true;
            }
        }
    }
    conCustomers.Close();
    return cust;
}

}

public string CustAbbName // Returns an abbreviated version of the
customer's name e.g. John Smith -> J SMITH
{
    get
    {
        Customer cust = GetCustomerDetails;
        if (cust.lastName.Length <= 5)
        {
            return Convert.ToString((cust.firstName[0] + " " +
cust.lastName).ToUpper());
        }
        else
        {
            return Convert.ToString((cust.firstName[0]) + " " +
cust.lastName.Substring(0, 5).ToUpper());
        }
    }
}

public string VoucherString // Voucher information to be printed
{
    get
    {
        if (voucher == "GOLD")
        {
            return "(-15% GOLD)";
        }
        else if (voucher == "SILVER")
        {
            return "(-10% SILV)";
        }
        else if (voucher == "BRONZE")
        {
            return "(-5% BRON)";
        }
        else
        {
            return "";
        }
    }
}
}

```

```

// CALCULATIONS
public double ApplyVoucher(double amount)
{
    if (amount < 0)
    {
        return 0;
    }
    else
    {
        //reapplying the vouchers
        if (voucher == "GOLD")
        {
            return Math.Round(amount * 0.85, 2);
        }
        if (voucher == "SILVER")
        {
            return Math.Round(amount * 0.9, 2);
        }
        if (voucher == "BRONZE")
        {
            return Math.Round(amount * 0.95, 2);
        }
        else
        {
            return amount;
        }
    }
}

public string CustTotalSaved() // Calculates the total amount of
money the customer has saved so far as a member.
{
    double totalSaved = 0;
    bool error = false;

    OleDbConnection con = new OleDbConnection();
    con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
    OleDbCommand cmd = con.CreateCommand();
    OleDbDataReader reader;
    con.Open();
    cmd.CommandText = "SELECT CustOrderID, Subtotal, Total, Voucher
FROM TableCustOrders WHERE CustID = @cid";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@cid", id);
    cmd.Connection = con;
    reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        double voidedTotal = 0;
        double returnedTotal = 0;
        try
        {
            OleDbConnection conItems = new OleDbConnection();
            conItems.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
            OleDbCommand cmdItems = conItems.CreateCommand();
            OleDbDataReader readerItems;

```



```

        conItems.Open();
        cmdItems.CommandText = "SELECT DISTINCT
TableProducts.Cost,
TableCustOrderItems.QuantityVoided,TableCustOrderItems.QuantityReturned FROM
(TableProducts INNER JOIN TableCustOrderItems ON TableProducts.ProductID =
TableCustOrderItems.ProductID) INNER JOIN TableCustOrders ON
TableCustOrderItems.OrderID = TableCustOrders.@oid";
        cmdItems.Parameters.Clear();
        cmdItems.Parameters.AddWithValue("@oid",
Convert.ToInt32(reader[0]));
        cmdItems.Connection = conItems;
        readerItems = cmdItems.ExecuteReader();
        while (readerItems.Read())
        {
            try
            {
                double multiplier = 1;
                if (reader[3].ToString() == "GOLD")
                {
                    multiplier = 0.85;
                }
                if (reader[3].ToString() == "SILVER")
                {
                    multiplier = 0.9;
                }
                if (reader[3].ToString() == "BRONZE")
                {
                    multiplier = 0.95;
                }
                voidedTotal += multiplier *
Convert.ToDouble(readerItems[0]) * Convert.ToDouble(readerItems[1]);
                returnedTotal += multiplier *
Convert.ToDouble(readerItems[0]) * Convert.ToDouble(readerItems[2]);
            }
            catch
            {
                if (!error)
                {
                    MessageBox.Show("Invoice creation error:
customer order data is missing.", "Order Data Error");
                    error = true;
                }
            }
        }
        conItems.Close();
        totalSaved += Convert.ToDouble(reader[1]) - voidedTotal
- returnedTotal - Convert.ToDouble(reader[2]);
    }
    catch
    {
        if (!error)
        {
            MessageBox.Show("Invoice creation error: customer
order data is missing.", "Order Data Error");
            error = true;
        }
    }
}

```

```

    }
    //MessageBox.Show(Convert.ToString(totalSaved));
    con.Close();
    if (totalSaved < 0)
    {
        totalSaved = 0;
    }
    return "You have saved £" + Math.Round(totalSaved, 2) + " with
us so far!";
}
public string NextVoucherCost() // Returns the amount of money the
customer of this order needs to spend to earn the next voucher
{
    // This is run when the customer hasn't used a voucher in their
order.
    // It will result in either the amount that needs to be spent to
obtain the next voucher, or achieve the next membership status.
    double amount = 0;
    double numOfStars = 0;
    Customer cust = GetCustomerDetails;
    double currentStars = cust.totalStars;
    if (cust.memType == "Bronze")
    {
        if ((currentStars / 150) == 2)
        {
            numOfStars = 450 - currentStars;
            amount = Math.Round(numOfStars / 3, 2);
            return "Spend £" + amount + " total in your next orders
to achieve Silver status!";
        }
        else
        {
            numOfStars = ((Math.Truncate(currentStars / 150) + 1) *
150) - currentStars;
            amount = Math.Round(numOfStars / 3, 2);
            return "Spend £" + amount + " total in your next orders
to earn a 5% discount voucher!";
        }
    }
    else if (cust.memType == "Silver")
    {
        if ((currentStars / 150) == 5)
        {
            numOfStars = 900 - currentStars;
            amount = Math.Round(numOfStars / 3, 2);
            return "Spend £" + amount + " total in your next orders
to achieve Gold status!";
        }
        else
        {
            numOfStars = ((Math.Truncate(currentStars / 150) + 1) *
150) - currentStars;
            amount = Math.Round(numOfStars / 3, 2);
            return "Spend £" + amount + " total in your next orders
to earn a 10% discount voucher!";
        }
    }
}

```

```

    }
    else if (cust.memType == "Gold")
    {
        numOfStars = (((Math.Truncate(currentStars / 150)) + 1) *
150) - currentStars;
        amount = Math.Round(numOfStars / 3, 2);
        return "Spend £" + amount + " total in your next orders to
earn a 15% discount voucher!";
    }
    return "";
}
public double VoidedTotal(List<OrderItem> list) // Returns the total
amount that has been voided
{
    double total = 0;
    foreach (OrderItem item in list)
    {
        total += item.product.cost * item.quantityVoided;
    }
    return Math.Round(total, 2);
}
public double ReturnedTotal(List<OrderItem> list) // Returns the
total amount that has been returned
{
    double total = 0;
    foreach (OrderItem item in list)
    {
        total += item.product.cost * item.quantityReturned;
    }
    return Math.Round(total, 2);
}
public override List<OrderItem> OrderItemList() // Returns a list of
all order items in this customer order. Overrides the method in the parent
class.
{
    // Loads all products of the order into a list of CustOrderItem
objects.
    bool error = false;
    List<OrderItem> orderItemList = new List<OrderItem>();
    OleDbConnection connection = new OleDbConnection();
    connection.ConnectionString = "Provider =
Microsoft.Jet.OLEDB.4.0; Data Source = shopDB.mdb";
    OleDbCommand cmd = connection.CreateCommand();
    OleDbDataReader reader;
    connection.Open();
    cmd.CommandText = "SELECT DISTINCT TableProducts.ProductID,
TableProducts.ProductName, TableProducts.Cost, TableProducts.Stock,
TableProducts.FileName, TableProducts.SupplierID,
TableProducts.Discontinued, TableProducts.LowStock, TableProducts.FullStock,
TableCustOrderItems.Quantity, TableCustOrderItems.QuantityVoided,
TableCustOrderItems.QuantityReturned FROM (TableProducts INNER JOIN
TableCustOrderItems ON TableProducts.ProductID =
TableCustOrderItems.ProductID) INNER JOIN TableCustOrders ON
TableCustOrderItems.OrderID = TableCustOrders.@oid";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@oid", orderID);

```

```

        cmd.Connection = connection;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            try
            {
                orderItemList.Add(new OrderItem(new
Product(Convert.ToInt32(reader[0]), Convert.ToString(reader[1]),
(float)Math.Round(Convert.ToDouble(reader[2]), 2),
Convert.ToInt32(reader[3]), Convert.ToString(reader[4]),
Convert.ToInt32(reader[5]), Convert.ToBoolean(reader[6]),
Convert.ToInt32(reader[7]), Convert.ToInt32(reader[8]),
Convert.ToInt32(reader[9]), Convert.ToInt32(reader[10]),
Convert.ToInt32(reader[11])));
            }
            catch
            {
                if (!error)
                {
                    MessageBox.Show("Order item data is missing.",
"Missing Data");
                    error = true;
                }
            }
        }
        connection.Close();
        return orderItemList;
    }
    public double Savings(List<OrderItem> list)
    {
        return Math.Round(subtotal - ApplyVoucher(VoidedTotal(list)) -
ApplyVoucher(ReturnedTotal(list)) -total, 2);
    }

    // BIG ALGORITHM: Generates invoice to be printed as a pdf or a
receipt
    public void GenerateInvoice(PrintPageEventArgs e, bool
pdf,List<InvoiceLine>lines)
    {
        Customer cust = GetCustomerDetails;
        List<OrderItem> orderItemList = OrderItemList();
        bool voidedItems = false;
        bool returnedItems = false;

        // fonts
        Font titleFont = new Font("Consolas", 13, FontStyle.Bold);
        Font title2Font = new Font("Consolas", 13, FontStyle.Bold |
FontStyle.Italic);
        Font boldFont = new Font("Consolas", 9, FontStyle.Bold);
        Font regularFont = new Font("Consolas", 9, FontStyle.Regular);
        Font italicFont = new Font("Consolas", 9, FontStyle.Italic);

        // brush
        SolidBrush drawBrush = new SolidBrush(Color.Black);

        // formats
        StringFormat drawFormatCenter = new StringFormat();

```

```

        drawFormatCenter.Alignment = StringAlignment.Center;

        StringFormat drawFormatLeft = new StringFormat();

        StringFormat drawFormatRight = new StringFormat();
        drawFormatRight.Alignment = StringAlignment.Far;

        // pens
        Pen blackDashedPen = new Pen(Color.Black);
        blackDashedPen.DashStyle =
System.Drawing.Drawing2D.DashStyle.Dash;
        int widthBound = 220;

        if ((cust == null) || (orderItemList == null))
        {
            MessageBox.Show("Invoice creation error: data is missing.",
"Invoice Error");
        }
        else
        {
            if (Orders.pageNum == 1) // Initialise the list of lines to
be drawn on the receipt.
            {
                if(lines != null)
                {
                    lines.Clear();
                }
                Orders.startIndex = 0;
                e.HasMorePages = false;

                // HEADER INFORMATION
                lines.Add(new InvoiceLine("PURLS OF WISDOM", titleFont,
(widthBound / 2), drawFormatCenter, 30));
                lines.Add(new InvoiceLine("3 Main Street", regularFont,
(widthBound / 2), drawFormatCenter, 20));
                lines.Add(new InvoiceLine("Birmingham", regularFont,
(widthBound / 2), drawFormatCenter, 20));
                lines.Add(new InvoiceLine("B5 3AL", regularFont,
(widthBound / 2), drawFormatCenter, 20));
                lines.Add(new InvoiceLine("01902 330033", regularFont,
(widthBound / 2), drawFormatCenter, 30));

                lines.Add(new InvoiceLine("-----
", regularFont, 10, drawFormatLeft, 20));

                // ORDER INFORMATION

                lines.Add(new InvoiceLine("Invoice", boldFont,
(widthBound / 2), drawFormatCenter, 30));
                lines.Add(new InvoiceLine("Order Date:", boldFont, 10,
drawFormatLeft, 0));
                lines.Add(new
InvoiceLine(orderDate.ToString("dd/MM/yyyy"), regularFont, widthBound - 10,
drawFormatRight, 20));
                lines.Add(new InvoiceLine("Order No:", boldFont, 10,
drawFormatLeft, 0));

```

```

        lines.Add(new InvoiceLine(orderID.ToString(),
regularFont, widthBound - 10, drawFormatRight, 20));
        lines.Add(new InvoiceLine("Date Printed:", boldFont, 10,
drawFormatLeft, 0));
        lines.Add(new
InvoiceLine(DateTime.Now.ToString("dd/MM/yyyy"), regularFont, widthBound -
10, drawFormatRight, 20));
        lines.Add(new InvoiceLine("Time Printed:", boldFont, 10,
drawFormatLeft, 0));
        lines.Add(new
InvoiceLine(DateTime.Now.ToString("HH:mm:ss"), regularFont, widthBound - 10,
drawFormatRight, 30));

        lines.Add(new InvoiceLine("-----
", regularFont, 10, drawFormatLeft, 20));

        // ORDER ITEM HEADERS

        lines.Add(new InvoiceLine("Product", boldFont, 10,
drawFormatLeft, 0));
        lines.Add(new InvoiceLine("Price", boldFont, widthBound
- 10, drawFormatRight, 20));

        lines.Add(new InvoiceLine("-----
", regularFont, 10, drawFormatLeft, 20));

        // PRODUCTS IN THE ORDER
        for (int i = 0; i < orderItemList.Count; i++) // Loops
through each order item in the order
        {
            lines.Add(new
InvoiceLine(orderItemList[i].product.CostString, regularFont, widthBound -
10, drawFormatRight, 0));
            lines.Add(new InvoiceLine(orderItemList[i].quantity
+ "x", regularFont, 10, drawFormatLeft, 0));

            string currentProductLine =
orderItemList[i].product.name;
            // Wraps text if the product name is too long to fit
in one line on the invoice
            if (currentProductLine.Length > 18)
            {
                while (currentProductLine.Length > 18)
                {
                    int space = currentProductLine.LastIndexOf('
', 18); // finds the last space before character 70
                    if (space != -1)
                    {
                        // if at least one space is found
                        lines.Add(new
InvoiceLine(currentProductLine.Substring(0, space), regularFont, 30,
drawFormatLeft, 20));
                        currentProductLine =
currentProductLine.Substring(space + 1); // sets the line to the remaining
string to be printed
                    }
                    else

```

```

        {
            // if no spaces are found
            lines.Add(new
InvoiceLine(currentProductLine.Substring(0, 18), regularFont, 30,
drawFormatLeft, 20));
            currentProductLine =
currentProductLine.Substring(18);
        }
        lines.Add(new InvoiceLine(currentProductLine,
regularFont, 30, drawFormatLeft, 20)); // prints the remaining string
    }
    else
    {
        lines.Add(new InvoiceLine(currentProductLine,
regularFont, 30, drawFormatLeft, 20)); // prints the remaining string
    }

    // IF ANY PRODUCTS WERE VOIDED
    if(deliveryStatus != "Voided")
    {
        if (orderItemList[i].quantityVoided != 0)
        {
            lines.Add(new InvoiceLine("- £" +
Math.Round(orderItemList[i].quantityVoided *
orderItemList[i].product.cost,2).ToString("0.00"), italicFont, widthBound -
10, drawFormatRight, 0));
            lines.Add(new InvoiceLine("- " +
orderItemList[i].quantityVoided + "x" + " VOIDED", italicFont, 30,
drawFormatLeft, 20));
            voidedItems = true;
        }
    }

    // IF ANY PRODUCTS WERE RETURNED
    if(deliveryStatus != "Returned")
    {
        if (orderItemList[i].quantityReturned != 0)
        {
            lines.Add(new InvoiceLine("- £" +
Math.Round(orderItemList[i].quantityReturned *
orderItemList[i].product.cost,2).ToString("0.00"), italicFont, widthBound -
10, drawFormatRight, 0));
            lines.Add(new InvoiceLine("- " +
orderItemList[i].quantityReturned + "x" + " RETURNED", italicFont, 30,
drawFormatLeft, 20));
            returnedItems = true;
        }
    }
}

lines.Add(new InvoiceLine("-----",
regularFont, 10, drawFormatLeft, 20));

// SUBTOTAL, TOTAL, VOIDS AND RETURNS

```

```

        lines.Add(new InvoiceLine("Subtotal:", boldFont, 10,
drawFormatLeft, 0));
        lines.Add(new InvoiceLine("£" + subtotal, regularFont,
widthBound - 10, drawFormatRight, 20));

        if (deliveryStatus == "Voided" || deliveryStatus ==
"Returned")
        {
            if (voucher != "N/A")
            {
                lines.Add(new InvoiceLine("Savings " +
VoucherString, boldFont, 10, drawFormatLeft, 0));
                lines.Add(new InvoiceLine("- £" +
Savings(orderItemList).ToString("0.00"), regularFont, widthBound - 10,
drawFormatRight, 20));
            }
            if (deliveryStatus == "Voided") //if order is voided
            {
                lines.Add(new InvoiceLine("Voided Order",
italicFont, 10, drawFormatLeft, 0));
                lines.Add(new InvoiceLine("- " + (subtotal-
Savings(orderItemList)).ToString("0.00"), regularFont, widthBound - 10,
drawFormatRight, 20));
            }
            if (deliveryStatus == "Returned") //if order is
returned
            {
                lines.Add(new InvoiceLine("Returned Order",
italicFont, 10, drawFormatLeft, 0));
                lines.Add(new InvoiceLine("- " + (subtotal -
Savings(orderItemList)).ToString("0.00"), regularFont, widthBound - 10,
drawFormatRight, 20));
            }
        }
        else
        {
            // If the order is not voided or returned and a
voucher is used
            // Display the savings from using a voucher
            // Display the amount deducted by voided or returned
items
            if (voucher != "N/A")
            {
                lines.Add(new InvoiceLine("Savings " +
VoucherString, boldFont, 10, drawFormatLeft, 0));
                lines.Add(new InvoiceLine("- £" +
Savings(orderItemList).ToString("0.00"), regularFont, widthBound - 10,
drawFormatRight, 20));
            }
            if (voidedItems)
            {
                lines.Add(new InvoiceLine("Voided Items ",
italicFont, 10, drawFormatLeft, 0));
                lines.Add(new InvoiceLine("- £" +
ApplyVoucher(VoidedTotal(orderItemList)).ToString("0.00"), regularFont,
widthBound - 10, drawFormatRight, 20));
            }
        }
    }
}

```



```

        }
        if (returnedItems)
        {
            lines.Add(new InvoiceLine("Returned Items ",
italicFont, 10, drawFormatLeft, 0));
            lines.Add(new InvoiceLine("- £" +
ApplyVoucher(ReturnedTotal(orderItemList)).ToString("0.00"), regularFont,
widthBound - 10, drawFormatRight, 20));

        }
    }

    lines.Add(new InvoiceLine("-----",
regularFont, 10, drawFormatLeft, 20));

    lines.Add(new InvoiceLine("TOTAL:", boldFont, 10,
drawFormatLeft, 0));
    lines.Add(new InvoiceLine("£" + Math.Round(total, 2),
boldFont, widthBound - 10, drawFormatRight, 20));

    lines.Add(new InvoiceLine("-----",
regularFont, 10, drawFormatLeft, 20));

    // CUSTOMER MEMBER INFORMATION

    // Customer information and membership insights are only
    displayed for members.
    if (cust.memType != "Deleted" && cust.memType != "Guest")
    {
        lines.Add(new InvoiceLine("Customer ID:", boldFont,
10, drawFormatLeft, 0));
        lines.Add(new InvoiceLine(cust.id.ToString(),
regularFont, widthBound - 10, drawFormatRight, 20));

        lines.Add(new InvoiceLine("Customer Name:",
boldFont, 10, drawFormatLeft, 0));
        lines.Add(new InvoiceLine(CustAbbName, regularFont,
widthBound - 10, drawFormatRight, 20));

        lines.Add(new InvoiceLine("Member Status:",
boldFont, 10, drawFormatLeft, 0));
        lines.Add(new InvoiceLine(cust.memType, regularFont,
widthBound - 10, drawFormatRight, 20));

        lines.Add(new InvoiceLine("Stars Earned:", boldFont,
10, drawFormatLeft, 0));
        lines.Add(new InvoiceLine(starsEarned.ToString(),
regularFont, widthBound - 10, drawFormatRight, 20));

        lines.Add(new InvoiceLine("Stars Balance:",
boldFont, 10, drawFormatLeft, 0));
        lines.Add(new
InvoiceLine(cust.totalStars.ToString(), regularFont, widthBound - 10,
drawFormatRight, 20));

        // Displays membership insights for the customer.

```

```

// If the customer used a voucher in their order,
display the total amount saved as a member with the shop.
// If the customer did not use a voucher in their
order, display the total amount the customer needs to spend to earn the next
voucher.

lines.Add(new
InvoiceLine("*****", regularFont, (widthBound / 2),
drawFormatCenter, 20));

string currentLine;
if (voucher == "N/A" || deliveryStatus == "Voided"
|| deliveryStatus == "Returned")
{
    currentLine = NextVoucherCost();
}
else
{
    currentLine = CustTotalSaved();
}

// Wraps text if the message is too long to fit in
one line on the receipt
if (currentLine.Length > 20)
{
    while (currentLine.Length > 20)
    {
        int space = currentLine.LastIndexOf(' ',
20); // finds the last space before character 70
        if (space != -1)
        {
            // if at least one space is found
            lines.Add(new
InvoiceLine(currentLine.Substring(0, space), italicFont, (widthBound / 2),
drawFormatCenter, 20));

            currentLine =
currentLine.Substring(space + 1); // sets the line to the remaining string
to be printed
        }
        else
        {
            // if no spaces are found
            lines.Add(new
InvoiceLine(currentLine.Substring(0, 20), italicFont, (widthBound / 2),
drawFormatCenter, 20));

            currentLine = currentLine.Substring(20);
        }
    }
    lines.Add(new InvoiceLine(currentLine,
italicFont, (widthBound / 2), drawFormatCenter, 20)); // prints the
remaining string
}
else
{
    lines.Add(new InvoiceLine(currentLine,
italicFont, (widthBound / 2), drawFormatCenter, 20)); // prints the whole
string
}

```

```

        lines.Add(new
InvoiceLine("*****", regularFont, (widthBound / 2),
drawFormatCenter, 20));

    }

    // THANK YOU MESSAGE
    lines.Add(new InvoiceLine("THANK YOU FOR", title2Font,
widthBound / 2, drawFormatCenter, 20));
    lines.Add(new InvoiceLine("SHOPPING WITH US",
title2Font, widthBound / 2, drawFormatCenter, 20));

    }
    else
    {
        e.HasMorePages = false;
    }

    // Loops through each line to be printed in the list of
lines.
    // Each line to be printed is in a list so that if the
invoice is to be saved as a pdf, the invoice will be printed along multiple
pages instead of cut off at the end of one page.

    if (pdf) // If the invoice is to be saved as a pdf
    {
        // Loops through each line from the start index
specified by the public variable on the orders form. This variable is public
so it can be accessed in this class.
        // startIndex is used to determine which index in the
list to start printing lines from, so lines can be printed along multiple
pages seamlessly.
        for (int i = Orders.startIndex; i < lines.Count; i++)
        {
            if (Orders.height < 1160)
            {
                e.Graphics.DrawString(lines[i].text,
lines[i].font, drawBrush, lines[i].width, Orders.height, lines[i].format);
                Orders.height += lines[i].heightInc;
            }
            else
            {
                e.HasMorePages = true;
                Orders.pageNum++;
                Orders.startIndex = i;
                break;
            }
        }
    }
    else // If the invoice is to be printed directly as a
receipt, there is no need for considering page breaks
    {
        for (int i = 0; i < lines.Count; i++)
        {
            e.Graphics.DrawString(lines[i].text, lines[i].font,
drawBrush, lines[i].width, Orders.height, lines[i].format);

```

```

    }
    }
    }
    }
    }
    Orders.height += lines[i].heightInc;
}

```

OrderItem

Code

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace YarnShop
{
    public class OrderItem
    {
        // Attributes
        private Product Product { get; set; }
        private int Quantity;
        private int QuantityRemaining;
        private int QuantityVoided;
        private int QuantityReturned;
        public OrderItem(Product product, int quantity, int quantityVoided,
int quantityReturned) // Constructor
        {
            this.Quantity = quantity;
            this.QuantityVoided = quantityVoided;
            this.QuantityReturned = quantityReturned;
            this.QuantityRemaining = quantity - quantityVoided -
quantityReturned;
            this.Product = product;
        }

        // Getters
        public int quantity
        {
            get { return Quantity; }
        }
        public int quantityVoided
        {
            get { return QuantityVoided; }
        }
        public int quantityReturned
        {
            get { return QuantityReturned; }
        }
        public int quantityRemaining
```

```

    {
        get { return QuantityRemaining; }
    }
    public Product product
    {
        get { return Product; }
    }
    public void SetQuantity(int newQuantity)
    {
        Quantity = newQuantity;
    }
    //Used as display members
    public string QuantityString
    {
        get
        {
            return Convert.ToString(quantity);
        }
    }
    public string name
    {
        get { return product.name; }
    }
    public string CostString
    {
        get { return product.CostString; }
    }
    public string Desc
    {
        get
        {
            return "x" + Convert.ToString(quantity) + " " +
product.name;
        }
    }
    public string CostTotalString
    {
        // Returns the total cost of this specific order item in the
order.
        get
        {
            return "£" + (product.cost*quantity).ToString("0.00");
        }
    }
}

```

InvoiceLine

Uses

SEE ERROR! REFERENCE SOURCE NOT FOUND.

Code

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace YarnShop
{
    // Used to create a list of lines to print on the invoice for an order.
    public class InvoiceLine
    {
        // Attributes
        private string Text { get; set; }
        private Font Font { get; set; }
        private float Width { get; set; }
        private StringFormat Format { get; set; }
        private float HeightInc { get; set; }

        public InvoiceLine(string text, Font font, float width, StringFormat
format, float heightInc) // Constructor
        {
            this.Text = text;
            this.Font = font;
            this.Width = width;
            this.Format = format;
            this.HeightInc = heightInc;
        }

        // Getters
        public string text
        {
            get { return Text; }
        }
        public Font font
        {
            get { return Font; }
        }
        public float width
        {
            get { return Width; }
        }
        public StringFormat format
        {
            get { return Format; }
        }
        public float heightInc
        {
            get { return HeightInc; }
        }
    }
}
```

Event

Code

```
using System;
using System.Collections.Generic;
using System.Data.OleDb;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public class Event
    {
        // Attributes
        private int ID { get; set; }
        private string Name { get; set; }
        private DateTime Date { get; set; }
        private DateTime StartTime { get; set; }
        private DateTime EndTime { get; set; }
        private string Host { get; set; }
        private float Price { get; set; }
        private int NumAttendees { get; set; }
        private int Capacity { get; set; }
        private int MaxGuests { get; set; }
        private string SkillLevel { get; set; }
        private string EventNotes { get; set; }
        private bool Cancelled { get; set; }

        public Event(int id, string name, DateTime date, DateTime startTime,
            DateTime endTime, string host, float price, int capacity, int maxGuests,
            string skillLevel, string eventNotes, bool cancelled) // Constructor
        {
            ID = id;
            Name = name;
            Date = date;
            StartTime = startTime;
            EndTime = endTime;
            Host = host;
            Price = price;
            Capacity = capacity;
            MaxGuests = maxGuests;
            SkillLevel = skillLevel;
            EventNotes = eventNotes;
            Cancelled = cancelled;

            NumAttendees = GetNumAttending;
        }

        // Getters
        public int id
        {
            get { return ID; }
        }

        public string name
```

```

    {
        get { return Name; }
    }
    public DateTime date
    {
        get { return Date; }
    }
    public DateTime startTime
    {
        get { return StartTime; }
    }
    public DateTime endTime
    {
        get { return EndTime; }
    }
    public string host
    {
        get { return Host; }
    }
    public float price
    {
        get { return Price; }
    }
    public int numAttendees
    {
        get { return NumAttendees; }
    }
    public int capacity
    {
        get { return Capacity; }
    }
    public int maxGuests
    {
        get { return MaxGuests; }
    }
    public string skillLevel
    {
        get { return SkillLevel; }
    }
    public string eventNotes
    {
        get { return EventNotes; }
    }
    public bool cancelled
    {
        get { return Cancelled; }
    }
    public string eventInfo
    {
        get
        {
            return StartTime.ToShortTimeString() + " " + Name;
        }
    }
    public string eventDateTime
    {
        get

```



```

        {
            return Date.ToShortDateString()+"
"+StartTime.ToShortTimeString()+" - "+EndTime.ToShortTimeString();
        }
    }

    // Get totals
    public int GetNumAttending // Gets the number of customers that are
attending the event i.e. are not on the waiting list.
    {
        get
        {
            OleDbConnection con = new OleDbConnection();
            con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0;
Data Source = shopDB.mdb";
            OleDbCommand cmd = con.CreateCommand();
            con.Open();
            cmd.CommandText = "SELECT SUM
(TableEventAttendees.NoOfAttendees) FROM (TableEventAttendees LEFT JOIN
TableEvents ON TableEventAttendees.EventID = TableEvents.EventID) WHERE
TableEventAttendees.WaitingList = False AND TableEvents.EventID =
TableEvents.@eid";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@eid", ID);
            cmd.Connection = con;
            int sum = 0;
            try
            {
                sum = Convert.ToInt32(cmd.ExecuteScalar());
            }
            catch
            {
                sum = 0;
            }
            con.Close();
            return sum;
        }
    }

    public int GetTotalAttendees // Gets the number of attendees on the
event attendees list inc waiting list
    {
        get
        {
            OleDbConnection con = new OleDbConnection();
            con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0;
Data Source = shopDB.mdb";
            OleDbCommand cmd = con.CreateCommand();
            con.Open();
            cmd.CommandText = "SELECT SUM
(TableEventAttendees.NoOfAttendees) FROM (TableEventAttendees LEFT JOIN
TableEvents ON TableEventAttendees.EventID = TableEvents.EventID) WHERE
TableEvents.EventID = TableEvents.@eid";
            cmd.Parameters.Clear();
            cmd.Parameters.AddWithValue("@eid", ID);
            cmd.Connection = con;
            int sum = Convert.ToInt32(cmd.ExecuteScalar());

```

```

        con.Close();
        return sum;
    }
}

public int GetTotalAttendingCustomers // Gets the number of
customers on the event attendees list excluding waiting list
{
    get
    {
        OleDbConnection con = new OleDbConnection();
        con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0;
Data Source = shopDB.mdb";
        OleDbCommand cmd = con.CreateCommand();
        con.Open();
        cmd.CommandText = "SELECT COUNT(*) FROM (TableEventAttendees
LEFT JOIN TableEvents ON TableEventAttendees.EventID = TableEvents.EventID)
WHERE TableEvents.EventID = TableEvents.@eid";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@eid", ID);
        cmd.Connection = con;
        int sum = Convert.ToInt32(cmd.ExecuteScalar());
        con.Close();
        return sum;
    }
}

// Get lists of attendees
public List<EventAttendee> GetEventAttendees // Gets the list of all
attendees for the event
{
    get
    {
        List<EventAttendee> attendees = new List<EventAttendee>();
        bool error = false;
        OleDbConnection con = new OleDbConnection();
        con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0;
Data Source = shopDB.mdb";
        OleDbCommand cmd = con.CreateCommand();
        OleDbDataReader reader;
        con.Open();
        cmd.CommandText = "SELECT DISTINCT TableCustomers.*,
TableEventAttendees.NoOfAttendees, TableEventAttendees.Paid,
TableEventAttendees.WaitingList FROM (TableCustomers INNER JOIN
TableEventAttendees ON TableCustomers.CustID = TableEventAttendees.CustID)
INNER JOIN TableEvents ON TableEventAttendees.EventID = TableEvents.@eid";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@eid", ID);
        cmd.Connection = con;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            try
            {
                attendees.Add(new EventAttendee(new
Customer(Convert.ToInt32(reader[0]), Convert.ToString(reader[1]),
Convert.ToString(reader[2]), Convert.ToString(reader[3]),
Convert.ToString(reader[4]), Convert.ToString(reader[5]),

```

```

Convert.ToInt32(reader[6]), Convert.ToInt32(reader[7]), id,
Convert.ToInt32(reader[8]), Convert.ToBoolean(reader[9]),
Convert.ToBoolean(reader[10]));
    }
    catch
    {
        if (!error)
        {
            MessageBox.Show("Event attendee data is
missing.", "Missing Data");
            error = true;
        }
    }
    }
    con.Close();
    return attendees;
}

}

public List<EventAttendee> GetCustomersUnpaid // Gets the list of
all attendees for the event that have not paid
{
    get
    {
        List<EventAttendee> attendees = GetEventAttendees;
        List<EventAttendee> unpaidAttendees = new
List<EventAttendee>();
        // Gets the list of customers that have not paid for an
event.

        foreach (EventAttendee a in attendees)
        {
            if (!a.paid) // if the attendee has not paid
            {
                unpaidAttendees.Add(a);
            }
        }
        return unpaidAttendees;
    }

}

public List<EventAttendee> GetAttendingCustomersPaid // Gets the
list of all attending customers that have paid and are not on the waiting
list
{
    get
    {
        List<EventAttendee> attendees = GetEventAttendees;
        List<EventAttendee> paidAttendees = new
List<EventAttendee>();
        // Gets the list of attending customers that have paid for
an event.

        foreach (EventAttendee a in attendees)
        {
            if (a.paid && !a.waitingList) // if the attendee has
paid and is not on the waiting list

```

```

        {
            paidAttendees.Add(a);
        }
    }
    return paidAttendees;
}
}
public List<EventAttendee> GetAttendingCustomers // Get all
attendees that are not on the waiting list
{
    get
    {
        List<EventAttendee> attendees = GetEventAttendees;
        List<EventAttendee> mainList = new List<EventAttendee>();
        foreach (EventAttendee a in attendees)
        {
            if (!a.waitingList) // if the attendee is not on the
waiting list
            {
                mainList.Add(a);
            }
        }
        return mainList;
    }
}
public List<EventAttendee> GetWaitingListAttendees // Get all
attendees that are on the waiting list
{
    get
    {
        List<EventAttendee> attendees = GetEventAttendees;
        List<EventAttendee> waitingListAttendees = new
List<EventAttendee>();
        // Gets the list of customers that have paid for an event.
        foreach (EventAttendee a in attendees)
        {
            if (a.waitingList) // if the attendee has paid
            {
                waitingListAttendees.Add(a);
            }
        }
        return waitingListAttendees;
    }
}
public List<EventAttendee> GetOrderedAttendeesList // Get all
attendees and return them in order of paid status
{
    get
    {
        List<EventAttendee> attendees = new List<EventAttendee>();
        attendees.AddRange(GetAttendingCustomers);
        attendees.AddRange(GetPaidWaitingListQueue.getList()); //
Adds the waiting list of paid attendees onto the end of the list of those
not on the waiting list
    }
}

```

```

        attendees.AddRange(GetUnpaidWaitingListQueue.getList()); //
        Adds the waiting list of attendees that have not paid last
        return attendees;
    }
}

// Get queues of attendees
public WaitingListQueue GetPaidWaitingListQueue // Get the queue of
all attendees on the waiting list that have paid
{
    get
    {
        List<EventAttendee> attendees = GetEventAttendees;
        WaitingListQueue paidWaitingList = new WaitingListQueue();
        foreach (EventAttendee a in attendees)
        {
            if (a.waitingList && a.paid) // if the attendee is on
the waiting list and has paid
            {
                paidWaitingList.Enqueue(a, attendees.IndexOf(a));
            }
        }
        return paidWaitingList;
    }
}

public WaitingListQueue GetUnpaidWaitingListQueue // Get the queue
of all attendees on the waiting list that have not paid
{
    get
    {
        List<EventAttendee> attendees = GetEventAttendees;
        WaitingListQueue unpaidWaitingList = new WaitingListQueue();
        foreach (EventAttendee a in attendees)
        {
            if (a.waitingList && !a.paid) // if the attendee is on
the waiting list and has not paid
            {
                unpaidWaitingList.Enqueue(a, attendees.IndexOf(a));
            }
        }
        return unpaidWaitingList;
    }
}
}
}
}

```

EventAttendee

Code

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

```

```

using System.Threading.Tasks;

namespace YarnShop
{
    // Used to record the paid status, waiting list status and number of
    // guests the customer has in an event.
    public class EventAttendee
    {
        // Attributes
        private Customer Customer { get; set; }
        private int EventID { get; set; }
        private int NoOfAttendees { get; set; }
        private bool Paid { get; set; }
        private bool WaitingList { get; set; }

        public EventAttendee(Customer cust, int eventID, int noOfAttendees,
            bool paid, bool waitingList) // Constructor
        {
            this.EventID = eventID;
            this.NoOfAttendees = noOfAttendees;
            this.Paid = paid;
            this.WaitingList = waitingList;
            Customer = cust;
        }

        // Getters
        public int eventID
        {
            get { return EventID; }
        }
        public int noOfAttendees
        {
            get { return NoOfAttendees; }
        }
        public bool paid
        {
            get { return Paid; }
        }
        public bool waitingList
        {
            get { return WaitingList; }
        }
        public Customer customer
        {
            get { return Customer; }
        }

        // Setters
        public void setWaitingList(bool newWaitingList)
        {
            WaitingList = newWaitingList;
        }
        public void setPaid(bool newPaid)
        {
            Paid = newPaid;
        }
        public void setNoOfAttendees(int newNum)
    }
}

```

```

        {
            NoOfAttendees = newNum;
        }
    }
}

```

Notice

Uses

Used to format each notice box correctly for display on Notices.cs.

Code

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data.OleDb;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public class Notice
    {
        // Attributes
        private int NoticeID { get; set; }
        private int ID { get; set; }
        private string Title { get; set; }
        private int Priority { get; set; }
        private DateTime DateIssued { get; set; }
        private DateTime DateToReview { get; set; }
        private bool Dismissed { get; set; }
        private bool Archived { get; set; }
        private string Type { get; set; }
        private string DescriptionText;
        public Notice(int newNoticeID, int newID, string newTitle, int
newPriority, DateTime newDateIssued, DateTime newDateToReview, bool
newDismissed, bool newArchived, string newType) // Constructor
        {
            NoticeID = newNoticeID;
            ID = newID;
            Title = newTitle;
            Priority = newPriority;
            DateIssued = newDateIssued;
            DateToReview = newDateToReview;
            Dismissed = newDismissed;
            Archived = newArchived;
            Type = newType;
            DescriptionText = SetDescription();
        }

        // Getters

```

```

public int noticeID
{
    get { return NoticeID; }
}
public int id
{
    get { return ID; }
}
public string title
{
    get { return Title; }
}
public int priority
{
    get { return Priority; }
}
public DateTime dateIssued
{
    get { return DateIssued; }
}
public DateTime dateToReview
{
    get { return DateToReview; }
}
public bool dismissed
{
    get { return Dismissed; }
}
public bool archived
{
    get { return Archived; }
}
public string type
{
    get { return Type; }
}
public string description
{
    get { return DescriptionText; }
}
public string NoticeTime() // Gets the string that informs how long
ago the notice's review date was
{
    string time = "";
    if (dateToReview < DateTime.Now) // If the review date is before
today's date.
    {
        TimeSpan ts = DateTime.Now - dateToReview;
        if (ts.TotalHours < 1) // If the review datetime was less
than 24 hours ago
        {
            time = Math.Round(ts.TotalMinutes).ToString() + " m";
        }
        else if (ts.TotalHours < 24) // If the review datetime was
less than 24 hours ago
        {
            time = Math.Round(ts.TotalHours).ToString() + " hr";

```



```

        }
        else if (ts.TotalDays < 3)
        {
            time = Math.Round(ts.TotalDays).ToString() + " days";
        }
        else
        {
            time = dateToReview.ToShortDateString();
        }
    }
    else
    {
        time = dateToReview.ToShortDateString();
    }
    return time;
}

public string PriorityText() // Show on the notice box the priority
of the notice
{
    switch (priority)
    {
        case 1:
            return "!";
        case 2:
            return "!!";
        case 3:
            return "!!!";
        default:
            return "";
    }
}

public Product GetProductDetails() // Get the details of the product
related to this notice
{
    bool error = false;
    Product product = null;
    OleDbConnection con = new OleDbConnection();
    con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
    OleDbCommand cmd = con.CreateCommand();
    OleDbDataReader reader;
    con.Open();
    cmd.CommandText = "SELECT * FROM TableProducts WHERE ProductID =
@pid";

    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@pid", id);
    cmd.Connection = con;
    reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        try
        {
            product = new Product(Convert.ToInt32(reader[0]),
reader[1].ToString(), Convert.ToSingle(reader[2]),
Convert.ToInt32(reader[3]), reader[10].ToString(),
Convert.ToInt32(reader[4]), Convert.ToBoolean(reader[11]),
Convert.ToInt32(reader[12]), Convert.ToInt32(reader[13]));

```

```

        }
        catch
        {
            if (!error)
            {
                MessageBox.Show("Product data is missing.", "Missing
Data");
                error = true;
            }
        }
    }
    con.Close();
    return product;
}

public CustOrder GetCustOrderDetails() // Get the details of the
customer order related to this notice
{
    bool error = false;
    CustOrder custOrder = null;
    OleDbConnection con = new OleDbConnection();
    con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
    OleDbCommand cmd = con.CreateCommand();
    OleDbDataReader reader;
    con.Open();
    cmd.CommandText = "SELECT * FROM TableCustOrders WHERE
CustOrderID = @oid";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@oid", id);
    cmd.Connection = con;
    reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        try
        {
            custOrder = new CustOrder(Convert.ToInt32(reader[0]),
Convert.ToInt32(reader[1]), Convert.ToDateTime(reader[2]),
Convert.ToDateTime(reader[3]), reader[4].ToString(),
Convert.ToDouble(reader[8]), Convert.ToDouble(reader[9]),
Convert.ToString(reader[11]), Convert.ToDateTime(reader[6]),
reader[5].ToString(), Convert.ToInt32(reader[7]), reader[10].ToString());

        }
        catch
        {
            if (!error)
            {
                MessageBox.Show("Customer order data is missing.",
"Missing Data");
                error = true;
            }
        }
    }
    con.Close();
}

```

```

        return custOrder;
    }
    public Order GetSupplierOrderDetails() // Get the details of the
supplier order related to this notice
    {
        bool error = false;
        Order supplierOrder = null;
        OleDbConnection con = new OleDbConnection();
        con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
        OleDbCommand cmd = con.CreateCommand();
        OleDbDataReader reader;
        con.Open();
        cmd.CommandText = "SELECT * FROM TableSupplierOrders WHERE
SupplierOrderID = @oid";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@oid", id);
        cmd.Connection = con;
        reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            try
            {
                supplierOrder = new Order(Convert.ToInt32(reader[0]),
Convert.ToInt32(reader[1]), Convert.ToDateTime(reader[2]),
Convert.ToDateTime(reader[3]), reader[4].ToString(),
Convert.ToDouble(reader[5]), Convert.ToDouble(reader[6]),
Convert.ToString(reader[7]));

            }
            catch
            {
                if (!error)
                {
                    MessageBox.Show("Supplier order data is missing.",
"Missing Data");
                    error = true;
                }
            }
        }
        con.Close();
        return supplierOrder;
    }
    public Event GetEventDetails() // Get the details of the event
related to this notice
    {
        bool error = false;
        Event shopEvent = null;
        OleDbConnection con = new OleDbConnection();
        con.ConnectionString = "Provider = Microsoft.Jet.OLEDB.4.0; Data
Source = shopDB.mdb";
        OleDbCommand cmd = con.CreateCommand();
        OleDbDataReader reader;
        con.Open();
        cmd.CommandText = "SELECT * FROM TableEvents WHERE EventID =
@oid";

```

```

cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@oid", id);
cmd.Connection = con;
reader = cmd.ExecuteReader();
while (reader.Read())
{
    bool cancelled = false;
    if (Convert.ToInt32(reader[11]) == -1 ||
Convert.ToBoolean(reader[11]))
    {
        cancelled = true;
    }
    try
    {
        shopEvent = new Event(Convert.ToInt32(reader[0]),
reader[1].ToString(), Convert.ToDateTime(reader[2]),
Convert.ToDateTime(reader[3]), Convert.ToDateTime(reader[4]),
reader[5].ToString(), Convert.ToSingle(reader[6]),
Convert.ToInt32(reader[7]), Convert.ToInt32(reader[8]),
reader[9].ToString(), reader[10].ToString(), cancelled);
    }
    catch
    {
        if (!error)
        {
            MessageBox.Show("Event data is missing.", "Missing
Data");
            error = true;
        }
    }
}
con.Close();
return shopEvent;
}
public string SetDescription() // Depending on the priority and the
type of the notice, the description is generated
{
    string description = "";
    switch (type)
    {
        case "Stock":
            Product product = GetProductDetails();
            description += "(" + product.id.ToString() + ") " +
product.name;
            switch (title)
            {
                case "Low Stock":
                    description += " stock levels are low.";
                    break;
                case "No Stock":
                    description += " is out of stock.";
                    break;
                case "Low Stock & Selling Fast":
                    description += " is selling fast and stock
levels are low.";
                    break;
            }
        }
    }
}

```

```

        case "Selling Fast":
            description += " is selling fast.";
            break;
        case "Low Sales":
            description += " has had a low amount of sales
in the past month.";
            break;
    }
    break;
    case "Order Collection":
        CustOrder custOrderNotCollected = GetCustOrderDetails();
        description += "Order #" +
custOrderNotCollected.orderID.ToString() + " has not been collected by
Customer #" + custOrderNotCollected.id.ToString();
        switch (title)
        {
            case "Order Collection Overdue: 1 day":
                description += " 1 day past the collection
window.";
                break;
            case "Order Collection Overdue: 3 days":
                description += " 3 days past the collection
window.";
                break;
            case "Order Collection Overdue: >5 days":
                description += " >5 days past the collection
window.";
                break;
        }
        break;
    case "Order Processing":
        CustOrder custOrderNotProcessed = GetCustOrderDetails();
        description += "Order #" +
custOrderNotProcessed.orderID.ToString() + " has not been processed for" ;
        switch (title)
        {
            case "Order Processing Overdue: 1 day":
                description += " 1 day past the delivery date.";
                break;
            case "Order Processing Overdue: 3 days":
                description += " 3 days past the delivery
date.";
                break;
            case "Order Processing Overdue: >5 days":
                description += " >5 days past the delivery
date.";
                break;
        }
        break;
    case "Supply Delivery":
        Order delayedOrder = GetSupplierOrderDetails();
        description += "Order #" +
delayedOrder.orderID.ToString() + "is overdue for delivery";
        switch (title)
        {
            case "Supply Delivery Overdue: 1 day":

```

```

description += " 1 day past the expected
delivery date.";
        break;
    case "Supply Delivery Overdue: 3 days":
        description += " 3 days past the expected
delivery date.";
        break;
    case "Supply Delivery Overdue: >5 days":
        description += " >5 days past the expected
delivery date.";
        break;
    }
    break;
case "Event Payments":
    Event shopEvent = GetEventDetails();
    List<EventAttendee> unpaidCustomers =
shopEvent.GetCustomersUnpaid;
    if (unpaidCustomers.Count > 1)
    {
        description += unpaidCustomers.Count.ToString() + "
customers have not paid for the event " + shopEvent.name + ".";
    }
    else if (unpaidCustomers.Count == 1)
    {
        description += unpaidCustomers[0].customer.idName +
" has not paid for the event " + shopEvent.name + ".";
    }
    else
    {
        description += "Resolved for " + shopEvent.name;
    }
    break;
case "Event Spaces Available":
    Event @event = GetEventDetails();
    List<EventAttendee> waitingListCustomers =
@event.GetWaitingListAttendees;
    if (waitingListCustomers.Count > 1)
    {
        description += "There are spaces available in the
event " + @event.name + " for customers on the waiting list.";
    }
    else if (waitingListCustomers.Count == 1)
    {
        description += "There is a space available in the
event " + @event.name + ".";
    }
    else
    {
        description += "Resolved for " + @event.name;
    }
    break;
}
return description;
}
}
}

```

Node

Uses

SEE: CLASS WAITINGLISTQUEUE

Code

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public class Node
    {
        // Attributes
        private EventAttendee attendeeData;
        private int waitingListPosition;
        private Node nextNode;

        public Node(EventAttendee attendeeData, int waitingListPosition) //
Constructor
        {
            this.attendeeData = attendeeData;
            this.waitingListPosition = waitingListPosition;
        }
        public EventAttendee GetAttendee()
        {
            return attendeeData;
        }
        public int GetWaitingListPosition()
        {
            return waitingListPosition;
        }
        public Node GetNext()
        {
            return nextNode;
        }
        public void SetNext(Node next)
        {
            nextNode = next;
        }
    }
}
```

WaitingListQueue

Uses

SEE HOMEPAGE CALENDAR

DESCRIPTION

THE calendar system dynamically generates a monthly view that consolidates events, collections, and deliveries, fulfilling the client's requirement for a centralized schedule overview.

Each time the calendar updates, it clears previous data from the FlowLayoutPanel controls representing each day of the week. The system retrieves the month name and year and displays it, while a DateTime object determines the first day of the month and total days in that month.

To align dates correctly, blank placeholders (CalendarBlank) are inserted before the first actual date. Then, the system loops through each day of the month, creating CalendarDay objects that store the date, events, and relevant shop activities for that day. These objects are then assigned to the appropriate day-of-week panel, ensuring proper alignment. The calendar updates when the user switches months by calling this routine again, ensuring the displayed data is always accurate and formatted correctly.

Note: See CalendarDay and CalendarBox code for the DisplayEvents, DisplayNumDueDeliveries and DisplayNumCollections subroutines.

Pseudocode

```
SUBROUTINE UpdateCalendar(month, year)
    # Clear previous calendar data
    ClearControls()

    # Get first day of the month and total days in month
    startOfMonth ← Date(year, month, 1)
    daysInMonth ← GetDaysInMonth(year, month)
    daysOfWeek ← GetDayOfWeek(startOfMonth)

    # Adjust daysOfWeek to match indexing (Sunday = 7)
    IF daysOfWeek = 0 THEN
        daysOfWeek ← 7
    ENDIF

    # Add blank placeholders to align first date correctly
    FOR i ← 1 TO daysOfWeek - 1
        dayName ← GetDayName(i)
        blankControl ← CreateBlankControl()
        ADD blankControl TO FlowLayoutPanel(dayName)
    ENDFOR

    # Populate the calendar with dates and data for each day
    FOR i ← 1 TO daysInMonth
        dateString ← FormatDate(i, month, year)
        date ← ConvertToDate(dateString)

        // Create calendar day object with necessary details
        calendarDay ← CreateCalendarDay(i, month, year)
        calendarDay.DisplayEvents(LoadEvents(i, month, year))
        calendarDay.DisplayNumDueDeliveries(i, month, year)
```



```

calendarDay.DisplayNumCollections(i, month, year)

# Assign the calendar day to the correct weekday panel
dayOfWeek ← GetDayOfWeek(date)
ADD calendarDay TO FlowLayoutPanel(dayOfWeek)
END FOR
END SUBROUTINE

```

Filtering Products

Description

To easily navigate through all products in the system, I have created a filtering system to easily find selections of products and allow the user to easily do so while creating orders and checking inventory.

This algorithm dynamically builds and executes a SQL query to filter products based on user selected criteria using the checked list boxes, check boxes and radio buttons on the user interface. It ensures that multiple filters can be applied simultaneously and results are shown in the data grid view.

Pseudocode

```

SUBROUTINE ApplyFilters()
  # Base query
  baseCommandText ← "SELECT * FROM TableProducts WHERE"

  # Initialize filter conditions
  categoryFilterCommand ← " AND Category IN ("
  fibreFilterCommand ← " AND FibreMaterial IN ("
  colourFilterCommand ← " AND Colour IN ("
  supplierFilterCommand ← " AND SupplierID IN ("

  # Get selected filters from UI
  selectedCategoryFilters ← GetCheckedItems(clsbCategory)
  selectedFibreFilters ← GetCheckedItems(clsbFibreMaterial)
  selectedColourFilters ← GetCheckedItems(clsbColour)
  selectedStockFilters ← GetCheckedItems(clsbStock)
  selectedSupplierFilters ← GetCheckedItems(clsbSuppliers)

  # Apply stock filters
  IF selectedStockFilters IS NOT EMPTY THEN
    filtersApplied ← TRUE
    FOR EACH stockFilter IN selectedStockFilters DO
      condition ← ""
      IF stockFilter = "In Stock" THEN
        condition ← " Stock > 0"
      ELSE IF stockFilter = "No Stock" THEN
        condition ← " Stock = 0"
      ELSE IF stockFilter = "Low Stock" THEN
        condition ← " Stock < LowStock"
      END IF

      # Append stock condition to query
      IF baseCommandText ENDS WITH "E" THEN
        baseCommandText ← baseCommandText + " (" + condition
      ELSE
        baseCommandText ← baseCommandText + " OR" + condition
      END IF
    END FOR
  END IF
END SUBROUTINE

```

```

        END IF
    END FOR
    baseCommandText ← baseCommandText + ")"
ELSE
    baseCommandText ← baseCommandText + " Stock >= 0"
END IF

# Apply product type filter
IF rbtnAll IS NOT CHECKED THEN
    baseCommandText ← baseCommandText + " AND ProductType = @ya"
ENDIF

# Apply category filters
IF selectedCategoryFilters IS NOT EMPTY THEN
    filtersApplied ← TRUE
    AppendFiltersToCommand(categoryFilterCommand,
selectedCategoryFilters)
    baseCommandText ← baseCommandText + categoryFilterCommand + ")"
END IF

# Apply fibre material filters
IF selectedFibreFilters IS NOT EMPTY THEN
    filtersApplied ← TRUE
    AppendFiltersToCommand(fibreFilterCommand, selectedFibreFilters)
    baseCommandText ← baseCommandText + fibreFilterCommand + ")"
END IF

# Apply colour filters
IF selectedColourFilters IS NOT EMPTY THEN
    filtersApplied ← TRUE
    AppendFiltersToCommand(colourFilterCommand, selectedColourFilters)
    baseCommandText ← baseCommandText + colourFilterCommand + ")"
END IF

# Apply supplier filters
IF selectedSupplierFilters IS NOT EMPTY THEN
    filtersApplied ← TRUE
    FOREACH supplierFilter IN selectedSupplierFilters DO
        IF supplierFilterCommand ENDS WITH "(" THEN
            supplierFilterCommand ← supplierFilterCommand +
supplierFilter.id
        ELSE
            supplierFilterCommand ← supplierFilterCommand + "," +
supplierFilter.id
        ENDIF
    END FOREACH
    baseCommandText ← baseCommandText + supplierFilterCommand + ")"
END IF

# Apply discontinued filter
IF chbxDiscontinuedFilter IS CHECKED THEN
    filtersApplied ← TRUE
    baseCommandText ← baseCommandText + " AND Discontinued = True"
ELSE
    baseCommandText ← baseCommandText + " AND Discontinued = False"
END IF

```

```

# Determine selected product type
IF rbtnYarn IS CHECKED THEN
    tempProductType ← "y"
ELSE IF rbtnAcc IS CHECKED THEN
    tempProductType ← "a"
END IF

# Execute query
connection ← OpenDatabaseConnection("shopDB.mdb")
cmd ← CreateCommand(baseCommandText, connection)
cmd.AddParameter("@ya", tempProductType)
reader ← ExecuteQuery(cmd)

# Load products into dgv, if any results
# If no results, show a message box
IF reader HAS ROWS THEN
    dgvProducts.DataSource ← reader
    currentCommand ← baseCommandText
    tbxSearch.Text ← ""
ELSE
    ShowMessage("No products found.", "No Results")
END IF

CloseDatabaseConnection(connection)
HighlightProductsStock()
END SUBROUTINE

# Function to append multiple filters to the command string
FUNCTION AppendFiltersToCommand(filterCommand, selectedFilters)
    FOREACH filter IN selectedFilters DO
        IF filterCommand ENDS WITH "(" THEN
            filterCommand ← filterCommand + "'" + filter + "'"
        ELSE
            filterCommand ← filterCommand + "," + "'" + filter + "'"
        END IF
    END FOREACH
    RETURN filterCommand
END FUNCTION

```

Event Waiting List Allocation

Code

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace YarnShop
{
    public class WaitingListQueue
    {
        private Node frontPointer;
        private Node rearPointer;
    }
}

```

```

private List<EventAttendee> list = new List<EventAttendee>();

public Node GetFront()
{
    return frontPointer;
}

public void SetFront(Node newFront)
{
    frontPointer = newFront;
}

public Node GetRear()
{
    return rearPointer;
}

public void SetRear(Node newRear)
{
    rearPointer = newRear;
}

public bool QueueEmpty()
{
    if (frontPointer == null)
    {
        return true;
    }
    else
    {
        return false;
    }
}

public bool Contains(EventAttendee attendee)
{
    foreach (EventAttendee a in list)
    {
        if (a.customer.id == attendee.customer.id)
        {
            return true;
        }
    }
    return false;
}

public int Count()
{
    if (list != null)
    {
        return list.Count;
    }
    else
    {
        return 0;
    }
}

```

```

public int CountAttendees()
{
    int i = 0;
    foreach (EventAttendee a in list)
    {
        i += a.noOfAttendees;
    }
    return i;
}

public void Enqueue(EventAttendee attendee, int position) // Order
by paid status first, then by position in waiting list.
{
    Node newNode = new Node(attendee, position);
    if (frontPointer == null) // If the queue is empty
    {
        frontPointer = newNode;
        rearPointer = newNode;
    }
    else
    {
        if (newNode.GetWaitingListPosition() <
rearPointer.GetWaitingListPosition()) // If the new node needs to be the
front of the queue
        {
            newNode.SetNext(frontPointer);
            frontPointer = newNode; // front is now the new node
        }
        else if (newNode.GetWaitingListPosition() >=
rearPointer.GetWaitingListPosition()) // if the new node needs to be at the
end of the queue
        {
            rearPointer.SetNext(newNode);
            rearPointer = newNode;
        }
        else // Finds where to insert the node
        {
            Node current = frontPointer;
            Node prev = null;
            while (current.GetWaitingListPosition() <=
newNode.GetWaitingListPosition())
            {
                prev = current;
                current = current.GetNext();
            }
            newNode.SetNext(current);
            prev.SetNext(newNode);
        }
        list.Add(attendee);
    }
}

public EventAttendee Dequeue()
{
    EventAttendee dequeuedAttendee;
    if (frontPointer == null)
    {

```

```

        MessageBox.Show("Waiting list is empty.");
        dequeuedAttendee = null;
    }
    else
    {
        dequeuedAttendee = frontPointer.GetAttendee();
        frontPointer = frontPointer.GetNext();
        // Gets the attendee at the top of the waiting list and
changes the pointer to the next attendee
    }
    list.Remove(dequeuedAttendee);
    return dequeuedAttendee;
}

public void Clear()
{
    while (frontPointer != null)
    {
        Dequeue();
    }
}

public Node DequeueNodes()
{
    Node dequeuedNode;
    if (frontPointer == null)
    {
        MessageBox.Show("Waiting list is empty.");
        dequeuedNode = null;
    }
    else
    {
        dequeuedNode = frontPointer;
        frontPointer = frontPointer.GetNext();
        // Gets the attendee at the top of the waiting list and
changes the pointer to the next attendee
    }
    list.Remove(dequeuedNode.GetAttendee());
    return dequeuedNode;
}

public void Remove(EventAttendee attendee, List<EventAttendee>
attendeeList)
{
    // Full list of attendees must be passed in as the indexes of
each customer in the list may have changed.

    List<EventAttendee> tempList = new List<EventAttendee>();
    if (frontPointer.GetAttendee() == attendee)
    {
        DequeueNodes();
    }
    else
    {
        while (!tempList.Contains(attendee)) //While the attendee
has not been retrieved from the priority queue
        {
            if (frontPointer != null) // while the queue isn't empty

```

```

        {
            tempList.Add(Dequeue()); //dequeues the queue and
            saves what has been dequeued in the temporary list
        }
        else
        {
            // if the queue is empty and the list doesn't
            contain the attendee that was to be removed, then the attendee wasn't in the
            queue in the first place
            break;
        }
    }
    for (int i = 0; i < tempList.Count; i++)
    {
        if (tempList[i].customer.id == attendee.customer.id)
        {
            tempList.RemoveAt(i);
            break; // Exit the loop after removing the item
        }
    }
    foreach (EventAttendee a in tempList)
    {
        Enqueue(a, attendeeList.IndexOf(a));
    }
}
}
public List<EventAttendee> getList()
{
    return list;
}
}
}

```

Testing

Test No	Form	Test Description	Test Type	Input	Expected Output	Output
1.	Homepage.cs	When the form loads, all events for the current month are displayed, all deliveries and collections are displayed in the correct dates and the current month is shown	Typical	Load form	All events in the correct boxes for their date All dates are shown correctly for this month All deliveries and collections due for this month are displayed in the correct boxes for their date	As expected

2.	Homepage.cs	Switch to the next month	Typical	Click forward button	Correct dates for February 2025 should be shown All events, collections and deliveries should be shown for that month	As expected
3.	Homepage.cs	Switch to the current month	Typical	Click 'Today'	Correct dates for the current month should be shown All events, collections and deliveries should be shown for that month	As expected
4.	Homepage.cs	Switch to the previous month	Typical	Click backward button	Correct dates for December 2024 should be shown All events, collections and deliveries should be shown for that month	As expected
5.	Homepage.cs	When right clicking on an event and clicking view event information, redirect the user to the and show all the information of that event	Typical	Right click on an event and click 'View Event Information'	Load Events.cs and display the information of the event clicked on	As expected
6.	Homepage.cs	"View Event Information" should only be displayed when the user clicks on an event	Erroneous	Right click on a delivery notice box in the calendar	No context menu strip should appear	As expected
CUSTOMERS.CS						
7.	Customers.cs	Load all details of the first customer in	Typical	Click on 'Annika Arumugam' in the data grid	All details of customer 'Annika Arumugam' should	As expected

		the data grid view		view and display all details for that customer.	be displayed in the correct textboxes.	
8.	Customers.cs	Click on a customer in the data grid view and display all details for that customer.	Typical	Click on a customer in the data grid view	All details of the customer selected should be displayed in the correct textboxes.	As expected
9.	Customers.cs	Click on the header of the data grid view This causes the selected index of the dgv to be -1.	Erroneous	Click on 'CustID' at the top of the data grid view.	All details of the first customer in the list should be displayed.	As expected
10.	Customers.cs	Search for a customer by first name	Typical	Type "Anni" in the search bar	The customer "Annika Arumugam" should be the only result	As expected
11.	Customers.cs	Search for a customer by email address	Typical	Type thecure@gmail.com in the search bar	The customer "Robert Smith" should be the only result	As expected
12.	Customers.cs	Search for a customer with details that do not match any customer's details	Erroneous	Type "abcdefg" in the search bar and click the search button	Error message is displayed: "No customers found"	As expected
13.	Customers.cs	Delete a customer that is in an event	Typical	Delete customer 'Michael Braun'	Data of customer Michael Braun is deleted Customer is removed from all events the customer was in: Event ID 25 and 26	As expected
14.	Customers.cs	Attempt to delete a customer that has placed an order and has not collected it	Erroneous	Attempt to delete customer 'Annika Arumugam'	Error message displayed indicating that all open orders must be closed before deleting customer data	As expected
NEWCUSTOMER.CS						

15.	NewCustomer.cs	Add a new customer	Typical	Enter valid inputs for all fields	Customer successfully added	As expected
16.	NewCustomer.cs	Add a new customer with blank fields	Typical	Enter valid inputs for all fields except 'Phone Number'	Customer successfully added Where fields were left blank, 'N/A' is displayed	As expected
17.	NewCustomer.cs	Attempt to add a new customer with an invalid email address	Erroneous	Add a customer with all valid details entered except for "asdfhjks" entered in 'Email Address' field	Error message is displayed: 'Enter a valid email address.'	As expected
18.	NewCustomer.cs	Attempt to add a new customer with an invalid phone number	Boundary	Add a customer with all valid details entered except for "071234" entered in 'Phone Number' field	Error message is displayed: 'Phone number must be longer than 7 characters.'	As expected
19.	NewCustomer.cs	Attempt to add a new customer with an invalid number of vouchers	Erroneous	Add a customer with all valid details entered except for "-1" entered in 'Vouchers Available' field	Error message is displayed: 'Enter a valid number.'	As expected
20.	NewCustomer.cs	Clear all details inputted	Typical	Enter values in all fields then click 'Clear'	All fields cleared and set to default	As expected
SUPPLIERS.CS						
21.	Suppliers.cs	Delete a supplier	Typical	Right click on a supplier Click 'Delete' Confirm deletion by clicking 'Yes'	Supplier deleted: Supplier no longer in data grid view In database: Set to a 'deleted state' All products by that supplier are discontinued	As expected
22.	Suppliers.cs	Open supplier creator	Typical	Click 'Add'	NewSupplier form opens with all	As expected

					fields blank/ set to default	
NEWSUPPLIER.CS						
23.	NewSupplier.cs	Add a new supplier	Typical	Enter valid inputs for all fields	Supplier successfully added	As expected
24.	NewSupplier.cs	Add a new supplier with some fields left blank	Typical	Enter valid inputs for all fields Leave 'URL' and 'Phone Number' blank	Supplier successfully added with 'N/A' saved in the fields left blank	As expected
25.	NewSupplier.cs	Attempt to add a supplier with no details entered	Erroneous	Add a supplier with no details entered in any textboxes	Error message: 'Enter inputs in all required fields'	As expected
26.	NewSupplier.cs	Attempt to add a supplier with no contact details	Erroneous	Add a supplier with all valid inputs Leave 'Email Address' and 'Phone Number' blank	Error message: 'Enter values in all required fields'	As expected
PRODUCTS.CS						
27.	Products.cs	Viewing a 'yarn' product	Typical	Click on a 'yarn' product in the data grid view	All product details for the selected yarn product should be displayed in the correct fields with the correct field names	As expected
28.	Products.cs	Viewing all yarn products	Typical	Click 'Yarn'	All products in the data grid view are yarn products All filters are changed to match yarn product attributes All field names are changed to match yarn product attributes	As expected
29.	Products.cs	Product stock levels indicated by colour in dgv	Typical	Click on a product highlighted in red or orange	Products in red are out of stock, products in orange are low on stock	As expected

30.	Products.cs	Filter products by one supplier which has no products	Typical	Select 'Cygnet Ltd' only	Error message: 'No Products Found'	As expected
31.	Products.cs	Filter by multiple suppliers that are in stock but discontinued	Typical	Select 'Discontinued' check box, 'In Stock' check box under 'Stock' and 'Hobbycraft' and 'Women's Institute' under 'Supplier'	All products shown are discontinued are are supplied by either Hobbycraft or Women's Institute All products shown are in stock (none are highlighted in the data grid view)	As expected
32.	Products.cs	Filter yarn products by one of each filter	Typical	Select 'In Stock','Hobbycraft','4 Ply', 'ACRYLIC' and 'Red' checkboxes	One product found that fulfils all filters selected	As expected
33.	Products.cs	Filter yarn products by two filters and one stock filter	Typical	Select 'Aran', 'Chunky' and 'ACRYLIC' checkboxes	All products shown have the yarn weight 'Aran' or 'Chunky' and are all of acrylic fibre All products shown are in stock, indicated by there being no products highlighted in red	As expected
34.	Products.cs	Viewing all accessory products	Typical	Click 'Accessories'	All products in the data grid view are accessory products All filters are changed to match accessory product attributes All field names are changed to match accessory product attributes	As expected
35.	Products.cs	Viewing all products	Typical	Click 'All'	Clears all filters and shows all products regardless of whether they are a yarn or accessory	As expected

					product in the data grid view	
36.	Products.cs	Search for a product that exists in the database with the filters applied	Typical	Enter “knitcraft” in the search bar and click the search button	Only products that match the input out of the filtered products are shown	As expected
37.	Products.cs	Search for a product that does not exist with the filters applied	Typical	Enter “mustard” in the search bar and click the search button	Error message: no products found	As expected
38.	Products.cs	Search for a product with no input in search bar	Erroneous	Click search button with nothing entered in the search bar	No changes are made to the products shown	As expected
39.	Products.cs	Clear all filters for accessory products	Typical	Click ‘Clear’	All checked list boxes cleared All accessory products are shown	As expected
40.	Products.cs	Search all products for products with similar names	Typical	Enter “Knitcraft” in the search bar	All products matching the input are shown	As expected
41.	Products.cs	Search for a product that does not exist	Typical	Enter “abcdefg” in the search bar and click the search button	Error message: no products found	As expected
42.	Products.cs	Editing a product: change the supplier	Typical	Edit Product 38 Change combo box item to ‘Women’s Institute’ and save	Product edited successfully	As expected
43.	Products.cs	Editing a product: change the image	Typical	Edit Product ID 15 Click ‘Upload’ Upload bow.jpg Save the product	Product image saved to bow.jpg	As expected
44.	Products.cs	Editing a product: remove the image	Typical	While editing Product ID 15, click ‘Remove’	Default image displayed	As expected

45.	Products.cs	Editing a product: reset the image	Typical	While editing Product ID 15, click 'Reset'	Original image bow.jpg displayed	As expected
46.	Products.cs	Editing a product: remove the image and save	Typical	While editing Product ID 15, click 'Remove' and save the product	Product image saved to default image	As expected
47.	Products.cs	Editing a product: enter values and cancel editing	Typical	Change product ID 1: Product name to "abcdef" and the supplier to 'Women's Institute' Click 'Cancel'	No edits are saved	As expected
48.	Products.cs	Discontinue a product	Typical	Right click product ID 34 Click 'Discontinue' Confirm discontinuation by clicking 'Yes'	Product ID 34 is discontinued Edits cannot be made	As expected
NEWPRODUCT.CS						
49.	NewProduct.cs	Add a new product	Typical	Enter valid inputs for all fields	Product successfully added	As expected
50.	NewProduct.cs	Attempt to add a new product with an invalid name	Erroneous	Enter valid inputs for all fields except 'Product Name' Enter "abcd\$£" in 'Product Name'	Error message	As expected
51.	NewProduct.cs	Attempt to add a new product with an invalid file name	Erroneous	Enter valid inputs in all fields except 'Save As:' Upload image as usual Enter "abc>@" in 'Save As' field	Error message	As expected
52.	NewProduct.cs	Clear all fields	Typical	Enter values in all fields Click 'Clear'	All fields cleared and set to default values	As expected

ORDERS.CS						
53.	Orders.cs	Open Orders and view all customer orders Details of the first customer order in the data grid view should be displayed	Typical	Load Orders.cs	All order details of the first order in the data grid view should be displayed in the correct fields with the correct field names	As expected
54.	Orders.cs	View all supplier orders	Typical	Click 'Supplier Orders'	All supplier orders should be displayed Details of the first order in the data grid view should be displayed All filters changed to supplier order specific filters i.e. collection date filter is hidden, payment status filter is hidden and supplier filter is shown	As expected
55.	Orders.cs	All suppliers with orders are loaded into filters	Typical	When viewing all supplier orders	All suppliers with orders recorded displayed in the filter list	As expected
56.	Orders.cs	Filter by suppliers	Typical	Select 'Hobbycraft' checkbox	All supplier orders for Hobbycraft shown	As expected
57.	Orders.cs	Filter supplier orders by order date	Typical	Select 15/1/2025 in order date filter	All supplier orders for 15/1/2025 shown	As expected
58.	Orders.cs	Filter supplier orders by order date and supplier	Typical	Select 15/9/2024 in order date filter Select 'Women's Institute' checkbox	All orders with the order date 15/9/2024 and supplier Women's Institute shown	As expected

59.	Orders.cs	Search for a supplier order by supplier name	Typical	Enter 'WYS' in search bar	All orders for the supplier 'WYS Woodland' shown	As expected
60.	Orders.cs	Search for a supplier order by name of a supplier that does not exist	Erroneous	Enter 'Annika' in search bar and click search button	Error message	As expected
61.	Orders.cs	Switch to viewing all customer orders	Typical	Click 'Customer Orders'	All customer orders should be displayed Details of the first order in the data grid view should be displayed All filters changed to customer order specific filters i.e. collection date filter is shown, payment status filter is shown and supplier filter is hidden	As expected
62.	Orders.cs	Click on a customer order and view all details of the order	Typical	Click on any row in the data grid view	Shows all details of the customer order Shows all details of the customer related to the order	As expected
63.	Orders.cs	View all products in the order	Typical	Click 'View Order Items'	Shows all products in the order and shows details of the first product in the order Hides all filters	As expected
64.	Orders.cs	Click on a product and view all details of the product	Typical	Click on a product in the order	All details of that product should be displayed in the correct fields with the correct field names	As expected
65.	Orders.cs	Search for a product in the order	Typical	Enter input in search bar that matches one of the products in the order	Only products that match input are displayed	Error: no search is made

66.	Orders.cs	Search for a product that does not exist in the order	Erroneous	Enter "abcdefghi" in the search bar and click search	Error message displayed	Error: no search is made
67.	Orders.cs	View order details	Typical	Click 'View Order Info'	Shows all order details of the order selected Shows all customer orders Shows all filters	As expected
68.	Orders.cs	Search for a customer order by customer name	Typical	Input "Annika" in the search bar	All orders shown are linked to customer "Annika Arumugam"	As expected
69.	Orders.cs	Search for a customer order by customer email address	Typical	Input "snaph" in the search bar	All orders shown are linked to customer "Sophie Naphray" whose email address matches the input	As expected
70.	Orders.cs	Search for a customer order by customer name of a customer that does not exist	Erroneous	Input "abcdefg" in the search bar and click the search button	Error message: No orders found	As expected
71.	Orders.cs	Filter all customer orders: archived orders	Typical	Select 'Archived'	All archived customer orders are shown	As expected
72.	Orders.cs	Filter all customer orders: open orders	Typical	Select 'Open'	All open customer orders are shown i.e. all orders that have not been collected, voided or returned	As expected
73.	Orders.cs	Filter all customer orders: paid and collected orders	Typical	Select 'Paid' and 'Collected'	All customer orders that have been paid for and collected are shown	As expected
74.	Orders.cs	Clear all filters	Typical	Clears all filters and shows all	All customer orders are shown	As expected

				customer orders		
75.	Orders.cs	Apply filters with no filters selected	Typical	Click 'Apply' when no filters are selected	All customer orders are shown	As expected
NEWCUSTORDER.CS						
76.	NewCustOrder.cs	Attempt to add a product that is out of stock to the order	Erroneous	Click 'Add' when a product out of stock is selected	Error message	As expected
77.	NewCustOrder.cs	Add a product to the shopping list	Typical	Select Product ID 1 and click 'Add'	Product is added to the shopping list and totals are updated Subtotal and total equal £1.99 Stars earned equals 6	As expected
78.	NewCustOrder.cs	Add multiple of one product to the shopping list	Typical	Select Product ID 6 and click 'Add' twice	Product is added to the shopping list with quantity 2 Subtotal and total equal £5.97 Stars earned equals 18	As expected
79.	NewCustOrder.cs	Create new product and add to the shopping list	Typical	Click 'New Product', create new product and add the new product to the shopping list	Product successfully created and displayed in the shopping list when added	As expected
80.	NewCustOrder.cs	View details of a product which is not currently selected in the shopping list	Typical	Click on 'Knitcraft Mustard Cotton Blend Plain DK Yarn..' in the shopping list	All product details of that product are displayed Product is selected in the data grid view	As expected
81.	NewCustOrder.cs	Increase quantity of existing product in shopping list	Typical	Click 'Add' to increase quantity of 'Knitcraft Mustard Cotton...' product	Quantity of that product is increased by 1 in the shopping list Totals updated accordingly	As expected

82.	NewCustOrder.cs	Remove a product from the shopping list	Typical	Remove 'Knitcraft Pale Peach...' using context menu strip	Product removed and totals updated accordingly	As expected
83.	NewCustOrder.cs	Remove all products from the shopping list	Typical	Remove all products from the shopping list using context menu strip	Totals updated to 0 Stars earned equals 0 No products in order	As expected
84.	NewCustOrder.cs	Attempt to place order	Erroneous	Click place order	Error message	As expected
85.	NewCustOrder.cs	Apply a voucher to the order, if the customer has vouchers available	Typical	Add products to the order Click 'up' on the numeric up down control for 'Vouchers' once	Vouchers used increases by 1 Vouchers available in customer info panel decreases by 1 Value in 'Savings' reflects the discount applied according to the customer's membership status Totals and stars earned are updated accordingly	As expected
86.	NewCustOrder.cs	Add products to the order while a voucher is applied	Typical	Add any product to the order	Totals and savings updated accordingly while regarding the voucher	As expected
87.	NewCustOrder.cs	Attempt to add another voucher	Erroneous	Click 'up' on the numeric up down control for 'Vouchers' once	Number of vouchers does not change, totals do not change	As expected
88.	NewCustOrder.cs	Remove voucher from order	Typical	Click 'down' on the numeric up down control for 'Vouchers' once	Voucher is removed and all totals and savings updated accordingly Subtotal = total	As expected

89.	NewCustOrder.cs	Attempt to add multiple of one product where there will be insufficient stock	Erroneous	Add Product ID 31 to the shopping list Click 'Add' 4 times	Error message: insufficient stock Only 3 lots of that product are in the shopping list	As expected
90.	NewCustOrder.cs	Decrease the quantity of a product in the shopping list	Typical	Click on any product with multiple lots added to the order Use numeric up down control for 'Quantity' and decrease the number Click 'Apply'	Quantity of selected product is decreased Totals updated accordingly	As expected
91.	NewCustOrder.cs	Input order date	Typical	Select 'Order Date' to be 26/01/2025	Order date is changed Expected delivery date is changed to date 3 days following order date Collection date end is changed to date 7 days following expected delivery date	As expected
92.	NewCustOrder.cs	Set the order to be in store	Typical	Select 'In Store' check box	Delivery status is set to 'Delivered' Status is set to 'Archived' Payment status is set to 'Paid' All dates are set to the current date and are not editable	As expected
93.	NewCustOrder.cs	Select another customer for the order	Typical	Select a different customer in the combo box	All details of that customer are displayed and the order will be placed for that customer now	As expected

94.	NewCustOrder.cs	Place customer order	Typical	Click 'Place Order'	Order placed successfully Inventory and membership details of related customer are updated accordingly	As expected
ORDERS.CS (Returning customer orders and printing an invoice)						
95.	Orders.cs	Return product	Typical	Return product in customer order created in previous tests	Inventory and membership details of related customer are updated accordingly Order payment status is changed to 'Partially Refunded'	As expected
96.	Orders.cs	Print invoice as a pdf	Typical	Print invoice for order made during testing Right click on order and click 'Print Invoice' Choose 'Microsoft Print to PDF' Click 'Print'	Print to PDF successful: all order information shown as required, including returns	As expected
97.	Orders.cs	Print invoice to printer	Typical	Right click on order and click 'Print Invoice' Choose correct printer for printing invoice	Print successful	As expected See Appendix: Printed Receipt
98.	Orders.cs	Return full order	Typical	Right click on order and click 'Return Order'	Return successful Delivery status set to 'Returned' Payment status set to 'Refunded' Inventory updated accordingly Membership details of customer are updated accordingly	As expected
99.	Orders.cs	Print invoice of returned order as a pdf	Typical	Print the invoice just returned as a pdf	Returns shown on invoice as required	As expected

100.	Orders.cs	Print invoice for customer order for a guest	Typical	Print invoice for a customer order where there is no related customer i.e. order placed under 'Guest'	Invoice should show no membership details	As expected
101.	Orders.cs	Print invoice with multiple pages	Typical	Print invoice for order ID 57 to pdf	PDF of invoice produced with multiple pages	As expected
NEWSUPPLIERORDER.CS						
102.	NewSupplier Order.cs	Select a product and show all product details	Typical	Click on a product in the data grid view	All details of that products are shown in the correct fields	As expected
103.	NewSupplier Order.cs	Switch supplier to a supplier with no products	Typical	Switch to supplier 'Sirdar Holdings' using the combo box	Error message 'No results found' Supplier is not changed	As expected
104.	NewSupplier Order.cs	Change supplier to another supplier while creating the order	Typical	Add products to the shopping list Change supplier using the combo box	Supplier details of new supplier displayed All items in the shopping list are cleared and all totals are cleared All products for the supplier selected are displayed in the data grid view	As expected
105.	NewSupplier Order.cs	Place supplier order	Typical	Click 'Place Order'	Supplier order placed successfully Inventory updated accordingly as order is delivered	As expected
106.	NewSupplier Order.cs	Place a new supplier order where the order has not been delivered	Typical	Input all valid details and add products to order Set delivery status to 'Order Made' Place order	Supplier order placed successfully Inventory is not updated as the order has not been delivered	As expected
ORDERS.CS (Voiding and returning supplier orders)						
107.	Orders.cs	Void items in a supplier order	Typical	Click 'View Order Items'	Order items successfully voided	As expected

				and right click on the product to be voided Void items in the order that has been placed while testing	No inventory changes	
108.	Orders.cs	Void full supplier order	Typical	Right click on order and click 'Void Order'	Order successfully voided No inventory changes	As expected
109.	Orders.cs	Return items in a supplier order	Typical	Click 'View Order Items' on the delivered supplier order created while testing Right click on the product to be returned Click 'Return'	Order items successfully returned Inventory updated accordingly	As expected
110.	Orders.cs	Return full supplier order	Typical	Right click on order and click 'Return Order'	Order successfully returned Inventory updated accordingly	As expected
EVENTS.CS						
111.	Events.cs	Open Events and view all active events for the current week	Typical	Load Events.cs	All active events for the current week displayed in the weekly calendar and in the data grid view	As expected
112.	Events.cs	View all events for the next week	Typical	Click forward button	All active events for the next week should be displayed in the weekly calendar and in the data grid view If there are no events for the next week, display that	As expected
113.	Events.cs	Switch to all events for the current week	Typical	Click 'Today'	All active events for the current week displayed in the weekly calendar	As expected

					and in the data grid view	
114.	Events.cs	Switch to the previous week	Typical	Click previous button	All active events for the previous week displayed in the weekly calendar and in the data grid view	As expected
115.	Events.cs	View event details of each event	Typical	Click on an event in the calendar	All details of that event displayed in the correct textboxes All attendees of that event displayed in the attendees list If the event has a waiting list, display the 'View Waiting List' button Event is selected in the data grid view	As expected
116.	Events.cs	View event details of each event	Typical	Click on an event in the data grid view	All details of that event displayed in the correct textboxes All attendees of that event displayed in the attendees list If the event has a waiting list, display the 'View Waiting List' button	As expected
117.	Events.cs	View attendee details	Typical	Click on an attendee in the list of attendees for the event	All details of that attendee displayed, including paid status	As expected
118.	Events.cs	View waiting list	Typical	Click 'View waiting list'	Displays attendees on the waiting list	As expected
119.	Events.cs	Switch to viewing all events	Typical	Click 'All Events'	Weekly view is hidden All active and upcoming events are displayed in the data grid view	74:38 Mostly as expected Shows all events that are

					All filters for events are shown	in the future except for events of the current date
120.	Events.cs	Filter cancelled events	Typical	Select 'Cancelled' Click 'Apply'	All events that are cancelled are displayed	As expected
121.	Events.cs	Filter cancelled and upcoming events	Typical	Select 'Cancelled' and 'Upcoming' Click 'Apply'	All cancelled events in the future are displayed	75:59 Shows all events that are cancelled or upcoming rather than events that are cancelled and upcoming .
122.	Events.cs	Filter upcoming events with spaces available	Typical	Select 'Spaces Available' and 'Upcoming' Click 'Apply'	All events in the future that aren't at capacity are displayed	Same as Test 119
123.	Events.cs	Filter cancelled events where there are customers that have paid for the event	Typical	Select 'Cancelled' and 'Paid' Click 'Apply'	All cancelled events where all customers have paid are displayed	As expected
124.	Events.cs	Clear all filters	Typical	Click 'Clear'	All filters are cleared and all upcoming and active events are displayed	77:15 All filters are cleared Events in the future are all shown except events for the

						current date: Same as Test 119
125.	Events.cs	View the weekly calendar of events	Typical	Click 'Weekly View'	All active events for the current week displayed in the weekly calendar and in the data grid view	As expected
126.	Events.cs	Edit order notes	Typical	Edit the order notes of an event and click 'Apply'	Order notes successfully changed	As expected
127.	Events.cs	Edit paid status of a customer	Typical	Click on an attendee that has not paid yet (highlighted in red) Check the 'Paid' checkbox and click 'Apply'	Customer should no longer be highlighted in list, indicating that the paid status of the customer has successfully been changed	As expected
128.	Events.cs	Attempt to cancel an event in the past	Erroneous	Right click on an event in the past Click 'Cancel Event'	Error message displayed	As expected
NEWEVENT.CS						
129.	NewEvent.cs	Add customer to attendee list	Typical	Click on a customer and click 'Add'	Customer added to attendee list successfully Customer highlighted in green in data grid view	As expected
130.	NewEvent.cs	Attempt to add a guest to a customer when no guests are allowed		Click up on the numeric up down control for 'Guests'	Error message	As expected
131.	NewEvent.cs	Add a guest to a customer and add the customer to	Typical	Increase maximum number of guests	Customer added successfully to attendee list with the guest shown	As expected

		the attendee list		Increase number of guests for the customer using the numeric up down control Add the customer		
132.	NewEvent.cs	Add a customer that has not paid	Typical	Uncheck 'Paid' and add the customer	Customer added to attendee list successfully Customer highlighted in red in attendee list	As expected
133.	NewEvent.cs	Reach capacity of event	Typical	Add enough customers to reach the capacity of 10 Click on another customer	'ON WAITING LIST' displayed to indicate the customer will be added to the waiting list	As expected
134.	NewEvent.cs	Add customers when the event is at capacity	Typical	Add customers when the event is at capacity	Customers are added to the waiting list Customers in the waiting list are highlighted blue in the list	As expected
135.	NewEvent.cs	Increase the capacity of the event and reallocate spaces to customers	Typical	Increase the capacity by 1 Refresh the waiting list	First customer on the waiting list is no longer highlighted in blue, indicating they are no longer on the waiting list	As expected
136.	NewEvent.cs	Attempt to decrease the capacity	Erroneous	Decrease 'Capacity'	Error message	As expected
137.	NewEvent.cs	Remove customer from main list	Typical	Right click on customer and click 'remove'	Customer removed successfully	As expected
138.	NewEvent.cs	Reallocate spaces to customers on the waiting list that have paid before those	Typical	Add a customer that has not paid to the waiting list, followed by a customer that	The customer that was originally on the waiting list that has paid is removed from the waiting list first	As expected

		that have not paid		has paid to the waiting list Increase the capacity to free up space in the event Click 'Refresh'		
139.	NewEvent.cs	Remove customer from waiting list	Typical	Right click on customer in the waiting list and click 'remove'	Customer removed successfully	As expected
140.	NewEvent.cs	Attempt to save event with date in the past	Erroneous	Enter valid inputs in all required fields Input 01/01/2025 in 'Date' Click 'Save'	Error message	As expected
141.	NewEvent.cs	Add event	Typical	Enter valid inputs in all required fields Click 'Save'	Event added successfully	As expected
NOTICES.CS (Viewing notices)						
142.	Notices.cs	Notices are all shown in default order	Typical	Load Notices.cs	All notices shown are ordered by review date in descending order For notices with the same review date, they are ordered by priority in descending order Latest notices are shown first	As expected
143.	Notices.cs	Order notices by ascending order of priority and ascending order of review date	Typical	Click 'Priority' and click 'Review Date'	Notices ordered by ascending order of review date and priority Earliest notices are shown first	As expected
144.	Notices.cs	Order notices by ascending order of priority and	Typical	Click 'Priority' so the arrow is pointing up	Notices shown in descending order of review date, and for notices with the	As expected

		descending order of review date		Click 'Review Date' so the arrow is pointing downwards	same review date, they are shown in ascending priority (lowest first)	
145.	Notices.cs	Filter all notices by high priority	Typical	Select 'High' and click 'Apply'	All notices shown have high priority	As expected
146.	Notices.cs	Filter all notices by collection related notices	Typical	Select 'Collection' and click 'Apply'	All notices shown related to collections	As expected
147.	Notices.cs	Filter all notices by 'Older' review date	Typical	Select 'Older' and click 'Apply'	All notices shown have a review date that is more than a week ago today	As expected
148.	Notices.cs	View all archived notices	Typical	Click the 'archive' icon	All archived notices shown	As expected
149.	Notices.cs	View all dismissed notices	Typical	Click the 'bell' icon	All dismissed but not archived notices shown	As expected
150.	Notices.cs	Filter all notices by low priority and collection related notices	Typical	Select 'Low' and 'Collection' and click 'Apply'	All notices with low priority and are collection related are shown Title should be 'Order Collection Overdue: 1 day'	As expected
151.	Notices.cs	Dismiss a notice for 3 days	Typical	Right click on a notice's header Click 'Dismiss' Click '3 days'	Dismisses notice successfully for 3 days- review date is set to 3 days later than today	As expected
152.	Notices.cs	Clear all filters	Typical	Click 'Clear'	All filters cleared	As expected
153.	Notices.cs	Go to the event for an event notice	Typical	Right click on an event notice	Events form opened All details of event related to notice are shown	As expected
154.	Notices.cs	Go to the customer order for a customer order notice	Typical	Right click on a customer order notice	Orders form opened All details of order related to notice are shown	As expected

155.	Notices.cs	Go to the supplier order for a supplier order notice	Typical	Right click on a supplier order notice	Orders form opened All details of order related to notice are shown	Loads the customer order with the same ID 96:08
156.	Notices.cs	Open a new supplier order for a product that is low/out of stock	Typical	Right click on a stock notice	Supplier order creator is opened Related product is added to the shopping list with quantity equal to the low stock threshold for that product	As expected
ISSUING AND UPDATING NOTICES						
157.	Products.cs	Issue a 'No Stock' notice for products out of stock	Typical	Change stock of product to 0	'No Stock' notice issued	As expected
158.	Products.cs	Issue a 'Low Stock' notice for products low on stock	Typical	Change stock of product to below low stock threshold	'Low Stock' notice issued	As expected
159.	Orders.cs	Archive stock notice when a supplier order for that notice has been delivered	Typical	Change delivery status of a supplier order containing a product with a notice to 'Delivered'	Notice related to product is archived	As expected
160.	Products.cs	Archive 'No Stock' notice	Typical	Add stock to a product out of stock	Notice related to product is archived	As expected
161.	NewSupplier Order.cs	Update 'Low Stock' notice	Typical	Place supplier order for a product low on stock	Notice related to product dismissed until delivery date of supplier order	As expected
162.	NewCustOrder.cs	Archive 'Low Sales' notice	Typical	Place customer order for a product with low sales	Notice related to product archived	As expected
163.	NewCustOrder.cs	Issue 'Selling Fast' notice	Typical	Order a large volume of one product in a customer order	Notice issued for the product	As expected

164.	NewCustOrder.cs	Issue collection overdue by 1 day notice	Typical	Create a customer order where the collection date has passed by 2 days and the order is ready to collect	Notice for overdue collection issued with low priority linked to the order ID	As expected
165.	NewCustOrder.cs	Issue collection overdue by more than 5 days notice	Typical	Create customer order where the collection date has passed by 6 days and the order is ready to collect	Notice for overdue collection issued with high priority linked to the order ID	As expected
166.	Orders.cs	Archive notices for collected orders	Typical	Set delivery status of an order 'Ready to Collect' to 'Collected'	Any related notices are archived	As expected
167.	NewCustOrder.cs	Issue order processing overdue by 3 days notice	Typical	Create a customer order where the delivery date has passed by 3 days	Notice for overdue processing issued with medium priority linked to the order ID	As expected
168.	Orders.cs	Archive processing overdue notice when the order is processed	Typical	Update customer order with delivery status 'Order Made', change to 'Ready to Collect'	Archives related overdue processing notice	As expected
169.	NewSupplierOrder.cs	Issue overdue supply delivery notice by 3 days notice	Typical	Create supplier order where the delivery date has passed by 3 days	Notice for overdue supply delivery issued with medium priority linked to the order ID	As expected
170.	Orders.cs	Archive supply delivery notices for supplier orders that have been delivered	Typical	Update supplier order out for delivery to 'Delivered'	Archives related supply delivery notice	As expected

171.	Orders.cs	Dismiss supply delivery notices until their delivery date if there was an existing notice for an overdue order	Typical	Update supplier order delivery date	Dismisses related notice until the delivery date	As expected
172.	Orders.cs	Archive all notices for orders that are voided	Typical	Void supplier order with a notice	Archives related notice	As expected
173.	NewSupplier Order.cs	Issue overdue supply delivery by 1 day notice	Typical	Create a supplier order where the order is out for delivery and the delivery date was 1 day ago	Notice issued for overdue supply delivery with low priority	As expected
174.	NewEvent.cs	Issue event payments notice	Typical	Create an event where customers have not paid	Notice related to event ID issued for event payments	As expected
175.	NewEvent.cs	Issue 'event spaces available' notice	Typical	Create an event where customers are on the waiting list but there are spaces available	Notice related to event ID issued for 'event spaces available'	As expected
176.	NewEvent.cs	Allocate spaces to customers on the waiting list in an event with a notice for available spaces	Typical	Resolve an existing 'event spaces available' notice by refreshing the waiting list and saving the event	Notice related to event is archived	As expected
177.	Events.cs	Cancel event with a notice	Typical	Cancel event with event payment notice	Archives related notice	Notice is not archived 120:17
178.	Events.cs	Update payments for an event with	Typical	In an event with an event payments	Archives related notice	As expected

		a payment notice		notice, set all customers' paid status to paid		
--	--	------------------	--	--	--	--

Evaluation

Testing Evaluation

Test 65 & 66

Description

Searching for a product in the order

Error

Any inputs into the search bar while the order items are being viewed should be used to find products in the order where the name matches the input. However, no results are shown when any input is entered.

Solution

REFER TO ORDERS.CS

BEFORE: `cmd.Parameters.AddWithValue("@pn", tbxSearch.Text);`

AFTER: `cmd.Parameters.AddWithValue("@pn", "%" + tbxSearch.Text + "%");`

CHANGES: Added wildcard "%" to the parameter so that the query returns all products where the search is part of or equal to the product name of any product in the order.

Test 119, 122, 124

Description

Show all upcoming and active events in the data grid view

Error

Displays all upcoming and active events, except for events of the current date

Solution

There are multiple places in the program where this error may occur, but are all based around the same or similar SQL condition.

Shown below is an example of the type of SQL statement that needs to be edited to solve this error.

REFER TO EVENTS.CS

BEFORE: `cmd.CommandText = "SELECT * FROM TableEvents WHERE Cancelled = False AND EventDate > @tod";`

AFTER: `cmd.CommandText = "SELECT * FROM TableEvents WHERE Cancelled = False AND EventDate >= @tod";`

CHANGES: Added '=' to ensure events where the event date is equal to the current date passed in are all shown

Test 121

Description

Show all future events that are cancelled

Error

Displays all future events and all cancelled events.

Solution

This issue arises in the conditions in the SQL statement generated. In the code snippet below, it demonstrates that 'OR' is used to concatenate the conditions together. This is what results in all future events and all cancelled events being shown.

To correct this error, I would separate the status filters generated from the selected checkboxes in the checkbox list. I would separate them into two lists that would store date related filters, and active filters.

Then I would use a combination of these lists and the 'AND' operator in SQL to concatenate conditions together appropriately, so conditions such as 'TableEvents.EventDate >= @tod AND TableEvents.Cancelled = True' are generated and concatenated with the command text to display the expected output for this test.

REFER TO EVENTS.CS

CURRENT:

```
if (selectedStatusFilters.Count != 0)
{
    filtersApplied = true;
    foreach (string statusFilter in selectedStatusFilters)
    {
        switch (statusFilter)
        {
            case "Upcoming":
                statusCondition = " TableEvents.EventDate >= @tod";
                break;
            case "Past":
                statusCondition = " TableEvents.EventDate < @tod";
                break;
            case "Active":
                statusCondition = " TableEvents.Cancelled = False";
                break;
            case "Cancelled":
                statusCondition = " TableEvents.Cancelled = True";
                break;
        }
        if (baseCommandText.EndsWith("E"))
        {
            if (selectedStatusFilters.Count > 1)
            {
                baseCommandText += " (" + statusCondition;
```

```

        }
        else
        {
            baseCommandText += " " + statusCondition;
        }
    }
    else if (baseCommandText.EndsWith("))")
    {
        if(selectedStatusFilters.Count > 1)
        {
            baseCommandText += " AND (" + statusCondition;
        }
        else
        {
            baseCommandText += " AND " + statusCondition;
        }
    }
    else
    {
        baseCommandText += " OR" + statusCondition;
    }
}
if (selectedStatusFilters.Count > 1)
{
    baseCommandText += "));";
}
}

```

Test 155

Description

Go to the supplier order for the supplier order notice

Error

Displays the customer order with the same ID, if any

Solution

This issue arises since the orders shown has not been switched to the supplier order view, so the order ID passed in it automatically found within the customer orders currently shown. To rectify this error, supplier orders should be shown first before finding the order with the ID. This can be done by passing in a Boolean value to the form specifying whether the form should load customer orders or supplier orders first.

Test 177

Description

Cancel event with a notice -> Archives related notices

Error

The notice is not archived

Solution

This issue arises in the subroutines processEventPayments() and processEventSpaces() in Homepage.cs. To retrieve all events that need to be processed, the initial SQL statement includes the condition **WHERE Cancelled <> True**

This condition ensures that only active events are reviews, so all cancelled events are not included in the processing of event notices.

A solution to this issue would be to remove the above condition, and check whether the event is cancelled before other conditions related to the notice type using the following if statement:

```
if (e.cancelled) // If the event is cancelled
{
    noticesToArchive.Add(eventID); // Archive the notice
}
```

Objectives Evaluation

Objectives	Objectives Details	Completion
Homepage	Interactive calendar dashboard of all events, deliveries and collections Access notices, events, orders, products, customers and suppliers	Completed in full
Customer Details Management	Add, view, edit and delete customer data Search for a customer by name or email address	Completed in full
Customer Loyalty Scheme	Assign customers membership statuses Bronze, Silver or Gold Allocate 3 stars to a customer for every £1 spent in an order Track and enable the customer to redeem vouchers in orders Update membership status according to the number of stars earned Update membership details when voids/returns are made	Completed in full
Supplier Details Management	Add, view, edit and delete supplier data Search for a supplier by name	Completed in full
Product and Inventory Management	Add, view, edit and discontinue products Highlight stock levels of products Filter products Search for a product by name	Completed in full
Customer Order Management	View and edit all customer orders Search for an order by customer details Filter orders View customer details of order View all products in the order Search for a product in the order Void/return order/order items Place an order	Completed in full

	Generate and print invoice (to pdf or direct printing) Update inventory and customer membership details when an order is placed	
Supplier Order Management	View and edit all supplier orders Search for an order by supplier details Filter orders View supplier details of order Search for a product in the order Void/return order/order items Place an order Update inventory when an order is placed	Completed in full
Event Management System	View all events altogether or in an interactive weekly calendar view Filter events Edit event details Edit attendees in the event Change the paid status and number of guests of a customer Create/edit an event Automatically allocate spaces to any customers on the waiting list Cancel an event in the future	When filtering events, some conditions can't be applied to be merged with others e.g. upcoming and cancelled events Otherwise completed in full
Notice System	View all notices Dismiss notices Go to the relevant form for a notice Sort notices by priority and review date Filter notices Automatically generate/update stock notices based on low stock, no stock, fast sales and low sales Automatically generate/update order notices based on the collection/delivery date and order status Automatically generate/update event notices based on the number of customers that have paid and spaces available for those on the waiting list and whether events are cancelled	Completed in full

Client Feedback

After the development of this program, I talked with my client to review how I have fulfilled my objectives and the original aim of this project. Here are the comments made by my client:

- "The layout of each part is clear and easy to understand, making navigation simple and intuitive: this is very useful for people who aren't as tech savvy, like me! I think the user friendly and clean design really adds to the appeal of this program."
- "The calendar is really useful—just awesome! It helps keep everything organized and ensures we never miss important dates."
- "The flexibility built into the program is great. It adapts to our needs rather than forcing us to work around it, meaning I will be relying on this program to manage the shop for me without me having to put in so much effort anymore."
- "Accessing suppliers and customers is much easier now, saving us valuable time and effort. The only thing I'd say is that customer details are usually searched for using postcodes, because a customer may not know or have no access to their email addresses. A postcode is an easier way of finding customer inform"
- "The stock listing feature is incredibly useful, especially the way it highlights items that are running low. It allows us to stay on top of inventory without constant manual checks, and the filters really help with easily finding products without the process being too complicated!"
- "It's easy to link orders and products to suppliers, as well as orders to customers, which makes processes far more efficient and seamless."
- "The loyalty scheme and the way it's automatically calculated is a stroke of genius. It removes the hassle of manual tracking and means customers get rewarded without me having to make so much effort for every customer! I think the only thing I'd want to add is having different types of vouchers, because I don't think a discount is always appropriate. Some customers may prefer deals like '3 for 2' or 'buy one get one free', and a system where these deals are automatically allocated to products could be really useful while placing orders. "
- "The invoice generation system ensures receipts are fit for purpose, making transactions smoother and more professional. The customised message at the end of the receipt will really help our business stand out as a shop that customers will want to return to."
- "The event waiting list allocation system is a fantastic feature. It allows us to see at a glance who has paid and who hasn't, and the automatic allocation system really helps with quickly managing customers in an event. It really saves me from having to work out the whole waiting list myself! But perhaps it could be nice to make it a bit more obvious which customers have been taken off the waiting list when I edit the event, in case I need to contact those customers about them being given a space."
- "Also, sometimes I might forget about other events while creating a new one and it could be helpful to be notified of any clashes with the events, especially since I don't really have the time to check all the events for each day and their timings."
- "Most of all, I love how the system is linked together, through the membership system and notices system particularly! The interconnectedness of the system really brings everything together, making the entire program feel cohesive and well-designed."
- "I know that currently it's just me in the shop, but in future I may decide to hire people to help me with managing everything. Obviously I can't make my employees do everything manually, and this program will help them to manage the shop alongside me, but just in case I don't want them to be able to do certain things like delete customers or void or return orders without me knowing."

- “Also, I know that I obviously won’t put information in the system that’s incorrect or valid, but since there’s nothing stopping me from entering a completely incorrect postcode for a supplier, I think something needs to be done about that when I hire people and they use this program.”

Future Improvements & Extensions

After obtaining feedback from my client, I have reviewed my objectives and identified areas that could be improved or extended accordingly with what my client has said.

User Access Levels

In the future, this business will be expanding further, meaning that there will be future employee expansion. The client has outlined that some sort of user access control system could be implemented to restrict certain actions, such as deleting customers, voiding/returning orders. An audit log visible to managers could be added, so managers of the shop can see what changes have been made and reasons could be recorded for changes such as returns. An approval system could also be implemented so a manager password needs to be entered to fulfil specific actions, so unauthorised changes are prevented. An alert system could also be set up to notify the shop owner of any critical changes, such as deletions and voids/returns being made.

Overall these improvements would enhance security, accountability and data accuracy, so the system can become more robust.

Address and Postcode Validation

Refer to Supplier Details Management

The inclusion of postcode and address validation could be implemented to prevent incorrect information from being recorded in supplier details. However, this feature was not included in the initial objectives, as implementing accurate validation would require integrating an external API to verify the validity of postcodes and addresses.

Additionally, not all suppliers have UK-based addresses. Ensuring international address validation would introduce further complexity, which, given the time constraints, was deemed unnecessary. After discussing this with the client, it was agreed that while validation would help prevent accidental input errors, it is not currently essential.

At present, the system is used solely by the shop owner. Therefore, the client has decided that address validation will only be necessary in the future if employees are introduced who need to interact with the system.

Customer Postcodes

Refer to Customer Details Management

An improvement could be made in retrieving customer details by using postcodes for identification instead of email addresses. Postcodes are commonly used in membership systems as they are less ambiguous than emails or phone numbers. This would require adding a 'Postcode' field to the customers table and implementing CRUD operations for it. Validation would only need to ensure the postcode follows UK formatting, as customers are required to reside in the UK.

Event Time Management

Refer to Event Management System

Given more time, I would improve the event system by detecting event clashes. While users could still create overlapping events, the system would highlight conflicts before confirming them. However, due to time constraints, this feature was not prioritized, as the client deemed it unnecessary for the current scale of operations.

Customer Loyalty Scheme

Refer to Customer Loyalty Scheme

A possible extension of the customer loyalty scheme would be the addition of different types of vouchers that apply to specific product categories and customer membership tiers. This would allow the shop to offer targeted discounts, such as percentage-based reductions on certain items or loyalty rewards for frequent purchases. Implementing this could help encourage repeat business, and promote specific products. While beneficial, this feature was not included due to time constraints and the client's current focus on core functionalities. However, as the business grows this could be a valuable feature for the shop.

Printing Receipts

*Refer **Error!** Reference source not found.*

Currently, receipts can only be printed with the width set to 220mm, which is specific to the label printer that I have used throughout development. Since my client may buy a different printer from the one I use and possibly use different paper sizes, my client and I agreed that in future I will ensure that the width of the invoice is adaptable to any printer.

Final Evaluation

Overall, I am highly satisfied with how my program meets its objectives, significantly aiding my client in managing shop operations. While some tests were initially unsuccessful, I identified solutions and would resolve these issues if given more time. My client has provided positive feedback, along with suggestions for improvement, which I have carefully considered.

If I were to restart this project, I would prioritize implementing user access levels to establish a secure foundation before developing other features. Additionally, I would focus on refactoring repeated code into reusable methods to enhance readability and maintainability, making future updates smoother. Since my client plans to request new features as the business grows, ensuring scalability is crucial.

While the system is user-friendly, data presentation could be improved beyond standard data grid views for better accessibility. Transitioning operations from paper to this system will boost efficiency, and features like invoice generation and the loyalty scheme are expected to improve sales. Based on my client's feedback, this program will be a valuable tool in streamlining daily business operations, with how I have effectively fulfilled the aim of this project and successfully brought all requirements into a full program ready for use by the client.

Appendix

1. Analysis

1.1 Full Structured Interview

1. What information about the yarn would you prefer to be displayed?

- Colour, the weight of the yarn, the quantitative weight of the yarn, and each of these at pricing. You'd also want similar yarns that are assorted colours of the same weight, material etcetera.
- Perhaps a filter button for filtering the products by colour, material or weight would be useful too, if I need to look for yarn the customer is interested in.

2. What information would be useful to view in terms of orders?

- For orders, the order date, first name and last name, phone number and email address. The total cost would also be important. Being able to see what's been ordered specifically would be helpful as well, so maybe when I right click on the order, I can view product information, and it would show all of that.

3. What information would be useful to view in terms of deliveries?

- For deliveries, I think that should be viewed separately. It would be helpful if that went straight onto a calendar, I would have thought, because that's easier to read, isn't it? If there is a field with the date in, a calendar view for all the inbound deliveries of the month would be helpful, as well as switching between months to see past/future deliveries.

4. What other data would you want to be able to store and link?

- Being able to record events like workshops, so when people contact us by phone or are in store, I can load up a form and then just type in the details like first name, last name and phone number. If I can type that in and then the program will add them as a customer, then link them to a workshop, it would be much better than just writing their information down in a notebook. Including something about paid status would be useful too, something toggleable I think.

5. What would you like the main interface to have?

- I think having important notices on the side and the logo in the corner would be great, and then in the centre a calendar view of all events, orders and inbound deliveries for the month and next months would be extremely useful, just for keeping everything in one place. It would help a lot with staying organised in one area, rather than having to open multiple screens and spend more time looking for events, orders and deliveries for the month.

6. What level of detail would be needed in the information displayed, or would you prefer separate windows for separate categories?

- On the main interface, with the calendar I think basic information like "Order inbound" should be shown just to not take up too much space, then when I click on it, I can see more information about the order. That sort of principle could be applied to workshops too, because I don't want the calendar to be too cluttered. If there is more, perhaps there could be a "plus three more events" at the bottom of the box or maybe have "4 orders inbound" and "2 events", then when you click on it could expand like a dropdown menu.

- I'd also like something with the workshops, inbound deliveries and orders where I can add a quick note to myself by clicking on the event on the front calendar and then just type. Say if there was a big order that had got a certain deadline to fulfil, I could make a small note to myself about what I need to do specifically with that order. That sort of flexibility would be particularly useful because it's likely that there is something else that needs to be done with the order that isn't standard. For example, if an order has been split up between two different suppliers, or if one item of the order is being delivered to the shop and another item of the order is being delivered to a collection point, which would be rare with the business.

7. What would you like to see when managing workshops?

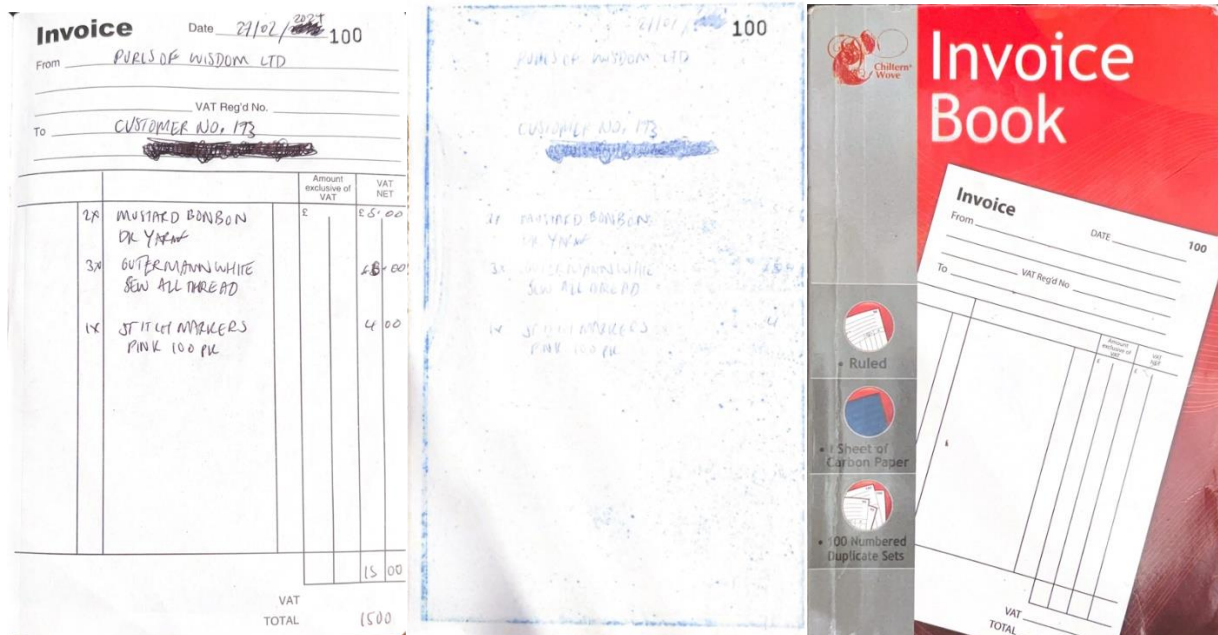
- Viewing it on a separate form from the main interface would be much better, I think. Then what I'd like is to be able to see how many people are booked in, how many people have paid etc, and being able to book customers into the workshop from that form itself would be much more useful. If the customer isn't already in the database, it would be more efficient to add a new customer from the workshops form and then book them in using their customer ID.
- I think a calendar view for just the workshops would be good here too but maybe have some space on the side where a list of the customers could come up when I click on the workshop. Then when I click on the customer, I can see whether they've paid or not, along with their details, and then remove them from the workshop but not delete the customer.

8. Are there any ways creating this program could help you increase your sales?

- Perhaps a loyalty program so that customers are more likely to come to our shop. Often people will visit the store once and not return, as we are a local business and are not very well known so far. To earn favour with customers, a membership system where they can earn loyalty points and earn vouchers with discount could be useful for increasing the number of returning customers.

1.2 Document Study

Existing Invoice Format



Receipt Design

SALT&PEPPER SHREDDED CHICKEN	3.29
STACKS SOUR CREAM & ONION 35G	0.39
STACKS ORIGINAL 35G	0.39
STACKS ORIGINAL 35G	0.39
BUY 3 FOR £1.00	-1.17
NY SALT & CHILLI CHUNKS	3.29
NY DINER HOT & SPICY STRIPS	3.29
JCB COIN CELL BATTERIES 15PK	0.99
SIGN SHACK	0.49
S/TREATS SHOE SHINE SPONGE NEU	0.69
KIWI SNEAKER ODOUR MAX DEFENCE	0.69
COMFY MEMORY FOAM INSOLES	0.89
SOFTY LUXURY SOFT TISSUES 70S	0.79
CHICKEN SALAD SANDWICH	1.85
BLT SANDWICH	1.85
3PK ROUND FOOD CONTAINERS ASST	1.99
WALKERS EXTRA FLAMIN' HOT 150G	1.65
MENS SS/LP CHECK PJS POPLIN	10.00
SWEET CHILLI CRISPS	1.00
SCHWARZKOPF H/S KERATIN	2.89
TRESERME HEAT DEFENCE SPRAY	3.99
COLGATE T/P PUMP COOL STRIPE	1.99
D/CARE TONGUE CLEANER	0.49
COLGATE T/P PUMP COOL STRIPE	1.99
ROBERTS MEDIUM WHITE BOOG	1.09
D/CARE TONGUE CLEANER	0.49
SENCE VITAMIN C 2 STEP MASK	0.69
SLICED BRIOCHE 500G	1.49
TOTAL TO PAY	48.86
CARD TENDER	48.86
CHANGE DUE	0.00

VAT @ 20.00%	Net	27.26	VAT	5.45
VAT @ 0.00%	Net	16.15	VAT	0.00

PLEASE RETAIN YOUR RECEIPT.
You will need it in case of product
exchange, refund or warranty claim.
TJ Morris Ltd www.tjmorris.co.uk
VAT NUMBER: 320093700
12/01/25 16:08 TILL2 DEON J 753630

PURLS OF WISDOM
3man Street
Birmingham
B5 3AL TEL: 01492 330033

Order No. 12345 04/10/24 13.16

Product	Qty	Total
Mustard Bonbon DK Yarn	2	2.98
Scarlet Bonbon DK Yarn	1	1.99
Black Bonbon DK Yarn	3	5.97
Subtotal		11.94
Savings:		-1.19
Voucher (-10% Gov)		10.75
TOTAL		10.75

Customer ID: 1
Customer Name: A. ARU
Membership Status: GOLD
Starts Earned: 32
Starts Balance: 164
You have saved 15.64 pounds with us so far!

THANK YOU FOR SHOPPING WITH US

Handwritten notes:
- Order Date
- Order Time (PM)
- Table Customer Order Items: Quantity
- Table Products: Cost + Table Customer Order Items: Quantity
- a voucher may be used to apply a discount (one at a time)
- If customer's membership type: GOLD -15% subtotal, SILVER -10% subtotal, BRONZE -5% subtotal (all members start with Bronze Membership)
- Display a message specifying how much the customer has saved since first registering. If the customer has not saved anything yet, or achieved a new membership status, no message is displayed. If the customer has achieved a new membership status, this message is changed to notify the customer of their membership status.
- All members start with Bronze membership status. If total cost >= 30.00 and purchase >= 10 items, upgrade membership.

Testing

Test 97: Printed Receipt

TEST NO 97

PURLS OF WISDOM
3 Main Street
Birmingham
B5 3AL
01902 330033

Invoice

Order Date: 26/01/2025
Order No: 72
Date Printed: 29/01/2025
Time Printed: 22:41:10

Product	Price
1x Knitcraft Mustard Cotton Blend Plain DK Yarn 100g	£1.99
1x Knitcraft Denim Cotton Blend Plain DK Yarn 100g	£1.99
2x Knitcraft Black Cotton Blend Plain DK Yarn 100g	£1.99
2x Knitcraft Crimson Everyday Chunky Yarn	£3.00
1x Knitcraft Pink Everyday Chunky Yarn	£2.79
1x Knitcraft Magenta Everyday Chunky Yarn	£2.79
1x James C Brett Forest Croftland Aran Yarn	£6.30
2x Lion Brand Almond Cream I Like Big Yarn 250g	£18.00
1x Lion Brand Chiffon I Like Big Yarn 250g	£18.00
- 1x RETURNED	- £18.00
3x Gutermann White Sew All Thread 250m	£2.50
Subtotal:	£87.34
Savings (-5% BRON)	- £4.37
Returned Items	- £17.10
TOTAL:	£65.87

Customer ID: 14
Customer Name: S APHON
Member Status: Silver
Stars Earned: 198
Stars Balance: 98

You have saved £4.37
with us so far!

**THANK YOU FOR
SHOPPING WITH US**